



SpearDAT MATLAB Interface User Guide

Modern C++ Missile DATCOM Extension Framework

Document version	0.1
Software baseline	SpearDAT v0.1 development series
Date	August 17, 2026

Author and maintainer

Christos Patrinos
christos.patrinis@outlook.com

SpearDAT builds upon the publicly released Missile DATCOM methodology through a substantially modernised and corrected C++17 implementation, supported by software verification and experimental aerodynamic validation.

Revision History

Revision	Date	Summary
0.1	August 17, 2026	Initial SpearDAT MATLAB Interface User Guide covering the public MATLAB workflow, engineering input model, result interpretation, experimental validation, applicability guidance, known implementation differences, and MATLAB input reference.

Contents

Revision History	i
1. Introduction	1
1.1. Purpose and Scope	1
1.2. SpearDAT Overview	1
1.3. Verification, Validation and Known Differences	2
1.4. MATLAB Interface Overview	2
1.5. Intended Audience	2
1.6. Guide Organisation and Conventions	3
2. Installation and MATLAB Configuration	5
2.1. System Requirements	5
2.2. Package Contents	5
2.3. Installing the Package	6
2.4. Configuring MATLAB	6
2.5. Verifying the Installation	7
2.5.1. Step 1: Verify the MATLAB Interface	7
2.5.2. Step 2: Verify the Solver Executable	7
2.5.3. Step 3: Run a Complete Analysis	8
3. MATLAB Quick Start	9
3.1. Example 1: Run a Single Analysis	9
3.2. Check the Execution Status	9
3.3. Inspect the Aerodynamic Table	10
3.4. Select Data for One Operating Condition	10
3.5. Plot the Results	11
3.6. Example 2: Run a Sideslip Sweep	11
3.7. Inspect an Individual Case	13
3.8. Retain Solver Files When Required	13
3.9. Quick-Start Result Map	14
4. The SpearDAT Analysis Model	15
4.1. How SpearDAT Represents an Analysis	15
4.2. Structure of the MATLAB Input	15
4.3. Requests, Configurations, Cases, Sweeps, and Zones	17
4.4. Schedules and Case Expansion	18
4.5. MATLAB Value and Structure Rules	18
4.5.1. Scalar Structures and Repeated Objects	19
4.5.2. Numeric, Logical, and Text Values	19
4.5.3. Field Names and Enumerated Choices	20
4.6. Attitude Modes and BETA/PHI Sweeps	20
4.6.1. Expansion Rules	21
4.7. Omitted Fields, Empty Values, Defaults, and Explicit Zero	22
4.8. Validation Before Execution	23
4.8.1. MATLAB Representation Errors	24
4.8.2. Engineering and Cross-Field Validation	24
4.9. Case Identity and Traceability	25

5. Flight Conditions and Reference Quantities	27
5.1. Operating Conditions Are Schedules Within a Case	27
5.2. Defining Incidence	28
5.3. Defining Speed	28
5.3.1. Mach-number mode	28
5.3.2. Velocity mode	29
5.4. Selecting the Atmospheric Description	29
5.4.1. Standard-atmosphere mode	30
5.4.2. Specified-Reynolds-number mode	30
5.4.3. Wind-tunnel mode	30
5.4.4. Matching atmospheric values to a speed schedule	30
5.5. Reference Quantities Define the Coefficient Convention	31
5.5.1. Explicit references versus geometry-derived references	32
5.6. Boundary-Layer and Surface Inputs	32
5.7. Worked Example: NASA Flight and Reference Conditions	33
5.7.1. Complete runnable input	33
5.8. Inspecting the Resolved Flight and Reference State	35
5.8.1. Plotting the Mach schedule	36
5.9. Useful Variations of the Same Analysis	37
5.9.1. Use a standard atmosphere instead of specified Reynolds number	37
5.9.2. Use freestream velocity as the primary speed input	38
5.9.3. Observe geometry-derived reference values	38
5.10. Common Flight and Reference Mistakes	39
5.11. MATLAB-to-Legacy Field Map	39
6. Body Geometry	41
6.1. Choosing a Body Representation	41
6.2. Axisymmetric Bodies	42
6.2.1. Analytic axisymmetric geometry	42
6.2.2. Station-defined axisymmetric geometry	45
6.3. Elliptical Bodies	46
6.3.1. Station-defined elliptical geometry	46
6.3.2. Analytic elliptical geometry	48
6.4. Discontinuities and Section Boundaries	49
6.5. The Body Base and Exit Diameter	50
6.6. Worked Example: NASA N3 Body Geometry	51
6.6.1. Complete runnable input	52
6.7. Inspecting the Resolved Body Geometry	54
6.8. Useful Variations of the NASA Body	56
6.8.1. Sharp N1 nose	56
6.8.2. More strongly blunted N2 nose	56
6.8.3. Use station geometry for a custom mold line	57
6.9. Common Body-Geometry Mistakes	57
6.10. MATLAB-to-Legacy Field Map	58
7. Fin Sets, Panels, and Controls	60
7.1. Fin Sets and Physical Panels	60
7.1.1. Set identity and cumulative build-up	61
7.1.2. Spacing between multiple sets	62
7.2. Spanwise Planform Definition	62
7.2.1. Automatic Root Placement on the Body	62

7.2.2.	Leading-edge coordinates and sweep	64
7.3.	Panel Count, Clocking, and Dihedral	65
7.3.1.	Panel Clocking and Aerodynamic Roll	67
7.4.	Airfoil-Section Selection	67
7.4.1.	NACA Sections	67
7.4.2.	HEX and ARC Sections	69
7.4.3.	User-Defined Sections	70
7.5.	Plain Trailing-Edge Devices	71
7.6.	Fixed Panel Deflections	74
7.6.1.	Panel Deflection Sign Convention	74
7.6.2.	Hinge location and skew	75
7.7.	Longitudinal Trim	75
7.7.1.	Trim Search and Interpolation	76
7.7.2.	Trim Deflection Sign Convention	77
7.8.	Resolved Fin Geometry	77
7.9.	Worked Example: Adding NASA C6 Canards and T2 Tails	79
7.9.1.	NASA Planform Reconstruction	79
7.9.2.	NASA HEX Section Reconstruction	83
7.9.3.	Complete aligned C6–T2 input	84
7.9.4.	Aligned and interdigitated tails	87
7.9.5.	NASA Component Build-Up	87
7.9.6.	NASA Canard-Deflection Series	88
7.9.7.	NASA C6 Trim Example	88
7.10.	Common Fin and Control Modelling Mistakes	89
7.11.	MATLAB-to-Legacy Field Map	90
8.	Optional and Advanced Features	92
8.1.	Inlet Modelling	92
8.1.1.	Five-Station Geometry and Installation	93
8.1.2.	Covered Inlets, Additive Drag, and Mass Flow	98
8.2.	Body Protuberances	99
8.3.	Experimental-Data Substitution	103
8.3.1.	Dataset Reference Quantities	104
8.3.2.	Configuration Selection in the Cumulative Build-Up	105
8.3.3.	Historical Configuration Incrementing	105
8.4.	Configuration Synthesis and Aerodynamic Interference	106
8.5.	Dynamic Derivatives and Spin/Magnus Calculations	107
8.5.1.	Derivative Reporting System	108
8.5.2.	Spin and Magnus Effects	109
8.6.	Pressure-Distribution Outputs	111
8.6.1.	Choosing a Body Method	111
8.6.2.	Reading the Body and Fin Pressure Files	112
8.7.	Worked Advanced Example: NASA N3–C6–T2 Vehicle	113
8.7.1.	Reusable NASA Baseline	114
8.7.2.	Illustrative Inlet Addition	116
8.7.3.	Illustrative Protuberance Addition	117
8.7.4.	Dynamic Derivatives and Spin/Magnus Output	118
8.7.5.	Pressure-Output Demonstration	118
8.7.6.	Synthetic EXPR Demonstration	119
8.8.	Common Advanced-Feature Modelling Mistakes	120
8.9.	MATLAB-to-Legacy Field Map	121

9. Results Interpretation and Experimental Validation	123
9.1. Purpose and Comparison Strategy	123
9.2. Running the NASA Baseline	124
9.3. Organisation of the Aerodynamic Table	125
9.3.1. Cases and Sweep Identity	125
9.3.2. Zones and Zone Types	125
9.3.3. Configuration Numbers and Build-Up Levels	126
9.3.4. Conditions, Controls, and Coefficients	127
9.4. Inspecting the Per-Case Result	127
9.4.1. Generated Input Text	128
9.5. Inspecting the Resolved Analysis State	128
9.5.1. Resolved Flight and Reference Quantities	129
9.5.2. Resolved Configuration	129
9.5.3. Resolved Body and Fin Geometry	130
9.6. Retaining and Inspecting the Detailed Solver Outputs	131
9.6.1. Retained Directory Structure	132
9.6.2. Numbered Solver Files	133
9.6.3. SpearDAT Batch and Transport Files	134
9.7. Preparing the NASA TM X-73,218 Experimental Data	135
9.8. Complete-Configuration Comparison	135
9.8.1. Longitudinal Agreement	135
9.8.2. Lateral-Directional Coefficients	138
9.9. Understanding the Mach 0.8 Pitching-Moment Difference	138
9.10. Validation with a Fixed Canard Deflection	139
9.11. Extended Validation with NASA/TM-2005-213541	142
9.11.1. Test Article and Selected Configuration	143
9.11.2. Digital Data and Comparison Procedure	144
9.11.3. Zero-Roll Longitudinal Comparison	146
9.11.4. Tail Increment and the High-Mach Pitching Moment	151
9.11.5. Aerodynamic Roll and Body-Tail Interference	152
9.11.6. Tail-Control Deflections	154
9.11.7. Overall Configuration 3 Assessment	157
9.12. Elliptical-Body Validation with NASA TM X-3129	159
9.12.1. Test Articles and Conditions	159
9.12.2. Comparison Quantities and Digitised Data	160
9.12.3. Normal Force and Aerodynamic-Centre Comparison	161
9.13. Installed-Inlet and Tri-Tail Validation with NASA TM-84560	164
9.13.1. SpearDAT Reconstruction and Inlet-Closure Assumption	165
9.13.2. Digitised Figure 5 Data and Complete-Configuration Comparison	167
9.13.3. Installed T1 Increment	168
9.14. Validation Summary	168
10. Applicability, Assumptions and Engineering Limitations	172
10.1. Applicability Guidance and Terminology	172
10.1.1. Method Dependence of Mach-Regime Labels	172
10.2. Planar Lifting Surfaces, Controls and Interference	173
10.2.1. Recommended working envelope	174
10.2.2. Aspect ratio, taper and planform geometry	176
10.2.3. Nonlinear and high-incidence aerodynamics	176
10.2.4. Pitching moment and aerodynamic-centre placement	177
10.2.5. Axial force and airfoil-section assumptions	177

10.2.6. Plain trailing-edge controls	177
10.2.7. Panel, body-fin and downstream-surface interference	178
10.2.8. Practical interpretation of planar-surface results	179
10.3. Axisymmetric Body Methods and Pressure Predictions	179
10.3.1. Body Applicability Summary	180
10.3.2. Body Geometry and Fineness Ratio	180
10.3.3. Subsonic and Transonic Normal Force	183
10.3.4. High-Incidence Crossflow Aerodynamics	183
10.3.5. Supersonic and Hypersonic Body Methods	184
10.3.6. Axial Force, Wave Drag and Base Drag	184
10.3.7. Body Pressure-Output Applicability	185
10.3.8. Practical Interpretation of Body Results	185
10.4. Specialised Aerodynamic Capabilities	186
10.4.1. Elliptical Bodies	186
10.4.2. Dynamic Derivatives	189
10.4.3. Spin and Magnus Effects	189
10.4.4. Base-Jet Plume Interaction	191
10.5. Inlets, Protuberances and Longitudinal Trim	194
10.5.1. Inlet Aerodynamics	194
10.5.2. Protuberance Drag	197
10.5.3. Longitudinal Static Trim	198
A. Known Differences	202
A.1. Purpose and Scope	202
A.2. What This Appendix Does Not Claim	202
A.3. What Is Included, and What Is Not	202
A.4. Missile DATCOM in Brief	203
A.5. Register Conventions and Evidence	206
A.6. Inlets and Diverters	210
A.7. Input Processing and Runtime Portability	223
A.8. Flight Attitude and Directional Interpolation	225
A.9. Body Geometry and Definition	230
A.10. Fins, Controls, and Trim	235
A.11. Dynamic Derivatives, Spin, and Magnus	247
A.12. Experimental Data Substitution	254
A.13. Interpolation and Numerical Utilities	259
A.14. Output, Reporting, and Diagnostics	265
A.15. Precision and Representation	272
A.16. Body Aerodynamic Methods and Pressure Outputs	278
A.17. Elliptical-Body Aerodynamic Corrections	282
B. MATLAB Input Reference	287
B.1. Purpose and Use of This Reference	287
B.2. MATLAB Call and Top-Level Structures	288
B.3. Request-Wide Settings	292
B.4. Case Definition	294
B.5. Output Requests	301
B.6. Body-Theory Selection	304
B.7. Flight Conditions	305
B.8. Reference Quantities	311
B.9. Body, Base-Jet, and Protuberance Definition	315

B.10. Fin Sets	332
B.11. Fixed Panel Deflections	344
B.12. Longitudinal Trim	346
B.13. Inlet Definition	349
B.14. Experimental Data	356
B.15. Enumeration Index	366
References	368

1. Introduction

1.1. Purpose and Scope

This guide documents the MATLAB interface to SpearDAT and the engineering workflow around it. It is intended for users who wish to define missile configurations and flight conditions in MATLAB, execute aerodynamic analyses, inspect structured numerical results, and understand the assumptions and applicability of the underlying aerodynamic methods without manually preparing or interpreting legacy input and output files.

The guide covers the complete public workflow: installation and first use; the SpearDAT analysis model; flight conditions and reference quantities; body, lifting-surface, control, inlet, protuberance, and other optional definitions; result interpretation and experimental validation; engineering applicability and limitations; and the field-level MATLAB input reference. Historical Missile DATCOM terminology is retained where it is needed to explain aerodynamic conventions, method lineage, legacy mappings, or documented limitations.

This is a user guide for the SpearDAT MATLAB interface rather than a replacement for the published aerodynamic-method reports. Where the engineering meaning or applicability of a method matters to normal use, the relevant guidance is brought into the main chapters. More detailed historical implementation differences are separated into [appendix A](#) so that they do not interrupt the normal MATLAB workflow.

1.2. SpearDAT Overview

SpearDAT is a modern C++17 aerodynamic analysis framework with a structured application programming interface and a MATLAB interface. Its aerodynamic foundation is the publicly released Missile DATCOM methodology for preliminary-design analysis of conventional missile configurations [1]. SpearDAT exposes that capability through typed engineering inputs, systematic validation and defaulting, automated case preparation, isolated solver execution, and structured numerical outputs.

SpearDAT is neither a wrapper around the historical executable nor a purely mechanical source-code translation. The historical methodology, geometry conventions, and coefficient definitions remain the principal technical foundation, while the implementation has been reconstructed in modern C++ and integrated with a public API designed for repeatable programmatic use. Verified defects, unsafe numerical behaviour, incomplete execution paths, and obsolete implementation assumptions are corrected or completed where required, with the resulting differences documented separately.

The aerodynamic methods remain analytical and semi-empirical preliminary-design methods rather than general CFD models. A successful calculation therefore means that SpearDAT was able to construct and execute the requested analysis; it does not by itself establish that every contributing correlation is being used within its strongest aerodynamic evidence range. [Chapter 10](#) provides the corresponding engineering applicability guidance.

1.3. Verification, Validation and Known Differences

Three related ideas are kept distinct throughout this guide.

Implementation verification Verification addresses whether the SpearDAT software performs the intended calculation consistently. The implementation is supported by automated unit, integration, and full-stack testing across the public input path, case preparation, aerodynamic execution, and result-processing workflow. Where a historical behaviour has been corrected, preserved, or intentionally changed, the relevant evidence is recorded with that difference in [appendix A](#).

Aerodynamic validation Validation addresses a different question: whether the resulting aerodynamic predictions reproduce physical behaviour with useful engineering fidelity. [Chapter 9](#) compares SpearDAT directly with published NASA experimental data for materially different configurations, including body–fin vehicles, control deflections, aerodynamic roll, elliptical bodies, and an installed-inlet configuration. The comparisons show both regions of useful agreement and regions of reduced fidelity.

Known differences [Appendix A](#) records verified differences between SpearDAT and the historical implementation, together with selected historical behaviours that are deliberately preserved and corrections whose effects are relevant to engineering review. This appendix is reference material; it is not required for defining or interpreting an ordinary SpearDAT analysis.

The published Missile DATCOM documentation remains an important reference for the lineage and intent of the inherited aerodynamic methods. It is not, however, treated as a requirement to reproduce defective, undefined, or unsafe software behaviour. Where such a behaviour has been verified, the corrected and documented SpearDAT implementation is the relevant software reference.

1.4. MATLAB Interface Overview

The MATLAB interface is designed around a direct engineering workflow. The user defines the analysis using MATLAB structures, calls SpearDAT, and receives one structured result containing the combined aerodynamic table, per-case information, resolved analysis state, and diagnostics. The principal workflow is shown in [figure 1.1](#).

SpearDAT prepares the required scalar analyses automatically and returns the results in a form suitable for ordinary MATLAB filtering, plotting, comparison, and automation. Generated input text and detailed solver-output files can also be retained when they are required for traceability, validation work, or engineering review, but retained legacy-format files are optional and are not part of the routine MATLAB workflow.

1.5. Intended Audience

This guide is written for three overlapping groups:

MATLAB users Engineers who want to define an analysis, call `spear_dat_mex`, and work directly with MATLAB tables and structures.

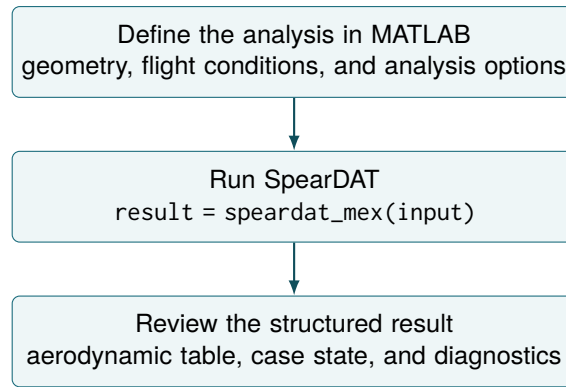


Figure 1.1.: Basic SpearDAT MATLAB workflow. The calling pattern and analysis model are introduced in [chapters 3 and 4](#), while detailed result interpretation is developed in [chapter 9](#).

Aerodynamics and preliminary-design users Readers who need to understand supported configurations, units, coefficient conventions, attitude definitions, controls, trim, aerodynamic applicability, and the engineering meaning of the returned data.

Advanced and validation users Analysts and developers who require resolved case state, diagnostics, generated input text, retained legacy-format outputs, experimental comparisons, or detailed traceability of an aerodynamic result.

No knowledge of Fortran namelists is required for normal MATLAB use. Legacy terms are introduced only where they remain relevant to the aerodynamic model, a documented limitation, a documented historical behaviour, or the mapping between a public SpearDAT field and the underlying solver convention.

1.6. Guide Organisation and Conventions

The guide is organised so that it can be read sequentially by a new user or used selectively as an engineering reference.

Getting started [Chapters 2 to 4](#) cover installation, the first executable MATLAB examples, and the conceptual model used to turn one MATLAB request into configurations, cases, sweeps, zones, and structured results.

Defining the aerodynamic model [Chapters 5 to 8](#) describe the public engineering inputs: flight conditions and reference quantities, body geometry, fin sets and controls, and the optional or specialised analysis features.

Interpreting confidence in the results [Chapter 9](#) explains the returned result structures and compares representative SpearDAT predictions with experimental data. [Chapter 10](#) then collects the principal method assumptions, correlation ranges, modelling limitations, and recommended engineering interpretation.

Reference material [Appendix A](#) documents verified historical implementation differences for users who need that level of traceability. [Appendix B](#) is the field-level MATLAB input reference and should be used when an exact public field name, accepted form, default, enumeration, or validation rule is required.

Public MATLAB field names, values, and executable names are typeset in a monospaced font. Historical input names and file names are identified explicitly as legacy mappings or legacy-format artefacts. Unless a section specifically discusses historical behaviour, statements in this guide describe the public SpearDAT interface and the current SpearDAT implementation.

2. Installation and MATLAB Configuration

2.1. System Requirements

SpearDAT is supplied as a platform-specific MATLAB package. Before installing it, confirm that the package matches the operating system and processor architecture of the computer on which MATLAB will run.

The user requires:

- MATLAB R2026a, or another compatible MATLAB release capable of loading the supplied platform-specific MEX binary;
- a SpearDAT package matching the operating system and processor architecture of the computer on which MATLAB will run;
- permission to install or extract the package and execute files from the selected installation folder; and
- sufficient storage for the installation and any retained analysis outputs.

SpearDAT v0.1 has been developed and exercised using MATLAB R2026a. Earlier MATLAB releases have not been formally qualified. Other releases may be compatible provided that they can load the supplied platform-specific MEX binary and support the MATLAB APIs used by SpearDAT, but compatibility outside R2026a should be regarded as unverified.

Platform-specific SpearDAT packages are provided for 64-bit Windows and macOS on Apple silicon. A package built for one platform or processor architecture cannot be used interchangeably with another.

A source repository, CMake, and a C++ compiler are not required to use the MATLAB interface.

2.2. Package Contents

A SpearDAT installation provides the MATLAB interface, the matching solver runtime, example analyses, and user documentation. The principal supplied components are listed in [table 2.1](#).

Table 2.1.: Principal components supplied with SpearDAT.

Component	Purpose
Platform-specific MEX binary	Provides the MATLAB entry point <code>speardat_mex</code> .
Solver executable	Performs the aerodynamic analysis using the matching SpearDAT runtime.
<code>setup_speardat.m</code>	Configures the MATLAB path for the installed package.
Example scripts	Provide ready-to-run analyses and starting points for new cases.
User guide and release information	Document installation, inputs, outputs, examples, supported platforms, and known limitations.

Important

The MEX binary is not a complete SpearDAT installation by itself. It must be used with the matching `speardat_solver_process` executable supplied for the same operating system and processor architecture.

The installed package uses the following directory layout:

```
SpearDAT/
|-- matlab/
|   |-- speardat_mex.<platform extension>
|   |-- setup_speardat.m
|   |-- examples/
|-- bin/
|   |-- speardat_solver_process[.exe]
|-- docs/
|   |-- SpearDAT_MATLAB_Interface_User_Guide.pdf
|-- LICENSE.txt
|-- README.txt
```

Listing 2.1: Installed SpearDAT package layout.

The relative locations of the `matlab` and `bin` directories must be preserved. Moving the MEX binary or solver executable away from the installed package may prevent SpearDAT from starting correctly.

2.3. Installing the Package

Install SpearDAT by extracting or copying the complete platform-specific package to a suitable location. The selected location should be writable when examples or retained outputs are to be stored within it; alternatively, those files may be written to a separate working directory.

Do not copy only the MEX binary. Keep the complete package structure shown above so that the MATLAB interface can locate the matching solver executable.

2.4. Configuring MATLAB

Start MATLAB and run the setup script from the installed package. In the following example, replace the placeholder path with the actual SpearDAT installation directory:

```
speardatRoot = "/path/to/SpearDAT";
run(fullfile(speardatRoot, "matlab", "setup_speardat.m"));
```

Listing 2.2: Configuring the installed SpearDAT package.

The setup script adds the installed SpearDAT MATLAB interface and examples to the MATLAB path and prints the resolved package locations.

2.5. Verifying the Installation

A complete SpearDAT installation has two parts, and both must be verified. The MATLAB interface locates the solver executable only when an analysis is actually requested, so a package with a correctly installed MEX binary and a missing or misplaced solver executable will appear to be installed correctly and will then fail on the first analysis.

2.5.1. Step 1: Verify the MATLAB Interface

Verify that MATLAB can locate the active MEX binary:

```
which speardat_mex -all
mexext

mexPath = which("speardat_mex");
assert(~isempty(mexPath), ...
    "SpearDAT is not available on the MATLAB path.");
```

Listing 2.3: Verifying the MATLAB interface.

The extension reported by mexext must match the MEX binary supplied with the installed package. Examples include mexw64 for 64-bit Windows and mexmac64 for macOS on Apple silicon.

If whichspeardat_mex-all reports more than one result, an earlier entry on the MATLAB path is shadowing the installed package. MATLAB uses the first entry, and the solver executable is then resolved relative to that copy rather than to the installation being verified. Remove the unintended path entries before continuing.

2.5.2. Step 2: Verify the Solver Executable

The MATLAB interface resolves the solver executable relative to the active MEX binary, using the bin directory of the installed package. Confirm that the executable is present and runnable:

```
matlabDirectory = fileparts(which("speardat_mex"));

if ispc
    solverName = "speardat_solver_process.exe";
else
    solverName = "speardat_solver_process";
end

solverPath = fullfile(matlabDirectory, "..", "bin", solverName);

assert(isfile(solverPath), ...
    "The SpearDAT solver executable was not found at: %s", ...
    solverPath);

if ~ispc
    assert(system("test -x \"" + solverPath + "\"") == 0, ...
        "The SpearDAT solver executable is not runnable: %s", ...
        solverPath);
end
```

Listing 2.4: Verifying the solver executable.

If the executable is missing, the package structure shown in [listing 2.1](#) has not been preserved. On macOS, if it is present but not runnable, the execute permission may have been lost during extraction or transfer; restore it before continuing.

Important

Both checks must pass. Locating the MEX binary alone does not confirm that an analysis can be executed.

2.5.3. Step 3: Run a Complete Analysis

After both components have been located successfully, continue with [chapter 3](#) and run Example 1. A successful Example 1 run is the only complete confirmation that the installation works end to end.

Note

On macOS, packages downloaded through a browser or transferred from another computer may be subject to operating-system security or quarantine restrictions. If the MEX binary or solver executable is blocked, follow the security instructions supplied with the SpearDAT distribution before repeating the verification steps.

3. MATLAB Quick Start

This chapter introduces the basic MATLAB workflow using two supplied examples. Example 1 performs a single analysis and introduces the returned result. Example 2 requests several sideslip conditions in one call and demonstrates how results from multiple cases are organised in a combined MATLAB table.

3.1. Example 1: Run a Single Analysis

After configuring the package as described in [chapter 2](#), run Example 1 from the MATLAB command window:

```
example_01_single_case
```

Listing 3.1: Running Example 1.

The example script obtains the complete SpearDAT input from the supplied helper function `make_speardat_example_vehicle`, assigns the example identity, executes the analysis, and leaves the variables `input` and `result` in the MATLAB workspace. Open `make_speardat_example_vehicle.m` in the MATLAB Editor to review the geometry, reference quantities, flight conditions, and analysis options shared by the two quick-start examples.

The execution part of the script follows this pattern:

```
result = speardat_mex(input);

if ~result.succeeded
    disp(result.diagnostics);
    error("SpearDAT:ExampleFailed", ...
        "The SpearDAT analysis did not complete successfully.");
end
```

Listing 3.2: Basic SpearDAT call.

3.2. Check the Execution Status

Every call returns one top-level MATLAB structure. The principal status fields are:

```
result.succeeded
result.hasErrors
result.caseCount
result.successfulCaseCount
result.failedCaseCount
```

For a successful single-case request, the expected values are shown in [table 3.1](#).

Diagnostics are reported at two levels. `result.diagnostics` contains request-level and batch-level entries, such as a rejected input or an invalid execution option. Diagnostics that belong to one individual case are reported in `result.cases(k).diagnostics` instead. When a request is unsuccessful, inspect both.

Table 3.1.: Expected top-level status for a successful single-case request.

Field	Expected value
<code>result.succeeded</code>	<code>true</code>
<code>result.hasErrors</code>	<code>false</code>
<code>result.caseCount</code>	1
<code>result.successfulCaseCount</code>	1
<code>result.failedCaseCount</code>	0

Note

In the current release, `result.hasErrors` is the logical complement of `result.succeeded`; the two fields never disagree. `result.successfulCaseCount` and `result.failedCaseCount` are the fields that distinguish a complete failure from a partially successful multi-case request.

The organisation of the combined aerodynamic table and the per-case result structures is developed further in [sections 9.3 and 9.4](#).

3.3. Inspect the Aerodynamic Table

The aerodynamic results are returned in a MATLAB table:

```
aero = result.aerodynamics;
size(aero)
head(aero)
```

Each row contains identifying information, the associated flight and control conditions, and the computed aerodynamic coefficients. Commonly used columns include:

AlphaDeg Angle of attack in degrees.

Mach Freestream Mach number.

CN, CM, and CA Normal-force, pitching-moment, and axial-force coefficients.

CY, CLN, and CLL Side-force, yawing-moment, and rolling-moment coefficients.

CL and CD Lift and drag coefficients.

DeltaDeg Control deflection in degrees associated with the row. For a trimmed result this is the solved deflection.

The table organisation and column semantics are described in detail in [section 9.3](#).

3.4. Select Data for One Operating Condition

SpearDAT groups related aerodynamic rows into numbered output groups called *zones*. A zone normally contains an angle-of-attack schedule for one combination of flight and control conditions. `ZoneNumber` is an output-group identifier; it does not describe a physical region of the vehicle.

Select the first zone from Example 1:

```
selected = aero(aero.ZoneNumber == 1, :);
head(selected)
```

Listing 3.3: Selecting one output zone.

ZoneNumber restarts at one within each case. For results containing more than one case, use both CaseIndex and ZoneNumber when selecting data. The ZoneType column identifies what the zone contains; the complete zone-type definitions are given in [section 9.3.2](#).

Note

CaseIndex is zero-based because it preserves the case identity used by the SpearDAT C++ API. MATLAB array positions are one-based. Therefore, `result.cases(1).caseIndex` is normally zero.

3.5. Plot the Results

The selected rows can be plotted using standard MATLAB functions:

```
figure
plot(selected.AlphaDeg, selected.CN, "o-")
grid on
xlabel("Angle of attack, \alpha (deg)")
ylabel("Normal-force coefficient, C_N")
title("SpearDAT normal-force result")

figure
plot(selected.AlphaDeg, selected.CM, "o-")
grid on
xlabel("Angle of attack, \alpha (deg)")
ylabel("Pitching-moment coefficient, C_M")
title("SpearDAT pitching-moment result")
```

Listing 3.4: Plotting normal-force and pitching-moment coefficients.

The engineering examples in [chapters 5 to 8](#) extend the same workflow to Mach schedules, geometry changes, control deflections, trim, and specialised outputs.

3.6. Example 2: Run a Sideslip Sweep

Example 2 requests several sideslip angles, denoted by β , in one SpearDAT call:

```
example_02_beta_sweep
```

Listing 3.5: Running Example 2.

This is presented as a separate example because each requested sideslip value is evaluated as an individual case, while all aerodynamic rows are returned in one combined table. The same result-handling approach applies to a sweep of aerodynamic roll angle, denoted by ϕ .

Confirm the number of completed cases and inspect the requested sweep values:

```

result.caseCount
result.successfulCaseCount

sweepValues = unique( ...
    result.aerodynamics.SweepValueDeg, "stable")

```

The leading metadata columns identify the origin of each row:

CaseIndex and CaseName Identify the individual case within the request.

SweepIndex, SweepVariable, and SweepValueDeg Identify the sweep position, the swept variable, and its value in degrees.

ZoneNumber Identifies the output zone within one case.

ZoneType Identifies the kind of result the zone contains: NoTrim for a standard untrimmed analysis, Untrimmed for a result computed at a commanded control deflection, and Trimmed for a solved trim result.

Configuration The configuration identifier reported for NoTrim zones. It is NaN for other zone types.

Note

Numeric columns that do not apply to a given row contain NaN rather than being empty. This affects Configuration, CommandedDeltaDeg, and SweepValueDeg for requests without a sweep. Use isnan rather than isempty when filtering on these columns.

The following example compares normal-force results from the first zone of each sideslip case:

```

aero = result.aerodynamics;
sweepValues = unique(aero.SweepValueDeg, "stable");

figure
hold on
for k = 1:numel(sweepValues)
    rows = aero.SweepValueDeg == sweepValues(k) & ...
        aero.ZoneNumber == 1;

    labelText = "\beta = " + ...
        string(sweepValues(k)) + " deg";

    plot(aero.AlphaDeg(rows), aero.CN(rows), "o-", ...
        "DisplayName", labelText);
end
hold off
grid on
xlabel("Angle of attack, \alpha (deg)")
ylabel("Normal-force coefficient, C_N")
legend("Location", "best")
title("SpearDAT sideslip sweep")

```

Listing 3.6: Comparing the cases in a sideslip sweep.

3.7. Inspect an Individual Case

Detailed information for each case is available under `result.cases`:

```
caseResult = result.cases(1);

caseResult.caseName
caseResult.succeeded
caseResult.diagnostics
caseResult.for006Status
caseResult.resolvedState
```

The fields `aerodynamicRowStart` and `aerodynamicRowCount` locate the case within the combined table:

```
rows = caseResult.aerodynamicRowStart + ...
    (0:(caseResult.aerodynamicRowCount - 1));

caseTable = result.aerodynamics(rows, :);
```

3.8. Retain Solver Files When Required

Normal MATLAB execution uses temporary working files and removes them after the result has been reconstructed. To retain the generated input and solver outputs, supply an options structure:

```
options = struct;
options.legacyFiles = true;
options.outputDirectory = fullfile(pwd, "speardat_output");

result = speardat_mex(input, options);
```

Listing 3.7: Retaining the solver-output bundle.

The selected output directory contains a run manifest, a batch status file, and one subdirectory for each case. These files are intended for traceability, detailed review, and direct comparison with the standalone solver.

Important

The output directory must be absent or empty. SpearDAT *rejects* a non-empty output directory and raises an error rather than writing into it, so a second retained run into the same directory will fail. Use a new directory for each retained run, or remove the previous one first.

Normal temporary execution is recommended unless the generated input and solver-output files are required for engineering review. When `legacyFiles` is true, `options.outputDirectory` is required; when it is false, the field must be omitted or empty.

3.9. Quick-Start Result Map

Table 3.2.: Main result fields introduced in this chapter.

Field	Purpose
<code>result.succeeded</code>	True only when the complete non-empty request succeeds.
<code>result.hasErrors</code>	Logical complement of <code>result.succeeded</code> .
<code>result.caseCount</code>	Number of individual cases produced by the request.
<code>result.successfulCaseCount</code>	Number of cases that produced a complete result.
<code>result.failedCaseCount</code>	Number of cases that did not.
<code>result.diagnostics</code>	Request-level and batch-level diagnostics. Per-case diagnostics are reported in <code>result.cases(k).diagnostics</code> .
<code>result.aerodynamics</code>	One combined table containing the aerodynamic rows from all requested cases.
<code>result.cases</code>	Ordered per-case structures containing identity, status, diagnostics, resolved state, generated input text, and table row ranges.
<code>result.legacyOutputDirectory</code>	Output directory used when retained solver files are enabled; empty during normal temporary execution.

4. The SpearDAT Analysis Model

Chapter 3 introduced the MATLAB interface through complete examples. This chapter defines the analysis model used by those examples. It explains how a SpearDAT input is organised, how the vehicle definition is separated from its operating conditions, and how one MATLAB request can produce one or more independently evaluated cases.

The chapter is concerned with the public MATLAB interface. It does not require knowledge of the historical input-deck format or of the software implementation. Detailed definitions of the flight-condition, reference, geometry, control, inlet, and experimental-data fields are introduced in the subsequent engineering chapters.

4.1. How SpearDAT Represents an Analysis

A SpearDAT analysis begins with one scalar MATLAB structure named `input`. The structure is passed to the MATLAB interface and one top-level result structure is returned:

```
result = speardat_mex(input);
```

Listing 4.1: The SpearDAT MATLAB calling pattern.

The input is a hierarchical engineering description rather than an unstructured collection of name–value pairs. Related quantities are grouped into branches: run-wide settings, flight conditions, reference quantities, body geometry, fin sets, controls, optional components, calculation requests, and output requests. This organisation makes the meaning and ownership of each field explicit. For example, a body station belongs to the body definition, whereas a Mach number belongs to the flight-condition definition.

At the highest level, the input contains two branches:

input.settings Settings that apply to the request as a whole, including the selected unit systems and run-wide options.

input.caseInput The vehicle configuration, operating conditions, calculation choices, and output requests that define the aerodynamic analysis.

SpearDAT checks the supplied structure before starting an analysis. It then prepares one or more cases from the request, evaluates those cases independently, and assembles their outputs into the single returned result structure. A request that contains no attitude sweep normally produces one case. A request containing several sideslip or aerodynamic-roll values produces one case for each retained value.

4.2. Structure of the MATLAB Input

The principal branches of the MATLAB input are shown in [table 4.1](#). The table is an orientation map rather than a field reference. Later chapters define the fields within each engineering branch.

The same hierarchy can be viewed as a compact MATLAB field tree:

Table 4.1.: Principal branches of the SpearDAT MATLAB input.

Branch	Purpose
<code>input.settings</code>	Unit systems and options that apply to the complete request.
<code>input.caseInput.id</code>	User-supplied identity for the requested analysis.
<code>input.caseInput.options</code>	Calculation requests such as dynamic derivatives, spin/Magnus effects, and input-checking-only operation.
<code>input.caseInput.flight</code>	Incidence, speed, and atmospheric conditions.
<code>input.caseInput.reference</code>	Reference area, reference lengths, moment centre, and related non-dimensionalisation inputs.
<code>input.caseInput.theory</code>	Selection of the applicable body aerodynamic method.
<code>input.caseInput.body</code>	Body geometry and body-mounted additions such as base-jet data and protuberances.
<code>input.caseInput.finSets</code>	Zero or more fin-set definitions.
<code>input.caseInput.deflections</code>	Commanded panel deflections and hinge-line data.
<code>input.caseInput.trim</code>	Trim request and the panels available to the trim solution.
<code>input.caseInput.inlet</code>	Optional inlet, diverter, placement, and mass-flow data.
<code>input.caseInput.experimentalData</code>	Experimental coefficient data used for substitution or incrementing.
<code>input.caseInput.output</code>	Requests for aerodynamic tables, pressure outputs, partial results, and selected detailed reports.

```

input
|-- settings
`-- caseInput
    |-- id
    |-- options
    |-- flight
    |-- reference
    |-- theory
    |-- body
    |-- finSets
    |-- deflections
    |-- trim
    |-- inlet
    |-- experimentalData
    `-- output

```

Listing 4.2: High-level structure of a SpearDAT input.

The optional second argument to `spear_dat_mex` is separate from this analysis structure:

```

options = struct;
options.legacyFiles = true;
options.outputDirectory = fullfile(pwd, "spear_dat_output");

result = spear_dat_mex(input, options);

```

Listing 4.3: Supplying MATLAB execution options separately from the aerodynamic input.

The distinction is deliberate. `input.settings.options` and `input.caseInput.options` select analysis behaviour, whereas the second argument controls how the MATLAB call is executed and whether its legacy-format files are retained. Retained-output operation was introduced in [chapter 3](#).

Not every branch must be populated for every analysis. A body-alone case, for example, has no fin-set, deflection, trim, or inlet definition. Fields that do not apply are normally omitted rather than populated with placeholder values. The distinction between omission, an empty MATLAB value, and an explicit numeric zero is defined later in this chapter.

4.3. Requests, Configurations, Cases, Sweeps, and Zones

Several levels of organisation appear between the MATLAB input and the aerodynamic table. The terms in [table 4.2](#) are used consistently throughout this guide.

Table 4.2.: Core SpearDAT analysis terminology.

Term	Meaning
Request	One call to <code>speardat_mex(input)</code> and the complete input structure supplied to that call.
Vehicle configuration	The physical configuration described by <code>input.caseInput</code> : body, fin sets, controls, inlet, protuberances, reference quantities, and the associated analysis choices.
Case	One independently evaluated analysis produced from the request. A generated case contains at most one sideslip value or one aerodynamic-roll value, but it may retain complete angle-of-attack, speed, and atmospheric schedules.
Sweep	An ordered family of cases generated from several requested values of one supported sweep variable. The current public sweep variables are sideslip angle and aerodynamic roll angle.
Schedule	An ordered set of values evaluated within a case, such as an angle-of-attack schedule or a Mach-number schedule. A schedule does not, by itself, create additional top-level cases.
Zone	A group of related aerodynamic rows within a completed case, normally for one combination of flight and control conditions. Zones are identified in the returned table by <code>ZoneNumber</code> and <code>ZoneType</code> .

Note

In this chapter, *vehicle configuration* means the complete physical definition supplied by the user. The numeric `result.aerodynamics.Configuration` column introduced in [chapter 3](#) identifies reported configuration build-up stages within the aerodynamic output and is a separate concept.

The current MATLAB interface accepts one vehicle configuration in each request. Analyses of genuinely different vehicles—for example, two bodies with different station geometry or two different fin arrangements—are made using separate calls. A sideslip or aerodynamic-roll sweep is different: the vehicle definition is unchanged, and only the selected attitude value differs between the generated cases.

4.4. Schedules and Case Expansion

A MATLAB vector does not have one universal meaning in the SpearDAT input model. Its meaning is determined by the field to which it is assigned.

An angle-of-attack vector is a schedule evaluated within a case. A Mach vector is likewise a speed schedule. Geometry vectors describe the stations or panels of one component. By contrast, a vector assigned to `input.caseInput.flight.betaDeg` or `input.caseInput.flight.rollDeg` requests a sweep and is expanded into independently evaluated cases.

The following fragment isolates the fields relevant to this distinction; the remainder of the vehicle definition is omitted for clarity.

```
input.caseInput.flight.attitudeMode = "BodyAlphaBeta";

input.caseInput.flight.alphaDeg = [0, 5, 10];
input.caseInput.flight.betaDeg = [-5, 0, 5];

input.caseInput.flight.speed.mode = "Mach";
input.caseInput.flight.speed.mach = [0.8, 1.2];
```

Listing 4.4: A sideslip sweep containing angle-of-attack and Mach schedules.

This request produces three cases:

Case position	Sideslip angle	Schedules retained in the case
1	-5°	$\alpha = [0, 5, 10]^\circ$, $M = [0.8, 1.2]$
2	0°	$\alpha = [0, 5, 10]^\circ$, $M = [0.8, 1.2]$
3	5°	$\alpha = [0, 5, 10]^\circ$, $M = [0.8, 1.2]$

The request therefore does not create eighteen top-level cases from every combination of three sideslip values, three angles of attack, and two Mach numbers. It creates three independently executed sideslip cases. Each case retains the complete angle-of-attack and Mach schedules, which determine the conditions represented by its output rows and zones.

Important

A SpearDAT case is not one row of the aerodynamic table. One case may contain several Mach conditions, an angle-of-attack schedule, and multiple output zones. `CaseIndex` identifies the independently executed case; `ZoneNumber` and the flight-condition columns identify the results within that case.

Only one attitude family can define a request: sideslip (`betaDeg`) or aerodynamic roll (`rollDeg`). They are not combined into a Cartesian product. The attitude modes, sweep ordering, duplicate handling, and treatment of explicit zero are defined in the next part of this chapter.

4.5. MATLAB Value and Structure Rules

A field path such as `input.caseInput.flight.alphaDeg` describes a sequence of nested MATLAB structures. The dots are structure-member operators: the value `alphaDeg` belongs to `flight`, which belongs to `caseInput`, which belongs to the top-level input structure.

SpearDAT applies strict representation rules at this MATLAB boundary. These rules detect misspelled fields, ambiguous array shapes, and unintended data types before they can change an aerodynamic analysis.

4.5.1. Scalar Structures and Repeated Objects

The top-level input value must be one scalar MATLAB structure. Most nested branches, including settings, caseInput, flight, reference, and body, must also be scalar structures.

Repeated engineering objects use structure vectors. Examples include:

- `input.caseInput.finSets;`
- `input.caseInput.deflections.finSets;`
- `input.caseInput.body.protuberances.items;`
- `input.caseInput.inlet.placements;` and
- `input.caseInput.experimentalData.datasets.`

A repeated-object branch may be a row or column structure vector. A two-dimensional structure matrix is rejected because its ordering would be ambiguous. Where a repeated object has an engineering identity—for example, a fin-set number or inlet-placement index—that identity is carried by a field within the object rather than by the MATLAB array shape alone.

4.5.2. Numeric, Logical, and Text Values

The accepted MATLAB forms are summarised in [table 4.3](#).

Table 4.3.: MATLAB representation rules used by the SpearDAT input.

Public value	Accepted MATLAB form	Important details
Numeric scalar	Real, full double scalar	Complex, sparse, single-precision, and integer-class arrays are not accepted.
Numeric vector	Real, full double scalar, row vector, or column vector	A general matrix is rejected rather than flattened implicitly. Row and column vectors retain the same requested order.
Integer-valued field	Real, full double scalar or vector, as appropriate	Every value must be finite and exactly integer-valued. Use 1, not <code>uint32(1)</code> .
Boolean	MATLAB logical scalar	Use <code>true</code> or <code>false</code> ; numeric 1 and 0 are not Boolean substitutes.
Text scalar	MATLAB string scalar or character row vector	Enumeration values and identifiers use this representation.
Text vector	String array, cell vector of text, or one character row	Used by fields that accept several names or report targets.

Ordinary MATLAB numeric literals are doubles, so the required form is natural in most scripts:

```
input.settings.lengthSystem = "Inches";
input.caseInput.options.dynamicDerivatives = false;

input.caseInput.flight.alphaDeg = 0:5:20;           % row vector
input.caseInput.flight.speed.mach = [0.8; 1.2]; % column vector

input.caseInput.finSets(1).set = 1;                 % representation example
```

Listing 4.5: Examples of accepted MATLAB value forms.

The following values are rejected at the MATLAB interface:

```
input.caseInput.flight.alphaDeg = single([0, 5, 10]);
input.caseInput.flight.speed.mach = [0.8, 1.2; 1.5, 2.0];
input.caseInput.options.dynamicDerivatives = 1;
input.caseInput.finSets(1).set = uint32(1);
```

Listing 4.6: Examples of values that do not satisfy the public MATLAB representation rules.

These are representation errors, not aerodynamic-limit decisions. The corresponding engineering value may be meaningful, but it must first be expressed in the public MATLAB form.

4.5.3. Field Names and Enumerated Choices

Public field names and enumerated text values are exact and case-sensitive. For example:

```
input.settings.lengthSystem = "Inches";
input.settings.derivativeSystem = "Degrees";

input.caseInput.flight.attitudeMode = "BodyAlphaBeta";
input.caseInput.flight.speed.mode = "Mach";
input.caseInput.flight.atmosphere.mode = "StandardAtmosphere";
```

Listing 4.7: Exact public names used by the MATLAB interface.

Values such as "inches", "DEG", "bodyAlphaBeta", or "mach" are not aliases for the public names above. Similarly, an unknown field is rejected even when it resembles a valid one. This strictness prevents a spelling error from being silently ignored and leaving the analysis to run with an unintended default.

Important

Use the MATLAB names shown in this guide and in the supplied examples. Names used in the historical input format, in language-neutral schema artefacts, or in internal source code are not alternative MATLAB spellings.

4.6. Attitude Modes and BETA/PHI Sweeps

SpearDAT supports the two incidence descriptions inherited from Missile DATCOM. The selected mode determines how `input.caseInput.flight.alphaDeg` is interpreted and which lateral attitude field may accompany it.

Table 4.4.: Public attitude modes.

attitudeMode	Associated fields	Interpretation
"BodyAlphaBeta"	alphaDeg, betaDeg	Body-axis angle of attack and body-axis sideslip angle.
"TotalAlphaRoll"	alphaDeg, rollDeg	Total incidence and aerodynamic roll angle, which orient the incidence plane around the vehicle axis.

In "TotalAlphaRoll" mode, rollDeg is the aerodynamic roll quantity traditionally named PHI in Missile DATCOM. It describes the orientation of the incidence plane; it is not a six-degree-of-freedom Euler roll attitude.

The two lateral fields are mutually exclusive. A request must not populate both betaDeg and rollDeg, and an explicitly selected mode must agree with the populated field:

```
% Body-axis alpha and sideslip
input.caseInput.flight.attitudeMode = "BodyAlphaBeta";
input.caseInput.flight.betaDeg = [-5, 0, 5];
input.caseInput.flight.rollDeg = [];

% Alternatively: total incidence and aerodynamic roll
input.caseInput.flight.attitudeMode = "TotalAlphaRoll";
input.caseInput.flight.betaDeg = [];
input.caseInput.flight.rollDeg = [-90, 0, 90];
```

Listing 4.8: Valid selection of either BETA or aerodynamic roll.

If attitudeMode is omitted, SpearDAT infers it only from the populated lateral-attitude field:

- a non-empty rollDeg selects "TotalAlphaRoll";
- otherwise, "BodyAlphaBeta" is selected.

Explicit selection is recommended in user scripts because it states the intended interpretation of alphaDeg directly. It also makes the script easier to review when an attitude vector is later changed or removed.

4.6.1. Expansion Rules

A scalar betaDeg or rollDeg produces one case. A vector produces one case for each retained value. Expansion follows these rules:

1. values must be finite;
2. the requested order is preserved;
3. exact duplicate values are removed;
4. the first occurrence of each exact value is retained;
5. an explicit zero is retained as an ordinary requested value; and
6. BETA and PHI are never combined into a Cartesian product.

For example:

```
input.caseInput.flight.attitudeMode = "BodyAlphaBeta";
input.caseInput.flight.betaDeg = [-5, 0, 5, 0, -5];

result = speardat_mex(input);
```

Listing 4.9: Stable removal of exact duplicate BETA values.

The retained values are -5° , 0° , and 5° , in that order. The duplicate entries generate warnings, but they do not prevent the three retained cases from running.

Duplicate removal is exact; SpearDAT does not normalise the values or silently replace them with an equivalent angular range. Distinct values separated by an integer multiple of 360° , such as 0° and 360° , remain separate requested cases. SpearDAT reports that they may represent physically equivalent attitudes, but preserves both values because the user supplied them explicitly.

Leaving both betaDeg and rollDeg empty requests no attitude sweep. If attitudeMode is also omitted, SpearDAT selects "BodyAlphaBeta". No BETA or PHI field is written for the generated case, and the default zero lateral-attitude condition is used. The case metadata records:

```
sweepVariable = "None"
sweepValueDeg = []
```

Supplying an explicit scalar zero is different. For example:

```
input.caseInput.flight.attitudeMode = "BodyAlphaBeta";
input.caseInput.flight.betaDeg = 0.0;
```

Listing 4.10: Recording an intentional zero-sideslip condition explicitly.

This still produces one case, but its metadata records sweepVariable="BetaDeg" and sweepValueDeg=0. The equivalent rule applies to an explicit zero aerodynamic-roll condition using "TotalAlphaRoll" and rollDeg=0.0.

Note

Omitted values and inferred modes are valid, but they contain less information about the analyst's intent. Where the selected attitude description or a zero BETA/PHI value forms part of the defined test condition, specify the mode and scalar value explicitly. This makes the intended condition visible in the input and preserves it in the case metadata.

4.7. Omitted Fields, Empty Values, Defaults, and Explicit Zero

SpearDAT preserves whether a public value was supplied. This matters because an omitted field may permit a default or a geometry-derived value, whereas an explicit zero may carry a valid engineering meaning.

At the MATLAB interface, a missing field and an empty value are both treated as unset:

```
% The xcg field is not created.
input.caseInput.reference = struct;

% Alternatively, an existing xcg field is cleared.
```

```
input.caseInput.reference.xcg = [];
```

Listing 4.11: Equivalent ways of leaving an optional field unset.

By contrast:

```
input.caseInput.reference.xcg = 0.0;
input.caseInput.flight.betaDeg = 0.0;
```

Listing 4.12: An explicit zero is a supplied value.

The first assignment requests a centre-of-gravity location of zero in the selected length system. The second requests one body-axis sideslip case at zero degrees and records that case as a BETA case. Neither value is treated as missing.

Table 4.5.: Presence semantics of a public MATLAB field.

MATLAB state	SpearDAT interpretation
Field absent	Unset. The field is not supplied to the corresponding analysis branch.
Field equal to []	Unset, with the same public meaning as an absent field.
Field equal to 0 or 0.0	Explicit numeric zero. It is retained when zero is valid for that field.
Logical field equal to false	Explicit Boolean false. Most calculation and output requests use false as their inactive default.
Empty repeated-object vector	No objects of that type are defined. For example, an empty finSets vector defines no fin sets.

Defaults are field-specific; omission does not have one universal numerical meaning. Important examples include:

- omitted unit selections use "Feet" and "Degrees";
- omitted Boolean calculation and output requests are normally inactive;
- some selectors can be inferred when the populated fields make the choice unambiguous; and
- selected reference quantities may be omitted so that they are derived or defaulted from the active geometry.

The engineering chapters state the omission and default behaviour of each field family. A user should not insert zero merely to make an optional branch look complete: zero is appropriate only when it is the intended value and is valid for that field.

4.8. Validation Before Execution

SpearDAT distinguishes between a MATLAB representation error and an invalid engineering request. Both are detected before the aerodynamic solver is started, but they are reported differently.

4.8.1. MATLAB Representation Errors

A representation error means that the supplied MATLAB value cannot be mapped unambiguously to the public input model. Examples include:

- the top-level input is not a scalar structure;
- a field name is unknown;
- a numeric field uses the wrong MATLAB class;
- a matrix is supplied where a scalar or vector is required;
- a numeric value is supplied where a logical scalar is required; or
- an enumeration value is not one of the exact public names.

These errors raise a MATLAB exception with identifier `SpearDAT:MATLABInput`. No SpearDAT result is returned because a valid public request could not be constructed. The message includes the field path at which conversion failed. For example, a misspelled speed field may be reported against `input.caseInput.flight.speed`.

4.8.2. Engineering and Cross-Field Validation

Once the MATLAB structure has been read successfully, SpearDAT checks whether its engineering choices form a supported and internally consistent analysis. Examples include:

- both BETA and PHI are populated;
- a selected attitude mode conflicts with its populated field;
- a required active-branch value is absent;
- related schedules have incompatible lengths;
- a count or station definition exceeds a supported limit;
- a mutually exclusive pair of engineering inputs is supplied; or
- a value is non-finite or outside the range accepted for that field.

An engineering validation failure is returned as a normal result structure:

```
result = speardat_mex(input);

assert(~result.succeeded);
assert(result.hasErrors);

disp(result.diagnostics);
```

Listing 4.13: Inspecting an analysis rejected before execution.

No scalar case is executed when the request cannot be prepared safely. The aerodynamic table and per-case result vector are therefore empty. Each diagnostic records, where applicable:

severity Information, warning, or error.

source The stage that produced the diagnostic.

code A machine-readable identifier.

path The affected public input path.

message A human-readable explanation.

recommendation A corrective action when one is available.

caseIndex The generated case concerned, when the diagnostic belongs to one case.

Warnings do not by themselves reject a request. Exact duplicate sweep values, for example, are removed with a warning and the retained cases continue through preparation and execution. Errors prevent the affected request from being executed.

Important

A MATLAB exception indicates that the public MATLAB structure itself could not be read. A returned result with `result.succeeded` equal to `false` indicates that the structure was read, but the requested analysis was rejected or did not complete. Inspect `result.diagnostics` in the second case.

Execution-time and output-processing diagnostics are introduced with the complete result model later in this guide.

4.9. Case Identity and Traceability

The optional field `input.caseInput.id` assigns a user-defined engineering identity to the requested analysis:

```
input.caseInput.id = "Roll sweep";
```

Listing 4.14: Assigning a descriptive request identifier.

This value becomes the user-supplied CASEID. It should describe the configuration, purpose, or test condition in language meaningful to the analyst. When a BETA or PHI vector expands the request into several scalar cases, the same CASEID is preserved unchanged in every generated case.

SpearDAT does not derive its generated case names from `input.caseInput.id`. It does not append the sweep index or attitude value to the user-supplied identifier. The two identities have different purposes:

SpearDAT generates a `caseName` for every prepared case, including a request that produces only one case. Names use a deterministic numeric sequence:

```
case_0000
case_0001
case_0002
```

The sequence is assigned after exact duplicate sweep values have been removed. For example:

```
input.caseInput.id = "Roll sweep";

input.caseInput.flight.attitudeMode = "TotalAlphaRoll";
input.caseInput.flight.rollDeg = [-90, 0, 90];
```

Listing 4.15: A user CASEID and a multi-value aerodynamic-roll request.

Table 4.6.: User-supplied and SpearDAT-generated case identities.

Identity	Assigned by	Purpose
<code>input.caseInput.id</code>	User	Descriptive engineering identity. Preserved as CASEID in every scalar case generated from the request.
<code>caseName</code>	SpearDAT	Unique execution identity, generated independently of <code>input.caseInput.id</code> . It is also used for a retained case directory.
<code>caseIndex</code>	SpearDAT	Zero-based numeric identity of the prepared case.
<code>sweepIndex</code>	SpearDAT	Zero-based position of the case in the retained sweep order.
<code>sweepVariable</code> <code>sweepValueDeg</code>	SpearDAT	Identify the attitude field and scalar value represented by the case.

Table 4.7.: Identity of the scalar cases generated by the preceding aerodynamic-roll request.

MATLAB element	CASEID	caseName	sweepValueDeg
<code>result.cases(1)</code>	"Rollsweep"	"case_0000"	-90
<code>result.cases(2)</code>	"Rollsweep"	"case_0001"	0
<code>result.cases(3)</code>	"Rollsweep"	"case_0002"	90

The three scalar cases retain the same user-supplied CASEID, while receiving distinct generated identities:

The MATLAB array position and the SpearDAT case index use different indexing conventions:

```
firstCase = result.cases(1);

assert(firstCase.caseIndex == 0);
assert(firstCase.caseName == "case_0000");
```

Listing 4.16: Relating MATLAB array positions to SpearDAT case indices.

The same generated identity columns are repeated on every row belonging to the case in `result.aerodynamics`. This permits rows to be grouped by case without relying on their physical position in the combined table. When legacy files are retained, the generated `caseName` names the corresponding case directory, while the user-supplied CASEID remains inside the generated analysis input.

The concepts established in this chapter apply to every engineering branch of the MATLAB interface. The following chapters use the same request, case, schedule, sweep, and traceability model while defining the flight conditions, reference quantities, vehicle geometry, controls, and optional analysis features in detail.

5. Flight Conditions and Reference Quantities

Once the vehicle geometry has been defined, an aerodynamic calculation still needs two independent pieces of information: the condition at which the vehicle is operating, and the convention used to non-dimensionalise and reference the resulting forces and moments. SpearDAT keeps these two concerns separate.

The operating condition is supplied beneath `input.caseInput.flight`. It defines the incidence schedule, the speed schedule, and the atmospheric or Reynolds-number description. The coefficient convention is supplied beneath `input.caseInput.reference`. It defines the reference area, reference lengths, moment-reference location, and selected surface-condition inputs.

Historically, these quantities correspond to the Missile DATCOM \$FLTCOM and \$REFQ namelists documented in the published Missile DATCOM User Guide [1]. SpearDAT creates those legacy inputs automatically; users of the MATLAB interface do not need to write either namelist.

This chapter concentrates on how these inputs are used together in an analysis. The complete field-by-field contract, including all omission rules and validation conditions, is given in sections B.7 and B.8.

5.1. Operating Conditions Are Schedules Within a Case

An aerodynamic analysis normally asks for more than one operating point. A single SpearDAT case may therefore contain an angle-of-attack schedule and a speed schedule simultaneously. For example,

```
input.caseInput.flight.alphaDeg = 0:2:24;

input.caseInput.flight.speed.mode = "Mach";
input.caseInput.flight.speed.mach = [0.8, 1.3, 1.75];
```

Listing 5.1: Angle-of-attack and Mach schedules within one case.

requests thirteen angles of attack at three Mach numbers. These are schedules inside the case; they do not create thirty-nine top-level SpearDAT cases. Case expansion is reserved for the supported sideslip and aerodynamic-roll sweeps described in sections 4.4 and 4.6.

This distinction is useful when interpreting the returned table. A scalar zero-sideslip request can produce one case containing all three Mach conditions and all thirteen angles of attack. The Mach and AlphaDeg columns then identify the operating point represented by each aerodynamic row, while CaseIndex continues to identify the independently executed case.

Important

A Mach vector and an angle-of-attack vector are not combined by SpearDAT into separate top-level cases. They remain schedules within each generated case. By contrast, a vector in betaDeg or rollDeg requests case expansion.

5.2. Defining Incidence

The principal incidence schedule is `input.caseInput.flight.alphaDeg`. It is always expressed in degrees and may contain between two and twenty values. The values describe the angle-of-attack schedule evaluated within each case.

The lateral description is selected by `input.caseInput.flight.attitudeMode`. SpearDAT supports the two representations introduced in [section 4.6](#):

"BodyAlphaBeta" The incidence is described by angle of attack together with sideslip β . The optional or swept value is supplied through `betaDeg`.

"TotalAlphaRoll" The incidence is described by the angle-of-attack schedule together with aerodynamic roll ϕ . The optional or swept value is supplied through `rollDeg`.

For a conventional body-axis polar at zero sideslip, an explicit specification is:

```
input.caseInput.flight.attitudeMode = "BodyAlphaBeta";
input.caseInput.flight.alphaDeg = 0:2:24;
input.caseInput.flight.betaDeg = 0.0;
```

Listing 5.2: An explicitly defined zero-sideslip polar.

The explicit zero is useful when zero sideslip is part of the prescribed test condition. It leaves the intent visible in the input and in the case metadata. Omitting `betaDeg` is also valid where the default zero lateral condition is sufficient, but the request then contains less traceability about why that condition was used.

Only one lateral-attitude family may be populated in a request. `betaDeg` and `rollDeg` are alternatives, not two independent schedules to be combined.

5.3. Defining Speed

SpearDAT accepts either Mach number or freestream velocity as the primary speed input. The mode is selected beneath `input.caseInput.flight.speed`.

5.3.1. Mach-number mode

For most Missile DATCOM studies, Mach number is the most direct description:

```
input.caseInput.flight.speed.mode = "Mach";
input.caseInput.flight.speed.mach = [0.8, 1.3, 1.75];
```

Listing 5.3: A three-point Mach schedule.

`mach` is dimensionless and may contain from one to twenty values. For executable aerodynamic analyses, use strictly positive Mach numbers.

5.3.2. Velocity mode

Velocity mode is selected explicitly when the operating condition is naturally known as a freestream speed:

```
input.caseInput.flight.speed.mode = "Velocity";
input.caseInput.flight.speed.velocity = [250, 350, 450];
```

Listing 5.4: Selecting velocity as the primary speed input.

The velocity units follow the *family* selected by `input.settings.lengthSystem`:

Length-system family	Velocity units	Selections
English	ft/s	"Inches" or "Feet"
Metric	m/s	"Centimetres" or "Metres"

This family rule is deliberate. Selecting "Centimetres" does *not* make velocity a centimetres-per-second quantity. The corresponding atmospheric quantities follow the same English versus metric family distinction.

Mach and velocity are alternative primary speed schedules in the public SpearDAT interface. Supplying both in one request is rejected. This is a stricter interface rule than the historical deck syntax: the Missile DATCOM User Guide also lists a mixed MACH–VINF–TINF flight-condition combination [1]. SpearDAT keeps one primary speed source authoritative instead of exposing that mixed legacy form.

5.4. Selecting the Atmospheric Description

The speed schedule does not by itself define density, viscosity, or Reynolds number. The atmospheric branch tells SpearDAT how those quantities are to be established. Three modes are available.

Table 5.1.: Atmospheric modes used by the MATLAB interface.

Mode	Supply	Typical use
"StandardAtmosphere"	Altitude, or omit altitude for sea level	Flight conditions described by a standard atmosphere.
"ReynoldsSpecified"	Reynolds number per unit length	Wind-tunnel or comparison data for which Reynolds number is prescribed directly.
"WindTunnel"	Freestream temperature and pressure	Conditions for which the tunnel or facility thermodynamic state is known explicitly.

The three mode names are SpearDAT interface concepts rather than legacy control-card names. The historical User Guide describes admissible combinations of MACH, ALT, REN, VINF, TINF, and PINF; the public MATLAB interface presents a smaller set of explicit, mutually exclusive definitions. Populate only the quantities belonging to the selected mode.

5.4.1. Standard-atmosphere mode

A standard-atmosphere request may be written as:

```
input.caseInput.flight.atmosphere.mode = "StandardAtmosphere";
input.caseInput.flight.atmosphere.altitude = [0, 0, 0];
```

Listing 5.5: Sea-level standard-atmosphere conditions.

Here the repeated altitude values correspond to the three values in the speed schedule. Altitude uses feet for the English family and metres for the metric family. If altitude is omitted entirely, the standard-atmosphere mode uses its sea-level default.

5.4.2. Specified-Reynolds-number mode

When a test condition is specified by Reynolds number rather than altitude, use:

```
input.caseInput.flight.atmosphere.mode = "ReynoldsSpecified";
input.caseInput.flight.atmosphere.reynoldsPerLength = ...
    [4.92e6, 4.92e6, 4.92e6];
```

Listing 5.6: Supplying Reynolds number per unit length.

The units are 1/ft for the English family and 1/m for the metric family. As with velocity, "Centimetres" still belongs to the metric family: the Reynolds quantity is supplied per metre, not per centimetre.

5.4.3. Wind-tunnel mode

For an explicitly specified thermodynamic state:

```
input.caseInput.flight.atmosphere.mode = "WindTunnel";
input.caseInput.flight.atmosphere.temperature = ...
    [288.15, 288.15];
input.caseInput.flight.atmosphere.pressure = ...
    [101325, 101325];
```

Listing 5.7: An illustrative metric wind-tunnel state.

Temperature and pressure are expressed in degrees Rankine and lbf/ft² for the English family, or kelvin and N/m² for the metric family.

5.4.4. Matching atmospheric values to a speed schedule

The public validator permits an atmospheric vector either to contain one value or to match the speed-schedule length. These two forms are not equivalent for a multi-speed execution. The current writer emits the values that were supplied; it does not broadcast a one-element atmospheric vector into the remaining speed positions. When several speed points are intended to share one atmospheric condition, repeat the value explicitly:

```
input.caseInput.flight.speed.mach = [0.8, 1.3, 1.75];

input.caseInput.flight.atmosphere.reynoldsPerLength = ...
    [4.92e6, 4.92e6, 4.92e6];
```

Listing 5.8: Repeating one atmospheric condition over three speed points.

If, for example, only one Reynolds-number value is supplied for this three-Mach schedule, only the first legacy REN position is populated. At later speed positions with no REN, ALT, or PINF, the preserved Missile DATCOM missing-atmosphere path assumes sea-level altitude and resolves the remaining freestream state from that assumption. A scalar reynoldsPerLength is therefore not a shorthand for repeating the same Reynolds number at every Mach point. The complete input contract is given in [section B.7](#).

5.5. Reference Quantities Define the Coefficient Convention

Aerodynamic coefficients are meaningful only together with the dimensions and moment reference used to form them. SpearDAT therefore treats reference quantities as part of the analysis definition rather than as plotting metadata.

For the principal static coefficients,

$$C_N = \frac{N}{qS_{\text{ref}}}, \quad C_A = \frac{A}{qS_{\text{ref}}}, \quad C_Y = \frac{Y}{qS_{\text{ref}}},$$

while the corresponding moment coefficients use reference lengths,

$$C_m = \frac{M_y}{qS_{\text{ref}}L_{\text{ref}}},$$

$$C_n = \frac{M_z}{qS_{\text{ref}}L_{\text{lat,ref}}}, \quad C_l = \frac{M_x}{qS_{\text{ref}}L_{\text{lat,ref}}}.$$

The MATLAB fields controlling this convention are summarised in [table 5.2](#).

The public names `xcg` and `zcg` follow the Missile DATCOM interface terminology. In comparison work, however, the quantity being matched may be an aerodynamic *moment-reference centre* rather than a measured mass centre of gravity. The source experiment should determine which physical point the values represent.

Important

Two coefficient datasets are not directly comparable merely because their symbols have the same names. Their reference area, moment-reference point, and reference lengths must also use the same convention.

Table 5.2.: Principal reference quantities.

MATLAB field	Physical role	Meaning
reference.sref	Reference area	Area used to non-dimensionalise force coefficients and, together with a reference length, moment coefficients.
reference.lref	Longitudinal reference length	Length used for pitching-moment non-dimensionalisation.
reference.latref	Lateral reference length	Length used for yawing- and rolling-moment non-dimensionalisation.
reference.xcg	Axial moment-reference coordinate	Axial coordinate of the point about which the relevant moments are reported; positive aft from the global origin.
reference.zcg	Vertical moment-reference coordinate	Vertical coordinate of the moment-reference point; positive upward.

5.5.1. Explicit references versus geometry-derived references

The principal reference values may be omitted when geometry-derived defaults are appropriate. For a configuration containing a body, SpearDAT can derive the reference area from the maximum body cross-sectional area and the longitudinal reference length from the maximum body diameter. The lateral reference length defaults to the resolved longitudinal reference length.

For a fin-only configuration, the applicable defaults are derived from the fin geometry instead. The complete derivation rules are listed in [section B.8](#).

Geometry-derived references are convenient during exploratory work. They are less suitable when reproducing published or contractual aerodynamic data, because the published coefficient convention is itself part of the dataset. For comparison work, specify the reported values explicitly.

5.6. Boundary-Layer and Surface Inputs

The reference branch also contains inputs that affect the skin-friction and surface-condition treatment.

reference.boundaryLayer accepts "Turbulent" or "Natural". If the field is omitted, the historical default is turbulent. Supplying the value explicitly is useful when the boundary-layer assumption forms part of the analysis definition.

Two alternative roughness descriptions are available: roughness and roughnessHeightRating. They are mutually exclusive. The roughness field retains a historical unit convention: it is supplied in inches for the English family and centimetres for the metric family, regardless of whether the selected dimensional system is respectively "Feet" or "Metres".

The advanced reference.scale input geometrically scales the vehicle. Because scaling interacts with dimensional geometry and reference quantities, use it only with the rules in [section B.8](#); do not pre-scale a geometry and then apply the same scale factor again.

5.7. Worked Example: NASA Flight and Reference Conditions

A useful reference case for the remainder of this guide is the canard-controlled missile reported by Kassner and Wettlaufer in NASA TM X-73,218 [3]. The model was tested at Mach numbers 0.8, 1.3, and 1.75 over angles of attack from 0° to 24° , at a Reynolds number of 6.25×10^5 based on the 12.70 cm cylindrical body diameter.

The report also gives

$$S_{\text{ref}} = 126.7 \text{ cm}^2, \quad L_{\text{ref}} = 12.70 \text{ cm},$$

and describes L_{ref} as the reference length for all coefficients. SpearDAT separates the longitudinal and lateral reference lengths, so the appropriate mapping for this dataset is

$$L_{\text{ref}} = L_{\text{lat,ref}} = 12.70 \text{ cm}.$$

The report states that all body-axis coefficients were reduced about model station 66 cm. This is an aerodynamic moment-reference location. It is not presented by the report as a measured mass-centre location. The SpearDAT field is nevertheless named `xcg`, following the historical input interface, so matching the NASA moment convention requires

$$x_{\text{cg}} = 66.0 \text{ cm}$$

A separate vertical reference coordinate is not tabulated. The coefficients are reduced about the model-centreline axis system, so `zcg=0.0` is used here as the centreline mapping. This value is an inference from the reported axis convention, not a separately quoted NASA dimension.

The tunnel Reynolds number can be converted to the quantity expected by `reynoldsPerLength`. The selected length system is "Centimetres", which belongs to SpearDAT's metric family; the required unit is therefore 1/m:

$$\frac{Re}{L} = \frac{6.25 \times 10^5}{0.127} = 4.9212598 \times 10^6 \text{ m}^{-1}.$$

5.7.1. Complete runnable input

The purpose of this chapter is to exercise the flight-condition and reference branches, not yet to reproduce the complete NASA body geometry. The following listing therefore uses a minimal axisymmetric support body with the NASA model's overall length and cylindrical diameter. Its nose is simply conical. It is sufficient to run the MATLAB path and inspect the flight/reference behaviour, but it must not be interpreted as the validation geometry used for comparison with the experimental coefficients. The detailed body definition is introduced in the next chapter.

The 2° angle-of-attack spacing is likewise an instructional SpearDAT schedule spanning the reported test range; it is not a claim that every experimental point in the NASA figures occurs at that spacing.

```

input = struct;

%% Units
input.settings.lengthSystem = "Centimetres";
input.settings.derivativeSystem = "Degrees";

input.caseInput.id = ...
    "Chapter 5 NASA flight/reference example";
input.caseInput.output.plot = true;
input.caseInput.theory.bodyMethod = "VanDykeHybrid";

%% Flight conditions
input.caseInput.flight.attitudeMode = "BodyAlphaBeta";
input.caseInput.flight.alphaDeg = 0:2:24;
input.caseInput.flight.betaDeg = 0.0;

input.caseInput.flight.speed.mode = "Mach";
input.caseInput.flight.speed.mach = [0.8, 1.3, 1.75];

input.caseInput.flight.atmosphere.mode = ...
    "ReynoldsSpecified";

rePerMetre = 6.25e5 / 0.127;
input.caseInput.flight.atmosphere.reynoldsPerLength = ...
    [rePerMetre, rePerMetre, rePerMetre];

%% Reference quantities
input.caseInput.reference.sref = 126.7;
input.caseInput.reference.lref = 12.70;
input.caseInput.reference.latref = 12.70;
input.caseInput.reference.xcg = 66.0;
input.caseInput.reference.zcg = 0.0;
input.caseInput.reference.boundaryLayer = "Turbulent";

%% Minimal support body -- not the NASA validation geometry
input.caseInput.body.shape = "Axisymmetric";
input.caseInput.body.axisymmetric.geometryMode = "Stations";
input.caseInput.body.axisymmetric.xOrigin = 0.0;

input.caseInput.body.axisymmetric.stations.x = ...
    [0.0, 38.10, 132.08];
input.caseInput.body.axisymmetric.stations.radius = ...
    [0.0, 6.35, 6.35];

input.caseInput.body.axisymmetric.stations. ...
    discontinuityStations = 2;

% Explicit zero requests the closed-base body-drag treatment.
input.caseInput.body.axisymmetric.exitDiameter = 0.0;

%% Execute
result = speardat_mex(input);

if ~result.succeeded
    disp(result.diagnostics)
    error("SpearDAT analysis did not complete successfully.")
end

```

```
aero = result.aerodynamics;
```

Listing 5.9: NASA flight and reference conditions applied to a minimal axisymmetric support body.

The example deliberately states the reference quantities even though several could be geometry-derived. Doing so reproduces the published coefficient convention explicitly rather than relying on the support body's defaults.

The Reynolds value is repeated three times because the same tunnel Reynolds condition is paired with each of the three Mach points.

5.8. Inspecting the Resolved Flight and Reference State

The returned result can be checked at two levels. First, the top-level status confirms that the request completed:

```
result.succeeded
result.caseCount
result.successfulCaseCount
```

Listing 5.10: Checking the completed request.

The example has one explicit scalar sideslip value, $\beta = 0^\circ$, so it should remain a single SpearDAT case despite the three Mach values and the angle-of-attack schedule.

Second, the resolved state records the flight and reference quantities actually present in the completed case:

```
caseResult = result.cases(1);

caseResult.resolvedState.flight
caseResult.resolvedState.reference
caseResult.resolvedState.configuration.bodyMethod
```

Listing 5.11: Inspecting the resolved flight and reference quantities.

The resolved state is a snapshot of the completed solver state, not merely an echo of the MATLAB fields that were supplied. Values introduced by the legacy case set-up or atmospheric calculations may therefore appear in `resolvedState.flight.points` even when the corresponding field was not populated explicitly in MATLAB. This is particularly useful when a reference quantity has been omitted and derived from geometry, or when a standard-atmosphere condition has been resolved internally.

For the present example, the reference structure should show the explicitly specified area, both reference lengths, the 66 cm axial moment-reference coordinate, the zero centreline coordinate, and the turbulent boundary-layer selection.

Note

For this station-defined support body, `resolvedState.configuration.bodyMethod` reports 'Sose' even though the request specifies "VanDykeHybrid". This is expected. Selecting "VanDykeHybrid" emits no explicit method-selection card and therefore re-

requests the normal legacy default path. The coordinate-defined axisymmetric-body path used by "Stations" subsequently activates the preserved second-order shock-expansion (SOSE) path internally. `resolvedState` records that completed solver state, so the reported method can differ from the method requested at the MATLAB interface.

5.8.1. Plotting the Mach schedule

The combined aerodynamic table contains the Mach number on each relevant row, so the three polars can be selected without treating the Mach values as separate cases.

```
aero = result.aerodynamics;

noTrim = strcmp(aero.ZoneType, "NoTrim");
machValues = unique(aero.Mach(noTrim), "stable");

figure
hold on

for k = 1:numel(machValues)
    machValue = machValues(k);

    rows = noTrim & ...
        abs(aero.Mach - machValue) < 1.0e-8;

    alpha = aero.AlphaDeg(rows);
    cn = aero.CN(rows);

    [alpha, order] = sort(alpha);
    cn = cn(order);

    plot(alpha, cn, "o-", ...
        "DisplayName", "M = " + string(machValue));
end

hold off
grid on
xlabel("Angle of attack, \alpha (deg)")
ylabel("Normal-force coefficient, C_N")
title("Body-alone normal-force schedule")
legend("Location", "best")
```

Listing 5.12: Comparing normal-force coefficient over the requested Mach schedule.

Pitching moment can be inspected in the same way:

```
figure
hold on

for k = 1:numel(machValues)
    machValue = machValues(k);

    rows = noTrim & ...
        abs(aero.Mach - machValue) < 1.0e-8;

    alpha = aero.AlphaDeg(rows);
```



```

cm = aero.CM(rows);

[alpha, order] = sort(alpha);
cm = cm(order);

plot(alpha, cm, "o-", ...
      "DisplayName", "M = " + string(machValue));
end

hold off
grid on
xlabel("Angle of attack, \alpha (deg)")
ylabel("Pitching-moment coefficient, C_M")
title("Body-alone pitching-moment schedule")
legend("Location", "best")

```

Listing 5.13: Comparing pitching-moment coefficient over the requested Mach schedule.

These curves are a software-execution example, not yet an experimental validation result. The support-body mold line is intentionally simplified, so agreement or disagreement with the NASA aerodynamic data is not meaningful at this stage. As a qualitative sanity check only, the NASA report notes that the body-alone normal-force coefficient increases as Mach number rises from the subsonic to the supersonic test conditions [3]; the same ordering is visible in the example output.

5.9. Useful Variations of the Same Analysis

The complete example can be modified without changing the underlying vehicle definition. The following variations illustrate how the operating-condition branches change.

5.9.1. Use a standard atmosphere instead of specified Reynolds number

Replace the atmospheric structure so that no stale `reynoldsPerLength` field remains:

```

inputStandard = input;

inputStandard.caseInput.flight.atmosphere = struct;
inputStandard.caseInput.flight.atmosphere.mode = ...
    "StandardAtmosphere";
inputStandard.caseInput.flight.atmosphere.altitude = ...
    [0.0, 0.0, 0.0];

resultStandard = speardat_mex(inputStandard);

```

Listing 5.14: Changing the example to a standard atmosphere.

The three altitude values correspond to the three retained Mach points. Because the selected length system is metric-family "Centimetres", these altitudes are expressed in metres.

5.9.2. Use freestream velocity as the primary speed input

Velocity mode is most naturally paired with an atmosphere from which the thermodynamic state can be resolved. The following fragment is illustrative and does not represent the NASA test condition:

```
inputVelocity = input;

inputVelocity.caseInput.flight.speed = struct;
inputVelocity.caseInput.flight.speed.mode = "Velocity";
inputVelocity.caseInput.flight.speed.velocity = ...
    [250.0, 350.0, 450.0];

inputVelocity.caseInput.flight.atmosphere = struct;
inputVelocity.caseInput.flight.atmosphere.mode = ...
    "StandardAtmosphere";
inputVelocity.caseInput.flight.atmosphere.altitude = ...
    [0.0, 0.0, 0.0];

resultVelocity = speardat_mex(inputVelocity);
```

Listing 5.15: Illustrative velocity-mode request.

The velocity values are in metres per second because the request uses the metric-family "Centimetres" length system. Replacing the complete speed and atmosphere structures avoids accidentally leaving mutually exclusive fields from the previous mode.

5.9.3. Observe geometry-derived reference values

The support body can also be used to demonstrate reference derivation. Preserve only the explicit boundary-layer choice and let the principal dimensional references resolve from geometry:

```
inputDerived = input;

inputDerived.caseInput.reference = struct;
inputDerived.caseInput.reference.boundaryLayer = ...
    "Turbulent";

resultDerived = speardat_mex(inputDerived);

resolvedReference = ...
    resultDerived.cases(1).resolvedState.reference
```

Listing 5.16: Allowing the principal reference quantities to be derived from the support body.

For this support body, the maximum diameter is 12.70 cm, so the geometry-derived area is $\pi(6.35 \text{ cm})^2 = 126.6769 \text{ cm}^2$, while the resolved longitudinal and lateral reference lengths are both 12.70 cm. The legacy reference defaults also give zero axial and vertical reference coordinates, zero roughness, and a unit scale factor when those fields are omitted. These values are consistent with the Missile DATCOM reference-quantity defaults; the small difference between 126.6769 cm^2 and the NASA value 126.7 cm^2 is simply the rounding used in the published report.

For comparison with the NASA dataset, the explicit reference block in [listing 5.9](#) remains preferable, because it records the published coefficient convention directly.

5.10. Common Flight and Reference Mistakes

Most errors in this part of the input come from mixing otherwise valid descriptions rather than from supplying an intrinsically invalid number.

Mach and velocity supplied together. Choose one primary speed mode. Clear or replace the old speed structure when changing modes.

Atmospheric modes mixed together. Altitude belongs to the standard-atmosphere mode, specified Reynolds number belongs to the Reynolds-specified mode, and temperature plus pressure belong to the wind-tunnel mode.

Metric-family quantities interpreted literally as centimetres. With "Centimetres", flight velocity is still m/s, altitude is still m, and Reynolds number per unit length is still 1/m. The geometry itself remains in centimetres.

One atmospheric value assumed to apply automatically to several speeds. The validator accepts a one-element atmospheric vector, but the current writer does not broadcast it across a multi-speed schedule. Repeat the value explicitly for every intended speed point. Otherwise later legacy positions can follow the ordinary missing-atmosphere fallback instead of using the condition intended for the first speed point.

Reference quantities omitted during data comparison. Geometry-derived defaults may be perfectly valid for analysis, but a published dataset can use a different convention. Match its S_{ref} , reference lengths, and moment-reference point before comparing coefficients.

An aerodynamic moment-reference point treated as a measured mass CG. The public fields are named `xcg` and `zcg` for historical compatibility. In validation work, document what the source data actually used as its reference point.

Roughness and roughness-height rating supplied together. These are alternative surface descriptions. Select at most one.

Geometry scaled twice. When `reference.scale` is used, do not also pre-scale the same input dimensions. The detailed scaling rules are in [section B.8](#).

5.11. MATLAB-to-Legacy Field Map

The MATLAB interface is the public workflow used throughout this guide. The legacy names in [table 5.3](#) are provided only as landmarks for readers comparing SpearDAT with the published Missile DATCOM User Guide [1] or with retained `for005.dat` input.

The exact field types, limits, defaults, units, and activation conditions are collected in [sections B.7](#) and [B.8](#). The next engineering chapter adds the body mold line to these same flight and reference definitions.

Table 5.3.: Flight-condition and reference-quantity field mapping.

MATLAB field	Legacy field	Purpose
flight.alphaDeg	ALPHA	Angle-of-attack schedule.
flight.betaDeg	BETA	Sideslip value in "BodyAlphaBeta" mode.
flight.rollDeg	PHI	Aerodynamic-roll value in "TotalAlphaRoll" mode.
flight.speed.mach	MACH	Mach-number schedule.
flight.speed.velocity	VINF	Freestream-velocity schedule.
flight.atmosphere.altitude	ALT	Standard-atmosphere altitude.
flight.atmosphere.reynoldsPerLength	REN	Reynolds number per unit length.
flight.atmosphere.temperature	TINF	Freestream temperature.
flight.atmosphere.pressure	PINF	Freestream pressure.
reference.sref	SREF	Reference area.
reference.lref	LREF	Longitudinal reference length.
reference.latref	LATREF	Lateral reference length.
reference.xcg	XCG	Axial moment-reference coordinate.
reference.zcg	ZCG	Vertical moment-reference coordinate.
reference.boundaryLayer	BLAYER	Boundary-layer selection.
reference.roughness	ROUGH	Surface roughness.
reference.roughnessHeightRating	RHR	Alternative roughness-height rating.
reference.scale	SCALE	Geometric scale factor.

6. Body Geometry

The body definition supplies the physical mold line on which the body aerodynamic methods operate. It establishes the longitudinal shape, local cross-section dimensions, nose and afterbody classifications, base geometry, and the geometric quantities from which several default reference values are derived.

SpearDAT exposes two body families through `input.caseInput.body`: an axisymmetric body with circular cross-sections, and an elliptical body with non-circular elliptical cross-sections. Each family can be defined either from standard analytic sections or from user-supplied longitudinal stations.

Historically, these capabilities correspond to the Missile DATCOM \$AXIBOD and \$ELLBOD namelists documented in the published Missile DATCOM User Guide [1]. SpearDAT creates the appropriate legacy input automatically; normal MATLAB users do not need to write either namelist.

This chapter concentrates on choosing a body representation, understanding the geometry that each representation creates, and checking the resolved body after execution. The complete field-by-field contract is given in [section B.9](#). Base-jet plume interaction and body-mounted protuberances share the body branch but are advanced add-on capabilities; their detailed inputs are kept in [sections B.9.5](#) and [B.9.6](#).

Important

Body dimensions use `input.settings.lengthSystem` literally. With "Centimetres", for example, body lengths, radii, widths, and `xOrigin` are entered in centimetres. This differs from flight-condition quantities such as velocity, altitude, and Reynolds number per unit length, which use the English-or-metric family rules described in [chapter 5](#).

6.1. Choosing a Body Representation

Two independent choices define the body representation.

First, `input.caseInput.body.shape` selects the cross-section family:

"Axisymmetric" Circular cross-sections normal to the body axis.

"Elliptical" Elliptical cross-sections whose height-to-width ratio may vary along the body.

Second, the active family's `geometryMode` selects how the longitudinal mold line is supplied:

"Analytic" Build the body from named nose, centrebody, and afterbody sections.

"Stations" Supply the physical mold line directly as longitudinal station arrays.

The four combinations have distinct engineering uses.

An explicit shape and `geometryMode` are recommended even where SpearDAT could infer them. They make the geometry unambiguous to a reader and prevent an old field from another representation from silently changing the intended interpretation.

Table 6.1.: Body representations available through the MATLAB interface.

Body family	Mode	Typical use
Axisymmetric	Analytic	Standard cones, tangent ogives, power-law or Haack-family noses, cylindrical or straight-sided centre sections, and conical or ogival afterbodies.
Axisymmetric	Stations	Measured or CAD-derived radius schedules, multi-segment bodies, exact secant-ogive coordinates, and other circular-section shapes that are not conveniently represented by the analytic primitives.
Elliptical	Analytic	Bodies whose horizontal and vertical dimensions can be represented by standard nose, centre, and aft sections with prescribed ellipticity.
Elliptical	Stations	Bodies whose half-width and half-height vary according to a supplied longitudinal schedule.

Important

Populate only one body family. Do not leave active axisymmetric and elliptical geometry in the same request.

Within the active family, do not mix an analytic mold-line definition with station arrays. The exception is the documented use of selected nose/centrebody/aft fields as *section metadata* for a station-defined body; in that case the station arrays remain the physical geometry.

The longitudinal coordinate `xOrigin` gives the global coordinate of the physical nose tip. It is often zero, but it does not have to be. A non-zero value is useful when source data use a fixed model-station coordinate system whose zero lies ahead of a blunted physical nose.

6.2. Axisymmetric Bodies

An axisymmetric body has a single radius at each longitudinal location. The same mold line is therefore seen in every plane through the body axis. This is the natural representation for conventional bodies of revolution.

6.2.1. Analytic axisymmetric geometry

Analytic mode builds the body sequentially from a nose, an optional centre section, and an optional afterbody:

```
input.caseInput.body.shape = "Axisymmetric";
input.caseInput.body.axisymmetric.geometryMode = "Analytic";
input.caseInput.body.axisymmetric.xOrigin = 0.0;

input.caseInput.body.axisymmetric.nose.type = "Ogive";
input.caseInput.body.axisymmetric.nose.length = 30.0;
input.caseInput.body.axisymmetric.nose.diameter = 10.0;

input.caseInput.body.axisymmetric.center.length = 50.0;
input.caseInput.body.axisymmetric.center.diameter = 10.0;
```

```
% Conical boattail from diameter 10 to diameter 6.
input.caseInput.body.axisymmetric.aft.type = "Conical";
input.caseInput.body.axisymmetric.aft.length = 12.0;
input.caseInput.body.axisymmetric.aft.diameter = 6.0;
```

Listing 6.1: Skeleton of an analytic axisymmetric body.

The nose diameter is its diameter at the downstream end. The centre section begins from that diameter and ends at center.diameter. Equal diameters therefore produce a cylinder; different values produce a straight-sided diameter transition over the specified centre length.

An afterbody begins from the end of the centre section, or directly from the nose when no centre section is present. Its downstream size is supplied by aft.diameter. A smaller terminal diameter produces a boattail; a larger one produces a flare.

The supported analytic afterbody selectors are summarised in [table 6.2](#).

Table 6.2.: Analytic afterbody types.

MATLAB value	Geometry
"Conical"	Straight-sided conical section or cone frustum.
"Ogive"	Tangent-ogive afterbody.

The supported analytic nose selectors are summarised in [table 6.3](#).

Table 6.3.: Analytic nose types.

MATLAB value	Geometry
"Conical"	Straight-sided conical nose.
"Ogive"	Tangent-ogive nose.
"Power"	Power-law nose; also supply a positive powerExponent.
"Haack"	Haack-series volume-constrained nose.
"Karman"	Von Karman length-diameter-constrained nose.

The "Ogive" selector is specifically a *tangent* ogive. A secant ogive is not a separate analytic selector in the public MATLAB model. Where the exact secant-ogive shape matters, define its mold line using "Stations" rather than relabelling it as a tangent ogive. The legacy User Guide gives the same recommendation for a secant-ogive body definition [1].

Blunted and truncated noses

The field nose.bluntness is a radius. Its interpretation depends on nose.truncated:

- with truncated=false, it defines spherical nose bluntness joined to the selected parent nose profile;
- with truncated=true, it is interpreted as the truncation radius; and
- a value of zero represents the sharp form of the selected analytic nose.

This distinction matters when reconstructing a published test model. A blunted tangent ogive is not created by shortening a cone or by assigning a generic flat nose. The physical nose length, parent ogive geometry, and spherical bluntness radius must describe the same profile.

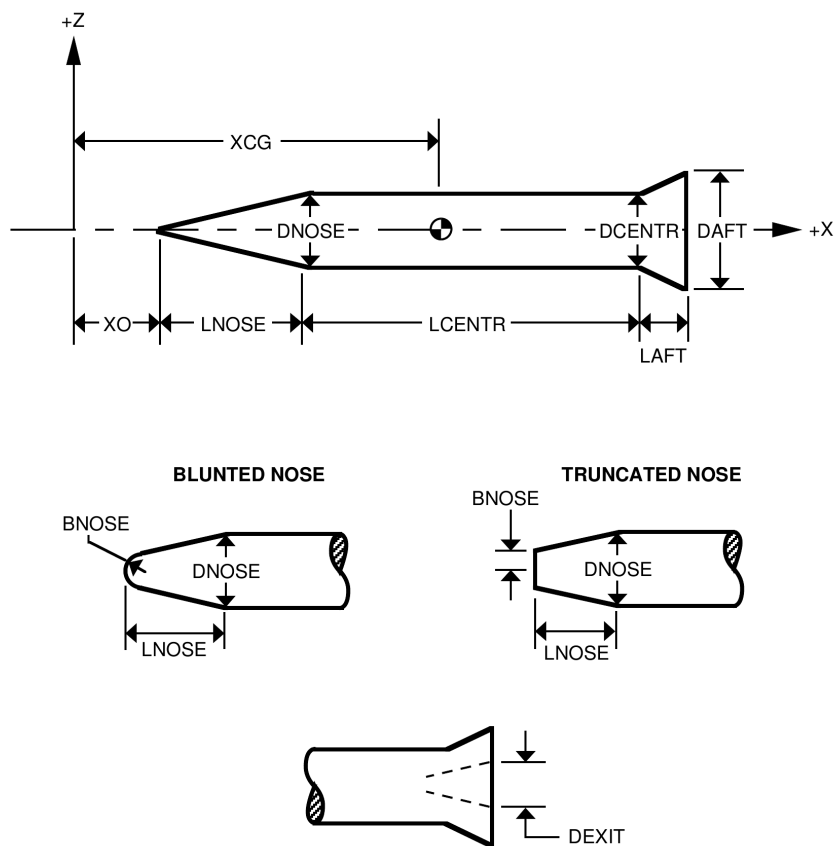


Figure 6.1.: Body-geometry input nomenclature. Reproduced from Figure 1 of the Missile DATCOM User Guide [1].

6.2.2. Station-defined axisymmetric geometry

Station mode supplies the mold line directly through paired longitudinal coordinates and radii:

```
input.caseInput.body.shape = "Axisymmetric";
input.caseInput.body.axisymmetric.geometryMode = "Stations";
input.caseInput.body.axisymmetric.xOrigin = 0.0;

input.caseInput.body.axisymmetric.stations.x = ...
    [0, 8, 24, 60];
input.caseInput.body.axisymmetric.stations.radius = ...
    [0, 4, 4, 2];

input.caseInput.body.axisymmetric.stations. ...
    discontinuityStations = [2, 3];
```

Listing 6.2: A station-defined axisymmetric body.

The two station vectors must have the same length. The coordinates are given in physical front-to-back order, and the radius must be non-negative. SpearDAT preserves the order supplied by the user; it does not sort a body schedule automatically.

The station count is a geometric description, not a CFD mesh-resolution setting. Add stations where they are needed to represent curvature, a change of slope, or a measured profile accurately. Simply adding large numbers of nearly redundant stations does not turn the underlying Missile DATCOM correlations into a higher-fidelity flow solver.

The MATLAB interface accepts between 3 and 49 user-supplied body stations in either station representation, together with at most 20 discontinuityStations indices. This user-input limit is different from the internally processed high-speed contour described in the legacy User Guide. Above Mach 1.2, Missile DATCOM may insert many additional points between the supplied coordinates, and the resulting internal contour can contain more than 1000 points [1]. Those are solver-generated points; they do not allow the MATLAB user to supply a 1000-point station vector.

Section metadata with station geometry

A station-defined body can also carry selected nose, center, and aft metadata. This is useful when the mold line is known from coordinates but downstream methods still need the engineering sections of the body to be identified.

For example,

```
input.caseInput.body.axisymmetric.nose.type = "Ogive";
input.caseInput.body.axisymmetric.nose.length = 18;

input.caseInput.body.axisymmetric.center.length = 36;

input.caseInput.body.axisymmetric.aft.type = "Conical";
input.caseInput.body.axisymmetric.aft.length = 22;
```

Listing 6.3: Adding section metadata without replacing the station-defined mold line.

identifies a nose boundary 18 selected length units aft of `x0origin`, followed by a 36-unit centre section. If either boundary falls between supplied stations, SpearDAT inserts an interpolated mold-line station at the required location. The supplied station profile remains authoritative: the "Ogive" and "Conical" values classify the interpreted sections; they do not regenerate those portions of the body as analytic curves.

This distinction is particularly useful for a secant ogive. If the station coordinates describe an exact secant-ogive mold line, those coordinates create the geometry. Supplying `nose.type = "Ogive"` in station mode does not convert it into the analytic tangent-ogive shape; it classifies the already-defined nose for downstream aerodynamic methods. If a positive station-defined nose is present and the type is omitted, SpearDAT resolves the usual ogive-style nose classification without altering the station mold line.

This mixed station-plus-metadata behavior is one area in which SpearDAT deliberately completes fragile historical handling. The compatibility details for axisymmetric and elliptical bodies are recorded in [entries KD-011](#) and [KD-012](#); the engineering interpretation is the one used throughout the MATLAB interface.

6.3. Elliptical Bodies

An elliptical body replaces the single circular radius with two half-dimensions at each longitudinal station. Its cross-section may therefore be wider than it is high, higher than it is wide, or change aspect ratio along the body.

The dimensionless ellipticity used by this interface is

$$e = \frac{\text{half-height}}{\text{half-width}} = \frac{\text{full height}}{\text{full width}}.$$

Thus $e = 1$ is circular, $e < 1$ is wider than high, and $e > 1$ is higher than wide.

6.3.1. Station-defined elliptical geometry

In station mode, the accepted dimensional field names are `halfWidth` and `height`. Despite its shorter name, `height` is the *half-height*; there is no public `halfHeight` field.

A cross-section can be defined from any two of `half-width`, `half-height`, and ellipticity. The following example supplies both half-dimensions directly:

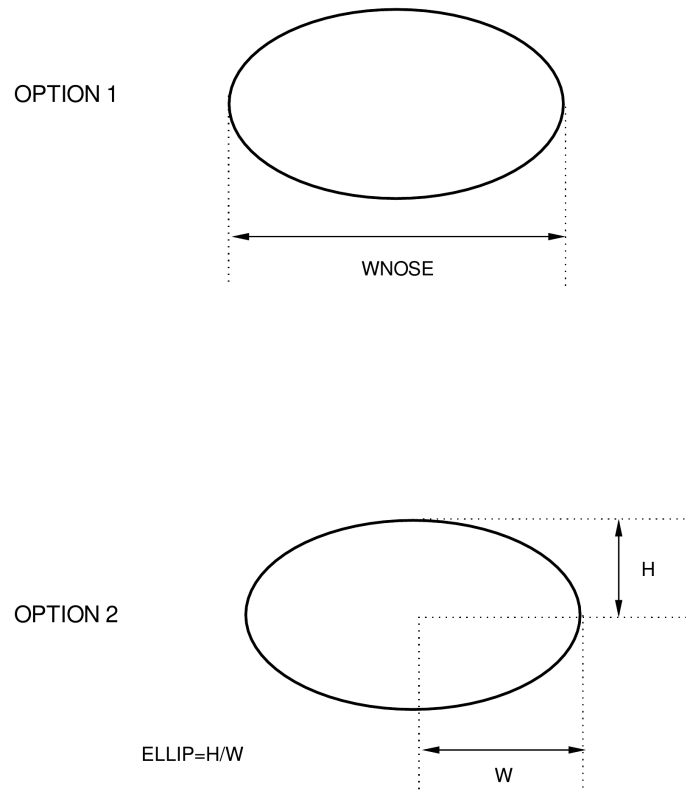
```
input.caseInput.body.shape = "Elliptical";
input.caseInput.body.elliptical.geometryMode = "Stations";
input.caseInput.body.elliptical.x0origin = 0.0;

input.caseInput.body.elliptical.stations.x = ...
    [0, 12, 30, 55];

input.caseInput.body.elliptical.stations.halfWidth = ...
    [0, 4, 4, 2];

input.caseInput.body.elliptical.stations.height = ...
    [0, 3, 5, 2];

input.caseInput.body.elliptical.stations. ...
```



NOTE: Option 1 input W_{NOSE} is TOTAL Width, Option 2 input W is HALF width

Figure 6.2.: Elliptical-body dimensional conventions for analytic and station geometry. Reproduced from Figure 3 of the Missile DATCOM User Guide [1].

```
discontinuityStations = [2, 3];
```

Listing 6.4: A station-defined elliptical body.

At the second station, for example, `halfWidth=4` and `height=3` describe a full width of 8 and a full height of 6.

The other accepted complete combinations are `halfWidth` plus ellipticity, or `height` plus ellipticity. Whichever combination is used, each populated vector must correspond one-for-one with the longitudinal station vector. Examples of these two equivalent formulations are shown below.

```
% Using the same x stations as the preceding example.
input.caseInput.body.elliptical.stations.halfWidth = ...
    [0, 4, 4, 2];

input.caseInput.body.elliptical.stations.ellipticity = ...
    [1.00, 0.75, 1.25, 1.00];
```

Listing 6.5: Elliptical stations from half-width and ellipticity.

The corresponding half-heights are then derived as `halfWidth.*ellipticity`, giving `[0, 3, 5, 2]` for this example.

```
% The public field name height contains the half-height.
input.caseInput.body.elliptical.stations.height = ...
    [0, 3, 5, 2];

input.caseInput.body.elliptical.stations.ellipticity = ...
    [1.00, 0.75, 1.25, 1.00];
```

Listing 6.6: Elliptical stations from half-height and ellipticity.

Here the corresponding half-widths are derived as `height./ellipticity`, recovering `[0, 4, 4, 2]`.

6.3.2. Analytic elliptical geometry

Analytic elliptical geometry uses the same longitudinal section idea as the axisymmetric branch, but the dimensional convention is different. The fields

- `nose.totalWidth`,
- `center.width`, and
- `aft.width`

are *full* widths. The corresponding heights are derived from the ellipticity assigned to each section.

```
input.caseInput.body.shape = "Elliptical";
input.caseInput.body.elliptical.geometryMode = "Analytic";
input.caseInput.body.elliptical.xOrigin = 0.0;

input.caseInput.body.elliptical.nose.type = "Ogive";
input.caseInput.body.elliptical.nose.length = 18;
```

```

input.caseInput.body.elliptical.nose.totalWidth = 8;
input.caseInput.body.elliptical.nose.ellipticity = 0.75;

input.caseInput.body.elliptical.center.length = 30;
input.caseInput.body.elliptical.center.width = 8;
input.caseInput.body.elliptical.center.ellipticity = 1.10;

input.caseInput.body.elliptical.aft.type = "Conical";
input.caseInput.body.elliptical.aft.length = 12;
input.caseInput.body.elliptical.aft.width = 4;
input.caseInput.body.elliptical.aft.ellipticity = 0.80;

```

Listing 6.7: An analytic elliptical body.

The distinction between half-dimensions in station mode and full widths in analytic mode is intentional. Copying a full width directly into `stations.halfWidth` would double the intended body size.

As with the axisymmetric branch, station-defined elliptical bodies may carry section metadata while the station arrays remain authoritative. The historical compatibility details of that mixed-input path are described in [entry KD-012](#).

6.4. Discontinuities and Section Boundaries

A *section boundary* and a *slope discontinuity* are not the same thing.

A section boundary identifies where the nose ends, where a centre section ends, or where an afterbody begins. It can be useful even when the mold line is smooth through that location because the aerodynamic method may need to know which portion of the body it is treating.

A slope discontinuity marks a genuine corner in the supplied station geometry. For example, a cone joining a cylinder has a change in surface slope at the shoulder; that station should be identified as discontinuous. By contrast, a proper tangent ogive joins its adjoining cylindrical surface tangentially and does not acquire a slope break merely because the engineering name changes from “nose” to “centrebody”.

In MATLAB, discontinuities are supplied as one-based station indices, not as longitudinal coordinates:

```

input.caseInput.body.axisymmetric.stations.x = ...
    [0, 10, 30, 60];

input.caseInput.body.axisymmetric.stations.radius = ...
    [0, 4, 4, 2];

% The value 2 means x(2) = 10, not x = 2.
input.caseInput.body.axisymmetric.stations. ...
    discontinuityStations = 2;

```

Listing 6.8: A discontinuity index refers to a station number.

Use these indices only for real slope breaks. Marking every station as discontinuous changes the geometric interpretation and is not a substitute for adding enough stations to describe a smooth curve.

Preserving genuine corners can also matter to the high-speed pressure methods. In the supersonic and hypersonic body-pressure paths, the discontinuity information is used when the processed contour is divided into body segments before the shock-expansion geometry and pressure calculations. Incorrect indices can therefore change more than the geometry report. `discontinuityStations` is still geometric metadata rather than a high-speed method selector, but it becomes particularly consequential when those pressure-calculation paths are active.

6.5. The Body Base and Exit Diameter

The external terminal diameter or width belongs to the body geometry. The separate `exitDiameter` field represents a nozzle or exit opening inside that external base for the body base-drag calculation.

Its omission is significant:

Field omitted. No body base-drag increment is requested through this input path.

Explicit zero. The body has a closed base for this calculation, so the full external base area contributes.

Positive value. The specified internal opening is removed from the external base area, producing an annular-base treatment.

Do not use an explicit zero merely as a placeholder for an unknown exit diameter. Omitted and zero are different modeling decisions. The exact field behavior is collected in [section B.9.4](#).

For example, if the analytic axisymmetric body introduced earlier has an internal exit opening of diameter 4 while its external base diameter is 6, the annular-base input is simply:

```
input.caseInput.body.axisymmetric.exitDiameter = 4.0;
```

Listing 6.9: Defining an internal exit diameter.

Base-jet plume interaction adds a separate set of inputs and should not be inferred merely from the presence of a non-zero exit diameter. For an applicable axisymmetric supersonic plume case, the additional branch has the following form:

```
input.caseInput.body.axisymmetric.exitDiameter = 4.0;

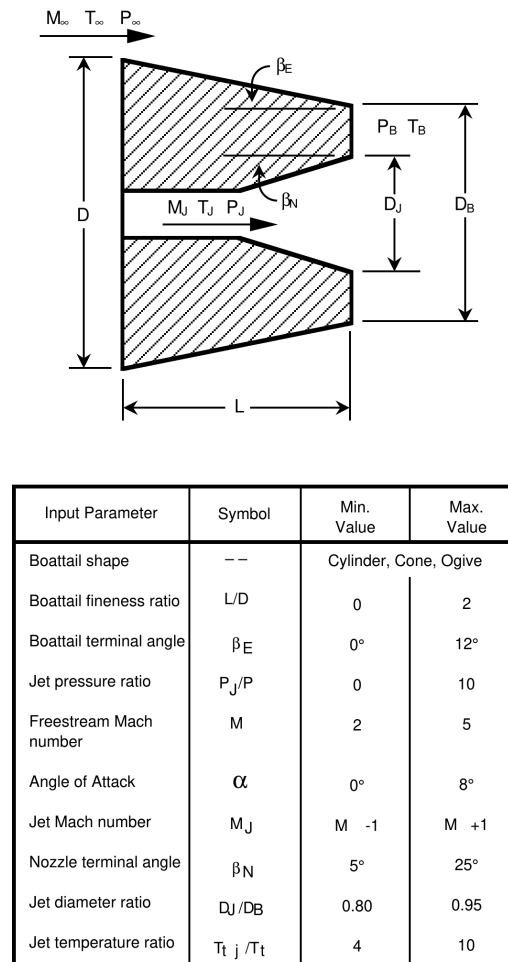
input.caseInput.body.baseJet.enabled = true;
input.caseInput.body.baseJet.nozzleExitHalfAngle = 10.0;
input.caseInput.body.baseJet.jetMach = [2.0, 2.1, 2.2];
input.caseInput.body.baseJet.pressureRatio = [1.20, 1.30, 1.40];
input.caseInput.body.baseJet.temperatureRatio = [1.10, 1.20, 1.30];
```

Listing 6.10: Illustrative base-jet plume inputs.

The plume vectors are schedules associated with the active freestream speed conditions; they are not independent Mach-number grids. For example, if `input.caseInput.flight.speed.mach` is `[2.0, 3.0, 4.0]`, the first values of `jetMach`, `pressureRatio`, and `temperatureRatio` apply to the freestream condition at $M = 2$, the second values to $M = 3$, and the third values to $M = 4$. The same positional rule applies when freestream velocity is the active speed input. In SpearDAT, each plume schedule may contain one value, which represents a constant schedule across the active freestream conditions, or one value per active speed condition. These inputs do not

create additional freestream cases and are not combined with the flight speed schedule as a Cartesian product.

The three-value plume schedules in the listing therefore assume three corresponding active freestream speed conditions in the same order. This is an input-layout example for the specialised plume model; it is not part of the NASA N3 analysis used below. The detailed activation and schedule rules are given in [section B.9.5](#).



Note; If input parameter is not between minimum and maximum value the code will extrapolate

Figure 6.3.: Base-jet plume-interaction nomenclature and documented correlation ranges. Reproduced from Figure 2 of the Missile DATCOM User Guide [1].

6.6. Worked Example: NASA N3 Body Geometry

The same experimental programme used in [chapter 5](#) provides a useful body-geometry example, but the essential source information is repeated here so that this chapter can be used independently.

Kassner and Wettlaufer tested a canard-controlled missile in the Ames 6- by 6-Foot Wind Tunnel and reported the results in NASA TM X-73,218 [3]. The basic model was a body of

revolution with a 12.70 cm cylindrical diameter and an aft model station of 132.08 cm. Three nose configurations were tested: a pointed three-caliber tangent ogive and two spherically blunted versions of the same three-caliber tangent-ogive family. The report identifies the semiblunt nose as configuration N3.

For N3, Figure 3 of the report gives a spherical bluntness radius of 1.42 cm and places the physical nose tip at model station 3.24 cm. The same drawing labels model station 13.49 cm as the *nose cap junction*. This is the junction between the interchangeable N1, N2, and N3 forward nose caps and the common continuation of the forebody; it is not the full-diameter shoulder of the complete aerodynamic nose. Consequently,

$$13.49 - 3.24 = 10.25 \text{ cm}$$

is the axial length of the removable N3 cap, not the length of the complete N3 aerodynamic nose.

The report separately identifies the nose family as three-caliber tangent ogives. The parent sharp ogive therefore has theoretical length

$$L_{\text{ogive}} = 3D = 3(12.70) = 38.10 \text{ cm.}$$

Using the NASA model-station coordinate system, the physical N3 aerodynamic nose length is reconstructed as

$$L_{\text{N3}} = 38.10 - 3.24 = 34.86 \text{ cm,}$$

and the downstream constant-diameter section extends from model station 38.10 cm to 132.08 cm:

$$L_{\text{center}} = 132.08 - 38.10 = 93.98 \text{ cm.}$$

These arithmetic steps are a SpearDAT reconstruction of the dimensions shown in the report, not a separate table of SpearDAT inputs published by NASA. The printed dimensions are rounded, so derived analytic quantities should be expected to agree closely rather than to machine precision.

The important coordinate choice is `xOrigin=3.24`. It places the *physical* semiblunt nose tip at the model station reported for N3 while preserving the fixed NASA model-station system used elsewhere in the dataset, including the 66 cm moment reference.

6.6.1. Complete runnable input

The following listing is self-contained. It repeats the NASA flight and reference conditions from the preceding chapter and replaces the earlier minimal support body with the reconstructed N3 analytic geometry.

The NASA report does not provide, in the body dimensions used here, a separate nozzle-exit diameter suitable for choosing a closed-base or annular-base SpearDAT treatment. The example therefore leaves `exitDiameter` omitted rather than inventing one. The example is intended first to establish the body geometry and its resolved state; experimental aerodynamic validation

requires the complete configuration and the appropriate treatment of all relevant test-model details.

```

input = struct;

%% Units
input.settings.lengthSystem = "Centimetres";
input.settings.derivativeSystem = "Degrees";

input.caseInput.id = "Chapter 6 NASA N3 body example";
input.caseInput.output.plot = true;
input.caseInput.theory.bodyMethod = "VanDykeHybrid";

%% Flight conditions
input.caseInput.flight.attitudeMode = "BodyAlphaBeta";
input.caseInput.flight.alphaDeg = 0:2:24;
input.caseInput.flight.betaDeg = 0.0;

input.caseInput.flight.speed.mode = "Mach";
input.caseInput.flight.speed.mach = [0.8, 1.3, 1.75];

input.caseInput.flight.atmosphere.mode = ...
    "ReynoldsSpecified";

rePerMetre = 6.25e5 / 0.127;
input.caseInput.flight.atmosphere.reynoldsPerLength = ...
    [rePerMetre, rePerMetre, rePerMetre];

%% Reference quantities
input.caseInput.reference.sref = 126.7;
input.caseInput.reference.lref = 12.70;
input.caseInput.reference.latref = 12.70;
input.caseInput.reference.xcg = 66.0;
input.caseInput.reference.zcg = 0.0;
input.caseInput.reference.boundaryLayer = "Turbulent";

%% NASA N3 semiblunt body geometry
input.caseInput.body.shape = "Axisymmetric";
input.caseInput.body.axisymmetric.geometryMode = "Analytic";

% NASA N3 physical nose tip in the report's model-station system.
input.caseInput.body.axisymmetric.xOrigin = 3.24;

% Three-caliber tangent-ogive parent with the N3 spherical bluntness.
input.caseInput.body.axisymmetric.nose.type = "Ogive";
input.caseInput.body.axisymmetric.nose.length = ...
    38.10 - 3.24;
input.caseInput.body.axisymmetric.nose.diameter = 12.70;
input.caseInput.body.axisymmetric.nose.bluntness = 1.42;
input.caseInput.body.axisymmetric.nose.truncated = false;

% Constant-diameter portion from the parent-ogive shoulder to MS 132.08.
input.caseInput.body.axisymmetric.center.length = ...
    132.08 - 38.10;
input.caseInput.body.axisymmetric.center.diameter = 12.70;

% exitDiameter is intentionally omitted in this geometry example.

```

```

%% Execute
result = speardat_mex(input);

if ~result.succeeded
    disp(result.diagnostics)
    error("SpearDAT analysis did not complete successfully.")
end

aero = result.aerodynamics;

```

Listing 6.11: Complete NASA N3 body-geometry example.

The example preserves the published NASA reference values explicitly even though the maximum body cross-section could supply closely related defaults. That keeps the aerodynamic coefficient convention independent of any rounding in the reconstructed geometry.

6.7. Inspecting the Resolved Body Geometry

A completed case contains a structured geometry snapshot beneath `resolvedState.geometry`. The body branch can be inspected directly:

```

caseResult = result.cases(1);

bodyGeometry = ...
    caseResult.resolvedState.geometry.body;

bodyGeometry
bodyGeometry.nose
bodyGeometry.center
bodyGeometry.aft

```

Listing 6.12: Inspecting the resolved body geometry.

The body structure contains calculated section properties as well as total properties. The principal groups are summarised in [table 6.4](#).

Among the most useful first checks for an axisymmetric body are therefore the maximum diameter and area, total physical length, total planform and wetted areas, total volume, and the theoretical lengths reported for the individual sections.

For the N3 reconstruction, a compact set of geometric sanity checks is:

```

bodyGeometry = ...
    result.cases(1).resolvedState.geometry.body;

noseTipStation = ...
    input.caseInput.body.axisymmetric.xOrigin;

aftStation = noseTipStation + bodyGeometry.totalLength;

fprintf("Maximum diameter: %.6f cm\n", ...
    bodyGeometry.maximumDiameter)

fprintf("Resolved physical body length: %.6f cm\n", ...
    bodyGeometry.totalLength)

```

Table 6.4.: Principal calculated quantities in the resolved body geometry.

Resolved field or group	Interpretation
nose, center, aft	Section planform area, wetted area, volume, centroids, fineness information, and theoretical section length where applicable.
totalPlanformArea, totalWettedArea, totalVolume	Calculated whole-body area and volume quantities.
totalVolumeCentroidX, totalPlanformCentroidX	Whole-body axial centroids of volume and planform area.
baseDiameter, baseArea	Resolved external base size and area.
maximumDiameter, maximumCrossSectionArea, totalLength	Maximum body size and resolved physical body length.
Vertical-planform fields	Additional elliptical-body quantities. They are unavailable for an axisymmetric body and therefore appear as [] in MATLAB.
Equivalent-circular-section fields	Equivalent circular quantities used for elliptical-body processing. They are likewise unavailable for an axisymmetric body.

```
fprintf("Reconstructed aft model station: %.6f cm\n", ...
    aftStation)

fprintf("Resolved sharp-parent nose length: %.6f cm\n", ...
    bodyGeometry.nose.theoreticalLength)
```

Listing 6.13: Checking the NASA N3 resolved geometry.

The expected interpretation is:

- the maximum diameter should be close to 12.70 cm;
- the resolved *physical* body length should be close to $132.08 - 3.24 = 128.84$ cm;
- adding the non-zero physical-tip origin should recover an aft model station close to 132.08 cm; and
- the calculated theoretical sharp-parent ogive length should be close to the reported three-caliber value of 38.10 cm.

The distinction between physical length and model-station coordinate is important. For the semiblunt N3 body, totalLength is not expected to equal 132.08 cm because the physical nose begins at model station 3.24 cm rather than at the virtual sharp-tip station 0.

A representative successful run of this example resolves maximumDiameter=12.70 cm, baseDiameter=12.70 cm, baseArea=126.6769 cm², and totalLength=128.84 cm. The nose theoretical length resolves to approximately 38.0702 cm, close to the nominal 38.10 cm three-caliber parent reconstructed from the rounded drawing dimensions. The nose and centrebody planform areas, wetted areas, and volumes also sum to the reported whole-body totals to the displayed precision.

The NASA body used here has no separate afterbody section. It is therefore normal for the resolved aft structure to report zero planform area, wetted area, volume, fineness, and theoretical length. Centroid fields for such a zero-area or zero-volume section have no physical

meaning and should not be interpreted even if the underlying numerical state contains a finite value.

The resolved state is more than an echo of the MATLAB input. Analytic geometry is discretised and processed by the underlying body-geometry routines before these properties are captured. The reported areas, volumes, centroids, and theoretical section quantities therefore provide useful evidence that the requested body was interpreted as intended.

Note

For an axisymmetric body, the vertical-planform and equivalent-circular-section fields in `resolvedState.geometry.body` are intentionally unavailable and MATLAB exposes them as []. For an elliptical body, the same structure populates these additional calculated quantities where applicable. They are outputs of the resolved geometry, not extra inputs that must be supplied by the MATLAB user.

6.8. Useful Variations of the NASA Body

The NASA report's three nose configurations make the same body a convenient way to study how analytic nose definitions change while the downstream cylindrical section remains fixed.

6.8.1. Sharp N1 nose

The pointed N1 configuration restores the virtual sharp tip as the physical nose tip and removes the spherical bluntness:

```
inputN1 = input;

inputN1.caseInput.id = "Chapter 6 NASA N1 body example";

inputN1.caseInput.body.axisymmetric.xOrigin = 0.0;
inputN1.caseInput.body.axisymmetric.nose.length = 38.10;
inputN1.caseInput.body.axisymmetric.nose.bluntness = 0.0;
inputN1.caseInput.body.axisymmetric.nose.truncated = false;

resultN1 = speardat_mex(inputN1);
```

Listing 6.14: Changing the worked example to the sharp N1 nose.

The centre-section length remains 93.98 cm, so the aft model station stays at 132.08 cm.

6.8.2. More strongly blunted N2 nose

The report's N2 nose places the physical tip at model station 6.23 cm and gives a spherical radius of 2.54 cm. Retaining the same three-caliber parent ogive gives:

```
inputN2 = input;

inputN2.caseInput.id = "Chapter 6 NASA N2 body example";

inputN2.caseInput.body.axisymmetric.xOrigin = 6.23;
inputN2.caseInput.body.axisymmetric.nose.length = ...
```

```

38.10 - 6.23;
inputN2.caseInput.body.axisymmetric.nose.bluntness = 2.54;
inputN2.caseInput.body.axisymmetric.nose.truncated = false;

resultN2 = speardat_mex(inputN2);

```

Listing 6.15: Changing the worked example to the NASA N2 nose.

Again, the parent ogive shoulder and the downstream cylindrical body remain in the same model-station locations; only the forward spherical truncation of the physical nose changes.

6.8.3. Use station geometry for a custom mold line

Where a source provides measured coordinates rather than a standard analytic profile, replace the analytic branch as a unit rather than leaving both forms populated:

```

inputStations = input;

inputStations.caseInput.body.axisymmetric = struct;
inputStations.caseInput.body.axisymmetric.geometryMode = ...
    "Stations";
inputStations.caseInput.body.axisymmetric.xOrigin = 0.0;

inputStations.caseInput.body.axisymmetric.stations.x = ...
    [0, 12, 30, 60, 76];
inputStations.caseInput.body.axisymmetric.stations.radius = ...
    [0, 4, 4, 3, 2];

inputStations.caseInput.body.axisymmetric.stations. ...
    discontinuityStations = [2, 3];

resultStations = speardat_mex(inputStations);

```

Listing 6.16: Replacing analytic geometry with a station mold line.

This last fragment is an illustrative custom body, not another reconstruction of the NASA test article. Replacing the complete axisymmetric structure prevents analytic fields from the previous example from remaining active accidentally.

6.9. Common Body-Geometry Mistakes

Most body-definition errors come from a geometry that is individually reasonable but inconsistent with the selected representation.

Both body families are populated. This is not an alternative multi-body representation. With an explicit body, shape, fields from the inactive family are rejected; if shape is omitted and both families are populated, the request is ambiguous and is rejected. Clear the inactive branch when changing representations.

Analytic and station mold lines are mixed. Station arrays are rejected in "Analytic" mode. In "Stations" mode, the supplied stations remain the physical mold line; the selected section metadata described in [section 6.2.2](#) is the documented exception and does not create a second analytic geometry.

Body dimensions are given in flight-condition metric units. With "Centimetres", body geometry is in centimetres, not metres. The metre-based rule in that same selection applies to the relevant flight quantities, not to the geometry branch.

Full elliptical width is supplied as a station half-width. `stations.halfWidth` is a half-dimension. Analytic `nose.totalWidth`, `center.width`, and `aft.width` are full widths.

`stations.height` is interpreted as full height. In elliptical station mode, the public field named `height` contains the body half-height.

Station coordinates are assumed to be sorted automatically. Supply them in strictly increasing physical order. SpearDAT preserves the schedule that is given.

A coordinate is supplied in `discontinuityStations`. The values are one-based station indices. A discontinuity at `x(3)` is written as index 3, not as the numerical value of `x(3)`.

Every engineering section boundary is marked as a corner. Mark only true slope discontinuities. A tangent junction can be an important section boundary without being a geometric slope break.

An exact secant ogive is expected to be generated by "Ogive". The analytic "Ogive" selector generates a tangent ogive. Use station coordinates for an exact secant-ogive mold line. In station mode, an "Ogive" nose classification may still be supplied or resolved for downstream methods; it does not replace the station-defined profile.

An omitted exit diameter is replaced by zero. Omission suppresses this body base-drag increment; zero requests a closed base. The two states are intentionally distinct.

Geometry is scaled twice. If `reference.scale` is used, do not also multiply the body dimensions manually by the same factor.

`x0origin` is treated as an extra body length. It locates the physical nose tip in the global geometry coordinate system. It translates the body; it is not added as another physical section.

6.10. MATLAB-to-Legacy Field Map

The MATLAB structure is the public interface used throughout this guide. The legacy names in [table 6.5](#) are provided as landmarks for readers comparing SpearDAT with the Missile DATCOM User Guide or with a retained `for005` input deck.

The legacy station count `NX` is generated from the MATLAB station arrays rather than supplied independently. The detailed limits, defaults, omission rules, and validation conditions are collected in [sections B.9.2 to B.9.4](#).

With the body now defined, the next configuration chapter can place lifting surfaces relative to this mold line and body coordinate system.

Table 6.5.: Principal body-geometry field mapping.

MATLAB field	Legacy field	Purpose
body.shape	\$AXIBOD/\$ELLBOD	Selects the active body family.
axisymmetric.xOrigin, elliptical.xOrigin	X0/X0	Global longitudinal coordinate of the physical nose tip.
axisymmetric.stations.x, elliptical.stations.x	X	Station longitudinal coordinates.
axisymmetric.stations.radius	R	Axisymmetric station radii.
elliptical.stations.halfWidth	W	Elliptical station half-widths.
elliptical.stations.height	H	Elliptical station half-heights.
elliptical.stations.ellipticity	ELLIP	Station height-to-width ratios.
stations.discontinuityStations	DISCON	Indices of station slope discontinuities.
nose.type	TNOSE	Analytic nose type or station-mode nose classification.
nose.length	LNOSE	Nose axial length.
axisymmetric.nose.diameter	DNOSE	Axisymmetric nose base diameter.
elliptical.nose.totalWidth	WNOSE	Elliptical analytic nose full width.
elliptical.nose.ellipticity	ENOSE	Elliptical analytic nose ellipticity.
nose.bluntness	BNOSE	Spherical bluntness or truncation radius.
nose.truncated	TRUNC	Selects truncation rather than smooth spherical bluntness.
nose.powerExponent	POWER	Power-law exponent.
center.length	LCENTR	Centre-section axial length.
axisymmetric.center.diameter	DCENTR	Axisymmetric centre-section terminal diameter.
elliptical.center.width	WCENTR	Elliptical centre-section terminal full width.
elliptical.center.ellipticity	ECENTR	Elliptical centre-section ellipticity.
aft.type	TAFT	Afterbody type.
aft.length	LAFT	Afterbody axial length.
axisymmetric.aft.diameter	DAFT	Axisymmetric terminal diameter.
elliptical.aft.width	WAFT	Elliptical terminal full width.
elliptical.aft.ellipticity	EAF	Elliptical afterbody ellipticity.
axisymmetric.exitDiameter, elliptical.exitDiameter	DEXIT	Nozzle or exit opening used by the base-drag calculation.

7. Fin Sets, Panels, and Controls

Lifting surfaces are described in SpearDAT as *fin sets*. A fin set is one spanwise planform, one section family, and a collection of identical physical panels placed around the body. The same representation can therefore be used for canards, wings, strakes, and tail fins; their aerodynamic role comes from where they are placed, how they are oriented, and whether their panels are fixed or movable.

The public MATLAB branches involved in this chapter are

- `input.caseInput.finSets`, which defines the physical lifting-surface geometry;
- `input.caseInput.deflections`, which assigns fixed panel incidences and optional hinge information; and
- `input.caseInput.trim`, which asks Missile DATCOM to vary selected panels in search of longitudinal static trim.

Historically these capabilities are associated principally with the Missile DATCOM `$FINSETn`, `$DEFLCT`, and `$TRIM` inputs. SpearDAT constructs those inputs from the typed MATLAB structure; normal users do not need to work with the legacy names directly.

This chapter is deliberately organised around the physical model rather than around a field catalogue. It explains how a planform becomes a set of panels, how fin position and vehicle roll differ, how section shape is selected, and how fixed deflections and trim act on the geometry. The exact limits, defaults, omission rules, and all subordinate fields remain in [sections B.10 to B.12](#).

Important

One element of `input.caseInput.finSets` describes one complete *set*, not one physical panel. A four-panel cruciform tail is normally one MATLAB fin-set structure with `nPanels=4`, not four separate fin-set structures.

7.1. Fin Sets and Physical Panels

A useful way to think about a fin set is to separate its definition into three layers:

1. the *spanwise planform*: where the root, tip, and any intermediate break points are located, and what the local chord is at each point;
2. the *section*: the two-dimensional cross-section used from root to tip; and
3. the *panel installation*: how many copies of that planform exist and where those copies are clocked around the vehicle.

For example, the following structure describes a simple two-panel rectangular surface. The same planform is repeated at two opposite roll positions:

```
fin = struct;
fin.set = 1;

fin.spanStations = [0 4];
```



```

fin.chord = [3 3];
fin.xLeadingEdge = [20 20];

fin.nPanels = 2;
fin.panelRollDeg = [90 270];
fin.dihedralDeg = [0 0];

fin.section.type = "Naca";
fin.section.naca.sectionType = "FourDigit";
fin.section.naca.designation = "0012";

input.caseInput.finSets = fin;

```

Listing 7.1: A simple two-panel fin set.

The two entries in panelRollDeg do not create two independent airfoil definitions. They place two copies of the same rectangular NACA 0012 surface on opposite sides of the vehicle.

SpearDAT accepts at most four fin sets and at most eight panels in any one set. The fin-set number is also the identity used by later deflection, trim, and output requests. For ordinary configurations it is best to number the sets in physical front-to-back order.

7.1.1. Set identity and cumulative build-up

Missile DATCOM synthesises a multi-fin vehicle cumulatively. Conceptually, for three defined sets the progression is

$$\text{body} \rightarrow \text{body} + \text{set 1} \rightarrow \text{body} + \text{sets 1-2} \rightarrow \text{body} + \text{sets 1-3}.$$

This is why set identity must agree with physical axial order. A forward canard and an aft tail are not interchangeable merely because both happen to be four-panel cruciform surfaces. Their interference and carry-over effects are evaluated in a directional, configuration-dependent synthesis.

When set is omitted, vector position supplies the identity. Thus

```
input.caseInput.finSets = [canards tails];
```

Listing 7.2: Implicit front-to-back fin-set numbering.

means that canards is set 1 and tails is set 2. Explicit numbers are useful whenever later branches refer to a particular set:

```

canards.set = 1;
tails.set = 2;
input.caseInput.finSets = [canards tails];

```

Listing 7.3: Equivalent explicit fin-set numbering.

Numbering gaps are supported, but they are usually unnecessary in a basic MATLAB workflow. SpearDAT preserves an explicit set number rather than renumbering the request automatically.

7.1.2. Spacing between multiple sets

The fore-to-aft ordering is more than bookkeeping. The historical methods model the wake from an upstream lifting surface when synthesising a downstream set. The Missile DATCOM User Guide therefore cautions against overlapping fin sets and against treating very closely spaced surfaces as though a fully rolled-up vortex wake necessarily exists between them [1].

This is a modelling consideration rather than a universal SpearDAT minimum spacing rule. Closely coupled canard–wing or canard–tail configurations can still be analysed, but the analyst should recognise that the underlying semi-empirical interference model has assumptions about wake development. The legacy discussion notes that a real vortex sheet can require several semispans of downstream distance to approach the idealised rolled-up state. That limitation becomes especially relevant when interpreting absolute agreement with experiment for compact multi-surface vehicles.

7.2. Spanwise Planform Definition

The planform is described at a sequence of spanwise break points. At each station SpearDAT can know the local chord, leading-edge position, section proportions, and trailing-edge-device geometry. Straight interpolation between successive break points forms the individual planform segments.

A two-station set therefore has one segment, a three-station set has two, and so on. Up to ten span stations may be supplied, which permits up to nine segments in one fin set.

For ordinary MATLAB use, the clearest representation is the explicit leading-edge form:

```
fin.spanStations = [0 3 7];
fin.chord = [5.0 4.0 2.5];
fin.xLeadingEdge = [28.0 29.0 31.0];
```

Listing 7.4: A tapered swept fin described at three span stations.

The three vectors correspond positionally. At the second span station, for example, the local chord is 4 units and its leading edge is 29 units aft of the body nose.

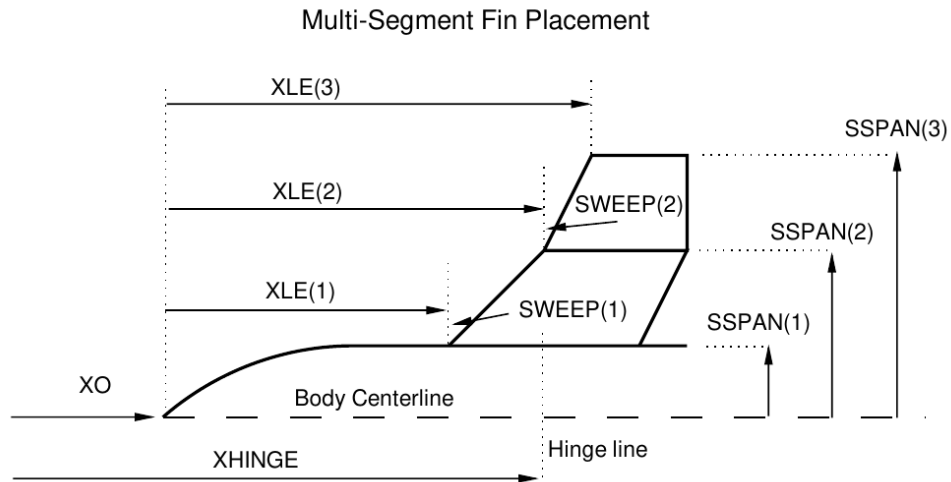
7.2.1. Automatic Root Placement on the Body

The span coordinate is measured from the vehicle centreline. There is, however, an important convenience for body-mounted fins. If the first supplied span station is zero, Missile DATCOM treats the supplied span schedule as relative to the exposed root and automatically adds the body radius evaluated at the centre of the root chord. In other words,

```
fin.spanStations = [0 8];
```

Listing 7.5: Requesting automatic root placement on the body.

should not be read as saying that the physical fin begins on the vehicle centreline. It asks the solver to place the exposed root on the local body mold line and to interpret the remaining span values relative to that root.



NOTE 1: XLE measured from body nose, XHINGE measured from origin

NOTE 2: Define either XLE(1) with various values of SWEEP
OR multiple values of XLE with no SWEEP

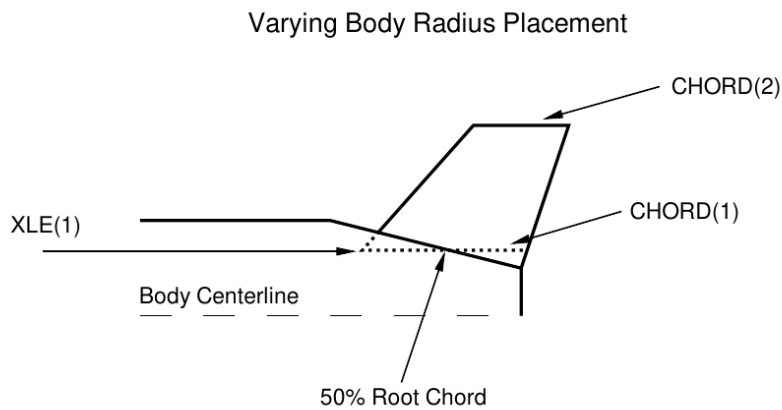


Figure 7.1.: Legacy fin-placement nomenclature, including multi-segment planforms and automatic root placement on a varying-radius body. Reproduced from Figure 6 of the Missile DATCOM User Guide [1].

This is particularly useful for a canard mounted on a nose or another varying-radius region. The important point is that the solver does not shorten, warp, or otherwise modify the supplied root chord. Suppose the root leading edge is at $x_{LE,1}$ and the supplied root chord is c_1 . When `spanStations(1)=0`, the body radius used for placement is evaluated at the axial midpoint of that straight root chord,

$$x_{50\%} = x_{LE,1} + \frac{c_1}{2}.$$

That local body radius is then added to the supplied span schedule. The root chord remains parallel to the body centreline at the resulting constant radial location. On a body whose radius changes along the chord, the body mold line therefore meets the idealised straight root at its midpoint; one part of the panel can lie slightly within the body envelope while another part lies slightly outside it. This is the historical approximation described in the Missile DATCOM User Guide [1].

The “50% Root Chord” annotation in the lower sketch of [figure 7.1](#) identifies this *axial midpoint used to sample the body radius*. It does not mean that only 50 percent of the supplied chord is retained or exposed. The complete value in `chord(1)` is still the root chord used by the fin geometry.

Note

Automatic root placement is a geometric convenience, not a general body–surface intersection calculation. For unusual installations, strongly curved roots, or surfaces that are deliberately offset from the body, inspect the resolved fin geometry and the printed geometry report rather than assuming that a zero first span station reproduces detailed CAD mounting geometry.

7.2.2. Leading-edge coordinates and sweep

The public field `xLeadingEdge` is measured from the body nose. That convention is worth separating from the deflection hinge coordinate used later: `hingeX` is measured from the active coordinate-system origin. The distinction matters when the body has a non-zero `xOrigin`.

The underlying solver also supports the historical `sweepDeg` and `sweepStation` description. The sweep reference is a chord fraction: zero refers to the leading edge and one to the trailing edge. In the current MATLAB interface, however, every populated spanwise array must match the `spanStations` length. For normal scripts it is therefore simplest to calculate the complete `xLeadingEdge` vector and omit the redundant sweep arrays.

For example, an unswept trailing edge with root chord 12, tip chord 7, and 8 units of exposed semispan can be written directly as

```
fin.spanStations = [0 8];
fin.chord = [12 7];
fin.xLeadingEdge = [44 49];
```

Listing 7.6: Explicit planform for an unswept trailing edge.

because the root trailing edge is at 56 and the tip trailing edge must remain at the same axial coordinate, giving $56 - 7 = 49$ for the tip leading edge.

The same intent can also be recorded explicitly with the legacy sweep fields. For the current MATLAB interface the complete leading-edge vector is still supplied, and any populated sweep arrays must have the same length as spanStations. A consistent unswept-trailing-edge definition is therefore

```
fin.spanStations = [0 8];
fin.chord = [12 7];
fin.xLeadingEdge = [44 49];

% Measure sweep on the trailing-edge line (STA = 1).
fin.sweepDeg = [0 0];
fin.sweepStation = [1 1];
```

Listing 7.7: Explicitly recording a trailing-edge sweep definition.

Here sweepStation=1 says that the quoted sweep angle applies to the trailing-edge line; zero would reference the leading edge. Because the complete xLeadingEdge geometry is already supplied, the sweep arrays are redundant information and must describe the same planform. For most MATLAB scripts the explicit leading-edge form alone is clearer; the sweep form is useful when an exact sweep angle is an important part of the engineering definition or when comparing directly with historical Missile DATCOM input. The detailed rules are retained in [section B.10.3](#).

7.3. Panel Count, Clocking, and Dihedral

Once a spanwise planform exists, nPanels determines how many copies of it are installed around the body. panelRollDeg then specifies the circumferential position of each copy.

SpearDAT follows the Missile DATCOM viewing convention: stand behind the vehicle and look forward along its longitudinal axis. Panel roll is measured clockwise from top vertical centre. A conventional four-panel cruciform is therefore

```
fin.nPanels = 4;
fin.panelRollDeg = [0 90 180 270];

% Cant the two side panels upward by 10 deg in the physical view.
fin.dihedralDeg = [0 -10 0 10];
```

Listing 7.8: A four-panel cruciform installation.

which places panels 1–4 at top, right, bottom, and left respectively in that view. The two side panels have equal physical upward cant even though their numeric dihedral signs are opposite. At the right-hand 90° position, negative dihedral rotates the tip toward top vertical centre; at the left-hand 270° position, positive dihedral rotates the tip in the same physical upward direction. This follows from the convention that positive dihedral acts in the direction of increasing panel roll.

dihedralDeg is also panel-specific. Positive dihedral acts in the direction of increasing panel roll. It is therefore interpreted together with the physical clocking of the panel rather than as one global “upward” direction for all surfaces.

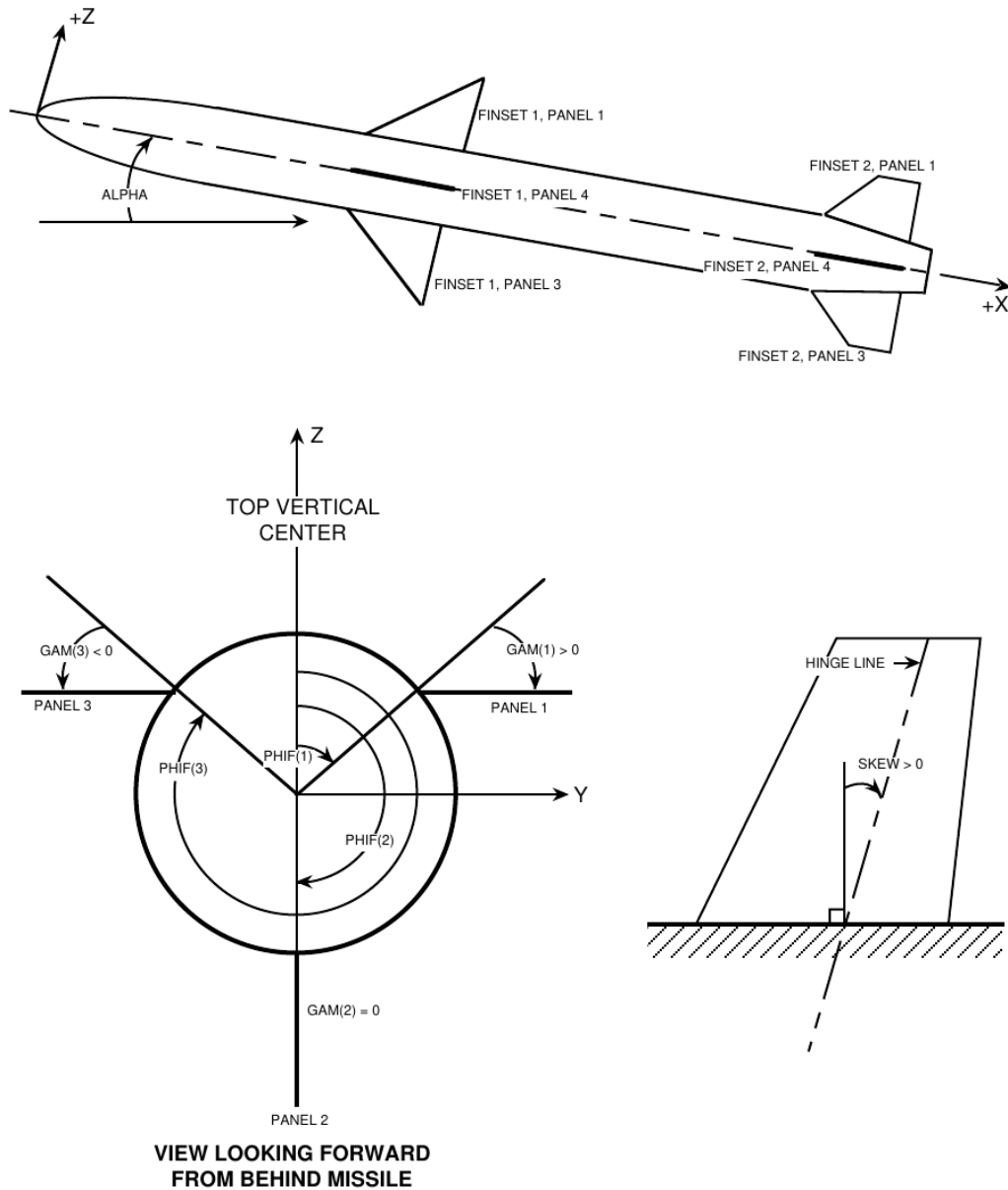


Figure 7.2.: Fin numbering, panel roll, dihedral, and hinge-skew nomenclature. Reproduced from Figure 8 of the Missile DATCOM User Guide [1].

7.3.1. Panel Clocking and Aerodynamic Roll

Two different roll-like angles appear in a complete analysis:

Panel roll, `panelRollDeg` fixes where a physical panel is installed around the body. It is part of the geometry.

Aerodynamic roll, `flight.rollDeg` changes the orientation of the complete vehicle relative to the freestream when "TotalAlphaRoll" is the selected attitude mode. It is part of the flight condition.

Changing one does not substitute for changing the other. Rotating a tail set by 45 degrees relative to an upstream canard set is a change in `panelRollDeg`; rolling the entire missile 45 degrees relative to the aerodynamic incidence plane is a change in `flight.rollDeg`.

This distinction is particularly useful for interdigitated canard–tail configurations. The two sets can have different physical clocking while the vehicle itself remains at zero aerodynamic roll.

7.4. Airfoil-Section Selection

The spanwise planform says where the fin is; the section definition says what its local cross-section looks like. One section *family* applies to an entire fin set, although supported proportions may vary from one span station to another.

SpearDAT exposes four section families:

Table 7.1.: Fin-section families available through the MATLAB interface.

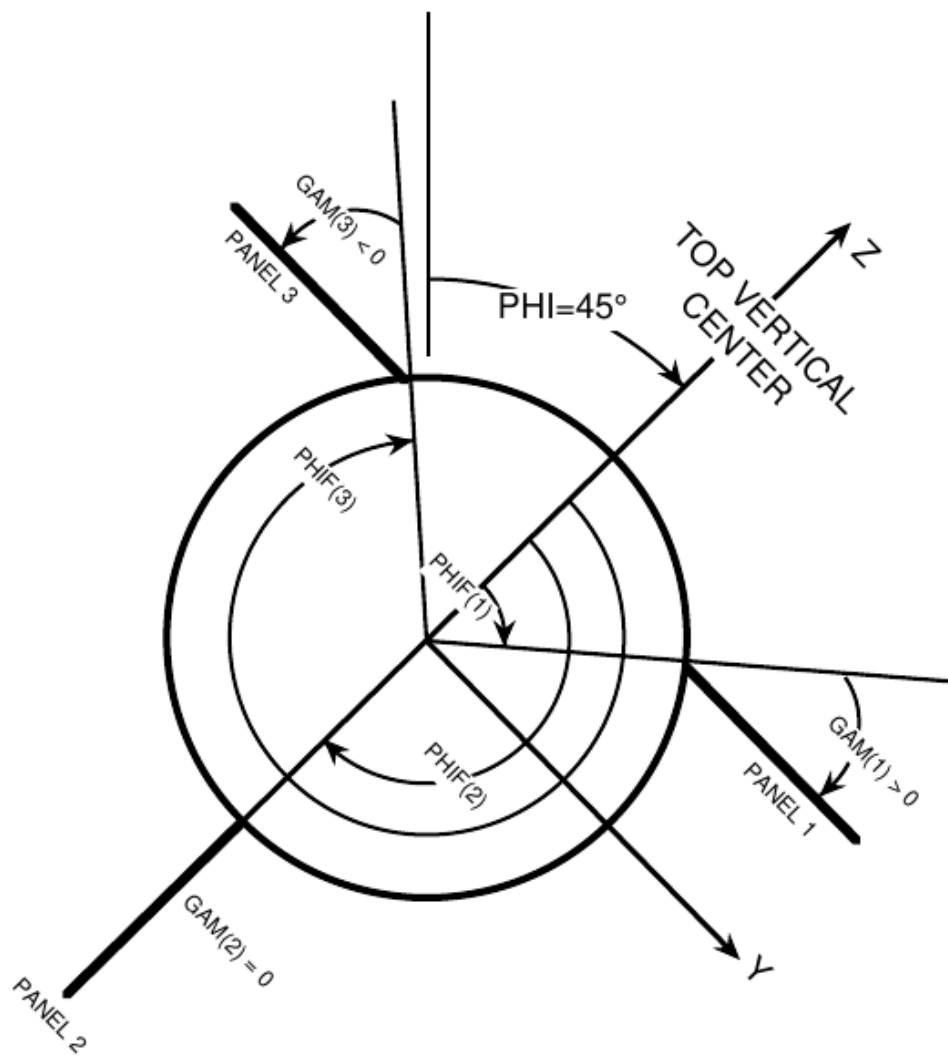
Value	Typical use	Description
"Hex"	Simple supersonic-style section	Piecewise-linear thickness shape described by upper-side maximum-thickness position and flat-length parameters.
"Arc"	Circular-arc section	Circular-arc thickness form using the supported upper/lower ordinate inputs.
"User"	Measured or custom profile	Normalised chordwise coordinates supplied directly as mean-line/thickness or upper/lower surfaces.
"Naca"	Supported NACA designation	Invokes the NACA families understood by the Missile DATCOM section parser, including one-series, four-digit, five-digit, six-series, and supported supersonic designations.

7.4.1. NACA Sections

For a familiar conventional section, the NACA path is concise:

```
fin.section.type = "Naca";
fin.section.naca.sectionType = "FourDigit";
fin.section.naca.designation = "0012";
```

Listing 7.9: Selecting a NACA 0012 section.



**VIEW LOOKING FORWARD
FROM BEHIND MISSILE**

**PHI IS THE BODY ROLL ANGLE
PHIF IS THE FIN PANEL ROLL ANGLE**

Figure 7.3.: Distinction between body aerodynamic roll and physical fin-panel roll. Reproduced from Figure 9 of the Missile DATCOM User Guide [1].

The designation is not a request to an external airfoil database. SpearDAT passes the supported family and designation to the Missile DATCOM section machinery. The exact accepted families and designation rules are given in [section B.10.5](#).

7.4.2. HEX and ARC Sections

HEX and ARC are useful when the intended section is naturally described by a small number of geometric proportions. The historical nomenclature is shown in [figure 7.4](#). These inputs are *dimensionless chord fractions*; they are not lengths in the selected lengthSystem. An “ordinate ratio” in this context means a surface ordinate normalised by the local chord, such as an upper-surface height written as z/c .

For a two-station HEX panel, for example,

```
fin.section.type = "Hex";
fin.section.hexArc.zUpper = [0.05 0.03];
fin.section.hexArc.lMaxUpper = [0.25 0.25];
fin.section.hexArc.lFlatUpper = [0.40 0.40];
```

Listing 7.10: A simple spanwise HEX section definition.

The arrays correspond to the same root and tip stations as the planform. At the root, $z_{\text{Upper}}=0.05$ means that the maximum upper-surface ordinate is 5 percent of the local chord above the chord line. For the symmetric shape represented by the automatic HEX path, the corresponding full local thickness is $0.10c$. For example, with a 200 mm local chord, the upper ordinate is 10 mm, the mirrored lower ordinate is 10 mm below the chord line, and the full thickness is 20 mm. The value is still entered as 0.05; it does not change when the request length system changes. The automatic section machinery subsequently interpolates the spanwise upper-side schedule internally when constructing the effective section used by the aerodynamic methods; that processing does not change the meaning or units of the user inputs.

$l_{\text{MaxUpper}}=0.25$ places the start of the maximum-thickness region at $x/c = 0.25$, or 25 percent chord aft of the leading edge. $l_{\text{FlatUpper}}=0.40$ then keeps that maximum thickness for another 40 percent of the chord, from $x/c = 0.25$ to $x/c = 0.65$. The remaining 35 percent forms the aft ramp back toward the trailing edge. The tip uses the same chordwise proportions in this example but has $z_{\text{Upper}}=0.03$, giving an effective symmetric maximum thickness of approximately $0.06c$. If $l_{\text{MaxUpper}}+l_{\text{FlatUpper}}=1$, there is no remaining aft-ramp length and the HEX section stays at its maximum thickness to the trailing edge.

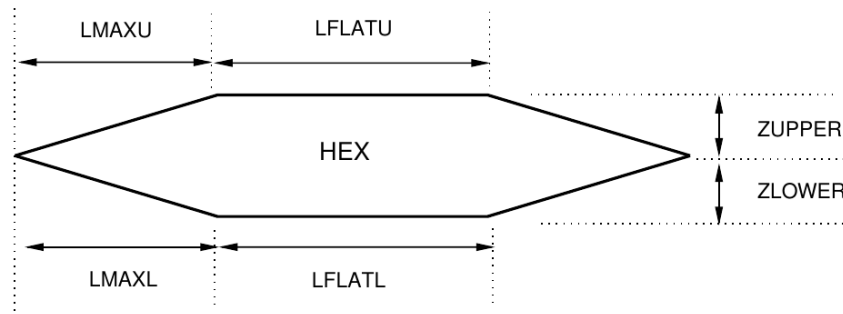
The circular-arc form is simpler because it does not use the HEX maximum- and flat-length fields:

```
fin.section.type = "Arc";
fin.section.hexArc.zUpper = [0.05 0.03];
```

Listing 7.11: A simple spanwise ARC section definition.

The meaning of z_{Upper} is the same dimensionless ordinate ratio. Thus the root value 0.05 describes a maximum upper ordinate of $0.05c$; the automatic ARC generator forms the corresponding symmetric circular-arc section, giving an effective full maximum thickness of approximately $0.10c$. For ARC, the maximum ordinate is generated at mid-chord rather than being specified through l_{MaxUpper} .

An important legacy behaviour is preserved: the automatic HEX and ARC paths generate the effective symmetric section used by the aerodynamic methods from the upper-side inputs. Corresponding lower-side fields can be accepted and echoed, but they do not independently create an asymmetric effective section in the validated legacy path. Use "User" when genuine camber or independent upper and lower surfaces are required.



NOTE: All parameters must be input at each span station



NOTE: ARC section only allows ZUPPER and ZLOWER

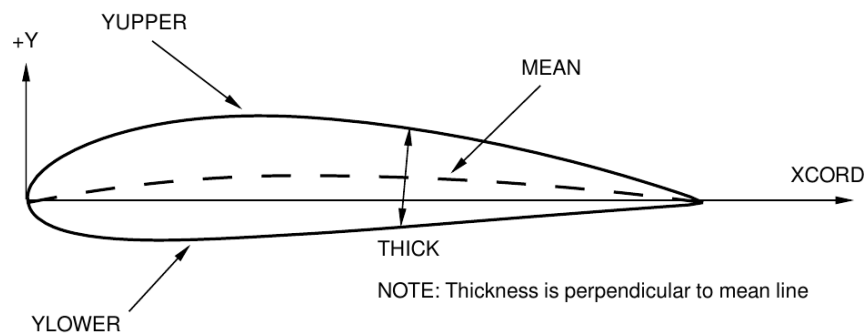


Figure 7.4.: Legacy HEX, ARC, and USER section nomenclature. Reproduced from Figures 10 and 11 of the Missile DATCOM User Guide [1].

7.4.3. User-Defined Sections

The USER path is the general option when a section comes from CAD, a measured profile, or an airfoil shape that cannot be represented cleanly by the simple automatic families. The chordwise coordinate is normalised from leading edge to trailing edge and may be accompanied by either

- mean-line ordinate plus full thickness; or
- explicit upper- and lower-surface ordinates.

For example, a simple symmetric profile can be supplied as

```
fin.section.type = "User";
fin.section.user.xChord = [0 0.25 0.5 0.75 1.0];
fin.section.user.mean = [0 0 0 0 0];
fin.section.user.thickness = [0 0.08 0.10 0.06 0];
```

Listing 7.12: A small custom symmetric section.

Here $xChord$ is the normalised chord coordinate x/c : the five entries sample the leading edge, 25, 50, and 75 percent chord, and the trailing edge. The zero mean vector places the mean line on the chord line, so the example has no camber. *thickness* is the *full* local thickness divided by chord, measured perpendicular to the mean line. The value 0.10 at mid-chord therefore means a full section thickness of $0.10c$; for this symmetric zero-mean example the upper and lower surfaces lie approximately at $+0.05c$ and $-0.05c$ there. The zero values at the first and last stations close the section at the leading and trailing edges.

If the upper and lower surface coordinates are already known directly, the alternative USER representation supplies $xChord$ together with $yUpper$ and $yLower$, each again as a fraction of local chord. This avoids reconstructing a mean line and thickness distribution from an existing measured or CAD-derived profile.

The field-level limits and closure responsibilities are described in [section B.10.5](#).

7.5. Plain Trailing-Edge Devices

A fin can include a plain trailing-edge device through `flapChordRatio`. At each span station the ratio is

$$\text{flapChordRatio} = \frac{\text{local movable trailing-edge chord}}{\text{local fin chord}}.$$

Thus a value of 0.25 means that the aft 25 percent of the local chord belongs to the device. A zero means that the device is absent at that station, while a value of one makes the complete local chord participate in the panel-style deflection.

A full-span 25-percent-chord device on the two-station example is simply

```
fin.spanStations = [0 8];
fin.chord = [12 7];
fin.xLeadingEdge = [44 49];
fin.flapChordRatio = [0.25 0.25];
```

Listing 7.13: A full-span 25-percent-chord trailing-edge device.

A partial-span device needs break points at both ends of its active region:

```

fin.spanStations = [0 2 6 8];
fin.chord = [4 4 3 3];
fin.xLeadingEdge = [10 10.7 12.0 12.0];
fin.flapChordRatio = [0 0.25 0.25 0];

```

Listing 7.14: A partial-span trailing-edge device.

The non-zero ratios define the device between stations 2 and 3. They do not command a deflection; they only define which part of the planform is movable. Motion is supplied separately through deflections or requested through trim.

For a CFOC-style trailing-edge device, the hinge line is implied by the planform and the local chord ratio; it is not supplied through hingeX or hingeSkewDeg. At an active span station, its longitudinal position is

$$x_{\text{hinge}} = x_{\text{LE}} + (1 - r_f) c,$$

where r_f is flapChordRatio and c is the local fin chord. For example, $r_f = 0.25$ places the inferred hinge at 75 percent chord. The hinge points implied at all active span stations must lie on one straight line. This remains a geometric requirement even when hinge moments are not of interest: the historical plain trailing-edge-device model assumes a straight hinge, and in fact does not calculate hinge moments for CFOC-defined devices. The separate hingeSkewDeg input belongs to the panel-hinge model and does not redefine or relax this trailing-edge-device hinge geometry.

The legacy model imposes genuine geometric restrictions. In particular, the active hinge line must remain straight; a cranked hinge is not represented by simply adding more planform stations. The movable device must also not grow in chord toward the tip: its outboard-to-inboard device-chord taper ratio may not exceed one. This restriction applies to the *actual movable chord*

$$c_f = r_f c,$$

where r_f is the local value supplied through flapChordRatio. The restriction therefore applies to c_f , not to flapChordRatio by itself. A ratio value may therefore increase outward if the local fin chord decreases enough that the resulting movable chord still stays constant or becomes smaller. Conversely, a geometry whose grey trailing-edge region becomes wider toward the tip is invalid even if every individual flapChordRatio value lies between zero and one.

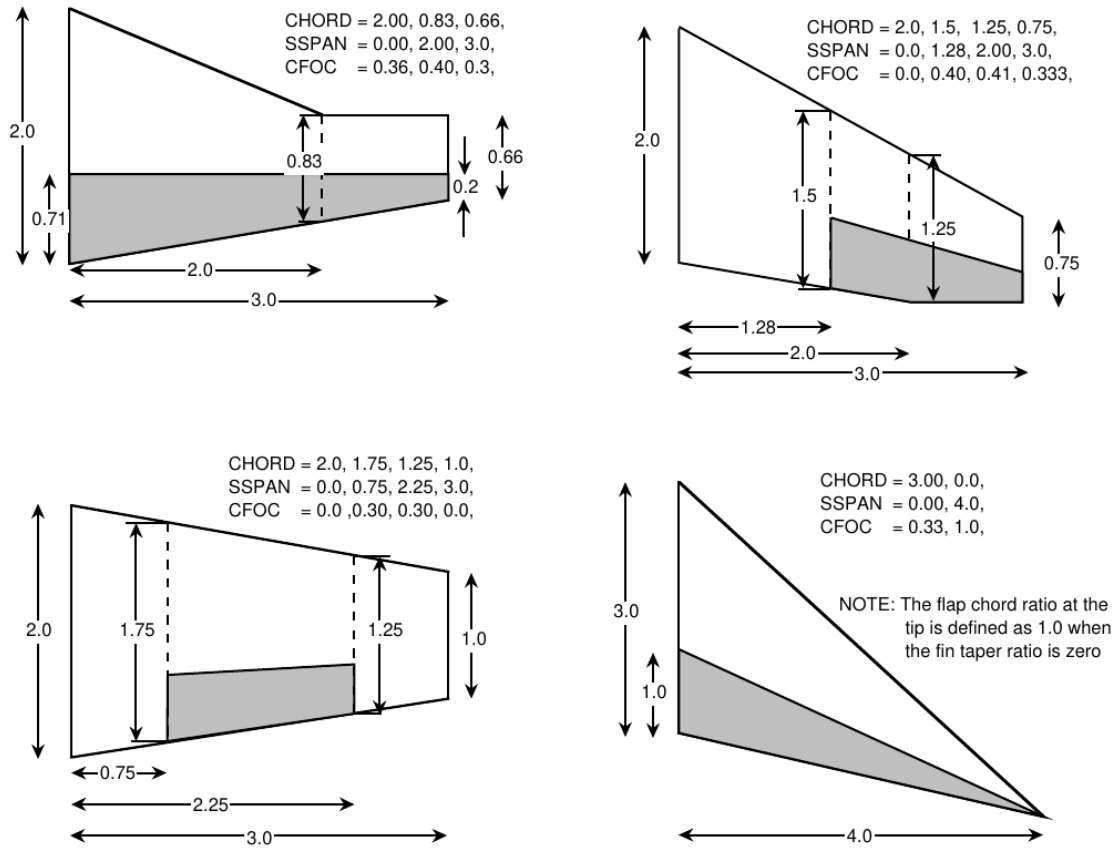
The following intentionally invalid example isolates the cranked-hinge restriction. The chord and leading edge are constant, but the three non-zero ratios imply hinge locations of 12.0, 12.4, and 13.0 in the active length units. Those three points are not collinear in the spanwise planform, so the hinge changes sweep at the middle station:

```

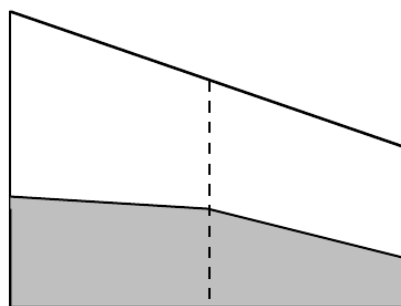
fin.spanStations = [0 4 8];
fin.chord = [4 4 4];
fin.xLeadingEdge = [10 10 10];
fin.flapChordRatio = [0.50 0.40 0.25];

```

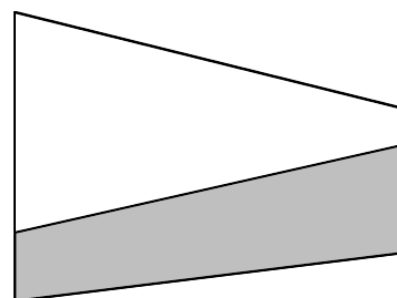
Listing 7.15: Illustrative invalid trailing-edge device: cranked hinge line.



Valid Trailing Edge Device Definitions



• Cranked Hinge Line Is Invalid



• Trailing Edge Device With Taper Ratio Greater Than One Is Invalid

Invalid Trailing Edge Device Definitions

Figure 7.5.: Examples of valid and invalid plain trailing-edge-device geometry. Reproduced from Figure 7 of the Missile DATCOM User Guide [1].

A different invalid construction can have a perfectly straight two-point hinge but an outward-growing movable chord. Here the local fin chord remains 4, while the movable chord grows from $0.20 \times 4 = 0.8$ to $0.40 \times 4 = 1.6$, giving a trailing-edge-device taper ratio of 2:

```
fin.spanStations = [0 8];
fin.chord = [4 4];
fin.xLeadingEdge = [10 10];
fin.flapChordRatio = [0.20 0.40];
```

Listing 7.16: Illustrative invalid trailing-edge device: device taper greater than one.

These listings are deliberately examples of geometries that should not be used. They reproduce the two failure modes illustrated in [figure 7.5](#): a broken hinge line and an outward-growing device chord. The restrictions belong to the aerodynamic model itself, not to hinge-moment reporting and not merely to MATLAB array-shape rules.

7.6. Fixed Panel Deflections

A fin-set definition establishes the zero-deflection geometry. Fixed panel incidences are then applied under `input.caseInput.deflections.finSets`. Each deflection element refers to one defined fin set and supplies one raw command per physical panel.

For a four-panel cruciform set,

```
defl = struct;
defl.set = 1;
defl.panelDeflectionDeg = [0 5 0 -5];
defl.hingeX = 35.0;
defl.hingeSkewDeg = 0;

input.caseInput.deflections.finSets = defl;
```

Listing 7.17: Applying a fixed pitch-pair deflection to fin set 1.

leaves panels 1 and 3 undeflected and commands panels 2 and 4 with opposite raw numerical signs. The two hinge fields describe the hinge line associated with fin set 1. `hingeX=35.0` places that line at longitudinal coordinate 35 in the active length units, measured from the geometry coordinate origin; `hingeSkewDeg=0` gives it zero sweepback under the Missile DATCOM hinge-line convention. A non-zero skew rotates the hinge line in the positive sweepback sense illustrated at the lower right of [figure 7.2](#). The hinge definition is separate from the four panel angles: `panelDeflectionDeg` commands panel incidence, whereas `hingeX` and `hingeSkewDeg` locate the line about which hinge-related geometry and report quantities are referenced.

7.6.1. Panel Deflection Sign Convention

The raw Missile DATCOM panel convention is defined by rolling-moment sense, not by a universal “trailing edge down is positive” rule. SpearDAT preserves that convention:

Important

A positive raw `panelDeflectionDeg` is the deflection sense that produces a negative body-axis rolling-moment increment at zero angle of attack and zero sideslip.

Consequently, the same physical pitch-control motion on panels mounted on opposite sides of a cruciform vehicle requires opposite raw numeric signs. For a cruciform whose panelRollDeg values are 0°, 90°, 180°, and 270°, the pair

```
delta = 10;
defl.panelDeflectionDeg = [0 delta 0 -delta];
```

Listing 7.18: Raw signs for an equal-magnitude physical pitch pair.

is the natural SpearDAT form for panels 2 and 4 acting together in one pitch sense. This is one reason not to copy signed panel angles from a wind-tunnel report blindly: the report may use a different panel-local sign convention. The complete raw convention is recorded in [section B.11.2](#).

7.6.2. Hinge location and skew

hingeX deserves particular care because it does *not* use the same origin convention as xLeadingEdge:

- xLeadingEdge is measured from the body nose;
- hingeX is measured from the active geometry coordinate origin.

If the body begins at xOrigin=0, these descriptions coincide. If the body uses a non-zero origin, the two coordinates must be converted deliberately.

Omitting hingeX does not mean that no hinge exists. The historical default places the hinge longitudinally at one-half of the exposed root chord,

$$x_{\text{hinge,default}} = x_{\text{origin}} + x_{\text{LE,root}} + \frac{c_{\text{root}}}{2}.$$

For a real all-moving canard, flap hinge, or comparison with measured hinge loads, supplying the physical longitudinal hinge location explicitly is preferable.

hingeSkewDeg describes the sweepback of that hinge line for the panel-style hinge model. If it is omitted, Missile DATCOM uses the historical default of 0°, corresponding to an unswept hinge line at constant longitudinal x across the span. Supplying hingeSkewDeg=0 makes that same geometry explicit. hingeSkewDeg is distinct from the spanwise planform sweep and from the plain trailing-edge-device geometry introduced above. The lower-right sketch in [figure 7.2](#) shows the hinge-line reference and the direction labelled SKEW > 0.

7.7. Longitudinal Trim

The trim branch asks Missile DATCOM to vary selected panels from one fin set until the pitching moment crosses zero. A typical request is

```
input.caseInput.trim.enabled = true;
input.caseInput.trim.finSet = 1;
input.caseInput.trim.panels = [2 4];
input.caseInput.trim.deflectionMinDeg = -15;
input.caseInput.trim.deflectionMaxDeg = 15;
```

Listing 7.19: Requesting longitudinal trim using panels 2 and 4.

Only one fin set participates in a trim request. The selected panel numbers must belong to that set.

7.7.1. Trim Search and Interpolation

The historical trim routine is deliberately simple. At each active Mach condition it evaluates ten equally spaced deflection values between the lower and upper bounds, including both endpoints. For each angle of attack it then

1. accepts an exactly zero sampled pitching moment if one occurs;
2. otherwise finds the first neighbouring pair whose pitching moments have opposite signs; and
3. linearly interpolates within that bracket to estimate where $C_m = 0$, then applies the same interpolation fraction to the other coefficients stored in the two sampled trim rows.

For example, suppose two neighbouring trial deflections are δ_1 and δ_2 , with pitching moments $C_{m,1}$ and $C_{m,2}$ on opposite sides of zero. The linear fraction from the first trial to the estimated zero crossing is

$$w = \frac{0 - C_{m,1}}{C_{m,2} - C_{m,1}},$$

and the reported trim deflection is estimated as

$$\delta_{\text{trim}} = \delta_1 + w(\delta_2 - \delta_1).$$

The same w is used to interpolate quantities such as normal force, axial force, lift, and drag between those two sampled rows. In other words, Missile DATCOM does not first interpolate δ_{trim} and then execute a new complete nonlinear aerodynamic solution at that exact angle; it reports an interpolated estimate from the already calculated trim matrix.

If no sampled pair brackets zero, that operating point is not reported as trimmed; the routine does not extrapolate outside the requested range.

Note

The returned trim deflection is an interpolation-based Missile DATCOM estimate, not the result of a continuously converged nonlinear root solve. Re-running a fixed-deflection case at the reported value need not produce machine-zero pitching moment. If a tighter root is required, use repeated fixed-deflection SpearDAT evaluations with an external root finder.

If the two bounds are omitted, the historical search interval is -25 to $+20$ degrees. When one bound is supplied, the other must be supplied as well.

7.7.2. Trim Deflection Sign Convention

The trim branch searches one scalar deflection value even when more than one panel is selected. To make that scalar useful for a conventional pitch pair, the historical trim path automatically reverses the raw sign on panels located on the left-hand side of the configuration. For a standard cruciform with side panels at 90° and 270°, selecting panels 2 and 4 therefore maps a positive trim trial $+\delta$ onto the raw fixed-deflection pattern

$$[0, +\delta, 0, -\delta].$$

Thus one positive trim value represents the two opposite raw DATCOM signs needed to move the side panels together in the same physical pitch-control sense. Under the historical convention this is reported as a practical “positive deflection is trailing edge down” trim sign. The advanced `asymmetryPanels` input can reverse that automatic trim sign treatment for selected panels. For example, if panels 2 and 4 are already the selected trim pair,

```
input.caseInput.trim.panels = [2 4];
input.caseInput.trim.asymmetryPanels = 4;
```

Listing 7.20: Reversing the trim sign treatment for one selected panel.

marks panel 4 for the additional ASYM sign reversal. The field does not select an extra movable panel: every entry in `asymmetryPanels` must already appear in `trim.panels`. It also changes only the trim sign treatment; it does not redefine the raw fixed-deflection convention in `panelDeflectionDeg`. The complete behaviour is collected in [section B.12.3](#).

This distinction matters when comparing a trim result with a later fixed- deflection run. The scalar trim value is not copied with the same numeric sign to every selected element of `panelDeflectionDeg`; it must first be translated back to the raw panel convention.

7.8. Resolved Fin Geometry

As with body geometry, SpearDAT exposes calculated fin geometry after the case has been prepared and executed. The four possible set slots are available at

```
finGeometry = result.cases(1).resolvedState.geometry.finSets;

% Inspect the first defined set.
fin1 = finGeometry(1);
disp(fin1)
disp(fin1.segments)
```

Listing 7.21: Inspecting resolved fin geometry.

The available flag distinguishes a populated set from an unused slot. Useful whole-set quantities include aspect ratio, taper ratio, exposed semispan, planform and wetted area, effective thickness-to-chord ratio, mean aerodynamic chord, geometric centre, centroid, and average sweep angles. For a multi-segment planform, `segments` also exposes the corresponding segment-level geometry.

A compact inspection pattern is

```

for k = 1:numel(finGeometry)
    if ~finGeometry(k).available
        continue
    end

    fprintf("Fin set %d\n", finGeometry(k).finSet);
    fprintf("  semispan:   %.6g\n", finGeometry(k).semiSpan);
    fprintf("  area:       %.6g\n", finGeometry(k).planformArea);
    fprintf("  AR:        %.6g\n", finGeometry(k).aspectRatio);
    fprintf("  taper:     %.6g\n", finGeometry(k).taperRatio);
    fprintf("  MAC:       %.6g\n", finGeometry(k).meanAerodynamicChord);
    fprintf("  LE sweep:  %.6g deg\n", ...
        finGeometry(k).averageLeadingEdgeSweepDeg);
end

```

Listing 7.22: A few useful fin-geometry sanity checks.

For detailed legacy-style confirmation, selected reports can also be requested with `output.printGeometry` and `output.printAero`:

```

input.caseInput.output.plot = true;
input.caseInput.output.printGeometry = ["FinSet1", "FinSet2"];
input.caseInput.output.printAero = ...
    ["FinSet1", "FinSet2", "Synths"];

```

Listing 7.23: Requesting fin and synthesis reports.

These selectors add the requested detailed sections to Missile DATCOM's main human-readable `for006.dat` report. They do not add extra columns to `result.aerodynamics`. To read the complete report directly from a MATLAB workflow, retain the legacy-format files for that execution:

```

options = struct;
options.legacyFiles = true;
options.outputDirectory = fullfile(pwd, "speardat_fin_reports");

reportResult = speardat_mex(input, options);

caseDirectory = string(reportResult.cases(1).legacyDirectory);
for006Path = fullfile(caseDirectory, "for006.dat");
for006Text = fileread(for006Path);
disp(for006Text)

```

Listing 7.24: Retaining and reading the detailed `for006` report.

The requested output directory must be absent or empty before the call. During normal temporary execution the structured result is still returned, but the current MATLAB interface does not expose the complete raw `for006.dat` text as a result field. Retained-output operation is introduced in [chapter 3](#) and specified in [section B.2.3](#). The field-by-field report selectors are listed in [section B.5](#).

7.9. Worked Example: Adding NASA C6 Canards and T2 Tails

Chapter 6 introduced the semiblunt N3 body from the Ames wind-tunnel investigation of Kassner and Wettlaufer. We can now extend that same vehicle with lifting surfaces.

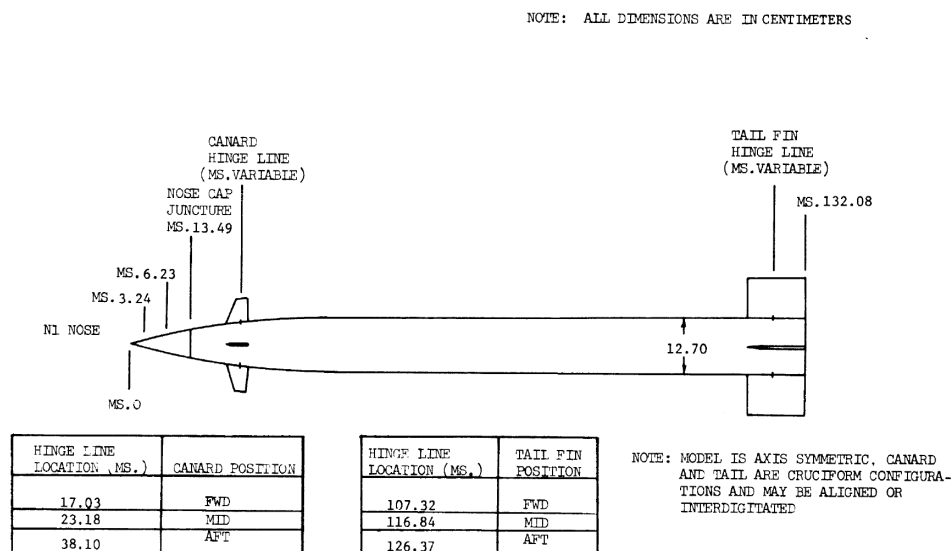
The NASA report tested several four-canard and four-tail arrangements. The configuration names used here are

C6 the large canard set at the aft canard station;

T2 the tail set at the middle tail station; and

BN3C6T2 the N3 body with both C6 canards and T2 tails.

The report gives the canard and tail hinge stations in its basic-model drawing and the panel dimensions in Table I. It also states that all canard and tail sets contain four panels and that the tail could be aligned with the canards or interdigitated by 45 degrees [3].



(a) Basic model drawing

Figure 3. - Basic model and components

Figure 7.6.: NASA basic model and the available canard/tail hinge stations. Reproduced from Figure 3(a) of NASA TM X-73,218 [3].

7.9.1. NASA Planform Reconstruction

The NASA control-panel drawing uses the lettered dimensions shown in figures 7.7 and 7.8, while the corresponding numerical values are listed in Table I of the NASA report. The quantities used for the planform reconstruction are:

A leading-edge sweep angle;

C root chord;

E exposed root-to-tip semispan;

F distance from the root hinge line to the root trailing edge;

I tip chord, when it is tabulated; and

hinge station the absolute NASA model station of the hinge line from the basic-model drawing.

Table I does not print a separate value of I for C6. That value is inferred below from the other reported dimensions and the unswept trailing edge shown in the canard drawing; it is therefore identified as a reconstruction rather than a directly tabulated dimension.

Table 7.2.: NASA C6 and T2 planform quantities used in the reconstruction.

Quantity	C6	T2	Interpretation
Leading-edge angle A	45°	45°	Leading-edge sweep in the panel drawing.
Root chord C	10.16 cm	17.78 cm	Exposed root chord.
Exposed span E	9.53 cm	8.89 cm	Root-to-tip span; used directly with a zero first SpearDAT span station so the local body radius is added automatically.
Hinge-to-TE F	4.37 cm	7.62 cm	Distance from the hinge station to the root trailing edge.
Tip chord I	inferred 0.63 cm	8.89 cm	T2 is tabulated directly in Table I of the NASA report. The C6 entry is blank; its value is reconstructed from the reported root geometry and the unswept trailing edge shown in the drawing.
Hinge station	38.10 cm	116.84 cm	NASA model-station coordinate, measured from the report origin.

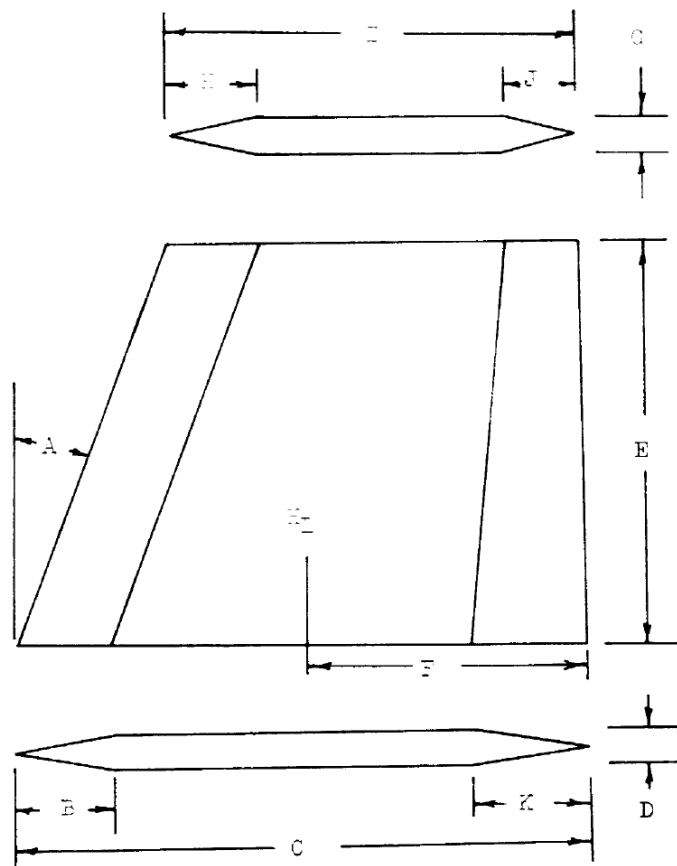
The root leading edge is not tabulated directly because the report gives the hinge station and the distance F from that hinge line to the root trailing edge. Dimension C is the complete root chord, so the distance from the root leading edge to the hinge is $C - F$. Consequently,

$$x_{\text{LE,root}} = x_{\text{hinge}} - (C - F).$$

For C6, the hinge is at model station 38.10 cm, $C = 10.16$ cm, and $F = 4.37$ cm. Hence

$$x_{\text{LE,root,C6}} = 38.10 - (10.16 - 4.37) = 32.31 \text{ cm}.$$

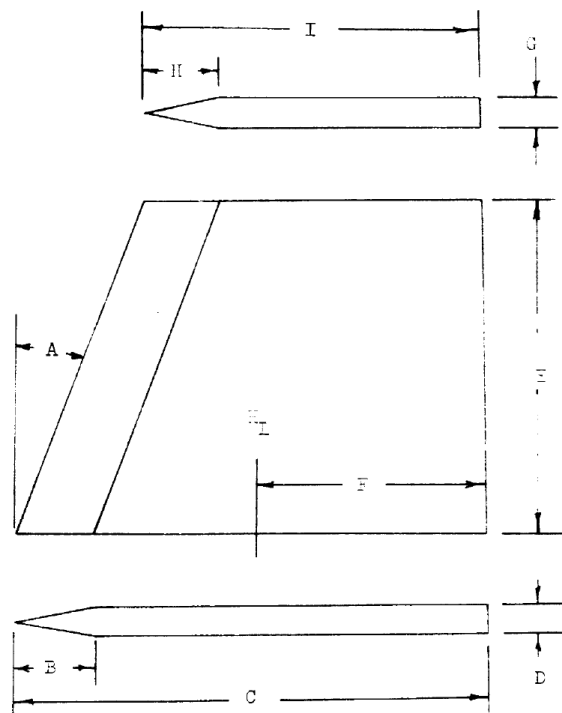
For T2, the corresponding values are 116.84 cm, 17.78 cm, and 7.62 cm, giving



NOTE: SEE TABLE I FOR DIMENSIONS OF CONTROL PANELS

(c) Canard configurations

Figure 7.7.: NASA canard planform and section dimension labels used by Table I. Reproduced from Figure 3(c) of NASA TM X-73,218 [3].



NOTE: SEE TABLE I FOR GEOMETRY OF CONTROL PANELS

(d) Tail configuration

Figure 7.8.: NASA tail planform and section dimension labels used by Table I. Reproduced from Figure 3(d) of NASA TM X-73,218 [3].

$$x_{LE,root,T2} = 116.84 - (17.78 - 7.62) = 106.68 \text{ cm.}$$

These are NASA *model-station* coordinates. The N3 physical nose begins at model station 3.24 cm, whereas SpearDAT `xLeadingEdge` is measured from that physical nose. Subtracting 3.24 cm therefore gives the root values used in MATLAB:

$$x_{LE,root,C6}^{SpearDAT} = 29.07 \text{ cm,} \quad x_{LE,root,T2}^{SpearDAT} = 103.44 \text{ cm.}$$

Dimension *E* is the exposed semispan, not an axial coordinate. With a leading-edge sweep *A*, the axial setback of the tip leading edge relative to the root leading edge is

$$\Delta x_{LE} = E \tan A.$$

Both surfaces have $A = 45^\circ$, so $\tan A = 1$. For C6 the 9.53 cm semispan therefore moves the tip leading edge 9.53 cm aft: its model station is $32.31 + 9.53 = 41.84$ cm, or $41.84 - 3.24 = 38.60$ cm in SpearDAT's nose-relative coordinate. For T2 the 8.89 cm semispan gives model station $106.68 + 8.89 = 115.57$ cm, or 112.33 cm nose-relative. Thus 38.60 cm and 112.33 cm are the *tip leading-edge x coordinates*; the semispans remain 9.53 cm and 8.89 cm respectively.

The C6 tip chord *I* can then be recovered from the unswept trailing edge in the NASA drawing. The root trailing edge is at

$$x_{TE,root} = 32.31 + 10.16 = 42.47 \text{ cm.}$$

An unswept trailing edge places the tip trailing edge at the same model station. Since the tip leading edge is at 41.84 cm,

$$I_{C6} = 42.47 - 41.84 = 0.63 \text{ cm.}$$

This 0.63 cm tip chord is therefore a SpearDAT reconstruction from the rounded NASA dimensions and drawing, not a separately printed C6 value in Table I.

Important

The NASA example demonstrates the two longitudinal origins discussed earlier. The fin `xLeadingEdge` values are measured from the physical N3 nose at model station 3.24 cm, whereas `hingeX` uses the NASA coordinate-system origin. The C6 hinge is therefore entered as 38.10 cm, not 34.86 cm.

7.9.2. NASA HEX Section Reconstruction

Table I of the NASA report also provides the root and tip section dimensions, using the letter definitions in [figures 7.7](#) and [7.8](#). Those dimensions do not map one-for-one to SpearDAT fields, so it is useful to show the conversion explicitly.

For C6, $D = 0.38$ cm is the *full* root-section thickness. The symmetric HEX input `zUpper`, however, is the distance from the chord line to the upper surface divided by local chord. For a symmetric section that upper distance is half the full thickness, which is why the reported 0.38 cm is divided by two:

$$Z_{\text{upper,root}} = \frac{D/2}{C} = \frac{0.38/2}{10.16} \approx 0.01870.$$

Dimension $B = 2.54$ cm is the root leading-edge wedge length, so it becomes the maximum-thickness location

$$\frac{B}{C} = \frac{2.54}{10.16} = 0.25,$$

which is supplied as `lMaxUpper(1)`.

Dimension $K = 2.54$ cm is the root trailing wedge length. The remaining middle part of the chord is the constant-thickness region, giving

$$\frac{C - B - K}{C} = \frac{10.16 - 2.54 - 2.54}{10.16} = 0.50,$$

which is supplied as `lFlatUpper(1)`.

The tip uses the analogous dimensions G , H , I , and J : I is the tip chord itself and therefore becomes `chord(2)`; G is the full tip thickness, so $(G/2)/I$ becomes `zUpper(2)`; H/I becomes `lMaxUpper(2)`; and $(I - H - J)/I$ becomes `lFlatUpper(2)`. For C6 the report gives $G = 0.15$ cm and $H = J = 0$; with the reconstructed $I = 0.63$ cm this gives approximately 0.11905, 0, and 1.0 for the three tip HEX proportions.

The T2 tail drawing uses the same principle but has no separate aft wedge label. At its root the constant-thickness part therefore extends from the end of the leading wedge B to the trailing edge, giving $(C - B)/C$. At the tip the corresponding quantities are G for full thickness, H for leading wedge length, and I for chord, giving $(G/2)/I$, H/I , and $(I - H)/I$. These are the values used in the listing below.

This is a SpearDAT reconstruction of the rounded NASA drawing dimensions. It should therefore be treated as the geometric model used for comparison, not as a claim that NASA published a Missile DATCOM input deck for these surfaces.

7.9.3. Complete aligned C6–T2 input

The following listing starts from the N3 body definition used in the preceding chapter, adds C6 as fin set 1 and T2 as fin set 2, and keeps the two cruciform sets aligned. The first span station is zero in each set, so the NASA exposed span dimensions are interpreted relative to the local body root.

```
input = struct;

%% Units
input.settings.lengthSystem = "Centimetres";
input.settings.derivativeSystem = "Degrees";
input.settings.options.buildOutput = true;

input.caseInput.id = "NASA N3 C6 T2 aligned";
input.caseInput.output.plot = true;
input.caseInput.output.printGeometry = ...
    ["Body", "FinSet1", "FinSet2"];
input.caseInput.output.printAero = ...
```



```

    ["Body", "FinSet1", "FinSet2", "Synths"];
input.caseInput.theory.bodyMethod = "VanDykeHybrid";

%% Flight conditions
input.caseInput.flight.attitudeMode = "BodyAlphaBeta";
input.caseInput.flight.alphaDeg = 0:2:24;
input.caseInput.flight.betaDeg = 0.0;

input.caseInput.flight.speed.mode = "Mach";
input.caseInput.flight.speed.mach = [0.8, 1.3, 1.75];

input.caseInput.flight.atmosphere.mode = "ReynoldsSpecified";
rePerMetre = 6.25e5 / 0.127;
input.caseInput.flight.atmosphere.reynoldsPerLength = ...
    [rePerMetre, rePerMetre, rePerMetre];

%% Reference quantities
input.caseInput.reference.sref = 126.7;
input.caseInput.reference.lref = 12.70;
input.caseInput.reference.latref = 12.70;
input.caseInput.reference.xcg = 66.0;
input.caseInput.reference.zcg = 0.0;
input.caseInput.reference.boundaryLayer = "Turbulent";

%% NASA N3 body from Chapter 6
input.caseInput.body.shape = "Axisymmetric";
input.caseInput.body.axisymmetric.geometryMode = "Analytic";
input.caseInput.body.axisymmetric.xOrigin = 3.24;

input.caseInput.body.axisymmetric.nose.type = "Ogive";
input.caseInput.body.axisymmetric.nose.length = 38.10 - 3.24;
input.caseInput.body.axisymmetric.nose.diameter = 12.70;
input.caseInput.body.axisymmetric.nose.bluntness = 1.42;
input.caseInput.body.axisymmetric.nose.truncated = false;

input.caseInput.body.axisymmetric.center.length = 132.08 - 38.10;
input.caseInput.body.axisymmetric.center.diameter = 12.70;

%% C6 canards: set 1
c6 = struct;
c6.set = 1;
c6.spanStations = [0 9.53];
c6.chord = [10.16 0.63];
c6.xLeadingEdge = [29.07 38.60];
c6.nPanels = 4;
c6.panelRollDeg = [0 90 180 270];
c6.dihedralDeg = [0 0 0 0];

c6.section.type = "Hex";
c6.section.hexArc.zUpper = ...
    [0.0187008 0.1190476];
c6.section.hexArc.lMaxUpper = [0.25 0.0];
c6.section.hexArc.lFlatUpper = [0.50 1.0];

%% T2 tails: set 2
t2 = struct;
t2.set = 2;
t2.spanStations = [0 8.89];

```

```

t2.chord = [17.78 8.89];
t2.xLeadingEdge = [103.44 112.33];
t2.nPanels = 4;
t2.panelRollDeg = [0 90 180 270];
t2.dihedralDeg = [0 0 0 0];

t2.section.type = "Hex";
t2.section.hexArc.zUpper = ...
    [0.0143420 0.0286839];
t2.section.hexArc.lMaxUpper = ...
    [0.2142857 0.4285714];
t2.section.hexArc.lFlatUpper = ...
    [0.7857143 0.5714286];

input.caseInput.finSets = [c6 t2];

%% Zero-incidence baseline plus physical hinge stations
c6Defl = struct;
c6Defl.set = 1;
c6Defl.panelDeflectionDeg = [0 0 0 0];
c6Defl.hingeX = 38.10;
c6Defl.hingeSkewDeg = 0;

t2Defl = struct;
t2Defl.set = 2;
t2Defl.panelDeflectionDeg = [0 0 0 0];
t2Defl.hingeX = 116.84;
t2Defl.hingeSkewDeg = 0;

input.caseInput.deflections.finSets = [c6Defl t2Defl];

%% Execute
result = speardat_mex(input);

if ~result.succeeded
    disp(result.diagnostics)
    error("SpearDAT analysis did not complete successfully.")
end

%% Inspect the two resolved fin sets
finGeometry = result.cases(1).resolvedState.geometry.finSets;
disp(finGeometry(1))
disp(finGeometry(2))

```

Listing 7.25: NASA N3 body with aligned C6 canards and T2 tails.

The automatic HEX path is appropriate to the sharp, piecewise-linear panel sections shown in the report. As explained earlier, the validated automatic HEX calculation uses the upper-side proportions to construct the effective symmetric section. The lower-side arrays are therefore not duplicated in this example.

The resolved geometry provides a useful check on that reconstruction. For the case above, SpearDAT reports a C6 semispan of 9.53 cm, leading-edge sweep of 45 degrees, effectively zero trailing-edge sweep, and taper ratio about 0.062, consistent with 0.63/10.16. T2 resolves to an 8.89 cm semispan, 45-degree leading-edge sweep, effectively zero trailing-edge sweep, and taper ratio 0.5. The corresponding planform areas are approximately 51.41 cm² for one C6

panel and 118.55 cm^2 for one T2 panel. These checks are particularly useful because the 0.63 cm C6 tip chord is reconstructed rather than printed directly in Table I of the NASA report.

7.9.4. Aligned and interdigitated tails

The report tested the tail both aligned with the canards and interdigitated by 45 degrees. In the aligned case above, both sets use `panelRollDeg` values of 0° , 90° , 180° , and 270° .

An interdigitated case is created by rotating the *tail panel clocking* relative to the canards. It is not created by changing `flight.rollDeg`, because that would roll the complete vehicle relative to the freestream. The exact signed 45-degree mapping should follow the NASA report's viewing convention when the comparison case is finalised; the engineering distinction is already clear:

$$\text{canard-tail interdigitation} \implies \text{relative panel clocking,}$$

not a new aerodynamic-roll flight condition.

7.9.5. NASA Component Build-Up

The NASA report presents body-alone, body-tail, body-canard, and body-canard-tail configurations. SpearDAT's `input.settings.options.buildOutput=true` requests the historical component/cumulative build-up report for the geometry that is already present in the current request. It does not add or remove hardware and it does not run extra vehicle configurations automatically. The resulting build-up tables are written into the main human-readable `for006.dat` report; they are report content rather than additional rows in `result.aerodynamics`. The retained-report workflow in [listing 7.24](#) can be used when those tables need to be reviewed directly.

With C6 defined as set 1 and T2 as set 2, the build-up order within the complete request is therefore

$$BN3 \rightarrow BN3C6 \rightarrow BN3C6T2.$$

The separate NASA *BN3T2* hardware configuration has no C6 canards at all, so it requires a separate SpearDAT request in which T2 is the only fin set. Within that independent request T2 becomes set 1:

```
inputT2 = input;

% Remove the canard and make the tail the first physical set in this request.
t2only = t2;
t2only.set = 1;
inputT2.caseInput.finSets = t2only;

% The hinge definition must reference the same new set identity.
t2onlyDefl = t2Defl;
t2onlyDefl.set = 1;
inputT2.caseInput.deflections.finSets = t2onlyDefl;

inputT2.caseInput.output.printGeometry = ["Body", "FinSet1"];
inputT2.caseInput.output.printAero = ["Body", "FinSet1", "Synths"];
inputT2.caseInput.id = "NASA N3 T2 only";
```

```
resultT2 = speardat_mex(inputT2);
```

Listing 7.26: Forming the NASA body–tail-only configuration.

The two output selectors ask for detailed legacy-style report sections; they do not define additional aerodynamics. `printGeometry` requests the body and fin-set geometry reports. `printAero` requests the body aerodynamic report, the fin-set report, and the synthesised configuration report ("Synths"). Those requested sections are printed in `for006.dat`; the normal structured MATLAB aerodynamic table is still returned independently. If the detailed report is required after the call, use the retained-file pattern in [listing 7.24](#).

This illustrates an important point: a SpearDAT set number identifies a surface within the *current aerodynamic request*. It is not a permanent hardware serial number. T2 is NASA's configuration label for the chosen tail geometry; it does not require the tail to be set=2 when no forward fin set exists in that particular request.

7.9.6. NASA Canard-Deflection Series

For the C6 data, NASA varied the two side canards while keeping the top and bottom panels fixed. In the report's panel numbering, panels 1 and 3 are the vertical pair and panels 2 and 4 are the side pair. That maps naturally onto the SpearDAT cruciform order used above.

The report quotes equal positive magnitudes for its two side-panel deflections, but SpearDAT's raw panel convention is rolling-moment based. A physical 10-degree pitch-pair magnitude is therefore entered with opposite raw signs:

```
input10 = input;

input10.caseInput.deflections.finSets(1).panelDeflectionDeg = ...
    [0 10 0 -10];

result10 = speardat_mex(input10);
```

Listing 7.27: A 10-degree C6 pitch-pair magnitude in SpearDAT raw signs.

When reproducing wind-tunnel or flight-test control cases, always check the source dataset's panel numbering, viewing direction, and local deflection sign convention before translating signed angles into `panelDeflectionDeg`. A label such as "positive 10 degrees" is not sufficient by itself to establish the raw Missile DATCOM sign on every panel.

7.9.7. NASA C6 Trim Example

The experiment reported that the canards could trim the complete model to roughly 7 degrees angle of attack with about 10 degrees of canard deflection, with little further increase in trim angle when the deflection was raised to 15 degrees [3]. The same C6/T2 model can therefore be used to demonstrate a bounded SpearDAT trim request over the experimentally relevant positive range.

The code begins from the aligned zero-incidence configuration, removes the fixed C6 command, selects fin set 1 for trim, allows panels 2 and 4 to vary, and limits the scalar trim search to 0–15 degrees. Component build-up reporting is turned off for this request because the historical

BUILD report is not normally provided for trimmed cases. The detailed trim-related printed reports are then requested explicitly.

```
trimInput = input;

% BUILD is a component build-up report; disable it for the trim request.
trimInput.settings.options.buildOutput = false;

% Keep the C6 and T2 hinge definitions but return the fixed panel commands
% to the zero-incidence baseline before enabling trim.
trimInput.caseInput.deflections.finSets(1).panelDeflectionDeg = ...
    [0 0 0 0];

trimInput.caseInput.trim.enabled = true;
trimInput.caseInput.trim.finSet = 1;
trimInput.caseInput.trim.panels = [2 4];
trimInput.caseInput.trim.deflectionMinDeg = 0;
trimInput.caseInput.trim.deflectionMaxDeg = 15;

trimInput.caseInput.output.printAero = ...
    ["FinSet1", "FinSet2", "Synths", "Trim"];

trimResult = speardat_mex(trimInput);
```

Listing 7.28: A NASA-inspired C6 trim search.

The baseline input used above has `buildOutput=true`, and the assignment `trimInput=input` copies that setting into the trim request. Setting `trimInput.settings.options.buildOutput=false` removes only the BUILD report request; it does not remove either fin set and it does not disable trim. If it were left true, SpearDAT would still accept the trim request but would issue a warning because Missile DATCOM does not provide its normal component build-up output for trimmed cases. Disabling BUILD therefore avoids requesting report content that is not expected to be available for this case.

The `printAero` line controls detailed human-readable report content in `for006.dat`. "FinSet1" and "FinSet2" request the two fin-set aerodynamic sections, "Synths" requests the synthesised complete-configuration section, and "Trim" requests the trim report. These strings do not enable the trim calculation; that is done by `trim.enabled=true` and the accompanying trim fields. The report can be retained and read with the same workflow shown in [listing 7.24](#).

For similar validation work, interpret the result at the level supported by the analysis method and the source data. Check whether trim is found across the expected angle-of-attack range, whether the required deflection has the expected order of magnitude, and whether stability/control trends are consistent. A wind-tunnel result and a semi-empirical prediction need not agree at every point for the example to be useful.

7.10. Common Fin and Control Modelling Mistakes

The most common errors in this part of the input are conceptual rather than syntactic:

One fin-set element is created for every physical panel. One `finSets` element describes the repeated geometry for a complete set. Use `nPanels` and `panelRollDeg` to place the physical copies around the body.

Fin-set numbering does not follow the physical fore-to-aft order. Set identity participates in the cumulative aerodynamic synthesis. Number the defined surfaces consistently from the forward set toward the aft set.

A zero root span station is interpreted as a centreline-mounted root. `spanStations(1)=0` requests the historical automatic body- surface placement described in [section 7.2.1](#); it does not put the exposed root on the vehicle centreline.

Panel clocking is confused with aerodynamic roll. `panelRollDeg` installs a panel around the body. `flight.rollDeg` rotates the complete vehicle relative to the incidence plane. They represent different physical operations.

Leading-edge coordinates and sweep data contradict one another. A complete `xLeadingEdge` vector already defines the station geometry. If `sweepDeg` and `sweepStation` are also supplied, they must describe the same planform rather than a second one.

flapChordRatio is treated as a commanded angle. The field defines movable trailing-edge chord geometry. Panel incidence or trim is commanded separately.

Raw panel deflection is treated as a universal trailing-edge-down sign. The fixed-deflection convention is based on rolling-moment sense. Translate physical panel motions using the panel clocking before assigning raw signs.

hingeX is measured from the body nose in every case. It is measured from the geometry coordinate-system origin. The two origins coincide only when the body nose is at zero.

A reported trim deflection is treated as an exact nonlinear root. The historical trim method samples a finite deflection matrix and interpolates a zero crossing. A fixed-deflection rerun at the reported value need not produce machine-zero pitching moment.

A reliable workflow is therefore to define one set at a time, inspect its resolved geometry, confirm panel clocking, and only then add fixed deflections, trim, or a downstream interacting set.

7.11. MATLAB-to-Legacy Field Map

The MATLAB structure is the public interface used throughout this guide. The legacy names in [table 7.3](#) are provided as landmarks for readers comparing SpearDAT with the Missile DATCOM User Guide or with a retained `for005` input deck.

The exact field types, limits, defaults, omission behaviour, section-specific rules, and validation conditions are collected in [sections B.10 to B.12](#). The next engineering chapter builds on these same geometry and control capabilities to introduce the remaining optional and advanced analysis features.

Table 7.3.: Principal fin-set, deflection, and trim field mapping.

MATLAB field	Legacy field	Purpose
<code>finSets(i).set</code>	FINSET1–FINSET4	Identifies the lifting-surface set.
<code>finSets(i).spanStations</code>	SSPAN	Spanwise planform break points.
<code>finSets(i).chord</code>	CHORD	Local chord at each span station.
<code>finSets(i).xLeadingEdge</code>	XLE	Leading-edge x coordinate at each span station.
<code>finSets(i).sweepDeg</code>	SWEEP	Optional sweep-angle definition.
<code>finSets(i).sweepStation</code>	STA	Chord fraction at which sweep is measured.
<code>finSets(i).leadingEdgeRadius</code>	LER	Leading-edge radius schedule.
<code>finSets(i).nPanels</code>	NPANEL	Number of physical panels in the set.
<code>finSets(i).panelRollDeg</code>	PHIF	Circumferential panel clocking.
<code>finSets(i).dihedralDeg</code>	GAM	Panel-specific dihedral.
<code>finSets(i).flapChordRatio</code>	CFOC	Plain trailing-edge-device chord ratio.
<code>finSets(i).section.type</code>	SECTYP	Selects HEX, ARC, USER, or NACA section treatment.
<code>section.hexArc.zUpper/zLower</code>	ZUPPER/ZLOWER	HEX/ARC section ordinates or thickness proportions.
<code>section.hexArc.lMaxUpper/lMaxLower</code>	LMAXU/LMAXL	HEX maximum-thickness locations.
<code>section.hexArc.lFlatUpper/lFlatLower</code>	LFLATU/LFLATL	HEX constant-thickness chord fractions.
<code>section.user.xChord, mean, thickness</code>	XCORD, MEAN, THICK	USER mean-line/thickness section representation.
<code>section.user.xChord, yUpper, yLower</code>	XCORD, YUPPER, YLOWER	USER explicit upper/lower section representation.
<code>section.naca.sectionType/designation</code>	NACA control card	Supported NACA-family section designation.
<code>deflections.finSets(i).panelDeflectionDeg</code>	DELTA1–DELTA4	Raw panel deflections for the selected fin set.
<code>deflections.finSets(i).hingeX</code>	XHINGE	Longitudinal hinge-line coordinate.
<code>deflections.finSets(i).hingeSkewDeg</code>	SKEW	Hinge-line sweepback.
<code>trim.enabled</code>	TRIM	Enables the longitudinal trim calculation.
<code>trim.finSet</code>	SET	Fin set used for trim.
<code>trim.panels</code>	PANL1–PANL8	Panels varied by the trim calculation.
<code>trim.deflectionMinDeg</code>	DELMIN	Lower trim-search deflection bound.
<code>trim.deflectionMaxDeg</code>	DELMAX	Upper trim-search deflection bound.
<code>trim.asymmetryPanels</code>	ASYM	Panels whose trim deflection sense is reversed.

8. Optional and Advanced Features

The preceding chapters define the core aerodynamic problem: flight conditions, reference quantities, the body, lifting surfaces, fixed controls, and trim. Missile DATCOM also contains a number of specialised capabilities that are useful when a configuration needs more than that basic description. SpearDAT exposes the most important of these through the same typed MATLAB interface.

This chapter introduces six groups of optional features:

- inlet and diverter geometry through `input.caseInput.inlet`;
- non-lifting body protuberances through `input.caseInput.body.protuberances`;
- experimental or externally generated aerodynamic data through `input.caseInput.experimentalData`;
- dynamic damping derivatives through `input.caseInput.options.dynamicDerivatives`;
- spin/Magnus calculations through `input.caseInput.options.spinMagnus`; and
- auxiliary pressure-distribution output through `input.caseInput.output.pressures`.

In addition, [section 8.4](#) summarises the automatic configuration-synthesis and aerodynamic-interference methods used when the body and lifting surfaces are combined. These methods are part of the aerodynamic solution rather than separate public input options, so they are included for interpretation of the model and its capabilities.

Historically these capabilities correspond principally to \$INLET, \$PROTUB, \$EXPR, and the DAMP, SPIN, and PRESSURES control cards. They are not required in order for Missile DATCOM to assemble and solve an ordinary body–fin configuration. They request additional geometry models, data substitution, specialised derivative calculations, or additional report files.

The purpose here is to explain what those capabilities mean physically and how they fit into a normal SpearDAT workflow. The exact field-by-field contract, limits, defaults, and validation rules remain in [sections B.4.3, B.5, B.9.6, B.13 and B.14](#).

Important

Several advanced results are still principally human-readable or auxiliary legacy outputs. The combined MATLAB aerodynamic table is constructed from the machine-readable `for003.dat` output; detailed inlet reports, dynamic derivative tables, Magnus output, and pressure-distribution files may need to be inspected in `for006.dat` or `for009.dat–for012.dat`. Use the execution option `legacyFiles=true` when those files need to be retained after a MATLAB call.

8.1. Inlet Modelling

The inlet model represents an external inlet and its installation on the body. It is more than a drag-area input: Missile DATCOM derives an inlet geometry, evaluates aerodynamic force and moment increments, and can optionally estimate supersonic additive or spillage drag. The model is intended for the classical external-compression inlet geometries documented in the Missile DATCOM User Guide [1].

One SpearDAT inlet definition describes *one common geometry*. The placements array then places one or more identical copies of that geometry around the body. It does not define a different profile for each placement.

Table 8.1.: Inlet geometry families available through SpearDAT.

MATLAB value	Legacy type	Interpretation
"TwoDimensionalTop"	2DTOP	Top-mounted two-dimensional inlet using five height and width stations.
"TwoDimensionalSide"	2DSIDE	Side-mounted two-dimensional inlet using five height and width stations.
"Axisymmetric"	AXI	Axisymmetric inlet using five diameter stations.

8.1.1. Five-Station Geometry and Installation

The inlet profile is defined at five longitudinal stations measured locally from the inlet leading edge:

1. inlet leading edge or tip;
2. cowl-lip leading edge;
3. midbody start;
4. boattail start; and
5. boattail end.

`xInlet` places station 1 globally relative to the missile nose. The values in `profile.x` are then local coordinates measured aft from that inlet leading edge. Thus the global coordinate of profile station k is

$$x_{\text{global},k} = x_{\text{inlet}} + x_{\text{profile},k}.$$

The top-mounted nomenclature in [figure 8.1](#) shows the five stations together with the diverter and compression-ramp quantities. The same station meanings are used for the side-mounted and axisymmetric inlet families.

The diverter is located relative to the inlet itself: `xDiverter` is measured from the inlet leading edge, while `diverterHeight` sets the radial separation at the diverter leading edge. The sign of that height has a useful physical meaning:

Positive height represents a diverter-mounted inlet separated from the body mold line.

Zero height represents a conformal installation.

Negative height represents a semi-submerged installation. SpearDAT accepts this with a warning so that the unusual geometry is deliberate.

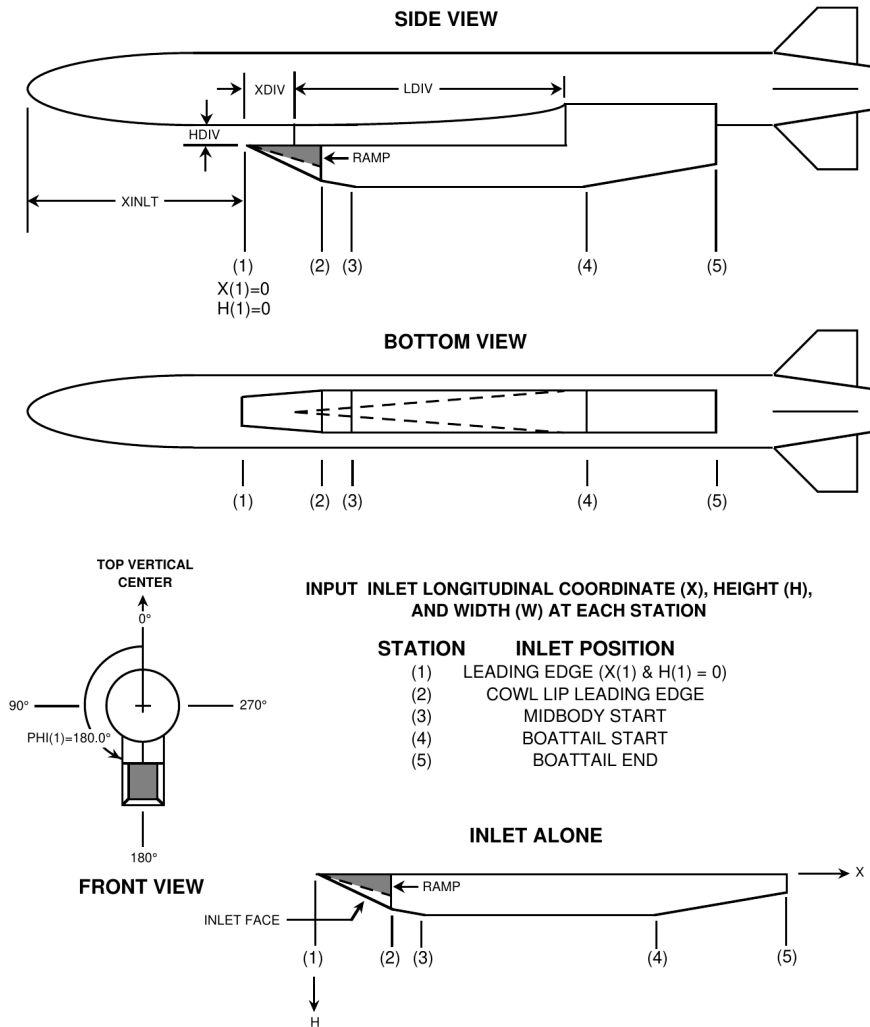


Figure 8.1.: Top-mounted two-dimensional inlet/diverter nomenclature. Reproduced from Figure 12 of the Missile DATCOM User Guide [1].

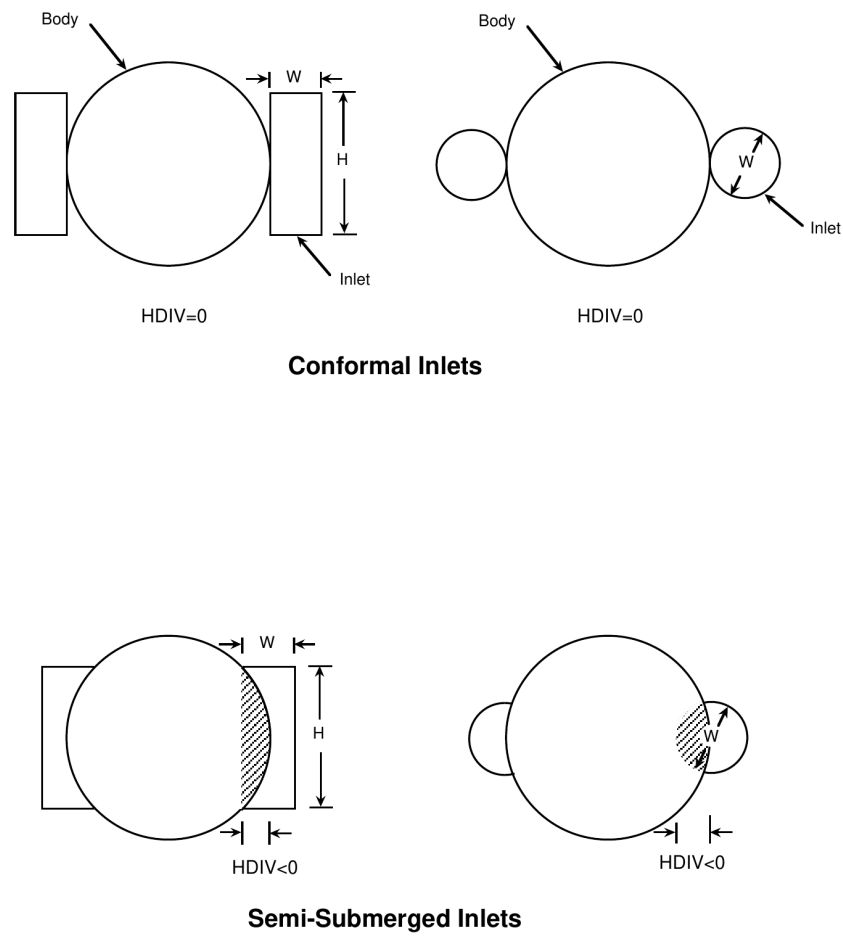


Figure 8.2.: Conformal and semi-submerged inlet definitions. Reproduced from Figure 15 of the Missile DATCOM User Guide [1].

The conformal and semi-submerged interpretations are illustrated directly in [figure 8.2](#).

The `rampAngleDeg` field describes the external-compression surface that turns the approaching flow before it reaches the inlet. For a two-dimensional inlet it is the compression-ramp angle; for an axisymmetric inlet the corresponding geometry is the external compression-cone half-angle. It is therefore an inlet-design quantity, not the vehicle angle of attack and not a circumferential placement angle. It is most directly relevant when the inlet is intended to operate as an external-compression inlet and the additive/spillage-drag calculation is requested. At a design condition the external compression system is ideally arranged so that the captured streamtube enters the inlet with little spillage; away from that condition, the same geometry can spill part of the approaching streamtube around the cowl.

The three SpearDAT inlet families use the same five station meanings but differ in the cross-section data supplied. The circular inlet shown in [figure 8.2](#) is the historical axisymmetric (AXI) family: in SpearDAT its five diameters are supplied through `profile.width` and no height vector is required.

A top-mounted two-dimensional example is:

```
inlet = struct;
inlet.enabled = true;
inlet.type = "TwoDimensionalTop";

inlet.xInlet = 55.0;
inlet.xDiverter = 1.0;
inlet.diverterHeight = 0.5;
inlet.diverterLength = 3.0;

inlet.placements(1).phiDeg = 0;

inlet.profile.x = [0 3 6 10 14];
inlet.profile.height = [0 1.8 2.4 2.2 1.5];
inlet.profile.width = [3.0 3.5 3.8 3.6 3.0];

inlet.covered = false;
inlet.rampAngleDeg = 5.0;
inlet.additiveDrag = false;

input.caseInput.inlet = inlet;
```

Listing 8.1: A top-mounted two-dimensional inlet.

A side-mounted two-dimensional inlet uses the same five local stations and requires both height and width at each station. The principal change is the inlet family and its circumferential placement:

```
inlet = struct;
inlet.enabled = true;
inlet.type = "TwoDimensionalSide";
inlet.xInlet = 55.0;
inlet.xDiverter = 1.0;
inlet.diverterHeight = 0.5;
inlet.diverterLength = 3.0;

% One copy on the right-hand side in the aft-looking-forward convention.
inlet.placements(1).phiDeg = 90;
```

```

inlet.profile.x = [0 3 6 10 14];
inlet.profile.height = [1.5 1.8 2.0 1.8 1.2];
inlet.profile.width = [0 3.0 3.5 3.2 2.4];

inlet.covered = false;
inlet.rampAngleDeg = 5.0;
inlet.additiveDrag = false;

input.caseInput.inlet = inlet;

```

Listing 8.2: A side-mounted two-dimensional inlet.

For an axisymmetric inlet, `profile.width` carries diameter rather than planar width. The height schedule is omitted:

```

inlet = struct;
inlet.enabled = true;
inlet.type = "Axisymmetric";
inlet.xInlet = 55.0;
inlet.xDiverter = 1.0;
inlet.diverterHeight = 0.0; % conformal installation
inlet.diverterLength = 3.0;

inlet.placements(1).phiDeg = 0;

inlet.profile.x = [0 3 6 10 14];
inlet.profile.width = [0 2.8 3.4 3.2 2.4]; % diameters

inlet.covered = false;
inlet.rampAngleDeg = 5.0; % compression-cone half-angle
inlet.additiveDrag = false;

input.caseInput.inlet = inlet;

```

Listing 8.3: An axisymmetric inlet.

The placement angle uses the same aft-looking-forward roll convention used for fin panels: zero degrees is top vertical centre and angles increase clockwise when viewed forward from behind the vehicle. Up to 20 entries in `placements` can be used to place identical copies of the same inlet geometry around the body. For example,

```

inlet.placements(1).phiDeg = 30;
inlet.placements(2).phiDeg = 330;

```

Listing 8.4: Two identical inlet copies at different roll positions.

Both copies share the same `xInlet`, diverter geometry, and five-station profile. This is an important distinction from defining two separate inlet geometries. The stock Missile DATCOM input model retains only one `$INLET` definition in a case; a later repeated `$INLET` namelist overwrites the earlier one rather than adding a second independent geometry. SpearDAT therefore exposes one inlet geometry with multiple identical circumferential placements, not a vector of distinct inlet profiles or different axial inlet stations.

For two-dimensional inlets the aft profile must actually contract through the boattail, so the station-5 area must be smaller than the station-4 area. For an axisymmetric inlet the equivalent condition is simply a smaller station-5 diameter. SpearDAT checks those conditions before execution.

8.1.2. Covered Inlets, Additive Drag, and Mass Flow

Setting `covered=true` models a plugged inlet with no internal flow. The historical geometry places the cover between the first two inlet profile stations, flush with the inlet face. This does not turn the inlet into a generic external blockage model; it selects the covered-inlet treatment and the additive-drag calculation is skipped. If the physical feature is better described as a lug, fairing, rail, or other non-lifting protrusion, the protuberance model in [section 8.2](#) is normally more appropriate.

For an open external-compression inlet, some of the freestream flow that would belong to the ideal capture streamtube may be unable to enter the inlet at an off-design condition. That flow is turned and spilled around the cowl. Missile DATCOM can estimate the associated additional axial-force contribution, historically called inlet *additive drag* or *spillage drag*.

The mass-flow ratio describes how much of that ideal capture-streamtube flow is actually admitted by the inlet. Conceptually,

$$\text{MFR} \approx \frac{\dot{m}_{\text{admitted}}}{\dot{m}_{\text{capture}}}.$$

A value approaching one therefore represents little spillage, while a smaller value represents a larger fraction of the capture streamtube being spilled. The current typed input accepts values strictly inside $0 < \text{MFR} < 1$. The additive-drag request is made explicitly:

```
inlet.additiveDrag = true;
inlet.rampAngleDeg = 5.0;

% One value for every active freestream speed condition.
inlet.massFlowRatio = [0.70 0.75 0.80];
```

Listing 8.5: Requesting inlet additive drag.

The schedule is positional. If the active flight-speed vector is

```
input.caseInput.flight.speed.mach = [0.8 1.3 1.75];
```

Listing 8.6: Freestream schedule corresponding to the inlet mass-flow schedule.

then the three mass-flow-ratio values in [listing 8.5](#) correspond to Mach 0.8, 1.3, and 1.75 in that order. They are not a second independent Mach grid and do not create a Cartesian product with the flight conditions. The historical inlet routine indexes MFR by the current Mach condition, so a per-speed vector also includes positions for lower-speed conditions at which additive drag will not ultimately be calculated. In the validated legacy path, additive drag is not calculated below Mach 1 and is calculated at zero angle of attack before the same increment is applied across the alpha table for that Mach condition.

Important

For the current SpearDAT release, use one explicit mass-flow-ratio value for every active Mach or velocity condition when `additiveDrag=true`. The request validator currently accepts a one-value MFR vector as shorthand, but the generated legacy deck serialises the supplied vector directly while the solver consumes MFR by speed index. Until scalar expansion is made explicit and full-stack validated, do not rely on a scalar MFR being broadcast across a multi-speed case.

Detailed inlet geometry and aerodynamic increments can be requested in the human-readable for006.dat report:

```
input.caseInput.output.printGeometry = ...
    ["Body", "Inlet"];
input.caseInput.output.printAero = ...
    ["Body", "Inlet", "Synths"];
```

Listing 8.7: Requesting detailed inlet reports.

8.2. Body Protuberances

Protuberances represent non-lifting external items attached to the body. The legacy model contains stock correlations for cylinders, blocks, streamlined fairings, launch lugs, and launch shoes. These are appropriate for features whose principal aerodynamic effect is a drag or local body-interference increment rather than the lifting-surface behaviour modelled by a fin set.

The available historical shapes are shown in [figure 8.3](#).

Table 8.2.: Protuberance types exposed by SpearDAT.

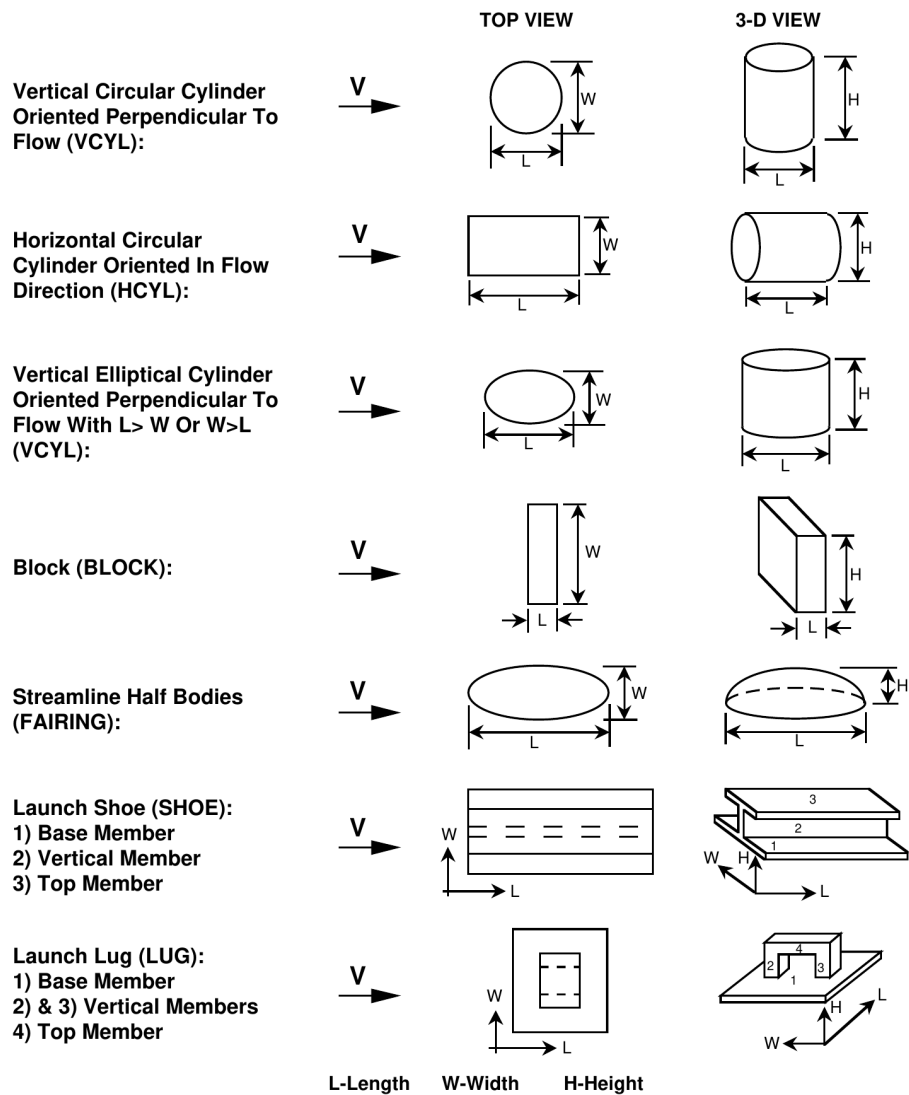
MATLAB value	Members	Typical interpretation
"VerticalCylinder"	1	Cylinder perpendicular to the flow.
"HorizontalCylinder"	1	Cylinder aligned with the flow.
"Block"	1	Rectangular external block.
"Fairing"	1	Streamlined half-body fairing.
"Lug"	4	Launch-lug approximation assembled from four members.
"Shoe"	3	Launch-shoe approximation assembled from three members.

Each entry in `items` defines one protuberance set at one longitudinal station. The item-level coordinate `x` is the approximate geometric-centroid location of the complete item, measured from the missile nose; it is not the forward edge of the member. Member `verticalOffset` is measured outward from the local body mold line to the bottom of the member. At most 20 protuberance items may be defined in one case. The member arrays have a separate legacy capacity, but with the currently supported stock types the 20-item limit is normally the controlling public limit.

The first four stock types use one member structure. The following compact snippets show the required form for each type. The dimensions are illustrative only.

```
member = struct;
member.length = 1.5;
member.width = 1.0;
member.height = 1.0;
member.verticalOffset = 0;

item = struct;
item.type = "VerticalCylinder";
item.x = 70.0;
item.countAroundBody = 2;
```



Note; Length, width, height, and offset must be input for each member of launch lug and launch shoe types

Note; Offset is the perpendicular distance from the missile mold line to the bottom of the protuberance or protuberance member

Figure 8.3.: Protuberance shapes available to the Missile DATCOM model. Reproduced from Figure 4 of the Missile DATCOM User Guide [1].


```
item.members = member;
```

Listing 8.8: Vertical-cylinder protuberance.

```
member = struct;
member.length = 3.0;
member.width = 0.8;
member.height = 0.8;
member.verticalOffset = 0;

item = struct;
item.type = "HorizontalCylinder";
item.x = 70.0;
item.countAroundBody = 2;
item.members = member;
```

Listing 8.9: Horizontal-cylinder protuberance.

```
member = struct;
member.length = 2.0;
member.width = 1.0;
member.height = 0.5;
member.verticalOffset = 0;

item = struct;
item.type = "Block";
item.x = 70.0;
item.countAroundBody = 4;
item.members = member;
```

Listing 8.10: Block protuberance.

```
member = struct;
member.length = 5.0;
member.width = 1.5;
member.height = 0.75;
member.verticalOffset = 0; % fairing is referenced to the mold line

item = struct;
item.type = "Fairing";
item.x = 70.0;
item.countAroundBody = 1;
item.members = member;
```

Listing 8.11: Fairing protuberance.

The value `countAroundBody` is the legacy NLOC multiplicity. It tells the protuberance correlation how many identical copies of the *complete item* exist around the body at the same axial station. Thus `countAroundBody=4` represents four copies of that item, not four member structures.

A subtle limitation is that Missile DATCOM does not ask for the individual circumferential angles of those copies. There is no protuberance PHI schedule analogous to a fin set, and the stock protuberance method calculates an axial-force contribution rather than resolving asymmetric lateral force and moment from each azimuth. Consequently, `countAroundBody=4` should not be interpreted as an explicit command to place protuberances at 0° , 90° , 180° , 270° .

A real vehicle may of course use such an evenly spaced arrangement, but the four azimuths are not represented individually by this model.

The composite "Lug" and "Shoe" types use several member structures to build one stock item. A lug requires exactly four members:

```
lugMember = struct;
lugMember.length = 1.2;
lugMember.width = 0.6;
lugMember.height = 0.4;
lugMember.verticalOffset = 0;

lugMembers = repmat(lugMember, 1, 4);
lugMembers(2).length = 2.0;
lugMembers(3).height = 0.6;
lugMembers(4).length = 1.0;

item = struct;
item.type = "Lug";
item.x = 70.0;
item.countAroundBody = 2;
item.members = lugMembers;
```

Listing 8.12: Four-member launch-lug approximation.

A launch shoe requires exactly three members:

```
shoeMember = struct;
shoeMember.length = 2.0;
shoeMember.width = 1.0;
shoeMember.height = 0.5;
shoeMember.verticalOffset = 0;

shoeMembers = repmat(shoeMember, 1, 3);
shoeMembers(2).length = 3.0;
shoeMembers(3).height = 0.7;

item = struct;
item.type = "Shoe";
item.x = 70.0;
item.countAroundBody = 2;
item.members = shoeMembers;
```

Listing 8.13: Three-member launch-shoe approximation.

To activate any of the preceding examples, assign the item—or a structure vector of several different items—to the body branch:

```
input.caseInput.body.protruberances.enabled = true;
input.caseInput.body.protruberances.items = item;
% For several sets, use for example: items = [item1 item2 item3];
```

Listing 8.14: Activating one or more protruberance items.

The multiple member structures in a lug or shoe describe the constituent geometry of one composite item; they are not separate circumferential locations. The entire composite item can then be repeated with countAroundBody.

Note

Choose between a fin set and a protuberance according to the aerodynamic role, not just the CAD appearance. A strake-like feature expected to generate important normal force or pitching/yawing moment should normally be represented as a lifting surface when the fin methods are appropriate. The stock protuberance model is best suited to non-lifting external items such as lugs, blocks, rails, and drag-producing protrusions.

8.3. Experimental-Data Substitution

The `experimentalData` branch allows aerodynamic coefficients from wind-tunnel tests, flight tests, CFD, another prediction method, or a reduced engineering database to participate in the Missile DATCOM configuration build-up. This is the public SpearDAT representation of the historical \$EXPR capability.

It is useful to think of EXPR as an *experimental substitution and synthesis* mechanism rather than a table that is simply pasted onto the final answer. Missile DATCOM still evaluates the theoretical vehicle. Before a supplied dataset is inserted, the experimental-data path can transform its reference convention to the active case and interpolate the supplied coefficient channels in angle of attack. At the selected configuration stage, the populated channels then become the data source for that stage. Channels that were not supplied remain theoretical, and later configuration synthesis can still combine that partially experimental stage with theoretical components defined farther downstream.

For example, if reliable measurements exist for a body plus fin sets 1 and 2, a "BodyFinSets12" dataset can supply that cumulative stage. If fin set 3 is then part of the model but no experimental body-plus-three-fin-set data exist, the later synthesis can use the experimental body-plus-sets-1-2 stage together with the theoretical contribution/interference treatment for fin set 3. This is why the configuration level attached to an EXPR dataset is important: it tells the solver *where in the cumulative build-up* the measured information is authoritative.

One dataset is a coefficient-versus-angle-of-attack table for one selected configuration level and, when stated explicitly, one Mach number. Separate datasets are used for separate Mach numbers or configuration levels. A dataset may contain all six static coefficient channels or only the channels that are trusted and available.

For ordinary direct substitution, leave configuration incrementing disabled:

```
input.caseInput.experimentalData.incrementMode = false;
```

Listing 8.15: Direct experimental-data substitution mode.

Here "direct" refers to the selected configuration stage: the supplied channels replace the corresponding theoretical channels at that stage after any required reference conversion and alpha interpolation. It does not mean that the solver stops performing theoretical aerodynamics for the rest of the vehicle. Omitted channels remain on the theoretical path; they are not materialised as zeros.

For example, the following deliberately synthetic dataset replaces normal force, pitching moment, and axial force for a complete body-plus-two-fin-set configuration at Mach 1.3:

```

synthetic = struct;
synthetic.mach = 1.3;
synthetic.configuration = "BodyFinSets12";
synthetic.alphaDeg = [0 4 8 12 16 20 24];

% Illustrative values only -- not taken from the NASA report.
synthetic.coefficients.cn = ...
    [0.00 0.42 0.88 1.38 1.92 2.48 3.05];
synthetic.coefficients.cm = ...
    [0.01 -0.04 -0.12 -0.23 -0.36 -0.52 -0.70];
synthetic.coefficients.ca = ...
    [0.10 0.11 0.13 0.16 0.20 0.25 0.31];

input.caseInput.experimentalData.incrementMode = false;
input.caseInput.experimentalData.datasets = synthetic;

```

Listing 8.16: Illustrative synthetic experimental-data substitution.

The coefficient arrays correspond positionally to `synthetic.alphaDeg`. At $\alpha = 12^\circ$, for example, the fourth entries are used. The assignment `experimentalData.datasets=synthetic` simply places this scalar MATLAB structure into the experimental-dataset collection, so this request contains one EXPR dataset. For several Mach numbers or configuration levels, build several structures and assign a structure vector such as `[dataset1, dataset2]`.

8.3.1. Dataset Reference Quantities

Experimental coefficients are nondimensional values and are meaningful only with the reference quantities under which they were reduced. The active SpearDAT vehicle may use a different reference area, reference length, lateral reference length, or moment reference location from the test database. In that case, describe the *original experimental convention* on the dataset:

```

% Active design/reference convention used by the SpearDAT case.
input.caseInput.reference.sref = 50.0;
input.caseInput.reference.lref = 8.0;
input.caseInput.reference.latref = 8.0;
input.caseInput.reference.xcg = 40.0;
input.caseInput.reference.zcg = 0.0;

% Convention under which the supplied experimental coefficients were reduced.
synthetic.reference.sref = 25.0;
synthetic.reference.lref = 5.0;
synthetic.reference.latref = 5.0;
synthetic.reference.xcg = 32.0;
synthetic.reference.zcg = 0.0;

```

Listing 8.17: Supplying the reference convention of an experimental dataset.

In this example, `synthetic.reference.sref=25` does not tell the solver to analyse a 25-unit-area version of the current vehicle. It records that the supplied nondimensional coefficients were formed using that reference area. SpearDAT converts the supplied force and moment coefficients to the active 50-unit-area convention before insertion; the reference lengths are used in the corresponding moment-coefficient rescaling, and a different `xcg/zcg` allows the moments to be shifted to the active moment-reference location. The same physical experimental

loads can therefore be compared or synthesised under the active design convention without pretending that the original coefficients used the active references.

If the experimental data were already reduced with exactly the active case references, the dataset-specific fields can be omitted and the active values are inherited. Do not overwrite the dataset reference fields with the desired new design references merely to force agreement; they should describe the references that were actually used to form the supplied data.

8.3.2. Configuration Selection in the Cumulative Build-Up

Configuration selectors mirror Missile DATCOM's cumulative synthesis order. Thus "BodyFinSet1" means body plus fin set 1, "BodyFinSets12" means body plus every fin set from 1 through 2, and "BodyFinSets123" means body plus every fin set from 1 through 3. They are not arbitrary set-membership lists. There is consequently no composite selector for body plus fin sets 1 and 3 while deliberately omitting set 2.

For example, suppose wind-tunnel or CFD data represent the complete body with fin sets 1 and 2 installed together. Attach those data to the cumulative "BodyFinSets12" stage:

```
dataset = struct;
dataset.mach = 1.30;
dataset.configuration = "BodyFinSets12";
dataset.alphaDeg = [0 4 8 12];

dataset.coefficients.cn = [0.00 0.40 0.85 1.35];
dataset.coefficients.cm = [0.02 -0.06 -0.18 -0.34];

input.caseInput.experimentalData.incrementMode = false;
input.caseInput.experimentalData.datasets = dataset;
```

Listing 8.18: Attaching experimental data to a cumulative body–fin configuration.

This does *not* tell the solver that separate experimental datasets exist for fin set 1 and fin set 2. It says that the supplied coefficients describe the already assembled body-plus-fin-sets-1–2 configuration as one cumulative stage. The solver therefore does not decompose those measurements back into an experimental body and two independent experimental fin contributions. If the vehicle also contains a later fin set 3, the normal theoretical build-up can continue from this supplied cumulative baseline where the EXPR synthesis rules permit that hybrid combination.

Conversely, a dataset for "FinSet3" alone is appropriate when isolated fin-set data are what the user actually has. Choose the configuration name that corresponds to the physical configuration represented by the data.

8.3.3. Historical Configuration Incrementing

The legacy INCRMT workflow is different from direct substitution. It uses a first, calibrated case to establish experimental-minus-theoretical corrections and then applies those stored corrections to later configurations within the *same stacked solver run*. The historical arrangement is shown in [figure 8.4](#).

The current public MATLAB call accepts one caseInput at a time and each call starts an independent isolated solver execution. Consequently, three separate speardat_mex calls cannot

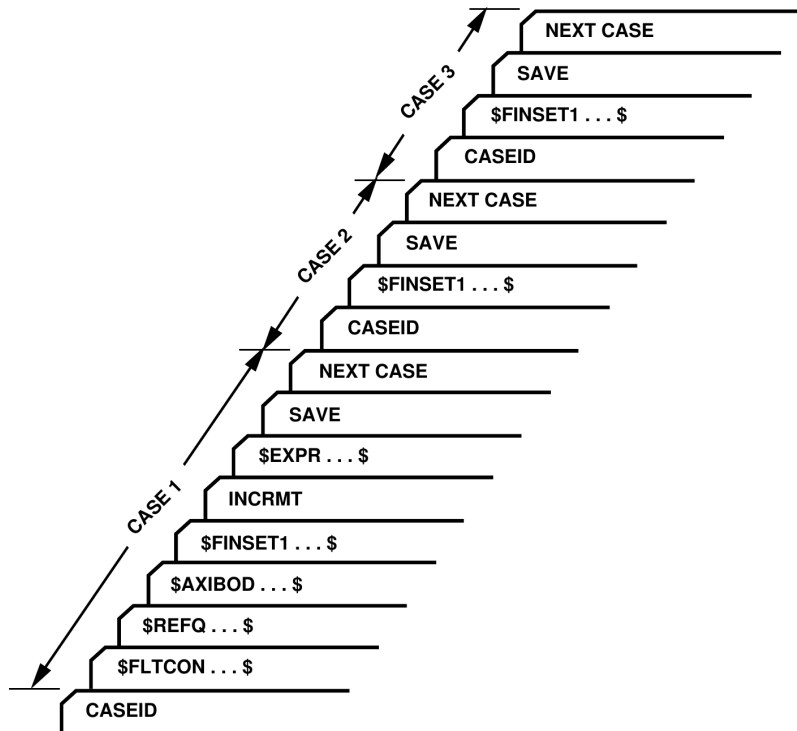


Figure 8.4.: Historical configuration-incrementing case sequence. Reproduced from Figure 17 of the Missile DATCOM User Guide [1].

reproduce the stacked case sequence in [figure 8.4](#); the calibration state would not persist from one process to the next. For ordinary MATLAB analysis, use `incrementMode=false` and the direct-substitution or hybrid build-up workflow. The incrementing flag is retained for advanced compatibility workflows where a complete stacked legacy sequence is deliberately managed.

8.4. Configuration Synthesis and Aerodynamic Interference

A complete body–fin configuration is not obtained by simply adding isolated body and fin coefficients. The presence of one component changes the local flow or effective loading seen by another: the body modifies the flow around an attached fin, neighbouring panels can influence one another, and vortical flow from an upstream component can alter the incidence seen by a downstream surface. Missile DATCOM represents these effects through a condition-dependent configuration synthesis using analytical and semi-empirical interference methods.

The configuration-synthesis methodology and its original method-selection rationale are described in Volume I of the Missile DATCOM Final Report, including the equivalent-angle-of-attack method, carryover interference, and body- and lifting-surface-vortex treatments [2]. The later Missile DATCOM User’s Manual documents the resulting synthesis capability in the operational 1997 revision [1].

Note

This section is provided for information only. SpearDAT has no MATLAB field for selecting an interference method or supplying interference factors. Once the body, fin geometry, panel orientation and deflection, and flight condition have been defined, the

applicable synthesis and interference calculations are performed automatically by the aerodynamic model.

A useful way to interpret the synthesis is through the *equivalent angle of attack* of each panel. The freestream angle of attack is not necessarily the local flow angle experienced by every lifting surface. Body-induced flow, nearby-panel effects, and vortices can increase or decrease the effective local incidence. Missile DATCOM estimates that local incidence, evaluates the panel aerodynamics at the resulting equivalent angle, and then projects the panel loads into the complete vehicle forces and moments.

The principal interference effects represented in this build-up are:

Body–fin carryover. The body changes the flow and effective loading of an attached fin, while the fin loading also contributes to the combined body–fin response. Geometry- and Mach-dependent carryover factors modify these contributions and their effective centres of pressure. The carryover treatment is also adjusted with incidence, and elliptical bodies receive an orientation-dependent correction relative to an equivalent circular body.

Panel/fin interference. Panels in the same fin set are not always aerodynamically independent, particularly when controls are deflected. Adjacent and opposing panels contribute to the equivalent panel incidence. In supersonic flow, the treatment also accounts for whether the relevant panel regions lie within the applicable Mach-cone influence.

Body-vortex interference. At incidence, separated vortical flow generated by the body nose can change the local flow seen by a fin. The model estimates vortex separation, position, and strength from the body geometry, incidence, and Mach number, and converts the induced influence at the panel into an equivalent-angle correction.

Upstream lifting-surface vortices. A forward fin set can shed vortical flow that changes the loading of a later fin set. The model estimates the vortex location and strength from the forward surface and its loading, tracks the vortex aft to the downstream surface, and integrates its influence over the aft panel to obtain an additional equivalent-angle correction.

The synthesis is cumulative from the forward configuration to the aft one. At each operating condition, the solver first establishes the body and fin characteristics, applies the body–fin and local panel corrections, and then adds each fin set in sequence. When an upstream lifting surface can influence a later set, its vortex effect is evaluated before the downstream panel loads are formed. A reported body-plus-fin-sets-1–2 configuration is therefore not, in general, the arithmetic sum of the isolated body, fin-set-1, and fin-set-2 coefficients; it contains the interference corrections used by the synthesis.

These models are static preliminary-design corrections evaluated independently at each operating condition. They are not a time-accurate wake propagation, free-wake calculation, or CFD solution. Their purpose is to capture the principal body–surface and surface–surface interactions within the engineering methodology used by Missile DATCOM.

8.5. Dynamic Derivatives and Spin/Magnus Calculations

The static aerodynamic coefficients described in the preceding chapters apply to a vehicle at a fixed flight condition. A dynamic model also needs the first-order change in those forces and moments when the vehicle has a small pitch, yaw, or roll rate, or when its angle of attack is

changing. The historical DAMP option provides these dynamic aerodynamic derivatives, and SpearDAT requests the calculation with

```
input.caseInput.options.dynamicDerivatives = true;
```

Listing 8.19: Requesting dynamic aerodynamic derivatives.

The calculation is performed about each nominal flight condition rather than by integrating a rotational motion through time. A body rate changes the local relative flow seen by parts of the vehicle away from the rotation centre. For example, during pitching motion an aft fin experiences a slightly different local flow direction from the one it sees in the corresponding static case. Equivalent effects occur during yawing and rolling motion. The solver combines body damping methods, fin-rate relations, and body–fin aerodynamic synthesis to determine how the aerodynamic coefficients respond to these small motions. The changes are then expressed as local derivatives about the nominal state.

The rate variables used by these derivatives are non-dimensionalised by the vehicle speed and the appropriate reference length. A physical schedule of p , q , or r is therefore not supplied to dynamicDerivatives; the option requests aerodynamic sensitivities, not a time-domain rotational simulation. The selected derivative reporting system is described in [section 8.5.1](#).

The principal quantities reported by this path include pitch-rate derivatives such as CNQ, CMQ, and CAQ; yaw-rate derivatives such as CYR, CLNR, and CLLR; roll-rate derivatives such as CYP, CLNP, and CLLP; and the angle-of-attack-rate derivatives CNAD and CMAD. Depending on the requested partial/build reporting, the solver can report body-alone and cumulative body–fin versions of these quantities.

The detailed dynamic-derivative tables are written principally to for006.dat, rather than to the ordinary combined MATLAB aerodynamic table. If those tables are required after the MATLAB call finishes, retain the legacy output bundle with legacyFiles=true; otherwise the solver's temporary numbered files are not kept. The retained-output workflow is shown explicitly in [listing 8.27](#).

8.5.1. Derivative Reporting System

input.settings.derivativeSystem controls the legacy DERIV reporting convention for aerodynamic derivatives. The two public values are:

```
input.settings.derivativeSystem = "Degrees"; % default: DERIV DEG
% input.settings.derivativeSystem = "Radians"; % DERIV RAD
```

Listing 8.20: Selecting the derivative reporting convention.

"Degrees" reports applicable angular derivatives on a per-degree basis, while "Radians" selects the per-radian reporting path. The choice changes the derivative scaling/reporting convention; it does not change the geometry or impose a physical degrees-per-second or radians-per-second body rate. Record this setting when comparing derivative tables from different analyses.

8.5.2. Spin and Magnus Effects

Spin is sustained rotation of a vehicle about its longitudinal body axis, as occurs in a spin-stabilised projectile. In flight-dynamics notation, this longitudinal spin is the body-axis roll rate p ; the term *spin* is used when that roll motion is sustained. When a spinning body flies at non-zero incidence, the interaction between the rotation and the crossflow can produce additional transverse aerodynamic loading. The resulting force and moment effects are commonly described as the *Magnus effect*.

SpearDAT requests the specialised body spin/Magnus calculation with `spinMagnus=true`. For a body-plus-fin vehicle, partial reporting can also be requested so that the body-alone correlation results are retained in the detailed report:

```
input.caseInput.options.spinMagnus = true;
input.caseInput.output.partialAll = true;
```

Listing 8.21: Requesting body spin/Magnus output.

The calculation is an empirical body correlation. The resolved geometry is reduced to quantities such as nose length, total body length, afterbody length, body diameter, centre-of-gravity position, and a measure of nose curvature. These geometry ratios are combined with Mach-dependent correlation data, which are interpolated at the requested Mach number and then scaled to the selected reference quantities. The resulting outputs describe the body's roll-damping response, a transverse side-force response associated with spin, and the incidence-dependent Magnus yawing-moment response.

The incidence dependence of the Magnus yawing-moment derivative is represented by

$$\frac{C_{NP}}{\sin \alpha} = C_{NP1} + C_{NP3} \sin^2 \alpha + C_{NP5} \sin^4 \alpha.$$

The spin/Magnus quantities are reported in `for006.dat`, in its Magnus derivatives section. In this historical notation, C_{NP} denotes the yawing-moment derivative due to longitudinal spin—equivalent to C_{n_p} in conventional derivative notation—and must not be confused with the static normal-force coefficient C_N . For a selected angle of attack, evaluate the right-hand side of the expression above and multiply it by $\sin \alpha$ to obtain $C_{NP}(\alpha)$. The three reported values C_{NP1} , C_{NP3} , and C_{NP5} therefore define one incidence-dependent yawing-moment derivative, not three separate moments. C_{NP5} is a five-degree secant/check value associated with the same relation; it is not an additional moment component. The correlation includes a warning against extrapolating the polynomial past its intended incidence range, particularly beyond its first positive extremum.

The same report section also presents `CLLP` and `CYP`. `CLLP` is the specialised body spin roll-damping derivative and `CYP` is the corresponding spin-related side-force derivative. These names also appear in the ordinary dynamic-derivative output, but the two sets of values are not interchangeable. Missile DATCOM evaluates them through different calculation paths: the spin/Magnus values come from the empirical body-alone spin correlation, whereas the ordinary dynamic-derivative values come from the general dynamic-derivative calculation and can be reported for the body alone or for cumulative body–fin configurations. The identically named values therefore need not be numerically equal, even when both refer to roll-rate effects; for a complete body–fin configuration the ordinary derivative also contains the configuration response that is absent from the body-only spin correlation. SpearDAT stores the specialised

spin/Magnus quantities separately from the ordinary dynamic-derivative arrays to preserve this distinction.

To apply these quantities in a flight-dynamics model, define the reduced spin rate

$$\hat{p} = \frac{p L_{\text{atref}}}{2V},$$

where p is the physical body-axis roll/spin rate and V is flight speed. The expressions below are most directly used with the per-radian derivative reporting convention; if per-degree reporting is selected, the derivative and rate units must be converted consistently before they are combined.

The specialised body roll-damping contribution is

$$\Delta C_{LL,\text{spin}} = C_{LLP,\text{spin}} \hat{p},$$

and the spin-related side-force contribution is

$$\Delta C_{Y,\text{spin}} = C_{YP,\text{spin}} \hat{p}.$$

For the usual $\beta = 0$ condition, incidence lies in the pitch plane and the Magnus moment acts in yaw. The corresponding yawing-moment coefficient increment is

$$\Delta C_{LN,\text{Magnus}} = C_{NP}(\alpha) \hat{p}.$$

There is therefore no separate pitching Magnus moment reported for this standard condition. A more general three-dimensional flight-dynamics model must orient the transverse Magnus response consistently with the vehicle's incidence direction.

With dynamic pressure

$$\bar{q} = \frac{1}{2} \rho V^2,$$

the corresponding dimensional loads are

$$\Delta Y_{\text{spin}} = \bar{q} S_{\text{ref}} \Delta C_{Y,\text{spin}},$$

$$\Delta L_{\text{spin}} = \bar{q} S_{\text{ref}} L_{\text{atref}} \Delta C_{LL,\text{spin}}, \quad \Delta N_{\text{Magnus}} = \bar{q} S_{\text{ref}} L_{\text{atref}} \Delta C_{LN,\text{Magnus}}.$$

These relations distinguish the reported spin derivatives from the actual flight-dynamics loads: the latter also depend on the instantaneous roll/spin rate, speed, dynamic pressure, angle of attack, and reference quantities.

The spin/Magnus correlation is a body-alone result even when fins are present in the analysed vehicle. `output.partialAll=true` is therefore useful when the body contribution is to be inspected explicitly. The detailed spin/Magnus quantities are reported in `for006.dat`; [listing 8.27](#) shows how to retain that file after the MATLAB call. Documented compatibility corrections affecting the historical spin/Magnus implementation are collected in [appendix A](#).

8.6. Pressure-Distribution Outputs

Pressure-distribution output involves two separate choices that should not be confused:

Body aerodynamic method is requested through `input.caseInput.theory.bodyMethod`; the public values are "VanDykeHybrid", "Sose", and "Hyper".

Pressure-output request is selected independently through `input.caseInput.output.pressures=true`.

Thus `pressures=true` means "write the applicable auxiliary pressure information." It does not by itself mean "use SOSE." Conversely, selecting "Sose" changes the requested body aerodynamic method even if no pressure file is requested. These methods affect the body coefficient calculation as well as any pressure distributions generated from the same method; they are not merely output-format options.

8.6.1. Choosing a Body Method

The three public method selections have different intended regimes:

Table 8.3.: Practical interpretation of the body-method selections.

MATLAB value	Method	Practical use
"VanDykeHybrid"	Van Dyke hybrid; legacy default	A sensible starting point for ordinary supersonic body analysis when the method is valid for the geometry and Mach number. Omitting <code>bodyMethod</code> requests the same default path.
"Sose"	Second-Order Shock Expansion (SOSE)	Explicit shock-expansion treatment for axisymmetric supersonic bodies. It is useful when a controlled SOSE comparison or the shock-expansion pressure path is the intended method, and it is also the method reached by some coordinate-defined body paths.
"Hyper"	Newtonian/hypersonic body method	A Newtonian-style high-Mach method intended for hypersonic applications. The legacy User Guide states that HYPER should normally be selected at Mach numbers greater than about 6.

There is therefore no single method that should be selected solely because a pressure file is desired. For a routine supersonic case, start from the default hybrid method unless the geometry, method-validity rules, or a validation plan calls for another method. Use SOSE when the second-order shock-expansion model is the desired controlled assumption. Use HYPER when the analysis is genuinely in the hypersonic/Newtonian regime rather than merely supersonic. When pressure or total coefficients are method-sensitive, comparing methods can

be useful, but the comparison should be treated as a modelling study rather than as three equivalent ways to print the same solution.

Note

"VanDykeHybrid" is a requested/default method, not an unconditional guarantee that the resolved solver method will be Hybrid. Missile DATCOM can resolve the body calculation to SOSE when the Hybrid method is not applicable; coordinate-defined axisymmetric geometry is one important path that can force SOSE. SpearDAT preserves the resolved method in the case state. Consequently, seeing "Sose" in resolved output after requesting or omitting "VanDykeHybrid" can be intended method substitution rather than an input error.

For a deliberately explicit supersonic pressure case:

```
input.caseInput.theory.bodyMethod = "Sose";
input.caseInput.output.pressures = true;
```

Listing 8.22: Selecting SOSE and requesting pressure output.

The principal auxiliary files are summarised in [table 8.4](#).

Table 8.4.: Principal pressure-related auxiliary files.

File	Typical content
for009.dat	Body contour geometry used by applicable pressure/body-method paths.
for010.dat	Body surface pressure-coefficient distributions at incidence.
for011.dat	Fin pressure-coefficient distributions.
for012.dat	Body pressure coefficient and local Mach number at zero incidence.

Not every file is populated for every geometry, Mach number, or method. The legacy User Guide limits body pressure-distribution output to supersonic conditions above Mach 1.2 and fin pressure output to Mach numbers above 1.05. For the Hybrid and SOSE body-pressure methods, body pressures are reported only through angles of attack up to 15 degrees; the legacy fin-pressure file is a zero-angle-of-attack section-pressure output. These are method-specific auxiliary products rather than a promise that four complete pressure datasets exist for every row of the ordinary aerodynamic solution.

8.6.2. Reading the Body and Fin Pressure Files

The legacy body-pressure format is illustrated in [figure 8.5](#).

In for010.dat, each ZONE identifies one Mach number and angle of attack. The first column, X/D, is the axial body location normalised by body diameter. The remaining columns are local surface pressure coefficients at circumferential angles around the body: CP(0), CP(30), CP(60), and so on through CP(180). Only one half-circumference is tabulated because the axisymmetric-body pressure field can be represented by symmetry. At zero angle of attack, a symmetric axisymmetric case should give the same pressure coefficient at all of those circumferential angles for a given X/D; at non-zero incidence, the circumferential columns separate as windward and leeward pressure varies.

```
VARIABLES=X/D,CP(0),CP(30),CP(60),CP(90),CP(120),CP(150),CP(180)
ZONE T="BODY CP AT MACH= 2.36 ALPHA= 0.00"
0.000000 0.28444 0.28444 0.28444 0.28444 0.28444 0.28444 0.28444
0.050000 0.27667 0.27667 0.27667 0.27667 0.27667 0.27667 0.27667
0.100000 0.26856 0.26856 0.26856 0.26856 0.26856 0.26856 0.26856
0.150000 0.26066 0.26066 0.26066 0.26066 0.26066 0.26066 0.26066
0.200000 0.25279 0.25279 0.25279 0.25279 0.25279 0.25279 0.25279
0.250000 0.24500 0.24500 0.24500 0.24500 0.24500 0.24500 0.24500
0.300000 0.23728 0.23728 0.23728 0.23728 0.23728 0.23728 0.23728
0.350000 0.22963 0.22963 0.22963 0.22963 0.22963 0.22963 0.22963
0.400000 0.22206 0.22206 0.22206 0.22206 0.22206 0.22206 0.22206
0.450000 0.21456 0.21456 0.21456 0.21456 0.21456 0.21456 0.21456
0.500000 0.20742 0.20742 0.20742 0.20742 0.20742 0.20742 0.20742
```

Figure 8.5.: Example for010.dat body pressure-distribution format. Reproduced from Figure 42 of the Missile DATCOM User Guide [1].

The fin pressure file, for011.dat, uses a different local coordinate system. Its columns are $Y/(B/2)$, X/C , and CP: $Y/(B/2)$ is a normalised spanwise location, X/C is the chordwise coordinate normalised by local chord, and CP is the local fin-section pressure coefficient. The file is organised into zones for the applicable fin set and Mach condition.

Because the current structured MATLAB output does not broadly parse for009.dat–for012.dat, retain the legacy bundle when those files are part of the analysis objective:

```
options = struct;
options.legacyFiles = true;
options.outputDirectory = fullfile(pwd, "speardat_pressure_case");

result = speardat_mex(input, options);

caseDirectory = string(result.cases(1).legacyDirectory);
for010Text = fileread(fullfile(caseDirectory, "for010.dat"));
disp(for010Text)
```

Listing 8.23: Retaining pressure and detailed report files.

The same retained directory contains for006.dat for detailed human- readable reports and the remaining numbered output files for that scalar case.

8.7. Worked Advanced Example: NASA N3–C6–T2 Vehicle

This worked example reuses the wind-tunnel missile family introduced in the preceding chapters so that the advanced inputs are attached to a vehicle the reader has already seen rather than to an unrelated synthetic missile. NASA TM X-73,218 is a 1977 static-aerodynamic investigation of a canard-controlled missile family tested with interchangeable noses, canards, and tails [3]. In the configuration names used throughout this guide:

- **N3** is the blunted member of the three-calibre tangent-ogive nose family used for the reference body;
- **C6** is the selected canard geometry and is represented as SpearDAT fin set 1; and
- **T2** is the selected tail geometry and is represented as SpearDAT fin set 2.

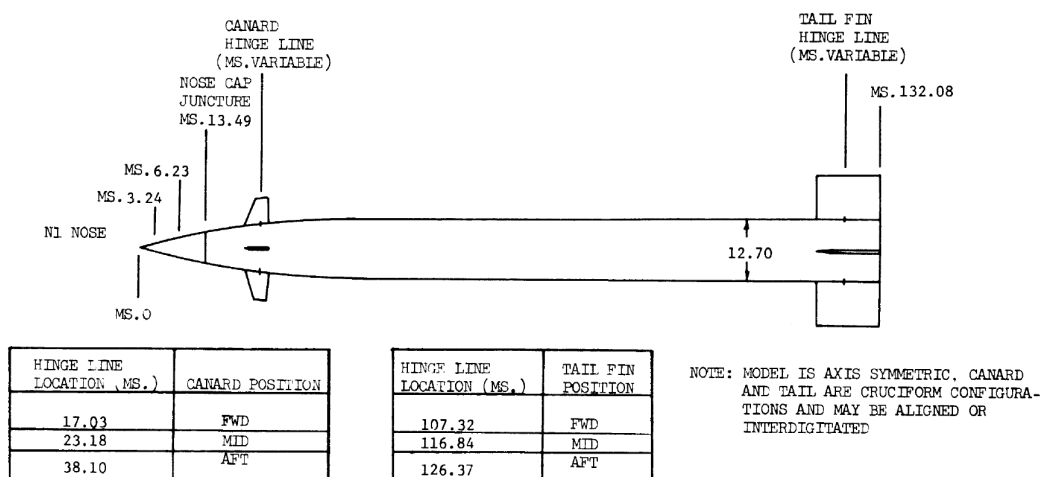
Chapter 6 reconstructed the N3 body, while chapter 7 added the C6 canards and T2 tails. The resulting N3–C6–T2 vehicle is therefore used here only as a convenient complete baseline.

A reader entering the guide at this section does not need to know the NASA configuration numbering: the code below defines the entire vehicle before any advanced feature is added.

Important

The NASA report does *not* define an inlet or protuberance model for this vehicle, and it does not provide the dynamic-derivative, spin/Magnus, or surface-pressure datasets demonstrated below. The inlet geometry, protuberance geometry, and EXPR coefficient values in this section are intentionally illustrative SpearDAT examples. They are not measurements, dimensions, or claims extracted from NASA TM X-73,218.

NOTE: ALL DIMENSIONS ARE IN CENTIMETERS



(a) Basic model drawing

Figure 8.6.: NASA basic model and available canard/tail stations, retained here as context for the advanced-feature examples. Reproduced from Figure 3(a) of NASA TM X-73,218 [3].

8.7.1. Reusable NASA Baseline

The following listing recreates the complete aligned N3–C6–T2 baseline so that this chapter can be used independently. The optional examples that follow copy this input and change only the feature being demonstrated.

```
input = struct;

%% Units and ordinary output
input.settings.lengthSystem = "Centimetres";
input.settings.derivativeSystem = "Degrees";

input.caseInput.id = "NASA N3 C6 T2 advanced baseline";
input.caseInput.output.plot = true;
input.caseInput.theory.bodyMethod = "VanDykeHybrid";

%% Flight conditions
```

```

input.caseInput.flight.attitudeMode = "BodyAlphaBeta";
input.caseInput.flight.alphaDeg = 0:2:24;
input.caseInput.flight.betaDeg = 0.0;
input.caseInput.flight.speed.mode = "Mach";
input.caseInput.flight.speed.mach = [0.8 1.3 1.75];
input.caseInput.flight.atmosphere.mode = "ReynoldsSpecified";

rePerMetre = 6.25e5 / 0.127;
input.caseInput.flight.atmosphere.reynoldsPerLength = ...
    [rePerMetre rePerMetre rePerMetre];

%% Reference quantities
input.caseInput.reference.sref = 126.7;
input.caseInput.reference.lref = 12.70;
input.caseInput.reference.latref = 12.70;
input.caseInput.reference.xcg = 66.0;
input.caseInput.reference.zcg = 0.0;
input.caseInput.reference.boundaryLayer = "Turbulent";

%% N3 body
input.caseInput.body.shape = "Axisymmetric";
input.caseInput.body.axisymmetric.geometryMode = "Analytic";
input.caseInput.body.axisymmetric.xOrigin = 3.24;
input.caseInput.body.axisymmetric.nose.type = "Ogive";
input.caseInput.body.axisymmetric.nose.length = 38.10 - 3.24;
input.caseInput.body.axisymmetric.nose.diameter = 12.70;
input.caseInput.body.axisymmetric.nose.bluntness = 1.42;
input.caseInput.body.axisymmetric.nose.truncated = false;
input.caseInput.body.axisymmetric.center.length = 132.08 - 38.10;
input.caseInput.body.axisymmetric.center.diameter = 12.70;

%% C6 canards -- fin set 1
c6 = struct;
c6.set = 1;
c6.spanStations = [0 9.53];
c6.chord = [10.16 0.63];
c6.xLeadingEdge = [29.07 38.60];
c6.nPanels = 4;
c6.panelRollDeg = [0 90 180 270];
c6.dihedralDeg = [0 0 0 0];
c6.section.type = "Hex";
c6.section.hexArc.zUpper = [0.0187008 0.1190476];
c6.section.hexArc.lMaxUpper = [0.25 0.0];
c6.section.hexArc.lFlatUpper = [0.50 1.0];

%% T2 tails -- fin set 2
t2 = struct;
t2.set = 2;
t2.spanStations = [0 8.89];
t2.chord = [17.78 8.89];
t2.xLeadingEdge = [103.44 112.33];
t2.nPanels = 4;
t2.panelRollDeg = [0 90 180 270];
t2.dihedralDeg = [0 0 0 0];
t2.section.type = "Hex";
t2.section.hexArc.zUpper = [0.0143420 0.0286839];
t2.section.hexArc.lMaxUpper = [0.2142857 0.4285714];
t2.section.hexArc.lFlatUpper = [0.7857143 0.5714286];

```



```

input.caseInput.finSets = [c6 t2];

%% Zero-incidence hinge definitions
c6Defl.set = 1;
c6Defl.panelDeflectionDeg = [0 0 0 0];
c6Defl.hingeX = 38.10;
c6Defl.hingeSkewDeg = 0;

t2Defl.set = 2;
t2Defl.panelDeflectionDeg = [0 0 0 0];
t2Defl.hingeX = 116.84;
t2Defl.hingeSkewDeg = 0;

input.caseInput.deflections.finSets = [c6Defl t2Defl];

```

Listing 8.24: Reusable NASA N3–C6–T2 baseline for the advanced examples.

8.7.2. Illustrative Inlet Addition

The first variation adds a synthetic top-mounted inlet on the cylindrical portion of the N3 body. The dimensions are chosen only to demonstrate the API; they are not present in the NASA model.

```

inletInput = input;
inletInput.caseInput.id = "NASA geometry plus illustrative inlet";

inlet = struct;
inlet.enabled = true;
inlet.type = "TwoDimensionalTop";
inlet.xInlet = 55.0;
inlet.xDiverter = 1.0;
inlet.diverterHeight = 0.5;
inlet.diverterLength = 3.0;
inlet.placements(1).phiDeg = 0;
inlet.profile.x = [0 3 6 10 14];
inlet.profile.height = [0 1.8 2.4 2.2 1.5];
inlet.profile.width = [3.0 3.5 3.8 3.6 3.0];
inlet.covered = false;
inlet.rampAngleDeg = 5.0;
inlet.additiveDrag = true;
inlet.massFlowRatio = [0.70 0.75 0.80];

inletInput.caseInput.inlet = inlet;
inletInput.caseInput.output.printGeometry = ...
    ["Body", "Inlet"];
inletInput.caseInput.output.printAero = ...
    ["Body", "FinSet1", "FinSet2", "Inlet", "Synths"];

inletResult = speardat_mex(inletInput);

```

Listing 8.25: Adding an illustrative inlet to the NASA-based vehicle.

The code first copies the complete N3–C6–T2 baseline, so the body, canards, tails, flight schedule, and reference quantities are unchanged. The inlet structure then adds one common

top-mounted inlet profile at $x = 55$ cm with a positive diverter height, making this a separated diverter-mounted installation. The five `profile.x` values are local to the inlet leading edge, while the height and width arrays define its changing cross-section through the cowl, midbody, and boattail stations.

`additiveDrag=true` activates the spillage-drag path and the three mass-flow-ratio entries correspond positionally to the existing Mach schedule [0.8, 1.3, 1.75]. They do not add three more flight conditions. The Mach-0.8 entry is still supplied to keep the MFR array aligned even though the legacy additive-drag method will not calculate an additive increment below Mach 1. The two output requests add detailed inlet geometry and inlet/synthesis aerodynamic sections to `for006.dat`; the ordinary structured `inletResult.aerodynamics` table remains the normal total configuration result.

8.7.3. Illustrative Protuberance Addition

A separate variation adds four small identical blocks at one axial station. Again, these blocks are not part of the NASA report; they simply demonstrate how a non-lifting external item is attached to the familiar vehicle model.

```
protubInput = input;
protubInput.caseInput.id = ...
    "NASA geometry plus illustrative protuberances";

member.length = 2.0;
member.width = 1.0;
member.height = 0.5;
member.verticalOffset = 0;

item.type = "Block";
item.x = 70.0;
item.countAroundBody = 4;
item.members = member;

protubInput.caseInput.body.protuberances.enabled = true;
protubInput.caseInput.body.protuberances.items = item;

protubResult = speardat_mex(protubInput);
```

Listing 8.26: Adding illustrative block protuberances to the NASA-based vehicle.

This variation again begins by copying the baseline and changes only the body protuberance branch. One "Block" item is defined at an approximate centroid station of 70 cm. The single member gives the dimensions of one block, while `countAroundBody=4` tells the stock protuberance correlation that four identical copies exist around the body at that axial station.

The number four is a multiplicity, not an explicit azimuth schedule. Missile DATCOM does not receive four angles and does not construct named copies at 0° , 90° , 180° , and 270° . This is consistent with the stock PROTUB method, which contributes axial force and does not resolve the asymmetric lateral force and moment of individually clocked protuberances. Use a more appropriate representation if the circumferential position of an external feature is aerodynamically important.

When several physically different protuberance sets are required, additional `items` elements can be added, up to the 20-item public limit. That is different from the inlet branch: protuberances

may contain several distinct items, whereas one SpearDAT inlet branch contains only one inlet geometry with identical circumferential placements.

8.7.4. Dynamic Derivatives and Spin/Magnus Output

The derivative variation leaves the geometry unchanged and activates the two rate-related calculations described earlier in this chapter. Since the detailed dynamic-derivative and Magnus results are reported principally in `for006.dat`, the example retains the complete output bundle.

```
derivInput = input;
derivInput.caseInput.id = "NASA geometry dynamic effects demonstration";
derivInput.caseInput.options.dynamicDerivatives = true;
derivInput.caseInput.options.spinMagnus = true;
derivInput.caseInput.output.partialAll = true;

derivOptions = struct;
derivOptions.legacyFiles = true;
derivOptions.outputDirectory = ...
    fullfile(pwd, "speardat_nasa_dynamic_effects");

derivResult = speardat_mex(derivInput, derivOptions);

for006Path = fullfile(...
    string(derivResult.cases(1).legacyDirectory), "for006.dat");
for006Text = fileread(for006Path);
```

Listing 8.27: Requesting dynamic derivatives and spin/Magnus output for the NASA-based vehicle.

The two option flags request rate-related aerodynamic information without adding a new flight-condition schedule. `dynamicDerivatives=true` requests the dynamic aerodynamic derivatives, while `spinMagnus=true` adds the specialised body spin/Magnus correlation. The ordinary static aerodynamic table remains available through `derivResult.aerodynamics`.

`partialAll=true` requests the body-alone partial report so that the spin/Magnus quantities can be inspected for a vehicle that also contains C6 and T2. It does not remove the fins from the main aerodynamic calculation. The important output-management step is `derivOptions.legacyFiles=true`: this retains the numbered solver files after the MATLAB call. The example then obtains the returned legacy directory and reads `for006.dat`, where the detailed dynamic-derivative and spin/Magnus information is reported. These quantities should be interpreted using the conventions described in [sections 8.5](#) and [8.5.2](#) rather than as physical body-rate time histories.

8.7.5. Pressure-Output Demonstration

For the pressure demonstration, the same geometry is used with an explicit SOSE body-method request and retained auxiliary pressure files:

```
pressureInput = input;
pressureInput.caseInput.id = "NASA geometry pressure demonstration";
pressureInput.caseInput.theory.bodyMethod = "Sose";
pressureInput.caseInput.output.pressures = true;
```

```

pressureOptions = struct;
pressureOptions.legacyFiles = true;
pressureOptions.outputDirectory = ...
    fullfile(pwd, "speardat_nasa_pressures");

pressureResult = speardat_mex(pressureInput, pressureOptions);

pressureDir = string(pressureResult.cases(1).legacyDirectory);
bodyPressure = fileread(fullfile(pressureDir, "for010.dat"));
finPressure = fileread(fullfile(pressureDir, "for011.dat"));

```

Listing 8.28: Requesting pressure-distribution files for the NASA-based vehicle.

This code makes two independent requests: it explicitly selects SOSE as the body method and separately enables pressure-file output. The complete N3-C6-T2 flight schedule is otherwise unchanged. The ordinary case therefore still contains Mach 0.8, 1.3, and 1.75 and the original alpha schedule, while the auxiliary pressure routines emit only the combinations for which their methods are applicable. The Mach-0.8 body-pressure point is excluded, and the Hybrid/SOSE body-pressure path is limited to angles of attack through 15 degrees. The pressure files are therefore a filtered auxiliary view of the normal analysis rather than a second flight-condition sweep.

The resulting files provide useful sanity checks. In the successful for010.dat output for Mach 1.3 and $\alpha = 0^\circ$, the columns CP(0) through CP(180) are equal at each X/D, as expected for an axisymmetric body at zero incidence. The generated for011.dat begins with Y/(B/2), X/C, CP and reports the fin section pressure coefficient over chordwise stations at successive normalised span locations. These are consistent with the file meanings described in [section 8.6.2](#) and confirm that both body and fin pressure paths were exercised by this example.

8.7.6. Synthetic EXPR Demonstration

The last variation demonstrates direct experimental-data substitution while keeping a clear boundary between the NASA geometry and the invented data. To avoid suggesting that a synthetic dataset exists at all three NASA test Mach numbers, this example reduces the active case to Mach 1.3 and uses a matching seven-point angle-of-attack schedule.

```

exprInput = input;
exprInput.caseInput.id = "NASA geometry synthetic EXPR demonstration";
exprInput.caseInput.flight.speed.mach = 1.3;
exprInput.caseInput.flight.atmosphere.reynoldsPerLength = rePerMetre;
exprInput.caseInput.flight.alphaDeg = [0 4 8 12 16 20 24];

synthetic = struct;
synthetic.mach = 1.3;
synthetic.configuration = "BodyFinSets12";
synthetic.alphaDeg = [0 4 8 12 16 20 24];

% Synthetic demonstration values -- not NASA measurements.
synthetic.coefficients.cn = ...
    [0.00 0.42 0.88 1.38 1.92 2.48 3.05];
synthetic.coefficients.cm = ...
    [0.01 -0.04 -0.12 -0.23 -0.36 -0.52 -0.70];
synthetic.coefficients.ca = ...

```

```
[0.10 0.11 0.13 0.16 0.20 0.25 0.31];

exprInput.caseInput.experimentalData.incrementMode = false;
exprInput.caseInput.experimentalData.datasets = synthetic;

exprResult = speardat_mex(exprInput);
```

Listing 8.29: Using explicitly synthetic EXPR data with the NASA-based geometry.

The code deliberately reduces the active analysis to one Mach number and seven alpha stations so that the synthetic dataset has an exact, easy-to-read match. The variable `synthetic` is simply one MATLAB structure containing one EXPR dataset; assigning it to `experimentalData.datasets` creates a one-element dataset collection. `configuration="BodyFinSets12"` attaches the data to the complete N3–C6–T2 cumulative stage because C6 is fin set 1 and T2 is fin set 2.

Only the supplied `cn`, `cm`, and `ca` channels are substituted. Lateral coefficients and any other omitted channels remain on the theoretical path. The successful run returns one scalar SpearDAT case and seven aerodynamic rows, corresponding to the seven requested alpha values at Mach 1.3; this is exactly the intended structure for a single direct- substitution dataset.

This example is intentionally a mechanics demonstration rather than a NASA validation exercise. The coefficient values are synthetic. A genuine wind-tunnel comparison would require the measured configuration, test Mach number, angle stations, coefficient signs, and original reference quantities to be matched to the source report before the values were inserted through `experimentalData`. If those references differ from the active SpearDAT convention, use the dataset-specific reference fields described in [section 8.3.1](#) so the conversion is explicit.

8.8. Common Advanced-Feature Modelling Mistakes

The advanced branches are easiest to use when each is treated as a specialised model rather than as a generic container:

Each inlet placement is treated as an independent geometry. One inlet branch defines one common profile. The placement vector creates identical circumferential copies at different roll angles.

Several different inlet geometries are expected to accumulate. The historical solver retains one `$INLET` definition per case; a later repeated `namelist` overwrites the earlier one without creating a second independent inlet geometry. Use placements only for identical copies of the common geometry.

The mass-flow-ratio vector is treated as an independent sweep. Its entries align positionally with the existing Mach or velocity schedule. They do not create new freestream conditions. For a multi-speed additive-drag case, use the explicit per-speed MFR vector documented in [section 8.1.2](#).

A covered inlet is used as a general external blockage model. `covered=true` selects the legacy plugged-inlet treatment. Use a protuberance or another appropriate geometry model for unrelated external blockage.

A lifting strake is represented only as a protuberance. The stock protuberance model is intended principally for non-lifting external items. Use a fin set when lifting-surface force and moment behaviour is the intended physical effect.

countAroundBody is treated as an explicit placement command. It specifies the number of identical copies in the stock protuberance set. Their individual azimuths are not input to PROTUB, and the method contributes axial force rather than resolving fin-like asymmetric force and moment. Composite lug/shoe members are a separate concept.

EXPR data are assumed to be increments by default. With incrementMode=false, populated coefficient channels are substituted at the selected configuration level. They are not automatically added as deltas to the theoretical coefficients.

Experimental data are assigned to a non-contiguous fin combination. Configuration selectors follow the same cumulative body-fin ordering used by the solver. A body-plus-fin-sets-1-and-3 selector does not exist.

DAMP is treated as a physical body-rate input schedule. dynamicDerivatives=true requests DATCOM's reduced-rate derivative calculation; it does not accept a user p , q , or r sweep.

SPIN is treated as a time-domain spinning-flight simulation. It activates a specialised aerodynamic correlation. Vehicle rotational dynamics must still be integrated by a separate flight-dynamics model.

PRESSURES is assumed to select SOSE automatically. output.pressures controls auxiliary output. The body method is selected separately through theory.bodyMethod.

Pressure and derivative details are expected in the combined MATLAB aerodynamic table.

Retain and inspect for006.dat and for009.dat–for012.dat when those advanced outputs are the purpose of the analysis.

8.9. MATLAB-to-Legacy Field Map

The MATLAB structure remains the public interface. The legacy names in [table 8.5](#) are included as landmarks for readers comparing a SpearDAT request with the Missile DATCOM User Guide or a retained for005.dat deck.

Table 8.5.: Principal optional and advanced field mapping.

MATLAB field	Legacy field/card	Purpose
inlet.type	INTYPE	Inlet geometry family.
inlet.xInlet	XINLT	Nose-to-inlet leading-edge distance.
inlet.xDiverter	XDIV	Inlet-local diverter leading-edge location.
inlet.diverterHeight	HDIV	Diverter, conformal, or submerged height.
inlet.diverterLength	LDIV	Diverter length.
inlet.placements(i).phiDeg	PHI	Circumferential position of each identical inlet copy.
inlet.profile.x/height/width	X/H/W	Five-station inlet profile.
inlet.covered	COVER	Covered/plugged inlet selection.
inlet.rampAngleDeg	RAMP	External-compression ramp angle.
inlet.additiveDrag	ADD	Additive/spillage-drag request.
inlet.massFlowRatio	MFR	Admitted mass-flow-ratio schedule.
body.protuberances.items(i).type	PTYPE	Stock protuberance type.
body.protuberances.items(i).x	XPROT	Protuberance-set axial station.

Continued on next page

MATLAB field	Legacy field/card	Purpose
body.protuberances.items(i).countAroundBody	NLOC	Number of identical repeated copies.
body.protuberances.items(i).members(j).length/width/height	LPROT/WPROT/HPROT	Member dimensions.
body.protuberances.items(i).members(j).verticalOffset	OPROT	Member offset from the body mold line.
experimentalData.incrementMode	INCRMT	Historical configuration-incrementing request.
experimentalData.datasets(i).mach	MACH	Dataset Mach number.
experimentalData.datasets(i).alphaDeg	ALPHA	Dataset angle-of-attack stations.
experimentalData.datasets(i).configuration	CONF	Configuration level receiving the data.
experimentalData.datasets(i).reference.sref	SREF	Reference area used to reduce the supplied data.
experimentalData.datasets(i).reference.lref	LREF	Longitudinal reference length of the supplied data.
experimentalData.datasets(i).reference.latref	LATREF	Lateral reference length of the supplied data.
experimentalData.datasets(i).reference.xcg/zcg	XCG/ZCG	Dataset moment-reference location.
experimentalData.datasets(i).coefficients.cn/cm/ca	CN/CM/CA	Supplied longitudinal coefficient channels.
experimentalData.datasets(i).coefficients.cy/csn/csl	CY/CSN/CSL	Supplied lateral-directional coefficient channels.
options.dynamicDerivatives	DAMP	Dynamic damping-derivative calculation.
settings.derivativeSystem	DERIV DEG/RAD	Derivative reporting convention.
options.spinMagnus	SPIN	Specialised spin/Magnus calculation.
output.partialAll	PART	Detailed partial/component reports, including the route used in the SPIN example.
output.pressures	PRESSURES	Auxiliary pressure-output request.
theory.bodyMethod="Sose"	SOSE	Second-Order Shock Expansion body method.
theory.bodyMethod="Hyper"	HYPER	Hypersonic/Newtonian body-method selection.
output.printGeometry="Inlet"	PRINT GEOM INLET	Detailed inlet geometry report in for006.dat.
output.printAero="Inlet"	PRINT AERO INLET	Detailed inlet aerodynamic report in for006.dat.

The writer generates legacy count fields such as NIN, NPROT, and experimental NALPHA from the corresponding MATLAB arrays rather than asking the user to maintain them independently. The complete field types, limits, omission behaviour, allowed configuration names, and specialised validation conditions are collected in [sections B.4.3, B.5, B.9.6, B.13 and B.14](#).

9. Results Interpretation and Experimental Validation

The preceding chapters developed a representative aerodynamic analysis from the underlying physical description of the vehicle. The flight conditions and reference quantities were established first, followed by the body geometry and then the lifting surfaces and controls. The resulting model is based on a wind-tunnel investigation published by NASA and represents an axisymmetric missile body fitted with a forward canard set and an aft tail set. Within the NASA report these components are identified as the N3 body, C6 canards, and T2 tails.

This chapter uses that established NASA-based model for two purposes. First, it explains how to interpret the structured numerical result returned by SpearDAT, including the relationship between cases, output zones, configuration levels, flight conditions, coefficients, and resolved state. Second, it compares selected SpearDAT predictions with four NASA wind-tunnel data sets. NASA TM X-73,218 [3] supplies the principal worked validation example. NASA/TM-2005-213541 [4] then provides a separate body–tail, aerodynamic-roll, and control exercise, NASA TM X-3129 [5] adds a body-alone assessment of a constant-ellipticity configuration over a broad subsonic-to-supersonic Mach range, and NASA TM-84560 [6] adds a materially different subsonic–transonic test of twin installed axisymmetric inlets and a tri-tail.

9.1. Purpose and Comparison Strategy

The starting point is the complete aligned N3–C6–T2 input developed in [section 7.9.3](#); the corresponding MATLAB definition is given in [listing 7.25](#). That listing already defines the geometry, flight-condition schedule, reference quantities, zero control deflections, and requested configuration build-up. It is therefore not repeated here. Instead, this chapter begins with the result returned by running that established input and uses the observed output to explain the SpearDAT result model.

The NASA TM X-73,218 investigation provides a useful external comparison because the published test programme includes the same body, canard, and tail family used in the preceding chapters. Measurements were obtained at Mach numbers 0.8, 1.3, and 1.75, at a Reynolds number of 6.25×10^5 based on body diameter, and over angles of attack from approximately 0° to 24° . The report also includes several canard-incidence conditions and component build-up measurements. The aerodynamic coefficients were reduced about model station 66.0 cm using a reference area of 126.7 cm^2 and a reference length of 12.70 cm, matching the reference convention adopted by the SpearDAT example [3].

The work is developed in stages. The structured MATLAB result is examined first so that its case, zone, configuration, flight-condition, control, and aerodynamic-coefficient fields can be interpreted in the context of a real vehicle. The same analysis is then repeated with retained legacy files enabled to show how the MATLAB result relates to the detailed solver reports and configuration build-up. Selected experimental curves from NASA TM X-73,218 are then digitised from the published figures and compared directly with the corresponding SpearDAT predictions. Three further validation exercises broaden the evidence beyond that worked configuration. The NASA 2005 strake–tail programme is read from its original electronic tabulated database, NASA TM X-3129 is used to assess the body-alone elliptical-body path at five Mach numbers using carefully reviewed values digitised from the published plots, and NASA TM-84560 is used to exercise the installed axisymmetric-inlet and tri-tail paths at subsonic–transonic conditions.

Important

The four validation data sets use two experimental-data routes. NASA TM X-73,218, NASA TM X-3129, and NASA TM-84560 require values to be digitised from published figures; only source markers that can be read consistently from the scanned plots are retained for comparison. For NASA TM-84560 the documentation exercise deliberately uses a coarser subset of readable Figure 5 markers rather than attempting to reproduce the full graphical resolution of the source plots. NASA/TM-2005-213541 instead provides the original ASCII-tabulated wind-tunnel data and does not require digitisation. The comparisons are used to assess aerodynamic trends, coefficient levels, control effectiveness, interference behaviour, inlet and non-circular-body response, and regions of reduced prediction fidelity rather than to imply precision beyond that supported by the source data.

9.2. Running the NASA Baseline

Run the complete aligned vehicle defined in [listing 7.25](#). The example produces one successful SpearDAT case:

```
result
```

Listing 9.1: Inspecting the top-level result returned by the NASA baseline.

For the present analysis, MATLAB reports:

```
result =

  struct with fields:

    succeeded: 1
    hasErrors: 0
    caseCount: 1
    successfulCaseCount: 1
    failedCaseCount: 0
    diagnostics: [1x0 struct]
    legacyOutputDirectory: ''
    aerodynamics: [195x22 table]
    cases: [1x1 struct]
```

Listing 9.2: Observed top-level result for the NASA baseline.

The status fields confirm that the single requested case completed successfully and produced no diagnostics. The empty `result.legacyOutputDirectory` is expected because this first run uses temporary solver files. Retained output is introduced later in this chapter.

The two principal result branches are:

result.aerodynamics One combined MATLAB table containing the aerodynamic rows returned for the completed request.

result.cases One structure per expanded SpearDAT case, containing case identity, execution status, generated input text, resolved solver state, and the row range occupied by that case in the combined aerodynamic table.

9.3. Organisation of the Aerodynamic Table

The baseline result contains 195 rows and 22 columns:

```
aero = result.aerodynamics;

size(aero)
aero.Properties.VariableNames
```

Listing 9.3: Inspecting the size and variable names of the aerodynamic table.

The row count follows directly from the requested schedules and build-up:

3 Mach numbers $\times$ 5 reported configuration levels $\times$ 13 angles of attack = 195 rows.

9.3.1. Cases and Sweep Identity

Two different zero-based indices appear in the combined table and in `result.cases`:

CaseIndex Identifies the prepared scalar case within the SpearDAT request. Cases are assigned in stable expansion order beginning at zero, and the same value is used to form generated names such as `case_0000`.

SweepIndex Identifies the position of that scalar case within the retained BETA or PHI sweep order. It also begins at zero. The associated `SweepVariable` and `SweepValueDeg` state which attitude quantity and scalar value that sweep position represents.

The distinction is one of purpose: `CaseIndex` is the execution identity of a prepared case, whereas `SweepIndex` describes its position in the attitude sweep that produced it. In the current BETA/PHI expansion path the cases are prepared in sweep order, so the two indices normally have the same numerical value. They are nevertheless retained separately because they describe different aspects of case identity and should not be treated as interchangeable fields.

The Chapter 7 baseline defined in [listing 7.25](#) contains one explicit BETA value, 0° . Consequently both indices are zero, while `SweepVariable` is `BetaDeg` and `SweepValueDeg` is zero. A request containing several BETA or PHI values would produce one scalar case for each retained value, numbered from zero in stable sweep order.

Note

`CaseIndex` and `SweepIndex` follow the zero-based SpearDAT C++ result convention. MATLAB array positions remain one-based. Therefore, the first MATLAB structure is `result.cases(1)`, while its `caseIndex` and `sweepIndex` values are normally zero.

9.3.2. Zones and Zone Types

A *zone* is one contiguous aerodynamic output group parsed from `for003`. It collects the rows belonging to one combination of result type and operating condition. For an ordinary build-up result, a zone contains the angle-of-attack schedule for one configuration at one Mach, sideslip, and aerodynamic-roll condition. Trim calculations create different zone types, described in [table 9.1](#).

ZoneNumber is one-based and restarts at one for every SpearDAT case. It is assigned from the order in which the zones appear in the parsed aerodynamic output; it is therefore an output-group identifier rather than a physical component number.

Table 9.1.: Zone types represented in the SpearDAT aerodynamic table.

ZoneType	When it is produced	Associated metadata
NoTrim	Ordinary untrimmed aerodynamic output and configuration build-up.	Configuration contains the reported build-up identifier. CommandedDeltaDeg is not applicable.
Untrimmed	An untrimmed control-deflection point evaluated as part of a trim calculation.	CommandedDeltaDeg contains the commanded deflection. Configuration is not applicable.
Trimmed	The solved trim result.	The solved deflection is carried in the row-level DeltaDeg column. Configuration and CommandedDeltaDeg are not applicable.

Only NoTrim zones occur in the present baseline because no trim calculation was requested. There are five reported configurations at each of the three Mach numbers, so this case contains 15 zones. Each zone contains the 13 requested angle-of-attack rows.

9.3.3. Configuration Numbers and Build-Up Levels

For NoTrim zones, the Configuration column preserves the Missile DATCOM build-up identifier. The complete mapping is shown in [table 9.2](#). Fin sets are ordered according to their FINSET number, and the body-fin levels are cumulative from fin set 1.

Table 9.2.: Missile DATCOM configuration identifiers used in no-trim aerodynamic output.

Configuration	Reported assembly	Interpretation
1	Body alone	Body aerodynamic result without lifting surfaces.
2	Fin set 1 alone	Isolated fin-set-1 result.
3	Fin set 2 alone	Isolated fin-set-2 result.
4	Fin set 3 alone	Isolated fin-set-3 result.
5	Fin set 4 alone	Isolated fin-set-4 result.
6	Body + fin set 1	First cumulative body-fin synthesis level.
7	Body + fin sets 1–2	Body with fin sets 1 and 2.
8	Body + fin sets 1–3	Body with fin sets 1 through 3.
9	Body + fin sets 1–4	Body with fin sets 1 through 4.

A configuration is reported only when the required component geometry exists. The NASA example contains two fin sets, so the values that actually appear are 1, 2, 3, 6, and 7. In this chapter fin set 1 is the C6 canard set and fin set 2 is the T2 tail set. Configuration 7 therefore represents the complete N3–C6–T2 vehicle.

The isolated fin-set rows are useful for understanding component behaviour, but they should not be treated as additive corrections that can always be summed manually. As discussed in [section 8.4](#), the cumulative body–fin levels include the applicable synthesis and interference effects.

The following selection displays the complete-vehicle result at Mach 1.3:

```

aero = result.aerodynamics;

completeM13 = aero( ...
    aero.Configuration == 7 & ...
    aero.Mach == 1.3, :);

completeM13(:, ...
    ["ZoneNumber", "Mach", "AlphaDeg", ...
    "CN", "CM", "CA", "CL", "CD"])

```

Listing 9.4: Selecting the complete N3–C6–T2 result at Mach 1.3.

Using both Configuration and Mach makes the selection independent of the numerical zone ordering.

9.3.4. Conditions, Controls, and Coefficients

The remaining metadata columns describe the operating condition and control state:

Mach, BetaDeg, RollDeg, and AlphaDeg Give the Mach number and attitude associated with the aerodynamic coefficients.

CommandedDeltaDeg and DeltaDeg Describe the control condition. CommandedDeltaDeg is used by untrimmed trim-search zones. The row-level DeltaDeg records the deflection associated with the aerodynamic row and carries the solved deflection in a trimmed zone. In the present zero-deflection baseline, DeltaDeg is zero.

The principal aerodynamic columns are:

CN, CM, and CA Normal-force, pitching-moment, and axial-force coefficients in the Missile DATCOM body-axis convention.

CY, CLN, and CLL Side-force, yawing-moment, and rolling-moment coefficients. These are zero for the present symmetric configuration at zero sideslip and zero aerodynamic roll.

CL and CD Lift and drag coefficients transformed from the body-axis force coefficients for the reported angle of attack.

For example, the complete vehicle at Mach 1.3 and $\alpha = 0^\circ$ has $C_N = C_M = C_L = 0$, while its axial-force and drag coefficients are both 0.5672. At non-zero incidence, the transformation between body-axis normal/axial force and wind-axis lift/drag causes CL and CD to differ from CN and CA.

9.4. Inspecting the Per-Case Result

The corresponding per-case structure is obtained using MATLAB array indexing:

```

caseResult = result.cases(1);

caseResult.caseIndex
caseResult.caseName
caseResult.sweepIndex
caseResult.sweepVariable

```

```
caseResult.sweepValueDeg
caseResult.succeeded
caseResult.for006Status
caseResult.aerodynamicRowStart
caseResult.aerodynamicRowCount
```

Listing 9.5: Inspecting the first completed SpearDAT case.

For this request, `caseResult.aerodynamicRowStart` is 1 and `caseResult.aerodynamicRowCount` is 195, so the single case occupies the whole combined table. For a multi-case request these fields locate only the rows belonging to the selected case. The row start follows MATLAB's one-based table indexing even though `caseIndex` and `sweepIndex` are zero-based identifiers.

The nested `caseResult.for006Status` structure confirms that the solver completed successfully, reached the normal end-of-job marker, and reported no errors. This status describes solver-output processing; it is separate from the aerodynamic coefficients themselves.

9.4.1. Generated Input Text

The generated solver input can be inspected directly without retaining any files:

```
caseResult.for005Text
```

Listing 9.6: Inspecting the generated solver input text.

For the NASA baseline, this text confirms the principal resolved requests: centimetre units, degree-based derivative reporting, component build-up, the three Mach numbers, the 13-point angle-of-attack schedule, the N3 body, the C6 and T2 fin sets, and zero panel deflections. It is primarily a traceability representation of the case that SpearDAT submitted to the solver; normal MATLAB workflows do not require the user to construct or edit it.

The field is available even though `caseResult.legacyDirectory` is empty. The latter is populated only when retained solver files are requested.

9.5. Inspecting the Resolved Analysis State

The resolved state records the numerical model after SpearDAT has interpreted the public input and prepared the solver case:

```
state = caseResult.resolvedState;

state.flight
state.reference
state.configuration
state.geometry
```

Listing 9.7: Opening the resolved analysis state.

The four branches describe the resolved flight schedule, reference convention, solver/configuration selections, and calculated geometry. This information is particularly useful when the solver derives quantities from the supplied geometry or when a compact confirmation of the analysis actually executed is required.

9.5.1. Resolved Flight and Reference Quantities

The flight branch contains the complete incidence schedule and three resolved speed points:

```
state.flight.alphaDeg
state.flight.betaDeg
state.flight.rollDeg

[state.flight.points.mach]
[state.flight.points.reynoldsPerLength]
```

Listing 9.8: Inspecting the resolved flight schedule.

The resolved values are the requested Mach numbers [0.8, 1.3, 1.75], zero sideslip, zero aerodynamic roll, and angles of attack from 0° to 24° in 2° increments. The Reynolds-number-per-length value is approximately $4.9213 \times 10^6 \text{ m}^{-1}$ at each Mach point. Multiplication by the 0.127-m body diameter gives the NASA diameter-based value of approximately 6.25×10^5 .

The reference branch confirms:

$$S_{\text{ref}} = 126.7 \text{ cm}^2, \quad L_{\text{ref}} = L_{\text{atref}} = 12.7 \text{ cm},$$

$$x_{\text{cg}} = 66.0 \text{ cm}, \quad z_{\text{cg}} = 0,$$

together with the turbulent boundary-layer selection. These values are the coefficient convention used by both the SpearDAT prediction and the later NASA comparison.

9.5.2. Resolved Configuration

The `state.configuration` branch provides a compact record of the configuration and calculation switches present in the resolved solver state:

```
state.configuration
```

Listing 9.9: Inspecting the resolved configuration.

The observed NASA baseline values are summarised in [table 9.3](#).

The branch is useful as a resolved-case summary rather than as a second input interface. For example, `deflectionsPresent` records that a deflection definition exists; it does not imply that the deflection value is non-zero. Similarly, `bodyMethod` records the selected body-method mode rather than exposing every lower-level method branch used internally by the aerodynamic calculation.

Table 9.3.: Resolved configuration state for the NASA baseline.

Field	Observed value	Meaning
lengthSystem	Centimetres	Dimensional geometry and reference quantities use centimetres.
derivativeSystem	Degrees	Derivative reporting uses the degree convention.
bodyShape	Axisymmetric	The active body is axisymmetric.
bodyMethod	VanDykeHybrid	Selected body aerodynamic method.
finSetsPresent	[1100]	Fin sets 1 and 2 are present; sets 3 and 4 are absent.
inletPresent	0	No inlet is defined.
trimRequested	0	No trim calculation is requested.
deflectionsPresent	1	A deflection definition is present. This records the presence of the definition even though the current panel deflections are all zero.
dynamicDerivativesRequested	0	Dynamic-derivative output is not requested.
spinRequested	0	Spin/Magnus output is not requested.
buildupRequested	1	Component and cumulative configuration build-up is requested.
partialOutputRequested	0	Global partial-output reporting is not requested.
plotRequested	1	Machine-oriented aerodynamic output is requested.
pressureOutputRequested	0	Pressure-distribution output is not requested.
lateralDirectionalSuppressed	0	Lateral-directional calculations have not been suppressed.

9.5.3. Resolved Body and Fin Geometry

The body branch contains derived integral quantities in addition to the individual nose, centre, and aft-section records. For the N3 body, SpearDAT reports a maximum and base diameter of 12.7 cm, a total length of 128.84 cm, and a total fineness ratio of approximately 10.145. The length excludes the 3.24-cm axial origin offset; adding that offset recovers the 132.08-cm overall model station used in the NASA geometry definition.

The geometry branch reserves four fin-set positions because Missile DATCOM supports up to four sets:

```
finSets = state.geometry.finSets;

[finSets.finSet]
[finSets.available]
```

Listing 9.10: Checking which resolved fin sets are present.

Fin sets 1 and 2 are available, while sets 3 and 4 retain empty derived fields. The first two entries correspond to the C6 canards and T2 tails, respectively. Their resolved total planform areas are approximately 51.4143 cm² and 118.5482 cm², with aspect ratios 1.7665 and 0.6667.

Each fin-set entry contains its own segment structure. MATLAB requires the outer structure array to be indexed before the nested segment is selected:

```
finSets = state.geometry.finSets;

c6Segment = finSets(1).segments(1);
t2Segment = finSets(2).segments(1);

c6Segment.segmentNumber
```

```

c6Segment.planformArea
c6Segment.meanAerodynamicChord

t2Segment.segmentNumber
t2Segment.planformArea
t2Segment.meanAerodynamicChord

```

Listing 9.11: Accessing the resolved segment of each active fin set.

Note

The expression `finSets.segments(1).segmentNumber` is invalid because `finSets.segments` first produces a comma-separated list containing the `segments` field from all four fin-set entries. Select one fin-set element first, for example `finSets(1).segments(1).segmentNumber`.

For the present single-segment planforms, the segment-derived quantities are also the fin-set quantities. The resolved C6 segment has a planform area of 51.4143 cm², mean aerodynamic chord of 6.7979 cm, leading-edge sweep of 45°, aspect ratio 1.7665, and taper ratio 0.0620. The T2 segment has a planform area of 118.5482 cm², mean aerodynamic chord of 13.8289 cm, leading-edge sweep of 45°, aspect ratio 0.6667, and taper ratio 0.5000.

These fields are derived inspection data. They do not replace the public geometry definition used to create the analysis, but they provide a useful check that the solver has resolved the intended planform before the experimental comparison is performed.

9.6. Retaining and Inspecting the Detailed Solver Outputs

The preceding result was reconstructed from temporary working files, which are removed automatically after the MATLAB result has been assembled. For detailed traceability, the same analysis can be repeated with retained solver files enabled.

Choose an output directory that does not already contain a previous retained run:

```

options = struct;
options.legacyFiles = true;
options.outputDirectory = ...
    fullfile(pwd, "speardat_nasa_validation_legacy");

retainedResult = speardat_mex(input, options);

retainedResult.succeeded
retainedResult.legacyOutputDirectory
retainedResult.cases(1).legacyDirectory

```

Listing 9.12: Repeating the NASA baseline while retaining the solver-output bundle.

For the present run, `retainedResult.succeeded` is true. The top-level `legacyOutputDirectory` identifies the retained batch root, while `retainedResult.cases(1).legacyDirectory` identifies the directory belonging to the first scalar case.

Important

The retained output directory must be absent or empty when the analysis starts. SpearDAT does not overwrite a non-empty retained run directory. If the example is repeated, use a new directory name or deliberately remove the previous output after it is no longer required.

9.6.1. Retained Directory Structure

The retained output has two levels. Files describing the complete SpearDAT batch are stored at the output-directory root, while the generated input, numbered solver files, and per-case SpearDAT transport files are stored beneath the corresponding scalar-case directory. The observed single-case layout is:

```
speardat_nasa_validation_legacy/
|-- speardat_batch_manifest.txt
|-- speardat_batch_status.txt
|-- speardat_solver_stdout.txt
|-- speardat_solver_stderr.txt
`-- case_0000/
    |-- for003.dat
    |-- for004.dat
    |-- for005.dat
    |-- for006.dat
    |-- for007.dat
    |-- for008.dat
    |-- for009.dat
    |-- for010.dat
    |-- for011.dat
    |-- for012.dat
    |-- speardat_case_status.txt
    `-- speardat_resolved_state.txt
```

Listing 9.13: Observed retained-output layout for the NASA baseline.

The case directory can be enumerated directly from MATLAB:

```
caseDirectory = string( ...
    retainedResult.cases(1).legacyDirectory);

files = dir(caseDirectory);
files = files(~[files.isdir]);

string({files.name})'
```

Listing 9.14: Listing the retained files for the NASA case.

The observed case directory contains 12 files. SpearDAT creates the complete numbered output set so that the retained directory is a stable execution and transport boundary; a file may therefore exist even when its associated optional output path produced no data.

9.6.2. Numbered Solver Files

The numbered files preserve the historical Missile DATCOM file-unit convention. Their roles and the observed contents of this NASA run are summarised in [table 9.4](#).

Table 9.4.: Numbered solver files retained for the NASA baseline.

File	Purpose	Observed in this run
for003.dat	Machine-readable aerodynamic output used to reconstruct <code>result.aerodynamics</code> . It is produced by the historical <code>PLOT</code> output path.	Populated with the 15 aerodynamic zones and 195 rows described earlier in this chapter.
for004.dat	Legacy common-block output produced when <code>WRITE</code> cards request selected internal quantities.	Empty because no <code>WRITE</code> request is present.
for005.dat	Generated solver input deck for the scalar case.	Populated with the same deterministic deck available in memory through <code>result.cases(1).for005Text</code> .
for006.dat	Main human-readable report containing input checking, geometry, method information, requested component reports, configuration aerodynamics, and other detailed solver output.	Populated. This is the principal detailed engineering report for the baseline case.
for007.dat	Internal legacy <code>FORMAT/WRITE</code> -card stream.	Empty because those cards are not used.
for008.dat	Checked and preprocessed input cards produced by the solver input-checking path.	Populated.
for009.dat	Body-geometry plot/output stream used by applicable body-method paths.	Populated with body-geometry zones.
for010.dat	Body surface pressure-coefficient output at incidence for applicable supersonic body-method paths.	Populated at Mach 1.3 and 1.75, even though the public pressure-output option is not requested; see the note below.
for011.dat	Fin pressure-coefficient output when the corresponding pressure-output path is active.	Contains only its variables header; no fin-pressure zones are present.
for012.dat	Body pressure coefficient and local Mach number at zero incidence for the applicable SOSE body-pressure path.	Populated at Mach 1.3 and 1.75, even though the public pressure-output option is not requested; see the note below.

Note

The baseline resolved state reports `pressureOutputRequested=0` and `plotRequested=1`, yet the retained `for010.dat` and `for012.dat` files contain body-pressure data. This is preserved Missile DATCOM output behaviour: the body-method support path enters its

auxiliary pressure-output loop when either the pressure-output request or the legacy plot request is active. The presence of those raw files therefore does not mean that `input.caseInput.output.pressures` was requested.

For user intent, inspect the resolved configuration state. When pressure distributions are deliberately required as an analysis output, request them explicitly as described in [section 8.6](#).

This distinction also illustrates why retained numbered files are best treated as solver evidence rather than as a second public result API. Some streams are historical working or plotting channels whose population depends on internal method and reporting paths.

The main aerodynamic and report files can be read directly:

```
for003Text = fileread( ...
    fullfile(caseDirectory, "for003.dat"));

for006Text = fileread( ...
    fullfile(caseDirectory, "for006.dat"));
```

Listing 9.15: Reading the principal retained aerodynamic outputs.

The structured MATLAB aerodynamic table is reconstructed from `for003.dat`; `for006.dat` remains the detailed human-readable report and is not duplicated as text inside the normal MATLAB result.

9.6.3. SpearDAT Batch and Transport Files

The files beginning with `spearDAT_` are not historical Missile DATCOM reports. They belong to the isolated SpearDAT execution boundary and preserve batch identity, process status, diagnostics, and the resolved solver state. Their roles are summarised in [table 9.5](#).

Table 9.5.: SpearDAT files in the retained NASA output bundle.

File	Location	Purpose and observed result
<code>spearDAT_batch_manifest.txt</code>	Batch root	Defines scalar-case execution order. The present manifest contains the single entry <code>case_0000</code> .
<code>spearDAT_batch_status.txt</code>	Batch root	Summarises request completion. The retained file records one case, one successful case, zero failed cases, and <code>all_succeeded=1</code> .
<code>spearDAT_solver_stdout.txt</code>	Batch root	Captures standard output from the isolated solver process. This run records the execution of <code>case_0000</code> and reports that 1/1 cases succeeded.
<code>spearDAT_solver_stderr.txt</code>	Batch root	Captures process-level standard-error text. It is empty for this clean run.
<code>spearDAT_case_status.txt</code>	Case directory	Records scalar-case identity and completion. The observed file reports <code>case_index=0</code> , <code>succeeded=1</code> , <code>resolved_state_captured=1</code> , and zero diagnostics.
<code>spearDAT_resolved_state.txt</code>	Case directory	Serialises the complete resolved flight, reference, configuration, body, and fin geometry state. SpearDAT reads this transport representation back to construct <code>result.cases(k).resolvedState</code> .

The distinction between the two groups is important. The numbered files are solver-format evidence and reports; the `spearDAT_` files describe the execution boundary used to transport that case safely into the structured MATLAB result.

9.7. Preparing the NASA TM X-73,218 Experimental Data

The principal experimental comparison uses the complete aligned BN3–C6–T2 configuration. In the NASA test programme this corresponds to the zero-deflection condition

$$D_1 = D_2 = D_3 = D_4 = 0^\circ, \quad \Phi_C = 0^\circ, \quad \Phi_T = 0^\circ,$$

which matches the aligned C6 and T2 panel clocking used by the Chapter 7 SpearDAT model. The corresponding SpearDAT result is configuration 7 in [table 9.2](#).

The experimental values were digitised from the total, main-balance series in the published NASA plots rather than from the component panel-balance summations. For each coefficient, the centre of the plotted experimental marker was read after rotating the scanned report page and calibrating the printed graph axes. The digitised values therefore retain the experimental angle-of-attack coordinates shown in the report rather than being forced onto the 2° SpearDAT incidence grid.

The wind-tunnel conditions printed on the relevant figures are approximately Mach 0.802, 1.306, and 1.752. These are compared with the SpearDAT conditions Mach 0.80, 1.31, and 1.75, respectively. The differences are small enough that no Mach interpolation is introduced. For the quantitative error measures, however, the SpearDAT coefficient curve is linearly interpolated in angle of attack to the digitised NASA marker locations before the residual is calculated.

The coefficient mapping used in the comparison is given in [table 9.6](#).

Table 9.6.: NASA main-balance quantities used for the complete-configuration comparison.

NASA quantity	SpearDAT column	Interpretation
C_N	CN	Normal-force coefficient.
C_M	CM	Pitching-moment coefficient.
C_A	CA	Axial-force coefficient.
C_Y	CY	Side-force coefficient.
C_{YM}	CLN	Yawing-moment coefficient.
C_{RM}	CLL	Rolling-moment coefficient.

9.8. Complete-Configuration Comparison

Figures 9.1–9.3 compare the digitised NASA measurements with the complete SpearDAT configuration. The experimental points are shown at their digitised incidence locations, while the SpearDAT prediction is plotted through its native 0° to 24° schedule.

9.8.1. Longitudinal Agreement

The comparison is strongest for the complete vehicle at the transonic and supersonic test conditions. In particular, the predicted normal-force and pitching-moment curves reproduce both the experimental trends and their coefficient levels over most of the measured incidence range. A compact quantitative summary is provided by the root-mean-square error (RMSE) in [table 9.7](#). The values are coefficient units and are computed after interpolating the SpearDAT curve to the experimental angle-of-attack coordinates.

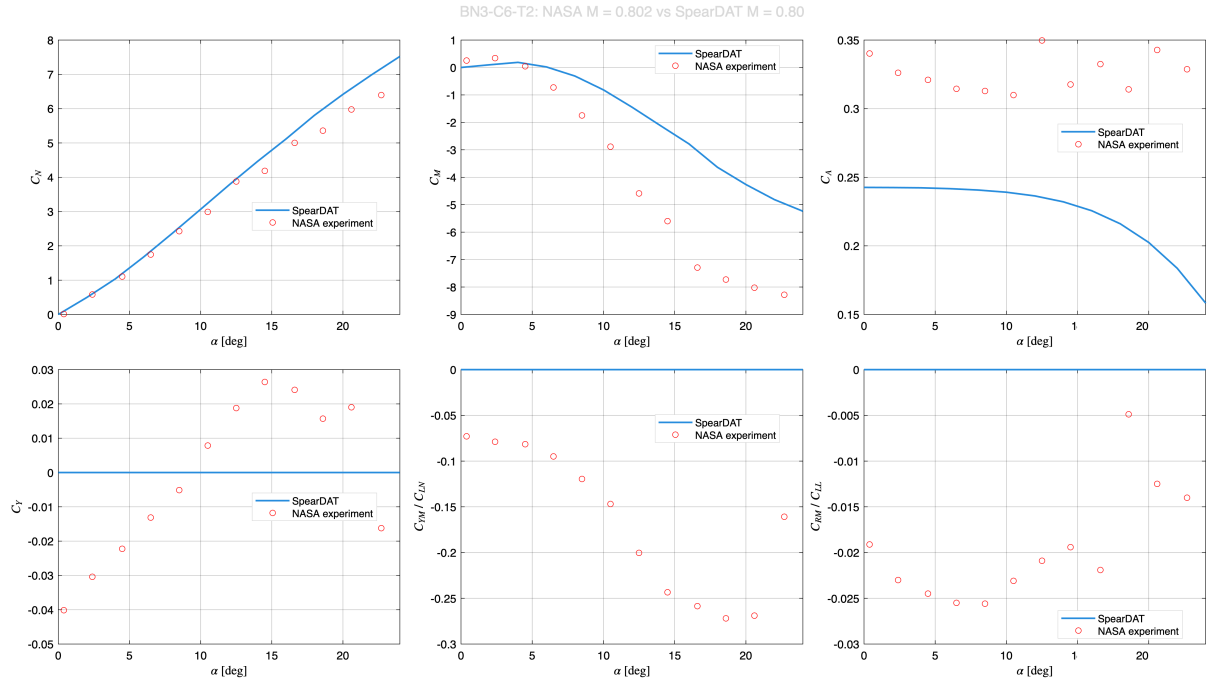


Figure 9.1.: BN3–C6–T2 comparison at NASA Mach 0.802 and SpearDAT Mach 0.80. The principal discrepancy is the subsonic pitching moment at moderate and high incidence.

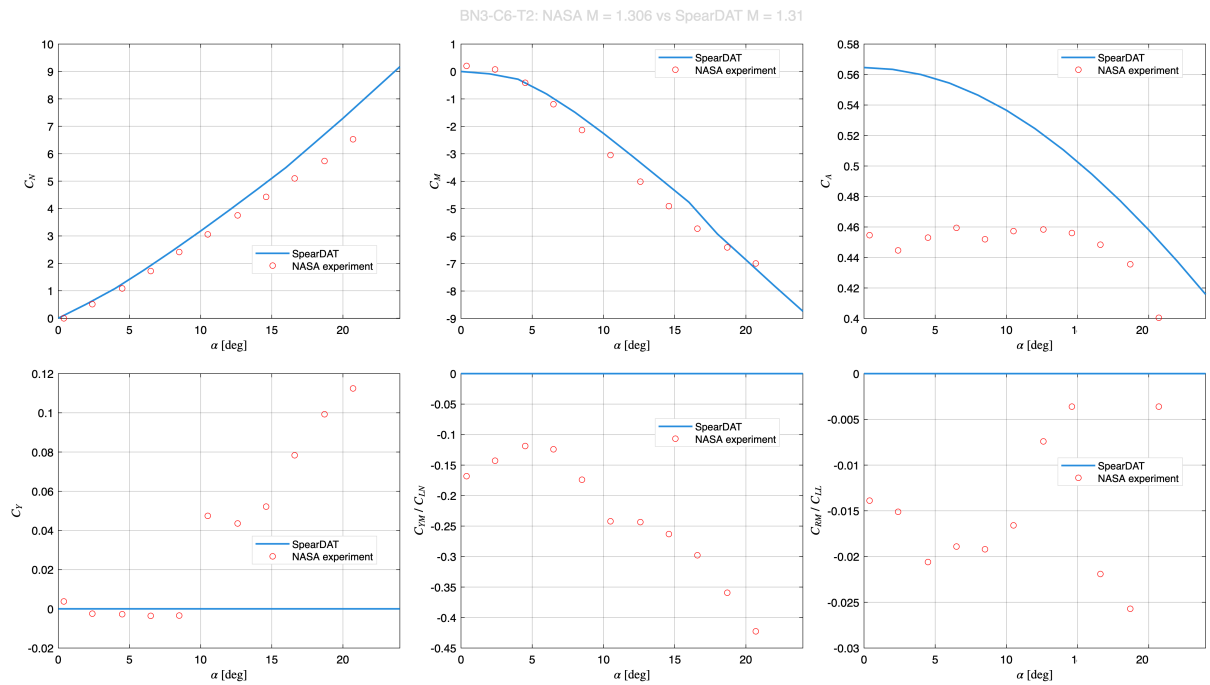


Figure 9.2.: BN3–C6–T2 comparison at NASA Mach 1.306 and SpearDAT Mach 1.31. The normal-force and pitching-moment predictions closely follow the experimental measurements over the tested incidence range.

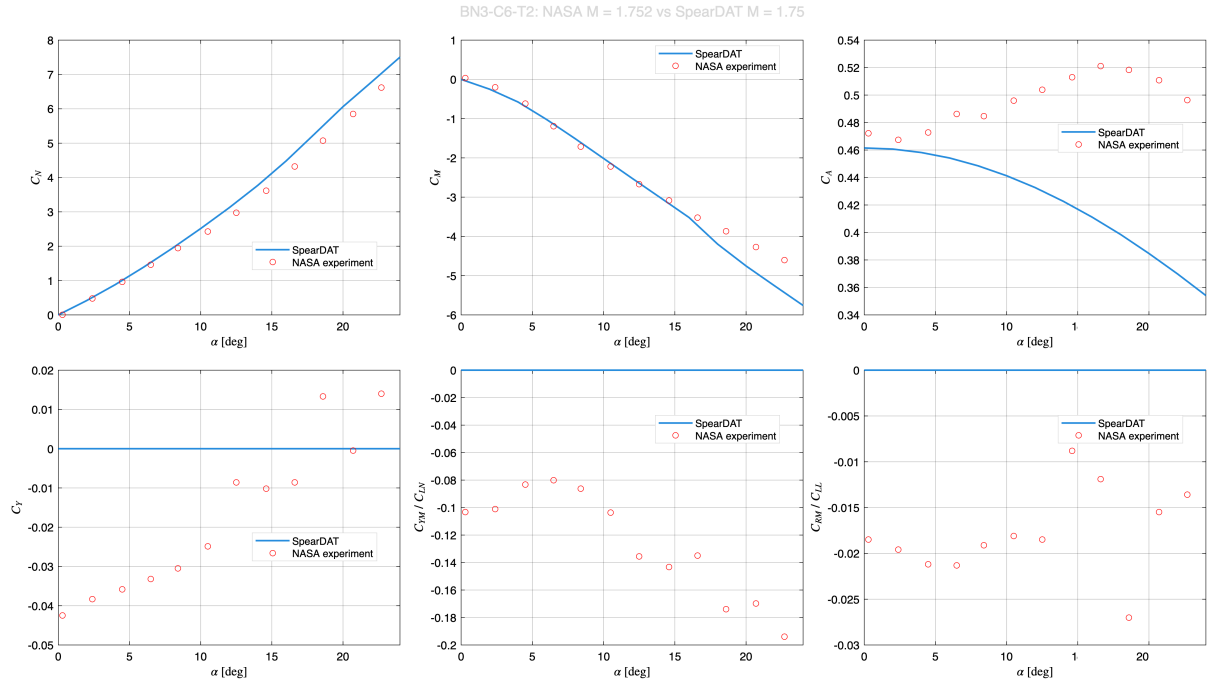


Figure 9.3.: BN3–C6–T2 comparison at NASA Mach 1.752 and SpearDAT Mach 1.75. The principal longitudinal trends and coefficient levels are reproduced closely, including the stabilising pitching-moment behaviour of the complete vehicle.

Table 9.7.: RMSE between SpearDAT configuration 7 and the digitised NASA BN3–C6–T2 main-balance data.

NASA Mach	SpearDAT Mach	C_N	C_M	C_A
0.802	0.80	0.385	2.630	0.103
1.306	1.31	0.544	0.442	0.081
1.752	1.75	0.293	0.349	0.083

The C_N comparison is consistently strong. Even at Mach 0.8, where the pitching moment is problematic, the predicted normal force follows the measured build-up well. At Mach 1.31 and 1.75 the C_M RMSE falls to approximately 0.44 and 0.35, respectively, and the calculated curves remain close to the wind-tunnel data across the incidence schedule. This is particularly important for the complete configuration because C_M depends not only on the total normal force but also on how the aerodynamic loading is distributed longitudinally between the body, canards, tails, and their interference contributions.

The axial-force prediction is less accurate quantitatively. SpearDAT captures the broad coefficient level and variation with incidence, but offsets of order 0.08–0.10 remain in the present comparison. Axial force combines several pressure-, friction-, base-, and lifting-surface contributions and is therefore a more demanding quantity for a preliminary-design method. The agreement in C_A should consequently be described as useful in trend and scale rather than point-for-point agreement.

Taken together, the transonic and supersonic results provide strong evidence that the SpearDAT/Missile DATCOM methodology captures the dominant longitudinal aerodynamics of this complete canard-controlled configuration. The comparison is especially compelling because no aerodynamic coefficients from the NASA experiment were supplied to the prediction: the curves follow from the independently reconstructed geometry, flight conditions, reference quantities, and the Missile DATCOM synthesis methods.

9.8.2. Lateral-Directional Coefficients

The lateral-directional panels require a different interpretation. The SpearDAT model is geometrically symmetric, the canards and tails are aligned in a cruciform arrangement, all panel deflections are zero, and the analysis is performed at $\beta = 0^\circ$ and $\phi = 0^\circ$. The deterministic prediction therefore returns zero C_Y , yawing moment, and rolling moment.

The wind-tunnel measurements contain small non-zero lateral-directional residuals. Such values can arise in a nominally symmetric experiment from model and balance tolerances and, particularly at incidence, from asymmetric separation and vortex development. The NASA report itself discusses asymmetric-flow behaviour in the test programme [3]. These residual measurements are useful evidence of the difference between an ideal symmetric aerodynamic model and a physical wind-tunnel article, but they are not an appropriate basis for judging the zero-sideslip deterministic lateral-directional prediction in the same way as C_N , C_M , and C_A .

9.9. Understanding the Mach 0.8 Pitching-Moment Difference

The principal exception to the otherwise strong complete-configuration comparison is the Mach 0.8 pitching moment. The discrepancy grows with incidence: SpearDAT predicts substantially less nose-down moment than the wind-tunnel data. This behaviour is not explained by an incorrect reference area, reference length, or moment-reference location; the same conventions are used in the NASA reduction and the SpearDAT case, and the much better agreement at Mach 1.3 and 1.75 would also be difficult to reconcile with a global reference-convention error.

The NASA component build-up provides a more useful diagnostic. Additional main-balance C_M curves were digitised for BN3–C6 and BN3–T2. The former can be compared directly with SpearDAT configuration 6, while BN3–T2 is used as experimental evidence for the stabilising influence of the aft tail. A compact comparison at representative angles is shown in [table 9.8](#).

Table 9.8.: Mach 0.8 pitching-moment build-up at representative incidence angles.

α	BN3–C6 C_M		BN3–C6–T2 C_M	
	NASA	SpearDAT cfg. 6	NASA	SpearDAT cfg. 7
10°	3.17	3.65	-2.60	-0.82
14°	3.84	4.76	-5.35	-2.12
20°	4.76	5.81	-7.93	-4.26

The BN3–C6 comparison shows that SpearDAT already predicts the forward body–canard combination as somewhat too destabilising. The larger difference, however, appears when the T2 tail is added. An effective tail contribution can be estimated from the change between the complete configuration and the body–canard configuration,

$$\Delta C_{M,T2} = C_{M,BN3C6T2} - C_{M,BN3C6}.$$

This quantity includes the tail and the interference changes caused by adding it; it is not an isolated-tail coefficient.

At 20°, the experiment gives an effective addition of approximately –12.69, whereas SpearDAT gives approximately –10.08. Thus, the subsonic synthesis produces about 2.6 coefficient units less stabilising pitching moment from the addition of the aft surface and its associated interference. Combined with the approximately 1.1-unit overprediction of the destabilising BN3–C6 moment at the same incidence, this accounts for most of the complete-vehicle discrepancy.

This behaviour is strongly Mach dependent. Repeating the same incremental comparison at Mach 1.75 gives an effective tail addition at $\alpha = 20^\circ$ of approximately –8.19 from the experiment and –8.34 from SpearDAT. The close supersonic increment makes a simple geometry, sign, or reference-location error unlikely. Instead, the evidence is consistent with a limitation in the subsonic prediction of the local tail loading and/or the body–canard–tail interference field. The NASA programme was itself concerned with the effect of canard-generated flow on the aft surfaces, making this a physically plausible region in which a semi-empirical preliminary-design method can depart from experiment [3].

Note

The diagnostic above identifies where the Mach 0.8 pitching-moment difference enters the configuration build-up; it does not prove which individual semi-empirical correlation is responsible. Separating body–fin carryover, canard-vortex influence, local tail incidence, and other synthesis terms would require a method-level validation study beyond the scope of this User Guide.

9.10. Validation with a Fixed Canard Deflection

The zero-deflection comparison tests the underlying complete-vehicle aerodynamics, but it does not by itself show whether the predicted response to a control command is realistic. The NASA programme also tested BN3–C6–T2 with the two side canards at a physical 15° pitch-pair deflection,

$$D_1 = D_3 = 0^\circ, \quad D_2 = D_4 = +15^\circ,$$

while the T2 tails remained fixed. This is the largest fixed canard magnitude in the published deflection series and therefore provides a useful additional test of control effectiveness [3].

As described in [section 7.9.6](#), the equal positive NASA panel-local deflections do not map to equal raw Missile DATCOM signs. For the cruciform C6 installation the corresponding SpearDAT fixed-deflection input is $[0, 15, 0, -15]$. The comparison therefore tests the same physical pitch-pair motion rather than merely reproducing the signed numbers printed in the wind-tunnel report.

The experimental total main-balance coefficients were digitised from the 15° BN3–C6–T2 figures at Mach 0.802, 1.302, and 1.753. SpearDAT was run at the same three Mach numbers so that each calculation corresponds directly to the wind-tunnel condition shown in the NASA figures.

Note

The Mach column in `result.aerodynamics` is recovered from the `for003` zone title, where Mach is written to two decimal places. Consequently the three SpearDAT conditions appear in that table as 0.80, 1.30, and 1.75 even though the generated input retains the more precise Mach values above. This output-formatting precision does not round the Mach number used by the aerodynamic calculation.

Figures 9.4–9.6 compare the resulting complete configuration with the digitised wind-tunnel measurements.

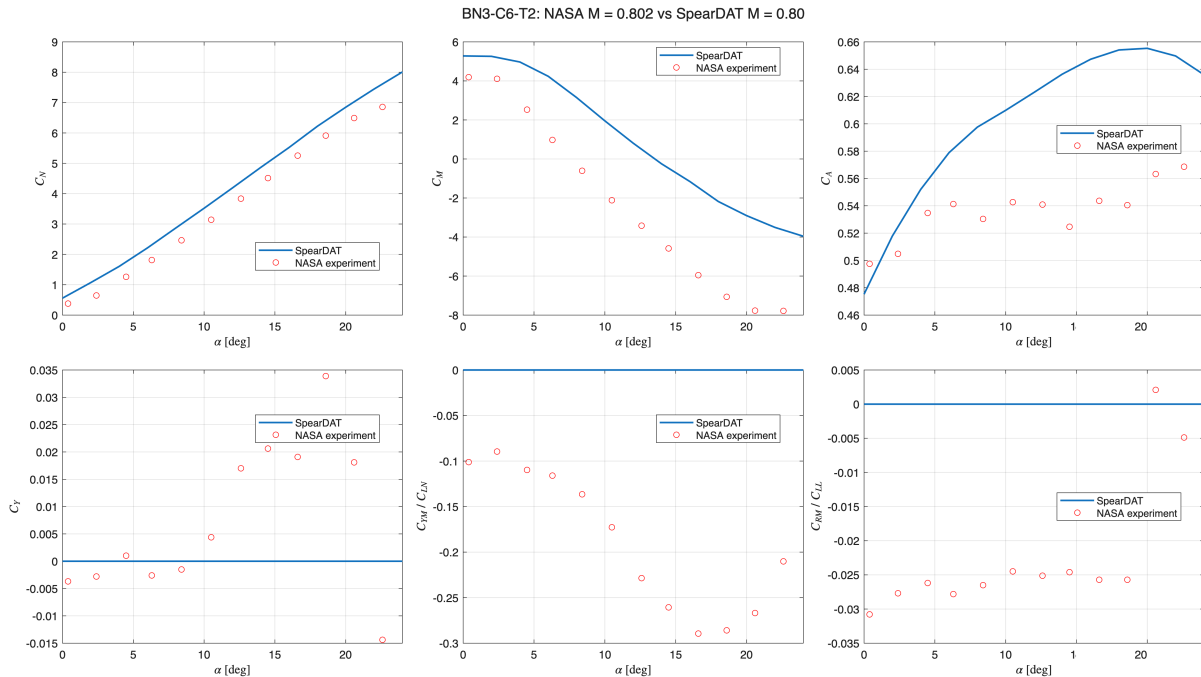


Figure 9.4.: BN3–C6–T2 comparison with $D_2 = D_4 = 15^\circ$ at Mach 0.802. The canard-induced force and moment shifts are captured, but the previously identified subsonic pitching-moment discrepancy remains at moderate and high incidence.

As for the zero-deflection case, the SpearDAT curves were interpolated only in angle of attack to the digitised experimental marker locations before calculating the residuals. The resulting longitudinal RMSE values are listed in [table 9.9](#).

The normal-force response remains strong at all three conditions. The C_N RMSE is approximately 8% of the measured coefficient range at Mach 0.802, 10% at Mach 1.302, and 5% at Mach 1.753. More importantly for a control-effectiveness test, SpearDAT reproduces the substantial increase in normal force produced by the 15° canard command and follows its variation with incidence.

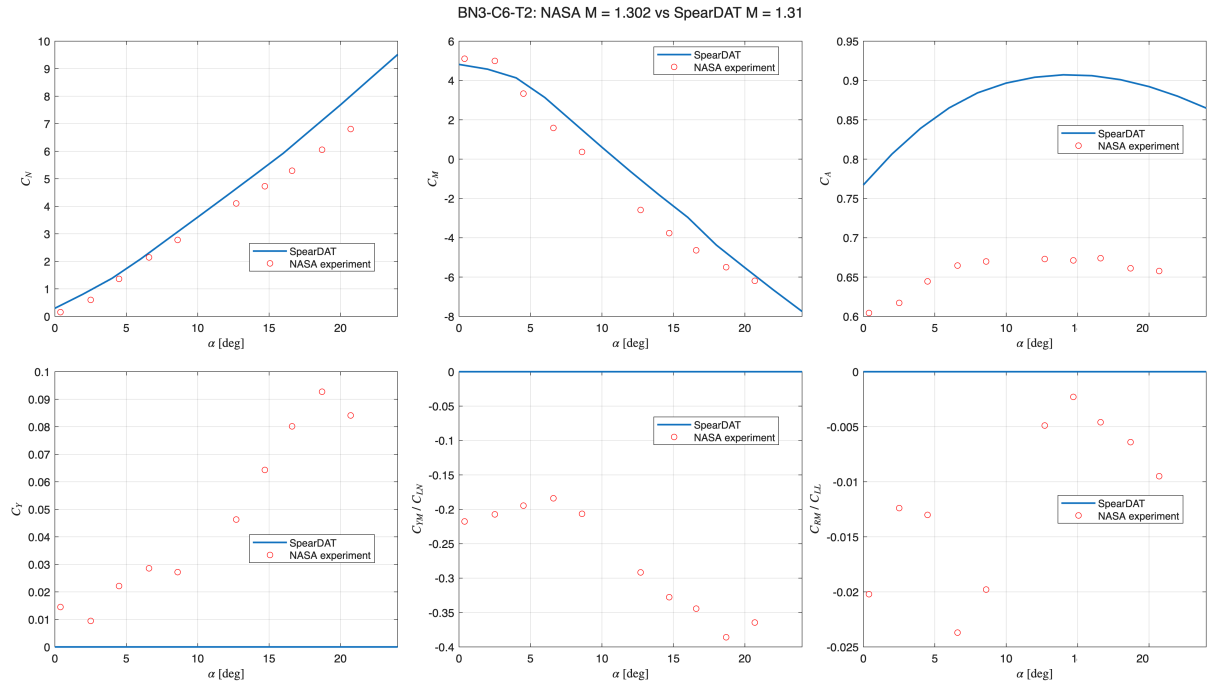


Figure 9.5.: BN3-C6-T2 comparison with $D_2 = D_4 = 15^\circ$ at Mach 1.302. SpearDAT reproduces the principal normal-force and pitching-moment response to the large canard command, while the axial-force level remains more approximate.

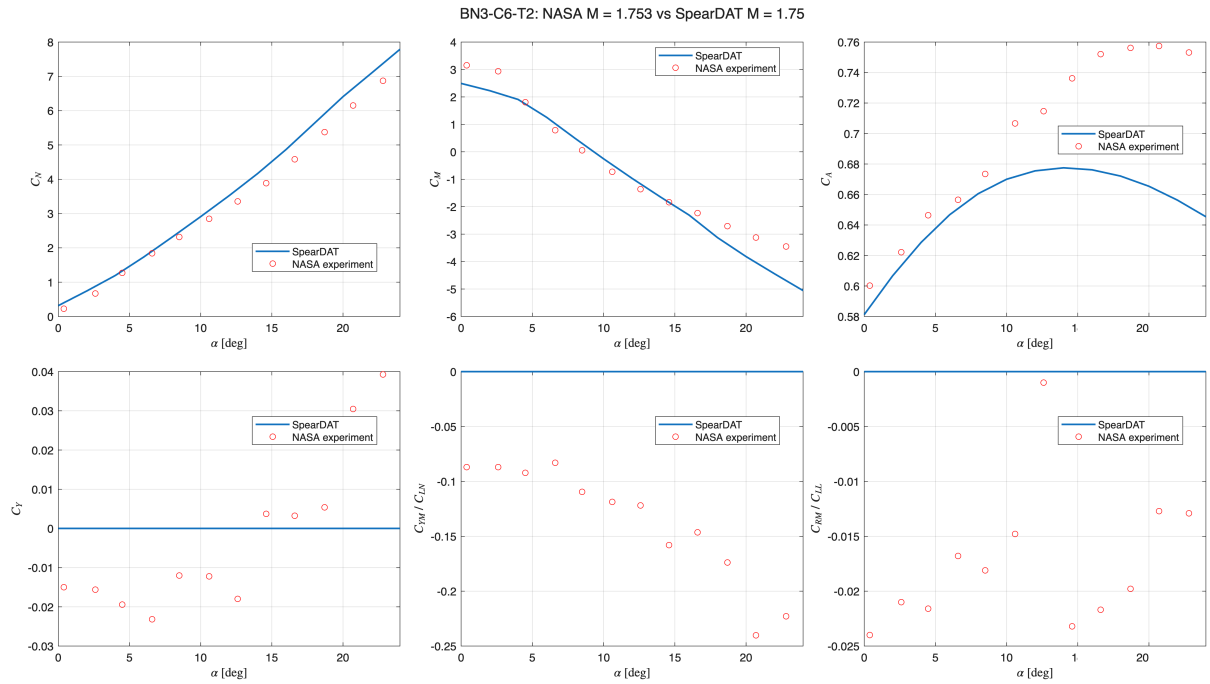


Figure 9.6.: BN3-C6-T2 comparison with $D_2 = D_4 = 15^\circ$ at Mach 1.753. The longitudinal force and moment response is reproduced closely across the measured incidence range.

Table 9.9.: RMSE between SpearDAT configuration 7 and the digitised NASA BN3–C6–T2 data with $D_2 = D_4 = 15^\circ$.

NASA Mach	SpearDAT Mach	C_N	C_M	C_A
0.802	0.802	0.527	3.617	0.077
1.302	1.302	0.664	1.000	0.213
1.753	1.753	0.364	0.597	0.057

The pitching-moment result reinforces the conclusion drawn from the zero-deflection configuration. At Mach 0.802 the commanded canards produce the correct large positive shift in C_M at low incidence, showing that the control direction and the forward-canard moment are being represented. As incidence increases, however, the calculation again becomes insufficiently nose-down relative to the experiment. The resulting C_M RMSE of 3.62 is therefore consistent with the same subsonic complete-configuration limitation identified in [section 9.9](#), rather than indicating a reversal or failure of the canard-deflection model.

At the higher Mach numbers the pitching-moment comparison is substantially better. The RMSE falls to about 1.00 at Mach 1.302 and 0.60 at Mach 1.753, while the calculation reproduces the experimentally observed change from a strongly positive low-incidence pitching moment to a negative moment as incidence increases. A separate comparison of the change from the zero-deflection case is particularly informative at Mach 1.75: the RMSE of the predicted control-induced increments is approximately 0.087 in C_N , 0.324 in C_M , and 0.028 in C_A . Thus the effect of applying the large canard command is reproduced more closely than the absolute coefficient levels alone might suggest.

The axial-force comparison is mixed. Agreement is close at Mach 0.8 and 1.75, with RMSE values of approximately 0.08 and 0.06, but the Mach 1.3 result contains a systematic positive offset of roughly two-tenths in C_A . Accordingly, the deflected case supports the same interpretation as the zero-deflection comparison: SpearDAT is particularly effective at predicting the dominant normal-force and pitching-moment behaviour, whereas axial force should be treated more cautiously when quantitative accuracy is required.

The experimental lateral-directional coefficients again contain small non-zero residuals while the ideal symmetric SpearDAT configuration returns zero. The interpretation given in [section 9.8.2](#) therefore also applies to the deflected pitch-pair case: the physical control command is symmetric in pitch and does not intentionally introduce a lateral-directional input.

Overall, the 15° comparison is an important extension of the zero-deflection validation. It shows that the agreement is not confined to one uncontrolled configuration: the solver captures the magnitude and direction of a large canard-control response, especially at the Mach 1.3 and 1.75 conditions, while independently reproducing the same Mach-0.8 pitching-moment limitation already isolated through the component build-up.

9.11. Extended Validation with NASA/TM-2005-213541

The preceding comparison is based on the 1977 NASA canard-missile programme used throughout the worked examples in Chapters 5–8. A second, independent NASA data set provides a useful extension because it exercises a different vehicle, a much wider Mach range, aerodynamic roll, tail control, and the interference between a body and a cruciform tail.

In 2005, Jerry M. Allen published NASA/TM-2005-213541, *Aerodynamics of an Axisymmetric Missile Concept Having Cruciform Strakes and In-Line Tail Fins From Mach 0.60 to 4.63* [4]. The programme was created to investigate control-tail loading in the presence of upstream aerodynamic interference. The generic model could be rolled remotely and its four tail fins could be deflected while the tunnel was running. Tests were performed in the NASA Langley 8-Foot Transonic Pressure Tunnel and in both test sections of the Unitary Plan Wind Tunnel. Across the complete programme the principal variables included Mach number, angle of attack, model roll angle, configuration build-up, strake length, nose size, and tail-fin deflection.

The report is especially valuable for validation because it records both six-component main-balance measurements of the complete configuration and three-component measurements on each of the four tail fins. The complete project contains 1059 wind-tunnel runs. Rather than requiring values to be read from printed graphs, NASA supplied the full data set electronically with the report in two ASCII forms: a column-tabulated format with run headers and a free-format SIF representation. The present comparison reads the original column-tabulated data directly. The experimental values in this section are therefore not digitised from plots.

9.11.1. Test Article and Selected Configuration

The NASA programme combined a small or large nose, optional long or short strakes, and an optional cruciform tail to produce the seven configurations shown in figure 9.7. Configuration 1, with the small nose, long strakes, and tails, was treated as the baseline test article. For the SpearDAT comparison here, however, Configuration 3 is more useful: it retains the small-nose body and the four tails but removes the upstream strakes. NASA identifies this arrangement as SN-no S-T. Configuration 6 uses the same small-nose body with both the strakes and tails removed and is therefore the corresponding body-alone reference.

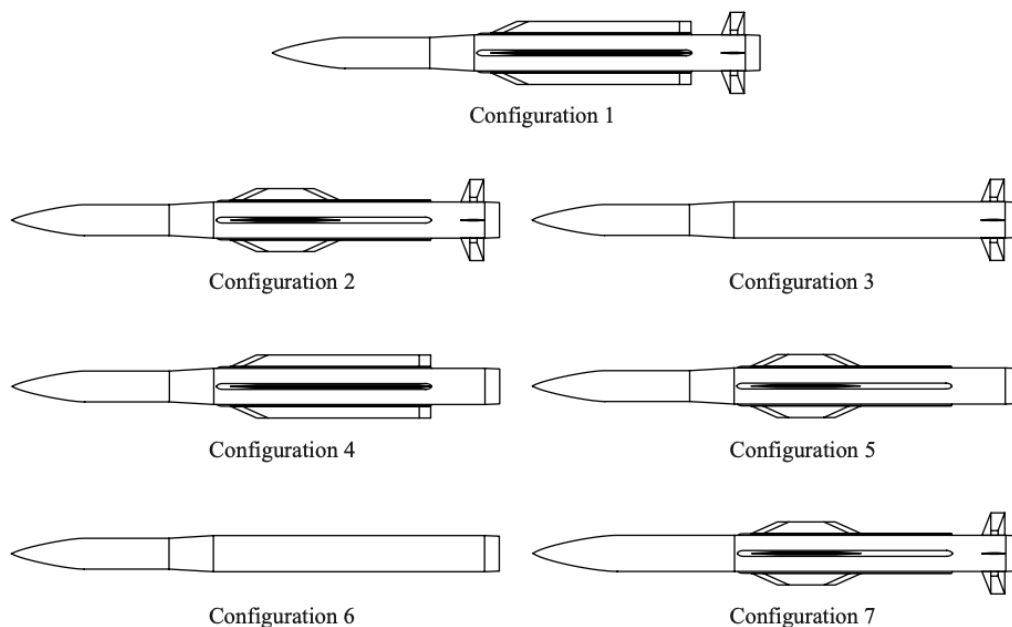


Figure 9.7.: Seven configurations tested in the NASA 2005 strake-tail programme. Configuration 3, used for the principal SpearDAT comparison in this section, is the small-nose body with no strakes and with the cruciform tail installed. Reproduced from Figure 5 of NASA/TM-2005-213541 [4].

The source report provides the body, nose, strake, mounting-pad, and tail dimensions explicitly. The most relevant drawings are retained as context in figures 9.8 to 9.10. The maximum body diameter is 3.15 in and the complete body length is 42.528 in. With the baseline small nose installed, the body is predominantly 3.15 in in diameter aft of the forward transition, followed by a short boattail to the 3.04 in base. NASA reduced the complete-configuration coefficients using the maximum body cross-sectional area, 0.0541 ft², the body diameter as the reference length, and a moment-reference station of 23.387 in.

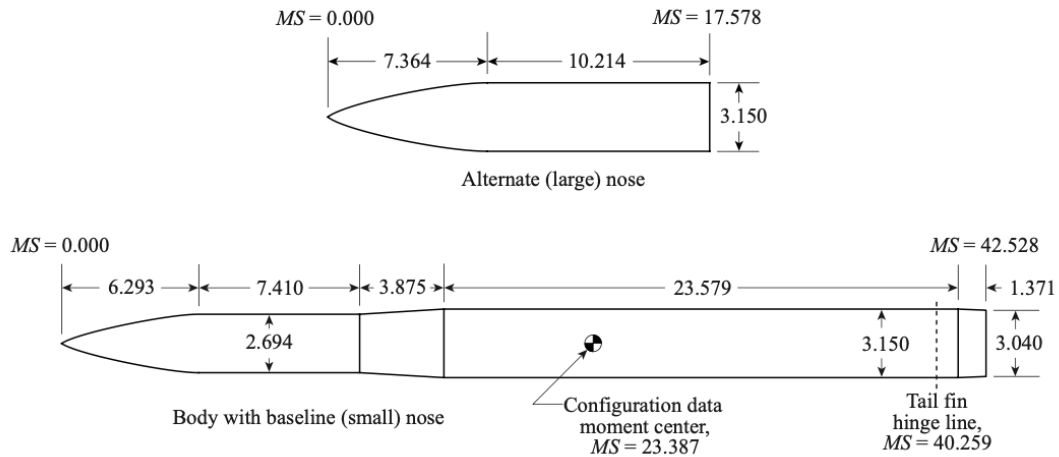


Figure 9.8.: NASA 2005 body and nose geometry, including the main-balance moment reference and the tail hinge station. Dimensions are in inches. Reproduced from Figure 6 of NASA/TM-2005-213541 [4].

The Configuration 3 SpearDAT reconstruction follows these published dimensions. The four identical tails have an exposed span of 1.953 in, root and tip chords of 1.992 and 1.213 in, and root and tip leading-edge stations of 39.165 and 39.795 in. The four panels are clocked at 0°, 90°, 180°, 270°. NASA gives a nominal root thickness of 0.110 in and states that the nominally sharp leading and trailing edges had a small radius of about 0.005 in. The exact chordwise break locations of the hexagonal section are not dimensioned in the report; the SpearDAT reconstruction therefore retains the documented 25–50–25 percent chord engineering approximation for that section shape. No experimental aerodynamic coefficients are used to tune the geometry or the prediction.

Table 9.10.: NASA 2005 configurations used for the body–tail build-up comparison.

NASA config.	NASA code	Physical model	Role in comparison
3	SN-no S-T	Small nose, strakes off, tails on	Body plus cruciform tail.
6	SN-no S-no T	Small nose, strakes off, tails off	Matched body-alone reference.

9.11.2. Digital Data and Comparison Procedure

All NASA runs are angle-of-attack sweeps at a fixed Mach number and fixed model roll angle for that run. The comparison reads the original NASA ASCII database, selects Configuration 3, and reproduces the corresponding Mach number, model roll angle, and tail-control setting

in SpearDAT. The complete NASA database contains 1059 runs and 11074 tabulated points. The Configuration 3 subset used here contains 103 runs and 1031 experimental points. The SpearDAT comparison covers the seven Mach numbers

$$0.60, \quad 0.90, \quad 1.19, \quad 1.70, \quad 2.36, \quad 2.86, \quad 3.95,$$

with the NASA unit Reynolds number of approximately $1.5 \times 10^6 \text{ ft}^{-1}$. Across the full Configuration 3 exercise, 1005 experimental locations lie within the calculated incidence ranges and are used in the point-by-point residual statistics.

The main-balance coefficient mapping is

$$\begin{aligned} \text{CNF} &\rightarrow C_N, & \text{CPM} &\rightarrow C_M, & \text{CAF} &\rightarrow C_A, \\ \text{CSF} &\rightarrow C_Y, & \text{CYM} &\rightarrow C_{LN}, & \text{CRM} &\rightarrow C_{LL}. \end{aligned}$$

SpearDAT is evaluated on its native incidence schedule; for numerical error metrics only, the calculated coefficient is linearly interpolated to the actual NASA incidence coordinate. Experimental data are not extrapolated.

The roll-angle comparisons require one additional convention check. NASA states that its six-component main-balance coefficients are presented in body axes. SpearDAT's "TotalAlphaRoll" mode represents the same total incidence and aerodynamic roll through its own resolved body-axis components. Before a direct comparison is made, the transverse SpearDAT force components are therefore expressed in the NASA reporting convention,

$$\begin{bmatrix} C_N^{\text{NASA}} \\ C_Y^{\text{NASA}} \end{bmatrix} = \begin{bmatrix} \cos \phi & -\sin \phi \\ \sin \phi & \cos \phi \end{bmatrix} \begin{bmatrix} C_N^{\text{SpearDAT}} \\ C_Y^{\text{SpearDAT}} \end{bmatrix},$$

with the corresponding transformation applied to pitching and yawing moment. For this model the longitudinal and lateral reference lengths are both the 3.15 in body diameter, so the moment transformation has the same numerical rotation. Axial force and rolling moment are unchanged by this rotation about the longitudinal axis.

9.11.3. Zero-Roll Longitudinal Comparison

The first comparison uses undeflected Configuration 3 at $\phi = 0^\circ$. At this symmetry condition the deterministic SpearDAT model has essentially zero lateral-directional coefficients, so the principal comparison is in C_N , C_M , and C_A . Unlike the selected examples used later for the aerodynamic-roll discussion, the full zero-roll Mach sequence is reproduced here because it provides a compact view of how the longitudinal agreement changes across the complete analysed speed range. The seven conditions are shown in [figures 9.11 to 9.17](#), and the corresponding RMSE values are listed in [table 9.11](#).

The normal-force prediction follows the experimental build-up well at most conditions, with particularly small errors from Mach 1.70 upward except for the transonic Mach-1.19 condition. Axial force remains close in coefficient level throughout the complete sequence. Pitching moment requires a more qualified interpretation: the absolute C_M agreement is useful through the lower part of the Mach range but becomes progressively less accurate at the higher supersonic conditions. Because pitching moment is sensitive to the longitudinal location of the aerodynamic loading, this does not by itself establish that the tail normal force or the body–tail interference is being predicted incorrectly.

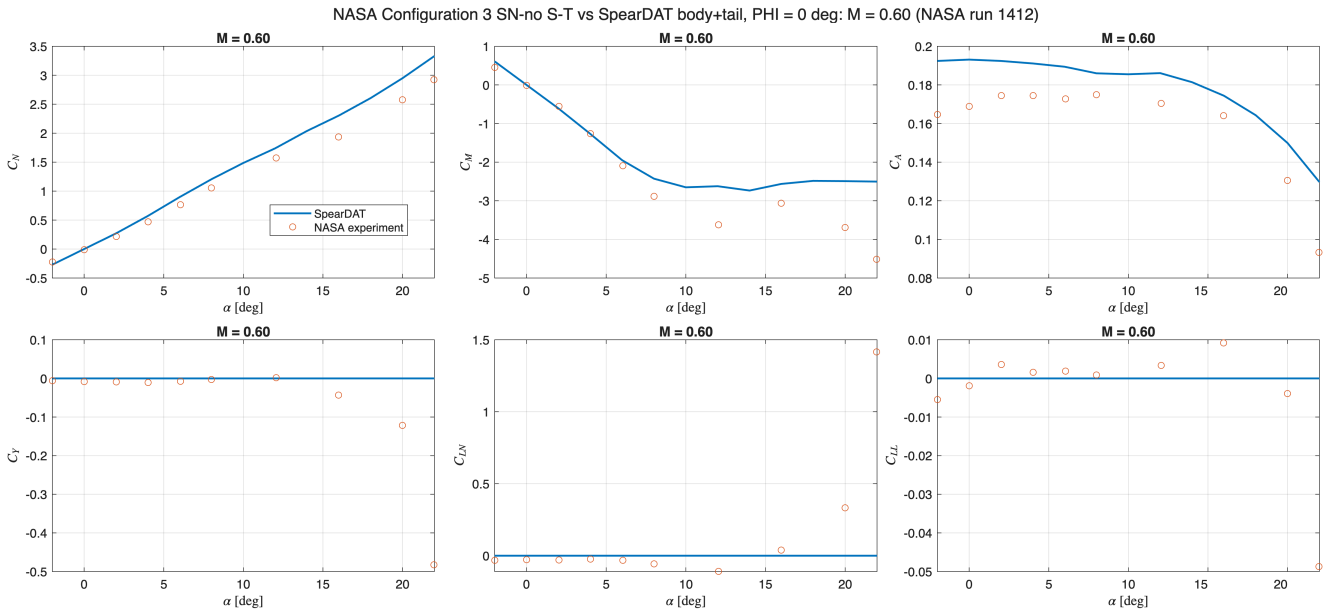


Figure 9.11.: NASA 2005 Configuration 3 compared with SpearDAT at $M = 0.60$, zero model roll, and zero tail deflection. The calculation follows the principal normal-force and axial-force trends, with a larger but still systematic difference in pitching moment.

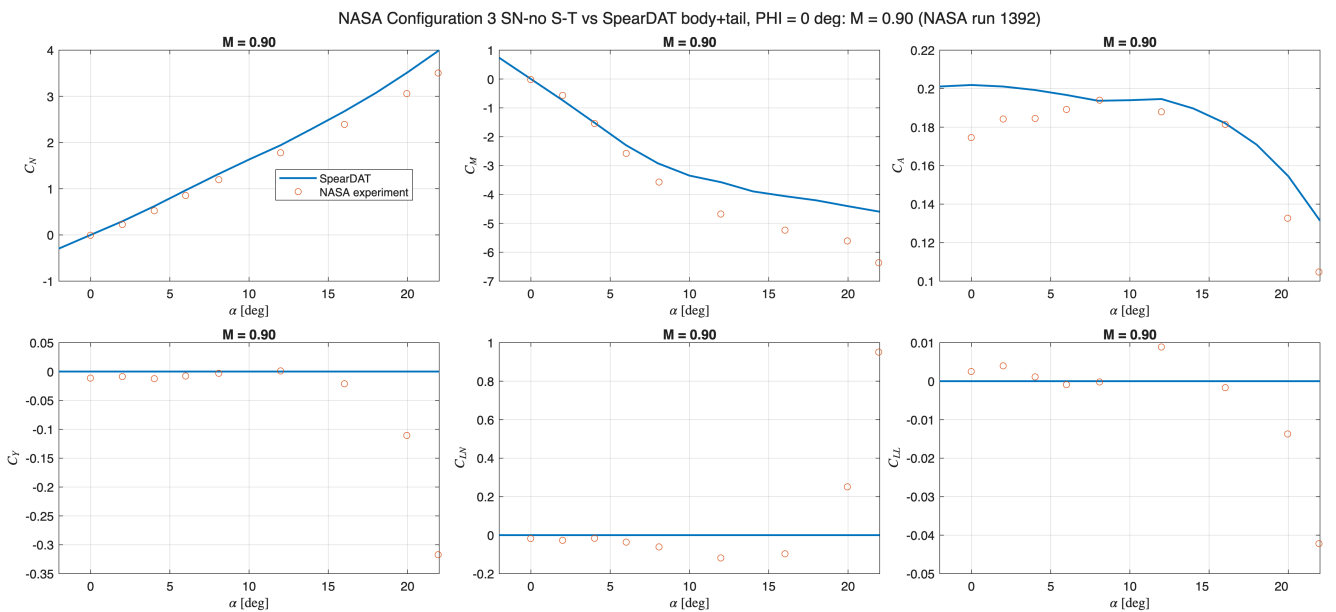


Figure 9.12.: NASA 2005 Configuration 3 compared with SpearDAT at $M = 0.90$, zero model roll, and zero tail deflection. The normal-force and pitching-moment trends are reproduced closely over most of the measured incidence range, while the measured lateral-directional residuals remain small relative to the longitudinal coefficients.

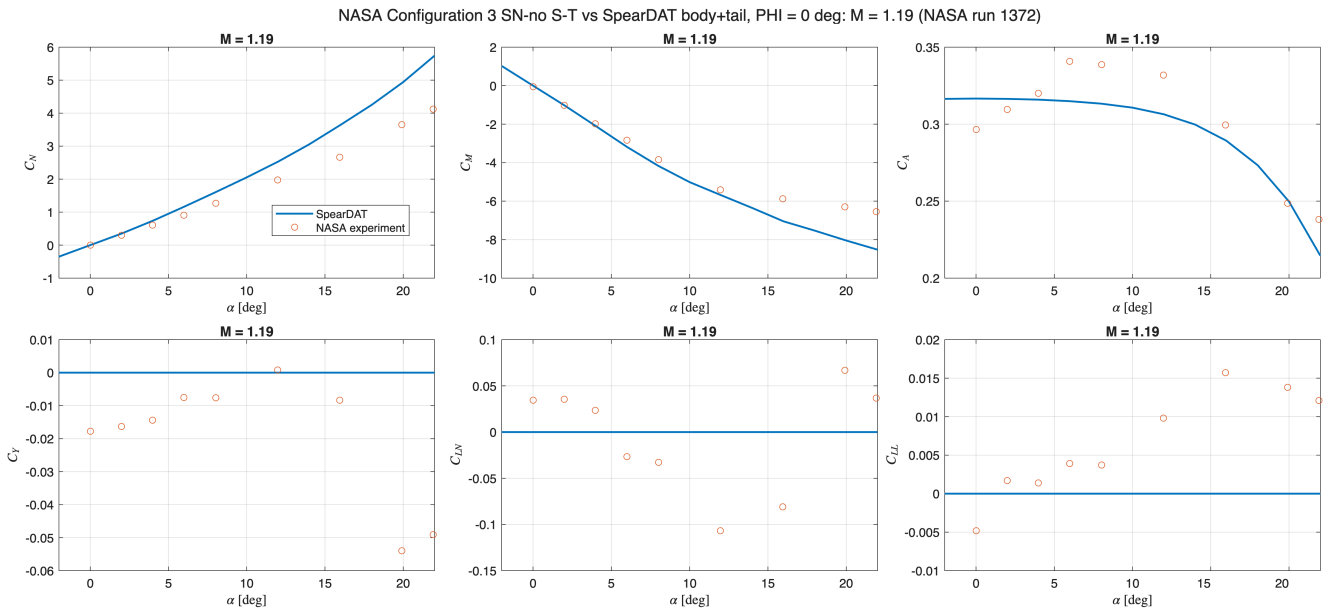


Figure 9.13.: NASA 2005 Configuration 3 compared with SpearDAT at $M = 1.19$, zero model roll, and zero tail deflection. This transonic condition shows a larger normal-force difference than the neighbouring Mach cases while retaining close axial-force agreement.

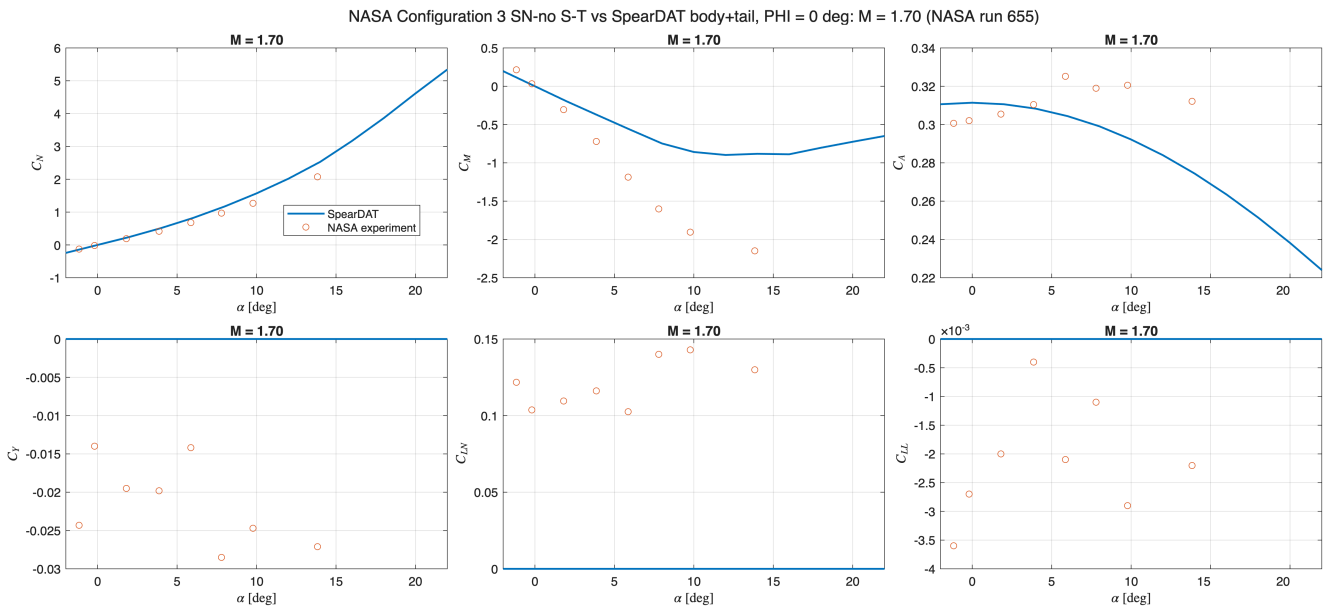


Figure 9.14.: NASA 2005 Configuration 3 compared with SpearDAT at $M = 1.70$, zero model roll, and zero tail deflection. The principal longitudinal trends and coefficient levels are reproduced closely through the measured low-supersonic incidence range.

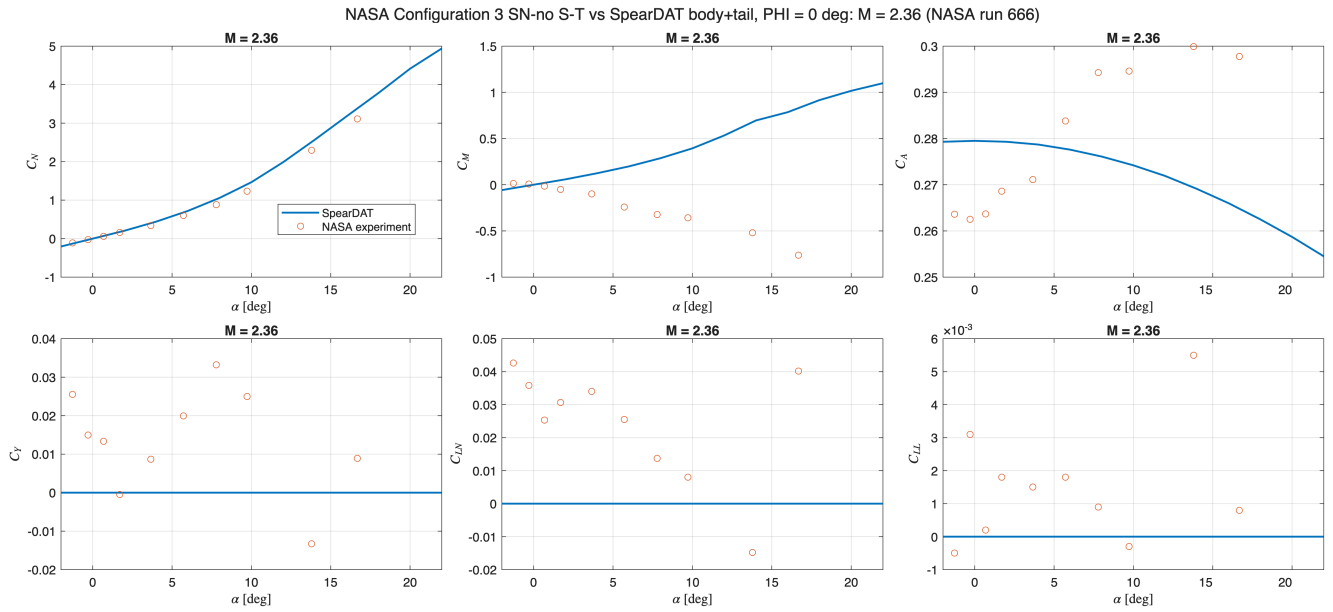


Figure 9.15.: NASA 2005 Configuration 3 compared with SpearDAT at $M = 2.36$, zero model roll, and zero tail deflection. Normal force and axial force retain close trend and level agreement, while the absolute pitching moment shows the high-supersonic departure examined through the component build-up below.

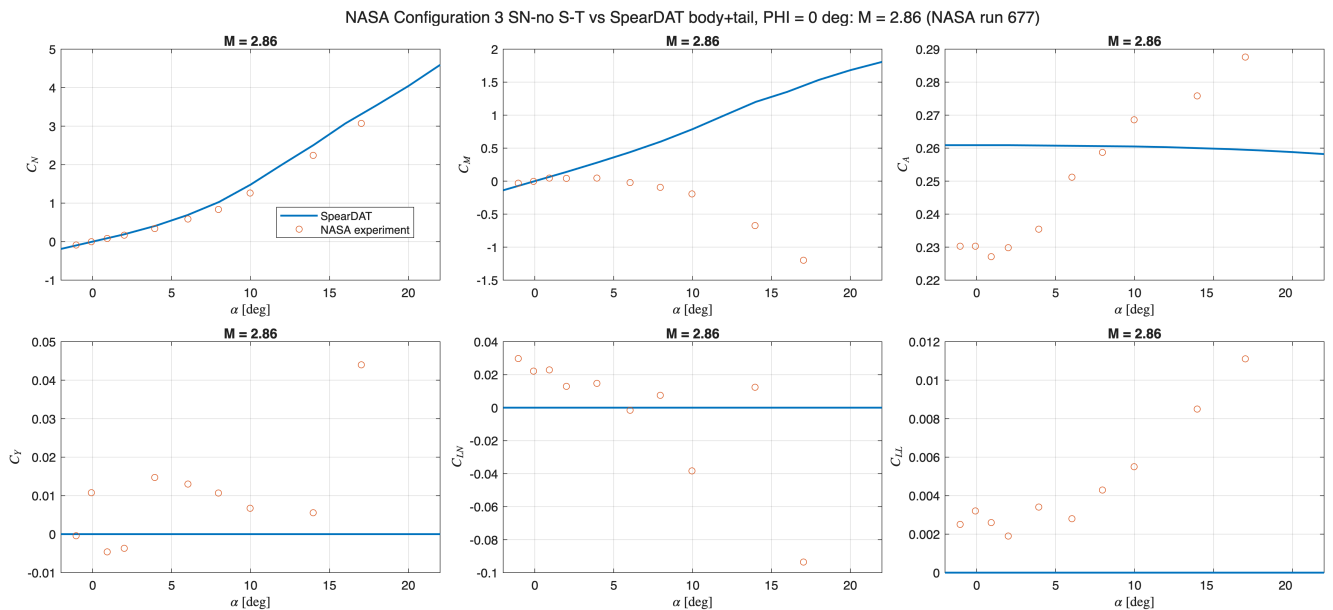


Figure 9.16.: NASA 2005 Configuration 3 compared with SpearDAT at $M = 2.86$, zero model roll, and zero tail deflection. The normal-force and axial-force predictions remain close while the body-derived pitching-moment difference becomes more evident at moderate and high incidence.

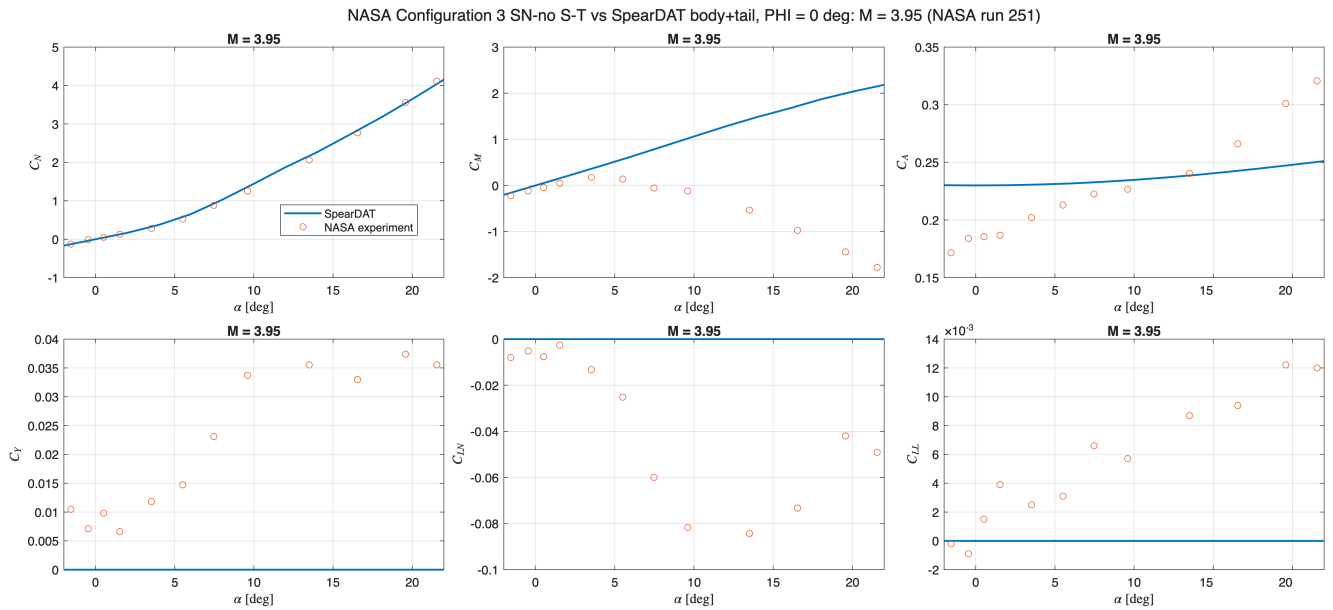


Figure 9.17.: NASA 2005 Configuration 3 compared with SpearDAT at $M = 3.95$, zero model roll, and zero tail deflection. Normal force remains very close to experiment, whereas the absolute pitching moment exhibits the largest high-supersonic discrepancy in the seven-Mach sequence.

Table 9.11.: RMSE for the undeflected NASA 2005 Configuration 3 body–tail comparison at zero model roll.

Mach	C_N	C_M	C_A
0.60	0.229	0.837	0.021
0.90	0.255	0.927	0.017
1.19	0.784	0.972	0.018
1.70	0.189	0.710	0.020
2.36	0.134	0.715	0.019
2.86	0.148	1.105	0.024
3.95	0.057	1.840	0.040

9.11.4. Tail Increment and the High-Mach Pitching Moment

The NASA configuration build-up makes that distinction testable. Subtracting Configuration 6 from Configuration 3 forms the effective increment produced by adding the tail,

$$\Delta C = C_{\text{body+tail}} - C_{\text{body}}.$$

This is not an isolated fin-alone coefficient: it includes the tail load and the change in body loading created by body–tail interference. The same subtraction is applied to the corresponding SpearDAT body-alone and body–tail configuration levels.

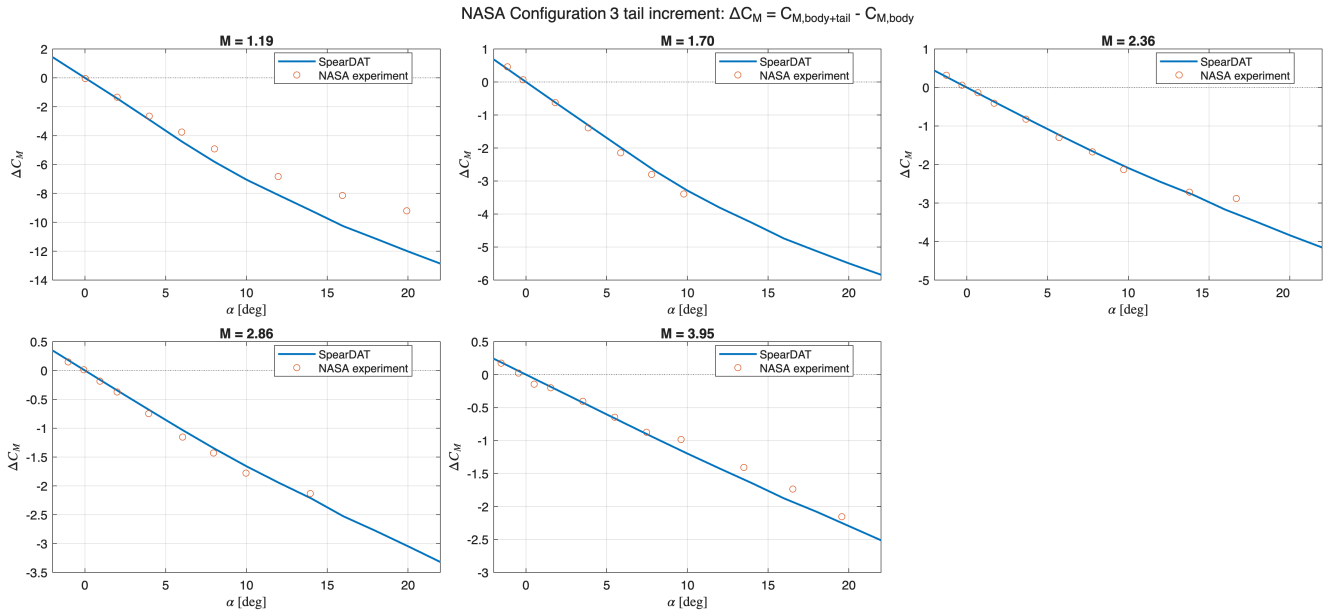


Figure 9.18.: Effective tail-plus-interference pitching-moment increment for NASA Configuration 3 relative to the matched Configuration 6 body. NASA increments are formed only over the incidence range measured in both wind-tunnel runs; no experimental extrapolation is used. The high-supersonic increments from Mach 1.70 to 3.95 are reproduced particularly closely.

Table 9.12.: RMSE of the effective NASA 2005 tail-plus-interference increment.

Mach	Common NASA points	ΔC_N	ΔC_M	ΔC_A
0.60	8	0.124	0.311	0.011
0.90	7	0.117	0.133	0.029
1.19	8	0.413	1.372	0.021
1.70	7	0.035	0.115	0.023
2.36	10	0.046	0.133	0.006
2.86	9	0.036	0.075	0.006
3.95	11	0.048	0.103	0.012

This diagnostic changes the interpretation of the high-Mach absolute pitching-moment difference. From Mach 1.70 through 3.95 the incremental ΔC_M RMSE remains between approximately 0.075 and 0.133, even while the absolute Configuration 3 C_M error grows. The tail and the associated body–tail interference contribution are therefore being reproduced much more closely than the total pitching moment alone suggests. The residual difference is principally inherited from the body-alone pitching-moment prediction.

The source methodology is consistent with this behaviour: the standard supersonic body calculation uses a simplified finite-incidence representation of the longitudinal body load distribution. The wind-tunnel comparison indicates reduced fidelity in that load location at the higher supersonic conditions, rather than a general loss of tail effectiveness. A method-level analysis is beyond the purpose of this chapter; the important validation result here is that the component build-up isolates the discrepancy while the high-Mach tail-plus-interference increment remains strong. The Mach 1.19 increment is a notable exception and indicates some cancellation between body and tail-build-up errors at that condition.

9.11.5. Aerodynamic Roll and Body–Tail Interference

Configuration 3 is axisymmetric only in its body. The complete vehicle contains four discrete tail panels, so rolling the model changes the circumferential orientation of those panels relative to the incidence plane. At $\phi = 0^\circ, \pm 45^\circ$, and $\pm 90^\circ$ the cruciform again passes through a symmetry orientation. At intermediate roll angles the total transverse loading is redistributed between the four fins and their body interference fields. The result can contain non-zero side force, yawing moment, and rolling moment even when the overall normal-force magnitude changes only modestly.

The complete roll analysis was performed at all seven Mach numbers used in the Configuration 3 comparison. Four representative conditions are reproduced in figures 9.19 to 9.22. They span the subsonic, transonic, and low-supersonic portions of the database and show the roll-dependent six-component behaviour before the high-supersonic body pitching-moment discrepancy discussed in section 9.11.4 increasingly dominates the C_M panel. The Mach 2.36, 2.86, and 3.95 roll sweeps were nevertheless retained in the numerical assessment and are included in the all-Mach error summary in table 9.13; their omission here is only a figure-selection choice, not an exclusion from the validation.

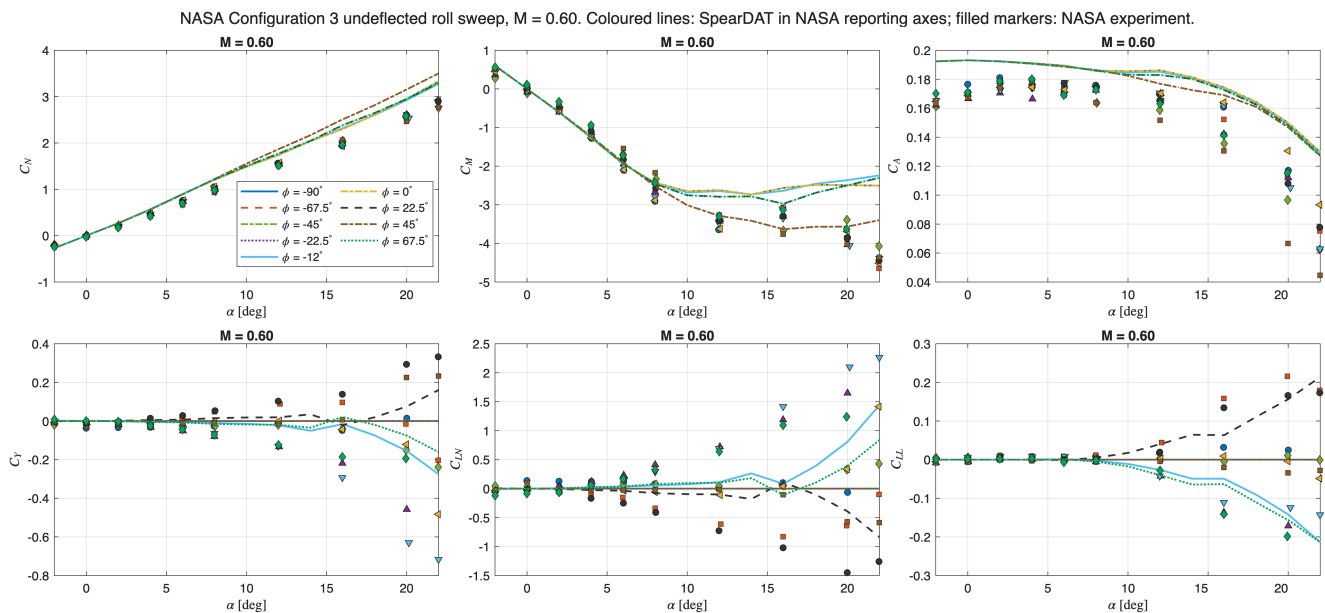


Figure 9.19.: Undelected NASA Configuration 3 aerodynamic-roll sweep at $M = 0.60$. SpearDAT is shown after expressing the transverse components in the NASA reporting convention. The comparison demonstrates how the cruciform tail redistributes load into lateral and rolling components while the overall normal-force response remains comparatively insensitive to model roll.

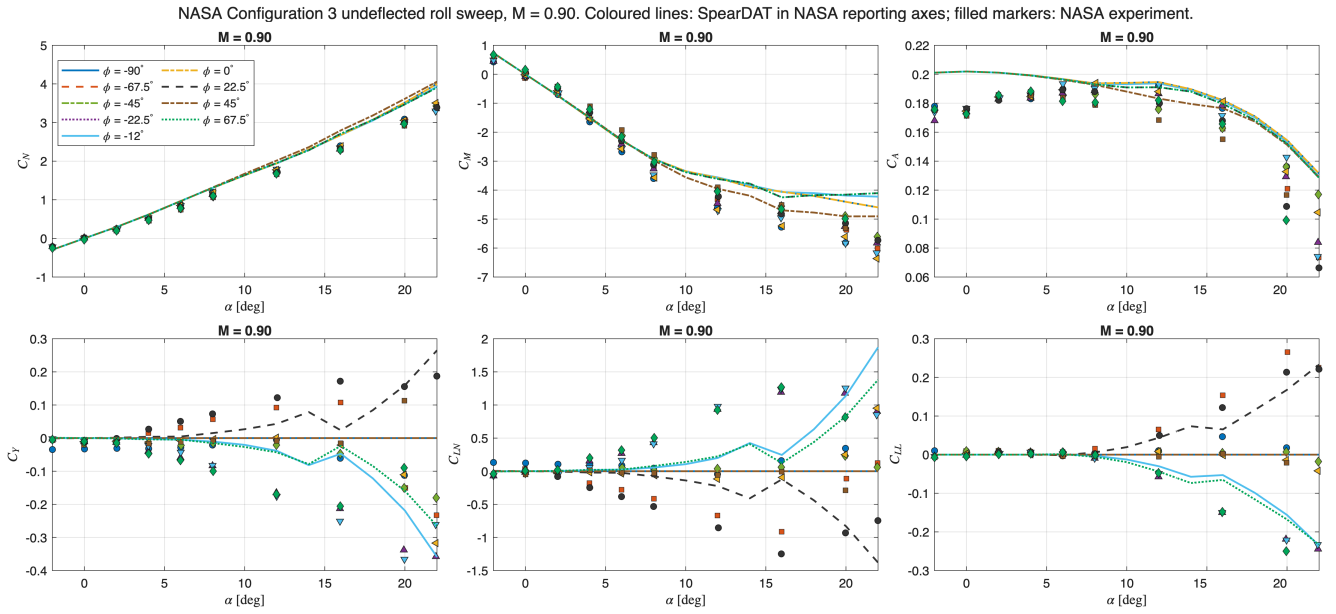


Figure 9.20.: Undeflected NASA Configuration 3 aerodynamic-roll sweep at $M = 0.90$. The calculated side force, yawing moment, and rolling moment follow the measured roll-dependent structure while the longitudinal normal-force curves remain closely grouped.

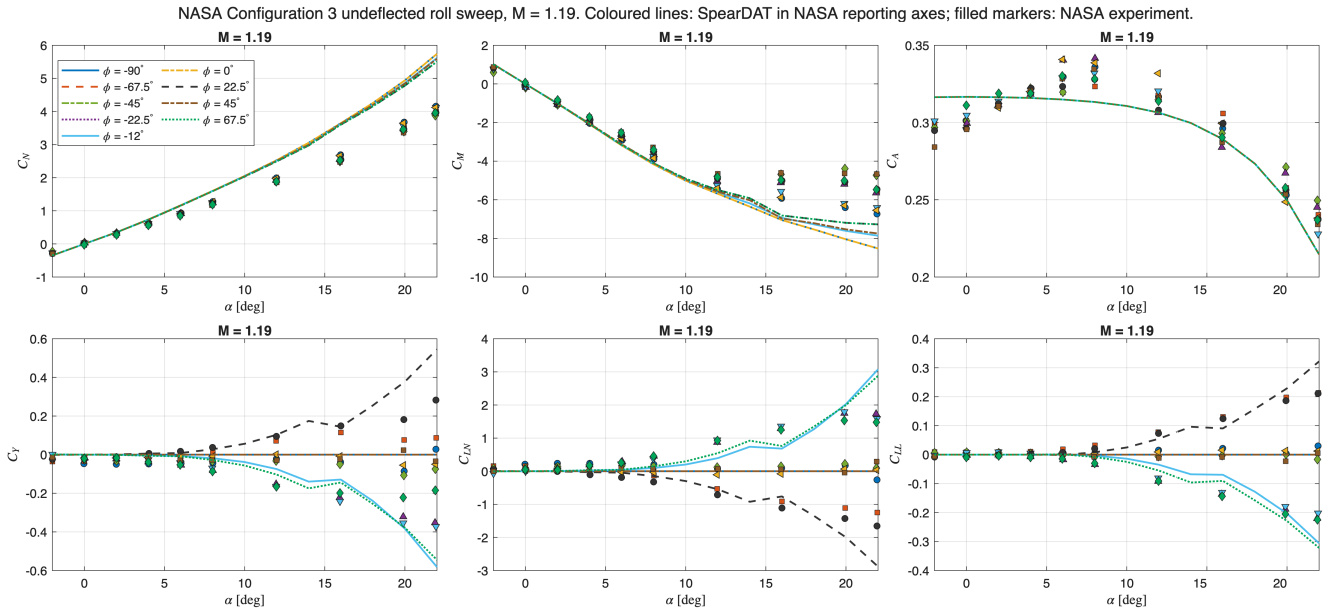


Figure 9.21.: Undeflected NASA Configuration 3 aerodynamic-roll sweep at $M = 1.19$. The condition provides a transonic test of circumferential panel loading and body-tail interference across the measured roll orientations.

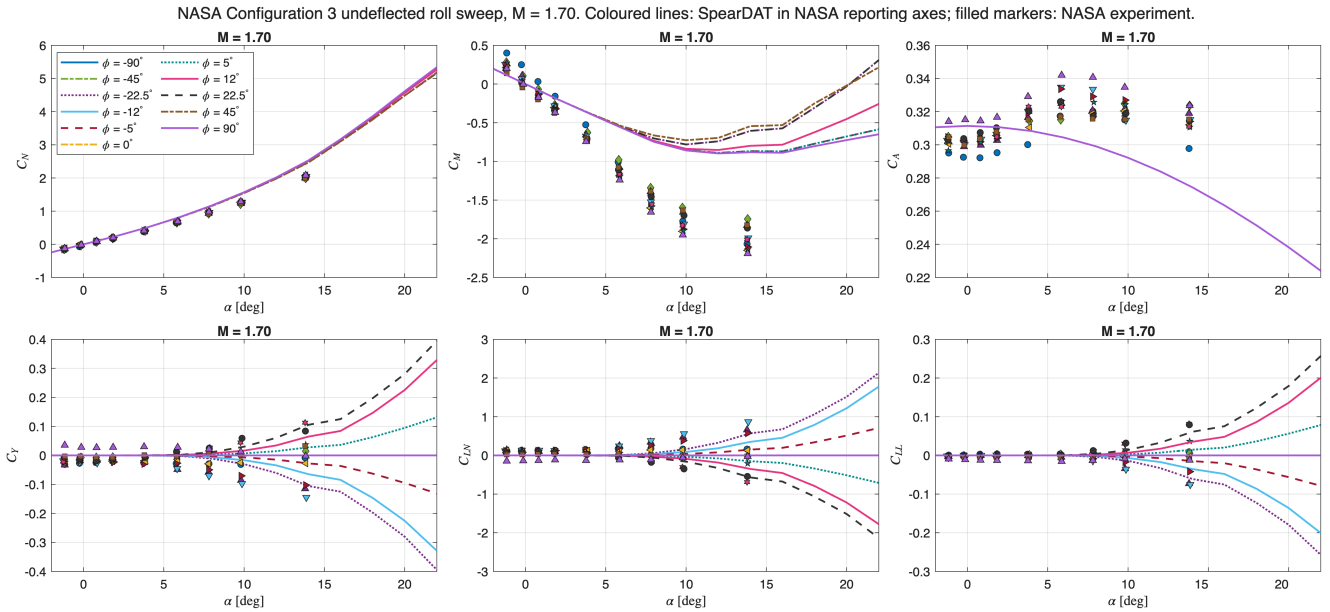


Figure 9.22.: Undeflected NASA Configuration 3 aerodynamic-roll sweep at $M = 1.70$. The low-supersonic comparison retains strong lateral-directional and rolling agreement while the normal-force curves remain comparatively insensitive to model roll.

This is a materially different validation from an axisymmetric body-alone comparison. The measured and calculated results both show that aerodynamic roll primarily redistributes the transverse load rather than changing the available normal-force magnitude. Across all undeflected Configuration 3 roll runs, including the higher-Mach cases not reproduced as figures here, the pooled RMSE is approximately 0.072 in C_Y , 0.283 in C_{L_N} , and 0.021 in C_{L_L} . The rolling-moment agreement is particularly close, while the yawing-moment comparison retains larger local differences as the asymmetric loading develops at high incidence.

9.11.6. Tail-Control Deflections

The digital database also permits the same reconstructed geometry to be tested with commanded tail motion. It is important to distinguish two uses of the word “roll”. The variable ϕ is the orientation of the complete model about its longitudinal axis. A *roll-control deflection*, by contrast, is a coordinated motion of the four tail panels. The comparison retains the NASA physical control labels and converts the individual panel signs into the raw Missile DATCOM convention before execution.

For example, NASA defines its $+5^\circ$ roll-control condition with all four physical tail angles equal to $+5^\circ$ [4]. For this panel orientation the raw SpearDAT/Missile DATCOM deflection signs are opposite, so the same physical condition is entered as

$$[-5, -5, -5, -5]^\circ.$$

The label “Roll $+5^\circ$ ” in the comparison figures therefore describes the NASA physical control command; it must not be confused with $\phi = +5^\circ$.

The two positive pitch-control cases provide a particularly useful check because they introduce a large commanded aerodynamic increment without changing the underlying body or tail geometry. Representative $+10^\circ$ and $+20^\circ$ results at Mach 1.19 are shown in figures 9.23 and 9.24.

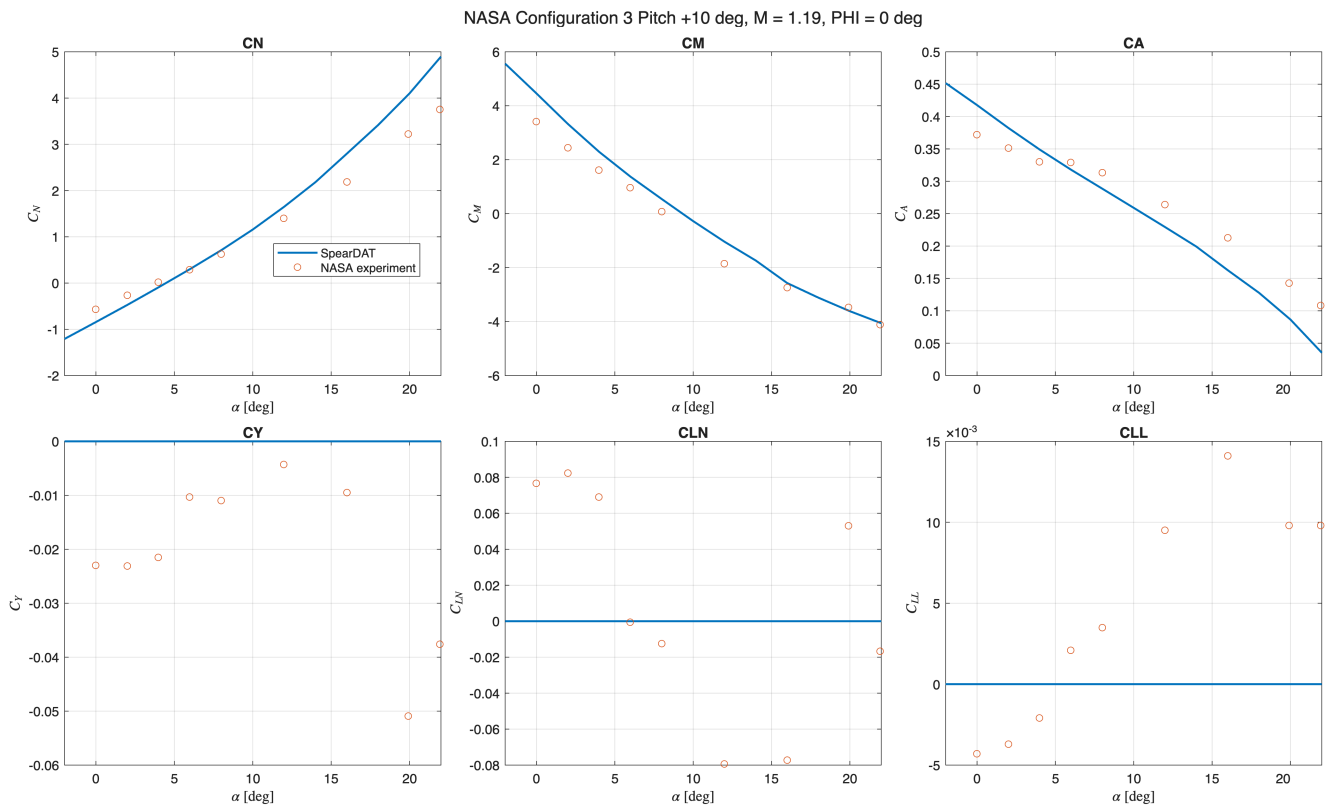


Figure 9.23.: NASA Configuration 3 with the physical +10° pitch-control setting at $M = 1.19$ and $\phi = 0^\circ$. SpearDAT reproduces the direction, scale, and incidence dependence of the principal longitudinal control response.

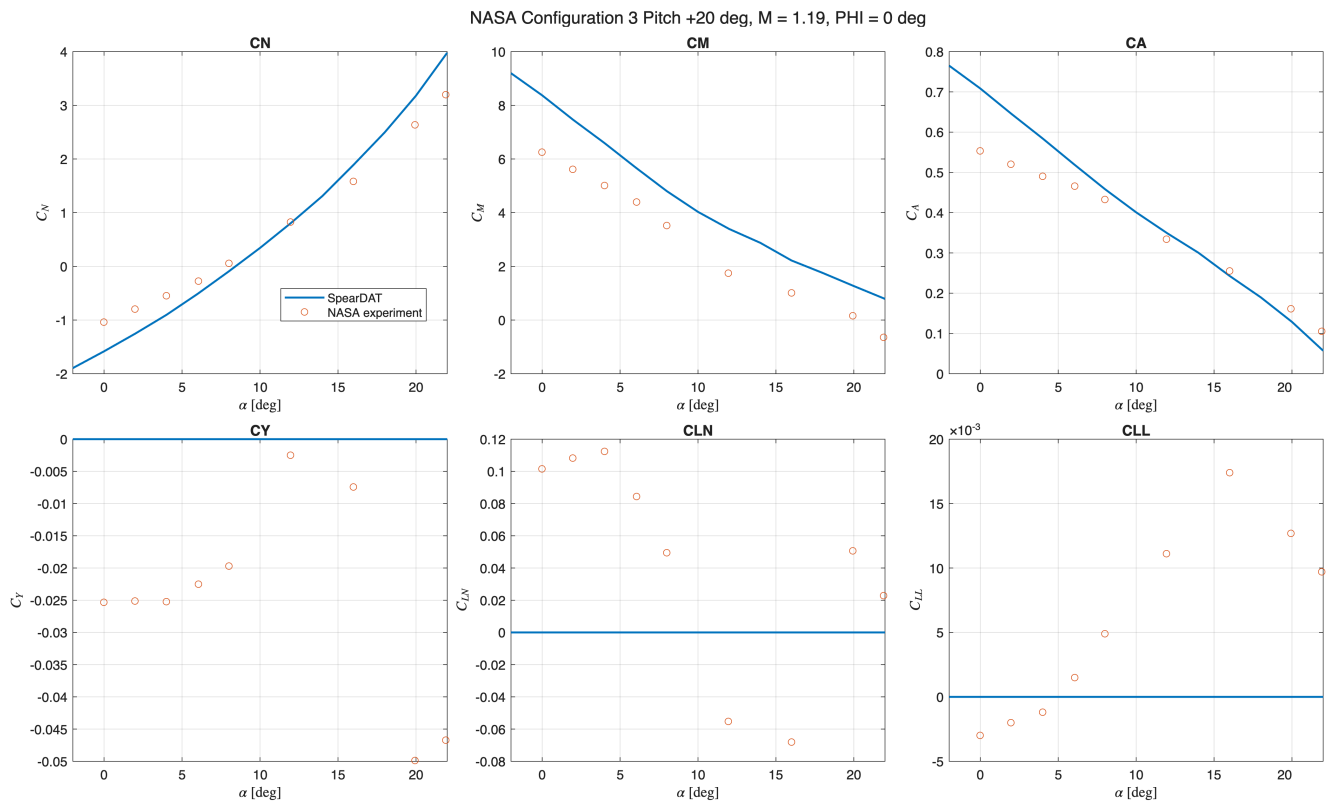


Figure 9.24.: NASA Configuration 3 with the physical +20° pitch-control setting at $M = 1.19$ and $\phi = 0^\circ$. The larger tail command is also captured with useful agreement in the longitudinal force and moment response.

The predicted normal-force, pitching-moment, and axial-force responses follow the measured control trends closely over most of the incidence sweep.

The roll-control case extends the exercise to a deliberately asymmetric six-component response. Figure 9.25 uses the Mach-0.90 case because it provides a particularly clear representative comparison of the commanded rolling response together with the accompanying lateral-directional coupling. SpearDAT captures the principal rolling-moment response and the systematic variation of the other components with model roll, although the measurements are not reproduced point for point at every incidence.

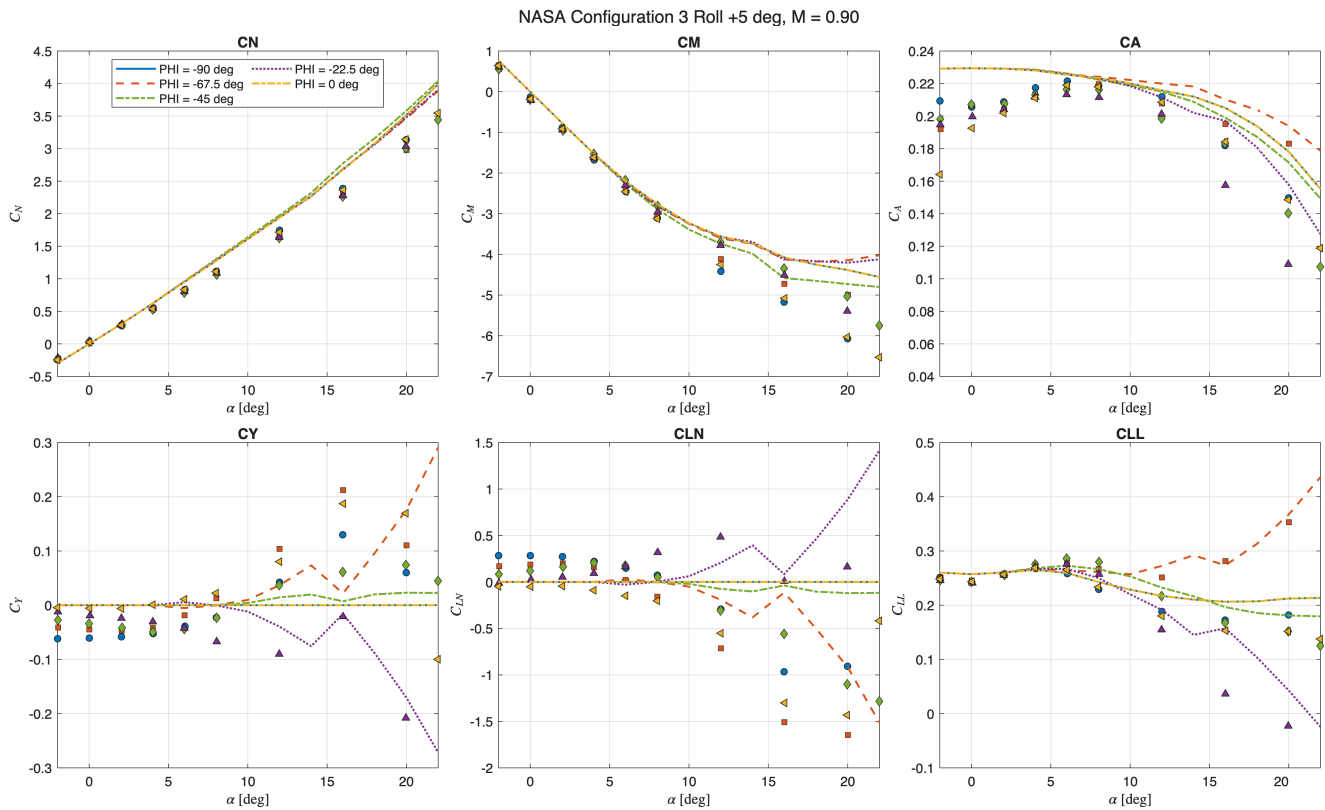


Figure 9.25.: NASA Configuration 3 with the physical Roll +5° tail-control command at $M = 0.90$. Multiple values of model roll ϕ are shown. The comparison tests commanded rolling moment together with the coupled normal, lateral, pitching, and yawing responses.

9.11.7. Overall Configuration 3 Assessment

The complete Configuration 3 exercise compares all six main-balance coefficients for the 103 selected NASA runs, including the undeflected roll sweeps and all available pitch- and roll-control families used in the analysis. Because the comparison should remain interpretable without access to the MATLAB script, table 9.13 reports the pooled six-component RMSE separately at every analysed Mach number. The overall statistics in table 9.14 then combine the 1005 matched experimental locations available for each coefficient and are therefore more conservative than any one representative figure.

The RMSE values are best read as a characteristic magnitude of the residual between prediction and experiment, expressed in the same nondimensional coefficient units as the quantity being

Table 9.13.: Pooled six-component RMSE by Mach number for the full NASA 2005 Configuration 3 comparison. All selected roll and control conditions available at each Mach number are included.

Mach	C_N	C_M	C_A	C_Y	C_{LN}	C_{LL}
0.60	0.282	0.803	0.061	0.136	0.462	0.024
0.90	0.348	1.003	0.093	0.101	0.460	0.027
1.19	0.847	1.975	0.138	0.078	0.330	0.087
1.70	0.188	0.640	0.021	0.028	0.157	0.011
2.36	0.138	0.753	0.020	0.036	0.108	0.009
2.86	0.137	1.158	0.025	0.026	0.096	0.009
3.95	0.074	1.865	0.041	0.039	0.180	0.023

Table 9.14.: Pooled six-component error over the full NASA 2005 Configuration 3 comparison.

Coefficient	Matched points	RMSE	Mean absolute error
C_N	1005	0.426	0.248
C_M	1005	1.323	0.818
C_A	1005	0.079	0.039
C_Y	1005	0.083	0.043
C_{LN}	1005	0.327	0.173
C_{LL}	1005	0.042	0.020

compared:

$$\text{RMSE} = \sqrt{\frac{1}{N} \sum_{i=1}^N (C_{\text{SpearDAT},i} - C_{\text{NASA},i})^2}.$$

An RMSE of 0.08 therefore indicates residuals with a root-mean-square magnitude of about 0.08 coefficient units; it is not an 8% error and it is not a signed bias. Because the residuals are squared, isolated larger departures influence RMSE more strongly than they influence the mean absolute error. The values should also be interpreted against the normal scale and range of each coefficient rather than compared mechanically across unlike coefficients. In the present data, the pooled statistics reinforce the graphical assessment: C_A , C_Y , and C_{LL} have relatively small absolute residuals, while C_M is the least accurate pooled quantity, consistent with the high-supersonic body pitching-moment behaviour isolated above.

Taken as a whole, this second NASA programme adds three forms of evidence that are not provided by the earlier N3–C6–T2 example. First, it validates a body plus cruciform-tail build-up over a much wider Mach range. Second, the roll-angle sweeps test the redistribution of panel loading and the associated body–tail interference through non-zero side force, yawing moment, and rolling moment. Third, the controlled runs test large commanded tail increments rather than only a fixed geometry. The plotted roll and control cases are therefore representative illustrations of a larger numerical comparison rather than the entire evidence set. The comparison is not exact at every point, but the normal-force, axial-force, lateral, and rolling trends are reproduced with useful engineering fidelity, the positive pitch-control cases correlate strongly, and the high-supersonic tail pitching-moment increment remains close to experiment even where the absolute body-derived pitching moment is less accurate.

9.12. Elliptical-Body Validation with NASA TM X-3129

The preceding comparisons exercise conventional circular missile bodies with planar lifting and control surfaces. SpearDAT also supports bodies of elliptical cross section, for which the aerodynamic response depends on both the cross-sectional axis ratio and the orientation of the body about its longitudinal axis. Published wind-tunnel data sets that combine a fully defined elliptical body, a broad Mach sweep, and sufficiently detailed coefficient plots are comparatively limited. NASA TM X-3129 provides a particularly useful body-alone data set because it isolates that capability without fin loads or body–fin interference.

Jorgensen and Nelson reported the investigation as NASA TM X-3129, *Experimental Aerodynamic Characteristics for Bodies of Elliptic Cross Section at Angles of Attack From 0 Degrees to 58 Degrees and Mach Numbers From 0.6 to 2.0* [5]. The models were tested in the NASA Ames 6- by 6-Foot Wind Tunnel. Six-component force and moment data were measured and reduced in body axes. The present comparison uses the constant-ellipticity body together with its matched circular reference, making the test a direct assessment of the non-circular body calculation rather than a complete-missile configuration.

9.12.1. Test Articles and Conditions

The circular reference body, identified by NASA as B_1 , consists of a 3-diameter tangent-ogive nose followed by a 7-diameter cylindrical afterbody. The body base diameter is

$$d = 2.60 \text{ in,}$$

so the complete length is $10d = 26.0 \text{ in}$. NASA used the base area, 5.31 in^2 , as the reference area and the body diameter as the reference length. The balance centre was located $4d$ forward of the body base.

The constant-ellipticity body, B_2 , has the same overall length and the same axial distribution of cross-sectional area as B_1 , but its semimajor to semiminor axis ratio is

$$\frac{a}{b} = 2$$

along the complete body. It was tested at two model roll angles. At $\phi = 0^\circ$ the flatter side of the elliptical nose pitches against the flow; at $\phi = 90^\circ$ the same physical body is rotated by one quarter turn. These two orientations therefore provide a strong check that the elliptical-body response changes coherently with circumferential orientation, not merely with the equivalent circular area distribution.

Table 9.15.: NASA TM X-3129 body-alone configurations used for the SpearDAT comparison.

Model	Cross section	Role in comparison
B_1	Circular, $a/b = 1$	Matched body-of-revolution reference: 3d tangent-ogive nose plus 7d cylinder.
$B_2, \phi = 0^\circ$	Elliptical, $a/b = 2$	Principal broad-side elliptical-body comparison.
$B_2, \phi = 90^\circ$	Same B_2 body, rolled 90°	Quarter-turn orientation check using exactly the same physical mold line.

Table 9.16.: NASA TM X-3129 conditions used for the constant-ellipticity comparison.

Mach	Re_d	NASA source figure
0.60	6.5×10^5	3
0.90	6.5×10^5	4
1.20	3.8×10^5	5
1.50	3.8×10^5	6
2.00	3.8×10^5	7

For the comparison, the five primary Mach conditions from NASA Figures 3–7 are retained. At Mach 0.6 and 0.9 the highest published Reynolds-number condition is used, $Re_d = 6.5 \times 10^5$. The Mach 1.2, 1.5, and 2.0 data use $Re_d = 3.8 \times 10^5$.

The SpearDAT B_2 reconstruction preserves the NASA area distribution station by station. If $r(x)$ is the radius of the equivalent circular B_1 body, the $\phi = 0^\circ$ half-width and half-height are defined as

$$W(x) = \sqrt{2}r(x), \quad H(x) = \frac{r(x)}{\sqrt{2}},$$

which gives $W/H = 2$ and

$$\pi W(x)H(x) = \pi r^2(x)$$

at every station. The model is therefore passed through SpearDAT as an elliptical body rather than as an equivalent circular approximation. No planar surfaces are present in this exercise.

9.12.2. Comparison Quantities and Digitised Data

NASA published the test results as plotted curves rather than as an electronic table, so the experimental markers used here were digitised from Figures 3–7 and then checked against the scanned source pages. Markers that are clipped by a plot boundary or cannot be separated reliably from a neighbouring series are not extrapolated. The SpearDAT curves are evaluated over the full incidence schedule, while experimental markers are shown only where a source value can be read consistently.

The comparison is deliberately focused on the two quantities that most directly test the elliptical-body longitudinal response:

$$C_N \quad \text{and} \quad \frac{x_{ac,N}}{d}.$$

NASA measured pitching moment directly but presented the derived normal-force aerodynamic-centre location in its place. With the balance centre located $4d$ forward of the base, the convention used here is

$$\frac{x_{ac,N}}{d} = \frac{C_M}{C_N} + 4.$$

The pair therefore separates two useful questions: whether the calculation reproduces the magnitude of the transverse body load, and whether it places that load at a sensible longitudinal location. When both C_N and $x_{ac,N}/d$ agree, the associated pitching-moment behaviour is constrained at the same time; agreement in aerodynamic-centre location alone should not be interpreted as exact agreement in absolute C_M when C_N itself differs.

The remaining published channels are not used as headline validation quantities in this section. NASA notes that the axial-force measurements can contain rear-sting support interference,

and the reported C_A also uses a specific base-pressure correction. The low-Mach C_Y , yawing-moment coefficient C_n , and C_Y/C_N plots contain high-incidence asymmetric separation effects at nominally zero sideslip, whereas those quantities become small at the higher supersonic conditions. A complete use of those panels would therefore require a separate digitisation and flow-physics assessment without adding a clearer test of the ELLBOD normal-force capability. They are left outside the present comparison rather than mixed into the principal confidence-building result.

9.12.3. Normal Force and Aerodynamic-Centre Comparison

Figures 9.26–9.30 show the same three physical conditions at all five Mach numbers. This makes the comparison particularly useful: the circular control, the broad-side elliptical body, and the same elliptical body rolled through 90° can be followed continuously from subsonic to supersonic flow.

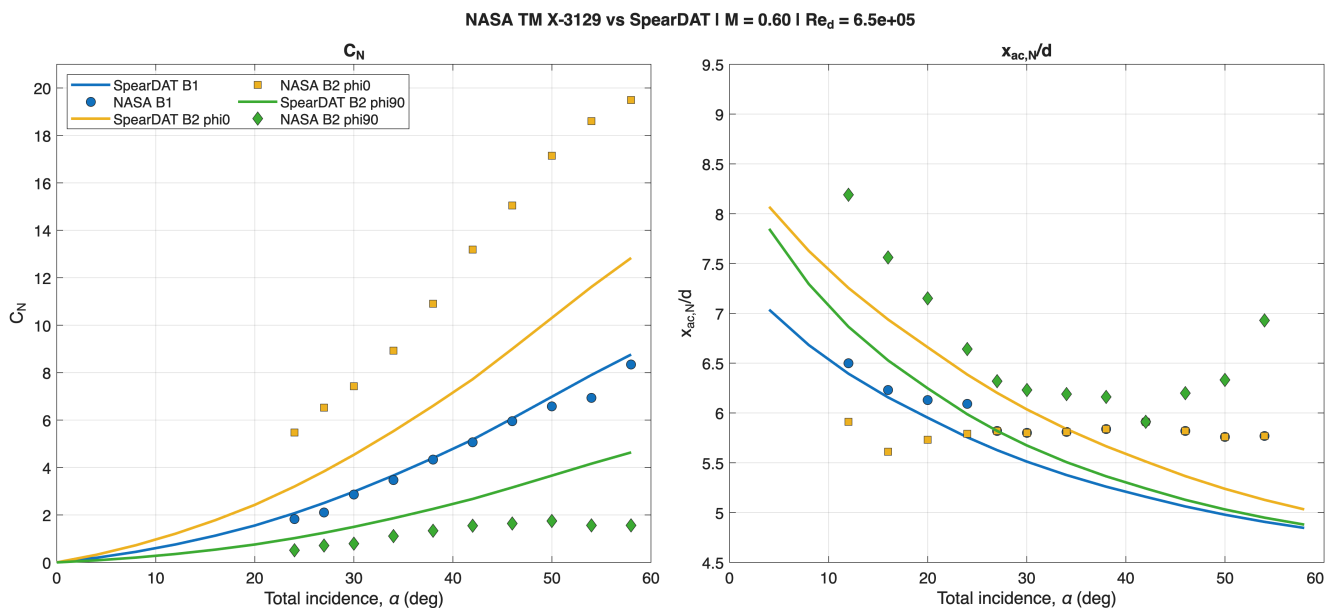


Figure 9.26.: NASA TM X-3129 B_1/B_2 comparison at $M = 0.60$ and $Re_d = 6.5 \times 10^5$. The circular-body normal force is reproduced closely. For B_2 , SpearDAT predicts the correct strong increase at $\phi = 0^\circ$ but underpredicts its magnitude. The measured $\phi = 90^\circ$ normal force flattens markedly at high incidence, a low-Mach behaviour not reproduced by the calculated curve.

The Mach-0.60 condition is the least uniform of the five comparisons. The circular body provides a strong control case, while the two elliptical orientations expose the limitations of a simple high-incidence prediction. In particular, the measured $\phi = 90^\circ$ branch approaches an approximately flat normal-force level at the upper end of the test range. SpearDAT continues to increase with incidence and therefore does not reproduce that feature. The discrepancy is useful rather than hidden: it identifies a specific low-Mach, high-incidence region in which the measured non-circular-body behaviour is more complex than the calculated trend.

At Mach 0.90 the comparison becomes much stronger. The B_1 circular-body curve follows the measured trend, and the $B_2, \phi = 90^\circ$ prediction tracks the measured normal force particularly well over most of the incidence range. The broad-side $B_2, \phi = 0^\circ$ response is still systematically low, but SpearDAT correctly predicts that this orientation generates substantially more normal force than either the circular body or the quarter-turn orientation.

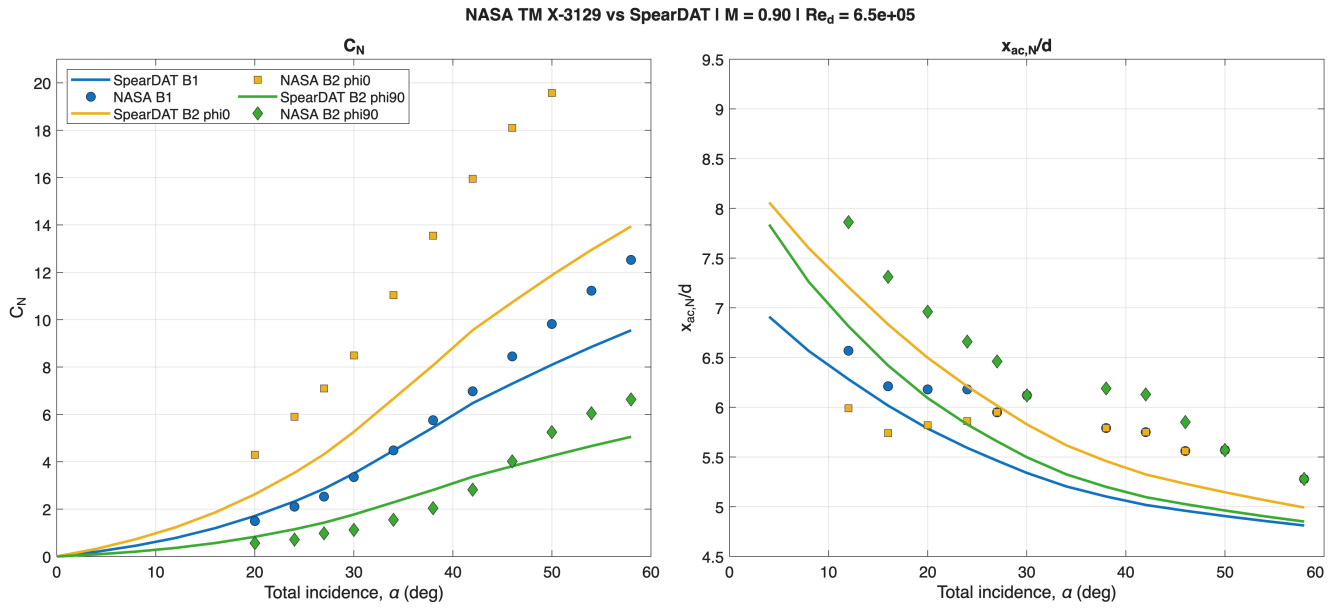


Figure 9.27.: NASA TM X-3129 B_1/B_2 comparison at $M = 0.90$ and $Re_d = 6.5 \times 10^5$. The circular and $B_2, \phi = 90^\circ$ normal-force trends are captured well. The $B_2, \phi = 0^\circ$ branch retains the correct ordering and incidence dependence but is underpredicted in magnitude.

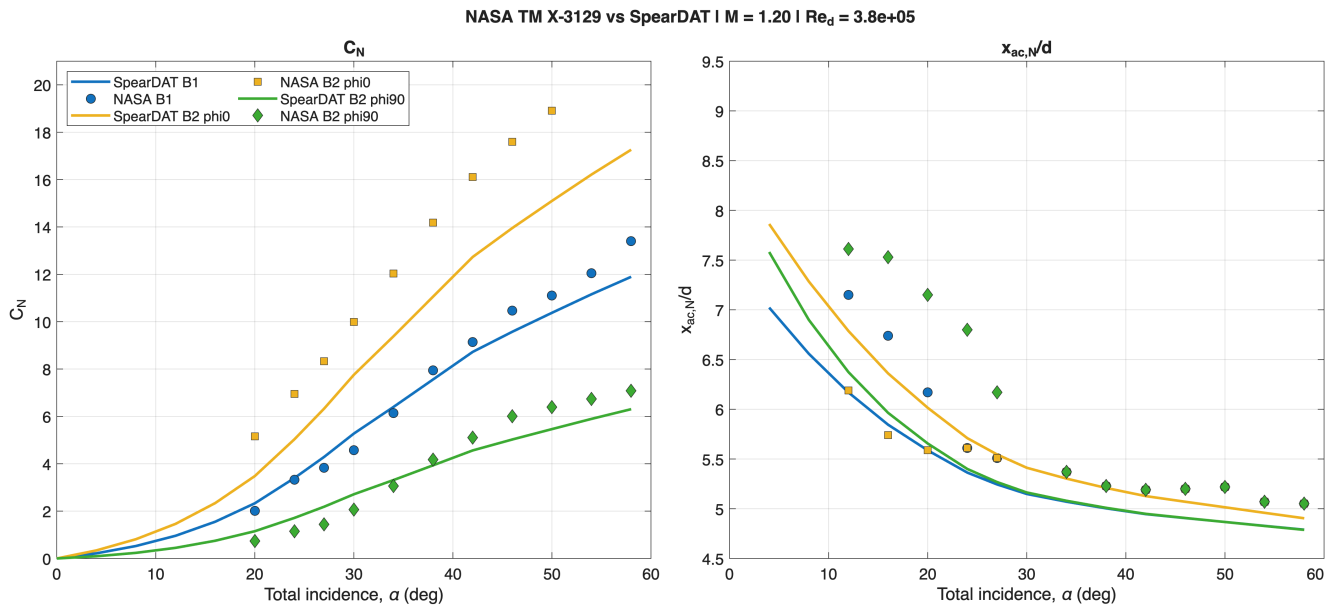


Figure 9.28.: NASA TM X-3129 B_1/B_2 comparison at $M = 1.20$ and $Re_d = 3.8 \times 10^5$. SpearDAT reproduces the principal circular and quarter-turn normal-force levels and the aerodynamic-centre migration. The broad-side elliptical normal force remains lower than measured but follows the correct nonlinear trend.

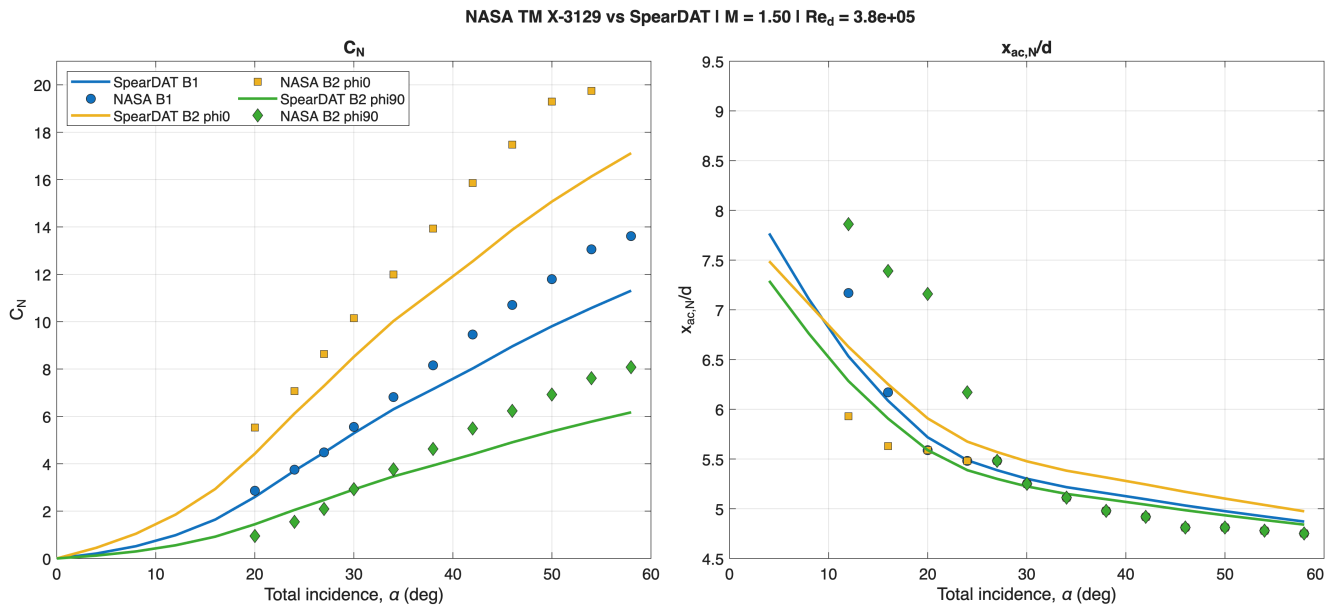


Figure 9.29.: NASA TM X-3129 B_1/B_2 comparison at $M = 1.50$ and $Re_d = 3.8 \times 10^5$. The three predicted normal-force families retain the measured ordering and the aerodynamic-centre locations converge closely with the experiment through the moderate- and high-incidence range.

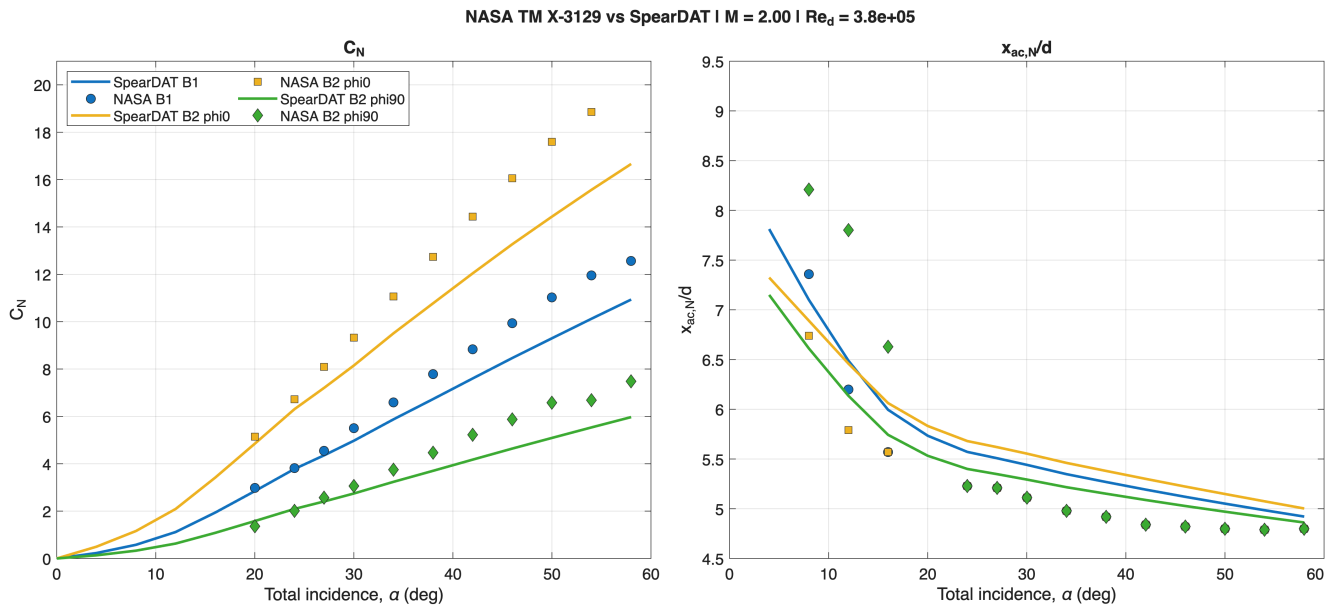


Figure 9.30.: NASA TM X-3129 B_1/B_2 comparison at $M = 2.00$ and $Re_d = 3.8 \times 10^5$. The circular and quarter-turn normal-force responses remain well ordered and close to the measurements, while the broad-side elliptical prediction retains a moderate underprediction. The measured and calculated aerodynamic-centre trends remain consistent over most of the incidence sweep.

From Mach 1.2 through 2.0 the overall pattern is consistent. The circular reference and the 90° orientation are reproduced with useful engineering fidelity, while the $\phi = 0^\circ$ elliptical-body normal force is systematically underpredicted. Importantly, that difference is a scale error rather than a loss of the underlying physical ordering: SpearDAT continues to predict

$$C_N(B_2, \phi = 0^\circ) > C_N(B_1) > C_N(B_2, \phi = 90^\circ)$$

over the principal incidence range, matching the central experimental effect of changing the circular body into a 2:1 ellipse and then rolling that same body.

The aerodynamic-centre comparison provides complementary evidence. The calculated curves generally reproduce the aft movement of the effective normal load with increasing incidence and become particularly close to the measured locations at the moderate and higher angles for Mach 1.2, 1.5, and 2.0. This matters because $x_{ac,N}/d$ is formed from both normal force and pitching moment: it shows that the stack is not merely generating a plausible force magnitude, but is also placing the resulting load at a physically sensible longitudinal location.

Taken together, the exercise provides a useful system-level check of SpearDAT's elliptical-body path. A single station-defined B_2 mold line is used for both roll orientations, the case sweep returns the expected circumferential change in aerodynamic response, and the same setup remains coherent across five Mach numbers. The agreement is not exact at every high-incidence condition, but the major geometry, roll-orientation, Mach, load, and aerodynamic-centre trends are reproduced in a way that supports using SpearDAT for preliminary analysis of non-circular bodies while making the reduced-fidelity regions visible to the engineer.

9.13. Installed-Inlet and Tri-Tail Validation with NASA TM-84560

The preceding validation exercises concentrate on conventional body–fin configurations and on body-alone cross-sectional effects. A fourth NASA data set is used here to exercise a different part of the aerodynamic stack: an air-breathing missile configuration with twin installed axisymmetric inlets and an aft tri-tail. Clyde Hayes reported this investigation in NASA TM-84560, *Aerodynamic Characteristics of a Series of Twin-Inlet Air-Breathing Missile Configurations, III—Axisymmetric and Two-Dimensional Inlets at Subsonic-Transonic Speeds* [6]. The tests formed the subsonic–transonic part of a wider inlet programme and were conducted at Mach numbers 0.60, 0.80, and 0.95. To simulate the boost condition, plugs were installed farther downstream in the internal ducts so that there was no through-flow, while the axisymmetric inlet mouths themselves remained uncovered.

This test arrangement introduces an important modelling limitation that should be understood before the comparison is read. Neither SpearDAT nor the publicly released Missile DATCOM baseline provides a configuration that combines an uncovered external inlet mouth with a plug farther downstream in the internal duct. SpearDAT therefore evaluates both available inlet-closure interpretations: `covered=false`, which preserves the uncovered inlet mouth but does not reproduce the downstream plug, and `covered=true`, which invokes the historical face-plug representation near the inlet entrance. Both SpearDAT curves are shown in the complete-configuration plots so that the sensitivity to this unresolved physical correspondence is visible. Neither curve is claimed to reproduce the wind-tunnel article exactly, and the uncovered case remains the principal comparison because it preserves the external inlet-mouth condition actually tested.

The experimental comparison is taken specifically from Figure 5 of NASA TM-84560, which presents the longitudinal component build-up for the axisymmetric-inlet configurations with

the internal ducts closed. The report used a modular test article, and the circular-inlet family is identified by NASA as inlet type I1. The present comparison uses configuration $B_1I_1T_1$: the B1 ogive-cylinder body, two I1 axisymmetric inlets, and the T1 tri-tail with zero pitch-control deflection. For T1, one surface is mounted at the top of the body and one surface is associated with each inlet fairing. The inlet/fairing assembly could be rotated circumferentially; the Figure 5 condition selected here has

$$\phi_I = 90^\circ,$$

placing the two inlets on the lateral sides of the body. Figure 5 is especially useful because it also contains the corresponding B_1I_1 body-inlet data, which provides a matched baseline from which the installed T1 contribution can be formed.



Figure 9.31.: Representative NASA TM-84560 model installed in the David Taylor 7- by 10-Foot Transonic Wind Tunnel. The photograph is included to show the twin axisymmetric inlet centerbodies, inlet fairings, and tail installation; the inlet clocking visible in the photograph is not the $\phi_I = 90^\circ$ condition used for the Figure 5 comparison below. Adapted from Figure 2(c) of NASA TM-84560 [6].

9.13.1. SpearDAT Reconstruction and Inlet-Closure Assumption

The B1 body has a diameter of 7.620 cm and a model-base station of 106.680 cm. NASA reduced the coefficients using the body cross-sectional area, approximately 45.6 cm², and the 7.620-cm body diameter as the longitudinal and lateral reference lengths. The moment reference is on the body centreline at model station 53.340 cm, equal to 50 percent of the complete body length.

The I1 inlet requires a deliberate reduction because NASA defines the external geometry in more detail than the historical five-station Missile DATCOM AXI input. In the NASA drawing, the shaped compression centerbody projects forward of the cowl lip. The published centerbody

coordinates place the cowl plane 4.783 cm behind the centerbody tip, while the cowl-lip station is M.S. 44.374 cm. SpearDAT therefore places the AXI inlet origin at

$$X_{\text{INLET}} = 44.374 - 4.783 = 39.591 \text{ cm.}$$

The five-station reduction used for the comparison is summarised in [table 9.17](#).

Table 9.17.: Five-station SpearDAT reduction of the NASA TM-84560 I1 axisymmetric inlet.

Station	Interpretation	Global M.S. [cm]	Local X [cm]	Diameter W [cm]
1	Compression-centerbody tip	39.591	0.000	0.000
2	Cowl-lip leading edge	44.374	4.783	4.445
3	Midbody/diverter start	46.355	6.764	4.445
4	Boattail start / section A	78.740	39.149	4.445
5	Boattail end / model base	106.680	67.089	2.286

The 4.445-cm section-A diameter and the 2.286-cm aft diameter are taken from the published inlet/fairing drawing. The same 4.445-cm diameter is retained at stations 2 and 3 as the first-order five-station representation of the nearly constant external fairing envelope between the cowl region and section A. The boundary-layer diverter is represented with a leading edge at M.S. 46.355 cm, a height of 0.635 cm, and a length of 14.224 cm. NASA also dimensions the forward compression surface at 12.5° . This value is supplied through the SpearDAT `rampAngleDeg` input, which corresponds to Missile DATCOM's single external-compression cone half-angle. The detailed curved NASA centerbody contour is therefore not reproduced point by point. In the present comparison additive drag is disabled and all three Mach numbers are below one, so supplying the published ramp angle does not activate the supersonic additive-drag calculation.

The T1 tail is reconstructed from its published projected planform. The exposed span is 10.160 cm, with root and tip chords of 17.188 and 5.080 cm and root and tip leading-edge stations of 89.492 and 101.600 cm. These dimensions give the published 50° leading-edge sweep and a common trailing edge at the model base. The three SpearDAT panels are clocked at $0^\circ, 90^\circ, 270^\circ$. The published root and tip thicknesses are retained, while the exact chordwise break positions of the straight-line tail section are not separately dimensioned; the same 25–50–25 percent engineering approximation used elsewhere in this guide is therefore used for the symmetric hexagonal section. The separate support fairing for the top tail surface is not modelled as an additional body protuberance.

The inlet-closure limitation stated at the start of this section is more important than the remaining geometric reductions. For the closed-duct axisymmetric tests, NASA installed plugs farther downstream in the internal ducts while leaving the inlet mouths uncovered. Neither SpearDAT nor the publicly released Missile DATCOM baseline provides that exact condition. In SpearDAT, `covered=false` leaves the external mouth open but does not represent the downstream internal plug, whereas `covered=true` invokes the historical face-plug representation between stations 1 and 2, flush with the inlet face. Neither SpearDAT option is therefore physically identical to the tunnel article. The uncovered case is used as the principal comparison because it preserves the external inlet-mouth geometry actually tested; the covered case is retained as a sensitivity so that the effect of the alternative inlet-closure representation remains visible rather than being selected according to which curve happens to agree more closely with a particular coefficient.

9.13.2. Digitised Figure 5 Data and Complete-Configuration Comparison

NASA TM-84560 publishes the longitudinal data used here as plotted markers and curves rather than as an electronic numerical table. The $B_1I_1T_1$ values were digitised from Figure 5 at Mach 0.60, 0.80, and 0.95. For this User Guide exercise, the digitisation is intentionally coarser than the graphical resolution of the original plots: a selected set of clearly readable markers is retained at approximately 2° incidence intervals rather than attempting to recover every resolvable source point. Marker centres are read from the scanned figures, so the resulting values should be interpreted as plot-derived engineering data rather than as exact wind-tunnel tabulations. The tail-equipped series also terminates at different incidences in the three Mach panels where the published measurements end; no experimental values are extrapolated beyond those endpoints.

The matched B_1I_1 series was digitised by the same procedure, but it is not shown as a separate headline validation case. Its role here is to provide the baseline needed to form the installed-tail increment discussed below.

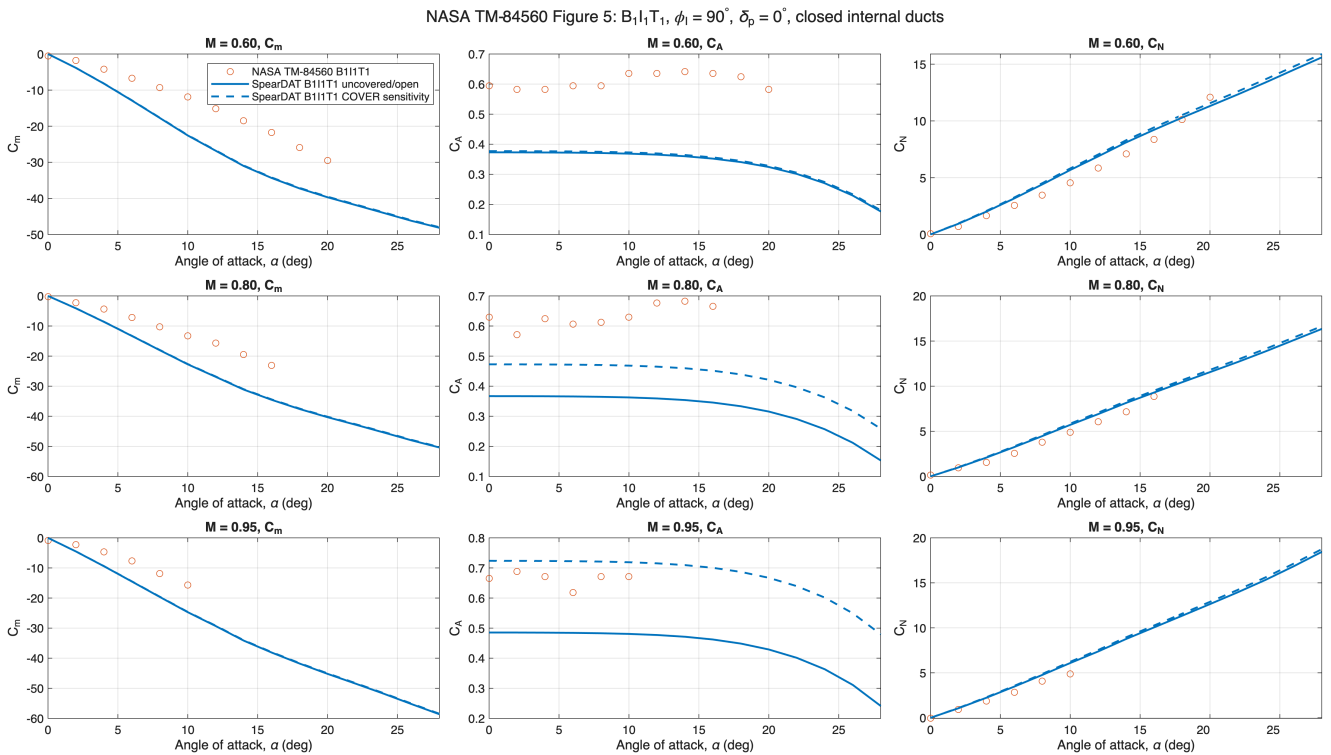


Figure 9.32.: NASA TM-84560 Figure 5 comparison for configuration $B_1I_1T_1$ at $\phi_I = 90^\circ$ and zero tail deflection. The red markers are digitised NASA data. The solid blue curve is the principal SpearDAT uncovered-inlet prediction; the dashed blue curve shows the separate COVER=true face-plug sensitivity. The normal-force trend is reproduced closely, while the calculated pitching moment is systematically more stabilising than the measurements. Axial force remains more sensitive to the inlet-closure interpretation, especially as the Mach number approaches the transonic condition.

The comparison in figure 9.32 is encouraging without being uniform across all three coefficients. The calculated normal force tracks the measured incidence dependence and coefficient level closely over the available ranges at all three Mach numbers. The pitching-moment curves have the correct stabilising direction and broadly similar incidence dependence, but SpearDAT

predicts a larger negative moment magnitude. Axial force remains of the correct engineering scale, although the growing separation between the uncovered and face-plug sensitivities toward Mach 0.95 demonstrates that the unrepresented downstream duct closure matters most strongly in this channel. The face-plug result is therefore treated as a model sensitivity rather than as an alternative fitted solution.

9.13.3. Installed T1 Increment

The matched component build-up in Figure 5 allows a more focused test than the absolute complete-configuration curves alone. At each common Mach number and angle of attack, the installed T1 contribution is formed as

$$\Delta C_i = C_i(B_1 I_1 T_1) - C_i(B_1 I_1),$$

so the increment includes the tail together with the associated installation and interference effects. It should not be interpreted as an isolated fin-alone aerodynamic coefficient. An effective longitudinal location for the incremental normal load is also formed from

$$x_{CP,\Delta} = x_{\text{ref}} - \frac{\Delta C_M}{\Delta C_N} L_{\text{ref}}.$$

Because this quantity is formed from the ratio of two digitised differences, individual experimental values become more sensitive to reading uncertainty when either increment is small; the overall location and trend are therefore more informative than the last reported decimal place.

The increment comparison provides the strongest result from this validation family. SpearDAT predicts a larger installed-tail force and stabilising moment than the experiment, but those two errors are proportioned consistently: their ratio places the incremental normal load at approximately the correct aft-body/tail location. For example, at Mach 0.60 and $\alpha = 6^\circ$, both the experimental and calculated $x_{CP,\Delta}$ values are approximately 97.2 cm even though the calculated ΔC_N and ΔC_M magnitudes are higher. The experimental centre-of-pressure estimates become more scattered at the higher Mach numbers, which is expected for a quantity derived from differences of coarsely digitised plot values, but the principal aft-load location remains coherent.

Taken together, NASA TM-84560 adds a useful fourth validation mechanism without requiring the User Guide to become a detailed inlet-method assessment. The exercise checks the combined body, installed-inlet, diverter, tri-tail, and configuration-build-up paths at three subsonic–transonic conditions. The complete $B_1 I_1 T_1$ normal force is reproduced with useful engineering fidelity, and the installed-tail increment has the correct direction, trend, and longitudinal load location. The larger absolute pitching-moment magnitude and the axial-force sensitivity to the unavailable downstream-duct-plug representation remain explicit reduced-fidelity aspects of the present SpearDAT modelling rather than being hidden or tuned away.

9.14. Validation Summary

The four NASA comparisons demonstrate both the value and the appropriate interpretation of SpearDAT results across materially different configurations and aerodynamic mechanisms. For the complete BN3–C6–T2 vehicle from NASA TM X-73,218, normal force is predicted well at

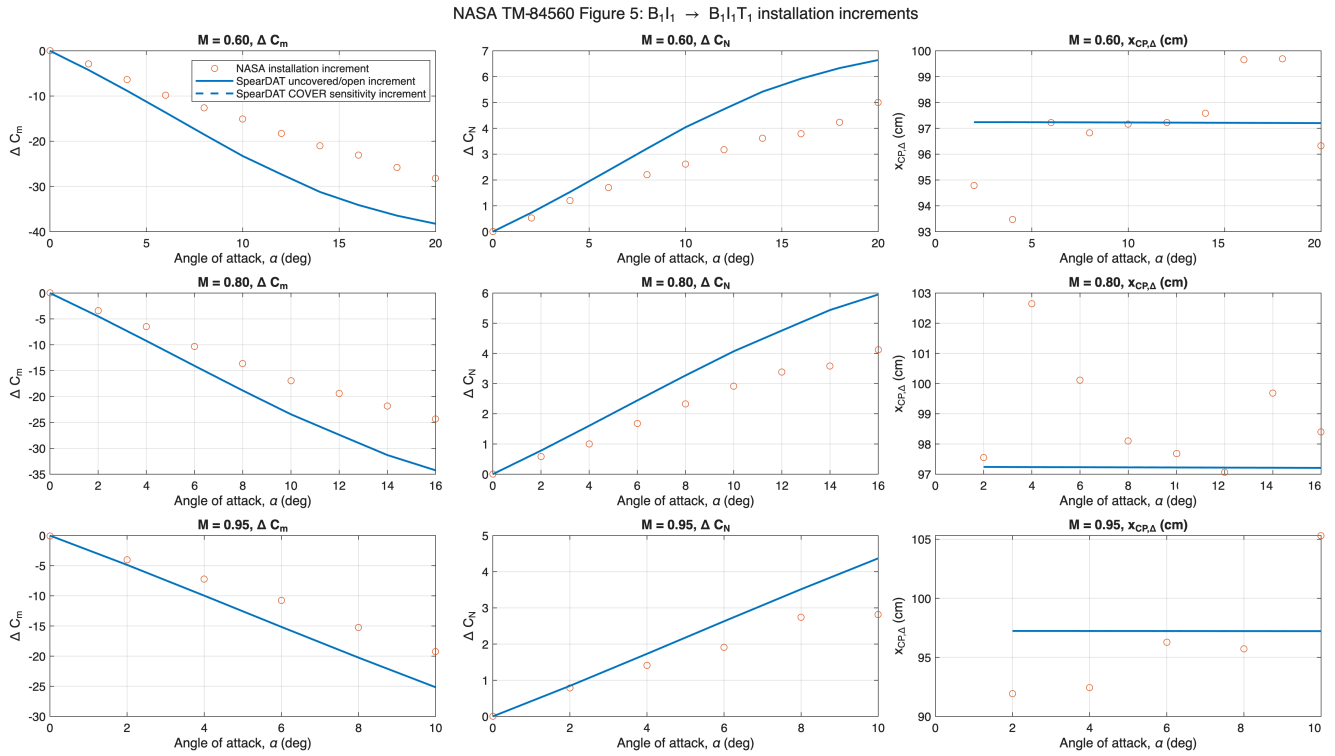


Figure 9.33.: Installed T1 increments obtained from the matched Figure 5 B_1I_1 and $B_1I_1T_1$ configurations. SpearDAT reproduces the sign and principal incidence dependence of both ΔC_N and ΔC_M , but the increment magnitude is systematically overpredicted. The inferred incremental centre of pressure remains concentrated in the correct aft-body region, close to 97 cm for the SpearDAT reconstruction and generally close to the experimental locations. The uncovered and face-plug SpearDAT increments are almost coincident, showing that the installed-tail conclusion is largely insensitive to the inlet-closure sensitivity used for the baseline.

all three test Mach numbers, while pitching moment agrees closely with experiment at Mach 1.3 and 1.75 in both the zero-deflection and 15° canard cases. The deflected comparison also shows that SpearDAT captures the direction and magnitude of a large pitch-control response, with particularly close control-induced increments at Mach 1.75. The Mach 0.8 comparison remains the principal reduced-fidelity region in that data set.

The NASA 2005 Configuration 3 exercise extends that evidence to a different body–tail geometry, seven Mach numbers, aerodynamic-roll sweeps, and tail control. It shows useful six-component agreement over more than one thousand experimental points and, in particular, reproduces the lateral and rolling responses generated when the cruciform-tail loading is redistributed with model roll. At the higher supersonic conditions the absolute pitching moment is less accurate, but the matched body-alone build-up shows that the residual is principally inherited from the body contribution: the incremental pitching moment from adding the tail and its associated interference remains very close to experiment from Mach 1.70 through 3.95.

NASA TM X-3129 adds a third and deliberately different validation case: body-alone aerodynamics for a circular reference and a constant 2:1 elliptical body tested at two roll orientations from Mach 0.6 to 2.0. SpearDAT reproduces the principal experimental ordering and the normal-force trends over the Mach sweep. The circular body and the $B_2, \phi = 90^\circ$ orientation correlate particularly well from Mach 0.9 upward, while the broad-side $B_2, \phi = 0^\circ$ normal force is systematically underpredicted but retains the correct incidence dependence and remains distinctly above the circular response. The Mach-0.60, $\phi = 90^\circ$ high-incidence flattening is not captured and is retained as an explicit reduced-fidelity region. The aerodynamic-centre comparisons are generally encouraging, especially from Mach 1.2 to 2.0, and show that the predicted longitudinal location of the normal load remains physically coherent as the body is made elliptical and rolled.

NASA TM-84560 adds a fourth validation mechanism: twin installed axisymmetric inlets and a tri-tail at Mach 0.60, 0.80, and 0.95. The selected $B_1 I_1 T_1$ comparison reproduces the complete-configuration normal-force trend closely, while the pitching moment is systematically more stabilising than measured. The source test used uncovered inlet mouths with plugs farther downstream in the ducts, a condition represented directly by neither SpearDAT nor the publicly released Missile DATCOM baseline. The open and face-plug SpearDAT results are therefore retained as an explicit inlet-closure sensitivity. The matched $B_1 I_1$ and $B_1 I_1 T_1$ build-up nevertheless gives a strong installation check: the calculated T_1 normal-force and pitching-moment increments are high in magnitude but retain the correct sign and incidence dependence, and their ratio places the incremental load in the same aft-body region as the experiment.

These are important outcomes rather than contradictions of the purpose of a preliminary-design method. SpearDAT combines analytical and semi-empirical component methods to provide rapid aerodynamic predictions over a wide configuration and flight-condition space. The four experimental programmes now exercise body-alone circular and elliptical aerodynamics, complete body–canard–tail and body–tail configurations, installed axisymmetric inlets and a tri-tail, aerodynamic roll, interference, and control from subsonic through high-supersonic conditions. Across those very different tests the calculations reproduce the dominant loads and trends over substantial parts of the measured envelopes while also making the regions of reduced fidelity visible to the engineer.

SpearDAT exposes these aerodynamic calculations through a modern, structured interface: the same analyses that generated these comparisons can be inspected through MATLAB tables, resolved case state, deterministic generated input, and retained solver reports. The experimental programme therefore supports two practical conclusions. First, the packaged

aerodynamic stack produces coherent and useful engineering predictions across a broad set of body shapes, component build-ups, inlet installations, roll orientations, and control states when the empirical scope of the underlying methods is respected. Second, SpearDAT makes that capability considerably easier to configure, interrogate, automate, and validate without hiding the limitations revealed by comparison with physical data.

10. Applicability, Assumptions and Engineering Limitations

SpearDAT provides rapid preliminary-design aerodynamic predictions by combining analytical and semi-empirical methods inherited from the Missile DATCOM methodology with the corrected and structured implementation described throughout this guide. The breadth of that capability is useful, but it also requires an important distinction: a configuration can be accepted by the interface and return finite aerodynamic coefficients without every underlying correlation necessarily being used inside the region where its accuracy is best established.

This chapter collects the principal applicability limits, modelling assumptions, and reduced-fidelity regions that matter when deciding how much confidence to place in a SpearDAT result. It is not a catalogue of internal source-code branches, and it does not restate the field-by-field input constraints already documented elsewhere. Instead, it translates the published Missile DATCOM methodology, the mature-program restrictions, and the behaviour verified during the SpearDAT implementation into guidance for an engineering user [2, 1].

Important

A finite SpearDAT result should not be interpreted automatically as a statement that the configuration lies inside the strongest validation range of every aerodynamic method contributing to that result. Where a geometry or flight condition approaches or exceeds the established range of an empirical correlation, the calculation can remain numerically well behaved while its aerodynamic uncertainty increases.

The guidance in this chapter is therefore deliberately conservative. A recommended working range is not the same as a hard software limit, and a condition outside that range is not automatically unusable. It is a signal that the result deserves additional sensitivity analysis, component build-up inspection, comparison with experiment or higher-fidelity analysis, or some combination of these.

10.1. Applicability Guidance and Terminology

Several different types of “limit” occur in the aerodynamic methodology. They should not be treated as interchangeable. The terminology used in this chapter is summarised in [table 10.1](#).

10.1.1. Method Dependence of Mach-Regime Labels

The terms *subsonic*, *transonic*, and *supersonic* appear throughout the Missile DATCOM documentation, but they do not define one universal set of switch points for every aerodynamic contribution. This is important when interpreting the calculation near Mach one.

The broad axisymmetric-body applicability summaries in the Missile DATCOM Final Report use approximately

$$M < 0.8, \quad 0.8 < M < 1.2, \quad M > 1.2$$

for the subsonic, transonic, and supersonic regimes, respectively [2]. The mature 1997 User’s Manual, however, lists the fin-alone aerodynamic methodology using

$$M < 0.8, \quad 0.8 < M < 1.4, \quad M > 1.4$$

Table 10.1.: Interpretation of applicability language used in this chapter.

Term	Meaning
Recommended working range	A deliberately conservative region considered appropriate for routine preliminary-design use on the basis of the published method applicability, available validation evidence, and the behaviour of the implemented correlations.
Extended / caution region	A region in which SpearDAT can still calculate a result, but the prediction relies more strongly on empirical continuation, extrapolation, limiting assumptions, or validation evidence that is thinner than in the recommended range.
Correlation bound	The finite range of a chart, table, or empirical fit used by one part of the aerodynamic build-up. Crossing a correlation bound does not necessarily cause a solver error: the method may clamp a parameter, extrapolate, or transition to another formulation.
Method transition	A planned change or blend between aerodynamic methods as Mach number, incidence, geometry, or another similarity parameter changes. A change in curve slope near such a transition is not by itself evidence of a numerical defect.
Unavailable / not represented	A physical effect or configuration for which the relevant model does not provide the required aerodynamic representation.

for the corresponding lifting-surface method families [1].

Even within the lifting-surface calculation, individual quantities can use different transition points. For example, the principal low-speed fin normal-force formulation hands over to the transonic normal-force treatment before $M = 0.8$, while selected pressure-, wave-, and leading-edge-drag terms use their own transitions in the neighbourhood of Mach one. These are method-selection details rather than competing definitions of the physical transonic regime.

Note

When a result changes behaviour near a nominal Mach-regime boundary, inspect the coefficient and component build-up before assuming that the change is non-physical. Different force, moment, drag, and interference contributions can change method at different Mach numbers.

10.2. Planar Lifting Surfaces, Controls and Interference

This section applies to the planar lifting surfaces represented through SpearDAT fin sets: fins, canards, wings, tails, strakes, and similar surfaces. The word *planar* describes the aerodynamic model, not the circumferential orientation of the surface. A panel can be clocked around the body and assigned a constant dihedral, but the panel itself is treated as a planar true-view lifting surface rather than as a warped or continuously twisted three-dimensional surface [1].

The underlying methods are strongest for conventional triangular and trapezoidal missile surfaces. SpearDAT can describe multi-segment planforms, but additional geometry stations do not turn the aerodynamic model into a general high-fidelity lifting-surface solver. Complex planforms are still reduced to the analytical and semi-empirical representations used by the underlying methodology.

10.2.1. Recommended working envelope

Tables 10.2 and 10.3 collect the principal guidance that can be stated cleanly at user-input level. The numerical values should be read as engineering applicability guidance rather than as a replacement for the input validator.

Table 10.2.: Recommended aerodynamic applicability guidance for planar lifting surfaces.

Quantity	Recommended / established region	Extended / caution region	Interpretation
Fin aspect ratio, AR	$0.5 \lesssim AR \lesssim 4$	$AR < 0.5$ or $AR > 4$	The principal fin methods have their strongest published applicability and validation support in the stated interval. A finite result outside it should not be treated as equally established.
Fin taper ratio, λ	$0 \leq \lambda \leq 1$	Outside the conventional triangular/trapezoidal range	The principal correlations are based on conventional non-expanding tapered planforms.
Angle of attack / panel incidence	Approximately $ \alpha \lesssim 30^\circ$ for routine use	Approximately 30° – 60° where the applicable nonlinear method continues	High-incidence capability exists, but the prediction increasingly depends on empirical maximum-lift, post-stall, cross-flow, and shock-detachment models.
Mach number	Method dependent	Normal-force evidence becomes progressively thinner beyond the range covered by the supporting databases; the reviewed fin normal-force validation extends to approximately $M = 4.6$	There is no single universal fin Mach cutoff. Individual force, moment, drag, and interference terms have different transitions and continuations.

The historical Missile DATCOM development targets covered a wider design space than the recommended ranges above. Those broader targets should not be confused with validation of every aerodynamic method throughout that complete space. For planar surfaces in particular, the published method assessment gives much stronger support to the $AR \approx 0.5$ – 4 range than to the very-low- or very-high-aspect-ratio extremes of the original design-space objectives [2].

Table 10.3.: Planform, panel and plain trailing-edge-control applicability guidance.

Quantity	Recommended / established region	Extended / caution region	Interpretation
Planform	Straight-taper triangular or trapezoidal	Strongly cranked, cut-out, or highly segmented planforms	Segmentation improves geometric representation but does not replace the underlying equivalent preliminary-design methods.
Plain-flap chord fraction, c_f/c	Approximately $0.05 \leq c_f/c \leq 0.50$ for the principal effectiveness correlation	Outside the tabulated effectiveness range	This is a correlation range, not a statement that every larger or smaller geometric input must fail numerically.
Plain trailing-edge deflection	Moderate deflection	Large deflection; the historical effectiveness correction remains defined to approximately 60°	Large-deflection results should be treated increasingly as empirical preliminary-design estimates.
Panels per fin set	One to eight identical panels are supported	Deflected sets with more than four panels require additional interpretation	For more than four panels, the mutual deflection-interference treatment is reduced to neighbouring panels in an equivalent cruciform representation.

10.2.2. Aspect ratio, taper and planform geometry

Aspect ratio is one of the most important parameters to check before relying on a fin result. The principal Missile DATCOM lifting-surface methodology was established over approximately

$$0.5 \leq AR \leq 4, \quad 0 \leq \lambda \leq 1,$$

with conventional triangular and trapezoidal planforms receiving the strongest support [2]. The software can evaluate geometries outside those bounds, but numerical availability should not be interpreted as equivalent aerodynamic validation.

Very-low-aspect-ratio surfaces deserve particular care. Their total normal force can remain plausible even when the predicted aerodynamic-centre location is much less reliable. A strake is a typical example: its contribution to stability can depend as strongly on where the load is placed as on the magnitude of the load itself. For $AR < 0.5$, results should therefore be treated as an outside-established-range calculation rather than as automatically invalid.

The same caution applies at the high- AR end. Calculations above $AR = 4$ can remain useful, and some methods have analytical or empirical continuation outside the central database, but broad validation of all relevant coefficient channels is less complete. Where such a surface materially controls stability or control authority, comparison with an independent source is recommended.

Small-span surfaces also require care in how sweep is specified. The Missile DATCOM User's Manual notes that small errors in planform coordinates can produce comparatively large errors in the derived sweep angle and recommends supplying an exact sweep definition where possible [1]. This is a geometric sensitivity rather than a SpearDAT numerical precision issue.

10.2.3. Nonlinear and high-incidence aerodynamics

The fin methods extend substantially beyond the small-angle linear range, but their physical basis changes as incidence increases. At low incidence the calculation is dominated by linear lifting-surface methods. At higher incidence the build-up introduces empirical maximum-lift, nonlinear normal-force, cross-flow, post-stall, and, at supersonic conditions, shock-detachment correlations [2].

The established method range extends to approximately 60° incidence, but this should not be read as a claim of uniform accuracy from zero to 60° . The experimental assessments used in the Missile DATCOM development provide substantially stronger evidence through the lower and moderate incidence range, with useful agreement demonstrated to approximately 30° for the configurations examined. Above this region the result should increasingly be interpreted as a semi-empirical preliminary-design estimate.

The transonic nonlinear treatment also deserves this interpretation. Rather than resolving the complete separated transonic flow field, the methodology bridges the nonlinear behaviour between lower-speed and supersonic endpoint models. This is consistent with the intended preliminary-design role of Missile DATCOM and SpearDAT, but it is not equivalent to a high-fidelity transonic viscous-flow solution.

10.2.4. Pitching moment and aerodynamic-centre placement

Important

Good agreement in fin normal force does not guarantee equally good agreement in pitching moment or centre of pressure. Normal force depends primarily on the magnitude of the aerodynamic loading; pitching moment additionally depends on where that loading acts.

This distinction becomes increasingly important at low aspect ratio, high incidence, and in strongly body-coupled or interference-dominated configurations. The nonlinear Missile DATCOM fin-moment methodology requires an engineering approximation for movement of the nonlinear load location as incidence increases. The original method assessment explicitly treats this as an engineering approximation rather than as a complete theoretical solution for the nonlinear centre of pressure [2].

For stability work, therefore, C_N , C_M , and any derived aerodynamic centre should be inspected together. A plausible C_N curve is not sufficient evidence by itself that a very-low-AR strake, highly loaded tail, or high-incidence surface is placing its load at the correct longitudinal location.

10.2.5. Axial force and airfoil-section assumptions

Fin axial force is assembled from several separate engineering contributions, including skin friction, pressure/form drag, wave drag, leading-edge bluntness, trailing-edge base drag, and finite-incidence drag. These terms do not share one common validity envelope. Some are blended through the transonic region, some change according to whether a leading edge lies ahead of or behind the Mach cone, and some use explicit high-Mach continuation after the original empirical table ends.

Consequently, confidence in C_N should not be transferred automatically to C_A . A configuration can lie comfortably inside the normal-force envelope while one of its drag contributions is closer to the edge of its own correlation range. This is especially relevant for unusually thick sections, large leading-edge radii, blunt trailing edges, or high-incidence conditions.

Detailed NACA and user-defined section geometry is used to obtain section properties and selected drag corrections, particularly at low speed. It should not be interpreted as a general compressible two-dimensional airfoil solver whose complete detailed section behaviour is propagated unchanged through the full Mach range.

The mature Missile DATCOM implementation also makes a specific distinction for camber. Camber effects on fin normal force and pitching moment are included for NACA and user-defined sections only at $M \leq 0.8$; for HEX and ARC section definitions, camber affects the drag treatment rather than supplying the same normal-force and pitching-moment correction [1]. Only one section family is selected for a given fin set, although its geometric proportions can vary between span stations.

10.2.6. Plain trailing-edge controls

Plain trailing-edge devices are a specialised control model and should not be confused with whole-panel or all-moving-surface deflection. The underlying two-dimensional effectiveness

correction is tabulated over approximately

$$0.05 \leq \frac{c_f}{c} \leq 0.50.$$

Within SpearDAT this should be treated as the principal correlation range for the flap chord fraction rather than as a universal geometric hard stop.

The historical control-effectiveness correction remains defined to large deflection angles, reaching approximately 60° . As deflection becomes large, however, the result becomes increasingly dependent on empirical correction rather than small-disturbance control theory. Large-deflection predictions should therefore be interpreted as preliminary-design estimates and checked independently when control authority is critical.

Several geometric and output restrictions are explicit in the Missile DATCOM User's Manual [1]:

- the active trailing-edge-device hinge line must be straight;
- the movable device must not increase in chord toward the tip, so its outboard-to-inboard device-chord taper ratio may not exceed one;
- hinge moments are not calculated for these plain trailing-edge devices;
- the increase in profile drag caused specifically by trailing-edge deflection is not calculated;
- the trailing-edge-device deflection angle is defined with respect to the freestream rather than relative to a swept hinge line; and
- the separate hinge-skew treatment does not automatically correct a swept trailing-edge-device hinge, so the user must account for that geometry where required.

The hinge-moment restriction above applies to the plain trailing-edge-device model. It does not mean that the separate whole-panel hinge-report capability is absent: an all-moving or panel-deflected surface uses a different hinge-line representation and can produce the corresponding Missile DATCOM panel hinge-moment coefficient output.

10.2.7. Panel, body-fin and downstream-surface interference

A complete body-fin vehicle is not formed by simply adding isolated component coefficients. As described in [section 8.4](#), the aerodynamic synthesis includes body-fin carryover, panel interaction, body-vortex effects, and the influence of vortices shed by an upstream lifting surface on later fin sets. These are important engineering corrections, but they remain static analytical and semi-empirical models rather than CFD, a free-wake solution, or a time-accurate vortex calculation [2, 1].

Within one fin set, the panel-interference treatment is organised around an equivalent cruciform representation. One to eight identical physical panels can be represented, but this should not be interpreted as a complete all-to-all aerodynamic interaction model for every arbitrary panel arrangement. In particular, when more than four panels are present, the deflection-interference treatment retains the neighbouring panels around the panel being evaluated in the equivalent cruciform representation. Asymmetric three-panel arrangements and strongly non-uniform panel deflections also deserve additional interpretation because the physical layout is being mapped into that same engineering framework.

Spacing between successive fin sets is a separate and especially important assumption. The sets must be ordered from nose to tail and their physical planforms should not overlap. More importantly, the downstream-interference model assumes that the vortex system shed by an

upstream surface has rolled up sufficiently by the time it reaches the control points of the next fin set [1]. The User's Manual notes that a real vortex sheet can require at least about four upstream-surface semispans of downstream distance to approach the fully rolled-up state assumed by the vortex-filament model.

Important

The approximately four-semispan wake-development distance is an aerodynamic modelling warning, not a universal SpearDAT input-rejection threshold. SpearDAT can analyse more closely spaced fin sets, but the closer the surfaces are, the more carefully the downstream interference result should be treated. Very compact canard–wing–tail, canard–strake–tail, and similar layouts are good candidates for comparison with wind-tunnel data, CFD, or another independent aerodynamic method.

Body-generated vortical interference is subject to the same general interpretation. The method uses analytical and empirical estimates of separation, vortex position, strength, and induced incidence. It captures an important physical mechanism but does not resolve the separated vortex field directly. Its contribution should therefore receive increasing scrutiny as incidence rises or when the downstream surface lies in a strongly vortex-dominated flow region.

10.2.8. Practical interpretation of planar-surface results

For a conventional missile fin, canard, wing, or tail, the most defensible routine starting point is a triangular or trapezoidal planform with approximately

$$0.5 \lesssim AR \lesssim 4, \quad 0 \leq \lambda \leq 1,$$

operated at moderate incidence and with adequate spacing between successive fin sets. Within that region, the Missile DATCOM lifting-surface methods have the strongest combination of published applicability and validation evidence [2].

Outside that region SpearDAT may still provide useful engineering information, but the coefficient channels should not be assumed to degrade uniformly. Very-low-AR surfaces can retain useful normal-force behaviour while pitching-moment or aerodynamic-centre fidelity deteriorates; large incidence increases dependence on empirical nonlinear models; axial-force terms have their own method-specific envelopes; and closely spaced fin sets increase dependence on the assumed rolled-up wake model.

For such configurations, use the configuration build-up and detailed reports to identify which component or interference term controls the result, vary the sensitive geometry or flight condition deliberately, and seek independent evidence when the affected coefficient is important to a design decision.

10.3. Axisymmetric Body Methods and Pressure Predictions

This section applies to the axisymmetric body methods used for conventional bodies of revolution, including pointed and blunted noses, cylindrical centrebodies, boattails, flares, and station-defined circular mold lines. The underlying Missile DATCOM capability spans a wide range of flight conditions, but the individual contributions to body normal force, pitching moment, axial force, base drag, and pressure distribution do not share one common applicability boundary [2, 1].

A useful overall starting point is the published principal body-method envelope: body fineness ratios of approximately

$$2 \lesssim L/D \lesssim 28$$

and incidence up to approximately

$$|\alpha| \lesssim 30^\circ.$$

These values are best treated as an overall preliminary-design envelope, not as proof that every internal correlation is directly validated throughout that complete rectangle of geometry and flight condition. Several body sub-methods have appreciably narrower source ranges and then change method, apply a continuation, or use extrapolation.

There is no corresponding universal minimum or maximum dimensional missile length or diameter. The body methods are governed principally by nondimensional geometry, Mach number, Reynolds number, incidence, and local surface shape. Absolute vehicle size matters through those quantities rather than through a single program-wide length interval.

10.3.1. Body Applicability Summary

Table 10.4 summarises the most useful body-level guidance. The second table, table 10.5, lists selected narrower correlation domains that are worth knowing because they can be reached inside an otherwise ordinary-looking body analysis.

10.3.2. Body Geometry and Fineness Ratio

The strongest body-method support is associated with conventional bodies of revolution constructed from familiar nose, cylindrical-centrebody, and afterbody forms. Conical and tangent-ogive noses, cylindrical centre sections, and conical or ogival boattails are therefore natural applications of the methodology. Spherical bluntness is supported through dedicated corrections and high-speed treatments, but it should be regarded as a separate physical model rather than as a trivial modification of a sharp nose.

The overall $L/D \approx 2\text{--}28$ envelope should not be applied blindly to every body segment. The transonic cone/ogive normal-force and centre-of-pressure correlations use much narrower nose- and centrebody-fineness coordinates. The normal-force treatment contains an analytical continuation toward slender noses, whereas the corresponding transonic load-location treatment has less direct support at very large nose fineness ratio. As a result, unusually slender noses can retain a plausible force slope while their predicted aerodynamic-centre location deserves greater scrutiny.

Important

As with planar lifting surfaces, body force and body moment do not necessarily lose fidelity at the same rate. A credible C_N prediction is not by itself evidence that the aerodynamic-centre position, C_M , or a derived static margin is equally well established.

Station-defined circular mold lines provide useful geometric flexibility, but that flexibility should not be interpreted as a general arbitrary-body flow solver. In the subsonic/transonic potential-force path, a station-defined body can reduce to a slender-body-theory representation for C_{N_α} and C_{M_α} . The detailed coordinates still define the physical geometry for other calculations, but the potential normal-force model itself does not become a geometry-resolving transonic CFD method merely because many stations are supplied.

Table 10.4.: Recommended applicability guidance for axisymmetric body aerodynamics.

Quantity	Recommended / established region	Extended / caution region	Interpretation
Overall body fineness ratio, L/D	Approximately 2–28	Outside the principal published body-method envelope	This is an overall methodology range. Individual nose, afterbody, drag, and pressure correlations can be narrower.
Body incidence	Approximately $ \alpha \lesssim 30^\circ$ for routine use	Above approximately 30°	The body calculation continues to high incidence, but increasingly through empirical crossflow and high-angle continuation models.
Subsonic/transonic body regime	Method dependent; broad body-method classification below about $M = 1.2$	Conditions near individual correlation boundaries	Potential force, pressure drag, crossflow, and afterbody terms use different Mach transitions.
Ordinary supersonic body analysis	Default hybrid method where its geometric assumptions are satisfied	Steep-tip, blunted, strongly curved, or otherwise difficult contours	The default high-speed path can fall back to second-order shock expansion when the hybrid solution is not applicable or does not remain physical.
Hypersonic body analysis	Modified Newtonian method principally for genuinely hypersonic conditions	Using the Newtonian method merely because the case is supersonic	The mature User's Manual permits the method above $M = 1.4$, but recommends it normally for Mach numbers greater than about 6.

Table 10.5.: Selected body-correlation ranges requiring additional interpretation.

Correlation / contribution	Principal source range	User interpretation
Cone/ogive transonic potential C_N and load location	Tabulated principally over $M \approx 0.8$ – 1.2 , with geometry-dependent nose and centrebody fineness-ratio coordinates	These are transonic chart domains, not universal body limits. Unusually slender noses or centrebodies can move the calculation into continuation or extrapolation, particularly for load location.
Blunted-nose transonic wave drag	Approximately $M = 0.8$ – 1.2 , nose L/D of roughly 0.75 – 5 , and $0 \leq 2R_b/D \leq 1$ in the underlying table	Useful for conventional spherical bluntness. Values outside the chart should not be interpreted as equally supported merely because a finite result is returned.
Subsonic/transonic conical-flare wave drag	Tabulated through roughly 10° flare half-angle and $M \approx 0.5$ – 1.3	More severe flare angles rely on use outside the central source correlation; the supersonic calculation subsequently uses a different body method.
Crossflow viscous normal force	Empirical functions of crossflow Mach and crossflow Reynolds number	At substantial incidence the body force becomes increasingly dependent on semi-empirical crossflow aerodynamics rather than small-disturbance potential flow.
Hybrid/SOSE pressure output	Supersonic; pressure tables limited to approximately $\alpha \leq 15^\circ$	The pressure output has a narrower incidence range than the total static coefficient capability.

Abrupt contour changes also deserve deliberate treatment. Genuine shoulders, boattail starts, flare starts, and other discontinuities should be represented consistently with the body geometry because the high-speed shock-expansion methods divide the mold line into aerodynamic segments. Conversely, adding unnecessary numerical corners to what should be a smooth mold line can create artificial local compression or expansion behaviour.

10.3.3. Subsonic and Transonic Normal Force

At low and transonic speeds, the body normal-force build-up combines a potential contribution with a viscous crossflow contribution. For standard cone/ogive configurations the potential term uses dedicated empirical charts; for more general station-defined bodies the alternative path uses slender-body theory. These methods are then continued to finite incidence with an Allen–Perkins-style high-angle form.

The standard transonic cone/ogive charts are concentrated around

$$0.8 \lesssim M \lesssim 1.2.$$

Below the transonic chart range, the method deliberately uses the lower boundary where appropriate rather than inventing a separate detailed compressible solution. At the geometry extremes, interpolation can become continuation or extrapolation. This is one reason the broad L/D envelope should not be interpreted as a guarantee of equal transonic accuracy for every combination of nose and centrebody proportions.

The body crossflow correction introduces its own effective-fineness, crossflow-Mach, and crossflow-Reynolds dependencies. In physical terms, this means that increasing incidence changes not only the magnitude of the force but also which empirical part of the body model is controlling it.

10.3.4. High-Incidence Crossflow Aerodynamics

The body methods continue well beyond 30° , including mathematically defined behaviour through very high incidence and reverse-flow orientations. This numerical capability should not be confused with uniform aerodynamic fidelity.

For routine preliminary-design use, approximately

$$|\alpha| \lesssim 30^\circ$$

is the more defensible body-incidence range. Above it, the body calculation increasingly depends on Jorgensen/Allen–Perkins-style crossflow and empirical high-angle continuation. The potential normal-force and pitching-moment terms are also folded into a high-angle form rather than obtained from a new three-dimensional separated-flow solution.

The viscous normal-force increment scales with the crossflow drag coefficient and with

$$\sin \alpha |\sin \alpha|,$$

while the corresponding viscous pitching moment places that crossflow loading relative to an effective body-planform centroid. The force magnitude and the moment therefore share the same crossflow model but not the same sensitivity to load placement.

This distinction is especially important for long bodies at large incidence: an acceptable estimate of side-on or normal loading can coexist with a less certain pitching moment because the latter depends additionally on the assumed longitudinal location of the effective crossflow load.

10.3.5. Supersonic and Hypersonic Body Methods

SpearDAT exposes three high-speed body-method selections: "VanDykeHybrid", "Sose", and "Hyper". They should be viewed as different modelling assumptions rather than as interchangeable ways to obtain the same solution.

The default Van Dyke hybrid method is a useful starting point for ordinary supersonic body analysis when its assumptions are satisfied. One explicit geometric requirement is that the pointed-nose tip angle must not exceed the local Mach angle. If that condition is violated, the hybrid method is not applicable and the body calculation falls back to second-order shock expansion. The high-speed stack also uses SOSE for blunted-nose cases and when a hybrid solution fails to remain physically admissible.

Second-Order Shock Expansion is therefore not merely an alternative output format. It is the more appropriate controlled assumption for cases involving stronger compression/expansion behaviour or when the hybrid method is geometrically invalid. The Missile DATCOM User's Manual recommends selecting SOSE when any Mach number is higher than approximately $M = 2$ [1]. This recommendation is a method-selection rule, not a claim that every other body calculation becomes impossible above Mach 2.

The "Hyper" selection uses a modified-Newtonian body method. The legacy manual allows that branch above $M = 1.4$ but states that it should normally be selected at Mach numbers greater than about 6 [1]. It is therefore best interpreted as a hypersonic/Newtonian approximation, particularly suitable when surface pressure is governed primarily by local inclination and impact-type behaviour rather than by the ordinary supersonic potential-flow assumptions.

Note

For a high-speed case whose result materially affects a design decision, method sensitivity can be informative. Agreement between Hybrid and SOSE over a conventional geometry increases confidence that the result is not dominated by one method-specific assumption. Large disagreement is a signal to inspect the geometry, local compression/expansion pattern, and available validation data rather than to select whichever result is more convenient.

10.3.6. Axial Force, Wave Drag and Base Drag

Body axial force is a synthesis of several contributions rather than one single-range correlation. Skin friction, forebody form drag, transonic wave drag, boattail or flare pressure drag, base drag, protuberance drag, and incidence-dependent drag can therefore have different applicability boundaries.

At subsonic/transonic conditions, forebody pressure drag changes treatment once the estimated drag-divergence Mach number is exceeded. The transonic wave-drag calculation is then faired between a subcritical pressure-drag value and supersonic area-rule results. This construction is appropriate to a preliminary-design method, but it is not a direct viscous solution of the complete transonic flow field.

The blunted-nose wave-drag charts are concentrated around $M \approx 0.8$ – 1.2 and a finite range of nose fineness and bluntness. Likewise, the conical-flare wave-drag chart is tabulated through approximately a 10° flare half-angle in the subsonic/transonic region. More unusual forebody or flare geometries should therefore be treated as correlation-edge cases even when the complete body analysis remains numerically available.

The boattail normal-force increment has a narrower supersonic similarity table than the overall high-speed body solver. Its direct chart support ends at a similarity value corresponding to approximately $M = 2.7$; beyond that point the contribution can rely on extrapolation even though the rest of the body solution remains fully active. This is a useful example of why a single “maximum body Mach number” would be misleading.

Base drag has a much broader built-in Mach range. The principal base-pressure tables extend to approximately $M = 10$, after which a $1/M^2$ continuation is used. Consequently, the uncertainty of total C_A at high Mach can be controlled by another contribution long before the base-drag model itself reaches the end of its direct table.

The incidence-dependent axial-force model also changes character at about

$$|\alpha| = 30^\circ.$$

Below that value the ordinary body force transformation is used; above it the high-incidence body treatment takes over, with an additional reverse-flow construction at very large incidence. This reinforces the use of $|\alpha| \lesssim 30^\circ$ as the routine body working region.

10.3.7. Body Pressure-Output Applicability

Body pressure distributions are an auxiliary engineering product of the supersonic body methods, not a CFD-quality surface solution. The Missile DATCOM User’s Manual restricts body pressure output to supersonic conditions and, for the Hybrid and SOSE methods, to approximately

$$\alpha \leq 15^\circ$$

[1]. This is narrower than the range over which total body force and moment coefficients can still be calculated.

The SOSE pressure path additionally reports local Mach number at zero angle of attack. Severe contour expansions or locally difficult shock-expansion states can make a local pressure or Mach state undefined even when the surrounding calculation remains numerically protected. SpearDAT exposes such non-physical local states rather than treating numerical completion as proof that the pressure method is aerodynamically valid there.

The modified-Newtonian hypersonic method can produce pressure distributions at larger incidence, but those values inherit the assumptions of Newtonian surface-impact theory. They should therefore be interpreted as hypersonic engineering pressure estimates, not as an extension of the Hybrid/SOSE pressure validation envelope.

10.3.8. Practical Interpretation of Body Results

For a conventional axisymmetric missile body, the most defensible routine use is a familiar body-of-revolution geometry inside the broad $L/D \approx 2\text{--}28$ envelope, at incidence up to roughly 30° , with the high-speed body method selected according to the flight regime and the geometry. Cone and tangent-ogive families with cylindrical centre sections and conventional boattails or flares are particularly natural applications of the methodology.

Outside that central region, the useful question is not simply whether SpearDAT returns a coefficient. The engineer should identify which part of the body build-up controls the result. Very slender or unusual noses may challenge transonic load-location correlations before they challenge normal-force magnitude; large incidence increases dependence on empirical crossflow;

aggressive flares or boattails can approach narrower drag correlations; and high-speed pressure results depend strongly on the selected Hybrid, SOSE, or Newtonian assumption.

Where body pitching moment, static margin, transonic drag rise, or local pressure is design-critical, inspect the body component outputs and repeat the analysis with a defensible alternative method or external aerodynamic source when the configuration lies near one of these applicability boundaries.

10.4. Specialised Aerodynamic Capabilities

Several SpearDAT capabilities sit outside the conventional static body-plus-fin prediction discussed in the preceding sections. Elliptical bodies modify the body-aerodynamic build-up itself, dynamic derivatives estimate local rate-dependent sensitivities, the spin/Magnus option applies a specialised body correlation, and the base-jet model estimates plume-induced changes to aft body flow. These capabilities should not be assigned one common flight envelope: each is governed by a different physical model and a different set of correlation assumptions.

The guidance below therefore separates four questions that are easy to combine incorrectly in routine use: how far a body may depart from circular cross sections, when a calculated aerodynamic derivative can be treated as a local linear derivative, where the empirical spin correlation is supported, and when the Chapman–Korst base-jet model is being used within its tabulated range.

10.4.1. Elliptical Bodies

SpearDAT’s elliptical-body capability represents a longitudinal sequence of elliptical cross sections. At each body station the two semi-axes may differ, and their ratio may vary along the body, so a configuration can transition from wider-than-high to higher-than-wide. The representation nevertheless remains an elliptical-section model: it should not be interpreted as a general arbitrary three-dimensional fuselage or blended-body solver.

The elliptical-body calculation is best understood as a correction to the ordinary axisymmetric-body calculation rather than as a completely separate aerodynamic solver. At each longitudinal station, SpearDAT first forms an equal-area circular section with

$$R_{eq} = \sqrt{WH},$$

where W and H are the physical elliptical half-width and half-height. The area of this circle is therefore the same as the area of the local ellipse. The resulting equivalent circular mould line is evaluated with the ordinary body methods at the requested Mach number and incidence.

That equivalent-circular calculation provides the baseline body aerodynamics. SpearDAT then compares the actual elliptical section with the equal-area circular section in the two principal body planes. Separate slender-body and nonlinear Newtonian correction ratios are integrated along the body and applied to the potential and nonlinear/viscous parts of the body normal-force and pitching-moment prediction. The two corrected planes are finally resolved at the requested aerodynamic roll angle. The elliptical correction can therefore change the reported body C_N and C_M , and after roll resolution can also change C_Y and C_{LN} .

There is no corresponding ellipse-specific correction to body axial force in this calculation. The body C_A produced for an elliptical configuration remains the result of the equal-area equivalent-circular body calculation. This distinction is important when a strongly non-circular

body is being used: the transverse force and moment channels receive explicit elliptical corrections, whereas axial force does not receive an independent correction for the non-circular cross-sectional shape.

The equivalent radius is therefore a geometric device used to construct the baseline body solution at every requested flight condition; it is not a separate low-speed or high-speed correlation. The Mach-number and incidence limitations of the underlying body methods described in [section 10.3](#) still apply. The elliptical corrections modify that baseline rather than replacing its aerodynamic method selection.

Cross-Sectional Axis Ratio

One part of the nonlinear elliptical correction depends explicitly on the local major-to-minor semi-axis ratio. If

$$A = \max(W, H), \quad B = \min(W, H),$$

then

$$\frac{A}{B} \geq 1.$$

This ratio is used by the empirical Newtonian correction applied to the nonlinear body normal-force contribution in each principal body plane. After the two corrected planes are resolved at the requested aerodynamic roll angle, that contribution can appear primarily in C_N or can be distributed between C_N and C_Y ; the associated load location also affects C_M and the roll-resolved C_{LN} . The ratio is therefore not a global accept/reject limit for the complete elliptical-body calculation, and it does not control the axial-force calculation.

The empirical table used by this nonlinear correction covers the physical major-to-minor ratio

$$1 \leq \frac{A}{B} \leq 20.$$

At $A/B = 20$, the calculation is using the final tabulated station. If $A/B > 20$, that nonlinear correction is extrapolated beyond its tabulated range.

The aerodynamic consequence of moving outside this range is specific to the elliptical correction rather than to the whole body solution. Uncertainty introduced by the extrapolation is most relevant to the nonlinear normal-force and pitching-moment prediction, and consequently to the corresponding roll-resolved side-force and yawing-moment components. The source correlation does not provide a quantitative error bound outside its tabulated range; thus this section does not prescribe one. Within the tabulated range, confidence still depends on the accuracy and applicability of the underlying body methods together with the empirical elliptical correction.

The implementation also treats a section that is numerically indistinguishable from circular as the continuous circular limit. This is a numerical robustness measure rather than an aerodynamic applicability boundary. Users should not attempt to exploit or avoid that tolerance when defining geometry; physically meaningful cross-section dimensions should be supplied instead.

Base-jet plume interaction is not available for elliptical-body configurations [1]. If plume interaction is important for a non-circular aft body, the current SpearDAT model does not provide a directly equivalent plume correction and an external aerodynamic assessment is appropriate.

Table 10.6.: Engineering interpretation of the elliptical-body applicability range.

Feature	Implemented / correlation information	Engineering interpretation
Cross-section geometry	Each local body section is an ellipse whose two semi-axes may vary longitudinally.	Arbitrary non-elliptical sections, chines, blended corners, and comparable three-dimensional shaping are not represented by this body model.
Equivalent-circular baseline	The local equal-area radius is $R_{eq} = \sqrt{WH}$, and the ordinary body calculation is evaluated on that equivalent circular mould line.	The underlying body Mach, incidence, nose, afterbody, pressure, and drag limitations still apply to the elliptical-body result.
Elliptical correction channels	Explicit corrections are applied to the normal-force and pitching-moment build-up in two principal body planes and are then resolved with aerodynamic roll.	The correction can affect C_N , C_M , C_Y , and C_{LN} . Body C_A remains on the equivalent-circular baseline and has no separate ellipse-specific correction.
Local major/minor ratio A/B	The nonlinear Newtonian correction is tabulated over $1 \leq A/B \leq 20$.	$A/B > 20$ requires extrapolation of that correction. Within the tabulated range, confidence remains governed by the applicability and accuracy of the underlying body methods and the empirical elliptical correction.

10.4.2. Dynamic Derivatives

The dynamic-derivative option calculates local first-order aerodynamic sensitivities associated with pitch rate, yaw rate, roll rate, and angle-of-attack rate. It is intended for linearised flight-dynamics work about a nominal operating condition. It is not a time-marching unsteady aerodynamic solution and does not impose a physical history of p , q , r , or $\dot{\alpha}$.

For the cumulative body–fin configurations, the pitch-, yaw-, and roll-rate terms are formed by perturbing the aerodynamic synthesis with an internal non-dimensional reduced-rate increment of

$$\hat{\omega} = 0.02$$

and differencing the perturbed and nominal static solutions. The corresponding difference is scaled by $1/0.02$. Body-alone damping relations and the fin $\dot{\alpha}$ contribution are evaluated through their dedicated methods and combined with these synthesis perturbations. The value 0.02 is therefore an internal derivative-construction step; it is not a recommended physical body rate and it is not entered by the MATLAB user.

The derivative validity necessarily inherits the assumptions of the underlying static aerodynamic model. If a fin normal-force method, body crossflow model, interference correction, or control-effectiveness correlation is already being used in an extrapolative region, the derivative obtained by perturbing that model is also extrapolative. A finite derivative does not restore confidence that was absent from the underlying static calculation.

Note

Treat the reported dynamic derivatives as local linear derivatives. For a stability or control analysis near an applicability boundary, repeat the SpearDAT calculation at neighbouring Mach numbers and incidence values. A derivative that changes abruptly as the nominal operating point moves through a method transition or a strongly nonlinear aerodynamic region deserves additional scrutiny before it is used in a linear model.

The derivative reporting convention also matters. Longitudinal rate derivatives use the usual $qL_{\text{ref}}/(2V)$ scaling, while lateral rates use the corresponding lateral reference length. Selecting degree- or radian-based derivative reporting changes the angular derivative convention; it does not change the physical perturbation model or impose a degrees-per-second or radians-per-second motion [1].

This capability should not be used as a substitute for a dedicated unsteady model when the design problem is governed by large-amplitude motion, dynamic stall, vortex-history effects, strongly time-dependent separation, aeroelastic response, or other phenomena for which the aerodynamic load depends materially on the previous motion rather than only on the local state.

10.4.3. Spin and Magnus Effects

The spin/Magnus calculation is a separate empirical correlation for a spin-stabilised body. Even when fins are present in the analysed configuration, the specialised spin quantities describe the body-alone correlation rather than the complete spinning body–fin vehicle [1]. This distinction is important when the result is used in a six-degree-of-freedom model: fin roll damping, fin asymmetry, control surface effects, and other complete-configuration spin phenomena are not added implicitly to the specialised Magnus result.

The correlation contains explicit Mach stations from approximately

$$M = 0.01 \quad \text{to} \quad M = 5.0.$$

It also reduces the physical body to a small set of calibre-based geometry quantities. Overall body fineness and the relative lengths of the nose and afterbody enter the empirical model separately. Satisfying the total-body fineness-ratio range therefore does not guarantee that the nose or afterbody also lies within its own correlation range.

The principal geometry ranges are

$$2.5 \leq \frac{L_{\text{body}}}{D} \leq 10, \quad 1.2 \leq \frac{L_{\text{nose}}}{D} \leq 5.5, \quad \frac{L_{\text{aft}}}{D} \leq 2.$$

Here L_{body} is the resolved total body length from nose to base, L_{nose} is the nose length, and L_{aft} is the afterbody or boattail length. The three ratios are retained separately because bodies with the same total L_{body}/D can have substantially different nose and afterbody proportions and therefore need not have the same spin/Magnus characteristics.

The diameter D is the resolved centre-section terminal diameter used by this specialised correlation. For a conventional missile with a nose that grows to a cylindrical main body and a boattail that subsequently contracts, this diameter will normally coincide with the main and maximum body diameter. It is not, however, defined as the maximum diameter of an arbitrary mould line. For a station-defined body, it is obtained at the interpreted downstream boundary of the centre section. SpearDAT stores the maximum mould-line diameter separately, and that maximum diameter should not be substituted mentally for D when interpreting the limits above.

Values outside the stated geometry intervals are reduced to the corresponding empirical boundary before the specialised correlation is evaluated. A finite result for a more extreme geometry therefore represents the boundary of the correlation rather than the original out-of-range geometry.

For the usual zero-sideslip condition, angle of attack places the crossflow in the pitch plane. Longitudinal spin then produces a transverse Magnus moment about the yaw axis. The specialised result can therefore be interpreted as an incidence-dependent yawing-moment derivative with respect to the reduced longitudinal spin rate. Using conventional notation, write this as

$$C_{n_p, \text{Mag}}(\alpha) = \sin \alpha \left[C_{NP1} + C_{NP3} \sin^2 \alpha + C_{NP5} \sin^4 \alpha \right].$$

The three reported coefficients C_{NP1} , C_{NP3} , and C_{NP5} therefore define *one* incidence-dependent Magnus yawing-moment derivative; they are not three independent moment increments.

For example, at $\alpha = 10^\circ$, the effective derivative is evaluated as

$$C_{n_p, \text{Mag}}(10^\circ) = \sin 10^\circ \left[C_{NP1} + C_{NP3} \sin^2 10^\circ + C_{NP5} \sin^4 10^\circ \right].$$

If the vehicle then has reduced longitudinal spin rate

$$\hat{p} = \frac{p L_{\text{atref}}}{2V},$$

the corresponding Magnus yawing-moment increment is

$$\Delta C_{LN, \text{Mag}} = C_{n_p, \text{Mag}}(10^\circ) \hat{p}.$$

At zero incidence the $\sin \alpha$ factor makes this Magnus yawing-moment contribution zero. At non-zero incidence, the polynomial describes how the yawing-moment sensitivity to spin changes with incidence.

The source correlation explicitly cautions against extending this incidence polynomial much beyond its first positive extremum. Here α is the independent variable, while C_{NP1} , C_{NP3} , and C_{NP5} are fixed for a particular body and Mach condition. Defining

$$f(\alpha) = \sin \alpha [C_{NP1} + C_{NP3} \sin^2 \alpha + C_{NP5} \sin^4 \alpha],$$

the location of an interior extremum satisfies

$$\frac{df}{d\alpha} = \cos \alpha [C_{NP1} + 3C_{NP3} \sin^2 \alpha + 5C_{NP5} \sin^4 \alpha] = 0.$$

For $0 < \alpha < 90^\circ$, the first positive extremum is therefore the smallest positive angle at which the bracketed polynomial becomes zero. Its location changes when the three correlation coefficients change, so two bodies with the same functional form can reach their first maximum or minimum at different incidence angles. This is why a single universal maximum angle of attack is not assigned to the Magnus correlation. Once the calculated incidence dependence has passed that first physical extremum and begins to develop the non-physical behaviour warned about by the source correlation, the result should be treated as outside the practical incidence range of the model.

Historical implementation differences affecting this specialised calculation are documented in [section A.11](#). The shared spin/Magnus calculation corrections are detailed in [section A.11.1](#), while the geometry-preparation correction is documented in [section A.11.2](#). These changes correct storage, interpolation, and geometry-preparation defects rather than enlarging the empirical correlation envelope. The limits above should therefore still be applied to the corrected SpearDAT results.

10.4.4. Base-Jet Plume Interaction

The base-jet option estimates how an operating jet modifies base pressure and induces separation on a cylindrical or contracting axisymmetric aft body. The calculation can also report the estimated separation location and Mach-cone angle at aft-fin roll positions. It is a specialised engineering correlation, not a plume-flow CFD calculation and not a general propulsion–airframe interaction model.

The mature Missile DATCOM documentation describes this option as a supersonic capability and recommends its use only above approximately $M = 1.2$ [1]. The stronger empirical range is narrower: the implemented Chapman–Korst database is tabulated for freestream Mach numbers from 2 to 5 and for the parameter ranges collected in [table 10.8](#). The solver can continue beyond several of these limits and issues an extrapolation warning; such a result belongs in the extended/caution region rather than the routine working envelope.

The configuration restrictions are as important as the scalar ranges. The base-jet calculation requires an axisymmetric body and a specified jet-exit diameter. It supports cylindrical aft bodies, conical boattails, and tangent ogives; an aft flare is rejected by the plume-interaction path. The equivalent base-jet option is not available for the elliptical-body definition [1].

For designs near the tabulated limits, inspect the reported base pressure and incremental ΔC_N , ΔC_M , and ΔC_A rather than judging the total configuration coefficients alone. If plume interaction is a major contributor to stability, control, or drag and the propulsion condition lies outside the ranges above, the result should be supported by propulsion wind-tunnel data, a dedicated plume model, or higher-fidelity coupled analysis.

Table 10.7.: Principal applicability bounds of the specialised body spin/Magnus correlation.

Quantity	Correlation range	Interpretation
Mach number	0.01–5.0	Outside the tabulated Mach stations, the spin/Magnus correlation is outside its direct empirical support.
Total body fineness L_{body}/D	2.5–10	L_{body} is the complete nose-to-base body length. The denominator D is the resolved centre-section terminal diameter, not a general maximum mould-line diameter. Values outside the interval are reduced to the boundary.
Nose fineness L_{nose}/D	1.2–5.5	This is an independent shape parameter of the correlation; satisfying the total-body fineness limit does not guarantee that the nose lies inside its own range.
Afterbody fineness L_{aft}/D	≤ 2	This is likewise an independent afterbody-proportion limit. Longer afterbodies are represented by the boundary value.
Incidence	Configuration dependent	The practical limit is tied to the first positive extremum of the incidence-dependent Magnus polynomial; its location changes with the correlation coefficients for the body and Mach condition.
Configuration	Body alone	The specialised correlation is not a complete spinning body–fin prediction.

Table 10.8.: Principal tabulated ranges for the base-jet plume-interaction correlation.

Parameter	Tabulated / accepted correlation range	Engineering interpretation
Freestream Mach number	2–5	The option is supersonic, but $1.2 < M < 2$ is already below the main Chapman–Korst database range.
Angle of attack	0° – 8°	Negative incidence and incidence above 8° require extrapolation of the separation correlation.
Jet/freestream static-pressure ratio	0–10	Outside this interval the base pressure correlation is extrapolative.
Jet/freestream stagnation-temperature ratio	4–10	Outside this interval the correlation is extrapolative.
Jet Mach number	$M_\infty - 1$ to $M_\infty + 1$	Larger mismatch between jet and freestream Mach lies outside the tabulated increment.
Nozzle exit half-angle	5° – 25°	Values outside the interval require extrapolation.
Boattail fineness L_B/D	≤ 2	More slender boattails lie outside the documented correlation range.
Boattail terminal angle	$\leq 12^\circ$	More aggressive contraction lies outside the documented range.
Nozzle-exit/base-diameter ratio	0.80–0.95	Smaller or larger exits lie outside the tabulated diameter-ratio range.

10.5. Inlets, Protuberances and Longitudinal Trim

The remaining optional installation features require a different kind of applicability judgement from the body and fin methods. An inlet is represented as an external five-station aerodynamic installation rather than as a complete internal-flow solution; a protuberance is represented primarily through its drag contribution rather than as a lifting component; and longitudinal trim is an interpolation procedure applied to repeated static aerodynamic solutions rather than a continuously iterated equilibrium solver. The corresponding input structures and examples are described in [sections 7.7, 8.1 and 8.2](#). This section concentrates on the aerodynamic interpretation of the resulting coefficients.

10.5.1. Inlet Aerodynamics

SpearDAT supports the three inlet families defined by the mature Missile DATCOM methodology: top-mounted two-dimensional inlets, side-mounted two-dimensional inlets, and axisymmetric inlets. The common inlet geometry can be repeated at different circumferential roll positions, and the installation may be diverter-mounted, conformal, semi-submerged, or covered [1]. The same documentation explicitly states that the method is not applicable to chin inlets.

Only one inlet geometry is defined within one SpearDAT inlet request. Multiple placements are repeated copies of that same five-station profile rather than independent inlet shapes. Geometrically different inlets should therefore be analysed in separate requests rather than represented as if they were repeated circumferential copies.

The five profile stations describe the inlet leading edge, cowl lip, midbody start, boattail start, and boattail end. This is a reduced external geometry, not an arbitrary inlet CAD surface or an internal-duct mesh. The final profile segment must contract: for a two-dimensional inlet,

$$H_5W_5 < H_4W_4,$$

while for an axisymmetric inlet,

$$W_5 < W_4.$$

These are geometric requirements of the implemented inlet representation, not claims that the physical inlet must close to zero area at the final station.

A covered inlet also has a specific meaning. It represents the historical face-plug treatment between the first two profile stations, with no internal flow admitted. It does not represent an arbitrary downstream duct closure. This distinction matters when experimental hardware contains an open inlet mouth with a plug farther downstream: that condition is not reproduced exactly by either the covered or uncovered idealisation.

Force and Moment Build-Up

The inlet increment is assembled from several engineering contributions rather than from one universal inlet correlation. The external transverse loading contains potential/slender-body terms, nonlinear crossflow/viscous terms, and the force associated with turning flow into an open inlet. The axial-force increment contains inlet skin-friction and pressure-drag terms and can also contain additive/spillage drag when that option is applicable. Installation offsets and circumferential placement convert these forces into the associated pitching, yawing, and rolling moments.

The inlet methods are documented for subsonic, transonic, and supersonic speeds [1], but the individual contributions do not share one universal Mach correlation. The nonlinear inlet crossflow model is evaluated separately in the pitch and lateral planes. For the pitch-plane contribution to C_N and C_M , the crossflow Mach number is

$$M_{\perp,\alpha} = M_{\infty} |\sin \alpha|,$$

where α is the flight-condition angle of attack. For the lateral-plane contribution to C_Y and C_{LN} , the corresponding quantity is

$$M_{\perp,\beta} = M_{\infty} |\sin \beta|,$$

where β is the stored flight-condition sideslip used by the inlet crossflow calculation.

The empirical crossflow drag-rise correlation is tabulated over

$$0.2 < M_{\perp} < 1.6.$$

The consequence of this limit therefore depends on both the freestream Mach number selected by the user and the relevant incidence angle. For example, at $M_{\infty} = 2$ and $\alpha = 10^\circ$,

$$M_{\perp,\alpha} \approx 0.35,$$

so the pitch-plane crossflow calculation remains inside the tabulated drag-rise interval even though the vehicle itself is supersonic. At $M_{\infty} = 4$ and $\alpha = 30^\circ$,

$$M_{\perp,\alpha} = 2.0,$$

which lies above the upper table station. At and outside the two tabulated endpoints, the implemented correlation uses the prescribed constant value $C_{D,\perp} = 1.2$ rather than extrapolating the drag-rise curve.

The tabulated crossflow-drag value represents an infinite-cylinder reference. A separate multiplier η , obtained from a derived inlet fineness ratio, accounts for the finite longitudinal extent of the inlet installation:

$$C_{D,\perp} = C_{D,\perp,\infty} \eta.$$

At low speed, this correction reduces the crossflow loading of a finite inlet relative to the infinite-cylinder idealisation, with shorter or less slender installations receiving the larger reduction. The full fineness-ratio correction is retained through $M_{\infty} \leq 0.9$, then progressively faired toward $\eta = 1$ over

$$0.9 < M_{\infty} < 1.2.$$

For $M_{\infty} \geq 1.2$, $\eta = 1$, so this particular finite-length reduction is no longer applied and the crossflow model uses the infinite-cylinder reference coefficient directly. This Mach fairing applies to the finite-length multiplier; it is separate from the $0.2 < M_{\perp} < 1.6$ drag-rise table described above.

One inherited modelling choice deserves explicit mention. The projected-planform linear inlet contribution is calculated internally but is suppressed in the final inlet normal-force, side-force, pitching-moment, and yawing-moment assembly. SpearDAT preserves this historical behaviour rather than enabling an unvalidated contribution. The implementation history and rationale are documented in [section A.6.3](#). The reported inlet increments should therefore be interpreted as the output of the inherited assembled method, not as a simple sum of every intermediate quantity that may appear in a method reconstruction.

Table 10.9.: Principal inlet applicability and modelling assumptions.

Feature		Implemented / documented capability	Engineering interpretation
Geometry family		Top-mounted two-dimensional, side-mounted two-dimensional, and axisymmetric inlets.	Chin inlets and arbitrary three-dimensional inlet installations are outside the documented method.
External profile		Five longitudinal stations with a contracting final segment.	The model is a reduced external aerodynamic representation, not a detailed cowl, centrebody, bleed-system, or internal-duct solution.
Circumferential placement		One common inlet profile may be repeated at arbitrary roll positions.	Different placement angles do not define different inlet geometries. Distinct inlet shapes require separate analyses.
Speed regime		General inlet aerodynamics are available at subsonic, transonic, and supersonic conditions.	Different force, moment, cross-flow, pressure-drag, and additive-drag terms use different correlations and transition behaviour.
Crossflow drag		Pitch-plane loading uses $M_{\perp,\alpha} = M_{\infty} \sin \alpha $; lateral-plane loading uses $M_{\perp,\beta} = M_{\infty} \sin \beta $. The drag-rise variation is tabulated for $0.2 < M_{\perp} < 1.6$.	The table limit depends on both freestream Mach and incidence. Outside the interval the prescribed $C_{D,\perp} = 1.2$ value is used rather than extrapolating the drag-rise curve.
Finite-length crossflow correction		A fineness-ratio multiplier modifies the infinite-cylinder crossflow coefficient at low speed, is faired toward unity over $0.9 < M_{\infty} < 1.2$, and is unity from $M_{\infty} = 1.2$ upward.	This is a finite-installation correction to the crossflow coefficient, not an additional Mach limit on the overall inlet calculation.
Covered inlet		Historical face-plug representation with internal flow suppressed.	Do not use the covered option as a generic model of any downstream duct closure or external blockage.

Additive and Spillage Drag

Additive drag represents the force associated with an external-compression inlet spilling part of its nominal capture streamtube at an off-design condition. It is therefore not a general inlet-performance model and does not predict the mass-flow ratio from the inlet geometry. The admitted capture-streamtube fraction is supplied by the user for each active speed condition.

The calculation is applied only when additive drag is requested, the inlet is open, and the freestream Mach number is at least unity. The mature documentation describes the intended application as a supersonic off-design external-compression inlet [1]. A covered inlet skips the calculation. The current SpearDAT interface requires

$$0 < \text{MFR} < 1$$

and one explicit mass-flow-ratio value for each active speed condition.

The most important interpretation limitation is its incidence treatment. Additive drag is evaluated once for the Mach condition at zero angle of attack, and the same axial-force increment is then applied across the complete angle-of-attack schedule for that Mach number. It should not be interpreted as an incidence-resolved spillage calculation. If inlet capture and spillage vary strongly with incidence, sideslip, control state, or forebody flow distortion, a dedicated inlet-performance or higher-fidelity flow model is more appropriate.

Attitude and Lateral-Increment Convention

The inlet lateral calculation retains one historical attitude convention that can be important in aerodynamic-roll sweeps. In a direct angle-of-attack and sideslip case, the stored sideslip is the expected body-axis value. In an incidence–aerodynamic-roll case, however, the direct inlet side-force terms use the single stored sideslip value rather than the row-dependent body-axis sideslip obtained by resolving the rolled incidence vector.

Consequently, an incidence–roll sweep whose stored sideslip is zero can report a zero direct inlet C_Y increment even on rows whose resolved body-axis sideslip is non-zero. Pitching, yawing, or rolling moments need not vanish at the same time because other inlet forces can act at circumferential and axial offsets. This convention is shared with the historical implementation and is documented in the inlet interpretation material of [section A.6.3](#). For studies in which inlet lateral loading is a primary quantity, explicit sideslip sweeps are easier to interpret than relying on an incidence–roll transformation.

10.5.2. Protuberance Drag

The stock protuberance capability is intended for non-lifting external items whose principal aerodynamic effect is drag: cylinders, blocks, streamlined fairings, launch lugs, and launch shoes. It should not be selected merely because a feature is geometrically small. A rail, strake, antenna, hardback, or other installation whose circumferential position produces important normal force, side force, pitching moment, or yawing moment requires a representation appropriate to those aerodynamic effects.

The fundamental limitation is that the protuberance model contributes *axial force only*. The drag of each item is evaluated at zero angle of attack, added to the body axial-force coefficient, and then treated as constant over the angle-of-attack schedule for that Mach condition [1]. The circumferential multiplicity specifies how many identical copies exist at one axial station, but

their individual azimuths are not supplied. The model therefore cannot resolve asymmetric lateral forces or moments from a clocked protuberance installation.

The item axial location is also aerodynamically meaningful rather than merely descriptive. It is used to estimate the local body boundary-layer thickness. Member height and stand-off are then used to reduce the drag when the protuberance is fully or partly immersed in that boundary layer. An item placed above the estimated boundary-layer edge receives no immersion reduction. The input location should therefore represent the approximate geometric centroid used by the correlation rather than an arbitrary convenient station.

Mach and Shape Dependence

The stock drag models use a common broad Mach architecture:

$$M \leq 0.6 \quad \text{low-speed correlation,}$$

$$0.6 < M < 1.0 \quad \text{cubic transition,}$$

and

$$M \geq 1.0 \quad \text{high-speed Newtonian/crossflow treatment.}$$

This is a method transition, not a statement that protuberance drag becomes invalid at $M = 0.6$ or $M = 1.0$. It does mean that a change in slope through this interval can arise from the intended correlation bridge rather than from a numerical discontinuity.

Several stock shapes contain additional geometry-dependent limiting behaviour. These should be interpreted as correlation behaviour rather than universal dimensions for all protuberances:

The high-speed branches use Newtonian-style pressure treatment together with a crossflow correction rather than extending the low-speed drag formula indefinitely. There is therefore no single user-facing “maximum Mach” that can be inferred merely from the $M = 0.6$ and $M = 1.0$ switch points. Confidence should instead be tied to whether the stock geometry is an adequate representation of the real installation and whether axial drag is the dominant aerodynamic effect.

Note

The stock protuberance correlations are most useful for drag-producing, non-lifting external hardware. If the feature is expected to alter stability or control through significant C_N , C_Y , C_M , C_{LN} , or C_{LL} , a finite protuberance C_A does not demonstrate that those omitted aerodynamic effects are negligible.

10.5.3. Longitudinal Static Trim

SpearDAT’s trim capability estimates a static longitudinal condition with zero pitching-moment coefficient by varying selected panels from one fin set. It is a pitch-trim calculation, not a general multi-axis equilibrium solution. The reported condition enforces

$$C_M = 0$$

within the trim procedure; it does not simultaneously require zero side force, yawing moment, rolling moment, translational acceleration, or rotational acceleration.

Table 10.10.: Selected geometry-dependent behaviour of the stock protuberance correlations.

Stock representation	Implemented geometry behaviour	Engineering interpretation
Cylinder aligned with the flow	At low speed the base drag coefficient varies with L/H between 0.6 and 1.4; values below 0.6 use the lower-end value and values above 1.4 use the upper-end value.	The low-speed length-ratio dependence is bounded rather than indefinitely extrapolated. Extreme L/H values therefore do not create a new trend beyond the two correlation ends.
Block / flat member	At low speed the length ratio $L/\min(W, H)$ changes treatment at 1.4 and 3.0. Above 3.0 the length correction remains at 0.45 of the underlying plate value.	Very long blocks do not receive an indefinitely decreasing length correction. Launch-lug and launch-shoe models inherit this member treatment.
Streamlined fairing	The low-speed fairing fineness relation changes formulation at $L/H = 20$.	This is a correlation switch rather than a hard geometry rejection and should not be interpreted as a universal maximum fairing fineness ratio.
Launch lug / shoe	Represented as the summed drag of four or three flat/block-like members, respectively.	The result is a component build-up approximation, not a resolved three-dimensional lug/shoe interference solution.

Only one fin set participates in a trim request. Any number of eligible panels within that set may be selected, with the pitch-oriented sign treatment described in [section 7.7](#). The aerodynamic limitations of the selected control surfaces still apply during trim. In particular, a plain trailing-edge device continues to use the same flap-effectiveness model and equivalent-deflection treatment discussed in [section 10.2.6](#); trim does not bypass those control-model assumptions.

Deflection Search and Interpolation

For each Mach condition, the trim procedure evaluates exactly ten equally spaced control-deflection values across the requested interval, including both endpoints. If explicit bounds are omitted, the historical interval is

$$-25^\circ \leq \delta \leq +20^\circ.$$

At each angle of attack, the resulting pitching-moment schedule is searched in deflection order. An exactly zero sampled value is accepted directly. Otherwise, the first adjacent pair with opposite signs is used to bracket the trim point. If the ten samples contain no zero and no sign change, trim is reported as unavailable for that angle of attack. The procedure does not extrapolate a trim solution outside the requested deflection interval.

Within a detected bracket, the trim deflection is obtained by linear interpolation of C_M . The remaining aerodynamic coefficients are then linearly interpolated from the same ten sampled solutions at that deflection. The solver is not rerun at the interpolated trim command. The reported C_M is therefore set to the trim condition by construction, while a separate fixed-deflection nonlinear calculation at the reported command can retain a small residual pitching moment.

The ten-point procedure is most reliable when $C_M(\delta)$ is locally smooth and approximately monotonic around the desired root. Strongly nonlinear control effectiveness, multiple zero crossings, a trim point very close to a method transition, or a deliberately broad deflection interval can all reduce the significance of the interpolated result. In such cases the reported trim deflection is best used as an initial estimate for a narrower SpearDAT sweep or an external numerical root search.

Taken together, the inlet, protuberance, and trim capabilities are valuable preliminary-design tools when their reduced physical representations match the engineering question being asked. Their numerical availability should not be used to infer internal-flow fidelity for an inlet, lateral-force fidelity for a protuberance, or exact nonlinear equilibrium for a trim solution.

Table 10.11.: Engineering interpretation of the longitudinal trim procedure.

Feature	Implemented behaviour	Engineering interpretation
Equilibrium quantity	Searches for $C_M = 0$ at each requested angle of attack.	This is static longitudinal trim, not simultaneous force-and-moment equilibrium in all axes.
Control scope	Selected panels from one fin set.	Multiple independent control sets are not solved simultaneously.
Deflection schedule	Ten equally spaced points including both requested bounds.	A very wide interval gives a coarser sampling of a nonlinear control response; a narrower physically plausible interval can improve the local interpolation.
Root selection	Exact sampled zero, otherwise the first adjacent sign change.	If several trim roots exist, the procedure returns the first bracket encountered in the sampled deflection order rather than performing a global root search.
No bracket	Trim result unavailable for that angle of attack.	The procedure does not extrapolate outside the requested control range.
Reported trim state	Linear interpolation of the sampled aerodynamic matrix; no final nonlinear rerun at the interpolated command.	The reported deflection is an engineering trim estimate. A fixed-deflection rerun or external root finder is appropriate when a tighter nonlinear $C_M = 0$ condition is required.

A. Corrections, Extensions, Portability Adaptations, and Known Differences

A.1. Purpose and Scope

This appendix records verified differences identified between SpearDAT and the historical Missile DATCOM implementation, together with selected historical behaviours that SpearDAT deliberately preserves and verified corrections that remain latent in the current public result path. The historical implementation is the publicly released software associated with the published Missile DATCOM manual [1].

It supports several related questions: why does a SpearDAT result differ from a historical result; why is an output or execution path available in one implementation but not the other; and why has a questionable historical construct been corrected, preserved, or documented as latent? Each entry names the affected capability, describes the historical and SpearDAT behaviours, states the conditions under which the issue is relevant, and records the evidence supporting the account.

The appendix is written to be read on its own. Where it uses a Missile DATCOM concept or a legacy programming term, it explains that term here rather than sending the reader elsewhere in this guide. Some of that material is repeated from earlier chapters deliberately: a reader consulting an entry should not have to page backwards to follow it.

Note

Most users will never need this appendix. Nothing in it is required to define an analysis, run SpearDAT, or interpret ordinary results. It is reference material for validation work, for comparison against historical results, and for engineering review.

A.2. What This Appendix Does Not Claim

This appendix records verified differences identified through the SpearDAT source-review and validation programme. It is not a proof that no additional differences exist. New entries may be added as validation coverage expands, but existing identifiers are never reused or renumbered.

Two further limits are worth stating plainly.

- An entry describes behaviour that has been demonstrated by the evidence recorded against it, and no more. Where the evidence is source-level analysis rather than a measured comparison, the entry says so, and it does not assert a magnitude.
- Absence of an entry for a given routine means no difference has been identified there. It does not mean the routine has been exhaustively reviewed.

A.3. What Is Included, and What Is Not

The register contains differences that a user can observe when running the historical implementation and SpearDAT on the same relevant case, including differences in numerical results, in availability, in output content, in diagnostics, or in whether the case completes at all.

Three classes of work are deliberately excluded.

SpearDAT development defects. Errors introduced during the C++ translation and later corrected to restore the intended or historical behaviour. These belong to the development record, not to the register of differences from Missile DATCOM.

Internal structural work. Changes to SpearDAT's internal storage, typing, validation, or data flow that leave externally relevant engineering behaviour unchanged.

Unverified or untraced candidates. A suspected defect, defensive guard, or source difference remains in the internal evidence record until both implementations have been examined and the affected quantity has been traced through its relevant consumers.

Three explicitly labelled exceptions are included even where no current public result differs.

Intentionally preserved behaviour. A questionable historical construct that SpearDAT has examined and deliberately reproduces, because there is not sufficient evidence to change it safely. These entries exist so that a reader who finds the construct in the historical source knows it was examined and that the decision was to leave it alone.

Latent verified corrections. A source-verified historical defect that SpearDAT corrects internally, but whose result does not reach the current public output along the verified consumer paths. These entries record that the construct was examined, and they become important if the suppressing path is ever changed.

Shared modelling conventions. Behaviour shared by both implementations that can easily be misread as a defect or a difference. Because no divergence or demonstrated defect exists, these conventions are documented alongside the relevant capability group without an entry number.

A.4. Missile DATCOM in Brief

Note

This section is an orientation for readers who have not used Missile DATCOM. Readers already familiar with it may proceed directly to [section A.5](#).

It is not a substitute for the published manual.

Cases and decks

Missile DATCOM is a batch program. The user writes an *input deck*: a text file of 80-column cards describing one or more *cases*. A case is one complete aerodynamic configuration together with the flight conditions at which it is to be evaluated. The program reads the deck, evaluates each case in turn, and writes its results to a set of output files.

Most of the deck is organised into *namelists* — named blocks of input values, each introduced by a name beginning with a dollar sign. The principal ones are:

Namelist	Governs
\$FLTCN	Flight conditions: Mach numbers, angles of attack, sideslip or roll, altitude or Reynolds number.
\$REFQ	Reference quantities used to non-dimensionalise the results, and the centre-of-gravity position.
\$AXIBOD	Axisymmetric body geometry.
\$ELLBOD	Elliptical body geometry.
\$FINSET	Fin-set geometry, one namelist per set, up to four.
\$DEFLCT	Control-surface deflections.
\$TRIM	Trim solution request and the panels used to trim.
\$INLET	Air-inlet geometry and placement.
\$PROTUB	Protuberances such as lugs, blocks, and fairings.
\$EXPR	Experimental data supplied to replace or increment computed values.

Users of the typed MATLAB interface normally do not write these decks by hand. SpearDAT converts the MATLAB input structure into a deterministic for005 deck automatically. The namelist names still matter here because they identify the historical capabilities and source paths discussed by the entries.

The coefficients

Aerodynamic results are reported as non-dimensional coefficients:

Symbol	Name	Meaning
CN	Normal force	Force perpendicular to the body axis, in the pitch plane.
CM	Pitching moment	Moment tending to rotate the vehicle nose-up or nose-down.
CA	Axial force	Force along the body axis; the drag-like component.
CY	Side force	Force perpendicular to the body axis, in the yaw plane.
CLN	Yawing moment	Moment tending to rotate the vehicle nose-left or nose-right.
CLL	Rolling moment	Moment tending to roll the vehicle about its own axis.

Because these are non-dimensional, each depends on a reference quantity appropriate to it. The force coefficients are non-dimensionalised using the reference area SREF. Pitching moment uses the longitudinal reference length LREF, while yawing and rolling moments use the lateral reference length LATREF. The centre-of-gravity position XCG defines the point about which the moments are reported.

Results using different reference quantities are not directly comparable until they have been transformed to a common convention. Several entries in this appendix concern exactly that transformation.

How a configuration is built up

Missile DATCOM works up to a complete configuration in stages. It first computes the body alone and each fin set alone, then combines them into body-fin configurations, then applies increments for inlets, protuberances, control deflections, and trim.

The combination step is not simple addition. Fins change the flow over the body and the body changes the flow over the fins, so the program applies interference and carry-over factors when it forms the combined result. This is why a configuration coefficient is not the sum of its component coefficients, and why a defect confined to one component can appear in a total that seems unrelated to it.

Angles

Missile DATCOM supports two related descriptions of incidence. A case may be defined using angle of attack and sideslip, or using total incidence together with aerodynamic roll. *Angle of attack* is the angle between the oncoming flow and the body axis in the pitch plane, and *sideslip* is the equivalent angle in the yaw plane. *Aerodynamic roll* describes the orientation of the vehicle about its own axis relative to the flow.

The program converts between these two representations internally. Several directional quantities become sensitive to that conversion near cardinal roll angles, where one description carries almost all of the incidence and the other carries almost none.

The legacy output files

The historical program writes to numbered files. SpearDAT retains them and can persist them on request, so a user working with retained output will encounter these names.

File	Contents
for005	The input deck.
for006	The main human-readable report, including an echo of the checked input.
for003	Machine-readable aerodynamic results. Written only when PLOT is requested.
for008	The checked and preprocessed input cards.
for009	Body geometry.
for010	Body pressure coefficients at incidence. Written when PRESSURES is requested, but may also be populated by the preserved legacy PLOT path for applicable body methods and conditions.
for011	Fin pressure coefficients. Written only when PRESSURES is requested.
for012	Body pressure coefficient and local Mach number at zero incidence for applicable body-pressure calculations. It may also be populated without an explicit PRESSURES request when the preserved legacy PLOT path enters the auxiliary body-pressure calculation.
for004	The COMMON-block dump produced by legacy WRITE cards.
for007	The internal FORMAT /WRITE-card scratch stream.

for002 is not listed because it is an internal namelist-reading scratch stream and is not exposed as a retained SpearDAT output.

Note

The conditional files are a common source of confusion. An empty `for003` usually means PLOT was not requested. The body-pressure streams `for010` and `for012` may, however, be populated even without an explicit PRESSURES request because the preserved legacy PLOT path can enter the auxiliary body-pressure calculation. The presence of one of these raw files therefore does not by itself show that pressure output was explicitly requested. Empty optional files likewise do not indicate a failed analysis.

A.5. Register Conventions and Evidence

This section defines how an entry is structured, what its classification fields mean, and what standard of evidence each entry meets. It also explains the handful of legacy programming concepts on which several entries turn.

A.5.1. How to Read an Entry

Each entry opens with an identifier of the form KD-`nnn` — KD for *known difference* — and a title, followed by a metadata block, followed by the account itself.

Field	Answers
Category	Why the difference exists.
Status	Whether the entry is published or has been superseded.
Compatibility	What relationship the difference creates between the two implementations.
Observability	Whether a user can see it.
Evidence	How strongly it has been demonstrated.
Affected	The user-facing capability, and then the routines involved.

Each field answers a distinct question. Explanatory prose is not duplicated across fields, even where one classification naturally implies another.

A.5.2. Category

Category	Meaning
calculation correction	A verified mathematical, typing, indexing, table, variable-selection, or storage defect is corrected.
numerical robustness	An undefined, singular, non-finite, out-of-bounds, overflow, or crash path is guarded.
capability completion	An incomplete, unavailable, or misleading historical capability is completed while retaining its engineering intent.
runtime portability	A compiler, linker, file-handling, or runtime assumption is not reliable on a contemporary toolchain.
output and reporting	Numerical methods are unchanged, but labels, columns, meta-data, diagnostics, or file content differ.
precision and representation	Differences are limited to floating-point precision, signed zero, final-digit rounding, or representation.
public-interface policy	An ambiguity in the historical deck interface is resolved through an implemented typed-input rule.
retained legacy behaviour	A questionable historical behaviour is deliberately preserved and documented.

This category is selective: it records only robustness changes that materially affect supported behaviour, result availability, or interpretation. Routine defensive checks and implementation-only improvements—such as local division-by-zero guards or replacing the legacy quadratic sorting routine with a more efficient implementation—are omitted unless they create a meaningful user-observable or compatibility difference.

A.5.3. Status

Value	Meaning
published	The entry meets the minimum evidence standard for its evidence label and forms part of this register.
superseded	The entry has been replaced. Its identifier is retained and carries a pointer to the replacement.

In a released register almost every entry reads *published*. The field earns its place chiefly by carrying *superseded* pointers, so that an identifier cited in an earlier revision of this guide still resolves.

A.5.4. Compatibility

Class	Meaning
numerically-different	A computed quantity differs.
precision-different	The same method is used; differences are limited to rounding, signed zero, or the final digit.
output-different	Engineering results are equivalent, but file content, labels, columns, metadata, or diagnostics differ.
availability-different	SpearDAT completes or exposes a capability where the affected historical path fails or is unusable.
intentionally-preserved	SpearDAT deliberately reproduces the historical behaviour rather than correcting it.

Some categories imply their compatibility class almost automatically — *retained legacy behaviour* implies *intentionally-preserved*, for instance. Both are stated regardless, because entries are read in isolation and a reader filtering on either field must be able to find the entry.

A.5.5. Observability

Value	Meaning
directly observable	Appears in normal output.
conditionally observable	Appears only under the stated conditions.
latent in verified paths	Does not reach any output along every path that has been examined.

For an entry describing a difference, this field asks whether a user can see *the difference*. For an entry recording a deliberately preserved behaviour there is no difference to see, so it asks instead whether a user can see *the retained behaviour itself*.

A.5.6. Evidence

Label	Meaning
S	Source verified. Both implementations have been read and the mechanism is structurally unambiguous.
R	Routine-level comparison against the historical program.
I	Integration test using real production dependencies.
F	Full-stack comparison of a complete generated deck.

An entry supported only by *S* is published only where the mechanism is unambiguous from the source, and it must trace the affected quantity to a result the user can see, or state explicitly that it does not reach one. Any statement of magnitude requires *F*.

A.5.7. Legacy Programming Concepts

Several entries turn on a property of the language the historical program was written in. This section explains those properties once. No knowledge of Fortran is assumed or required beyond what appears here.

Types are assigned by name

Fortran of this era assigns a type to a variable based on the first letter of its name unless the programmer states otherwise. Names beginning with the letters I through N hold whole numbers; all others hold real numbers.

This is convenient and dangerous in equal measure. A quantity that should hold a fractional value, but whose name happens to begin with one of those letters, silently holds only the whole-number part unless the programmer remembers to declare it. Several historical defects arise from this rule. In one verified inlet calculation, a length array that should have held fractional values was not declared explicitly and therefore held whole numbers, causing short lengths to be truncated to zero.

Shared storage laid out by position

The historical program shares data between its routines using *COMMON blocks*: regions of memory that several routines declare and access. Crucially, the correspondence is by *position*, not by name. Two routines may give the same region different names and different shapes, and the program will compile and run.

A disagreement in such declarations can therefore corrupt an unrelated value without producing a compile-time error. Entries involving this mechanism state the conflicting layouts explicitly.

Columns carry meaning

Source code of this era is written in a fixed layout where the column a character occupies determines its role. A misplaced digit, continuation marker, or field boundary can therefore alter a constant or a statement without producing an obvious syntax error. Entries involving fixed-form source explain the affected columns before quoting the construct.

Names and their layers

Four naming layers appear in this appendix, and entries distinguish them on first use:

Public SpearDAT fields are written as `reference.latref`. These are the names used by the MATLAB interface.

Legacy run or shared quantities are written as `LATR`. These represent the active case state inside the historical program.

Legacy local or input quantities are written as `LATREF`. These may represent values read from a particular input block, or values held locally by one routine.

Legacy routines are written as `SUBEXP`.

The distinction between the second and third layers matters more than it should. The historical program sometimes uses names differing by a single letter for quantities that are not interchangeable. Where a single entry involves several such names, it opens with a short table mapping each one to its meaning.

A.5.8. Further Conventions

Magnitudes

Where a difference has been measured, the entry quotes the measured values and names the case that produced them. Where it has not, the entry says nothing about magnitude. No entry estimates the size of a difference.

Quoted source

A few entries quote a short fragment of the historical source, where the mechanism is clearer from the construct than from any description of it. Every such fragment is preceded by a plain-language sentence explaining what it does, so that the fragment confirms the explanation rather than carrying it. Line numbers are not cited, since a reader is not assumed to have the historical listing.

Identifiers and maintenance

Entries are identified as KD-*nnn*, allocated in a single sequence independent of category. An identifier is never reused and never renumbered, including when an entry is reclassified. Entries are added as coverage expands and are never deleted; an entry that is superseded retains its identifier and carries a pointer to its replacement.

Entries are grouped by the engineering capability they affect.

A.6. Inlets and Diverter

An *inlet* is an air intake mounted on the body of an air-breathing configuration. A *diverter* spaces the inlet away from the body so that part of the body boundary layer is directed around the intake rather than into it.

Missile DATCOM treats the inlet as an increment applied after the basic body and fin configuration has been calculated. Normal-force, side-force, pitching- moment, and yawing-moment increments are assembled from potential-flow, flow-turning, linear planform, and viscous cross-flow contributions. Axial force is calculated separately from external wetted area, skin friction, pressure drag, and, when requested, additive drag. Geometry preparation and an equivalent axisymmetric-body construction support these aerodynamic calculations.

The findings in this section should therefore not be read as evidence that the inlet capability is generally unreliable. They are the exceptions isolated during a broader validation campaign. One produces a measured difference in a public coefficient, two are inherited modelling behaviours that SpearDAT deliberately preserves, three are verified source corrections that remain latent in the current public result path, and two define stable behaviour for historical geometry paths whose original results are not well defined.

A.6.1. Validation Coverage and Practical Significance

The inlet review combined source-level comparison with routine, integration, and full-stack testing. The historical Fortran and SpearDAT C++ paths were examined together, and the quantities produced by each affected routine were traced through their downstream consumers before an item was classified as user-visible or latent.

The full-stack inlet campaign covered the following areas.

Area	Coverage
Inlet geometry	Top-mounted two-dimensional, side-mounted two-dimensional, and axisymmetric inlets.
Installation	Diverter-mounted, conformal, and semi-submerged configurations.
Options	Covered inlets, additive drag, multiple identical circumferential placements, arbitrary inlet roll positions, and active inlet cases with dynamic-derivative and spin requests.
Execution range	Reduced diagnostic cases and full roll/Mach/incidence sweeps, including 73-case batches.
Compared evidence	Input acceptance, geometry and aerodynamic report sections, inlet incremental coefficients, final machine-readable aerodynamic output, and targeted intermediate values where a difference required root-cause isolation.

The current inlet findings can be interpreted at a glance as follows.

Entry	Relationship	Current user impact
KD-004	Corrected numerical behaviour	A measured increase in the inlet axial-force coefficient for affected diverter-mounted configurations.
KD-001	Intentionally preserved behaviour	No SpearDAT–historical difference. Both implementations suppress the same linear planform contribution.
<i>Shared modelling convention</i>	Shared modelling convention	No SpearDAT–historical difference. In ALPHA–PHI cases, direct inlet side force uses the stored scalar BETA, not the row-dependent sideslip derived from aerodynamic roll.
KD-002	Latent correction	No current public-result difference; the corrected quantity feeds only the suppressed contribution in KD-001 .
KD-003	Latent correction	No current public-result difference; the corrected centre of pressure feeds only the suppressed contribution in KD-001 .
KD-005	Latent correction	No difference has been observed through the supported execution path; the correction protects an unresolved-scale path.
KD-006	Robustness correction	Defines how the equivalent inlet body is trimmed when no valid preceding station exists. Ordinary inlet geometries are unaffected.
KD-007	Robustness correction	Defines the nose properties of a monotonically widening equivalent inlet body instead of passing unset values into the critical-Mach and pressure-drag path.

Note

The principal measured coefficient difference in the inlet comparison is [entry KD-004](#). Entries KD-002 to KD-005 are latent along the verified production paths, while entries KD-006 and KD-007 replace historically undefined geometry states with explicit behaviour. The unnumbered note records an inlet-attitude convention shared by both implementations.

A.6.2. User-Visible Numerical Correction

KD-004 Fractional diverter lengths were truncated in the wetted-area calculation

Category	calculation correction
Status	published
Compatibility	numerically-different
Observability	conditionally observable
Evidence	S, F
Affected	Inlet axial-force coefficient and therefore total configuration axial force; routine ILTSWT .

Part of an inlet's axial-force increment is the skin-friction drag acting on its external surface. The calculation depends on the *wetted area*: the area of inlet and diverter surface exposed to the external flow, after removing surfaces hidden by the body or by their mutual interface.

Historical implementation. The historical routine computes two diverter lengths and stores them in LD. The array is dimensioned but is not declared as real:

```
DIMENSION LD(2)
```

Under the implicit-typing rule described in [section A.5.7](#), a name beginning with L holds whole numbers unless declared otherwise. The fractional part of each positive diverter length is therefore discarded when it is stored. Any non-integer length is reduced, and a length below 1.0 in the solver's internal units becomes zero.

The diverter wetted area is calculated from those truncated lengths. The inlet–diverter overlap area is calculated independently and is still subtracted. The historical total can consequently omit some or all of the diverter surface while retaining the associated overlap subtraction.

SpearDAT implementation and motivation. SpearDAT stores both lengths as floating-point values and evaluates the existing wetted-area formula without truncation. This is a correction of the historical storage type, not a replacement aerodynamic method: the geometric formula, overlap treatment, skin-friction correlation, and subsequent coefficient build-up are otherwise unchanged.

Result impact. The correction is relevant when a positive diverter height activates the diverter branch and either computed length has a fractional part. The effect is largest when a length is below 1.0, because the historical value then becomes zero.

A full-stack comparison of the axisymmetric diverter case isolated the following values:

	Historical	SpearDAT
Diverter wetted area, SWETD	0.000000	0.264620
Total wetted area, SWET	0.497345	0.761965
Inlet axial-force coefficient, CA-INLT	0.002891	0.004430

For this geometry, the historical inlet axial-force coefficient is approximately 35 per cent below the corrected value. This percentage applies only to the inlet increment in this case. The change in total configuration axial force depends on the number of inlets and on the magnitude of the body, fin, pressure-drag, and other contributions in the same case.

Verification. The Fortran and C++ routines were compared directly. Intermediate wetted-area terms were instrumented in both implementations, and the resulting inlet increment was confirmed through the complete solver output.

Historical results for diverter-mounted configurations should be re-evaluated where inlet friction drag is important.

A.6.3. Intentionally Preserved Historical Behaviour

KD-001 The linear planform inlet contribution is explicitly suppressed

Category	retained legacy behaviour
Status	published
Compatibility	intentionally-preserved
Observability	directly observable
Evidence	S
Affected	Inlet normal force, side force, pitching moment, and yawing moment; routines LINFOR , LINXCP , and ILTARO .

The inlet method calculates a linear contribution from the projected inlet extents and the corresponding planform aspect ratios. It also calculates the longitudinal centre of pressure required to convert that force into pitching- and yawing-moment increments.

Historical implementation. The linear quantities are calculated and retained in working storage, but the final inlet build-up multiplies each one by zero. For example, the normal-force assembly contains:

```
CNINLT = CNP + 0.*CNLIN + CNCF + CNMOM
```

The same explicit suppression is applied to the linear pitching-moment, side-force, and yawing-moment terms.

SpearDAT implementation and motivation. SpearDAT deliberately preserves the zero multipliers. The available methodology and validation evidence do not establish why the historical implementation disabled the contribution, nor whether restoring it would improve agreement with experimental data. Enabling it would alter the inherited aerodynamic model for every case in which the linear term is non-zero. SpearDAT therefore retains the verified historical behaviour by default rather than introducing an unvalidated method change.

Result impact. There is no compatibility difference between SpearDAT and the historical program for this item: both omit the linear planform contribution from the reported inlet increments. The entry is relevant when a user reconstructs the inlet build-up from the published method or inspects the source and finds non-zero linear working quantities that do not appear in the final coefficients.

This suppression is also why the corrections in [entries KD-002](#) and [KD-003](#) remain latent in the current public result path.

Verification. The production, assembly, and reporting paths were traced in both implementations. The linear quantities are produced by **LINFOR** and **LINXCP**, but the reviewed final consumer in **ILTARO** suppresses them before the inlet coefficients are assembled.

A shared modelling convention: inlet sideslip in ALPHA–PHI cases

This behaviour is documented for interpretation rather than as a numbered entry. It is not a suspected defect that SpearDAT examined and chose to preserve; it is a modelling convention shared by both implementations, recorded because its output pattern is easily misread as an error.

Missile DATCOM permits the flight attitude to be supplied either as direct body-axis angle of attack and sideslip, or as total incidence together with an aerodynamic roll angle. For an ALPHA-PHI condition, **ALPBET** resolves the supplied attitude into row-dependent body-axis angles:

$$\alpha_{\text{body}} = \tan^{-1} (\tan \alpha_{\text{total}} \cos \phi_{\text{aero}}) ,$$

$$\beta_{\text{body}} = -\tan^{-1} (\tan \alpha_{\text{total}} \sin \phi_{\text{aero}}) .$$

For example,

$$\alpha_{\text{total}} = 10^{\circ}, \quad \phi_{\text{aero}} = 90^{\circ}$$

corresponds approximately to

$$\alpha_{\text{body}} = 0^{\circ}, \quad \beta_{\text{body}} = -10^{\circ}.$$

The inlet calculation does not use this row-dependent β_{body} when forming its direct side-force contribution. Instead, **ILTARO** uses the single stored flight-condition BETA value. In a typical ALPHA-PHI deck, that stored value remains zero because sideslip was not supplied as the independent flight variable.

The direct inlet lateral terms contain the factor

$$\cos \beta_{\text{stored}} \sin \beta_{\text{stored}}.$$

When $\beta_{\text{stored}} = 0$,

$$\cos 0 \sin 0 = 0,$$

so the reported direct inlet side-force increment CY-INLT can remain zero even though the resolved body-axis sideslip for that aerodynamic-roll row is non-zero.

A zero inlet side force does not require the inlet rolling- and yawing-moment increments to be zero. Missile DATCOM also forms moments by applying the inlet axial- and normal-force increments at the inlet's physical circumferential position. A side-mounted inlet can therefore produce

$$\text{CY-INLT} = 0, \quad \text{CLN-INLT} \neq 0, \quad \text{CLL-INLT} \neq 0.$$

This is not internally contradictory. The zero applies to the direct side-force component evaluated using stored BETA; the moments can also contain contributions from other inlet forces acting at an offset from the body centreline.

The behaviour was isolated by comparing two configurations differing only in how the attitude was supplied: an aerodynamic-roll sweep of a side-mounted inlet, in which CY-INLT remained zero while the inlet moment increments were non-zero, and an explicit-sideslip sweep of the same geometry, in which CY-INLT became non-zero as soon as the stored sideslip did.

SpearDAT deliberately retains this historical coupling. Replacing the stored BETA with the row-dependent sideslip from **ALPBET** would redefine the inlet incidence convention and alter accepted historical baselines.

Note

In an ALPHA-PHI sweep, a zero CY-INLT does not necessarily mean that the inlet has zero body-axis lateral incidence. It may mean only that the inherited inlet side-force model evaluated the contribution using the unchanged scalar BETA.

A.6.4. Latent Verified Corrections

The following corrections are present in SpearDAT but do not alter current public results along the verified production paths. They are recorded because the historical defects are source-verifiable and because they would become active if the suppressed linear contribution were restored or if an unresolved scale value reached the inlet geometry routine.

KD-002 Vertical projected inlet extents used the wrong running maxima

Category	calculation correction
Status	published
Compatibility	numerically-different
Observability	latent in verified paths
Evidence	S
Affected	Linear inlet side-force slope and vertical planform aspect ratio; routine LINFOR .

The linear inlet model requires the maximum projected inlet extent on each side of the body. The routine therefore keeps separate running maxima for right, left, upper, and lower extents while it processes the inlet placements.

Historical implementation. The right and left candidates are compared with the right and left maxima. The upper and lower candidates, however, are also compared with the *horizontal* maxima before being written to the vertical maxima:

Extent	Historical comparison	SpearDAT comparison
Upper	RB2 > RIMAX	RB2 > UPMAX
Lower	RB2 > LEMAX	RB2 > LOMAX

Because the historical test does not consult the vertical value it is about to replace, the stored upper or lower extent is not guaranteed to be the largest one encountered. It can also depend on the order in which inlet placements are listed.

SpearDAT implementation and motivation. SpearDAT compares each candidate with the maximum for the same plane. This restores the intended running-maximum operation and removes the input-order dependence from the intermediate vertical extent.

Result impact. The corrected extent feeds the linear side-force slope and vertical aspect ratio. Those quantities currently enter only the linear contribution suppressed by [entry KD-001](#). No public coefficient, report value, or MATLAB result changes along the verified paths.

Verification. The Fortran and C++ routines were compared directly, and the corrected values were traced to the suppressed linear consumer. No magnitude is quoted because the affected contribution does not reach current output.

KD-003 A supersonic yaw-plane centre of pressure was assigned to pitch-plane storage

Category	calculation correction
Status	published
Compatibility	numerically-different
Observability	latent in verified paths
Evidence	S
Affected	Linear inlet yawing-moment centre of pressure; routine LINXCP .

The linear inlet model evaluates separate centre-of-pressure curve fits in the pitch and yaw planes. In the supersonic branch, each plane has three ranges selected from a Mach–aspect-ratio parameter.

Historical implementation. For the middle yaw-plane range, the routine tests the yaw-plane parameter BETARB and evaluates the yaw-plane curve, but writes the result to the pitch-plane variable XCPA:

```
IF ((BETARB.GE.1.5).AND.(BETARB.LT.3.5)) XCPA =
1  -0.01122*BETARB**2.+0.07837*BETARB+0.33188
```

The pitch-plane value is therefore overwritten by a yaw-plane calculation, and the yaw-plane output XCPB is not assigned by this branch. From the routine interface, the resulting yaw-plane value is stale or undefined.

SpearDAT implementation and motivation. SpearDAT writes the middle-range yaw result to XCPB. This matches the variable used by the lower and upper yaw-plane ranges and the repeated pitch/yaw structure of the surrounding curve-fit code.

Result impact. The corrected centre of pressure is used only to form the linear yawing-moment contribution. That contribution is suppressed by [entry KD-001](#), so the correction does not alter current public coefficients or MATLAB output.

Verification. Both source implementations were compared directly. The five correctly formed neighbouring branches establish the intended destination unambiguously, and the affected result was traced to the suppressed linear consumer. No measured difference is claimed.

KD-005 The inlet-geometry scale test was always true

Category	calculation correction
Status	published
Compatibility	numerically-different
Observability	latent in verified paths
Evidence	S
Affected	Scaling of all inlet and diverter input dimensions; routine GEOINL .

Missile DATCOM provides a configuration scale factor. Inlet dimensions should be multiplied by it only when the factor is present and differs from one.

Historical implementation. The inlet geometry routine joins those two requirements with an *or*:

```
IF ((SCALE.NE.UNUSED).OR.(SCALE.NE.1.)) THEN
```

No value can make both comparisons false. A scale factor of one differs from the not-supplied marker, while the not-supplied marker differs from one. The historical condition is therefore always true, and the routine always enters the scaling block.

SpearDAT implementation and motivation. SpearDAT uses an *and*: the scale factor must be present *and* different from one. This implements the stated purpose of the guard. It also prevents all inlet dimensions from being multiplied by the near-zero not-supplied marker if the routine is called before the scale value has been resolved.

Result impact. No difference has been observed through the supported full-stack execution path. Upstream geometry processing normally resolves the scale factor to one before **GEOINL** is called, so the historical routine multiplies the geometry by one and produces the same result. The correction protects standalone, incomplete, or future call paths in which the unresolved marker could otherwise reach the inlet routine.

Verification. The condition and its correction were compared directly in the Fortran and C++ sources. The normal production path was also traced to the upstream scale normalisation. Because the defective branch is latent in the verified path, no numerical magnitude is claimed.

A.6.5. Defined Behaviour for Historically Undefined Geometry Paths

The following two corrections concern boundary cases in the construction and use of the equivalent axisymmetric inlet body. For ordinary inlet geometries, the historical and SpearDAT paths are unchanged. In the affected cases, however, the historical source either addresses storage outside the generated geometry or passes nose descriptors that were never assigned. There is therefore no stable historical numerical result that SpearDAT should reproduce.

KD-006 The equivalent inlet body could be copied from an invalid starting position

Category	numerical robustness
Status	published
Compatibility	availability-different
Observability	conditionally observable
Evidence	S
Affected	Equivalent axisymmetric geometry used by the inlet critical-Mach and pressure-drag calculations; routine ILTRVX .

Missile DATCOM estimates the inlet pressure-drag contribution by reducing the installed inlet geometry to a temporary *equivalent axisymmetric body*. This construction allows the existing body pressure-drag method to be reused for an inlet whose actual cross-section is generally non-axisymmetric. The surrogate exists only within the inlet axial-force calculation; it does not replace the physical inlet or the missile body elsewhere in the analysis.

The construction begins with the inlet geometry supplied by the user: axial stations, inlet width and height, and any applicable diverter or body-overlap description. At each station, the geometry routines use these quantities to form a signed effective cross-sectional area change

A_{eff} . Depending on the inlet type, this may represent a change relative to an initial inlet section, an inlet-plus-diverter area, or an area after the portion submerged within the body has been removed.

ILTRVX converts that signed area change into a signed equivalent radius:

$$R_{\text{eq}} = \begin{cases} \sqrt{A_{\text{eff}}/\pi}, & A_{\text{eff}} > 0, \\ 0, & A_{\text{eff}} = 0, \\ -\sqrt{|A_{\text{eff}}|/\pi}, & A_{\text{eff}} < 0. \end{cases}$$

Because R_{eq} is derived from a signed area change rather than entered as physical geometry, a negative value is only an internal marker. It means that the derived area change lies below the chosen reference level; it does not describe a circle, inlet surface, or missile surface with a physically negative radius. These negative signed values are excluded before the equivalent body is passed to the pressure-drag calculation.

Illustrative equivalent-body construction

Suppose, for illustration, that the inlet geometry stage uses 2.00 in^2 as its reference cross-sectional area. At three successive axial stations, suppose the corresponding physical sections have areas of 2.00 , 3.00 , and 4.00 in^2 . The area changes and equivalent radii are then:

Station	Section area	Effective change	Signed equivalent radius
1	2.00 in^2	0.00 in^2	0.000 in
2	3.00 in^2	$+1.00 \text{ in}^2$	$+\sqrt{1/\pi} = 0.564 \text{ in}$
3	4.00 in^2	$+2.00 \text{ in}^2$	$+\sqrt{2/\pi} = 0.798 \text{ in}$

For the pressure-drag method, this is treated like a small axisymmetric nose that begins at zero radius and widens through the following stations. If a section area were instead 1.50 in^2 , its effective change would be -0.50 in^2 , and the temporary signed radius would be

$$-\sqrt{0.50/\pi} = -0.399 \text{ in}.$$

Again, that value is not physical geometry. It marks a station lying below the reference-area level and is used only while identifying the part of the signed area-change curve that can form the positive-radius equivalent body.

After adding intermediate stations, the ordinary pattern expected by the historical trimming logic is a zero-radius anchor followed by one or more positive radii. The zero station defines the nose origin of the surrogate body.

Historical implementation. The routine scans for the first strictly positive signed radius. It then begins copying one station earlier, on the assumption that the preceding station is the valid zero-radius nose anchor:

```

DO 70 I=1,K
  IF (RTEMP2(I).GT.0.0) GO TO 80
70 CONTINUE
80 CONTINUE
M = 1
DO 90 N=I-1,K
  IF (RTEMP2(N).LT.0.0) GO TO 100
  X(M)=XTEMP2(N)
  R(M)=RTEMP2(N)
  M=M+1
90 CONTINUE

```

The first loop leaves I at the first positive-radius station. The second loop starts at $I - 1$, copies zero and positive radii, and stops if it encounters a negative signed radius. The code therefore does not intend to send negative radii to the pressure-drag method; their sign acts as a boundary marker.

The assumption that a valid station always exists at $I - 1$ creates two boundary failures.

First, the first generated radius can already be positive. For example, a covered inlet can retain the full positive area of its first physical section instead of beginning with the zero area-change station used by an uncovered inlet. In that situation $I = 1$, so the copy begins at station zero:

$$N = I - 1 = 0.$$

The temporary arrays are defined from station 1 onwards. Reading $XTEMP2(0)$ and $RTEMP2(0)$ therefore accesses storage outside the declared generated geometry. The value can depend on compiler layout, adjacent variables, and runtime checking.

Second, the derived area-change curve can contain no positive station at all. This can occur in a degenerate or method-inapplicable geometry for which the inlet area never rises above the selected reference level after the applicable baseline and overlap subtractions. In that case there is no positive axisymmetric surrogate to construct.

The historical search nevertheless reaches its end without recording that no positive radius was found. The subsequent $I-1$ start is then based on loop-exhaustion state rather than on a valid geometry location. The copied range is not a defined implementation of an empty equivalent body and can return an empty, partial, or otherwise runtime-dependent result.

SpearDAT implementation and motivation. SpearDAT records explicitly whether a positive radius was found.

If the first positive radius occurs after station 1, copying still begins at the preceding station, preserving the historical and intended boundary point. If the first positive radius is station 1, copying begins at station 1 because no valid predecessor exists. If no positive radius exists, SpearDAT returns an empty equivalent body:

```

if (first_pos == 0) {
  nstep = 0;
  return;
}

const int copy_start =
  (first_pos > 1) ? (first_pos - 1) : first_pos;

```


The correction does not alter interpolation, area-to-radius conversion, or the ordinary trimming rule. It only supplies explicit outcomes where the historical indexing cannot represent the requested predecessor.

Result impact. The correction is relevant only when:

- the first signed equivalent radius is already positive, so there is no preceding zero-radius anchor; or
- the generated area-change curve never becomes positive, so no equivalent pressure-drag body exists.

The usual pattern—a zero-radius anchor followed by positive radii—is unchanged.

In an affected historical run, the equivalent-body station count and geometry can depend on out-of-range or post-search storage. Those values feed the subsequent nose-property, critical-Mach, and pressure-drag calculations. Possible symptoms therefore include non-repeatable inlet axial force, invalid intermediate geometry, a runtime bounds failure, or no usable aerodynamic result.

SpearDAT instead produces a deterministic equivalent body, or an explicitly empty one where no positive body exists. The compatibility classification is therefore availability-different rather than numerically-different: there is no single defined historical coefficient against which the corrected boundary case can be compared.

Verification. The Fortran and C++ implementations were compared directly, together with the upstream construction of AREAI. That trace confirmed that the signed radius is derived from an effective area change and is not a user-entered physical radius. The normal path retains the same preceding zero-radius station and the same subsequent positive stations. The two boundary cases were traced to the historical I-1 start and to the explicit SpearDAT found-index checks.

No historical coefficient magnitude is quoted because the affected source path does not define a stable numerical result.

KD-007 Equivalent-body nose properties could remain unset when the radius increased to the final station

Category	numerical robustness
Status	published
Compatibility	availability-different
Observability	conditionally observable
Evidence	S
Affected	Inlet critical-Mach and pressure-drag calculations; equivalent-body nose length, fineness ratio, and volume in routine ILTARO .

For the inlet pressure-drag calculation, Missile DATCOM represents the installed inlet by the temporary axisymmetric surrogate described in [entry KD-006](#). Its axial stations and radii are derived from the inlet's effective cross-sectional area-change distribution. Only the usable zero-and-positive-radius portion is retained. This *equivalent body* is not the missile body or a literal reconstruction of the inlet; it is the simplified body of revolution supplied to the existing body pressure-drag methods.

Those downstream methods require the nose portion of the surrogate to be characterised separately from any following cylindrical, level, or contracting portion. After **ILTRVX** has assembled the equivalent-body stations, **ILTARO** therefore searches along their radii to locate the end of the surrogate nose. It tracks the largest radius encountered and normally treats the station immediately before the first radius that no longer increases as the nose endpoint.

From that identified endpoint it calculates:

- LNOSE, the equivalent nose length;
- FRNOSE, the equivalent nose fineness ratio; and
- VOLNOS, the volume enclosed by the equivalent nose.

These values are then used by the critical-Mach calculation and, where applicable, by the inlet pressure-drag calculation.

Historical implementation. All three quantities are assigned only inside the branch that detects a radius which is no greater than the preceding running maximum:

```

RMAX=R(1)
INDEX=1
DO 80 I=2,NSTEP
  IF (R(I).LE.RMAX) THEN
    LNOSE=X(INDEX)
    FRNOSE=LNOSE/(2.*RMAX)
    VOLNOS=DVOL(INDEX,X,R)
    GOTO 90
  ENDIF
  RMAX=R(I)
  INDEX=I
80 CONTINUE
90 CONTINUE

```

For a conventional equivalent body that reaches a maximum and then levels or narrows, this branch assigns the nose descriptors as intended.

For a body whose radius increases monotonically through the final station, the condition is never true. The loop finishes with the final station correctly held as the running maximum, but LNOSE, FRNOSE, and VOLNOS have never been assigned.

The routine nevertheless passes those values to the downstream calculations:

```

CALL MDIV(FRNOSE,NX,R,X,LNOSE,CRMACH)
...
CALL CDPRES(NX,X,R,SREF,MACH(IM),FRNOSE,
&          VOLNOS,LNOSE,CRMACH,CDPO,CAPR)

```

The resulting critical Mach number, pressure-drag branch, or runtime behaviour can therefore depend on whatever values occupy the uninitialised local storage.

SpearDAT implementation and motivation. SpearDAT retains the same running-maximum search. It records whether the historical stop condition was encountered. When no stop occurs, it uses the final and widest station as the nose endpoint and evaluates the same fineness-ratio and volume formulas there:

```

if (!nose_found && nstep >= 1) {
    lnose = x[index];
    frnose = (rmax > 0.0) ? lnose / (2.0 * rmax) : 0.0;
    volnos = dvol(index, x, r, state);
}

```

This is the continuous completion of the historical search logic: when every new station is wider than the previous maximum, the maximum is located at the last station.

Result impact. The correction is relevant only when the equivalent inlet body widens monotonically from its first station to its last. Bodies that level off or narrow before the final station follow the same branch in both implementations.

In an affected historical case, the unset values can influence:

- the critical Mach number used to choose the subsonic or transonic/supersonic pressure-drag path;
- the equivalent-nose inputs supplied to the wave-drag calculation; and
- whether those downstream routines complete with finite, repeatable values.

SpearDAT supplies deterministic nose properties based on the actual final maximum. As with [entry KD-006](#), no unique historical coefficient difference is stated because the original path does not define the values passed downstream.

Verification. The Fortran and C++ implementations were compared directly. The assignment sites and all downstream consumers of LNOSE, FRNOSE, and VOLNOS were traced. The SpearDAT fallback was confirmed to use the final running-maximum station and the same **DVOL** calculation used by the ordinary historical branch.

No measured full-stack magnitude is claimed. The correction is documented from the source-defined failure and the deterministic completion implemented by SpearDAT.

A.7. Input Processing and Runtime Portability

SpearDAT provides typed MATLAB and C++ inputs, but it deliberately retains the validated Missile DATCOM deck-processing path beneath those interfaces. Each public request is checked, expanded into scalar cases, and written as a deterministic for005 deck before it reaches the SpearDAT solver. The inherited card and namelist readers therefore remain part of the production execution path.

The historical program processes input as fixed-width card images. During case preparation it copies namelist cards into the internal for002 scratch deck, records their order, and later rewinds that scratch deck to read the active namelists. Experimental-data blocks are handled separately: **READNL** deliberately leaves \$EXPR blocks for the later **SUBEXP**/**REXP** scan, which can rewind and search the same scratch deck repeatedly.

This capability group currently contains one published difference. It concerns the low-level handling of end of file. It can determine whether an input path completes on a contemporary runtime, but it does not change the interpretation of a valid card, the accepted deck syntax, or any aerodynamic method.

Note

The practical distinction is availability, not numerical fidelity. When both implementations complete the same deck, this item does not by itself produce a coefficient difference. On an affected runtime, however, the historical executable can stop during input processing before aerodynamic results are available.

KD-008 Double end-of-file probing can fail on contemporary Fortran runtimes

Category	runtime portability
Status	published
Compatibility	availability-different
Observability	conditionally observable
Evidence	S, F
Affected	End-of-file handling in the shared 80-column card reader; observed while scanning the internal for002 deck for \$EXPR data. Routine READCD and its input-reader call paths.

The historical card reader does not report end of file after the first failed read. It attempts to read the same sequential file a second time and reports end of file only if both attempts fail.

Historical implementation. The two-read protocol is explicit in **READCD**:

```

DO 1000 J=1,2
  READ(IUNIT,1020,END=1000)(KOL(I),I=1,80)
  GO TO 1010
1000 CONTINUE
IEOF=.TRUE.
```

After the first read reaches the end marker, control returns to the loop and a second sequential read is issued without first repositioning the file. Some historical Fortran environments tolerated this convention. A contemporary sequential-file runtime may instead reject the second read because the file is still positioned after its end marker.

The failure was reproduced with a modern gfortran runtime while the experimental-data reader was scanning unit 2, the internal for002 scratch deck. The runtime terminated the read with the diagnostic:

```
Sequential READ or WRITE not allowed after EOF marker, possibly use REWIND
or BACKSPACE
```

The problem was exposed by the \$EXPR path because that path scans the scratch deck for matching Mach and configuration records and must also handle skipped records and a no-match end-of-file result. The underlying issue belongs to the shared physical card reader, not to the aerodynamic meaning of experimental-data substitution.

SpearDAT implementation and motivation. SpearDAT preserves the two-attempt logical protocol in **READCD**: a successful line is returned immediately, while two failed attempts set the end-of-file flag. The implementation uses a C++ input stream, where a second read request on a stream already carrying its end-of-file state fails normally rather than causing a Fortran runtime abort. The reader can therefore reproduce the inherited control flow without relying on an obsolete sequential-file convention.

This is a portability adaptation at the file-I/O boundary. It does not insert a card, discard a card, reinterpret a namelist, or change how a completed case is passed to the aerodynamic solver.

Result impact. The difference is relevant only when an input-reader path reaches the second end-of-file probe on a runtime that prohibits it.

- The historical executable terminates during input processing with an error, so the affected case produces no complete aerodynamic result.
- SpearDAT reports the end-of-file condition to the surrounding reader, which can then complete normal case termination or return the ordinary no-match result.
- For a deck that both implementations read successfully, this adaptation does not alter the resulting aerodynamic coefficients.

No special input syntax or user workaround is required in SpearDAT.

Verification. The historical and C++ implementations of **READCD** were compared directly. The modern-runtime failure was then reproduced through a complete experimental-data deck rather than inferred from the source alone.

To continue using the historical solver as an aerodynamic comparison oracle, a test-only Fortran replacement was introduced that returned `IEOF=.TRUE.` on the first end-of-file encounter. That replacement changed only the brittle physical-card operation. The historical **READIN**, **READNL**, namelist readers, **SUBEXP**, experimental-data transformations, and aerodynamic solver remained in place.

The SpearDAT input path was tested at several levels: low-level no-match and end-of-file handling, high-level namelist dispatch and scratch-deck rewinding, saved-case inheritance and termination, and complete experimental-data runs. This separation was necessary to distinguish a runtime-I/O limitation from a difference in input meaning or aerodynamic calculation.

Note

Other input-system changes made during the C++ translation—including typed namelist storage, corrected internal lookup-table initialisation, and stream ownership changes—are not listed here. They either restored the historical input behaviour after SpearDAT development issues were found or modernised internal data handling without creating a user-visible difference.

A.8. Flight Attitude and Directional Interpolation

Missile DATCOM can describe incidence in either of two forms. In *angle-of-attack and sideslip mode*, the body-axis angles ALPHA and BETA are supplied directly. In *total-incidence and aerodynamic-roll mode*, ALPHA represents the total incidence magnitude and PHI specifies the orientation of that incidence around the body axis. The routine **ALPBET** converts between the two representations.

The conversion has an important geometric consequence. At an aerodynamic roll of $+90^\circ$ or -90° , the incidence lies entirely in the body sideslip plane: the body-axis angle of attack is zero and the body-axis sideslip carries the incidence. At a roll near 180° , a positive total incidence can appear as a negative body-axis angle of attack. These sign and component changes are part of the Missile DATCOM convention; they are not known differences.

The entries in this section concern what happens when the historical numerical implementation operates at or near the points where one body-axis component vanishes. One correction stabilises the angle conversion itself. The other prevents a collapsed interpolation coordinate from producing unusable directional derivatives.

Entry	Affected operation	Practical consequence
KD-009	Body directional-derivative interpolation	Removes the large or overflowing derivative values produced by the historical implementation at cardinal roll angles.
KD-010	Conversion between the two attitude representations	Avoids division by a near-zero tangent and removes spurious residual angle components at cardinal roll angles.

Note

The corrections preserve the Missile DATCOM definitions of total incidence, body-axis angle of attack, sideslip, and aerodynamic roll. They change the numerical treatment of singular or nearly singular representations, not the meaning of the requested flight condition.

KD-009 Body directional-derivative interpolation collapsed at cardinal roll angles

Category	numerical robustness
Status	published
Compatibility	numerically-different
Observability	conditionally observable
Evidence	S, F
Affected	Body side-force and yawing-moment derivatives with respect to sideslip, and derivative outputs that consume them; routines BODYA and BODYE .

The body routines estimate directional derivatives by perturbing the body-axis sideslip angle by one degree. They convert that perturbed condition back into total incidence and aerodynamic roll, evaluate the appropriate body force and moment curves at the perturbed condition, and subtract the nominal values.

Historical implementation. For an axisymmetric body, the relevant part of the historical operation is:

```
BETNEW=BBETA(I)+1.
PHINEW=UNUSED
CALL ALPBET(BALPHA(I),ALTNEW,BETNEW,PHINEW)
CALL POLINT(BALPHA,CNB,NALP,ALTNEW,CNNPHI,DYDX)
CALL POLINT(BALPHA,CMB,NALP,ALTNEW,CMNPHI,DYDX)
```

The elliptical-body routine has the same interpolation structure, using its vertical-plane normal-force and pitching-moment curves.

The first argument to **POLINT** is the interpolation coordinate. Here it is the body-axis angle-of-attack schedule BALPHA. At an aerodynamic roll of $\pm 90^\circ$, every total-incidence station maps to a body-axis angle of attack of zero. In floating-point arithmetic the values are either identical or separated only by round-off of approximately machine precision.

Polynomial interpolation requires distinct coordinate values. Interpolating against the collapsed BALPHA schedule therefore introduces divisions by zero or by extremely small differences between neighbouring stations. The resulting body derivatives can be enormous even though the underlying force and moment curves remain finite.

Illustrative numerical example

Consider three total-incidence stations at an aerodynamic roll of $+90^\circ$. At this roll orientation, the incidence vector lies entirely in the body sideslip plane. The body-axis angle of attack BALPHA should therefore be exactly zero at every total-incidence station, while the corresponding body-axis sideslip carries the incidence. In floating-point arithmetic, however, the trigonometric conversion leaves extremely small non-zero BALPHA values because the computed cosine of 90° is not exactly zero. A representative floating-point evaluation of the same transformation gives:

ALPTOT	BALPHA	Illustrative curve value
0°	0.0°	0.0
5°	$3.1 \times 10^{-16}^\circ$	0.5
10°	$6.2 \times 10^{-16}^\circ$	1.0

At the 10° station, the corresponding body-axis sideslip is approximately -10° . The derivative calculation adds one degree, giving $BETNEW \approx -9^\circ$, and the forward angle conversion returns a perturbed total incidence $ALTNEW \approx 9^\circ$.

The historical interpolation therefore has the numerical form

$$Q(9^\circ) = \text{POLINT}(\{0, 3.1 \times 10^{-16}, 6.2 \times 10^{-16}\}, \{0, 0.5, 1.0\}).$$

The available interpolation coordinates span only 6.2×10^{-16} degrees, yet the requested coordinate is 9 degrees. Using the rounded values above in the inherited **POLINT** algorithm gives a value of approximately 6×10^{31} . Small changes in the round-off residues can change even the sign of that result. The number has no aerodynamic meaning; it is the consequence of extrapolating from an effectively zero-width coordinate interval.

SpearDAT first evaluates the spread of the proposed interpolation coordinate. For this example,

$$\max(\text{BALPHA}) - \min(\text{BALPHA}) = 6.2 \times 10^{-16} \text{ deg},$$

which is far below the collapse threshold for a 9° query. It therefore uses the distinct total-incidence schedule instead:

$$Q(9^\circ) = \text{POLINT}(\{0, 5, 10\}, \{0, 0.5, 1.0\}) = 0.9.$$

The value 0.9 is illustrative rather than a Missile DATCOM validation result. The important point is that SpearDAT evaluates the same finite force or moment curve using a coordinate that still distinguishes its stations.

SpearDAT implementation and motivation. SpearDAT checks whether the body-axis interpolation schedule has a meaningful spread. When it has collapsed, SpearDAT uses total incidence ALPTOT as the interpolation coordinate because that schedule remains distinct at all roll orientations.

The interpolation ordinates are also selected for the active incidence plane. For an axisymmetric body, SpearDAT uses the unrotated normal-force and pitching-moment curves. For an elliptical body, it uses the corresponding vertical-plane curves. Away from the collapsed condition, the historical BALPHA-based interpolation is retained.

This correction does not introduce a new aerodynamic correlation. It changes only the coordinate used to evaluate an existing force or moment curve when the historical coordinate no longer contains enough information to support an interpolation.

Result impact. The immediate affected quantities are the body side-force derivative CYB and yawing-moment derivative CLNB. They can also enter later configuration synthesis and derivative processing.

In complete roll-sweep validation, the historical implementation produced fixed-width overflow fields or derivative values many orders of magnitude larger than the neighbouring roll stations at $\text{PHI} = \pm 90^\circ$. The same instability appeared in the historical dynamic-derivative output as an extreme CYR value. SpearDAT removed the cardinal-roll spike and produced finite derivative values. Away from the singular roll stations, the compared derivative paths continued to match the historical output to its displayed precision.

Static force and moment coefficients that do not depend on this interpolation are not changed by the correction.

Verification. The Fortran and C++ implementations of both **BODYA** and **BODYE** were compared directly. The collapse follows explicitly from the **ALPBET** attitude transformation and the repeated BALPHA values supplied to **POLINT**.

The user-visible consequence was confirmed through full solver runs covering complete aerodynamic-roll sweeps, dynamic derivatives in degree and radian reporting modes, and the surrounding non-singular roll stations. Separate production-path tests exercised the axisymmetric and elliptical body executives with their real aerodynamic and interpolation dependencies.

KD-010 Attitude conversion used a near-zero tangent as a divisor

Category	numerical robustness
Status	published
Compatibility	numerically-different
Observability	conditionally observable
Evidence	S, I
Affected	Conversion between body-axis ALPHA/BETA and total-incidence/ PHI representations; routine ALPBET and its directional-derivative callers.

The forward mode of **ALPBET** converts body-axis angle of attack and sideslip into total incidence and aerodynamic roll. Geometrically, the aerodynamic roll angle is the direction of a two-component vector whose components are the tangents of the two body-axis angles.

Historical implementation. When both body-axis angles are non-zero, the historical routine forms their ratio and then applies manual quadrant corrections:

```
IF(ALPHA.NE.0. .AND. BETA.NE.0.) PHI=ATAN(TAN(BETA/RAD)/
1                                TAN(ALPHA/RAD))*RAD
```

Separate branches handle the exact cases in which ALPHA is zero. They do not protect the much more common numerical case in which ALPHA should be zero geometrically but contains a small residual from trigonometric round-off. The routine then divides by the tangent of that residual. Although the arctangent can still return a plausible angle, the operation is unnecessarily ill-conditioned and its quadrant depends on a sequence of exact comparisons and manual adjustments.

The reverse conversion has a related representation issue. At $\text{PHI} = \pm 90^\circ$, multiplication by the computed cosine of the roll angle can leave a body-axis angle-of-attack component of order 10^{-15} degrees instead of the exact geometric value zero. Downstream routines can then treat that residual as a genuine interpolation coordinate.

SpearDAT implementation and motivation. SpearDAT computes total tangent magnitude with a two-component hypotenuse operation and computes aerodynamic roll with a two-argument arctangent. The two-argument form determines the quadrant directly and does not divide one component by the other.

For the reverse conversion, SpearDAT preserves the historical trigonometric formula but rounds angle components smaller than a tightly bounded tolerance to zero. The convention and signs remain those of Missile DATCOM, including the distinction between positive and negative 180° representations.

The motivation is to express the same geometric conversion without depending on exact floating-point equality or division by a component that is geometrically zero.

Result impact. For ordinary non-zero angle components, both implementations produce the same attitude representation apart from expected final-digit precision. The difference becomes relevant when one or both components are zero or very close to zero, particularly:

- at aerodynamic roll angles near $+90^\circ$ or -90° ;
- when a sideslip perturbation is converted during body or configuration derivative calculations; and
- at zero total incidence, where both body-axis components should resolve to zero without assigning an arbitrary roll direction.

The correction prevents small representation errors from being amplified by downstream interpolation and finite-difference calculations. The separate interpolation correction that removes the demonstrated large derivative values is documented in [entry KD-009](#); this entry records the more general angle-conversion weakness on which that and other derivative paths depend.

Verification. The historical and SpearDAT implementations were compared directly. The two-argument arctangent and hypotenuse formulation is algebraically the same two-component construction as the historical ratio and square-root form, but remains defined when one component approaches zero.

The conversion was exercised through the production body and configuration-derivative paths, including sideslip perturbations and cardinal roll conditions. No independent coefficient

magnitude is assigned to this entry because the largest demonstrated output effect also includes the collapsed interpolation corrected separately by [entry KD-009](#).

A.9. Body Geometry and Definition

Missile DATCOM provides two fundamentally different ways to define a body.

Analytic definition (Option 1). The user supplies section lengths, diameters or widths, and shape types such as cone, ogive, power-series, or Haack. Those quantities generate the body station table.

Station definition (Option 2). The user supplies the physical mould line directly. An axisymmetric body is defined by NX/X/R; an elliptical body is defined by NX/X/W/H, with optional ellipticity and discontinuity data.

Section metadata such as LNOSE, LCENTR, LAFT, TNOSE, and TAFT can also be useful with a station-defined body. In that context, the station table should remain the physical geometry. The additional fields should identify the nose, centrebody, and afterbody portions of that geometry and classify those portions for downstream aerodynamic methods.

The historical implementation did not apply that distinction consistently. Some Option 1 assumptions were applied before the Option 2 station table was interpreted, and the elliptical-body reader used a particularly fragile rule for locating section boundaries. SpearDAT completes this mixed-input capability by treating the station table as authoritative and interpreting the section fields against it.

Entry	Body definition	Principal difference
KD-011	Axisymmetric AXIBOD Option 2	Prevents analytic-body diameter defaults from overriding section properties that can be derived from X/R; completes consistent section interpretation and diagnostics.
KD-012	Elliptical ELLBOD Option 2	Replaces exact station matching and an upstream index offset with physical section lengths and interpolated section boundaries on X/W/H.

Note

These entries concern mixed station geometry and section metadata. Pure station-defined bodies without that metadata, and analytic Option 1 bodies, retain their established interpretation. The changes do not replace the supplied mould line with an analytic shape.

KD-011 Axisymmetric section metadata could be overridden by analytic-body defaults

Category	capability completion
Status	published
Compatibility	numerically-different
Observability	conditionally observable
Evidence	S, I, F
Affected	Station-defined axisymmetric body section boundaries, derived diameters, method-classification flags, geometry reports, and downstream aerodynamic results; routines MAJERR , GEOAXI , and USERS .

When **NX** is supplied in **\$AXIBOD**, **GEOAXI** selects the coordinate-defined path and calls **USERS**. The physical body is therefore the radius schedule **X/R**, not an analytic nose, cylinder, or afterbody generated from section diameters.

The user may nevertheless supply **LNOSE** and **LCENTR** to identify where the nose and centrebody end. **USERS** already contained the useful ability to insert a new station when either boundary falls between two supplied stations.

Historical implementation. Before **USERS** is called, **MAJERR** checks section lengths and diameters using rules intended for analytic bodies:

```
IF(LNOSE .GT. UNUSED .AND. DNOSE .LE. UNUSED)DNOSE=1.
IF(LCENTR .GT. UNUSED .AND. DCENTR .EQ. UNUSED)DCENTR=DNOSE
```

It also prints warnings that a nose or centrebody base diameter was not supplied. Those messages are accompanied by state changes: the nose diameter is set to 1.0 and the centrebody diameter is copied from it.

For a station-defined body, both quantities can be obtained from the radius at the interpreted section boundary. The premature defaults can therefore make two decks with the same **X/R** mould line and the same section lengths produce different geometry properties and aerodynamic results solely because one deck explicitly repeats the boundary diameters and the other does not.

Illustrative interpretation

Consider the following part of a station-defined body:

Axial station <i>X</i>	Radius <i>R</i>
0	0
12	4
60	4
76	2

Suppose the user supplies **LNOSE** = 12 and **LCENTR** = 48, but omits **DNOSE** and **DCENTR**. The station table clearly places both section boundaries at radius 4, corresponding to diameter 8.

The historical error-checking defaults can instead place the value 1.0 in both diameter fields before the station geometry routine is reached. SpearDAT derives both diameters as 8 from the supplied mould line. The example is illustrative, but it shows why the old warning was

not merely cosmetic: its associated default could change quantities used by downstream body methods.

SpearDAT implementation and motivation. For coordinate-defined AXIBOD and ELLBOD bodies, SpearDAT prevents **MAJERR** from applying the analytic-body diameter substitutions. The warnings and defaults remain active for body paths where explicit analytic section dimensions are genuinely required.

Within **USERS**, SpearDAT applies the following interpretation:

- NX/X/R is the authoritative physical mould line.
- LNOSE is a distance from the body origin to the nose boundary.
- LCENTR is a length measured after the nose, so the centrebody boundary is LNOSE + LCENTR.
- If either boundary lies between stations, a radius is linearly interpolated and a new station is inserted.
- DISCON indices at or after an insertion are shifted so that existing discontinuities remain attached to the same physical stations.
- DNOSE, DCENTR, DAFT, and LAFT are derived from the interpreted station table. A supplied LAFT is treated as a consistency check rather than as a competing geometry definition.

SpearDAT also diagnoses negative or overlong section lengths, boundaries that must be inserted or clamped, and a centrebody definition that cannot be applied. Missing TNOSE and TAFT values are defaulted after the station geometry has been interpreted. These fields classify the nose and afterbody for downstream method selection; they do not generate or replace the Option 2 mould line.

Result impact. The difference is conditional on combining station-defined AXIBOD geometry with section metadata. It is most apparent when section diameters are omitted, when a boundary falls between supplied stations, or when the supplied lengths are inconsistent with the total body length.

The corrected interpretation can change:

- section fineness ratios and integrated geometry quantities;
- nose, centrebody, and afterbody method selection;
- body pressure, normal-force, moment, and axial-force calculations; and
- the geometry and aerodynamic data written to retained output files.

Pure NX/X/R cases without mixed section metadata are unchanged. Analytic Option 1 definitions continue to receive the historical warnings and defaults where appropriate.

Verification. The historical and SpearDAT implementations of **MAJERR**, **GEOAXI**, and **USERS** were compared directly. Testing first established that the legacy axisymmetric path already inserted intermediate section stations; the correction therefore retained that useful behaviour rather than replacing it.

Full-stack probe decks then compared otherwise equivalent definitions with and without explicit boundary diameters, exercised nose and centrebody insertion, verified DISCON index preservation, and covered inconsistent, negative, and overlong section metadata. After correction, no-diameter and consistent explicit-diameter definitions of the same station body produced the same interpreted geometry.

KD-012 Elliptical section lengths were not interpreted as physical lengths on the station-defined body

Category	capability completion
Status	published
Compatibility	numerically-different
Observability	conditionally observable
Evidence	S, I, F
Affected	Station-defined elliptical body section boundaries, equivalent circular geometry, geometry reports, pressure-method inputs, and downstream aerodynamic results; routines MAJERR , GEOELL , and USEREL .

When NX is supplied in \$ELLBOD, **GEOELL** selects the coordinate-defined path. The physical mould line comes from the station arrays X/W/H; the program also forms an equivalent circular radius from the width and height at each station for body methods that require it.

The section metadata should therefore partition the supplied station body without redefining it. In particular, LNOSE should identify the nose length from the body origin, and LCENTR should identify the centrebody length after the nose.

Historical implementation. The original **USEREL** instead searched for exact numerical matches and selected the station immediately before each match:

```
DO 1010 I=2,NNX
  IF(LNOSE .EQ. X(I))INOSE=I-1
  IF(LCENTR .EQ. X(I))ICENTR=I-1
1010 CONTINUE
```

This has three consequences.

- A boundary is recognised only when its value exactly equals an existing axial station.
- The selected boundary is shifted to the preceding station by the I-1 assignment.
- LCENTR is compared directly with an absolute X value rather than with LNOSE + LCENTR.

A section length that falls between stations cannot be represented by this path. Even an exact match does not select the station the user named.

Illustrative interpretation

Assume a station-defined elliptical body has axial stations at

$$X = \{0, 4, 8, 12, 24, 36, 48, 60, 76\}.$$

The user supplies

$$\text{LNOSE} = 12, \quad \text{LCENTR} = 48.$$

The natural physical interpretation is a nose from 0 to 12, a centrebody from 12 to 60, and an afterbody from 60 to 76.

The historical exact-match rule finds $X = 12$ but selects the preceding station $X = 8$. It separately compares the value 48 with the absolute station $X = 48$ and again selects the preceding station $X = 36$. The resulting section interpretation is therefore:

Section	Historical	SpearDAT
Nose	$0 \rightarrow 8$	$0 \rightarrow 12$
Centrebody	$8 \rightarrow 36$	$12 \rightarrow 60$
Afterbody	$36 \rightarrow 76$	$60 \rightarrow 76$

This example exposes the interpretation problem without depending on a particular aerodynamic case.

SpearDAT implementation and motivation. SpearDAT treats NX/X/W/H as the authoritative elliptical mould line and gives the section lengths their direct physical meaning:

- the nose boundary is at LNOSE;
- the centrebody boundary is at LNOSE + LCENTR; and
- the afterbody length is derived from the final station.

If a requested boundary lies between existing stations, SpearDAT inserts it. The width and height are interpolated from the neighbouring stations, the ellipticity and equivalent circular radius are recomputed, NX is updated, and affected DISCON indices are shifted.

This is more than a report-only convenience. The inserted station is retained in the active body state, consumed by the elliptical-body executive, and propagated into downstream equivalent-body and pressure-method calculations.

As in [entry KD-011](#), the old analytic-body diameter warnings and default substitutions are suppressed only for coordinate-defined Option 2 bodies. TNOSE and TAFT remain classification flags for the interpreted sections; they do not switch the case back to analytic geometry. SpearDAT reports inserted or clamped boundaries, inconsistent LAFT, and this classification-only use of the type flags.

Result impact. The difference applies when station-defined ELLBOD geometry is combined with section lengths or type metadata. The historical interpretation can partition the same physical station table at different axial locations, or fail to represent a requested boundary between stations. That can change:

- nose, centrebody, and afterbody dimensions and fineness ratios;
- integrated planform, wetted-area, volume, and centre quantities;
- the equivalent circular body used by selected aerodynamic methods;
- pressure and body aerodynamic results; and
- the geometry reported to the user.

Pure station-defined elliptical bodies without the mixed metadata remain on the established Option 2 path. Analytic ELLBOD Option 1 bodies are also unaffected.

Verification. The historical and SpearDAT implementations of **MAJERR**, **GEOELL**, and **USEREL** were compared directly. The exact-match and I-1 behaviour is explicit in the historical source.

The correction was then exercised across pure station-defined bodies, discontinuity handling, mixed section metadata, boundaries located both on and between stations, and clamped or

inconsistent section lengths. Regression runs confirmed that previously accepted pure Option 2 cases were unchanged while the mixed-metadata cases changed intentionally.

The inserted elliptical stations were traced beyond **USEREL**: they were present in the active body state, consumed by the elliptical-body aerodynamic executive, converted to equivalent circular radius, and passed into the downstream pressure-method path. This consumer trace establishes that the corrected section interpretation is part of the production aerodynamic calculation rather than an isolated geometry-report change.

Note

The **MAJERR** warning suppression is included within [entries KD-011](#) and [KD-012](#) rather than published as a separate entry. In the historical program the warnings were coupled to default substitutions that changed active geometry state, so they are part of the mechanism of the two capability differences rather than an independent reporting issue.

A.10. Fins, Controls, and Trim

Missile DATCOM can model as many as four longitudinal fin sets, each containing one to eight panels. A panel may be fixed, deflected as an all-moving surface, or represented with a trailing-edge flap. The program first calculates each fin set in isolation, then combines the fins with the body and with upstream or downstream sets through interference and carry-over methods.

The trim calculation is a further layer above that synthesis. For each angle of attack, Missile DATCOM evaluates the selected control panels at ten deflection angles, searches the resulting pitching-moment schedule for $CM = 0$, and interpolates the remaining coefficients at the trim deflection.

This section records differences at all three levels:

- primary fin geometry and aerodynamic calculations;
- user-defined airfoil preparation and panel-level synthesis; and
- trim input, trim search, and report generation.

The distinction matters because an error in an isolated fin quantity can propagate into the complete body-fin configuration, while a report-only error can leave every coefficient unchanged.

Entry	Type	Current significance
KD-013	Primary aerodynamic correction	Corrects the sign of fin normal force between the zero-lift angle and twice that angle.
KD-014	Geometry correction	Places a zero-root-span fourth fin set on the body using its own geometry rather than repeating the third-set test.
KD-015	User-airfoil correction	Initialises the leading-edge surface/thickness values and the optional supersonic shape factor consistently.
KD-016	Output correction	Prints each fin set using its own panel count. Coefficients are unchanged.
KD-017	Trim robustness	Returns the first sampled deflection when that point already satisfies $CM = 0$.
KD-018	Intentionally preserved behaviour	Retains one questionable three-panel equivalent-deflection assignment for historical compatibility.
KD-019	Capability completion	Makes all eight trim asymmetry flags available to the input reader, matching the eight flags consumed by the trim calculation.

Note

The normal-force correction in entry KD-013 is the most broadly significant item in this group because it changes a primary aerodynamic coefficient in an ordinary operating range for a cambered fin. The remaining entries apply to narrower geometry, airfoil, reporting, panel-layout, or trim conditions.

A.10.1. Fin Geometry and Aerodynamics

KD-013 Fin normal force used the wrong sign above the zero-lift angle

Category	calculation correction
Status	published
Compatibility	numerically-different
Observability	conditionally observable
Evidence	S, R
Affected	Fin-alone normal force and the body-fin configurations that consume it; routine FINCN .

A symmetric fin normally has a zero-lift angle close to zero. A cambered section can instead produce zero normal force at a non-zero incidence. During the fin-section analysis, Missile DATCOM determines that incidence and passes it into **FINCN** under the variable name **AL0**. The force calculation must therefore use the incidence relative to zero lift:

$$ALPT = ALPTOT - AL0.$$

A positive **ALPT** is above zero lift and should use the positive normal-force branch. A negative value is below zero lift and should use the negative branch.

Historical implementation. The historical routine calculates the corrected incidence and then compares it with the zero-lift angle a second time:

```
ALPT=ALPTOT(I)-ALO
...
IF (ALPT .LT. ALO) GO TO 1160
```

Substituting the definition of ALPT into the condition gives

$$ALPTOT < 2 ALO.$$

The documentation embedded in the historical **FINCN** source describes the positive branch as the case in which total incidence is greater than ALO. The implemented test does not switch branch until the incidence reaches twice ALO.

Illustrative sign error

For a fin with

$$ALO = 2^\circ, \quad ALPTOT = 3^\circ,$$

the physical incidence relative to zero lift is

$$ALPT = 3^\circ - 2^\circ = +1^\circ.$$

The fin is above its zero-lift angle and should therefore produce positive normal force under the routine's sign convention. The historical comparison instead evaluates

$$1^\circ < 2^\circ,$$

selects the below-zero-lift branch, takes the absolute incidence, and applies a negative sign to the linear and nonlinear force terms.

The erroneous interval is

$$ALO < ALPTOT < 2 ALO$$

for positive zero-lift angles, with the corresponding reversed interval for negative values.

SpearDAT implementation and motivation. SpearDAT selects the branch from the sign of the already corrected incidence:

$$ALPT \geq 0.$$

Equivalently, the switch occurs at $ALPTOT = ALO$, as stated by the historical source-file documentation and by the meaning of a zero-lift correction. The linear and nonlinear force expressions, panel projection, and subsequent interpolation remain otherwise unchanged.

The routine's Lucero correction is also unchanged. This is an original Missile DATCOM high-Mach multiplier applied only to the nonlinear part of fin normal force when a qualifying

supersonic fin planform and leading-edge condition are present. It is separate from the zero-lift-angle branch corrected here.

Result impact. The correction applies when a non-zero ALO is active. In the historical routine that correction is used for subsonic cases through Mach 0.8. Symmetric sections with ALO = 0 are unaffected because the historical and corrected branch thresholds then coincide.

For an affected cambered fin, the historical program can report a fin normal force with the wrong sign in the interval described above. The fin-alone coefficient is subsequently used by pitching-moment, body-fin synthesis, control, and derivative paths, so the difference can propagate into complete configuration results rather than remaining confined to a diagnostic table.

Verification. The Fortran and C++ implementations of **FINCN** were compared directly. A targeted historical routine case with ALO = 2° and ALPTOT = 3° reproduced the negative force, while the corrected C++ branch produced the positive result. Additional tests exercised subsonic zero-lift correction, transonic interpolation, the unchanged high-Mach Lucero correction, all four fin-set storage selections, and isolation of neighbouring fin sets.

No general percentage correction is quoted: its magnitude depends on fin geometry, section properties, panel orientation, Mach number, and the position of the requested incidence within the erroneous band.

KD-014 The fourth fin set was skipped when placing zero-root-span fins on the body

Category	calculation correction
Status	published
Compatibility	numerically-different
Observability	conditionally observable
Evidence	S, I
Affected	Fourth-fin-set span stations, derived fin geometry, and downstream fin/body aerodynamics; routine GEOM .

A fin-set root semi-span of zero tells Missile DATCOM that the supplied span is measured from the body surface. After the body has been generated, the geometry executive evaluates the local body radius at the fin mid-chord and adds that radius to the active span stations. This converts the exposed span description into the centreline-referenced geometry required by later fin methods.

Historical implementation. The historical source contains four nearly identical blocks, one for each fin set. The fourth block repeats the third set's storage:

```
1060 IF(FDATA3(2) .NE. 0.)GO TO 1080
      XCR02=FDATA3(102)+FDATA3(52)/2.
      ...
      IF(FDATA3(I+1) .NE. UNUSED)
1      FDATA3(I+1)=FDATA3(I+1)+RCR02
```

At the preceding third-set block, a zero root span has already been increased by the local body radius. The repeated test therefore normally sees a non-zero third-set value and exits immediately. Fin set 4 is never examined and its zero-root-span geometry remains unadjusted.

Illustrative placement

Assume the body radius at the fourth fin set's mid-chord is 2 inches and the user supplies the exposed semi-span stations

$$SSPAN = \{0, 6\} \text{ in.}$$

The zero first value means that the fin root begins at the body surface. The centreline-referenced span stations required by the subsequent fin methods are therefore

$$\{0, 6\} + \{2, 2\} = \{2, 8\} \text{ in.}$$

The historical fourth-set block re-examines the third set instead, so the fourth set can remain at $\{0, 6\}$ inches. SpearDAT performs the conversion for fin set 4 itself and obtains $\{2, 8\}$ inches. The example does not add material to the fin; it shows the missing conversion from a body-surface reference to the body-centreline reference used internally.

SpearDAT implementation and motivation. SpearDAT applies one common body-surface-placement operation independently to fin sets 1 through 4. For each set it:

- checks that a body is present and the first span station is zero;
- evaluates the body radius at the set's own mid-chord position;
- applies the elliptical-body projection when required; and
- adds that radius only to defined span entries for that set.

The correction restores the repeated structure evident in the first three historical blocks. It does not change the placement rule used for those sets.

Result impact. The difference requires all of the following:

- fin set 4 is defined;
- an axisymmetric or elliptical body is present;
- the fourth set's first semi-span station is zero; and
- the fin mid-chord lies within the body station range.

Under those conditions, the historical and SpearDAT fourth-fin geometries use different root locations and effective spans. The difference can propagate to fin area, aspect ratio, aerodynamic-centre locations, isolated fin coefficients, interference factors, and the complete body-plus-four-fin-set configuration. Fin sets 1 to 3 and fourth sets supplied with an explicit non-zero root station are unaffected.

Verification. The four Fortran geometry blocks were compared with the C++ common operation. Integration tests used the real body conversion, body geometry, fin geometry, and interpolation dependencies. They verified local-radius placement, preservation of unused span entries, independent fourth-set adjustment, and unchanged neighbouring fin-set data.

KD-015 User-defined airfoil preparation contained three initialisation defects

Category	calculation correction
Status	published
Compatibility	numerically-different
Observability	conditionally observable
Evidence	S
Affected	User-defined fin-section surface coordinates, thickness, camber, maximum thickness, thickness-gradient parameter, and optional supersonic shape factor; routine USEFOL .

A USER airfoil may be supplied in either of two equivalent forms:

- upper and lower surface ordinates YUN/YLN; or
- thickness and camber distributions THN/CAM.

USEFOL reconstructs the missing representation and derives the section quantities used by the fin aerodynamic methods. Three independent historical defects affect that preparation.

Historical implementation: stale index in the surface reconstruction. After counting the valid chord stations, the loop index I contains the station after the last valid point. In the thickness-and-camber input path, the routine correctly sets the first upper surface point to zero but writes the lower-surface zero to that stale index:

```
YUN(1)=0.
YLN(I)=0.
```

The leading-edge lower ordinate YLN(1) is therefore not initialised as intended.

This assignment does not replace a user-supplied thickness value. In this input path, the user has supplied THN/CAM, and **USEFOL** is constructing the missing upper- and lower-surface coordinates. The routine treats station 1 as the common leading-edge anchor: it explicitly sets YUN(1) to zero and intends to set YLN(1) to zero as well. Because the reconstruction loop begins at station 2, an uninitialised YLN(1) is never corrected later and leaves the derived surface description incomplete.

Historical implementation: stale index in the thickness reconstruction. In the opposite input path, a loop first computes camber at stations 2 through NP. On exit, I again holds the value after the last station. The routine then writes

```
THN(I)=0.
```

instead of setting the leading-edge thickness THN(1) to zero. That value is subsequently part of the thickness distribution used to determine maximum thickness and the near-leading-edge thickness-gradient parameter DELTAY.

This is the opposite conversion path. Here the user supplied the surface ordinates rather than the thickness distribution, so THN(1) is a derived value. Within the Missile DATCOM user-airfoil representation, setting it to zero records that the upper and lower surfaces meet at the leading-edge station and anchors the thickness curve used for subsequent interpolation. This matters particularly for DELTAY, which is formed from interpolated thickness values close to the leading edge.

Historical implementation: implicit integer shape factor. The optional variable KSHARP begins with the letter K. Because it is not declared explicitly in **USEFOL**, the routine treats the dummy

argument as an integer even though its caller supplies a real-valued fin work quantity. The intended default test

```
IF(KSHARP .EQ. UNUSED)KSHARP=0.
```

therefore compares the real sentinel with storage interpreted through the wrong type. The test does not reliably recognise an omitted value, so the default shape factor is not established.

SpearDAT implementation and motivation. SpearDAT writes the geometric leading-edge conditions explicitly:

Input representation	Explicit leading-edge value
THN/CAM supplied	YUN(1) = YLN(1) = 0
YUN/YLN supplied	CAM(1) = THN(1) = 0

It also declares KSHARP as a floating-point quantity and applies the intended zero default when the sentinel is present. The interpolation formulae used to reconstruct the remaining stations are unchanged.

Result impact. The entry applies only to fin sets using a user-defined section. The first two defects affect opposite input representations, while the third applies when the optional sharpness factor is omitted.

Depending on the supplied station table and which derived quantities are used by the active Mach regime, the historical values can alter the reconstructed airfoil coordinates, reported section geometry, maximum thickness, DELTAY, and section aerodynamic parameters. Those section properties feed the isolated fin normal-force, pitching-moment, and axial-force paths and can therefore reach complete configuration results.

The source establishes the incorrect destinations and type unambiguously, but no general magnitude is stated because the resulting difference is specific to the user-defined airfoil.

Verification. Both input representations and the default-setting path were reviewed in the Fortran and C++ sources. The stale-index destinations follow directly from the completed Fortran loops, and the KSHARP mismatch follows from the implicit-typing rule described in [section A.5.7](#). The C++ routine tests exercise reconstruction in both directions and the optional defaults.

A.10.2. Synthesis Output and Trim

KD-016 Panel-characteristics reports used the last fin set's panel count

Category	output and reporting
Status	published
Compatibility	output-different
Observability	conditionally observable
Evidence	S
Affected	Panel-characteristics sections of synthesis partial output; routine SYNPAR .

The synthesis partial report includes one panel-characteristics section for each fin set. Each row gives the panel number, its equivalent angle, and its normal-force contribution. The number of rows should therefore follow the NPANEL value of the fin set named in the section heading.

Historical implementation. In the first report loop, the historical routine assigns

```
NFINS=NFIN(ISET)
```

while printing each fin set in the presence of the body. When that loop ends, NFINS contains the count for the final set.

A separate loop then prints the panel-characteristics sections, but it never updates NFINS:

```
DO 1040 ISET=1,NSETS
...
DO 1000 IFIN=1,NFINS
```

Every section consequently uses the last active fin set's panel count.

For example, if fin set 1 has four panels and fin set 2 has three, the historical fin-set-1 section prints only three panel rows. Reversing the counts can make the first section print a fourth row that does not belong to its declared panel set.

SpearDAT implementation and motivation. SpearDAT selects NFIN(ISET) inside the panel-characteristics loop. Each heading and each group of rows therefore refer to the same fin set.

Result impact. This is a report-only difference. It appears when:

- synthesis partial output is requested; and
- at least two active fin sets have different panel counts.

The panel aerodynamic arrays and the complete configuration coefficients are not recalculated by **SYNPAR**; only the number of rows emitted for each section changes. A user diagnosing panel loads from the historical report can nevertheless be shown missing or unrelated panel data under the wrong fin-set heading.

Verification. The lifetime of NFINS across the two historical report loops was compared with the C++ per-set selection. The correction was tested with different panel counts and leaves the stored aerodynamic state unchanged.

KD-017 An exact trim at the first sampled deflection used an undefined index

Category	numerical robustness
Status	published
Compatibility	availability-different
Observability	conditionally observable
Evidence	S
Affected	Static longitudinal trim deflection and all coefficients interpolated at that deflection; routine TRIMIT .

For each requested angle of attack, the trim routine stores pitching moment at ten sampled control deflections. It searches for either an exact zero or a sign change between neighbouring samples. The index II identifies the sample at which the search succeeds.

Historical implementation. The ordinary search assigns II inside a loop beginning with the second sample:

```

      IF(CMUT(1) .EQ. 0.)GO TO 1140
      DO 1130 I=2,10
        II=I
        IF(CMUT(I) .EQ. 0.)GO TO 1140
        ...
1130  CONTINUE
1140  CONTINUE
      IF(CMUT(II) .EQ. 0.)DELTRM(IALP)=DELTAB(II)

```

When the first sampled deflection already gives $CMUT(1) = 0$, control jumps directly to the trim-result block without entering the loop. II has not been assigned for that angle of attack. The routine then uses it to index the moment schedule and the deflection schedule.

The historical result is compiler- and execution-state dependent: it can select an unrelated sample, access outside the intended range, or fail before a usable trim result is produced.

SpearDAT implementation and motivation. SpearDAT explicitly sets $II=1$ when the first sample is an exact solution. The trim deflection is then the first sampled value $DELTAB(1)$, and the remaining force and moment coefficients are interpolated at that same deflection.

This does not change the ten-point search, the sign-change interpolation, or the behaviour when the first point is non-zero. It defines the result of a corner case for which the intended answer is already present in the sampled data.

Result impact. The difference occurs only when the pitching moment is exactly zero at the lowest sampled deflection for a particular angle of attack. SpearDAT returns a deterministic valid trim point. The historical executable has no defined value for the index used to construct that row.

Exact equality is a narrow condition, but it can occur deliberately when a trim range starts at a known solution or incidentally through the displayed precision and aerodynamic build-up. Other angles of attack in the same run and trim solutions found at samples 2 through 10 are unaffected.

Verification. The branch and the first assignment of II were traced in the Fortran and C++ implementations. The C++ path was exercised with the first sample set to an exact pitching-moment zero and with ordinary exact-zero, sign-change, and no-trim cases later in the schedule.

KD-018 A three-panel equivalent-deflection assignment is deliberately preserved

Category	retained legacy behaviour
Status	published
Compatibility	intentionally-preserved
Observability	conditionally observable
Evidence	S
Affected	Panel-to-panel interference for one branch of a three-panel fin set; routine PANLCN .

A *physical panel* is one of the actual fin surfaces specified by the user through its panel angle and deflection. The panel-interference method used inside **PANLCN**, however, is organised around

four temporary cruciform positions. For each calculation, the physical panel being evaluated is placed in temporary position 1, and the remaining physical-panel deflections are mapped into positions 2 to 4. These temporary positions are a calculation device; they are not additional fins in the user's geometry.

For a three-panel set, one of the four temporary positions must represent an empty location. The mapping depends on which physical panel is being evaluated, the angular gaps between panels, and whether selected panels have zero deflection.

Historical implementation. In the final fallback branch for the third physical panel, the historical mapping is:

```
DELTMP(1)=DELT(3)
...
DELTMP(2)=DELT(1)
DELTMP(3)=0.0
DELTMP(4)=DELT(3)
```

The third panel's own deflection therefore appears twice in the equivalent cruciform arrangement, while the second physical panel's deflection is absent. Comparison with the neighbouring first- and second-panel branches suggests that DELT(2) may have been intended in the fourth temporary position, but the source does not provide enough evidence to establish that replacement as the validated aerodynamic model.

Illustrative three-panel mapping

Consider three physical panels at 0° , 120° , and 240° , with deflections

$$\text{DELT} = \{+5^\circ, -2^\circ, +3^\circ\}.$$

When the third physical panel is evaluated, its $+3^\circ$ deflection is placed in temporary position 1. For these panel spacings and non-zero first and second deflections, the final fallback branch is selected. The historical temporary arrangement is then:

Temporary position	Deflection	Physical origin
1	$+3^\circ$	panel 3, the panel being evaluated
2	$+5^\circ$	panel 1
3	0°	empty equivalent position
4	$+3^\circ$	panel 3 repeated

The physical panel-2 deflection of -2° is therefore absent. A symmetry-based reading of the neighbouring branches would place $\text{DELT}(2) = -2^\circ$ in temporary position 4 instead, but that alternative has not been validated as the intended aerodynamic method. SpearDAT consequently retains the historical $+3^\circ$ assignment.

SpearDAT implementation and motivation. SpearDAT reproduces the historical $\text{DELTMP}(4) = \text{DELT}(3)$ assignment. Changing it would alter panel-interference, equivalent-angle, panel-force, and complete configuration results for affected three-panel cases without an independent experimental or methodological basis for selecting the alternative.

Result impact. There is no SpearDAT–historical compatibility difference for this item. The retained behaviour is relevant only when all of the following select the final fallback branch:

- the fin set contains exactly three panels;
- the third panel is being evaluated;
- none of the three angular-gap tests identifies a gap greater than 120°; and
- the preceding zero-deflection tests do not select another mapping.

When that branch is active, both implementations use the same duplicated deflection in the equivalent interference model. The entry is recorded so that a reader inspecting the asymmetry in the source does not mistake it for an unreviewed translation error.

Verification. The complete three-panel branch was compared statement by statement with its C++ translation and with the analogous mappings for the other evaluated panels. The questionable assignment is explicit and is preserved in the production C++ routine.

KD-019 Only four of eight trim asymmetry flags were exposed by the historical reader

Category	capability completion
Status	published
Compatibility	availability-different
Observability	conditionally observable
Evidence	S
Affected	Input and use of ASYM(5:8) in the \$TRIM namelist; routines RTRIM and TRIMIT .

The trim namelist provides one ASYM flag per panel. A true value reverses the automatically selected trim-deflection sense for that panel. This allows the user to recover the basic panel-deflection sign convention or to construct an intentionally asymmetric trim pattern.

The trim calculation declares and consumes eight flags:

```
COMMON /TRIMIN/ ... ASY(8)
...
DO 1052 II=1,8
...
IF(ASY(II))DELTA1(II)=-DELTA1(II)
```

Equivalent loops are used for all four possible trim fin sets.

Historical implementation. The historical \$TRIM reader is table-driven. The routine **RTRIM** does not parse each public variable independently. Instead, it passes the variable names, their storage locations, and an IDIM descriptor table to the generic namelist parser **NAMER**.

There are twelve IDIM entries because the \$TRIM namelist exposes twelve variable names:

Variable or group	IDIM	Parser meaning
SET	+1	one real scalar
PANL1–PANL8	–1 for each	one logical scalar for each panel-selection flag
DELMIN, DELMAX	+1 for each	one real scalar for each deflection limit
ASYM	–4	logical array with four accessible entries

For this parser, the sign identifies the input type: positive entries describe real values and negative entries describe logical values. The absolute value gives the number of entries that may be read. Thus, –1 describes one logical scalar, while –4 describes a four-element logical array.

This metadata controls only how the input deck is decoded. After **RTRIM** has populated the trim state, **TRIMIT** performs the aerodynamic trim calculation using those stored values.

The historical table ends with:

```
DATA IDIM / 1,-1,-1,-1,-1,-1,-1,-1,-1,1,1,-4 /
```

Consequently, the parser exposes only ASYM(1:4). This conflicts with both the documented eight-element ASYM input and the later **TRIMIT** loop, which checks one asymmetry flag for each of panels 1 through 8. A deck request such as ASYM(6)=.TRUE. therefore has no corresponding historical parser destination.

SpearDAT implementation and motivation. SpearDAT changes the final parser dimension from –4 to –8 and maps it to eight logical storage entries. The generic reader can therefore populate ASYM(1:8), matching the values subsequently consumed by **TRIMIT**. This matches:

- the declared ASY(8) calculation state;
- the eight public panel-selection flags PANL1 through PANL8; and
- the loops in **TRIMIT** that apply asymmetry by panel.

The default remains false for every panel. Existing decks that specify only the first four flags retain their established behaviour.

Result impact. The difference is relevant when a trim fin set contains five to eight panels and the user requests a reversed trim-deflection sense for one of those later panels. SpearDAT can represent and apply that request. The historical reader does not expose the corresponding array elements, so those panels retain their default sign handling.

Because the flags alter individual panel deflections before configuration synthesis, the difference can affect the computed trim deflection and the normal, axial, side-force, pitching-, yawing-, and rolling-moment coefficients reported at trim.

Verification. The array dimensions in **RTRIM** were compared with the ASY(8) declaration and the eight-panel loops in **TRIMIT**. The SpearDAT reader maps all eight logical values into the same typed array consumed by the trim calculation.

Note

The source review changed the preliminary classification of [entry KD-019](#). It is not an intentionally preserved four-flag limitation: the SpearDAT reader exposes eight flags,

while the historical reader exposes four even though the calculation consumes eight. It is therefore recorded as a completed input capability.

A.11. Dynamic Derivatives, Spin, and Magnus

Missile DATCOM's **DAMP** option calculates aerodynamic derivatives associated with pitch, yaw, roll, and angle-of-attack rate. These are coefficient derivatives under DATCOM's nondimensional reduced-rate conventions; they are not the response of a time-domain simulation to a physical rate expressed in degrees or radians per second.

The **SPIN** option activates a separate, specialised correlation for a spin-stabilised body. In the verified execution path, **BDAMP** prepares the body geometry and calls **SPIN83**. That routine calculates six scalar quantities for each Mach condition:

Quantity	Meaning in SPIN83
CLLP	Rolling moment due to roll rate: the body roll-damping derivative.
CYP	Side force due to roll rate.
CNP1, CNP3, CNP5	Coefficients of the incidence-dependent Magnus moment representation.
CNPY5	The five-degree secant value used to describe the Magnus moment slope.

Four of the entries below arise inside this one routine. The first is a shared storage-layout mismatch and dominates the historical full-stack symptoms: it simultaneously corrupts an unrelated angle-of-attack-rate derivative and prevents the calculated Magnus quantities from reaching their intended report fields. The remaining **SPIN83** corrections concern its Mach table, output initialisation, and treatment of an input argument. A fifth entry concerns the equivalent nose geometry prepared by **BDAMP** before the spin correlation is called.

Entry	Mechanism	Practical significance
KD-020	Conflicting /DBODY/ layouts	Corrupts the first six body CNAD values and leaves the printed Magnus quantities stale.
KD-021	Malformed fixed-form Mach table	Corrupts the intended high-Mach interpolation in most SPIN83 outputs; masked in the historical report by the KD-020 storage mismatch.
KD-022	Two misspelled output initialisers	Latent in the verified path because later interpolation overwrites the intended outputs.
KD-023	Input argument modified in place	Latent in the current caller, but changes the caller's distance from physical units to body diameters.
KD-024	Equivalent ogive radius left unset	Can make spin roll damping non-finite for a coordinate-defined body.

Note

The verified historical deck behaviour distinguishes DAMP from SPIN. DAMP is the principal path for the ordinary dynamic-derivative tables. Enabling SPIN alone does not necessarily produce the same useful printed sections as a combined DAMP and SPIN run. The entries below describe the specialised spin/Magnus calculation when that calculation is reached.

A.11.1. The Shared SPIN83 Calculation**KD-020 Magnus results overwrite the first six angle-of-attack-rate derivatives**

Category	calculation correction
Status	published
Compatibility	numerically-different
Observability	conditionally observable
Evidence	S, F
Affected	Body CNAD(1:6) values and all six printed Magnus quantities when the spin calculation is active; routines BDAMP , SPIN83 , and their dynamic-derivative report consumers.

The historical program shares body dynamic derivatives through the /DBODY/ common block. In a Fortran common block, variables correspond by position in shared memory rather than by name. Every routine using the block must therefore agree on the number, size, and order of all preceding values.

Historical implementation. The declaration used by **BDAMP** places two twenty-element angle-of-attack-rate derivative arrays immediately before the six Magnus scalars:

```
COMMON /DBODY/ ... BCNAD(20), BCMAD(20),
1  CLLP, CYP, CNP1, CNP3, CNP5, CNPY5
```

In that layout, BCNAD occupies positions 361 through 380, BCMAD occupies positions 381 through 400, and the Magnus scalars occupy positions 401 through 406.

SPIN83 declares the same shared block differently:

```
COMMON /DBODY/ DUM(360), CLLP, CYP, CNP1, CNP3, CNP5, CNPY5,
1  DUM1(40)
```

The resulting positional disagreement is:

Shared positions	BDAMP and report meaning	SPIN83 meaning
361–366	BCNAD(1:6)	CLLP, CYP, CNP1, CNP3, CNP5, CNPY5
401–406	The six intended Magnus scalars	Part of the unused DUM1 region

The Magnus calculation therefore succeeds numerically but writes its answers over the first six BCNAD elements. Those elements are the body normal-force derivatives associated with angle-of-attack rate. The report later reads the intended Magnus positions 401 through 406, which **SPIN83** never updated.

SpearDAT implementation and motivation. SpearDAT does not reproduce the conflicting memory views. The ordinary dynamic derivatives and the six Magnus scalars are separate

named fields in the typed body-derivative state. **SPIN83** writes directly to the Magnus fields and cannot overwrite the CNAD array.

This is not a change to the empirical spin correlation. It restores the destination implied by the output names, by the **BDAMP** layout, and by the Magnus report.

Result impact. When the spin calculation is active, the historical program exhibits two linked symptoms:

- the first six CNAD values are replaced by unrelated roll-damping, roll-side-force, and Magnus quantities; and
- the printed Magnus values come from storage that was not populated by the current calculation.

Complete historical runs repeatedly printed the same sentinel-scale value, approximately 1.745×10^{-32} , in all six Magnus fields. Those values are stale storage rather than an aerodynamic prediction and must not be used as a numerical oracle. SpearDAT preserves the calculated CNAD schedule and reports the actual named Magnus outputs.

The defect affects dynamic and Magnus results. It does not change the static aerodynamic coefficients calculated before the derivative path is entered.

Verification. The two historical /DBODY/ declarations were mapped position by position and compared with the named C++ state. Full-stack DAMP+SPIN sweeps confirmed both observable symptoms: the historical CNAD discrepancy appears only when the spin path is active, and the historical Magnus report remains stale while SpearDAT reports the values written by **SPIN83**.

The same report structure and stale historical Magnus fields were also reproduced when an active inlet was present, confirming that the mechanism is in the shared derivative storage rather than in a particular static configuration.

KD-021 A fixed-form source character changed the Mach 2.0 interpolation station to Mach 12.0

Category	calculation correction
Status	published
Compatibility	numerically-different
Observability	latent in verified paths
Evidence	S
Affected	Mach interpolation of CLLP, CNP1, CNP3, CNP5, and CNPY5 above Mach 1.75; routine SPIN83 .

SPIN83 first builds each spin derivative at seventeen fixed Mach stations and then linearly interpolates the resulting table at the requested Mach number. The Mach abscissa must therefore be ordered and must correspond element by element with every derivative table.

Historical implementation. The relevant fixed-form data statement spans two source lines:

```
DATA CMACH / ... 1.5,1.75,
1 1          2.0,2.5,3.0,4.0,5.0 /
```

The first 1 shown on the continuation line is the valid fixed-form continuation character in column 6. The second 1 lies inside the statement field. Blanks inside a numeric token are ignored by this Fortran source form, so the compiler joins that stray digit to 2.0 and reads 12.0.

The local portion of the table is consequently:

Table element	Intended Mach	Historical Mach	compiled
12	1.75	1.75	
13	2.00	12.00	
14	2.50	2.50	

The error makes the thirteenth station six times too large and makes the seventeen-point Mach schedule non-monotonic.

SpearDAT implementation and motivation. SpearDAT defines the table explicitly with $\text{CMACH}(13) = 2.0$. This restores the ordered sequence

..., 1.75, 2.0, 2.5, 3.0, 4.0, 5.0

expected by the interpolation routine. No empirical ordinate or interpolation formula is otherwise changed.

Result impact. The missing Mach 2.0 station corrupts the intended interpolation for requested Mach numbers above 1.75. It affects roll damping and the four Magnus-moment outputs that use Mach-varying source tables.

The CYP side-force derivative is not independently changed by this defect in the affected high-Mach region because its source ordinate is constant there. The malformed abscissa is still present in that interpolation, but both adjacent ordinate values are the same.

In the verified historical full-stack path, the independent numerical signature of this table error is hidden by the shared-storage mismatch in [entry KD-020](#): the calculated Magnus values do not reach the printed Magnus fields. The table correction is therefore classified as latent in the current comparison even though its source-level effect on the intended **SPIN83** calculation is direct.

Verification. The fixed-form columns and the compiled numeric token were checked directly against the explicit C++ table. The affected output list follows from the six calls that interpolate the generated tables at the requested Mach number. No separate magnitude is quoted because the historical report does not expose the calculated scalars independently of [entry KD-020](#).

KD-022 Two Magnus outputs were not initialised by their intended statements

Category	calculation correction
Status	published
Compatibility	numerically-different
Observability	latent in verified paths
Evidence	S
Affected	Initial state of the cubic and quintic Magnus coefficients CNP3 and CNP5; routine SPIN83 .

At entry, **SPIN83** intends to initialise all six output quantities to zero before constructing and interpolating its Mach-dependent tables.

Historical implementation. Four of the statements use the correct output names, but the two middle initialisers reverse the letters N and P:

```
CLLP=0.
CYP=0.
CNP1=0.
CPN3=0.
CPN5=0.
CNPY5=0.
```

CPN3 and CPN5 are not aliases for CNP3 and CNP5. Under the historical implicit-typing rules they become separate local real variables. The intended shared output fields retain whatever values were already present until the later interpolation calls write them.

SpearDAT implementation and motivation. SpearDAT explicitly initialises all six named Magnus outputs, including CNP3 and CNP5. This makes the output state deterministic from routine entry and removes the two unused misspelled locals.

Result impact. No user-visible difference has been demonstrated in the current production path. After building the tables, the routine unconditionally interpolates into the real CNP3 and CNP5 outputs before scaling and returning. Those writes mask the bad initialisation during ordinary completion.

The correction is retained in this register because the historical statements do not perform their documented purpose and leave two shared outputs dependent on prior storage for part of the routine. It should not be confused with the separate shared-layout defect in [entry KD-020](#), which does reach the verified full-stack output.

Verification. The six intended output names, the two misspelled local names, and the later interpolation destinations were traced directly in both implementations. The consumer trace establishes why the defect is real but latent in every normal completion path examined.

KD-023 The vertex-to-CG distance was converted to body diameters in the caller's variable

Category	calculation correction
Status	published
Compatibility	numerically-different
Observability	latent in verified paths
Evidence	S
Affected	The caller-owned VCG argument passed to SPIN83 .

Despite its name, VCG does not mean “vertical centre of gravity.” The historical routine documentation defines it as the axial distance from the body vertex to the centre of gravity. It enters **SPIN83** in the user’s internal length units and is converted to body diameters, or calibres, for the empirical correlation.

Historical implementation. The conversion is performed by assigning back to the dummy argument:

```
VCG=VCG/D
```

Arguments in this Fortran calling convention are normally passed by reference. The assignment therefore changes the caller’s variable as well as the value used locally by **SPIN83**.

For example, if the vertex-to-CG distance is 24 inches and the body diameter is 4 inches, the routine correctly needs the dimensionless value 6 for its correlation. The historical statement also leaves the caller’s variable equal to 6, rather than preserving the original 24-inch distance.

SpearDAT implementation and motivation. SpearDAT receives the distance by value and creates a separate local value:

$$\text{VCG_CAL} = \frac{\text{VCG}}{D}.$$

The correlation uses VCG_CAL; the caller’s input remains in its original units.

Result impact. The side effect is latent in the verified full-stack path. **BDAMP** first copies the active longitudinal CG location into a local scalar, passes that local scalar to **SPIN83**, and does not use it again after the call. The shared reference CG is therefore not changed by the current historical deck path.

The correction nevertheless matters to the routine contract. It prevents a caller from receiving the same variable back with a different unit and avoids making correctness depend on the present caller discarding that value.

Verification. The assignment and pass-by-reference effect were compared with the C++ by-value interface. The **BDAMP** caller was then traced to confirm that the modified historical local is not reused and that the difference remains latent in the verified product path.

A.11.2. Geometry Required by the Spin Correlation

KD-024 Coordinate-defined bodies could reach the spin correlation without an equivalent nose radius

Category	numerical robustness
Status	published
Compatibility	availability-different
Observability	conditionally observable
Evidence	S, F
Affected	Spin-body roll damping CLLP and the usability of the Magnus result set for coordinate-defined axisymmetric bodies; routines BDAMP and SPIN83 .

The spin correlation requires an equivalent ogive radius OR. This is a dimensionless nose-curvature quantity expressed in body diameters. It enters the roll-damping correlation through the term

$$VOR = \frac{VN^2}{OR} - 0.5,$$

where VN is nose length in body diameters.

Historical implementation. **BDAMP** assigns OR only for recognised analytic nose classifications:

```
IF(TYPEN.EQ.0.) OR=500.
IF(TYPEN.EQ.1.) OR=(LN/DN)**2+0.25
IF(TYPEN.NE.2.) GO TO 1500
...
OR=0.25*SQRT(R2)
```

A coordinate-defined AXIBOD body can have valid inferred LNOSE and DNOSE values while its analytic nose-type flag remains unspecified. None of the assignments above is then guaranteed to execute, leaving OR unset. When the value reaching **SPIN83** is zero or otherwise invalid, the VN^2/OR operation is non-finite and the roll-damping result becomes unusable.

Illustrative geometry fallback

Consider a coordinate-defined body for which geometry processing has inferred

$$LNOSE = 12 \text{ in}, \quad DNOSE = 4 \text{ in}.$$

The nose length is therefore three body diameters. With no valid OR, the historical spin expression contains a division by an unset or zero value. SpearDAT forms the geometry-derived tangent-ogive equivalent radius

$$OR = \left(\frac{12}{4}\right)^2 + 0.25 = 9.25.$$

The correlation term is then finite:

$$VOR = \frac{3^2}{9.25} - 0.5 \approx 0.473.$$

The example illustrates the numerical role of the fallback; it is not a quoted aerodynamic result for a particular missile.

SpearDAT implementation and motivation. SpearDAT retains the historical equivalent-radius rules when a recognised nose type supplies a valid value. If the value is absent or invalid but positive LNOSE and DNOSE values are available, it uses

$$OR = \left(\frac{LNOSE}{DNOSE}\right)^2 + 0.25.$$

This is the same tangent-ogive expression used by the historical TNOSE=1 path, now applied to geometry that is known through its station-derived nose length and diameter.

If no valid equivalent radius can be formed, SpearDAT marks all six Magnus scalars consistently as non-finite rather than allowing one calculation to fail while other fields retain values from earlier state.

Result impact. The difference applies to a coordinate-defined axisymmetric body when the spin/Magnus path is reached without a usable analytic nose classification. Before the fallback was added, that path produced a non-finite CLLP in the named C++ Magnus output. SpearDAT now produces a finite correlation input derived from the active geometry.

The historical printed Magnus value cannot be used to measure this correction independently because [entry KD-020](#) prevents the computed scalar from reaching its intended report field. The practical improvement is therefore availability: the SpearDAT Magnus result set no longer fails solely because a station-defined body lacks an analytic shape flag.

Verification. The OR assignment paths in **BDAMP** and the division in **SPIN83** were reviewed together. Full-stack DAMP+SPIN sweeps using coordinate-defined body geometry exposed the non-finite named roll-damping value before the fallback and completed with finite outputs after the correction.

The broader derivative campaign also confirmed that DAMP-only, combined DAMP+SPIN, derivative-reporting-unit, and active-inlet derivative paths complete with the documented corrections isolated from the ordinary dynamic-derivative results.

Note

Three corrections in this group require different interpretations. [Entries KD-020 and KD-024](#) explain demonstrated user-visible failures. [Entry KD-021](#) changes the intended spin interpolation but is masked in the historical report by [entry KD-020](#). [Entries KD-022 and KD-023](#) are verified source defects that remain latent in the current full-stack caller and consumer paths.

A.12. Experimental Data Substitution

The \$EXPR input allows measured aerodynamic data to be introduced into a Missile DATCOM run. Before those values are used, **SUBEXP** converts the experimental dataset from its own reference area, reference lengths, and centre-of-gravity location to the active run reference system and interpolates it onto the active angle-of-attack schedule.

When the experimental data are used as increments rather than direct substitutions, the coefficient-specific routines **CNINC**, **CMINC**, **CYINC**, and **CSNINC** derive correction factors from the experimental case and apply those factors to the corresponding theoretical configuration data. In the zero-aerodynamic-roll path, each routine separates a total theoretical quantity into potential and viscous contributions before applying the experimental correction.

The two entries in this section affect different stages of that process:

Entry	Stage	Principal difference
KD-025	Increment calibration and reconstruction	Uses radians, rather than degree values, in the trigonometric potential/viscous decomposition shared by four coefficient routines.
KD-026	Experimental reference conversion	Uses the active lateral reference length when rescaling and shifting lateral-directional moment coefficients.

Note

These corrections apply only when experimental-data processing reaches the affected branches. Ordinary theoretical calculations without \$EXPR data are unchanged. Direct replacement and increment modes also do not necessarily exercise the same internal operations.

KD-025 Experimental increment decomposition passed degree values directly to trigonometric functions

Category	calculation correction
Status	published
Compatibility	numerically-different
Observability	conditionally observable
Evidence	S, F
Affected	Experimental increment corrections to CN, CM, CY, and CSN; routines CNINC , CMINC , CYINC , and CSNINC .

In the zero-aerodynamic-roll increment path, the historical program does not apply one multiplier directly to the complete theoretical coefficient. Instead, it estimates a potential contribution from the local coefficient slope, defines the remaining contribution as viscous, and derives separate experimental correction factors for those parts.

The four routines use the same trigonometric shape but apply it to different quantities:

Routine	Final coefficient	Quantity decomposed into potential and viscous parts
CNINC	CN	Theoretical normal force CN.
CMINC	CM	Theoretical pitching moment CM.
CYINC	CY	Total transverse-force magnitude formed from $\sqrt{CN^2 + CY^2}$; the corrected CY is recovered afterwards.
CSNINC	CSN	Total moment magnitude formed from $\sqrt{CM^2 + CSN^2}$; the corrected CSN is recovered afterwards.

Historical implementation. The active angle-of-attack schedule ALPHA is stored in degrees, but each routine passes those values directly to the Fortran SIN and COS intrinsic functions, which interpret their arguments as radians. For example, **CNINC** contains:

```
CNPTH(I) = .5 * CNATH(1) * SIN(2. * ALPHA(I)) * COS(ALPHA(I) / 2.)
```

The equivalent expressions in **CMINC**, **CYINC**, and **CSNINC** differ only in the coefficient and slope names. The routines already multiply the calculated slope by RAD to express it per radian, so the missing operation is the corresponding conversion of the angle used inside the trigonometric shape.

Illustrative decomposition factor

At $\text{ALPHA} = 5^\circ$, the dimensionless trigonometric factor should be

$$\sin(10^\circ) \cos(2.5^\circ) \approx 0.1735.$$

The historical statement instead evaluates

$$\sin(10 \text{ rad}) \cos(2.5 \text{ rad}) \approx 0.4358.$$

For this station, the historical potential contribution is therefore about 2.51 times the value obtained from the intended angular units before the experimental multipliers are applied. At other angles the error is not a constant scale factor: the radian arguments can change the magnitude and, depending on the angle, the sign of the trigonometric product.

This example demonstrates only the internal decomposition factor. It is not a measured difference in a final Missile DATCOM coefficient, because the calibration and reconstruction steps also depend on the experimental and theoretical curves.

SpearDAT implementation and motivation. SpearDAT first converts the active angle to radians:

$$\text{ALPHA_RAD} = \frac{\text{ALPHA}}{\text{RAD}},$$

and evaluates the shared expression using $\sin(2 \text{ ALPHA_RAD})$ and $\cos(\text{ALPHA_RAD}/2)$. The coefficient-specific calibration, offset handling, interpolation, and potential/viscous recombination are otherwise retained.

Result impact. The defect is reached when experimental increment processing uses the zero-roll potential/viscous branch. The non-zero-PHI branch uses a direct multiplier table and does not evaluate this trigonometric expression.

The first calibration case can conceal the defect in its own output. At the calibration stations, the viscous correction table is derived using the same potential/viscous split that is then used to reconstruct the coefficient, so the supplied experimental data can still be recovered. The wrong split becomes important when those stored correction factors are applied to a later case or configuration whose theoretical curve differs from the calibration case.

The affected values are primary static force and moment coefficients. They can therefore propagate into derived slopes, centres of pressure, body-fin synthesis, and later report or derivative calculations that consume the incremented curves.

Verification. All four historical expressions were compared with their C++ counterparts. They contain the same degree-valued trigonometric arguments and SpearDAT applies the same radians conversion in each routine.

A full aerodynamic-executive increment test exercises the corrected **CNINC** zero-roll path and intentionally expects the SpearDAT result rather than the historical oracle. The analogous **CMINC**, **CYINC**, and **CSNINC** mechanisms are source-verified; no independent full-stack magnitude is claimed for those three routines.

KD-026 Lateral moment conversion used the longitudinal reference length

Category	calculation correction
Status	published
Compatibility	numerically-different
Observability	conditionally observable
Evidence	S, I
Affected	Experimental yawing- and rolling-moment coefficients CSN and CSL when dataset reference quantities differ from the active run; routine SUBEXP .

Experimental coefficients may have been reduced using reference quantities that differ from those of the active Missile DATCOM case. **SUBEXP** converts the coefficients to the run reference system before substitution or increment processing.

Four similarly named lengths appear in this operation and must be kept distinct:

Historical name	Meaning	Public source
LREFR	Active run longitudinal reference length	reference.lref
LATR	Active run lateral reference length	reference.latref
LREF	Longitudinal reference length used to reduce the experimental dataset	\$EXPR field LREF
LATREF	Lateral reference length used to reduce the experimental dataset	\$EXPR field LATREF

CM is a longitudinal pitching-moment coefficient and is normalised with the longitudinal reference length. CSN and CSL are lateral-directional yawing- and rolling-moment coefficients and are normalised with the lateral reference length.

Historical implementation: reference-length rescaling. The longitudinal conversion follows the expected pattern:

```
IF(LREF .EQ. LREFR)GO TO 1020
IF(CM(I) .NE. UNUSED)CM(I)=CM(I)*LREF/LREFR
```

The lateral conversion tests the correct pair of lateral lengths but divides by the active longitudinal length:

```
IF(LATREF .EQ. LATR)GO TO 1030
IF(CSN(I) .NE. UNUSED)CSN(I)=CSN(I)*LATREF/LREFR
IF(CSL(I) .NE. UNUSED)CSL(I)=CSL(I)*LATREF/LREFR
```

The numerator is the dataset's lateral reference length and is correct. The denominator should be the active run's lateral reference length LATR, not its longitudinal length LREFR.

Historical implementation: reference-point shifts. When the experimental and active centres of gravity differ, **SUBEXP** also translates the moment coefficients to the active reference point. The historical lateral shifts again use the longitudinal length:

$$\begin{aligned} \text{CSN}(I) &= \text{CSN}(I) + \text{DXCG} * \text{CY}(I) / \text{LREFR} \\ \text{CSL}(I) &= \text{CSL}(I) + \text{DZCG} * \text{CA}(I) / \text{LREFR} \end{aligned}$$

Because these are yawing- and rolling-moment coefficients, the denominator should again be LATR. This second defect is independent of the preceding dataset-length rescaling: it can be reached even when LATREF already equals LATR.

Illustrative reference conversion

Assume the active run uses

$$\text{LREFR} = 10, \quad \text{LATR} = 5,$$

while the experimental dataset uses LATREF = 4 and contains CSN = 0.020.

The historical rescaling gives

$$0.020 \frac{4}{10} = 0.008,$$

whereas the lateral-reference conversion gives

$$0.020 \frac{4}{5} = 0.016.$$

Now suppose the experimental reference point is one length unit aft of the active point, so DXCG = 1, and the converted side-force coefficient is CY = 0.10. The historical yawing-moment increment is

$$\frac{1(0.10)}{10} = 0.010,$$

while the lateral-reference increment is

$$\frac{1(0.10)}{5} = 0.020.$$

The numbers are illustrative and are not taken from a validation deck. They show that the two defects act at separate stages and can both affect the final lateral moment.

SpearDAT implementation and motivation. SpearDAT applies the reference conversion consistently:

- CM is rescaled by LREF / LREFR;
- CSN and CSL are rescaled by LATREF / LATR;
- the DXCG CY contribution to CSN is divided by LATR; and
- the DZCG CA contribution to CSL is divided by LATR.

The reference-area conversion, pitching-moment conversion, interpolation, and experimental-data selection logic remain unchanged.

Result impact. Both defects are masked when the active run uses equal longitudinal and lateral reference lengths:

$$\text{LREFR} = \text{LATR}.$$

This is common for missile cases that use one body diameter or one common length for both moment normalisations.

When those run reference lengths differ, the first defect additionally requires the dataset lateral reference LATREF to differ from LATR, so that the rescaling branch executes. The second defect requires a non-zero DXCG or DZCG and the corresponding force and moment coefficients. It is therefore reachable independently of the first defect.

Affected CSN and CSL values can be substituted directly or passed into later experimental increment and synthesis paths. Longitudinal CM scaling is not changed by this correction.

Verification. The four reference-length names and both historical denominator uses were traced directly in **SUBEXP**. Routine-level integration tests exercise experimental reference conversion, interpolation, centre-of-gravity shifts, and the corrected C++ CSN/CSL values.

Existing validation cases whose active longitudinal and lateral reference lengths are equal cannot expose this difference and are not treated as numerical evidence for it. No campaign-wide magnitude is stated.

Note

The two entries have different masking behaviour. The degree/radian defect may be hidden by calibration at the first experimental case and emerge when the derived increment factors are reused. The lateral-reference defect is hidden whenever the active longitudinal and lateral reference lengths are equal, regardless of the coefficient values.

A.13. Interpolation and Numerical Utilities

Several Missile DATCOM aerodynamic methods rely on small numerical utilities rather than implementing interpolation and differentiation locally. **INTERP** evaluates a tabulated quantity, normally with a five-point Lagrange formula, while **FDP5** evaluates the local derivative of a table using a corresponding five-point formula. **SVRINT** is a separate geometric utility that calculates the interference between a lifting surface and the physical right- and left-hand vortices, together with two mathematical body-image vortices. A body-image vortex is an auxiliary vortex placed at a reflected position inside the body so that the simplified flow model accounts for the presence of the solid body. It is not an additional physical vortex. The qualifier “physical”—called “real” in the routine documentation—distinguishes the external vortices from these mathematical images.

These routines are low in the call hierarchy, so a defect can appear far from the source of the problem. In particular, invalid short-table handling in **INTERP** and **FDP5** was observed as non-finite general-nose body results rather than as an error labelled “interpolation.”

Entry	Utility behaviour	Principal difference
KD-027	Interpolation and differentiation of short tables	Uses a valid local linear or secant fallback when fewer than five usable points are available.
KD-028	Vortex-interference geometry with zero normal separation	Uses the variable that the historical routine had already guarded, avoiding a division-by-zero exception without changing the intended correlation.

Note

A numerical helper must respect the amount of information supplied by its caller. A five-point formula is appropriate only when five valid points exist. Likewise, preserving the intended mathematical result does not require preserving an intermediate division by zero whose result happened to be tolerated by one floating-point environment.

KD-027 Three- and four-point tables were forced through five-point formulas

Category	numerical robustness
Status	published
Compatibility	availability-different
Observability	conditionally observable
Evidence	S, F
Affected	Interpolation and derivative evaluation for short internal tables, including several coordinate-defined general-nose body calculations; routines INTERP and FDP5 .

INTERP and **FDP5** are wrappers around five-point numerical formulae. Their role is to select a suitable group of neighbouring table stations and pass those five abscissae and ordinates to the underlying interpolation or differentiation routine.

The historical utilities contain a special fallback for a table with exactly two usable points. They do not provide corresponding handling for tables with three or four usable points.

Historical implementation. In **INTERP**, the two-point case is sent to a linear expression. Every other case is sent to **INTER5**:

```

      IF((N-J+2).EQ.1)GO TO 1010
      ...
      CALL INTER5(X,TX(I-2),TX(I-1),TX(I),TX(I+1),TX(I+2),
1          TY(I-2),TY(I-1),TY(I),TY(I+1),TY(I+2),Y)
      ...
1010 Y=TY(J-2)+(X-TX(J-2))*(TY(N)-TY(J-2))/
1          (TX(N)-TX(J-2))

```

FDP5 has the same structure: two usable points select a secant slope, while every other non-empty table is sent to the five-point **FD5** formula.

Clamping the centre index cannot create missing data. With only three or four usable stations, the requested five-point stencil extends before the start of the valid table. The routine consequently reads values that do not belong to the supplied dataset and may access outside

the declared Fortran array range. The result depends on unrelated storage and can become non-finite.

Illustrative short table

Consider the three-point linear table

X	0	1	2
Y	0	2	4

and a request at $X = 1.5$. The table contains enough information for a valid local result but not for a five-point polynomial. The appropriate linear interpolation is

$$Y(1.5) = 2 + (1.5 - 1) \frac{4 - 2}{2 - 1} = 3,$$

and the corresponding local secant derivative is

$$\frac{4 - 2}{2 - 1} = 2.$$

The historical wrappers nevertheless request five table values. The precise failure can be seen by storing the three valid points in array positions 1 through 3. For **INTERP** this corresponds to $J = 3, N = 3$; for **FDP5** it corresponds to $J = 1, N = 3$. In both routines, the subsequent clamp sets the five-point centre index to

$$I = N - 2 = 1.$$

The called five-point routine is therefore given the positions

$$I - 2, \dots, I + 2 = -1, 0, 1, 2, 3.$$

The Fortran arrays are declared with their first valid element at index 1. Indices -1 and 0 are therefore outside the table, and in this case outside the declared array bounds. The first two abscissae and ordinates passed to **INTER5** or **FD5** consequently come from unrelated memory.

The underlying five-point formula divides by differences between its supplied abscissae. Invalid values can therefore produce repeated or otherwise nonsensical stations, leading to a zero or non-finite denominator. In the actual three-point regression tests, the historical **INTERP** value and **FDP5** derivative both returned NaN; SpearDAT returned 3 and 2, respectively. The historical NaN is the observed result for those tests, not a language-defined answer: different surrounding memory could instead produce a different finite value or a bounds failure.

SpearDAT implementation and motivation. SpearDAT first counts the usable stations.

For **INTERP**:

- one usable point returns that point's ordinate;
- two to four usable points use the nearest valid linear interval; and
- five or more usable points retain the historical five-point path.

For **FDP5**:

- fewer than two usable points cannot define a derivative and return the routine's neutral initial value;
- two to four usable points use the secant slope of the nearest valid interval; and
- five or more usable points retain the historical five-point differentiation path.

The fallback does not claim high-order information that the table does not contain. For a locally linear table, as in the example above, it returns the exact value and slope.

Result impact. The difference is activated by callers that supply a usable table of fewer than five points. Tables with five or more valid stations continue through the established high-order formula.

The defect was encountered in several coordinate-defined general-nose body paths. The historical **HYBG1**, **HYBG2**, and **HYBSET** calculations could propagate the invalid utility result into non-finite body grids and aerodynamic outputs. After the short-table fallbacks were added, those same SpearDAT paths produced finite grids.

Because the historical result comes from invalid table access, it is not a numerical oracle that SpearDAT should reproduce. The relevant comparison is availability and mathematical validity: SpearDAT evaluates the information that is actually present in the table.

Verification. The short-table branch conditions and five-point stencil accesses were compared directly in both source pairs. Tests exercise one-, two-, three-, four-, and five-point inputs, query locations within and outside the tabulated interval, and the transition from the short-table fallback to the five-point path.

Full aerodynamic cases then confirmed the downstream symptom: general-nose paths that produced non-finite historical results through these dependencies complete with finite SpearDAT grids. The updated SpearDAT values are therefore used as the reference for those affected cases.

KD-028 A zero-distance guard was calculated but not used in the divisions it was intended to protect

Category	numerical robustness
Status	published
Compatibility	availability-different
Observability	conditionally observable
Evidence	S, I
Affected	Lifting-surface vortex-interference factor and execution under floating-point exception trapping; routine SVRINT .

SVRINT estimates how four vortex contributions influence a lifting surface: the physical right and left vortices and their two mathematical body-image vortices. The image vortices are generated by reflecting the physical-vortex locations through the local circular-body boundary; they represent the body's influence in the potential-flow construction rather than additional vortices in the actual flow. For each contribution, the routine forms:

- F , the vortex position measured along the lifting-surface plane; and
- H , the signed distance normal to that plane.

Two ratios, FS and FR, describe the vortex position relative to the surface tip S and local body radius RL. When H is zero, the vortex lies in the surface plane and those ratios require special handling.

Historical implementation. The historical routine clearly creates a protected denominator HH:

```
HH=H
IF(H.EQ.0.0) HH=0.0001
FS=(F-S)/H
FR=(F-RL)/H
AA=1.0
IF(FS.LT.0.0) AA=-AA
B=S-RL+H*ATAN(ABS(FS))*AA
AA=1.0
IF(FR.LT.0.0) AA=-AA
B=B-H*ATAN(ABS(FR))*AA
```

The two divisions use the original H, not the guarded HH. The guard therefore has no effect on FS or FR.

A separate condition is checked first for a vortex exactly on the wing tip, $F = S$ and $H = 0$. That documented singularity returns the historical sentinel $VRINT = -10^{10}$ and does not reach the divisions discussed here.

Why the historical answer can look correct

Take an in-plane, non-tip contribution with

$$H = 0, \quad S = 5, \quad RL = 1, \quad F = 3.$$

The historical arithmetic attempts

$$FS = \frac{3-5}{0} = -\infty, \quad FR = \frac{3-1}{0} = +\infty$$

on an IEEE-style floating-point system. The routine subsequently uses these ratios only through expressions of the form

$$H \tan^{-1}(|FS|) \quad \text{and} \quad H \tan^{-1}(|FR|).$$

The factor that becomes zero is not introduced artificially by the numerical guard. It is the original geometric variable H, the normal distance between the vortex and the lifting-surface plane. For the in-plane case being considered, that physical distance is exactly zero.

The inverse tangent maps either infinity to its finite limiting angle $\pi/2$. The two angular contributions to B are then of the form $0 \times \pi/2$ and vanish. Equivalently, their mathematical limit is

$$\lim_{H \rightarrow 0} H \tan^{-1} \left(\left| \frac{\text{non-zero distance}}{H} \right| \right) = 0.$$

Only these two angular terms vanish. The logarithmic contribution A, the remaining part of B, the individual $L(I)$ value, and the final VRINT combination are not forced to zero. The final interference factor can consequently agree with the guarded calculation even though a division-by-zero exception was raised on the way.

With the intended guard, the ratios in this example are finite:

$$FS = -20000, \quad FR = 20000.$$

Their inverse tangents are already extremely close to $\pi/2$. The formula still multiplies them by the physical distance $H = 0$, so the same angular terms vanish, but the intermediate division is now finite and no floating-point exception is raised. The protected value HH is used only to evaluate the ratios; it does not replace H in the physical geometric contribution.

SpearDAT implementation and motivation. SpearDAT retains the historical guard value but uses it in the two denominators:

```
HH=H
IF(H.EQ.0.0) HH=0.0001
FS=(F-S)/HH
FR=(F-RL)/HH
```

The vortex geometry, logarithmic contribution, sign handling, real/image vortex sequence, final four-term combination, and explicit wing-tip sentinel are otherwise unchanged.

This is an example of why a faithful reimplementation is not always a statement-for-statement copy. Copying the divisions by H would preserve an avoidable floating-point exception, not an intended aerodynamic method. Using HH preserves the evident guard and the verified numerical result while making that result independent of whether the runtime continues after division by zero.

Result impact. For ordinary geometry with $H \neq 0$, the historical and SpearDAT expressions are identical. For tested in-plane, non-tip geometries, both implementations produce matching VRINT values on hardware that permits division by zero and continues with infinities.

The distinction is execution robustness. The historical routine raises the floating-point divide-by-zero condition. A build configured to trap that condition can stop before returning the otherwise finite interference factor; behaviour on a floating-point environment that does not provide the same infinity semantics is also not portable. SpearDAT completes the calculation without raising that exception.

The special wing-tip result remains $VRINT = -10^{10}$ in both implementations.

Verification. The unused HH assignment and the two divisions were compared directly with the C++ implementation. Routine tests cover in-plane $H = 0$ cases and confirm matching finite results without the C++ divide-by-zero exception. A separate test confirms that the explicit wing-tip singularity still returns the historical -10^{10} sentinel.

Note

Entry KD-027 corrects demonstrated non-finite aerodynamic results. Entry KD-028 normally preserves the numerical value but removes dependence on permissive floating-point exception handling. Together they illustrate two different responsibilities of a numerical reimplementation: choose a formula supported by the available data, and evaluate that formula without relying on accidental machine behaviour.

A.14. Output, Reporting, and Diagnostics

Missile DATCOM produces two output forms that serve different purposes. The main for006 report is arranged for a person to read, with headings, fixed-width tables, and explanatory notes. The optional for003 file is a compact, zone-based aerodynamic dataset intended for plotting and post-processing.

SpearDAT retains both forms but deliberately removes several ambiguities that made automated interpretation unreliable. It completes the advertised for003 row schema, adds identifying metadata to zone titles, represents an undefined centre of pressure explicitly, and prevents a fixed-size historical side-store from corrupting long multi-case runs.

Entry	Output area	Principal difference
KD-029	for003 aerodynamic rows	Writes the two CL/CD values already advertised by the ten-column file header.
KD-030	for003 zone titles	Adds configuration, Mach, sideslip, aerodynamic roll, and deflection metadata where applicable.
KD-031	for006 centre-of-pressure table	Avoids meaningless ratios and overflow asterisks when normal force is numerically zero, and prints NaN when the location is genuinely undefined.
KD-032	Long multi-case runs	Prevents writes beyond the historical twenty-case /UDATA/ side-store while allowing reports and plot output to continue.

Note

The for003 changes are deliberate schema and metadata extensions. A SpearDAT for003 file is therefore not intended to be byte-for-byte identical to the historical file. Validation compares the shared legacy columns and zone structure, then verifies the added columns and metadata independently.

A.14.1. Machine-Readable Aerodynamic Output

KD-029 The machine-readable header advertised ten columns but most historical rows contained eight

Category	output and reporting
Status	published
Compatibility	output-different
Observability	conditionally observable
Evidence	S, F
Affected	The for003 aerodynamic rows emitted by PLOT3 and PLTUT9 .

The global for003 header declares the following variables:

```
VARIABLES=ALPHA,CN,CM,CA,CY,CLN,CLL,DELTA,CL,CD
```

A reader following that declaration reasonably expects every aerodynamic row to contain ten values in the same order.

Historical implementation. The ordinary static-output writer **PLOT3** and the untrimmed trim-matrix writer **PLTUT9** emit only:

ALPHA, CN, CM, CA, CY, CLN, CLL, DELTA.

The advertised CL and CD fields are absent. The trimmed writer **PLTTRM** already emits all ten values, so a single historical file can contain rows with different column counts under one global header.

SpearDAT implementation and motivation. SpearDAT appends CL and CD to the affected eight-column rows. They are derived from the same **CLOCD** transformation used by the human-readable static report:

$$CL = CN \cos \alpha - CA \sin \alpha,$$

$$CD = CA \cos \alpha + CN \sin \alpha.$$

The completed row schema is therefore:

```
ALPHA CN CM CA CY CLN CLL DELTA CL CD
```

The first eight values are not redefined by this extension. The two new values are derived report quantities rather than replacements for the native body-axis coefficients.

Result impact. The aerodynamic calculation is unchanged. The difference is in the machine-readable representation:

- historical **PLOT3** and **PLTUT9** rows have eight values despite the ten-name header;
- SpearDAT rows have ten values and conform to that header; and
- historical **PLTTRM** rows already have ten values and retain the same layout.

The file is deliberately not byte-comparable with the historical for003. A post-processor should compare the first eight columns where they are shared and verify the added CL/CD columns from CN, CA, and ALPHA.

For an ordinary no-trim zone, the eighth column DELTA is commonly zero. It must not be mistaken for the newly added lift coefficient in column nine.

Verification. Accepted full-stack cases retain the historical zone and row counts. Their first eight columns match to the applicable printed tolerance, while columns nine and ten agree with the **CLOCD** transformation and with the corresponding values in the static report.

KD-030 Zone titles now identify the condition represented by each aerodynamic block

Category	output and reporting
Status	published
Compatibility	output-different
Observability	conditionally observable
Evidence	I, F
Affected	Zone titles in for003; routines PLOT3 , PLTUT9 , and PLTTRM .

A *zone* is one block of for003 rows sharing one configuration and operating condition. Large build-up, roll-sweep, and trim runs can contain hundreds or thousands of zones. The title must distinguish blocks that may have the same Mach number but represent different physical conditions.

Historical implementation. An ordinary historical title commonly contains only the trim state and Mach number:

```
ZONE T="NO TRIM MACH= 0.40"
```

That title does not identify whether the rows represent the body, an isolated fin set, or a cumulative body–fin configuration. It also omits fixed sideslip and aerodynamic roll. In a trim-matrix file it does not identify the commanded deflection associated with an untrimmed zone.

SpearDAT implementation and motivation. SpearDAT adds the metadata available at each writer:

Zone type	SpearDAT title fields
Ordinary no-trim output	CONFIG, MACH, BETA, ROLL
Untrimmed trim-matrix point	MACH, BETA, ROLL, DELTA
Trimmed result	MACH, BETA, ROLL

For example:

```
ZONE T="NO TRIM CONFIG=8 MACH= 0.40 BETA= 0.00 ROLL=-90.00"
```

The CONFIG identifier is the established build-up index passed from **PRIIOM**. It is not the deck's CASE number. Its values have the following fixed meanings:

CONFIG	Aerodynamic configuration
1	Body alone
2	Fin set 1 alone
3	Fin set 2 alone
4	Fin set 3 alone
5	Fin set 4 alone
6	Body with fin set 1
7	Body with fin sets 1 and 2
8	Body with fin sets 1, 2, and 3
9	Body with fin sets 1, 2, 3, and 4

The sequence reflects Missile DATCOM's component build-up: isolated components are followed by progressively more complete body–fin configurations.

BETA is the case-level fixed-sideslip input. ROLL is the case-level aerodynamic roll angle PHI. In an ALPHA+PHI run, internally derived body-axis sideslip varies with the row's angle of attack and therefore cannot be represented by one zone-level BETA value.

Result impact. No aerodynamic coefficient changes. The richer titles allow a parser, GUI, or MATLAB consumer to associate each block with its configuration and operating condition without reconstructing that identity from file order.

As with [entry KD-029](#), the textual file is intentionally different from the historical output. Zone counts and aerodynamic rows remain the appropriate structural comparison; title strings are validated against the enriched SpearDAT schema.

Verification. Full-stack build-up, roll-sweep, deflection, trim, and experimental-data cases retain the historical number and ordering of zones. The enriched titles parse into the expected configuration, Mach, sideslip, roll, and commanded deflection fields.

A.14.2. Human-Readable Reports and Long Runs

KD-031 Centre-of-pressure reporting used exact zero tests for normal force

Category	numerical robustness
Status	published
Compatibility	output-different
Observability	conditionally observable
Evidence	S, F
Affected	The X-C.P. column in static for006 reports; routine PRIIOM .

The reported longitudinal centre of pressure is a force-to-moment ratio, measured from the moment reference centre in longitudinal reference lengths. When the normal-force coefficient is usable, the report forms

$$XCP = \frac{CM}{CN}.$$

At exactly zero normal force, the historical method attempts to infer the limiting location from the corresponding slopes:

$$XCP = \frac{CMA}{CNA}.$$

This second expression is meaningful only when a usable normal-force slope exists.

Historical implementation. Both body/body–fin and fin-alone report branches use exact comparisons:

```
IF(CN(I) .NE. 0.)XCP(I)=CM(I)/CN(I)
IF(CN(I) .EQ. UNUSED)XCP(I)=1./UNUSED
IF(CN(I) .EQ. 0. .AND. CNA(I) .EQ. 0.)XCP(I)=1./UNUSED
IF(CN(I) .EQ. 0. .AND. CNA(I) .NE. 0.)XCP(I)=CMA(I)/CNA(I)
```


A tiny round-off residual is not exactly zero, so the first statement divides by it. The resulting centre-of-pressure value can be enormous even though the printed CN field rounds to 0.000.

Where the routine recognises an exactly undefined condition, it assigns 1/UNUSED. Because UNUSED is a tiny sentinel, this also produces an enormous finite number. The report then writes XCP into a fixed-width F10.3 field. A value too large for that field appears as ten asterisks rather than as a number.

Illustrative near-zero result

Assume an output station contains

$$CN = 10^{-14}, \quad CM = 0.020,$$

and the local slopes are

$$CNA = 0.20, \quad CMA = -0.10.$$

The historical exact test treats CN as non-zero and calculates

$$XCP = \frac{0.020}{10^{-14}} = 2 \times 10^{12}.$$

That number cannot fit in the fixed-width report field and is printed as asterisks. The slope-based limiting value is instead

$$\frac{-0.10}{0.20} = -0.5.$$

The example illustrates the decision logic; it is not taken from a validation deck.

SpearDAT implementation and motivation. SpearDAT treats coefficients whose magnitude is no greater than 10^{-10} as numerically zero for this report calculation.

The decision sequence is:

1. If CN is usable and not effectively zero, use CM/CN.
2. If CN is non-zero but effectively zero, and CM is also effectively zero, form CM/CN. This covers a common-projection case in which both coefficients contain the same very small multiplier. The ratio is retained only when it is finite and its magnitude does not exceed 10^6 . If CN is exactly zero, this direct ratio is not attempted.
3. Otherwise, if CNA is usable and not effectively zero, use CMA/CNA.
4. If neither force nor slope can define a location, store and print NaN.

Illustrative numerical example

Suppose an unprojected configuration has

$$CN_0 = 0.40, \quad CM_0 = -0.20,$$

and a roll-dependent projection multiplies both quantities by the same small factor $p = 10^{-12}$. The stored body-axis values are then

$$CN = p CN_0 = 4 \times 10^{-13}, \quad CM = p CM_0 = -2 \times 10^{-13}.$$

Both are effectively zero for reporting purposes, but their ratio remains

$$\frac{CM}{CN} = \frac{p CM_0}{p CN_0} = \frac{-0.20}{0.40} = -0.5.$$

The small common projection cancels, so -0.5 is a meaningful limiting centre-of-pressure location rather than a huge division-by-round-off result. SpearDAT accepts this ratio only when the denominator is not exactly zero and the result is finite and bounded. At an exact projection of $p = 0$, both coefficients are exactly zero and the direct ratio is $0/0$; SpearDAT then uses the slope fallback or reports NaN if the slopes cannot define the location.

The explicit non-finite marker communicates that no longitudinal centre-of-pressure location exists for that row. It is preferable to a huge sentinel-derived number or overflow asterisks, either of which can be mistaken for a very remote but valid location.

Result impact. The change affects only the derived X-C.P. report field. CN, CM, CNA, CMA, and all other aerodynamic coefficients remain unchanged.

Two observable historical symptoms are folded into this entry:

- a huge finite centre-of-pressure value when CN is a tiny non-zero residual; and
- fixed-width asterisks when the ratio or sentinel-derived value does not fit the field.

Where CN and CNA are both effectively zero while a finite moment remains, SpearDAT prints NaN. Validation permits that marker only in the X-C.P. column, not in the primary force, moment, lift, drag, or derivative fields.

Verification. The exact historical tests and the SpearDAT tolerance/fallback sequence were compared directly. A canard case reproduced the historical overflow behaviour from a tiny CN; an opposed-canard case at $ROLL = \pm 90^\circ$ established the genuinely undefined condition with near-zero CN and CNA but finite moment. Full-stack reruns confirmed that SpearDAT removes huge finite and asterisk X-C.P. fields and emits NaN only where the location is undefined.

KD-032 The report routine could write beyond a twenty-case side-store

Category	numerical robustness
Status	published
Compatibility	availability-different
Observability	conditionally observable
Evidence	S, F
Affected	Runs containing more than twenty cases; the historical /UDATA/ save performed by PRIIOM .

After printing and plotting a configuration, **PRIIOM** copies selected values into /UDATA/. This is a secondary shared store, not the calculation that produced the current report row.

The historical dimensions are:

```
COMMON /UDATA/ UCN(20,20,20),UCM(20,20,20),UCA(20,20,20),
1              DEL(20,20)
```

The three indices of UCN, UCM, and UCA are angle-of-attack station, case number, and Mach index. The second dimension therefore reserves storage for only twenty cases.

Historical implementation. The current case counter is copied directly into the array subscript and no capacity check is made:

```
NC = CASE
...
UCN(I,NC,IM) = CN(I)
UCM(I,NC,IM) = CM(I)
UCA(I,NC,IM) = CA(I)
```

At case 21, the second subscript is outside its declared range. A typical unchecked Fortran build continues and writes into memory following the intended array region. What value is corrupted depends on the compiler's common-block layout and cannot be treated as defined program behaviour. A different build could stop on a bounds check, corrupt later state, or fail during a later case even though the out-of-range statement itself appeared to complete.

Within **PRIIOM**, the current for006 report is written first. If plot output is requested, **PLOT3** then writes the current for003 zone. Only after both output operations does the routine copy CN, CM, CA, and one representative deflection into /UDATA/.

Skipping this final copy therefore cannot alter the report or plot rows that have already been written for the current configuration. In the validated translated path, /UDATA/ is retained only as a legacy side-store; the primary SpearDAT result capture does not read it back to construct later aerodynamic coefficients or output rows.

The historical out-of-bounds write is nevertheless unsafe. Even if the saved values themselves are not needed later, writing beyond the common block can overwrite unrelated adjacent memory and thereby affect later cases or program execution. No such report corruption was observed in the verified 73-case legacy run, but that observation does not make the write defined or portable.

SpearDAT implementation and motivation. SpearDAT retains the twenty-case side-store for compatibility with the translated state layout, but does not allow it to control whether a valid multi-case aerodynamic run can complete.

For case numbers 1 through 20, the values are saved normally. For a case number above 20, SpearDAT skips only the /UDATA/ copy. The human-readable report and for003 zone have already been emitted before this decision and therefore continue to be available for every case. Invalid non-positive case indices and invalid Mach indices remain diagnostic errors.

This preserves the useful externally visible behaviour of a long run without reproducing an out-of-bounds memory write. Modern structured SpearDAT results are captured per prepared case and do not depend on this fixed historical side-store as their primary storage.

Result impact. The difference is reached by a deck containing more than twenty CASE blocks. The historical executable may appear to complete, but its writes after case 20 have undefined effects. SpearDAT completes safely and retains the report and plot output for every case; only the obsolete fixed-capacity side copy is omitted for later cases.

No particular historical coefficient corruption is asserted because the source does not define which adjacent storage receives an out-of-range write.

Verification. The issue was exposed by a 73-case roll sweep. An earlier strict C++ guard stopped at case 21, while the historical executable continued through the deck. After the save policy was narrowed to skip only the full side-store, SpearDAT completed all 73 cases through the end-of-job marker. The for006 and shared for003 values matched the historical run to their accepted printed tolerance.

Note

The first two entries intentionally enrich the machine-readable file. [Entry KD-031](#) improves the meaning of one human-readable derived field, and [entry KD-032](#) protects execution after that output has already been calculated. None of the four entries changes the underlying static aerodynamic coefficient build-up.

A.15. Precision and Representation

The historical Missile DATCOM implementation and SpearDAT do not use identical floating-point representations. Most real-valued quantities in the historical program are held in default Fortran REAL storage, which is single precision in the validated historical build. SpearDAT stores and evaluates the translated numerical model in C++ double precision.

This difference does not normally change which aerodynamic method is selected, which table is addressed, or which equations are evaluated. It can nevertheless change the final stored bit pattern, the sign attached to a computed zero, the last printed digit, and the amount of cancellation retained in a sensitive intermediate calculation.

Note

“Precision difference” is not used as a general explanation for an unresolved mismatch. It is accepted only after the same inputs, method, branch, data ordering, coefficient definition, and output transformation have been confirmed in both implementations.

KD-033 Floating-point precision and representation can differ without changing the selected aerodynamic method

Category	precision and representation
Status	published
Compatibility	precision-different
Observability	conditionally observable
Evidence	S, R, I, F
Affected	Floating-point calculations and textual output throughout the program, especially accumulated results, cancellation-sensitive analytic body methods, signed zero, and the internal leading-edge calculation in IDEAL .

Numerical representations. Single precision retains substantially fewer significant binary digits than double precision. Each historical arithmetic operation therefore rounds to a coarser representable value, while SpearDAT normally carries more of the intermediate result into the next operation.

The wider representation is not intended to redefine the Missile DATCOM correlations. SpearDAT retains the translated equations, constants, branch conditions, interpolation choices, and coefficient definitions unless another entry in this appendix states a specific correction. The expected difference is the numerical evaluation of those equations on a modern floating-point representation.

Ordinary representation differences

The most common consequences are small and local:

- the final printed digit may differ when a value lies close to a rounding boundary;
- a sequence of additions or integrations may accumulate slightly differently because each intermediate sum is rounded differently;
- a result that is mathematically zero may retain a negative sign and appear as -0.000 rather than 0.000 ; and
- a machine-readable value may contain more significant digits even when the values agree to the precision supported by the historical result.

Positive and negative zero compare as equal numerically. The sign records the direction from which the operation reached zero, but it does not represent a different aerodynamic magnitude. A text comparison that treats -0.000 and 0.000 as different strings therefore reports a representation difference rather than an engineering difference.

Similarly, suppose two internal values differ only far beyond the precision retained by the historical calculation. If a fixed-width report rounds one value down and the other up, the last displayed digit can differ even though the same method and data were used. Validation therefore compares parsed numeric values with a tolerance appropriate to the output family rather than requiring byte-identical text.

Accumulation and integration

Surface build-up, numerical integration, and repeated coefficient summation can expose more than one final-bit difference. A small rounding change made at each station is carried into later partial sums, and the total depends on the precision of every intermediate result.

Such a difference remains a precision difference only when the following have been confirmed:

- both implementations include the same stations and components;
- the terms have the same signs, units, and reference quantities;
- the same interpolation and integration formula is used; and
- no term is omitted, duplicated, or read from a different table location.

A larger discrepancy is not accepted merely because the calculation contains many additions. Indexing, method selection, common-block layout, and unit conversion are investigated first.

Cancellation-sensitive body calculations

A more visible precision effect can occur when an analytic body method subtracts two nearly equal quantities. The true residual may be much smaller than either operand. Single precision can lose most of the significant digits in that residual, while double precision retains more of them. If the residual is then divided by another small quantity or used in a sensitive correlation, the difference can be amplified.

This behaviour was established in an ogive calculation at Mach 2.36 and $\alpha = 15^\circ$ using the Van Dyke Hybrid body-pressure method. In the first-order axial-flow path, **VANDYK** calls **DELV**, which accumulates the axial and radial perturbation velocities VX and VR . Those values are then passed to **AXBNDY**, where the local source-strength coefficient $C(I)$ is obtained from the tangency boundary condition.

The decisive historical statements are:

```
C(I) = RBP(I)*(1.0+VX+TTX)-TTR-VR
DEN = (XB(I)-XI)*(BETA*(F2/TAU-TAU*F1)
1      +2.*RBP(I)*(F1-F2))
C(I) = C(I)/DEN
```

The first statement is the numerator of the final division:

$$C_{\text{num}} = \text{RBP}(I)(1 + VX + \text{TTX}) - \text{TTR} - \text{VR}.$$

At body station $I = 18$, the targeted historical trace gave approximately

$$\begin{aligned} VX &= 1.06383073, & VR &= -3.04749346, \\ \text{TTX} &= -1.04584134, & \text{TTR} &= 3.04745555. \end{aligned}$$

The dominant radial terms almost cancel:

$$-\text{TTR} - \text{VR} = -3.04745555 + 3.04749346 \approx 3.79 \times 10^{-5}.$$

After the remaining slope term is included, the historical numerator is only

$$C_{\text{num},F} \approx 3.0994 \times 10^{-5},$$

even though the individual terms being combined are of order one to three. The historical denominator is also small:

$$\text{DEN} \approx 1.7365 \times 10^{-3}.$$

The resulting historical source strength is therefore

$$C(18)_F = \frac{3.0994 \times 10^{-5}}{1.7365 \times 10^{-3}} \approx 1.7849 \times 10^{-2}.$$

Re-evaluating the printed historical quantities with single-precision arithmetic reproduced that value closely. The double-precision SpearDAT state retained a slightly different residual numerator,

$$C_{\text{num},C++} \approx 7.7191 \times 10^{-5},$$

which gives

$$C(18)_{C++} \approx 4.4446 \times 10^{-2}.$$

The absolute numerator difference is only about 4.62×10^{-5} , but the numerator itself is a cancellation residual and is divided by 1.74×10^{-3} . Consequently, the local source-strength coefficient differs by a factor of about 2.5. This is a conditioning effect: the small residual has a large relative uncertainty even though its input terms differ only in digits that single precision cannot retain reliably.

The source-strength difference then propagates through the Hybrid solution:

$C(I) \longrightarrow \text{ZE0X, PHIX, PHIR} \longrightarrow \text{CPV} \longrightarrow \textbf{WAVE/SIMP} \longrightarrow \text{integrated coefficients and pressure output.}$

This is why selected curved analytic noses can show visible differences in for003 and for010, rather than merely a different last digit.

Higher-precision investigation supported the residual retained by the double-precision SpearDAT path. The difference is therefore not evidence that the programs selected different geometry or pressure methods; it is an amplified consequence of evaluating the same poorly conditioned recursion at different precision.

This case also shows why “precision-different” does not always mean “only the final printed digit differs.” Most cases exhibit only final-digit or very small accumulated differences, but a poorly conditioned intermediate expression can magnify the effect. Such cases are identified individually in the validation evidence rather than covered by one unrestricted tolerance.

The IDEAL leading-edge singularity

IDEAL applies Weber's conformal-mapping method to an airfoil section. It creates 32 cosine-spaced chord stations from

$$\theta_i = \frac{i\pi}{32}, \quad x_i = \frac{1}{2}(\cos \theta_i + 1).$$

The order runs from the trailing edge towards the leading edge. At the final station,

$$i = 32, \quad \theta_{32} = \pi, \quad x_{32} = 0,$$

so index 32 represents the exact leading edge.

The routine calculates five internal singularity distributions. The relevant ones here are:

- ST2NU, the slope of the thickness distribution; and
- ST4NU, the vortex distribution due to camber.

For $i \neq j$, the historical ST2NU term contains

```
S2=-2.*(-1.)**(J-I)*SIN(THMU(J))/(SIN(THNU(I))*
1 (COS(THMU(J))-COS(THNU(I))))
```

and the corresponding ST4NU expression also contains $1/\sin(\text{THNU}(I))$. At $i = 32$,

$$\sin(\text{THNU}(32)) = \sin(\pi) = 0.$$

Because the inner index only runs from $j = 1$ to 31, the special $i = j$ branch is never available at $i = 32$. The routine therefore evaluates the general expression whose denominator is zero at the leading edge.

The mathematical Weber formulation requires the limiting value as $\theta \rightarrow \pi$. Obtaining that value requires combining the vanishing trigonometric terms with the local leading-edge behaviour of the thickness or camber distribution, for example through an analytic limit or L'Hôpital's rule. Neither implementation performs that limiting operation.

Instead, both programs evaluate $\sin(\pi)$ numerically. In floating-point arithmetic, the stored value of π and the trigonometric-library result do not generally make this expression exactly zero:

- in the historical single-precision calculation, the residual $\sin(\pi)$ is small but limited by single-precision representation;
- in double precision, the residual is normally many orders of magnitude smaller; and
- because that residual appears in the denominator, the nominally more accurate double-precision evaluation can produce a much larger magnitude for ST2NU(32) or ST4NU(32).

This apparently paradoxical result does not mean that lower precision gives the correct leading-edge value. Both values are substitutes created by round-off at a point where the unreduced expression is singular. The proper answer is the analytic limit, which neither code calculates.

The internal discrepancy does not alter the validated aerodynamic result for two concrete reasons. First, the ideal angle, zero-lift angle, and ideal lift coefficient are formed from finite camber stations and ST5NU values:


```

DAI05= .3739*ZC05+.04745*ST5NU(N-1)
DAI95=-.3739*ZC95+.04745*ST5NU(1)
...
AI=RAD*(DAI05+DAI+DAI95)
ALO=RAD*(DA95+DALO)
CLI=2.*PI/RAD*(AI-ALO)

```

Those scalar formulas do not use ST2NU(32) or ST4NU(32). Second, the validated downstream airfoil calculations use the finite interior distribution stations rather than the singular index-32 value. The printed section properties and final aerodynamic coefficients therefore match even though a routine-level comparison of the last internal distribution element can look dramatically different.

This example has the opposite impact from the **AXBNDY** case: the **IDEAL** discrepancy is large internally but does not propagate to the validated output, whereas the small **AXBNDY** residual difference is amplified into observable pressure and coefficient differences.

SpearDAT policy. SpearDAT retains double precision rather than deliberately rounding every operation back to the historical single-precision representation. Reproducing single-precision round-off operation by operation would make the result more dependent on a particular compiler and evaluation order without improving the engineering method.

The comparison policy is therefore:

Observation	Treatment
Signed zero or a final printed-digit change	Treat as a representation difference after the underlying numeric values are shown to agree.
Small accumulated difference	Accept with the output-specific tolerance only after term selection, ordering, units, and formulas have been verified.
Amplified cancellation-sensitive difference	Use routine-level or higher-precision evidence to determine which value is numerically better supported; do not force SpearDAT to reproduce lost single-precision digits.
Different method, branch, table station, sign, or coefficient definition	Do not classify as precision; investigate and document the actual cause.
Finite result in one implementation and a non-finite or unavailable result in the other	Do not classify as precision unless the singular mechanism and its downstream effect have been demonstrated explicitly.

Result impact. For ordinary cases, users should expect agreement in the engineering result with possible differences in signed zero, the last displayed digit, or small accumulated quantities. Text files containing floating-point results are not required to be byte-identical.

Cancellation-sensitive analytic methods can show a larger difference even when both programs execute the same nominal correlation. Those cases are reviewed separately and are not used to widen the tolerance for unrelated outputs.

The leading-edge **IDEAL** example is an internal representation difference only in the validated path: neither implementation evaluates the mathematical limit, but the discrepant intermediate does not alter the reported aerodynamic coefficients.

Verification. Routine, integration, and full-stack comparisons cover body, fin, synthesis, and report paths. Ordinary differences were traced to rounding, signed zero, or accumulated arithmetic after method and data equivalence had been established.

The cancellation-sensitive Hybrid case was checked against a higher-precision evaluation, which supported retaining the double-precision SpearDAT result. The **IDEAL** leading-edge discrepancy was traced through its downstream consumers and confirmed not to affect the validated aerodynamic outputs.

No single global numeric tolerance is inferred from this entry. Each output family retains its own acceptance criterion, and a mismatch outside that criterion remains an investigation item until its mechanism is established.

Note

Precision explains how the same numerical method can produce slightly different representations; it does not excuse an unexplained engineering difference. The purpose of this entry is to state where such differences are expected and the evidence required before they are accepted.

A.16. Body Aerodynamic Methods and Pressure Outputs

The body pressure methods calculate both integrated aerodynamic coefficients and diagnostic distributions at internal body stations. A diagnostic quantity can become undefined even when the program still has enough finite state to continue part of the calculation. SpearDAT distinguishes such an invalid physical state from an ordinary zero-valued result.

KD-034 A non-positive SOSE static pressure makes the reported local Mach number undefined

Category	numerical robustness
Status	published
Compatibility	availability-different
Observability	conditionally observable
Evidence	S, F
Affected	The second-order shock-expansion body-pressure method, routine SOSE , the pressure-to-Mach inversion in EQN44 , and the M-LOCAL column of for012.

SOSE transports a nondimensional local static pressure P along the body and converts it to a local Mach number for the zero-angle pressure report. The conversion is performed by **EQN44** using the isentropic total-to-static pressure relation

$$M_{\text{LOCAL}} = \sqrt{\frac{2}{\gamma - 1} \left[\left(\frac{P_T}{P} \right)^{(\gamma-1)/\gamma} - 1 \right]}.$$

For the SOSE gas model, $\gamma = 1.4$. A real-valued local Mach number from this expression requires a positive static pressure and a physically valid total-to-static pressure ratio.

Historical implementation. The complete historical **EQN44** calculation is one unguarded statement:

```
AMX=SQRT(2./(GAM-1.)*((PT/P)**((GAM-1.)/GAM)-1.))
```

At each reported station, **SOSE** calls that routine and writes the result directly to the local-Mach output:

```
1150 CALL EQN44(GAM,PT,P,AMX)
      CP=(P-1.)/(GAM/2.)/AMS
      ...
      IF(IPRINT.EQ.1.AND.ALPH.EQ.0.)
1    WRITE(12,1350)XPT,CP(1),AMX
1350 FORMAT(4X,3F14.6)
```

If $P = 0$, the ratio PT/P divides by zero. On a permissive IEEE floating-point runtime this commonly becomes positive infinity, and the reported local Mach tends to infinity:

$$P \rightarrow 0^+ \implies \frac{PT}{P} \rightarrow +\infty \implies M_{LOCAL} \rightarrow +\infty.$$

If $P < 0$, the code raises a negative pressure to the fractional power $(\gamma - 1)/\gamma = 2/7$. The real-valued isentropic inversion is then invalid, and the floating-point result is normally NaN. A runtime configured to trap division by zero or invalid arithmetic can stop instead of writing either marker.

The fixed F14.6 format does not define a portable textual representation for these non-finite values. The validated historical runtime printed Infinity and NaN, but another compiler may use a different spelling, overflow asterisks, or terminate when the exception is raised.

Observed severe-geometry case

The behaviour was reproduced at Mach 3 for a coordinate-defined body containing a severe radius contraction:

	X (in)	R (in)
upstream station	9.57	3.26
downstream station	10.00	0.825

The radius changes by -2.435 inches over only 0.43 inches of axial distance, giving the approximate local slope

$$\frac{0.825 - 3.26}{10.00 - 9.57} \approx -5.66.$$

The same contraction appears in the SOSE pressure report using the normalised axial coordinate $X/D_{\max}$, where $D_{\max} = 12.8$ inches for this body. The two dimensional geometry stations therefore correspond to

$$\frac{9.57}{12.8} = 0.747656, \quad \frac{10.00}{12.8} = 0.781250.$$

Thus, the reported range

$$0.747656 \leq X/D_{\max} \leq 0.781250$$

does not enter the pressure-to-Mach equation. It simply identifies, in the coordinate system printed by `for012`, the body segment over which the abrupt contraction occurs and the invalid pressure states were observed.

Across calculation stations on that segment, the SOSE expansion solution drove the local static pressure to zero or below. The resulting infinite or non-real local Mach is best interpreted as a warning that the semi-empirical pressure method has left its valid numerical or physical domain for this abrupt geometry. It is not, by itself, a CFD-like prediction of the detailed flow through the discontinuity.

SpearDAT implementation and motivation. SpearDAT separates numerical protection from user-facing reporting.

At the utility level, **EQN44** first verifies that the gas and pressure inputs can define a real Mach number:

```
if (gam <= 1.0 || p <= 0.0 || pt < p) {
    amx = 0.0;
    return;
}
```

It also rejects a non-positive value under the final square root. Returning zero internally prevents a divide-by-zero or invalid-power operation from propagating uncontrolled infinities and NaN values through later control logic.

That protected internal zero must not be presented as the physical result. A local Mach number of zero describes stagnant flow; it does not describe a failed pressure inversion. The **SOSE** output layer therefore examines the pressure state when writing `for012`:

```
if (!std::isfinite(p) || !std::isfinite(amx)) {
    print("NaN");
} else if (p == 0.0) {
    print("Infinity");
} else if (p < 0.0) {
    print("NaN");
} else {
    print(amx);
}
```

The resulting policy is:

SOSE pressure state	for012 M-LOCAL	Meaning
Finite and positive	Numeric value	The isentropic inversion produced a usable local Mach number.
Exactly zero	Infinity	The positive-pressure limit of the inversion is unbounded.
Negative	NaN	No real local Mach number is defined by the isentropic expression.
Non-finite pressure or Mach	NaN	The local pressure-method state is already invalid or indeterminate.

The explicit strings are written in the same fourteen-character field occupied by the historical numeric value, so the row structure of `for012` remains stable.

Relationship to the aerodynamic coefficients

The pressure-to-Mach inversion is used for two different purposes within the SOSE calculation, and the distinction matters.

During the construction of the pressure solution, SOSE sometimes converts a local pressure into Mach number before a matching point, a compression-surface decision, or a Prandtl–Meyer expansion. In those locations, the resulting Mach number participates in subsequent branch selection or pressure transport. Protecting the inversion there prevents a division by zero or invalid fractional power from injecting uncontrolled non-finite values into the remaining station calculations.

After SOSE has finished determining the pressure state at a body station, it performs one further pressure-to-Mach conversion for the reported `M-LOCAL` value. At this stage, the station pressure `P` and the loading quantities `LAMDA`, `GAMMA`, and `DELTA` have already been established by the pressure method.

The station contributions are then calculated from those quantities:

$$\begin{aligned} CP &= \frac{P - 1}{(\gamma/2) M_\infty^2}, \\ ACAI &= 8 CP R RX, \\ ACNI &= 8 LAMDA R, \\ ACMI &= -ACNI (XPT + R RX). \end{aligned}$$

The reported local Mach number is not present in these station-load equations. Consequently, printing `Infinity` or `NaN` in the `M-LOCAL` column does not itself modify the pressure coefficient or the axial-force, normal-force, and pitching-moment contributions already defined for that station. The output marker reports the failure of the pressure-to-Mach interpretation; it is not substituted into the aerodynamic integration.

This distinction should not be read as validation of a non-positive SOSE pressure. The aerodynamic station loads still originate from a pressure state for which the local isentropic Mach interpretation has failed. The explicit marker therefore allows the user to recognise that the case has entered a questionable method-validity region, even though the program can continue and produce integrated coefficients.

Result impact. For valid positive pressures, the historical and SpearDAT local-Mach equations are the same and produce the same engineering quantity subject to ordinary precision differences.

For zero or negative pressure:

- the historical routine performs exceptional arithmetic directly and may print a runtime-dependent non-finite representation or stop under floating-point exception trapping;
- SpearDAT avoids that exceptional arithmetic in **EQN44**; and
- SpearDAT deliberately restores the physical diagnostic meaning in `for012` by printing `Infinity` or `NaN`, rather than the protected internal value `0`.

The difference is therefore primarily one of robustness and diagnostic clarity, not a claim that SpearDAT repairs the underlying SOSE pressure method for a geometry outside its useful domain.

Verification. The unguarded historical equation, all **SOSE** call sites, and the zero-angle for 012 write were compared directly with the C++ implementation.

The Mach-3 severe-contraction case reproduced the historical Infinity/NaN interval. The SpearDAT rerun completed without relying on divide-by-zero continuation and emitted the explicit marker corresponding to each non-positive pressure state. Valid neighbouring stations continued to print ordinary numeric local Mach values in the same three-column output layout.

Note

A protected numerical value and a reported physical value serve different purposes. Internally replacing an invalid inversion with zero can keep the program stable; printing that zero would falsely imply stagnant flow. SpearDAT therefore protects the calculation while reporting that the local Mach number is unbounded or undefined.

A.17. Elliptical-Body Aerodynamic Corrections

Elliptical-body aerodynamics are obtained by correcting an equivalent circular body in two orthogonal planes. The correction factors are evaluated repeatedly along the interpolated body mould line, so a numerically fragile local cross-section can affect the longitudinal integrals used to scale the final body force and moment components.

KD-035 Near-circular elliptical sections could lose significance in the Newtonian correction

Category	numerical robustness
Status	published
Compatibility	numerically-different
Observability	conditionally observable
Evidence	S
Affected	Newtonian correction factors for coordinate-defined elliptical bodies; the nonlinear body normal-force and pitching-moment corrections and their roll-resolved lateral counterparts; routines CNNEWT , GETCNO , and BODYE .

CNNEWT evaluates an empirical Newtonian correction factor for an elliptical cross-section. Its inputs are the major semi-axis A, the minor semi-axis B, and the orientation of that ellipse relative to the crossflow.

For a circular section,

$$A = B,$$

the correction must be

$$\frac{C_N}{C_{N0}} = 1.$$

The historical routine recognises that exact case explicitly:

```
CNCN0=1.
...
IF(A .EQ. B)GO TO 1010
```

The difficulty occurs when the two axes are extremely close but are not represented by exactly equal floating-point values.

The removable singularity

For the orientation in which the major axis is perpendicular to the crossflow, define

$$q = \frac{B}{A}, \quad 0 < q \leq 1.$$

Apart from the empirical table factor, the historical expression contains

$$-\frac{q^2}{(1-q^2)^{3/2}} \ln \left[\frac{A}{B} \left(1 + \sqrt{1-q^2} \right) \right] + \frac{1}{1-q^2}.$$

For the orthogonal orientation, define

$$r = \frac{A}{B}, \quad r \geq 1.$$

The corresponding expression contains

$$\frac{r^2}{(r^2-1)^{3/2}} \tan^{-1} \left(\sqrt{r^2-1} \right) - \frac{1}{r^2-1}.$$

As the ellipse approaches a circle,

$$q \rightarrow 1, \quad r \rightarrow 1,$$

the individual terms in each expression become very large because their denominators approach zero. Their difference nevertheless has a finite limit. In both orientations, the bracketed geometric expression tends to

$$\frac{2}{3}.$$

At a unit axis ratio, the empirical correction table gives 1, and the remaining multiplier tends to 3/2. The complete correction therefore approaches

$$1 \times \frac{3}{2} \times \frac{2}{3} = 1,$$

which is the circular-body value already used by the exact-equality branch.

The singularity is therefore removable mathematically, but it is not removed algebraically in the historical source. The routine obtains the finite result by subtracting two separately large quantities.

Why exact equality is not sufficient

The historical protection applies only when the two stored values satisfy the bit-for-bit comparison

$$A = B.$$

That condition is adequate for an explicitly circular section whose width and height were created from the same value. It is not adequate for every near-circular section produced by interpolation.

GETCNO evaluates the elliptical-body correction at 101 uniformly spaced longitudinal stations. At each station it independently interpolates body half-width and half-height, then assigns the larger value to **AMAJOR** and the smaller value to **AMINOR** before calling **CNNEWT**.

If the width and height distributions cross along the body, one interpolated station can be circular while neighbouring stations are only nearly circular. Round-off can also leave two values that are physically indistinguishable but not exactly equal. Those stations bypass the historical equality test and enter the cancellation-sensitive formulas.

In single precision, much of the significant information in the small difference between the two large terms can then be lost. The resulting correction can depart sharply from the smooth circular limit even though the section itself differs negligibly from a circle. The problem is not the Newtonian method or the empirical correction table; it is the numerical form used to evaluate their near-circular limit.

SpearDAT implementation and motivation. SpearDAT retains the historical exact-circle checks and adds a scale-relative near-circle test:

```
double scale = std::max(std::abs(a), std::abs(b));
if (std::abs(a - b) <= 1.0e-6 * scale) {
    return; // CNCNO remains the circular limit, 1.0
}
```

Thus, when the two semi-axes differ by no more than one part in one million relative to the larger axis, SpearDAT uses the analytic circular limit

$$\text{CNCNO} = 1.$$

The test is relative rather than dimensional. It therefore applies consistently whether the body dimensions are expressed in inches, feet, metres, or another supported length unit.

This guard does not blend elliptical sections towards a new empirical model. It only replaces an ill-conditioned evaluation of a known removable singularity by its limiting value. Outside the near-circular neighbourhood, SpearDAT uses the same axis ratios, the same NACA TN D-7228 empirical table, the same two orientation formulas, and the same $\cos^2 \phi / \sin^2 \phi$ directional blending as the historical routine.

How the local correction reaches reported coefficients

For each of the 101 stations, **GETCNO** stores the Newtonian correction and integrates two quantities along the body:

$$C_{NNEW} = \int \left(\frac{C_N}{C_{N0}} \right)_{\text{Newtonian}} R dx,$$

$$C_{MNEWT} = \int \left(\frac{C_N}{C_{N0}} \right)_{\text{Newtonian}} R (X_{CG} - x) dx.$$

The first weights the correction by the equivalent local radius; the second also includes the moment arm to the centre of gravity.

BODYE compares these elliptical-body integrals with the corresponding equivalent-circular-body integrals. Their ratios scale the nonlinear Newtonian portions of the body normal-force and pitching-moment results in the horizontal and vertical body planes.

Those plane results are subsequently resolved using aerodynamic roll. Depending on the flight attitude, the affected nonlinear terms can therefore appear in the reported body contributions to:

- normal-force coefficient C_N ;
- pitching-moment coefficient C_M ;
- side-force coefficient C_Y ; and
- yawing-moment coefficient C_{LN} .

A single unstable station need not dominate either integral, but the historical correction is evaluated at every station and is multiplied into the quadrature input. A spurious near-circular value can therefore disturb the integrated scaling factors rather than remaining an unused local diagnostic.

Result impact. The correction is conditional. It is relevant to elliptical-body paths in which an interpolated section is nearly circular, particularly where the width and height distributions cross and the identity of the major axis changes from one body plane to the other.

Exactly circular sections already return 1 in both implementations. Clearly elliptical sections outside the relative near-circle threshold continue to use the historical equations unchanged.

Within the threshold, SpearDAT returns the continuous circular limit rather than a cancellation-dominated value. The affected Newtonian body force and moment components can therefore differ from the historical result. No campaign-wide coefficient magnitude is assigned here because this entry is source-verified: the visible effect depends on the location, weighting, moment arm, surrounding cross-sections, and aerodynamic roll of the particular body.

Verification. The Fortran and C++ implementations of **CNNEWT** and **GETCNO** were compared directly. The exact-circle branch, both elliptical orientation formulas, the empirical correction table, and the directional blending were found to be otherwise equivalent.

The limiting behaviour of both orientation formulas confirms $C_N/C_{N0} \rightarrow 1$ as $A/B \rightarrow 1$. The SpearDAT guard therefore selects the existing mathematical limit rather than introducing an unrelated fallback.

The consumer trace through **GETCNO** and **BODYE** establishes that the correction enters the Newtonian normal-force and pitching-moment integrals and can reach the roll-resolved body coefficients. Crossover-sensitive translation tests identified this path as the case in which SpearDAT values should be used as the numerical reference, but no full-stack magnitude is asserted by this source-level entry.

Note

The exact circle was already handled correctly. The correction addresses the numerically more difficult case in which interpolation produces an ellipse that is effectively circular but not bit-for-bit equal in its two stored semi-axes.

B. MATLAB Input Reference

B.1. Purpose and Use of This Reference

This appendix lists the MATLAB input fields accepted by SpearDAT. It is intended as a lookup reference for exact field paths, MATLAB value forms, units, allowed choices, omission behaviour, and activation conditions. The engineering chapters explain how the corresponding capabilities are used; this appendix states what may be supplied to the interface.

The reference follows the MATLAB hierarchy seen by the user. Public paths and enumerated text values are therefore written exactly as they must appear in a MATLAB script. They are case-sensitive.

Important

This appendix lists the complete MATLAB input surface accepted by SpearDAT. It does not imply that every field may be populated in the same request. Some fields select mutually exclusive analysis modes, some activate subordinate branches, and some become required only when a related capability is enabled. The availability and activation information must therefore be read together with each field definition.

B.1.1. Reference Conventions

The tables use the following terms.

Required The field must be supplied whenever the stated parent branch or capability is active.

Optional The field may be omitted. The table states the resulting default or omission behaviour.

Conditionally required Another field or selector determines whether the field must be supplied.

Advanced The field is supported but is intended for specialised analysis or diagnostic use rather than ordinary aerodynamic requests.

Legacy diagnostic The field requests historical report or dump content. It is not part of the normal structured aerodynamic table.

Unset The field is absent or is assigned []. In both cases SpearDAT treats the value as not supplied.

Explicit zero A supplied numeric zero. It is retained as a value where zero is valid and is not treated as unset.

Unless a field entry states otherwise, the following MATLAB representation rules apply:

- numeric scalars and vectors are real, full MATLAB double values;
- Boolean fields are MATLAB logical scalars;
- text fields accept a MATLAB string scalar or character row vector;
- text vectors accept a string array, a cell vector of text, or one character row vector;
- nested objects are scalar structures;
- repeated objects are row or column structure vectors; and

- unknown fields are rejected at every public structure level.

The complete representation rules and the distinction between omitted and explicit values are explained in [chapter 4](#).

B.2. MATLAB Call and Top-Level Structures

The MATLAB interface accepts one required analysis-input structure and one optional execution-options structure:

```
result = speardat_mex(input);  
result = speardat_mex(input, options);
```

Listing B.1: Supported MATLAB calling forms.

The two structures have different roles. `input` describes the aerodynamic analysis. The optional second argument, `options`, controls file retention for that MATLAB execution.

B.2.1. Call Arguments

Table B.1.: MATLAB arguments accepted by `speardat_mex`.

Argument	MATLAB form	Availability	Description
<code>input</code>	Scalar structure	Required	Complete SpearDAT analysis request. Its only accepted top-level fields are <code>settings</code> and <code>caseInput</code> .
<code>options</code>	Scalar structure	Optional; default is omitted	Execution-only options. Its only accepted fields are <code>legacyFiles</code> and <code>outputDirectory</code> . An empty second argument has the same effect as omission.
<code>result</code>	Scalar structure returned by SpearDAT	Optional output	Structured preparation, execution, diagnostic, case, and aerodynamic results. The result model is documented separately from this input reference.

The function accepts one or two input arguments and returns at most one output. A malformed argument structure raises a MATLAB input error before an analysis request is prepared.

B.2.2. Top-Level Analysis Input

No other field is accepted directly under `input`. In particular, the execution-only fields `legacyFiles` and `outputDirectory` belong to the optional second argument, not to `input.settings`.

Table B.2.: Fields accepted directly under input.

Field	MATLAB form	Units	Availability/ default	Description
<code>input.settings</code>	Scalar structure	—	Optional; defaults apply when omitted	Unit selections, request-wide calculation options, and advanced legacy diagnostics. See section B.3 .
<code>input.caseInput</code>	Scalar structure	—	Optional at the representation boundary; engineering requirements depend on the requested analysis	Vehicle configuration, flight conditions, reference quantities, calculation requests, and requested outputs. The branch is defined in the following sections of this appendix.

B.2.3. Execution Options

The optional options structure controls whether the generated input and solver-output bundle is retained after MATLAB reconstructs the structured result.

A valid retained-output request is:

```
options = struct;
options.legacyFiles = true;
options.outputDirectory = fullfile(pwd, "speardat_output");

result = speardat_mex(input, options);
```

Listing B.2: Retaining the complete solver-output bundle.

When `legacyFiles` is false, `outputDirectory` must be omitted or empty. When `legacyFiles` is true, an empty output-directory value is rejected. Retained-output operation and directory contents are introduced in [chapter 3](#).

Inspecting Retained Legacy Files

When retained output is enabled, `result.legacyOutputDirectory` contains the retained batch root. Each generated scalar case also reports its own case directory through `result.cases(k).legacyDirectory`. A request containing several expanded BETA or PHI cases therefore has one retained subdirectory per case, for example `case_0000`, `case_0001`, and so on.

Each retained case directory contains the exact generated `for005.dat` input deck and the numbered legacy output files `for003.dat` through `for012.dat`. Files whose corresponding output path is inactive for the selected analysis may be empty. The main human-readable Missile DATCOM report is `for006.dat`.

The retained bundle also contains SpearDAT process and transport files. These are not historical Missile DATCOM files; they are used by the MATLAB execution path to describe the batch,

Table B.3.: Fields accepted under the optional execution options structure.

Field	MATLAB form	Units	Availability/ default	Description
options.legacyFiles	Logical scalar	—	Optional; default false	When false, SpearDAT uses a temporary working directory and removes it after reconstructing the result. When true, the complete batch and per-case legacy-format files are retained beneath options.outputDirectory.
options.outputDirectory	Text scalar	Path	Required when options.legacyFiles is true; otherwise omitted or empty	Directory used as the retained batch root. The path may not name a non-directory object. If the directory already exists, it must be empty; otherwise SpearDAT creates it.

preserve per-case status, and transfer resolved solver state back into the structured MATLAB result.

The per-case status and resolved-state files are therefore different from the numbered legacy reports. For example, `speardat_case_status.txt` may state that a case succeeded and that its resolved state was captured, while `for006.dat` contains the actual human-readable Missile DATCOM aerodynamic report. Likewise, `speardat_resolved_state.txt` is a SpearDAT transport artefact rather than an additional aerodynamic report.

The retained directory can be inspected directly from MATLAB:

```
retainedRoot = string(result.legacyOutputDirectory);
caseDirectory = string(result.cases(1).legacyDirectory);

disp(retainedRoot)
disp(caseDirectory)
dir(caseDirectory)
```

Listing B.3: Locating the retained files for the first case.

For example, the complete `for006` report for the first case can be read without manually reconstructing its path:

```
caseDirectory = string(result.cases(1).legacyDirectory);

for006Path = fullfile(caseDirectory, "for006.dat");
for006Text = fileread(for006Path);

disp(for006Text)
```

Table B.4.: SpearDAT files contained in a retained solver-output bundle.

File	Location	Purpose
speardat_batch_manifest.txt	Batch root	Lists the generated scalar case directories in execution order. For a single-case request it contains only case_0000; an expanded BETA or PHI request contains one ordered entry per generated case.
speardat_batch_status.txt	Batch root	Summarises completion of the complete request, including the total, successful, and failed case counts and whether every case succeeded.
speardat_solver_stdout.txt	Batch root	Captures the standard output stream of the isolated solver process. Normally it contains progress and completion messages, such as which case is running and the final number of successful cases.
speardat_solver_stderr.txt	Batch root	Captures the standard error stream of the isolated solver process. An empty file is normal for a clean run. If the subprocess reports a process-level or runtime error to standard error, the text is retained here for diagnosis.
speardat_case_status.txt	Per-case directory	Records the identity and completion status of one scalar case, whether its resolved state was captured successfully, and any case-level diagnostics generated by the solver-process boundary.
speardat_resolved_state.txt	Per-case directory	Serialises the completed <i>resolved case state</i> : the alpha and BETA/PHI conditions, resolved flight points, reference quantities, unit systems, selected body method and active options, together with calculated body and fin geometry where available. SpearDAT reads this transport file back after solver execution to reconstruct <code>result.cases(k).resolvedState</code> .

Listing B.4: Reading a retained for006 report from MATLAB.

For a multi-case request, change the index in `result.cases(k).legacyDirectory` to inspect the corresponding case. This avoids relying on the generated directory name or on the ordering of files in the batch root.

Note

SpearDAT captures the raw numbered solver outputs internally during execution, including the for006 report, even when temporary execution is used. The current MATLAB result does not, however, expose the raw for003–for012 text directly. The generated input deck is available in memory as `result.cases(k).for005Text`, and `result.cases(k).for006Status` reports the completion status derived from the main for006 report. The latter is status information only; it is not the report text itself. To inspect the complete for006 report, or any of the other numbered legacy output files, enable `options.legacyFiles`, supply `options.outputDirectory`, and read the retained file from `result.cases(k).legacyDirectory`.

The purpose of the individual numbered legacy output files is described in the “The legacy output files” subsection of [section A.4](#).

B.3. Request-Wide Settings

The `input.settings` branch contains selections that apply to the complete request. It may be omitted when its defaults are suitable.

B.3.1. Settings Branch

Although the unit selectors have defaults, explicit assignments are recommended in reusable analysis scripts:

```
input.settings.lengthSystem = "Inches";
input.settings.derivativeSystem = "Degrees";
```

Listing B.5: Selecting the request unit conventions explicitly.

Explicit selections make the interpretation of dimensional inputs and derivative outputs visible without relying on inherited defaults.

B.3.2. Run-Wide Options

These fields select analysis behaviour. They are unrelated to the optional second options argument used to retain files.

B.3.3. Advanced Legacy Diagnostics

The only field accepted directly under `input.settings.legacy` is the scalar structure `debug`:

Table B.5.: Fields accepted directly under `input.settings`.

Field	MATLAB form	Units	Availability/ default	Description
<code>input.settings.lengthSystem</code>	Text scalar; enumerated	—	Optional; default "Feet"	Selects the dimensional length system used by the request. Individual engineering-field tables identify the quantities governed by this selection. Allowed values are listed in table B.6 .
<code>input.settings.derivativeSystem</code>	Text scalar; enumerated	—	Optional; default "Degrees"	Selects whether applicable aerodynamic derivatives are expressed per degree or per radian. Allowed values are listed in table B.7 .
<code>input.settings.options</code>	Scalar structure	—	Optional; all contained flags default to false	Request-wide calculation and reporting options. See section B.3.2 .
<code>input.settings.legacy</code>	Scalar structure	—	Optional; inactive when omitted	Advanced historical diagnostic requests. These fields are intended for detailed solver review rather than ordinary structured aerodynamic results. See section B.3.3 .

Table B.6.: Allowed values of `input.settings.lengthSystem`.

MATLAB value	Selected system
"Inches"	Inch-based dimensional inputs.
"Feet"	Foot-based dimensional inputs; public default.
"Centimetres"	Centimetre-based dimensional inputs.
"Metres"	Metre-based dimensional inputs.

Table B.7.: Allowed values of `input.settings.derivativeSystem`.

MATLAB value	Selected derivative convention
"Degrees"	Applicable derivatives are expressed per degree; public default.
"Radians"	Applicable derivatives are expressed per radian.

Table B.8.: Fields accepted under `input.settings.options`.

Field	MATLAB form	Units	Availability/ default	Description
<code>input.settings.options.noLateralDirectional</code>	Logical scalar	—	Optional; default false	Suppresses lateral-directional output when set to true.
<code>input.settings.options.buildOutput</code>	Logical scalar	—	Optional; default false	Requests component build-up output. A simultaneous trim request is accepted with a warning because component build-up output is not normally provided for trimmed cases.

Table B.9.: Fields accepted under `input.settings.legacy`.

Field	MATLAB form	Availability/ default	Description
<code>input.settings.legacy.debug</code>	Scalar structure	Advanced; inactive when omitted	Requests historical diagnostic, namelist-echo, and selected internal-block dump output.

Note

The fields under `input.settings.legacy` expose specialised historical diagnostic facilities. They are not required to run an aerodynamic analysis, and they should not be used as substitutes for the structured result fields.

B.4. Case Definition

The `input.caseInput` branch describes the aerodynamic problem to be analysed. It contains the user-defined case identity, requested calculation and output options, flight conditions, reference quantities, vehicle configuration, and optional analysis features.

Only one `caseInput` structure is supplied in a MATLAB request. SpearDAT may expand that request into several scalar cases when `flight.betaDeg` or `flight.rollDeg` contains more than one retained value. The user-supplied identifier is preserved, while SpearDAT assigns a separate generated identity to each prepared case, as explained in [section 4.9](#).

B.4.1. Case-Input Branch

Table B.10.: Fields accepted under `input.settings.legacy.debug`.

Field	MATLAB form	Units	Availability/ default	Description
<code>input.settings.legacy.debug.dumpCase</code>	Logical scalar	—	Legacy diagnostic; default false	Requests the supported current-case internal dump in the main report.
<code>input.settings.legacy.debug.dumpBlocks</code>	Text vector	—	Legacy diagnostic; default empty	Requests one named internal-block dump per vector element. Each name must be non-empty, contain no comma, contain no newline, and be no longer than 75 characters.
<code>input.settings.legacy.debug.writeFormat</code>	Text scalar	—	Legacy diagnostic; unset by default	Supplies historical format text associated with selected block writes. The text must begin with an opening parenthesis, contain no newline, and be no longer than 73 characters. The current translated output path otherwise uses its default numeric formatting.
<code>input.settings.legacy.debug.printNamelists</code>	Logical scalar	—	Legacy diagnostic; default false	Requests printed namelist echo/debug output.
<code>input.settings.legacy.debug.writeBlocks</code>	Structure vector	—	Legacy diagnostic; default empty	Requests selected internal block ranges. Each element has the fields defined in table B.11 .

Table B.11.: Fields in each `input.settings.legacy.debug.writeBlocks` element.

Element field	MATLAB form	Units	Availability/ default	Description
<code>name</code>	Text scalar	—	Required for each element	Internal block name. It must be non-empty, contain no comma or newline, and be no longer than 60 characters.
<code>start</code>	Integer-valued double scalar	Index	Required for each element	First requested one-based element. It must be positive.
<code>end</code>	Integer-valued double scalar	Index	Required for each element	Last requested one-based element. It must satisfy $\text{end} \geq \text{start}$. The complete generated write request must also fit within the supported line length.

Table B.12.: Fields accepted directly under `input.caseInput`.

Field	MATLAB form	Units	Availability/ default	Description
<code>input.caseInput.id</code>	Text scalar	—	Optional; unset by default	User-defined engineering identifier. It is preserved as CASEID and is not used to construct SpearDAT's generated caseName. See section B.4.2 .
<code>input.caseInput.comments</code>	Text vector	—	Optional; default empty	User comments associated with the analysis. Each element represents one physical comment line. See section B.4.2 .
<code>input.caseInput.options</code>	Scalar structure	—	Optional; all flags default to false	Dynamic derivatives, spin/Magnus calculations, and checking-only operation. See section B.4.3 .
<code>input.caseInput.output</code>	Scalar structure	—	Optional; flags default to false; target vectors are empty	Machine-readable aerodynamic output, pressure files, partial reports, and selected detailed reports. See section B.5 .
<code>input.caseInput.theory</code>	Scalar structure	—	Optional; default "VanDykeHybrid"	Body aerodynamic method. See section B.6 .
<code>input.caseInput.flight</code>	Scalar structure	Angular, speed, and atmosphere units	Conditionally required	Attitude definition, incidence schedule, speed schedule, and atmospheric conditions. See section B.7 .
<code>input.caseInput.reference</code>	Scalar structure	Length and area units	Optional; some values may be derived	Aerodynamic reference area, reference lengths, moment centre, and surface condition. See section B.8 .
<code>input.caseInput.body</code>	Scalar structure	Selected length system	Optional; required for body-based configurations	Axisymmetric or elliptical body geometry, with optional base-jet and protuberance inputs. See section B.9 .

Continued on the next page

Table B.12.: Fields accepted directly under `input.caseInput` (continued).

Field	MATLAB form	Units	Availability/default	Description
<code>input.caseInput.finSets</code>	Structure vector	Length units and degrees	Optional; default empty	Zero to four fin-set definitions, including planform, panel orientation, and airfoil-section data. See section B.10 .
<code>input.caseInput.deflections</code>	Scalar structure	Length units and degrees	Optional; inactive when omitted	Fixed fin-panel deflections and optional hinge-line data. See section B.11 .
<code>input.caseInput.trim</code>	Scalar structure	Degrees	Optional; inactive when omitted	Longitudinal static-trim request, selected fin set, movable panels, search bounds, and sign reversals. See section B.12 .
<code>input.caseInput.inlet</code>	Scalar structure	Length units, degrees, and dimensionless ratios	Optional; inactive when omitted	Inlet type, diverter geometry, circumferential placement, five-point profile, cover state, ramp angle, and optional additive-drag mass-flow schedule. See section B.13 .
<code>input.caseInput.experimentalData</code>	Scalar structure	Degrees, selected reference units, and dimensionless coefficients	Optional; inactive when omitted	Experimental or externally generated aerodynamic coefficient data used for direct substitution, hybrid theoretical/experimental build-up, or advanced incrementing workflows. See section B.14 .

The branch may be constructed incrementally. A typical opening definition is:

```
input.caseInput.id = "Baseline";
input.caseInput.comments = ...
    "Axisymmetric reference configuration";

input.caseInput.output.plot = true;
input.caseInput.theory.bodyMethod = "VanDykeHybrid";
```

Listing B.6: Beginning a user-defined case request.

The remaining engineering branches are then added according to the configuration and operating conditions being analysed.

B.4.2. Case Identity and Comments

The fields in [table B.13](#) provide descriptive traceability. They do not alter the aerodynamic geometry or operating condition.

A string array or cell vector may be used for several comments:

Table B.13.: Case identity and comment fields.

Field	MATLAB form	Units	Availability/default	Description and constraints
<code>input.caseInput.id</code>	Text scalar	—	Optional; unset by default	Descriptive user identifier preserved as CASEID in every scalar case generated from the request. Carriage-return and newline characters are rejected. A value longer than 73 characters produces a warning because it exceeds the documented legacy payload width.
<code>input.caseInput.comments</code>	Text vector	—	Optional; default empty	Each vector element becomes one comment line and must not contain an embedded carriage return or newline. A comment longer than 79 characters produces a warning because the complete prefixed line may exceed the historical 80-column width. An empty text element represents an empty comment line.

```
input.caseInput.id = "Baseline";

input.caseInput.comments = [
    "Reference body and three fin sets"
    "Standard atmosphere at sea level"
    "Static aerodynamic calculation"
];
```

Listing B.7: Adding several case comments.

The identifier should be short enough to remain legible in reports and specific enough to distinguish the engineering purpose of the request. SpearDAT does not append the BETA or PHI value to this identifier. Expanded cases instead receive independent generated names such as "case_0000" and retain separate sweep metadata.

Note

Use `input.caseInput.id` for the engineering name of the analysis. Use `result.cases(n).caseName`, `caseIndex`, and the sweep metadata to distinguish the scalar cases generated during execution.

B.4.3. Case Calculation Options

The `input.caseInput.options` structure selects calculations that belong to the aerodynamic case rather than to the overall MATLAB execution.

For example:

```
input.caseInput.options.dynamicDerivatives = true;
input.caseInput.options.spinMagnus = false;
input.caseInput.options.validateOnly = false;
```

Listing B.8: Requesting dynamic derivatives.

`validateOnly` controls whether the normal aerodynamic calculation is performed. When it is false, as in the example above, SpearDAT performs the normal calculation. When it is true, SpearDAT generates the case normally but requests the solver's checking-only path: the generated input is read and checked, and the normal aerodynamic coefficient calculation is not carried out. This is useful when reviewing a newly constructed case before relying on its calculated results.

SpearDAT's MATLAB representation checks and engineering validation are applied in either mode. A checking-only request is therefore different from an unsuccessful request: it is an intentional analysis option, not an error. Because the aerodynamic calculation is skipped, calculated-output requests may return no aerodynamic rows or auxiliary calculation results; SpearDAT reports a warning when such outputs are requested together with `validateOnly=true`.

Table B.14.: Fields accepted under `input.caseInput.options`.

Field	MATLAB form	Units	Availability/ default	Description
<code>input.caseInput.options.dynamicDerivatives</code>	Logical scalar	—	Optional; default false	Requests dynamic aerodynamic derivatives. Reported rate derivatives use Missile DATCOM's non-dimensional reduced-rate conventions; this field does not accept a physical body-rate schedule.
<code>input.caseInput.options.spinMagnus</code>	Logical scalar	—	Optional; default false	Requests spin/Magnus calculations. A request made without dynamic derivatives, partial output, or build-up output is accepted with a warning because the historical reporting path may provide limited or no useful printed spin/Magnus output.
<code>input.caseInput.options.validateOnly</code>	Logical scalar	—	Optional; default false	Requests checking of the generated aerodynamic input without carrying out the normal aerodynamic calculation. Output requests may consequently produce no calculated aerodynamic results.

B.5. Output Requests

The `input.caseInput.output` structure controls which machine-readable and detailed solver outputs are requested. Every Boolean field defaults to false; every target vector defaults to empty.

Important

Set `input.caseInput.output.plot` to true for ordinary MATLAB coefficient workflows. The `result.aerodynamics` table is constructed from the machine-readable aerodynamic output requested by this field. An analysis may otherwise complete successfully while returning no machine-readable aerodynamic rows.

B.5.1. Primary Output Fields

A typical output request is:

```
input.caseInput.output.plot = true;
input.caseInput.output.pressures = false;
input.caseInput.output.partialAll = false;

input.caseInput.output.printGeometry = [
    "Body"
    "FinSet1"
];

input.caseInput.output.printAero = [
    "Body"
    "Synths"
];
```

Listing B.9: Requesting structured and selected detailed output.

The target names are exact and case-sensitive. A target referring to undefined body, fin-set, or inlet geometry is accepted with a warning because the requested report may be empty or unavailable.

B.5.2. Geometry Print Targets

A body, fin-set, or inlet target should normally be requested only when the corresponding geometry is present in the same caseInput.

B.5.3. Aerodynamic Print Targets

The selector requests report content; it does not itself define the corresponding component or enable a trim calculation. For example, "Trim" does not replace the `input.caseInput.trim` definition.

Table B.15.: Primary fields accepted under `input.caseInput.output`.

Field	MATLAB form	Units	Availability/ default	Description
<code>input.caseInput.output.plot</code>	Logical scalar	—	Optional; default false	Requests machine-readable aerodynamic output. This output is parsed into the combined MATLAB aerodynamic table and should normally be enabled when coefficient data are required.
<code>input.caseInput.output.pressures</code>	Logical scalar	—	Optional; default false	Requests applicable body, fin, or inlet pressure-related auxiliary output. It does not select a body aerodynamic method. A request with no body, finset, or inlet geometry produces a warning.
<code>input.caseInput.output.partialAll</code>	Logical scalar	—	Optional; default false	Requests broad geometric and aerodynamic partial output. This is useful for detailed investigation but produces a warning because the resulting report can be very large.
<code>input.caseInput.output.printExtrapolation</code>	Logical scalar	—	Optional; default false	Enables detailed extrapolation-related printed output. The default suppresses this report content.
<code>input.caseInput.output.printGeometry</code>	Text vector; enumerated	—	Optional; default empty	Requests selected geometry reports. Exact target names are listed in table B.16 .
<code>input.caseInput.output.printAero</code>	Text vector; enumerated	—	Optional; default empty	Requests selected aerodynamic reports. Exact target names are listed in table B.17 .

Table B.16.: Allowed values of `input.caseInput.output.printGeometry`.

MATLAB value	Requested geometry report
"Body"	Active axisymmetric or elliptical body geometry.
"FinSet1"	Fin set 1 geometry.
"FinSet2"	Fin set 2 geometry.
"FinSet3"	Fin set 3 geometry.
"FinSet4"	Fin set 4 geometry.
"Inlet"	Inlet geometry.

Table B.17.: Allowed values of `input.caseInput.output.printAero`.

MATLAB value	Requested aerodynamic report
"Body"	Body aerodynamic report.
"FinSet1"	Fin set 1 aerodynamic report.
"FinSet2"	Fin set 2 aerodynamic report.
"FinSet3"	Fin set 3 aerodynamic report.
"FinSet4"	Fin set 4 aerodynamic report.
"Synths"	Synthesised configuration aerodynamic report.
"Trim"	Trim aerodynamic report.
"Bend"	Bending-related aerodynamic report.
"Hinge"	Hinge-moment-related aerodynamic report.
"Inlet"	Inlet aerodynamic report.

B.5.4. Backward-Compatible Convenience Fields

The MATLAB interface also accepts the convenience fields in [table B.18](#). They are retained for compatibility with earlier API examples. New scripts should normally use `printGeometry` and `printAero`, which express the requested targets directly.

Table B.18.: Backward-compatible output convenience fields.

Field	MATLAB form	Availability/default	Equivalent request
<code>input.caseInput.output.printBodyGeometry</code>	Logical scalar	Compatibility field; default false	Equivalent to including "Body" in <code>printGeometry</code> .
<code>input.caseInput.output.printBodyAerodynamics</code>	Logical scalar	Compatibility field; default false	Equivalent to including "Body" in <code>printAero</code> .
<code>input.caseInput.output.printFinGeometry</code>	Four-element logical vector	Compatibility field; all elements default to false	Elements 1–4 request geometry reports for fin sets 1–4, respectively.
<code>input.caseInput.output.printFinAerodynamics</code>	Four-element logical vector	Compatibility field; all elements default to false	Elements 1–4 request aerodynamic reports for fin sets 1–4, respectively.

For example:

```
% Preferred form for new scripts
input.caseInput.output.printGeometry = ...
    ["Body", "FinSet1"];
```

```
% Backward-compatible equivalent
input.caseInput.output.printBodyGeometry = true;
input.caseInput.output.printFinGeometry = ...
    [true, false, false, false];
```

Listing B.10: Equivalent modern and compatibility output requests.

If the same report target is requested through both forms, SpearDAT emits the request once rather than duplicating it.

B.5.5. Interactions and Diagnostic Warnings

Output requests may be valid but still unlikely to produce useful content. SpearDAT therefore reports warnings or information in the following situations:

- `input.settings.options.buildOutput` is requested together with `trim`; component build-up output is not normally provided for a trimmed case;
- `spinMagnus` is requested without dynamic derivatives, partial output, or build-up output; the historical report path may expose limited spin/Magnus content;
- `partialAll` is enabled; the report may become very large;
- `validateOnly` is combined with calculated-output requests; checking-only operation may suppress those results;
- `pressures` is enabled without any body, fin-set, or inlet geometry; no component supports the requested pressure output;
- a body, fin-set, or inlet print target refers to a component that has not been defined; and
- `pressures` is requested without an explicit body method; the default "VanDykeHybrid" path remains active.

These diagnostics do not all represent errors. They identify requests that can be prepared but may not produce the report content the user expects.

B.6. Body-Theory Selection

The `input.caseInput.theory` structure contains one field:

Table B.19.: Fields accepted under `input.caseInput.theory`.

Field	MATLAB form	Units	Availability/ default	Description
<code>input.caseInput.theory.bodyMethod</code>	Text scalar; enumerated	—	Optional; default "VanDykeHybrid"	Selects the body aerodynamic method. Exact values are listed in table B.20 . The single selector prevents simultaneous SOSE and HYPER requests.

Table B.20.: Allowed values of `input.caseInput.theory.bodyMethod`.

MATLAB value	Selected method
"VanDykeHybrid"	Default Van Dyke hybrid body-method path. Omitting <code>bodyMethod</code> has the same method-selection effect.
"Sose"	Selects the second-order shock-expansion body method.
"Hyper"	Selects the hypersonic body-method path.

For a script intended to be reviewed or reused, the method may be stated explicitly even when the default is required:

```
input.caseInput.theory.bodyMethod = "VanDykeHybrid";
```

Listing B.11: Selecting the body method explicitly.

`bodyMethod` controls the body aerodynamic calculation whether or not pressure files are requested. Conversely, `input.caseInput.output.pressures` requests applicable auxiliary pressure output but does not change the selected body method.

Note

An omitted `bodyMethod` and an explicit "VanDykeHybrid" selection use the same aerodynamic method. The explicit form is preferable when the method choice forms part of a controlled test definition because it records the analyst's intent directly in the MATLAB input.

B.7. Flight Conditions

The `input.caseInput.flight` structure defines the incidence schedule, attitude convention, primary speed schedule, and atmospheric or Reynolds-number definition for the analysis.

A normal aerodynamic calculation requires a complete flight-condition definition. Within one scalar SpearDAT case, the angle-of-attack schedule and the speed schedule are independent axes: the solver evaluates the requested angles of attack at each active Mach number or freestream velocity. A vector of sideslip or aerodynamic-roll values has different semantics. It requests separate scalar cases, as described in [section 4.6](#).

Note

The `flight` branch may be absent from a partially constructed MATLAB structure, but a geometry definition by itself is not a complete aerodynamic analysis. Once any field in `flight` is populated, SpearDAT requires a valid angle-of-attack schedule and one valid primary speed schedule.

B.7.1. Flight Branch

Table B.21.: Fields accepted directly under `input.caseInput.flight`.

Field	MATLAB form	Units	Availability/default	Description
<code>input.caseInput.flight.alphaDeg</code>	Real vector	deg	Required when the flight branch is active; 2–20 finite values	Body-axis angle of attack in "BodyAlphaBeta" mode, or total incidence in "TotalAlphaRoll" mode. The number of values is derived automatically.
<code>input.caseInput.flight.attitudeMode</code>	Text scalar; enumerated	—	Optional; inferred when omitted	Selects the incidence representation. Exact values are listed in table B.22 .
<code>input.caseInput.flight.betaDeg</code>	Real scalar or vector	deg	Optional; default empty	Sideslip angle for "BodyAlphaBeta" mode. Multiple retained values request separate scalar cases. Mutually exclusive with <code>rollDeg</code> .
<code>input.caseInput.flight.rollDeg</code>	Real scalar or vector	deg	Optional; default empty	Aerodynamic roll angle for "TotalAlphaRoll" mode. Multiple retained values request separate scalar cases. Mutually exclusive with <code>betaDeg</code> .
<code>input.caseInput.flight.speed</code>	Scalar structure	Varies	Required when the flight branch is active	Selects Mach number or freestream velocity as the primary speed schedule. See section B.7.3 .
<code>input.caseInput.flight.atmosphere</code>	Scalar structure	Varies	Optional; explicit definition recommended	Selects a standard-atmosphere, specified-Reynolds-number, or wind-tunnel definition. See section B.7.4 .

All numeric flight fields must contain finite real values. MATLAB row and column vectors are both accepted; SpearDAT interprets them as one-dimensional schedules.

B.7.2. Incidence and Attitude Convention

Table B.22.: Allowed values of `input.caseInput.flight.attitudeMode`.

MATLAB value	Interpretation
"BodyAlphaBeta"	<code>alphaDeg</code> is body-axis angle of attack and <code>betaDeg</code> is body-axis sideslip. This is the inferred mode when <code>attitudeMode</code> and <code>rollDeg</code> are both omitted.
"TotalAlphaRoll"	<code>alphaDeg</code> is total incidence and <code>rollDeg</code> is aerodynamic roll angle. This mode is inferred when <code>attitudeMode</code> is omitted and <code>rollDeg</code> is populated.

The two attitude conventions are alternatives. A request must not populate both `betaDeg` and `rollDeg`, and an explicit `attitudeMode` must agree with the populated field.

The angle-of-attack schedule always contains at least two and no more than twenty values. Its values are evaluated within each scalar case; they do not create additional top-level entries in `result.cases`.

By contrast, each retained value of `betaDeg` or `rollDeg` creates one scalar case. SpearDAT preserves the order of first occurrence and removes later exact duplicates. Distinct values that differ by an integer multiple of 360 degrees are retained because they are not exact duplicates, but SpearDAT reports that they may represent equivalent physical attitudes.

```
input.caseInput.flight.attitudeMode = "BodyAlphaBeta";
input.caseInput.flight.alphaDeg = [-4 0 4 8 12];
input.caseInput.flight.betaDeg = [-5 0 5];
```

Listing B.12: Defining body-axis angle of attack and sideslip.

The request in [listing B.12](#) produces three scalar cases. Each case contains all five angle-of-attack values and one sideslip value.

```
input.caseInput.flight.attitudeMode = "TotalAlphaRoll";
input.caseInput.flight.alphaDeg = [0 5 10 15];
input.caseInput.flight.rollDeg = [0 90 180 270];
```

Listing B.13: Defining total incidence and aerodynamic roll.

In "TotalAlphaRoll" mode, the solver resolves the total incidence into body-axis incidence components internally. The reported body-axis angle of attack and sideslip may therefore differ from the supplied total-incidence value, particularly near cardinal roll angles.

B.7.3. Primary Speed Schedule

The `input.caseInput.flight.speed` structure accepts the fields in [table B.23](#).

`input.settings.lengthSystem` selects the English or metric unit family for freestream velocity, but the flight-condition convention uses the canonical unit of that family rather than the literal geometry unit. Therefore "Inches" and "Feet" both require velocity in feet per second, while "Centimetres" and "Metres" both require velocity in metres per second. Selecting "Centimetres" does not make the velocity unit centimetres per second.

Table B.23.: Fields accepted under `input.caseInput.flight.speed`.

Field	MATLAB form	Units	Availability/default	Description
<code>input.caseInput.flight.speed.mode</code>	Text scalar; enumerated	—	Optional; inferred when unambiguous	Selects the active primary speed field. Allowed values are "Mach" and "Velocity".
<code>input.caseInput.flight.speed.mach</code>	Real scalar or vector	—	Required in "Mach" mode; 1–20 values	Mach-number schedule. Use finite, strictly positive values for an executable aerodynamic calculation.
<code>input.caseInput.flight.speed.velocity</code>	Real scalar or vector	English: ft/s; metric: m/s	Required in "Velocity" mode; 1–20 values	Freestream-velocity schedule. Values must be finite and strictly positive.

Exactly one primary speed source is permitted. When mode is specified, it is authoritative: "Mach" requires `mach` and prohibits `velocity`, while "Velocity" requires `velocity` and prohibits

mach. When mode is omitted, SpearDAT infers the mode only when exactly one of the two schedules is populated.

The speed schedule may contain a different number of values from alphaDeg. For example, three Mach numbers and five angles of attack define fifteen angle-of-attack/Mach combinations within each scalar case.

Important

The current request validator accepts an explicit Mach value of zero, but the solver execution path treats zero Mach as a speed that is too small when no freestream velocity is supplied. Use strictly positive Mach values in executable analyses.

```
input.caseInput.flight.speed.mode = "Mach";
input.caseInput.flight.speed.mach = [0.5 0.8 1.2];
```

Listing B.14: Selecting a Mach-number schedule.

```
input.caseInput.flight.speed.mode = "Velocity";
input.caseInput.flight.speed.velocity = [250 350 450];
```

Listing B.15: Selecting a freestream-velocity schedule.

B.7.4. Atmosphere and Reynolds-Number Definition

The atmosphere branch determines how the solver obtains Reynolds number and the thermodynamic state associated with each active speed point.

As with freestream velocity, `input.settings.lengthSystem` selects the unit family but the flight-condition quantities use the canonical units defined for that family. For "Inches" or "Feet", altitude is entered in feet and Reynolds number per unit length in inverse feet. For "Centimetres" or "Metres", altitude is entered in metres and Reynolds number per unit length in inverse metres. Consequently, selecting "Centimetres" does not make altitude a centimetre value or Reynolds number per unit length an inverse-centimetre value.

Table B.24.: Fields accepted under `input.caseInput.flight.atmosphere`.

Field	MATLAB form	Units	Availability/default	Description
<code>input.caseInput.flight.atmosphere.mode</code>	Text scalar; enumerated	—	Optional; inferred when unambiguous	Selects the active atmosphere definition. Exact values are listed in table B.25 .
<code>input.caseInput.flight.atmosphere.altitude</code>	Real scalar or vector	English: ft; metric: m	Optional in StandardAtmosphere mode	Geometric altitude used with the 1962 Standard Atmosphere. Omission in this mode requests the sea-level default.
<code>input.caseInput.flight.atmosphere.reynoldsPerLength</code>	Real scalar or vector	English: 1/ft; metric: 1/m	Required in ReynoldsSpecified mode	Reynolds number per unit length. Values must be finite and strictly positive.

Continued on the next page

Table B.24.: Fields accepted under `input.caseInput.flight.atmosphere` (continued).

Field	MATLAB form	Units	Availability/default	Description
<code>input.caseInput.flight.atmosphere.temperature</code>	Real scalar or vector	English: degree Rankine; metric: kelvin	Required in WindTunnel mode	Freestream static temperature. Values must be finite and strictly positive.
<code>input.caseInput.flight.atmosphere.pressure</code>	Real scalar or vector	English: lbf/ft ² ; metric: N/m ²	Required in WindTunnel mode	Freestream static pressure. Values must be finite and strictly positive.

Table B.25.: Allowed values of `input.caseInput.flight.atmosphere.mode`.

MATLAB value	Required fields	Fields that must remain empty
"StandardAtmosphere"	altitude is optional	reynoldsPerLength, temperature, and pressure.
"ReynoldsSpecified"	reynoldsPerLength	altitude, temperature, and pressure.
"WindTunnel"	Both temperature and pressure	altitude and reynoldsPerLength.

When mode is omitted, an unambiguous populated definition is accepted: altitude alone selects the standard-atmosphere path, Reynolds number per length alone selects the specified-Reynolds path, and a complete temperature/pressure pair selects the wind-tunnel path. Supplying only one member of the temperature/pressure pair is invalid, as is populating more than one atmosphere definition.

For each atmosphere field, the current validator accepts an empty vector when the selected mode permits omission, a one-element vector, or a vector having the same length as the active Mach or velocity schedule. No atmosphere vector may contain more than twenty values.

For current execution behaviour, use one atmosphere value per active speed point whenever the speed schedule contains more than one value. A one-element atmosphere vector is not duplicated into the remaining generated-deck positions; later speed points then follow the solver's ordinary missing-input fallback. To apply one constant altitude, Reynolds number, temperature, or pressure to several speed points, repeat that value explicitly.

Important

Flight-condition units are an intentional exception to the literal geometry unit selected by `lengthSystem`.

The English family uses feet, feet per second, inverse feet, degrees Rankine, and pounds-force per square foot. The metric family uses metres, metres per second, inverse metres, kelvin, and newtons per square metre.

The following examples show the three supported atmosphere modes.

```
input.caseInput.flight.speed.mode = "Mach";
input.caseInput.flight.speed.mach = [0.6 0.9 1.2];

input.caseInput.flight.atmosphere.mode = ...
    "StandardAtmosphere";
```

```
input.caseInput.flight.atmosphere.altitude = ...
    [10000 10000 10000];
```

Listing B.16: Mach schedule with a standard atmosphere.

The altitude is repeated deliberately so that all three Mach points use the same standard-atmosphere altitude.

```
input.caseInput.flight.speed.mode = "Mach";
input.caseInput.flight.speed.mach = [0.6 0.9 1.2];

input.caseInput.flight.atmosphere.mode = ...
    "ReynoldsSpecified";
input.caseInput.flight.atmosphere.reynoldsPerLength = ...
    [2.0e6 2.8e6 3.6e6];
```

Listing B.17: Supplying Reynolds number per unit length.

```
input.settings.lengthSystem = "Metres";

input.caseInput.flight.speed.mode = "Velocity";
input.caseInput.flight.speed.velocity = [100 150];

input.caseInput.flight.atmosphere.mode = "WindTunnel";
input.caseInput.flight.atmosphere.temperature = ...
    [288.15 288.15];
input.caseInput.flight.atmosphere.pressure = ...
    [101325 101325];
```

Listing B.18: Defining wind-tunnel conditions.

The temperature and pressure values are repeated so that both velocity points use the same wind-tunnel conditions.

Note

The underlying solver may sort angle-of-attack and speed-dependent data into ascending order. Scripts should therefore interpret returned rows by their reported AlphaDeg, Mach, and case metadata rather than assuming that output row order reproduces the order of the input vectors.

B.7.5. Flight-Condition Validation Summary

The most common invalid flight-condition requests are:

- supplying fewer than two or more than twenty alphaDeg values;
- supplying both betaDeg and rollDeg;
- selecting an attitude mode that disagrees with the populated attitude field;
- supplying both Mach and velocity schedules, or neither;
- using non-positive freestream velocity, Reynolds number, static temperature, or static pressure;
- combining atmosphere fields from more than one atmosphere mode; and

- supplying an atmosphere vector whose length is neither one nor the length of the active speed schedule.

SpearDAT reports these conditions before solver execution. Non-finite values such as NaN and Inf are rejected in every numeric flight field.

B.8. Reference Quantities

The `input.caseInput.reference` structure defines the quantities used to non-dimensionalise forces and moments, the point about which moments are reported, and selected surface and scaling conventions.

For these reference quantities, `input.settings.lengthSystem` is applied literally: "Inches" uses inches and square inches, "Feet" uses feet and square feet, "Centimetres" uses centimetres and square centimetres, and "Metres" uses metres and square metres. The surface-roughness field has the special unit convention stated in [section B.8.4](#).

The branch is optional and may be supplied partially. An omitted field is not replaced by an explicit zero in the generated request. Instead, the solver uses its documented default or derives the quantity from the active scaled geometry where applicable.

B.8.1. Reference Branch

Table B.26.: Fields accepted under `input.caseInput.reference`.

Field	MATLAB form	Units	Availability/ default	Description
<code>input.caseInput.reference.sref</code>	Real scalar	Selected length squared	Optional; derived from active geometry when omitted	Aerodynamic reference area. Must be finite and strictly positive when supplied.
<code>input.caseInput.reference.lref</code>	Real scalar	Selected length	Optional; derived from active geometry when omitted	Longitudinal reference length used for pitching moment. Must be finite and strictly positive when supplied.
<code>input.caseInput.reference.latref</code>	Real scalar	Selected length	Optional; defaults to the resolved longitudinal reference length	Lateral reference length used for yawing and rolling moments. Must be finite and strictly positive when supplied.
<code>input.caseInput.reference.xcg</code>	Real scalar	Selected length	Optional; default zero	Longitudinal moment-reference position measured from the global origin; positive aft. Explicit zero is valid.
<code>input.caseInput.reference.zcg</code>	Real scalar	Selected length	Optional; default zero	Vertical moment-reference position; positive upward. Explicit zero is valid.
<code>input.caseInput.reference.boundaryLayer</code>	Text scalar; enumerated	—	Optional; default "Turbulent"	Selects fully turbulent flow or natural transition. Exact values are listed in table B.28 .

Continued on the next page

Table B.26.: Fields accepted under `input.caseInput.reference` (continued).

Field	MATLAB form	Units	Availability/default	Description
<code>input.caseInput.reference.roughness</code>	Real scalar	English: in; metric: cm	Optional; default zero; advanced	Equivalent surface-roughness height. Must be finite and non-negative. Mutually exclusive with <code>roughnessHeightRating</code> .
<code>input.caseInput.reference.roughnessHeightRating</code>	Real scalar	Dimensionless rating	Optional; default zero; advanced	Roughness Height Rating. Must be finite and non-negative. Mutually exclusive with <code>roughness</code> .
<code>input.caseInput.reference.scale</code>	Real scalar	—	Optional; default 1	Vehicle scale factor. Must be finite and strictly positive. Scaling behaviour is described in section B.8.5 .

B.8.2. Coefficient and Moment Conventions

Table B.27.: Principal uses of the reference quantities.

Quantity	Principal use
<code>sref</code>	Reference area used to non-dimensionalise aerodynamic force coefficients.
<code>lref</code>	Longitudinal reference length used with the pitching-moment coefficient.
<code>latref</code>	Lateral reference length used with yawing- and rolling-moment coefficients.
<code>xcg, zcg</code>	Coordinates of the point about which aerodynamic moments are reported.

Reference quantities are part of the coefficient convention. Results produced with different values of `sref`, `lref`, `latref`, `xcg`, or `zcg` are not directly comparable until they have been transformed to a common reference convention.

B.8.3. Omission, Defaults, and Derived Values

When the complete reference branch is omitted, the solver uses the following baseline behaviour:

- for a configuration containing a body, `sref` is derived from the maximum body cross-sectional area and `lref` from the maximum body diameter;
- for a valid fin-only configuration, `sref` is derived from the maximum fin-panel area and `lref` from the fin-panel mean geometric chord;
- `latref` defaults to the resolved `lref`;
- `xcg` and `zcg` default to zero;
- `boundaryLayer` defaults to fully turbulent flow;
- surface roughness defaults to zero; and

- scale defaults to one.

Missile DATCOM permits configurations that contain lifting surfaces without a body. The fin-only fallbacks above are therefore defined behaviour rather than an error case, and SpearDAT preserves the same reference defaults.

The same omission semantics apply field by field. A request may therefore override only the quantities needed for a particular coefficient convention:

```
input.caseInput.reference.xcg = 3.5;
input.caseInput.reference.zcg = 0;
```

Listing B.19: Overriding only the moment-reference position.

For `xcg` and `zcg`, omission and an explicit zero currently produce the same numerical moment-reference location because the solver default is zero. Their traceability is different: an omitted field is not written into the generated reference block, whereas an explicit zero is written and records that the origin was selected intentionally. Use explicit zero when that location is part of the defined test condition.

For `sref`, `lref`, and `latref`, zero is invalid; omit the field to request derivation.

B.8.4. Boundary Layer and Surface Roughness

Table B.28.: Allowed values of `input.caseInput.reference.boundaryLayer`.

MATLAB value	Selected treatment
"Turbulent"	Fully turbulent boundary layer; public and solver default.
"Natural"	Natural-transition boundary-layer treatment.

Table B.29.: Alternative surface-roughness inputs.

Field	Units	Default	Purpose
<code>input.caseInput.reference.roughness</code>	English family: in; metric family: cm	0	Equivalent surface-roughness height. Use this when a physical roughness height is known.
<code>input.caseInput.reference.roughnessHeightRating</code>	Dimensionless historical rating	0	Roughness Height Rating representation. Use this instead of roughness when the surface is specified by its historical rating.

`roughness` and `roughnessHeightRating` are alternative descriptions and must not be supplied together. Surface roughness is useful when the vehicle finish is not aerodynamically smooth: the selected roughness enters the skin-friction treatment and can therefore affect viscous drag and axial-force predictions.

The physical roughness field is a deliberate exception to the literal `lengthSystem` unit. It uses inches for either English selection ("Inches" or "Feet") and centimetres for either metric selection ("Centimetres" or "Metres"). `roughnessHeightRating` is the historical rating representation.

A value of zero is valid for either roughness representation and requests the smooth-surface baseline. Negative values are rejected.

B.8.5. Vehicle Scale Factor

scale changes the physical size represented by dimensional inputs. The implemented solver applies the following factors before converting the request to its internal unit system:

Supplied quantity	Applied factor
Geometry lengths	scale.
sref	scale ² .
lref, latref, xcg, and zcg	scale.

Important

The scale field is intended to scale an otherwise unscaled geometry and its dimensional reference quantities. If the dimensions and areas have already been multiplied by the desired scale factor, leave scale equal to 1; supplying the factor again would scale them a second time.

For example, sref=10 with scale=2 resolves to a reference area of 40 in the selected area units before internal unit conversion.

B.8.6. Reference-Quantity Example

The following example defines a complete inch-based reference convention:

```
input.settings.lengthSystem = "Inches";

input.caseInput.reference.sref = 90.763;
input.caseInput.reference.lref = 10.75;
input.caseInput.reference.latref = 10.75;
input.caseInput.reference.xcg = 77.291;
input.caseInput.reference.zcg = 0;

input.caseInput.reference.boundaryLayer = "Turbulent";
input.caseInput.reference.roughness = 0.00016;
input.caseInput.reference.scale = 1;
```

Listing B.20: Defining reference quantities explicitly.

Because lengthSystem is "Inches", sref is interpreted in square inches and lref, latref, xcg, and zcg in inches. The physical roughness value is also in inches because an English unit family is active.

Because roughness is supplied, the alternative roughnessHeightRating field must remain unset.

All numeric reference fields must be finite. SpearDAT rejects non-positive reference area, reference lengths, or scale; it rejects negative roughness values; and it rejects simultaneous roughness and roughnessHeightRating definitions.

B.9. Body, Base-Jet, and Protuberance Definition

The `input.caseInput.body` branch defines an axisymmetric or elliptical body and, where applicable, its nozzle-exit, base-jet, and non-lifting protuberance data.

All dimensional fields in this section use `input.settings.lengthSystem` literally. Thus "Inches" means inches, "Feet" means feet, "Centimetres" means centimetres, and "Metres" means metres. This differs from the canonical metre-or-foot convention used by the flight-condition quantities.

A non-unity `input.caseInput.reference.scale` is a separate vehicle scaling instruction. Enter the body and protuberance dimensions in the selected length system and do not apply the same scale factor manually and through `scale`; the general double-scaling warning is given in [section B.8.5](#).

Important

One request may contain at most one body family. Set `input.caseInput.body.shape` to either "Axisymmetric" or "Elliptical", populate only the corresponding geometry branch, and leave the inactive branch unset.

SpearDAT can infer the body family when exactly one branch is populated, but an explicit shape is clearer and prevents an otherwise ambiguous request when both branches contain data.

B.9.1. Body Branch and Shape Selection

Table B.30.: Fields accepted under `input.caseInput.body`.

Field	MATLAB form	Availability/default	Description
<code>input.caseInput.body.shape</code>	String scalar or character row	Optional; recommended explicitly	Selects the active body family. Allowed values are "Axisymmetric" and "Elliptical".
<code>input.caseInput.body.axisymmetric</code>	Scalar structure	Active only for an axisymmetric body	Analytic or station-defined circular cross-section body geometry. See section B.9.2 .
<code>input.caseInput.body.elliptical</code>	Scalar structure	Active only for an elliptical body	Analytic or station-defined non-circular body geometry. See section B.9.3 .
<code>input.caseInput.body.baseJet</code>	Scalar structure	Optional; axisymmetric bodies only	Base-jet plume-interaction inputs. See section B.9.5 .
<code>input.caseInput.body.protuberances</code>	Scalar structure	Optional; inactive when empty	Non-lifting body-mounted protuberance definitions. See section B.9.6 .

B.9.2. Axisymmetric Bodies

An axisymmetric body has circular cross-sections normal to its longitudinal axis. It may be described by standard analytic nose, centrebody, and afterbody sections, or directly by longitudinal stations and radii.

Axisymmetric Root Fields

Table B.31.: Fields accepted under `input.caseInput.body.axisymmetric`.

Field	Units/form	Availability/default	Description
<code>input.caseInput.body.axisymmetric.geometryMode</code>	Text enum	Optional; inferred when omitted	Selects "Analytic" or "Stations". An explicit value is recommended.
<code>input.caseInput.body.axisymmetric.xOrigin</code>	Length	Optional; historical default 0	Longitudinal coordinate of the body nose tip in the global geometry coordinate system.
<code>input.caseInput.body.axisymmetric.stations</code>	Scalar structure	Active in station mode	Longitudinal coordinates, radii, and optional slope-discontinuity indices.
<code>input.caseInput.body.axisymmetric.nose</code>	Scalar structure	Analytic geometry or station-mode section metadata	Nose geometry in analytic mode. In station mode, selected fields define the nose boundary and section classification while the station mold line remains authoritative.
<code>input.caseInput.body.axisymmetric.center</code>	Scalar structure	Analytic geometry or station-mode section metadata	Centreboddy geometry in analytic mode. In station mode, length identifies the centreboddy extent within the supplied mold line.
<code>input.caseInput.body.axisymmetric.aft</code>	Scalar structure	Analytic geometry or station-mode section metadata	Afterbody geometry in analytic mode. In station mode, the type classifies the interpreted aft section and the supplied length is consistency metadata.
<code>input.caseInput.body.axisymmetric.exitDiameter</code>	Length	Optional; omission-significant	Nozzle or exit diameter used by the base-drag calculation. See section B.9.4 .

If `geometryMode` is omitted, populated station-coordinate arrays select station mode; otherwise populated analytic geometry selects analytic mode. Explicit selection remains preferable because it makes the intended interpretation visible in the MATLAB script.

Axisymmetric Station Geometry

Station geometry directly defines the body mold line. The number of active stations is generated from the length of the supplied coordinate vector.

Table B.32.: Axisymmetric station fields.

Field	Units/form	Requirement	Description
<code>input.caseInput.body.axisymmetric.stations.x</code>	Length vector	Required; 3–49 values	Longitudinal station coordinates. The final coordinate is the end of the body. Use strictly increasing values in physical front-to-back order.
<code>input.caseInput.body.axisymmetric.stations.radius</code>	Length vector	Required; same length as x	Non-negative body radius at each station.
<code>input.caseInput.body.axisymmetric.stations.discontinuityStations</code>	Integer-valued numeric vector	Optional; at most 20 values	One-based indices of stations at which the surface slope is discontinuous. Indices must be unique and lie within the active station range.

SpearDAT preserves station order and does not sort the mold line. A non-increasing x vector therefore produces a warning rather than being silently rearranged.

```
input.caseInput.body.shape = "Axisymmetric";
input.caseInput.body.axisymmetric.geometryMode = "Stations";
input.caseInput.body.axisymmetric.xOrigin = 0;

input.caseInput.body.axisymmetric.stations.x = ...
    [0 8 24 60];
input.caseInput.body.axisymmetric.stations.radius = ...
    [0 4 4 2];

% Mark the shoulder and start of the afterbody.
input.caseInput.body.axisymmetric.stations.discontinuityStations = ...
    [2 3];
```

Listing B.21: Defining an axisymmetric body by stations.

The values in `discontinuityStations` are station numbers, not longitudinal distances. In the example, the entries 2 and 3 refer to `x(2)` and `x(3)`.

Optional section metadata in station mode. Station mode may also use selected fields from the nose, center, and aft branches. These fields do not replace or regenerate the physical mold line: `stations.x` and `stations.radius` remain the authoritative geometry.

The optional metadata has the following interpretation:

- `nose.length` is the physical nose length measured from `xOrigin`;
- `center.length` is the centrebody length measured after the nose, so the centrebody ends at `xOrigin+nose.length+center.length`;
- the resolved aft length is the remaining distance to the final body station; a supplied `aft.length` is checked as consistency metadata rather than used to reshape the body;
- `nose.type` and `aft.type` classify the interpreted station-defined sections for downstream aerodynamic methods; they do not generate an analytic nose or aft-body mold line; and

- nose, centrebody, and aft diameters are obtained from the interpreted station table at the resolved section boundaries.

When a requested nose or centrebody boundary lies between two supplied stations, SpearDAT inserts an interpolated station at that boundary. Existing discontinuityStations indices are shifted as necessary so that they continue to refer to the same original physical stations.

```
input.caseInput.body.shape = "Axisymmetric";
input.caseInput.body.axisymmetric.geometryMode = "Stations";
input.caseInput.body.axisymmetric.xOrigin = 0;

input.caseInput.body.axisymmetric.stations.x = ...
    [0 12 30 60 76];
input.caseInput.body.axisymmetric.stations.radius = ...
    [0 4 4 3 2];

% The station arrays still define the physical mold line.
input.caseInput.body.axisymmetric.nose.type = "Ogive";
input.caseInput.body.axisymmetric.nose.length = 18;

input.caseInput.body.axisymmetric.center.length = 36;

input.caseInput.body.axisymmetric.aft.type = "Conical";
input.caseInput.body.axisymmetric.aft.length = 22;
```

Listing B.22: Adding section metadata to an axisymmetric station-defined body.

Here the requested nose boundary is at $x = 18$, the centrebody boundary at $x = 54$, and the final body station at $x = 76$. SpearDAT interpolates mold-line stations at 18 and 54 because those boundaries are not present in the original coordinate vector. The resulting aft length is $76 - 18 - 36 = 22$, which is consistent with the supplied aft.length. The "Ogive" and "Conical" values classify the two sections and may affect downstream aerodynamic method selection, but they do not alter the supplied radius profile.

Axisymmetric Analytic Geometry

Analytic mode constructs the body from named nose, centrebody, and afterbody sections. At least one analytic geometry field must be populated, and the station arrays must remain unset.

Table B.33.: Axisymmetric analytic-body fields.

Field	Units/form	Availability/ default	Description
input.caseInput.body.axisymmetric.nose.type	Text enum	Optional; historical default "Ogive"	Nose profile. Allowed values are listed in table B.34 .
input.caseInput.body.axisymmetric.nose.length	Length	Supply for an active nose	Nose axial length.
input.caseInput.body.axisymmetric.nose.diameter	Length	Optional; historical default 1	Nose diameter at its base.
input.caseInput.body.axisymmetric.nose.bluntness	Length	Optional; default 0	Spherical bluntness radius, or truncation radius when truncated is true.

Field	Units/form	Availability/default	Description
input.caseInput.body.axisymmetric.nose.truncated	Logical scalar	Optional; default false	Indicates that the nose is truncated rather than smoothly blunted.
input.caseInput.body.axisymmetric.nose.powerExponent	Dimensionless scalar	Required only for "Power"	Positive exponent in the power-law nose definition.
input.caseInput.body.axisymmetric.center.length	Length	Optional; historical default 0	Centrebody axial length.
input.caseInput.body.axisymmetric.center.diameter	Length	Optional; historically inherited from nose diameter	Centrebody diameter.
input.caseInput.body.axisymmetric.aft.type	Text enum	Optional; historical default "Conical"	Afterbody profile: "Conical" or "Ogive".
input.caseInput.body.axisymmetric.aft.length	Length	Optional; historical default 0	Afterbody axial length.
input.caseInput.body.axisymmetric.aft.diameter	Length	Supply for an active afterbody	Terminal body diameter. Use a positive value different from the upstream centrebody diameter for a genuine boattail.

All analytic dimensions must be finite and non-negative. Zero is accepted for sections intentionally having zero length or size, but a complete intended section should be defined explicitly rather than relying accidentally on a historical default.

Table B.34.: Allowed analytic nose types for axisymmetric and elliptical bodies.

Value	Meaning
"Conical"	Straight-sided conical nose.
"Ogive"	Tangent-ogive nose.
"Power"	Power-law nose; requires a positive powerExponent.
"Haack"	Haack-series volume-constrained nose.
"Karman"	Von Karman length-diameter-constrained nose.

The "Ogive" selector denotes the tangent-ogive analytic primitive: its generating circular arc is tangent to the adjoining cylindrical section at the nose shoulder. A secant ogive uses a different circular arc that is not tangent at that junction. SpearDAT does not expose a separate secant-ogive analytic selector; define an exact secant-ogive mold line with station coordinates. A power-law profile may be used only as an approximation, not as an equivalent definition.

```
input.caseInput.body.shape = "Axisymmetric";
input.caseInput.body.axisymmetric.geometryMode = "Analytic";
input.caseInput.body.axisymmetric.xOrigin = 0;

input.caseInput.body.axisymmetric.nose.type = "Ogive";
input.caseInput.body.axisymmetric.nose.length = 18;
input.caseInput.body.axisymmetric.nose.diameter = 8;
input.caseInput.body.axisymmetric.nose.bluntness = 0;
input.caseInput.body.axisymmetric.nose.truncated = false;

input.caseInput.body.axisymmetric.center.length = 30;
input.caseInput.body.axisymmetric.center.diameter = 8;
```

```

input.caseInput.body.axisymmetric.aft.type = "Conical";
input.caseInput.body.axisymmetric.aft.length = 12;
input.caseInput.body.axisymmetric.aft.diameter = 4;

input.caseInput.body.axisymmetric.exitDiameter = 2;

```

Listing B.23: Defining an analytic axisymmetric body.

B.9.3. Elliptical Bodies

An elliptical body has elliptical cross-sections whose height-to-width ratio may vary along the longitudinal axis. As for an axisymmetric body, the geometry may be analytic or station-defined.

Elliptical Root Fields

Table B.35.: Fields accepted under input.caseInput.body.elliptical.

Field	Units/form	Availability/default	Description
input.caseInput.body.elliptical.geometryMode	Text enum	Optional; inferred when omitted	Selects "Analytic" or "Stations".
input.caseInput.body.elliptical.xOrigin	Length	Optional; historical default 0	Longitudinal coordinate of the body nose tip.
input.caseInput.body.elliptical.stations	Scalar structure	Active in station mode	Longitudinal coordinates and elliptical cross-section dimensions.
input.caseInput.body.elliptical.nose	Scalar structure	Analytic geometry or station-mode section metadata	Analytic nose definition, or station-mode nose boundary and classification metadata.
input.caseInput.body.elliptical.center	Scalar structure	Analytic geometry or station-mode section metadata	Analytic centrebody definition, or station-mode centrebody-length metadata.
input.caseInput.body.elliptical.aft	Scalar structure	Analytic geometry or station-mode section metadata	Analytic aft-body definition, or station-mode aft classification and length-consistency metadata.
input.caseInput.body.elliptical.exitDiameter	Length	Optional; omission-significant	Nozzle or exit diameter used by the base-drag calculation. See section B.9.4 .

Elliptical Station Geometry

At every station, the cross-section can be defined by any two of its half-width, half-height, and ellipticity. The exact MATLAB station-field names are halfWidth, height, and ellipticity. There is no public halfHeight field.

Despite its shorter name, height maps to the legacy H input and must contain the body *half-height* at each station. Ellipticity is the ratio of half-height to half-width, which is the same as the full-height to full-width ratio.

Table B.36.: Elliptical station fields.

Field	Units/form	Requirement	Description
<code>input.caseInput.body.elliptical.stations.x</code>	Length vector	Required; 3–49 values	Longitudinal station coordinates. The final coordinate is the end of the body. Use strictly increasing values.
<code>input.caseInput.body.elliptical.stations.halfWidth</code>	Length vector	Conditional	Non-negative body half-width at each station.
<code>input.caseInput.body.elliptical.stations.height</code>	Length vector	Conditional	Non-negative body half-height at each station. This is the accepted public field name; <code>halfHeight</code> is not accepted.
<code>input.caseInput.body.elliptical.stations.ellipticity</code>	Dimensionless vector	Conditional	Positive height-to-width ratio at each station.
<code>input.caseInput.body.elliptical.stations.discontinuityStations</code>	Integer-valued numeric vector	Optional; at most 20 values	Unique one-based indices of stations at which the surface slope is discontinuous.

The populated dimensional or ellipticity vectors must match the length of `x`. Supply one of these three complete combinations:

1. `halfWidth` and `height`;
2. `halfWidth` and `ellipticity`; or
3. `height` and `ellipticity`.

Important

The MATLAB interface accepts half-width and full-width concepts in different geometry branches; they are not interchangeable inputs within station mode.

In "Stations" mode:

- `stations.halfWidth` is the body half-width;
- `stations.height` is the body half-height; and
- full-width station fields such as `stations.totalWidth` are not accepted.

In "Analytic" mode, `nose.totalWidth`, `center.width`, and `aft.width` are full widths. The corresponding heights are derived from the supplied ellipticities.

```
input.caseInput.body.shape = "Elliptical";
input.caseInput.body.elliptical.geometryMode = "Stations";
input.caseInput.body.elliptical.x0Origin = 0;

input.caseInput.body.elliptical.stations.x = ...
    [0 12 30 55];
input.caseInput.body.elliptical.stations.halfWidth = ...
    [0 4 4 2];
input.caseInput.body.elliptical.stations.height = ...
    [0 3 5 2];
```

```
input.caseInput.body.elliptical.stations.discontinuityStations = ...
    [2 3];
```

Listing B.24: Defining an elliptical body by stations.

Because both half-dimension vectors are supplied in this example, SpearDAT does not require a separate ellipticity vector. At the second station, halfWidth=4 and height=3 describe a full cross-section width of 8 and a full cross-section height of 6.

Optional section metadata in station mode. The nose, center, and aft branches may also provide section metadata while stations.x, stations.halfWidth, stations.height, and stations.ellipticity remain the physical mold-line definition.

The station-mode interpretation is the same as for an axisymmetric body:

- nose.length defines the end of the nose from xOrigin;
- center.length defines the centrebody length after the nose;
- aft.length is consistency metadata for the remaining body length;
- nose.type and aft.type classify the interpreted sections and can affect downstream aerodynamic methods and coefficients; and
- analytic width and ellipticity fields such as nose.totalWidth, center.width, and aft.width do not replace the station geometry in "Stations" mode.

SpearDAT inserts an interpolated cross-section when a requested nose or centrebody boundary falls between supplied stations. The interpolated half-width, half-height, and ellipticity are taken from the neighbouring station-defined geometry.

The preceding station example may therefore be enriched as follows:

```
input.caseInput.body.elliptical.nose.type = "Ogive";
input.caseInput.body.elliptical.nose.length = 16;

input.caseInput.body.elliptical.center.length = 25;

input.caseInput.body.elliptical.aft.type = "Conical";
input.caseInput.body.elliptical.aft.length = 14;
```

Listing B.25: Adding section metadata to an elliptical station-defined body.

For the body ending at $x = 55$, these values identify a nose from 0 to 16, a centrebody from 16 to 41, and an aft section from 41 to 55. Boundaries at 16 and 41 are interpolated because neither is present in the original station vector. The "Ogive" and "Conical" values classify those station-defined sections; they do not generate analytic widths or heights.

Elliptical Analytic Geometry

Table B.37.: Elliptical analytic-body fields.

Field	Units/form	Availability/ default	Description
<code>input.caseInput.body.elliptical.nose.type</code>	Text enum	Optional; historical default "Ogive"	Nose profile. Uses the values in table B.34 .
<code>input.caseInput.body.elliptical.nose.length</code>	Length	Supply for an active nose	Nose axial length.
<code>input.caseInput.body.elliptical.nose.totalWidth</code>	Length	Optional; historical default 1	Full nose width at its base.
<code>input.caseInput.body.elliptical.nose.ellipticity</code>	Dimensionless scalar	Optional; historical default 1	Positive height-to-width ratio at the nose base.
<code>input.caseInput.body.elliptical.nose.bluntness</code>	Length	Optional; default 0	Nose bluntness or truncation radius.
<code>input.caseInput.body.elliptical.nose.truncated</code>	Logical scalar	Optional; default false	Indicates a truncated nose.
<code>input.caseInput.body.elliptical.nose.powerExponent</code>	Dimensionless scalar	Required only for "Power"	Positive exponent in the power-law nose definition.
<code>input.caseInput.body.elliptical.center.length</code>	Length	Optional; historical default 0	Centrebody axial length.
<code>input.caseInput.body.elliptical.center.width</code>	Length	Optional; historically inherited from nose width	Full centrebody width at its base.
<code>input.caseInput.body.elliptical.center.ellipticity</code>	Dimensionless scalar	Optional; historical default 1	Positive height-to-width ratio at the centrebody base.
<code>input.caseInput.body.elliptical.aft.type</code>	Text enum	Optional; historical default "Conical"	Afterbody profile: "Conical" or "Ogive".
<code>input.caseInput.body.elliptical.aft.length</code>	Length	Optional; historical default 0	Afterbody axial length.
<code>input.caseInput.body.elliptical.aft.width</code>	Length	Supply for an active afterbody	Full terminal body width.
<code>input.caseInput.body.elliptical.aft.ellipticity</code>	Dimensionless scalar	Optional; historical default 1	Positive height-to-width ratio at the afterbody base.

Analytic lengths, widths, and bluntness values must be finite and non-negative. Every populated ellipticity must be finite and strictly positive.

```
input.caseInput.body.shape = "Elliptical";
input.caseInput.body.elliptical.geometryMode = "Analytic";
input.caseInput.body.elliptical.xOrigin = 0;

input.caseInput.body.elliptical.nose.type = "Ogive";
input.caseInput.body.elliptical.nose.length = 18;
input.caseInput.body.elliptical.nose.totalWidth = 8;
input.caseInput.body.elliptical.nose.ellipticity = 0.75;

input.caseInput.body.elliptical.center.length = 30;
input.caseInput.body.elliptical.center.width = 8;
input.caseInput.body.elliptical.center.ellipticity = 1.10;

input.caseInput.body.elliptical.aft.type = "Conical";
input.caseInput.body.elliptical.aft.length = 12;
input.caseInput.body.elliptical.aft.width = 4;
input.caseInput.body.elliptical.aft.ellipticity = 0.80;
```



```
input.caseInput.body.elliptical.exitDiameter = 2;
```

Listing B.26: Defining an analytic elliptical body.

B.9.4. Nozzle Exit Diameter and Base Drag

Both body families expose an `exitDiameter` field. It is not a general terminal-body diameter: the analytic aft-body diameter or width defines the external body geometry, while `exitDiameter` defines an internal nozzle or exit opening for the base-drag calculation.

Table B.38.: Omission-sensitive `exitDiameter` behaviour.

MATLAB state	Base-drag interpretation	Meaning
Field omitted or []	No body base-drag increment	The exit-diameter input is not emitted. This is not equivalent to zero.
Explicit 0	Full closed-base drag	The full external base area contributes to the base-drag calculation.
Positive value	Annular-base drag	Only the base area outside the specified nozzle or exit contributes.

Important

Do not replace an omitted `exitDiameter` with zero as a convenient default. Omission suppresses the body base-drag increment, whereas explicit zero requests full closed-base drag.

A positive exit diameter should also be physically compatible with the body base dimensions. An opening larger than the available base is not a meaningful vehicle definition even when a permissive numerical path could process it.

B.9.5. Base-Jet Plume Interaction

The base-jet branch activates the specialised plume-interaction treatment for an axisymmetric body. It is an advanced option and requires a physically compatible axisymmetric aft-body and nozzle definition.

Table B.39.: Fields accepted under `input.caseInput.body.baseJet`.

Field	Units/form	Availability/default	Description
<code>input.caseInput.body.baseJet.enabled</code>	Logical scalar	Optional; default inactive	Set to true to activate base-jet plume inputs. Plume fields are rejected when this is false or unset.
<code>input.caseInput.body.baseJet.nozzleExitHalfAngle</code>	Degrees	Conditional	Non-negative nozzle exit half-angle. The historical plume correlation is documented over 5–25 degrees and extrapolates outside that range.

Field	Units/form	Availability/ default		Description
<code>input.caseInput.body.baseJet.jetMach</code>	Dimensionless vector	Conditional; most 20 values	at	Jet Mach number at the nozzle exit.
<code>input.caseInput.body.baseJet. pressureRatio</code>	Dimensionless vector	Conditional; most 20 values	at	Jet-to-freestream static- pressure ratio; every value must be positive.
<code>input.caseInput.body.baseJet. temperatureRatio</code>	Dimensionless vector	Conditional; most 20 values	at	Jet-to-freestream stagnation- temperature ratio; every value must be positive.

The vector fields are schedules associated with the active freestream speed conditions. A one-element vector is the compact constant-schedule form. A multi-value schedule must be compatible with the active speed count, and the three populated plume schedules must be mutually compatible. SpearDAT preserves their supplied order.

Note

The base-jet branch is available only when `input.caseInput.body.shape="Axisymmetric"`. It must not be populated for an elliptical body. Base-jet inputs also do not replace `exitDiameter`; define the nozzle opening explicitly when the plume and annular-base geometry require it.

```
input.caseInput.body.baseJet.enabled = true;
input.caseInput.body.baseJet.nozzleExitHalfAngle = 10;

input.caseInput.body.baseJet.jetMach = ...
    [2.0 2.1 2.2];
input.caseInput.body.baseJet.pressureRatio = ...
    [1.20 1.30 1.40];
input.caseInput.body.baseJet.temperatureRatio = ...
    [1.10 1.20 1.30];
```

Listing B.27: Defining a three-condition base-jet schedule.

This example assumes that the flight branch also contains three active freestream speed conditions in the corresponding order.

B.9.6. Protuberances

Protuberances represent non-lifting external items attached to a body, such as cylinders, blocks, fairings, launch lugs, and launch shoes. Each entry in `items` defines one protuberance set at one longitudinal station. The set may contain several identical circumferential copies.

Protuberance Collection

For a clear and self-documenting request, set `enabled` to `true` whenever `items` is populated.

Table B.40.: Fields accepted under `input.caseInput.body.protuberances`.

Field	MATLAB form	Availability/ default	Description
<code>input.caseInput.body.protuberances.enabled</code>	Logical scalar	Optional; default false	Explicitly activates protuberance processing. Populated items also activate the branch, but produce a warning when enabled remains false.
<code>input.caseInput.body.protuberances.items</code>	Structure row or column vector	Required when active; 1–20 items	Ordered protuberance-set definitions.

Protuberance Items

Table B.41.: Fields accepted in each protuberance item.

Field	Units/form	Requirement	Description
<code>input.caseInput.body.protuberances.items(i).index</code>	Integer-valued scalar	Optional	Explicit protuberance-set number. If every item omits this field, <code>items(1)</code> , <code>items(2)</code> , and so on become sets 1, 2, and so on in MATLAB vector order. If one item supplies an index, every item must supply one; the values must be unique and form the contiguous sequence 1 through the number of items. SpearDAT orders the generated sets by this index.
<code>input.caseInput.body.protuberances.items(i).type</code>	Text enum	Required	Stock protuberance type. Allowed values and member counts are listed in table B.42 .
<code>input.caseInput.body.protuberances.items(i).x</code>	Length	Required	Axial coordinate of the approximate geometric centroid of the complete protuberance item, measured from the missile nose. The local body mold line at this axial position is the zero-offset reference for its members. It is not the leading edge or forward end of a member.

Field	Units/form	Requirement	Description
<code>input.caseInput.body.protuberances.items(i).countAroundBody</code>	Integer-valued scalar	Required; non-negative	Number of identical copies of the <i>complete item</i> around the body at the same axial station. This is a multiplicity count, not the number of geometric members and not a list of user-defined circumferential angles. Use a positive value for an active physical set.
<code>input.caseInput.body.protuberances.items(i).members</code>	Structure vector	Required	Geometry entries used to construct one copy of the selected stock type. "Lug" requires exactly four members and "Shoe" exactly three; the other stock types require one. These members do not represent separate circumferential locations.

When indices are omitted, the structure-vector position determines the set number. For example:

```

blockMember = struct;
blockMember.length = 2.0;
blockMember.width = 1.0;
blockMember.height = 0.5;
blockMember.verticalOffset = 0;

fairingMember = struct;
fairingMember.length = 5.0;
fairingMember.width = 1.5;
fairingMember.height = 0.75;
fairingMember.verticalOffset = 0;

items(1).type = "Block";
items(1).x = 40;
items(1).countAroundBody = 4;
items(1).members = blockMember;

items(2).type = "Fairing";
items(2).x = 70;
items(2).countAroundBody = 1;
items(2).members = fairingMember;

input.caseInput.body.protuberances.enabled = true;
input.caseInput.body.protuberances.items = items;

```

Listing B.28: Defining two protuberance sets in vector order.

Because neither element contains index, `items(1)` becomes protuberance set 1 and `items(2)` becomes set 2. Set 1 describes four identical copies of one block item at the axial station $x = 40$. Set 2 describes one fairing item at $x = 70$.

The value four in `items(1).countAroundBody` does not create four entries in `items(1).members`. It tells the legacy protuberance model that the complete one-member block item occurs four times around the body. The MATLAB interface does not accept a separate roll angle for each copy; `countAroundBody` is the legacy multiplicity input rather than an explicit circumferential-placement schedule.

Explicit indices are useful when the MATLAB vector is assembled in an order that should not determine the generated set numbers. For example, assigning `items(1).index=2` and `items(2).index=1` would cause SpearDAT to generate the fairing as set 1 and the block set as set 2. Partial indexing is not permitted.

Table B.42.: Protuberance types and geometry entries required for one item.

Value	Members per item	Physical interpretation
"VerticalCylinder"	1	One cylinder perpendicular to the freestream direction.
"HorizontalCylinder"	1	One cylinder aligned with the freestream direction.
"Block"	1	One rectangular block.
"Fairing"	1	One streamlined half-body fairing.
"Lug"	4	One launch-lug approximation constructed from four sequential flat/block-like member contributions.
"Shoe"	3	One launch-shoe approximation constructed from three sequential flat/block-like member contributions.

The member count in this table is mandatory for the selected stock type. A "Shoe" must contain three member structures and a "Lug" four; the remaining stock types must contain one. SpearDAT checks the structure-vector length before execution. For example, supplying one or two members for a "Shoe" produces an input-validation error and the solver is not run.

The member count belongs to one protuberance item; it is not a limit on the number of copies around the body. Setting `countAroundBody=4` for a "Shoe" requests four identical copies of the complete three-member shoe at the same axial station. The typed MATLAB hierarchy hides the legacy flat-array bookkeeping, but the underlying aerodynamic method still sums the three or four constituent member contributions for each composite shoe or lug.

The 100-entry limit applies to the flattened member-geometry arrays for the complete request:

$$N_{\text{members}} = \sum_{i=1}^{N_{\text{items}}} \text{numel}(\text{items}(i).\text{members}).$$

It does not count the physical copies represented by `countAroundBody`. Thus one shoe item contributes three entries to this total whether `countAroundBody` is 1, 4, or another accepted value.

A separate limit permits at most 20 protuberance items. Consequently, with the currently supported stock types, a valid request cannot ordinarily reach the 100-member-array limit: 20 four-member lugs would use 80 entries, while 20 three-member shoes would use 60. The

100-entry check preserves the capacity of the underlying legacy arrays and protects malformed or future extended definitions; it does not mean that 100 independent shoe items may be defined.

Protuberance Members

Table B.43.: Fields accepted in each protuberance member.

Field	Units	Requirement	Description
<code>input.caseInput.body.protuberances.items(i).members(j).length</code>	Length	Required	Member length.
<code>input.caseInput.body.protuberances.items(i).members(j).width</code>	Length	Required	Member width.
<code>input.caseInput.body.protuberances.items(i).members(j).height</code>	Length	Required	Member height.
<code>input.caseInput.body.protuberances.items(i).members(j).verticalOffset</code>	Length	Optional; effective default 0	Outward perpendicular distance from the local body mold line at <code>item.x</code> to the bottom surface of this member. Zero places the member bottom on the mold line; a positive value represents physical stand-off or clearance. It is not an axial coordinate, circumferential angle, or member orientation.

Local mold-line reference and vertical offset. `item.x` first selects the axial station of the complete protuberance item. The local body mold line at that station is then the zero-offset reference. Conceptually, if the local body radius is $R(x)$, the bottom of a member with `verticalOffset=d` is represented at an outward distance $R(x) + d$ from the body centreline in the radial direction of that copy.

This is a correlation input rather than a three-dimensional CAD placement: SpearDAT does not create a mesh, calculate mounting brackets, or check whether the resulting member intersects another component. The practical interpretation is:

- `verticalOffset=0` places the bottom of the member on the local mold line;
- a positive value represents a real stand-off, spacer, gap, or another member supporting it above the body surface; and
- a negative value would place the member bottom inside the body mold line and is normally not physically meaningful.

It is therefore possible to represent a member whose bottom is separated from the body, but the offset should correspond to a physical installation rather than being used to create an arbitrary “floating” protuberance. For a composite “Lug” or “Shoe”, each constituent member has its own offset, allowing the flat/block-like contributions to be placed at different stand-off distances while sharing the same item-level axial station.

Member dimensions and offsets must be finite. The interface deliberately retains finite engineering values rather than imposing an arbitrary plausibility envelope, so the user remains responsible for supplying positive, physically meaningful dimensions and offsets.

A “Fairing” is treated as attached directly to the mold line and therefore always uses zero offset. A non-zero verticalOffset for “Fairing” produces a warning because the aerodynamic treatment substitutes zero.

```
shoeMembers(1).length = 4.0;
shoeMembers(1).width = 1.5;
shoeMembers(1).height = 0.25;
shoeMembers(1).verticalOffset = 0.0;

shoeMembers(2).length = 2.0;
shoeMembers(2).width = 0.4;
shoeMembers(2).height = 0.75;
shoeMembers(2).verticalOffset = 0.25;

shoeMembers(3).length = 4.0;
shoeMembers(3).width = 1.5;
shoeMembers(3).height = 0.25;
shoeMembers(3).verticalOffset = 1.00;

shoe = struct;
shoe.type = "Shoe";
shoe.x = 60;
shoe.countAroundBody = 4;
shoe.members = shoeMembers;

input.caseInput.body.protuberances.enabled = true;
input.caseInput.body.protuberances.items = shoe;
```

Listing B.29: Defining four copies of one three-member launch shoe.

This request contains one protuberance item at $x = 60$. Because its type is “Shoe”, the three supplied member structures form one DATCOM launch-shoe approximation. Supplying any other member count would fail SpearDAT input validation. The offsets place the bottoms of the three member contributions at 0, 0.25, and 1.00 selected length units outward from the local body mold line.

The separate value countAroundBody=4 requests four identical copies of that complete three-member shoe around the body. It does not mean that the shoe may contain up to four members, and it does not assign three or four separate axial positions. All three members share the item-level $x=60$; all four copies reuse the same three-member geometry. No individual circumferential angles are supplied through this interface.

```
member = struct;
member.length = 2.0;
member.width = 1.0;
member.height = 0.5;
member.verticalOffset = 0;
```

```

item = struct;
item.type = "Block";
item.x = 40;
item.countAroundBody = 4;
item.members = member;

input.caseInput.body.protuberances.enabled = true;
input.caseInput.body.protuberances.items = item;

```

Listing B.30: Defining four identical block protuberances.

The example defines one protuberance set. The member geometry is described once, while `countAroundBody=4` requests four identical copies around the body at the same axial station.

`item.x=40` places the approximate geometric centroid of each copy 40 selected length units aft of the missile nose. It does not place the front edge of the two-unit-long block at 40. The member dimensions describe the block itself, while `x` selects the axial station used by the protuberance and boundary-layer calculation.

Because `member.verticalOffset=0`, the bottom of each block is referenced directly to the local body mold line at $x = 40$. Changing that value to 0.25 would retain the same axial station and block dimensions but represent the bottom of each block 0.25 selected length units outward from the local surface.

No explicit index is needed because the vector contains only one item; it therefore becomes protuberance set 1.

B.9.7. Body-Branch Validation Summary

Before execution, SpearDAT checks the body branch for the following structural and engineering conditions:

- only one body family is active, and inactive-family fields are empty;
- the selected or inferred geometry mode agrees with the populated fields;
- station-defined mold lines remain authoritative when optional nose/centrebody/aft section metadata is supplied;
- station arrays have compatible lengths and contain 3–49 stations;
- station radii and half-dimensions are non-negative, and elliptical ratios are positive;
- discontinuity indices are unique, one-based, and within the active station range;
- power-law noses have a positive exponent;
- exit diameters are finite and non-negative while preserving omission separately from explicit zero;
- base-jet data are axisymmetric-only, enabled explicitly, finite, and schedule-compatible; and
- protuberance identities, types, item counts, and member counts are internally consistent; and
- the required three- or four-member composite definition is kept separate from the circumferential copy count.

Warnings identify non-increasing station order, automatically activated protuberance content, and solver substitutions such as a non-zero fairing offset. SpearDAT does not silently sort geometry schedules or reinterpret an inactive body branch.

B.10. Fin Sets

The `finSets` structure vector defines lifting surfaces attached to the body. Each element describes one complete fin set: its spanwise planform, the number and circumferential orientation of its panels, and one airfoil section family used from root to tip.

A request may contain at most four fin sets. Their public set numbers are 1–4, corresponding to the four fin-set identities recognised by the analysis. Set numbers need not be contiguous: for example, sets 1 and 3 may be defined without set 2. They must nevertheless be unique, and increasing set number must follow the physical fore-to-aft order of the vehicle, as determined from the root leading-edge coordinate. When set numbers are omitted, vector position assigns them and the structure vector must consequently be ordered fore to aft.

All dimensional fin geometry uses the selected length system. All angular fields in this section are in degrees. Ratios and normalised chord coordinates are dimensionless.

Important

A fin set is one spanwise geometry repeated at one or more roll positions around the body. Do not create one `finSets` element per physical panel. Use `nPanels` and `panelRollDeg` within a single fin-set element to describe the circumferential copies.

B.10.1. Fin-Set Root Fields

Table B.44.: Fields accepted in each `input.caseInput.finSets(i)` element.

Field	Units/form	Availability/ default	Description
<code>input.caseInput.finSets(i).set</code>	Integer-valued scalar	Optional; inferred from vector position	Public fin-set number, from 1 to 4. When all set numbers are omitted, element 1 maps to set 1, element 2 to set 2, and so on. Explicit numbers may contain gaps but must be unique.
<code>input.caseInput.finSets(i).section</code>	Scalar structure	Optional; historical section default is hexagonal	Airfoil-section family and any family-specific data. See section B.10.5 .
<code>input.caseInput.finSets(i).spanStations</code>	Length vector	Required for an explicitly defined planform; 1–10 values	Semi-span locations of the planform break points, measured from the vehicle centreline. The first value may be zero to request automatic root placement on the body mold line.
<code>input.caseInput.finSets(i).chord</code>	Length vector	Required with <code>spanStations</code>	Chord at each span station. Its length must match <code>spanStations</code> .

Field	Units/form	Availability/ default	Description
<code>input.caseInput.finSets(i).xLeadingEdge</code>	Length vector	Required for the current MATLAB planform form	Nose-referenced leading-edge coordinate at every span station. The first value is the leading edge of the root chord. Its length must match <code>spanStations</code> .
<code>input.caseInput.finSets(i).sweepDeg</code>	Degree vector	Advanced legacy field; one value per span station when populated	Sweepback angle measured at the chord fraction selected by <code>sweepStation</code> . The underlying solver supports a root-XLE plus sweep reconstruction, but the current MATLAB contract requires populated station arrays to match <code>spanStations</code> ; see section B.10.3 .
<code>input.caseInput.finSets(i).sweepStation</code>	Ratio vector	Advanced legacy field; historical default 1	Chord fraction at which <code>sweepDeg</code> is measured: 0 is the leading edge and 1 is the trailing edge. When populated through MATLAB, its length must match <code>spanStations</code> .
<code>input.caseInput.finSets(i).leadingEdgeRadius</code>	Length vector	Optional; historical default 0	Leading-edge radius at each span station. It is not required for a NACA section generated from its designation.
<code>input.caseInput.finSets(i).nPanels</code>	Integer-valued scalar	Optional; 1–8; historical default 4	Number of circumferential panels in the fin set. When omitted, SpearDAT derives the count from <code>panelRollDeg</code> when that vector is populated, otherwise from <code>dihedralDeg</code> . If neither vector is populated, the historical default is four panels.
<code>input.caseInput.finSets(i).panelRollDeg</code>	Degree vector	Optional; maximum 8	Roll position of each panel, measured clockwise from top vertical centre when viewed forward from behind the vehicle. When omitted, the active panels are evenly spaced around the body.
<code>input.caseInput.finSets(i).dihedralDeg</code>	Degree vector	Optional; effective default 0	Dihedral of each panel. Positive dihedral acts in the direction of increasing <code>panelRollDeg</code> . When supplied, the vector length must equal the effective panel count.
<code>input.caseInput.finSets(i).flapChordRatio</code>	Ratio vector	Optional; one value per span station; maximum 10 entries	Plain trailing-edge-device chord divided by local fin chord. Zero means no device at that station, values below 1 define a trailing-edge flap, and 1 represents the full local chord. The limit of 10 is the vector length, not the maximum permitted ratio.

B.10.2. Set Numbering and Physical Order

The set field identifies a fin set for later references such as deflection, trim, output selection, and experimental-configuration selection. It also identifies that set's position in Missile DATCOM's cumulative front-to-back synthesis. The number does not generate the geometry, but it must agree with the physical axial order defined by the root `xLeadingEdge` value.

When set is omitted, vector position supplies the set number. The following two definitions are equivalent:

```
% Implicit numbering: first element is set 1, second is set 2.
input.caseInput.finSets = [forwardSet, aftSet];

% Equivalent explicit numbering.
forwardSet.set = 1;
aftSet.set = 2;
input.caseInput.finSets = [forwardSet, aftSet];
```

Listing B.31: Implicit and explicit fin-set numbering.

With implicit numbering, vector position therefore determines both identity and required physical order, so the vector must be arranged from nose to tail. Explicit numbering is recommended whenever a later branch refers to a set.

Numbering gaps are valid:

```
canardSet.set = 1;
tailSet.set = 3;

input.caseInput.finSets = [canardSet, tailSet];
```

Listing B.32: A valid gapped fin-set identity sequence.

The gap does not create an intermediate surface and does not cause SpearDAT to renumber the tail. For every pair of defined sets, however, the root `xLeadingEdge` coordinate must increase with set number. Thus set 3 must be physically aft of set 1. When explicit set numbers are supplied, the MATLAB structure-vector order is not itself the ordering criterion; retaining set-number order in the vector is nevertheless recommended for readability. SpearDAT preserves the supplied identities and validates them against the geometry rather than silently sorting or renumbering the request.

B.10.3. Planform Definition

A fin planform is divided into as many as ten spanwise segments by the values in `spanStations`. Each station is a planform break point at which the local chord, leading-edge position, section proportions, or trailing-edge-device geometry may change.

The first station is the root station. The first `xLeadingEdge` value is therefore the nose-referenced axial coordinate of the root-chord leading edge, while the first chord value is the root chord. When the first `spanStations` value is zero, the solver places that root on the local body mold line; the zero does not mean that the exposed root lies on the vehicle centreline.

Direct Leading-Edge Coordinates

The supported and recommended MATLAB representation supplies one leading-edge coordinate at every span station:

```
fin.spanStations = [0 8];
fin.chord = [12 7];
fin.xLeadingEdge = [44 49];
```

Listing B.33: Defining a swept trapezoid with explicit leading-edge coordinates.

The root leading edge is 44 selected length units aft of the missile nose. The tip leading edge is at 49. Together with the span and chord values, these two coordinates define the complete trapezoid without a separate sweep input.

For a multi-segment planform, add one entry to all active spanwise arrays at every break point. SpearDAT does not sort these stations or infer missing intermediate break points.

Legacy Root-Coordinate-plus-Sweep Form

The underlying Missile DATCOM solver also supports a legacy form in which the user supplies only the root leading-edge coordinate and then specifies the sweep of each segment. The sweep is measured at a selected chord fraction:

- sweepStation=0 measures leading-edge sweep;
- sweepStation=1 measures trailing-edge sweep; and
- an intermediate value measures sweep along the corresponding fractional-chord line.

Consider the two-station tapered fin used in [listing B.33](#):

$$c_{\text{root}} = 12, \quad c_{\text{tip}} = 7, \quad x_{\text{LE,root}} = 44.$$

The root trailing edge is therefore

$$x_{\text{TE,root}} = x_{\text{LE,root}} + c_{\text{root}} = 44 + 12 = 56.$$

The equivalent legacy input is:

```
% Legacy solver concept, not a valid root-only MATLAB request in the
% current typed interface:
% XLE(1) = 44
% SWEEP   = 0 deg
% STA     = 1      % sweep is measured at the trailing edge
```

Listing B.34: Legacy trailing-edge-sweep description equivalent to [listing B.33](#).

Because STA=1, the specified zero-degree sweep applies to the *trailing-edge line*, not to the leading edge and not to the complete fin. An unswept trailing edge keeps the tip trailing edge at the same axial coordinate:

$$x_{\text{TE,tip}} = x_{\text{TE,root}} = 56.$$

The tip chord is only 7 units long, so its leading edge must be

$$x_{LE,tip} = x_{TE,tip} - c_{tip} = 56 - 7 = 49.$$

The resulting planform is therefore a tapered trapezoid with an unswept trailing edge and a swept-back leading edge. It is not rectangular. A rectangular unswept fin would require equal root and tip chords as well as unchanged leading- and trailing-edge coordinates.

Important

SpearDAT does expose the legacy STA quantity through the public MATLAB field `sweepStation`; similarly, `sweepDeg` maps to `SWEEP`. The present limitation is more specific: the typed MATLAB validator requires every populated spanwise array, including `xLeadingEdge`, `sweepDeg`, and `sweepStation`, to have the same length as `spanStations`. Consequently, the legacy combination of a multi-station span vector with only one root `xLeadingEdge` value is not currently a valid SpearDAT MATLAB request. For current scripts, calculate and supply the complete `xLeadingEdge` vector as shown below. Supplying complete `XLE`, `SWEEP`, and `STA` vectors together is possible but redundant and can introduce inconsistent geometry.

A user who prefers to design from sweep angles can calculate the explicit leading-edge coordinates in MATLAB before constructing the request. The following code reproduces the two-station example using an unswept trailing edge:

```
span = [0 8];
chord = [12 7];
rootXLeadingEdge = 44;

sweepReference = 1;      % 0 = leading edge, 1 = trailing edge
segmentSweepDeg = 0;     % unswept trailing edge

rootReferenceX = rootXLeadingEdge + ...
    sweepReference * chord(1);
tipReferenceX = rootReferenceX + ...
    diff(span) * tand(segmentSweepDeg);
tipXLeadingEdge = tipReferenceX - ...
    sweepReference * chord(2);

fin.spanStations = span;
fin.chord = chord;
fin.xLeadingEdge = [rootXLeadingEdge tipXLeadingEdge];
% Result: [44 49]
```

Listing B.35: Converting a desired sweep reference into explicit MATLAB leading-edge coordinates.

A complete `xLeadingEdge` vector may be supplied together with complete sweep and sweep-reference vectors. This passes the basic array-shape rules, and SpearDAT writes all three definitions. It does not discard one or resolve a conflict between them. Such input is redundant and can be numerically inconsistent, so ordinary MATLAB requests should use the explicit leading-edge form alone.

Plain Trailing-Edge Devices

flapChordRatio defines the chordwise extent of a plain trailing-edge device at each span station:

$$\text{flapChordRatio} = \frac{\text{local flap chord}}{\text{local fin chord}}.$$

The value is measured forward from the trailing edge. For example:

- 0 means that no trailing-edge device exists at that station;
- 0.25 means that the aft 25 percent of the local chord moves, with the hinge at approximately 75 percent chord; and
- 1 means that the full local chord participates in the panel-style deflection.

The “maximum 10” shown in [table B.44](#) is the maximum number of vector entries because a fin set has at most ten span stations. It is not a permitted ratio of 10. Engineering values should normally remain between 0 and 1.

A non-zero flapChordRatio defines device geometry; it does not by itself command a deflection. The device is moved through `input.caseInput.deflections` or varied by a trim request.

A full-span 25-percent-chord flap on a two-station fin is:

```
fin.spanStations = [0 8];
fin.chord = [12 7];
fin.xLeadingEdge = [44 49];
fin.flapChordRatio = [0.25 0.25];
```

Listing B.36: A full-span 25-percent-chord trailing-edge device.

A partial-span device requires break points at both ends of the active region:

```
fin.spanStations = [0 2 6 8];
fin.chord = [4 4 3 3];
fin.xLeadingEdge = [10 10.7 12.0 12.0];
fin.flapChordRatio = [0 0.25 0.25 0];
```

Listing B.37: A partial-span trailing-edge device.

The device occupies the span interval from station 2 to station 3. The zero values exclude the root and tip regions. The active hinge must remain straight, and the device geometry must not have an invalid increasing taper ratio. These are physical restrictions of the legacy method rather than arbitrary three-dimensional control-surface modelling.

B.10.4. Panel Count, Roll Position, and Dihedral

nPanels is the number of copies of the spanwise planform around the body. The effective count is limited to eight. If explicit roll positions are supplied, panelRollDeg must contain exactly one value per panel.

The roll-angle convention is the same convention used for inlet placement: the observer is behind the vehicle and looks forward along the longitudinal axis. Angles increase clockwise from top vertical centre. Thus a conventional four-panel cruciform may be written as:

```

fin.nPanels = 4;
fin.panelRollDeg = [0 90 180 270];
fin.dihedralDeg = [0 0 0 0];

```

Listing B.38: A four-panel cruciform orientation.

With this convention, the four panels are located at top, right, bottom, and left respectively when viewed from aft looking forward. Values separated by 360 degrees are physically equivalent but remain distinct supplied values; use a single consistent angular convention throughout a model.

B.10.5. Airfoil-Section Selection

The user may define a specific airfoil section for each fin set. This section controls the two-dimensional cross-section used by the fin aerodynamic methods; it is separate from the spanwise planform described previously. SpearDAT supports simple automatically generated HEX and ARC sections, custom USER coordinates, and supported NACA designations.

One section family applies to the entire fin set. Its proportions may vary from span station to span station where that family supports spanwise arrays, but the family itself cannot change between stations.

Table B.45.: Allowed values of `input.caseInput.finSets(i).section.type`.

Value	Active subordinate branch	Meaning
"Hex"	section.hexArc	Hexagonal or diamond-like section. All six HEX/ARC arrays may be used.
"Arc"	section.hexArc	Circular-arc section. Only zUpper and zLower are active.
"User"	section.user	User-supplied normalised section coordinates, defined by one of two supported coordinate representations.
"Naca"	section.naca	A section generated by the Missile DATCOM NACA designation parser.

Populating section-specific data without `section.type` is rejected because the intended interpretation is ambiguous. Fields belonging to inactive section branches must remain unset.

Hexagonal and Circular-Arc Sections

Table B.46.: Fields accepted under `input.caseInput.finSets(i).section.hexArc`.

Field	Units	Availability/default	Description
<code>input.caseInput.finSets(i).section.hexArc.zUpper</code>	Chord fraction	Optional; maximum 10	Upper-surface thickness or ordinate ratio at each span station. Active for "Hex" and "Arc".

Field	Units	Availability/ default		Description
<code>input.caseInput.finSets(i).section. hexArc.zLower</code>	Chord fraction	Optional; maximum 10	maxi-	Lower-surface thickness or ordinate ratio at each span station. Active for "Hex" and "Arc".
<code>input.caseInput.finSets(i).section. hexArc.lMaxUpper</code>	Chord fraction	Optional; only	"Hex"	Upper-surface chord fraction from the section leading edge to maximum thickness.
<code>input.caseInput.finSets(i).section. hexArc.lMaxLower</code>	Chord fraction	Optional; only	"Hex"	Lower-surface chord fraction from the section leading edge to maximum thickness.
<code>input.caseInput.finSets(i).section. hexArc.lFlatUpper</code>	Chord fraction	Optional; only	"Hex"	Upper-surface fraction of chord over which the section remains at constant thickness.
<code>input.caseInput.finSets(i).section. hexArc.lFlatLower</code>	Chord fraction	Optional; only	"Hex"	Lower-surface fraction of chord over which the section remains at constant thickness.

When these arrays are populated, their lengths must match `spanStations`. An ARC request must not populate `lMaxUpper`, `lMaxLower`, `lFlatUpper`, or `lFlatLower`.

Important

The automatic HEX and ARC paths preserve a significant legacy behaviour. The effective section used by the aerodynamic methods is generated as a symmetric NACA supersonic S-* section from the upper-side inputs.

For HEX, `zUpper`, `lMaxUpper`, and `lFlatUpper` control the generated section. For ARC, `zUpper` controls the generated circular-arc thickness. The corresponding lower-side fields are accepted, stored, and may appear in input echoes, but they do not alter the effective generated section in the validated legacy path.

Use "User" when a genuinely asymmetric, cambered, or independently defined upper/lower section is required.

User-Defined Sections

The USER option allows the user to provide the actual normalised airfoil shape rather than selecting an automatically generated section. It is the appropriate choice for measured coordinates, CAD-derived profiles, cambered or asymmetric sections, and section shapes that cannot be represented by the HEX, ARC, or supported NACA families.

The section is represented as a single-valued upper and lower surface over a common chordwise coordinate. A user-defined section contains at most 50 normalised chordwise points. `xChord` is required and one of two complete ordinate representations must accompany it.

Table B.47.: Fields accepted under `input.caseInput.finSets(i).section.user`.

Field	Units	Availability	Description
<code>input.caseInput.finSets(i).section.user.xChord</code>	Chord fraction	Required; 1–50 values	Chordwise station measured from the section leading edge.
<code>input.caseInput.finSets(i).section.user.mean</code>	Chord fraction	Required with thickness in representation 1	Mean-line ordinate at each xChord.
<code>input.caseInput.finSets(i).section.user.thickness</code>	Chord fraction	Required with mean in representation 1	Full local thickness, measured perpendicular to the mean line.
<code>input.caseInput.finSets(i).section.user.yUpper</code>	Chord fraction	Required with yLower in representation 2	Upper-surface ordinate at each xChord.
<code>input.caseInput.finSets(i).section.user.yLower</code>	Chord fraction	Required with yUpper in representation 2	Lower-surface ordinate at each xChord.

The two accepted combinations are:

1. xChord, mean, and thickness; or
2. xChord, yUpper, and yLower.

Every populated ordinate vector must have the same number of elements as xChord. Do not mix an incomplete mean/thickness definition with an incomplete upper/lower definition.

For a conventional closed section, xChord should increase from 0 at the leading edge to 1 at the trailing edge. Closure means:

- in the mean/thickness representation, the thickness is zero at each closed edge; and
- in the upper/lower representation, the upper and lower ordinates are equal at each closed edge.

The current SpearDAT validator checks that the arrays are finite, complete, consistently sized, and no longer than 50 points. It does not presently enforce monotonic chordwise ordering or geometric closure. The user must therefore supply a physically sensible profile. Because both surfaces are functions of a single xChord vector, a perfectly vertical leading or trailing edge is not represented naturally; use a short chordwise ramp if a box-like section is required.

```
fin.section.type = "User";
fin.section.user.xChord = [0 0.25 0.5 0.75 1.0];
fin.section.user.mean = [0 0 0 0 0];
fin.section.user.thickness = [0 0.08 0.10 0.06 0];
```

Listing B.39: A simple closed symmetric user-defined section.

The first and last thickness values are zero, so the upper and lower surfaces meet at the leading and trailing edges.

NACA Sections

The NACA branch invokes the designation families supported by the Missile DATCOM parser. It is not a general lookup into an arbitrary airfoil database. Use "User" for custom profiles, externally sourced coordinates, CFD-derived sections, or airfoils outside the supported designation syntax.

Table B.48.: Fields accepted under `input.caseInput.finSets(i).section.naca`.

Field	Form	Availability	Description
<code>input.caseInput.finSets(i).section.naca.sectionType</code>	Text enum	Required "Naca"	Selects the designation family. Allowed values are listed in table B.49 .
<code>input.caseInput.finSets(i).section.naca.designation</code>	Text scalar	Required "Naca"	Family-specific designation text, excluding the generated fin-set and family prefixes. It must be non-empty and no longer than 15 characters.

Table B.49.: Allowed NACA section-family values.

MATLAB value	Generated family token	Designation family
"OneSeries"	1	NACA 1-series.
"FourDigit"	4	Four-digit and modified four-digit series.
"FiveDigit"	5	Five-digit and modified five-digit series.
"SixSeries"	6	NACA 6-series.
"Supersonic"	S	Supported supersonic diamond, circular-arc, and hexagonal forms.

The designation may contain digits, uppercase A, commas, equals signs, periods, hyphens, and spaces. Lowercase letters and other punctuation are rejected.

A conventional NACA 0012 section is written as:

```
fin.section.type = "Naca";
fin.section.naca.sectionType = "FourDigit";
fin.section.naca.designation = "0012";
```

Listing B.40: Selecting a NACA 0012 fin section.

SpearDAT combines the public set number, family token, and designation when it constructs the solver input. The user therefore supplies only "0012", not a complete legacy card such as "NACA-1-4-0012".

B.10.6. Fin-Set Examples

The following examples progress from a minimal planform to a multi-segment, partial-flap configuration. Each example is valid under the current MATLAB station-array rules.

Simple Rectangular Two-Panel Set

This is the smallest practical explicit planform: two span stations, constant chord, constant leading-edge position, and a generated NACA 0012 section.

```
fin = struct;
fin.set = 1;

fin.spanStations = [0 4];
fin.chord = [3 3];
fin.xLeadingEdge = [20 20];

fin.nPanels = 2;
fin.panelRollDeg = [90 270];
fin.dihedralDeg = [0 0];

fin.section.type = "Naca";
fin.section.naca.sectionType = "FourDigit";
fin.section.naca.designation = "0012";

input.caseInput.finSets = fin;
```

Listing B.41: A simple rectangular bilateral fin set.

The two identical panels are placed on opposite sides of the body. Equal chord and leading-edge coordinates make the planform rectangular.

Four-Panel Multi-Station HEX Set

The next example defines one four-panel cruciform set with three spanwise planform stations and an automatically generated HEX section:

```
fin = struct;
fin.set = 1;

fin.spanStations = [0.0 0.5 1.0];
fin.chord = [1.0 0.8 0.5];
fin.xLeadingEdge = [3.0 3.1 3.3];

fin.nPanels = 4;
fin.panelRollDeg = [0 90 180 270];
fin.dihedralDeg = [0 0 0 0];

fin.section.type = "Hex";
fin.section.hexArc.zUpper = [0.05 0.04 0.03];
fin.section.hexArc.zLower = [0.05 0.04 0.03];
fin.section.hexArc.lMaxUpper = [0.30 0.30 0.30];
fin.section.hexArc.lMaxLower = [0.30 0.30 0.30];
fin.section.hexArc.lFlatUpper = [0.20 0.20 0.20];
```

```
fin.section.hexArc.lFlatLower = [0.20 0.20 0.20];

input.caseInput.finSets = fin;
```

Listing B.42: A four-panel multi-station fin-set definition.

Because `spanStations(1)=0`, the root is placed relative to the local body mold line. The first leading-edge coordinate, 3.0, locates the root chord axially. The two subsequent coordinates create the swept, tapered outboard segments. As explained in [section B.10.5](#), the effective generated HEX section is controlled by the upper-side arrays.

Two Mirrored Wings with Sweep, Dihedral, and a Partial Flap

The following example resembles a simple aircraft-style bilateral wing. It has two mirrored panels, several planform break points, a span-limited 25-percent-chord trailing-edge flap, mirrored dihedral, and a rectangular outer segment.

```
wing = struct;
wing.set = 1;

wing.spanStations = [0 2 6 8];
wing.chord = [4 4 3 3];

% Construct explicit leading-edge coordinates from desired segment
% leading-edge sweep angles. The last segment is unswept.
segmentSweepDeg = [20 18 0];
wing.xLeadingEdge = [10, ...
    10 + cumsum(diff(wing.spanStations) .* ...
        tand(segmentSweepDeg))];

% No flap at the root or tip; a 25%-chord flap between stations 2 and 3.
wing.flapChordRatio = [0 0.25 0.25 0];

wing.nPanels = 2;
wing.panelRollDeg = [90 270];
wing.dihedralDeg = [-5 5];

wing.section.type = "Naca";
wing.section.naca.sectionType = "FourDigit";
wing.section.naca.designation = "0012";

input.caseInput.finSets = wing;
```

Listing B.43: A multi-segment bilateral fin set with a partial-span flap.

The first two segments are swept because their leading-edge coordinates increase with span. The last two stations have the same chord and, because the last requested segment sweep is zero, the same leading-edge coordinate; the outer segment is therefore rectangular. The opposite dihedral signs create a mirrored bilateral installation under the DATCOM panel convention.

The flap exists only between span stations 2 and 3. It remains stationary until a later deflection or trim request acts on the panels. Its two active boundary points define one straight hinge line, while the zero values exclude the root and tip regions.

B.10.7. Fin-Set Validation Summary

Before execution, SpearDAT checks that:

- no more than four fin sets are supplied;
- set numbers are unique integers from 1 to 4 and their root `xLeadingEdge` coordinates increase strictly in set-number order; when set numbers are omitted, vector position supplies that order;
- spanwise arrays contain no more than ten finite values and every populated station array matches `spanStations`;
- the effective panel count is between 1 and 8, and panel-specific arrays match it;
- one section family governs each set and inactive section branches are empty;
- ARC data use only the two ARC-supported ordinate fields;
- USER data contain `xChord` plus one complete ordinate representation, with matching lengths and no more than 50 points; physical ordering and closure remain the user's responsibility; and
- NACA data contain an allowed section family and a valid, non-empty designation of at most 15 characters.

Numbering gaps are accepted. SpearDAT does not renumber a supplied set, sort a planform schedule, or silently convert one section family into another.

B.11. Fixed Panel Deflections

The deflections branch fixes the incidence of individual panels in one or more defined fin sets. It may also supply a hinge-line position and hinge-line skew for each affected set.

The branch contains one accepted field, `input.caseInput.deflections.finSets`, which is a structure vector with at most four elements.

B.11.1. Deflection Fin-Set Fields

Table B.50.: Fields accepted in each `input.caseInput.deflections.finSets(i)` element.

Field	Units/form	Availability/default	Description
<code>input.caseInput.deflections.finSets(i).set</code>	Integer-valued scalar	Optional; inferred from vector position	Defined fin-set identity, from 1 to 4. The referenced set must exist in <code>input.caseInput.finSets</code> .
<code>input.caseInput.deflections.finSets(i).panelDeflectionDeg</code>	Degree vector	Required for a panel-deflection request	Fixed deflection of each panel in the referenced set. The vector must contain exactly one value per effective panel and no more than eight values.

Field	Units/form	Availability/ default	Description
<code>input.caseInput.deflections.finSets(i).hingeX</code>	Length	Optional; solver default is the exposed-root mid-chord location	Axial hinge-line position measured from the active geometry coordinate origin. It is not automatically measured from the body nose when the body origin is non-zero.
<code>input.caseInput.deflections.finSets(i).hingeSkewDeg</code>	Degrees	Optional; historical default 0	Sweepback angle of the hinge line for the referenced fin set.

When all deflection set values are omitted, vector position maps the first element to fin set 1, the second to fin set 2, and so on. Explicit set numbers are clearer when the main fin-set numbering contains a gap.

B.11.2. Raw Panel-Deflection Sign Convention

The raw panel-deflection sign is based on rolling-moment sense, not on a single vehicle-independent “trailing edge down” rule:

Important

A positive raw panel deflection is the sense that produces a negative body-axis rolling-moment increment at zero angle of attack and zero sideslip.

The corresponding physical trailing-edge motion therefore depends on the panel roll position. For the conventional cruciform orientation $[0, 90, 180, 270]$, positive raw deflection rotates each panel in the same rolling-moment sense when viewed from aft looking forward; it does not move every trailing edge in the same vertical direction.

This convention applies to `panelDeflectionDeg`. The printed trim deflection uses the different pitch-trim convention described in [section B.12.3](#).

B.11.3. Hinge Coordinates

`hingeX` uses the same global longitudinal coordinate system as the fin and body geometry. For an axisymmetric body with `input.caseInput.body.axisymmetric.xOrigin` non-zero, a supplied hinge coordinate is not automatically shifted to remain nose-relative. Similarly, an elliptical body uses its own active `xOrigin`.

Omitting `hingeX` allows the solver to place the hinge at the historical default, approximately one-half of the exposed root chord measured from the active coordinate origin. Supplying the value explicitly is recommended for a physical control surface.

B.11.4. Deflection Example

For the four-panel fin set in [listing B.42](#), fixed panel incidences may be supplied as:

```
defl = struct;
defl.set = 1;
defl.panelDeflectionDeg = [0 5 0 -5];
defl.hingeX = 3.55;
defl.hingeSkewDeg = 0;

input.caseInput.deflections.finSets = defl;
```

Listing B.44: Applying fixed deflections to fin set 1.

The second and fourth values apply to panels 2 and 4 respectively. Their opposite numeric signs are raw panel commands; interpret them using the panel roll positions and the rolling-moment convention rather than a generic elevator convention.

hingeX=3.55 places the common hinge line at axial coordinate 3.55 in the active geometry coordinate system. In this example the root leading edge is at 3.0, so the hinge is 0.55 selected length units aft of that root leading edge. hingeSkewDeg=0 makes the hinge line unswept.

B.11.5. Deflection Validation Summary

SpearDAT checks that deflection-set identities are unique and within 1–4, that every referenced fin set exists, and that each deflection vector is finite and matches the effective panel count of its set. An effective count is obtained from the fin-set panel declaration and its panel-orientation data. A deflection request may not contain more than four affected sets or more than eight panel values per set.

B.12. Longitudinal Trim

The trim branch requests longitudinal static trim. For each active angle of attack, the analysis varies selected panels from one fin set and searches for a zero pitching-moment condition.

Only one fin set may participate in a trim request. The user selects the movable panels and may constrain the search interval with minimum and maximum deflection bounds.

B.12.1. Trim Fields

Table B.51.: Fields accepted under `input.caseInput.trim`.

Field	Units/form	Availability/ default	Description
<code>input.caseInput.trim.enabled</code>	Logical scalar	Optional; default false	Explicitly requests longitudinal trim. Set this to true for a deliberate trim analysis.

Field	Units/form	Availability/ default	Description
<code>input.caseInput.trim.finSet</code>	Integer-valued scalar	Optional; historical fallback is set 1	Identity of the one fin set used for trim. The selected set must exist. If omitted, fin set 1 must exist for the fallback to be valid.
<code>input.caseInput.trim.panels</code>	Integer vector	Optional; recommended explicitly	One-based panel numbers varied by the trim solution. Values must be unique, from 1 to 8, and no greater than the selected set's effective panel count.
<code>input.caseInput.trim.deflectionMinDeg</code>	Degrees	Optional only as a pair; historical default -25	Lower deflection bound searched by the trim solution. Supply together with <code>deflectionMaxDeg</code> .
<code>input.caseInput.trim.deflectionMaxDeg</code>	Degrees	Optional only as a pair; historical default +20	Upper deflection bound searched by the trim solution. Supply together with <code>deflectionMinDeg</code> .
<code>input.caseInput.trim.asymmetryPanels</code>	Integer vector	Optional; default empty	Selected panel numbers for which the trim sign treatment is reversed. Every entry must also appear in panels.

Populating any trim field other than enabled causes the trim request to be emitted even when enabled remains false. SpearDAT reports that situation as a warning. For an intentional request, set `enabled=true` so the activation is explicit and immediately visible in the MATLAB script.

B.12.2. Trim Search and Bounds

The trim routine does not perform a continuously iterated nonlinear root solve. For each Mach condition it evaluates ten equally spaced deflection values from the lower bound to the upper bound, including both endpoints. At each angle of attack it then examines the resulting pitching-moment values:

1. an exactly zero sampled pitching moment is accepted directly;
2. otherwise, the first adjacent pair whose pitching moments have opposite signs brackets the trim point; and
3. the reported trim deflection and other coefficients are obtained by linear interpolation between that pair.

If no zero or sign change occurs within the ten samples, the operating point is reported as not trimmed. The routine does not extrapolate outside the requested bounds.

Note

The returned deflection is Missile DATCOM's interpolation-based trim estimate. A separate fixed-deflection run at that exact value is not guaranteed to give an exact nonlinear zero of pitching moment. A user requiring a tighter root can run repeated

fixed-deflection SpearDAT cases and apply a higher-level numerical root finder externally.

The two bounds are inseparable:

- supply both `deflectionMinDeg` and `deflectionMaxDeg`; or
- omit both and use the historical -25 to $+20$ degree search interval.

Both values must be finite and the minimum must not exceed the maximum.

B.12.3. Trim Sign Convention and `asymmetryPanels`

The raw deflection convention in [section B.11.2](#) would give opposite numeric signs to the two sides of a conventional pitch-control pair. The trim calculation therefore applies a pitch-oriented sign treatment: panels on the left-hand side of the configuration, with roll positions strictly between 180 and 360 degrees, are reversed automatically.

The resulting printed trim command follows this practical convention:

Important

A positive trim deflection represents trailing-edge-down pitch-control sense for the normally selected pitch-trim panels.

`asymmetryPanels` reverses this trim treatment for listed selected panels. It is intended for configurations whose desired actuation sense differs from the normal pitch-pair convention. Every `asymmetry` entry must already be a member of `panels`; it cannot introduce an additional movable panel.

B.12.4. Trim Example

The following request varies panels 2 and 4 of fin set 1 between -25 and $+20$ degrees:

```
input.caseInput.trim.enabled = true;
input.caseInput.trim.finSet = 1;
input.caseInput.trim.panels = [2 4];
input.caseInput.trim.deflectionMinDeg = -25;
input.caseInput.trim.deflectionMaxDeg = 20;
```

Listing B.45: Requesting longitudinal trim with a pitch pair.

For a cruciform set at $[0, 90, 180, 270]$, the calculation handles the left-side sign reversal internally so that its reported trim command has the pitch-oriented trailing-edge-down meaning described above.

B.12.5. Trim Validation Summary

SpearDAT checks that the selected set exists, panel numbers are unique and within that set, asymmetry entries are a subset of the selected panels, and the deflection bounds are supplied as a finite ordered pair. Populated trim fields with `enabled=false` produce a warning rather than silent deactivation because the generated request is still a trim request.

A component-build-up request used together with trim may not produce the expected component report. This interaction is reported diagnostically rather than being silently removed from the request.

B.13. Inlet Definition

The `inlet` branch describes one inlet geometry that may be copied to one or more circumferential roll positions. Supported geometries are side-mounted two-dimensional, top-mounted two-dimensional, and axisymmetric inlets. The branch includes the inlet and diverter location, a five-point longitudinal profile, cover and ramp options, and an optional mass-flow schedule for additive drag.

All inlet dimensions use the selected length system. Placement and ramp angles are in degrees. Mass-flow ratio is dimensionless: it represents the fraction of the inlet capture-streamtube flow admitted by the inlet. A value approaching 1 represents little or no spillage, while a lower value represents increasing flow spillage at an off-design condition.

Important

placements contains repeated copies of the same inlet geometry. It does not contain a separate profile for each copy. To model geometrically different inlets, use separate analysis requests.

B.13.1. Inlet Root Fields

Table B.52.: Fields accepted under `input.caseInput.inlet`.

Field	Units/form	Availability/ default	Description
<code>input.caseInput.inlet.enabled</code>	Logical scalar	Optional; default false	Explicitly activates the inlet branch. Populated inlet data also activate the branch, but SpearDAT then reports an auto-activation warning.
<code>input.caseInput.inlet.type</code>	Text enum	Required when the inlet branch is active	Selects the inlet geometry family. Allowed values are listed in table B.53 .
<code>input.caseInput.inlet.xInlet</code>	Length	Required when active	Longitudinal distance from the missile nose tip to the inlet leading edge.
<code>input.caseInput.inlet.xDiverter</code>	Length	Required when active	Longitudinal distance from the inlet leading edge to the diverter leading edge. This is inlet-local, unlike <code>xInlet</code> .

Field	Units/form	Availability/ default	Description
input.caseInput.inlet.diverterHeight	Length	Required when active	Diverter height at its leading edge. Zero defines a conformal inlet. A negative value represents a semi-submerged inlet and produces a warning, not an error.
input.caseInput.inlet.diverterLength	Length	Required when active	Diverter length. It must be finite and non-negative.
input.caseInput.inlet.placements	Structure vector	Required when active; 1–20 elements	Circumferential copies of the inlet geometry. See section B.13.3 .
input.caseInput.inlet.profile	Scalar structure	Required when active	Five inlet-local longitudinal stations and their height/width or diameter data. The global coordinate of station k is <code>xInlet+profile.x(k)</code> . See section B.13.4 .
input.caseInput.inlet.covered	Logical scalar	Optional; default false	true models a plugged inlet with no internal flow; the legacy geometry places the plug between profile stations 1 and 2, flush with the inlet face. Additive-drag calculation is skipped for a covered inlet.
input.caseInput.inlet.rampAngleDeg	Degrees	Optional	External-compression inlet ramp angle. Explicit zero is a valid zero-angle ramp.
input.caseInput.inlet.additiveDrag	Logical scalar	Optional; default false	Requests the inlet additive-drag contribution. When true, <code>massFlowRatio</code> is required.
input.caseInput.inlet.massFlowRatio	Dimensionless vector	Required only when additiveDrag is true	Fraction of capture-streamtube mass flow admitted by the inlet. A value of 1 would represent zero spillage, but the accepted input range is strictly $0 < \text{MFR} < 1$. Supply one explicit value per active Mach or velocity condition, aligned positionally with the active speed schedule; maximum length is 20.

As with trim, populated inlet fields activate the generated inlet request even when enabled remains false. Set `enabled=true` for a deliberate inlet analysis.

B.13.2. Inlet Types

The text values are exact and case-sensitive.

Table B.53.: Allowed values of `input.caseInput.inlet.type`.

MATLAB value	Geometry	Profile use
"TwoDimensionalSide"	Side-mounted two-dimensional inlet	Requires five local <code>profile.x</code> values plus five heights and widths. The width normally begins at zero at the inlet leading edge.
"Axisymmetric"	Axisymmetric inlet	Requires five local <code>profile.x</code> values and five diameters in <code>profile.width</code> ; height may be omitted. The leading-edge diameter normally begins at zero.
"TwoDimensionalTop"	Top-mounted two-dimensional inlet	Requires five local <code>profile.x</code> values plus five heights and widths. The height normally begins at zero at the inlet leading edge.

B.13.3. Circumferential Placements

Each placement element identifies one copy of the common inlet geometry.

Table B.54.: Fields accepted in each `input.caseInput.inlet.placements(i)` element.

Field	Units/form	Availability/default	Description
<code>input.caseInput.inlet.placements(i).index</code>	Integer-valued scalar	Optional; inferred from vector position	Placement identity. If any placement supplies an index, every placement must supply one and the values must form the contiguous sequence from 1 to the number of placements.
<code>input.caseInput.inlet.placements(i).phiDeg</code>	Degrees	Required	Inlet roll orientation, measured clockwise from top vertical centre when viewed forward from behind the vehicle.

When no indices are supplied, vector order determines the generated placement order. The number of placement elements becomes the number of identical inlet copies; users do not supply that count separately.

For two symmetric side inlets:

```
p(1).index = 1;
p(1).phiDeg = 90;

p(2).index = 2;
p(2).phiDeg = 270;
```

```
input.caseInput.inlet.placements = p;
```

Listing B.46: Two inlet placements at opposite roll positions.

The two entries reuse the same inlet profile. They create one copy on each side of the body; they do not define two different inlet geometries.

B.13.4. Five-Point Inlet Profile

The inlet is not defined by one rectangular opening. It is represented by a longitudinal sequence of five cross-sections that describe the inlet lip, midbody, and aft closure:

1. inlet leading edge or tip;
2. cowl-lip leading edge;
3. midbody start;
4. boattail start; and
5. boattail end.

x_{Inlet} locates station 1 globally from the missile nose. profile.x is a separate local coordinate measured aft from that inlet leading edge. Therefore:

$$x_{\text{global},k} = x_{\text{inlet}} + x_{\text{profile},k}.$$

The first local value is normally zero. For example, if $x_{\text{Inlet}}=40$ and $\text{profile.x}(3)=1.5$, profile station 3 lies at global axial coordinate 41.5 in the selected length system.

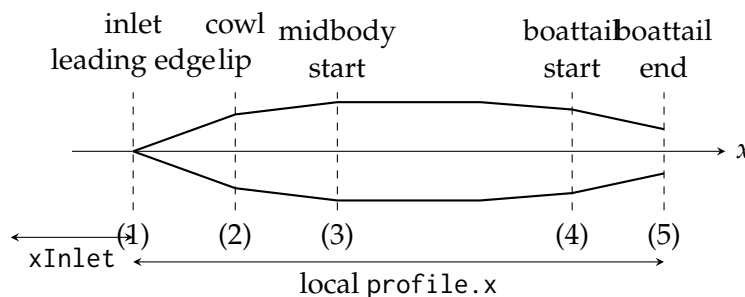


Figure B.1.: Schematic meaning of the five inlet profile stations. The station names and ordering follow the detailed top-mounted, side-mounted, and axisymmetric layouts in Figures 12–14 of the Missile DATCOM User's Manual [1]. Dimensions are not to scale.

Table B.55.: Fields accepted under `input.caseInput.inlet.profile`.

Field	Units	Availability	Description
<code>input.caseInput.inlet.profile.x</code>	Length vector	Required; exactly five values	Inlet-local longitudinal coordinates for stations 1–5, measured from the inlet leading edge. The first value is normally zero. These are not global nose-referenced coordinates.
<code>input.caseInput.inlet.profile.height</code>	Length vector	Exactly five for two-dimensional inlets; omit for axisymmetric inlet	Inlet height at each profile station. Values must be finite and non-negative.
<code>input.caseInput.inlet.profile.width</code>	Length vector	Required; exactly five values	Inlet width for two-dimensional types, or inlet diameter for the axisymmetric type. Values must be finite and non-negative.

Aft Boattail Requirement

The word “boattail” does not mean that the inlet must close to zero at station 5. It means that the final portion contracts from station 4 to station 5:

- for a two-dimensional inlet, the final height-times-width area must be smaller than the area at the start of the boattail, $H_5W_5 < H_4W_4$; and
- for an axisymmetric inlet, the final diameter must be smaller than the diameter at station 4, $W_5 < W_4$.

The user supplies the five dimensions; SpearDAT performs these comparisons automatically during input validation. An invalid profile produces an error and the solver is not run. The formula is included here so that the user can design a valid profile, not because the comparison must be coded separately in the MATLAB script.

For reference, the two-dimensional validation is equivalent to:

```
profile.height(5) * profile.width(5) < ...
    profile.height(4) * profile.width(4)
```

Listing B.47: Two-dimensional inlet boattail-area condition.

B.13.5. Additive Drag and Mass-Flow Ratio

An open external-compression inlet operating away from its design condition may spill part of the approaching streamtube around the cowl. Turning that flow creates an additional axial-force contribution, historically called inlet additive drag or spillage drag.

The mass-flow ratio is the admitted fraction of the inlet capture-streamtube flow. A value approaching 1 corresponds to little spillage; smaller values represent more spillage. The accepted input range is strictly between zero and one.

The calculation is activated explicitly in MATLAB:

```
inlet.additiveDrag = true;
inlet.rampAngleDeg = 5;

% One value per active speed condition in this example.
inlet.massFlowRatio = [0.55 0.60 0.68];
```

Listing B.48: Activating inlet additive drag.

When `additiveDrag=true`, `massFlowRatio` is mandatory. In the current SpearDAT release, supply one explicit value for every active Mach or velocity condition. The vector must align positionally with the speed schedule in `input.caseInput.flight.speed`.

If the schedule contains subsonic conditions, their vector positions must still be represented even though the legacy additive-drag method normally does not calculate this contribution below Mach 1. For example, a three-speed schedule requires three MFR entries. Do not rely on a scalar value being broadcast in the current release.

The legacy method is intended mainly for supersonic external-compression inlets. It calculates additive drag at zero angle of attack and applies that increment across the angle-of-attack table for the same speed. If `covered=true`, the inlet admits no flow and the additive-drag calculation is skipped.

A mass-flow vector is rejected when `additiveDrag` is false because it would otherwise be inactive and misleading.

B.13.6. Side-Mounted Two-Dimensional Inlet Example

A side-mounted two-dimensional inlet uses both height and width schedules. The following example places one inlet on the right-hand side of the body. Station 1 is the inlet leading edge; its width begins at zero and expands toward the cowl and midbody before contracting through the boattail.

```
inlet = struct;
inlet.enabled = true;
inlet.type = "TwoDimensionalSide";

inlet.xInlet = 4.0;
inlet.xDiverter = 0.35;
inlet.diverterHeight = 0.1;
inlet.diverterLength = 0.5;

inlet.placements(1).index = 1;
inlet.placements(1).phiDeg = 90;

inlet.profile.x = [0.0 0.5 1.0 1.5 2.0];
inlet.profile.height = [1.0 1.2 1.3 1.2 0.8];
inlet.profile.width = [0.0 2.1 2.2 2.0 1.5];

inlet.covered = false;
inlet.rampAngleDeg = 5.0;
inlet.additiveDrag = false;

input.caseInput.inlet = inlet;
```

Listing B.49: A complete side-mounted two-dimensional inlet.

The value 4.0 assigned to `xInlet` places profile station 1 four selected length units aft of the missile nose. The value 0.35 assigned to `xDiverter` places the diverter leading edge 0.35 units aft of the inlet leading edge, not 0.35 units from the nose. The placement angle of 90 degrees puts the copy on the right-hand side in the aft-looking-forward convention.

At station 4 the area is $1.2 \times 2.0 = 2.4$, while at station 5 it is $0.8 \times 1.5 = 1.2$. The aft area therefore contracts and satisfies the boattail rule.

B.13.7. Top-Mounted Two-Dimensional Inlet Example

A top-mounted inlet uses the same five local station meanings but the orientation of the two-dimensional geometry is interpreted as top-mounted. The leading-edge height normally begins at zero:

```
inlet = struct;
inlet.enabled = true;
inlet.type = "TwoDimensionalTop";

inlet.xInlet = 6.0;
inlet.xDiverter = 0.40;
inlet.diverterHeight = 0.15;
inlet.diverterLength = 0.60;

inlet.placements(1).phiDeg = 0;

inlet.profile.x = [0.0 0.6 1.1 1.7 2.2];
inlet.profile.height = [0.0 1.0 1.2 1.1 0.7];
inlet.profile.width = [2.0 2.1 2.2 2.0 1.6];

inlet.covered = false;
inlet.rampAngleDeg = 5.0;
inlet.additiveDrag = true;

% Example for a case with three active speed conditions.
inlet.massFlowRatio = [0.65 0.65 0.65];

input.caseInput.inlet = inlet;
```

Listing B.50: A complete top-mounted two-dimensional inlet.

The placement at 0 degrees locates the inlet at top vertical centre. The three mass-flow-ratio entries form an explicit per-speed schedule; replace their count and values so they align with `input.caseInput.flight.speed`. Additive drag is requested, but the legacy method contributes it only where its operating criteria are satisfied, principally at supersonic conditions.

B.13.8. Axisymmetric Inlet Example

An axisymmetric inlet is described by five diameters. The public `profile.width` field carries those diameters, and the height vector is omitted:

```
inlet = struct;
inlet.enabled = true;
inlet.type = "Axisymmetric";
```

```

inlet.xInlet = 4.0;
inlet.xDiverter = 0.35;
inlet.diverterHeight = 0.0;
inlet.diverterLength = 0.5;

inlet.placements(1).phiDeg = 0;

inlet.profile.x = [0.0 0.5 1.0 1.5 2.0];
inlet.profile.width = [0.0 2.1 2.2 2.0 1.5];

inlet.covered = false;
inlet.rampAngleDeg = 0.0;
inlet.additiveDrag = false;

input.caseInput.inlet = inlet;

```

Listing B.51: A complete axisymmetric inlet.

The diameter expands from the tip through the cowl and midbody. It then reduces from 2.0 at station 4 to 1.5 at station 5, satisfying the axisymmetric boattail requirement. `diverterHeight=0` defines a conformal installation.

B.13.9. Inlet Validation Summary

Before execution, SpearDAT checks that:

- an active inlet has an explicit type and at least one placement;
- no more than 20 placements are supplied;
- explicit placement indices form one unique contiguous sequence;
- placement angles and scalar geometry fields are finite;
- the five-point profile has the required type-dependent shape;
- profile heights, widths, and diameters are finite and non-negative;
- the station-5 area or diameter is smaller than the corresponding station-4 value; this boattail comparison is performed automatically;
- diverter length is non-negative, while a negative diverter height is retained with a semi-submerged-geometry warning;
- additive drag has a valid mass-flow schedule satisfying $0 < \text{MFR} < 1$; and
- the mass-flow schedule contains one explicit value per active speed condition and matches the active speed count.

An inlet populated with `enabled=false` is not discarded. SpearDAT preserves the request, activates the inlet branch, and reports a warning so the script can be made explicit.

B.14. Experimental Data

The `experimentalData` branch supplies aerodynamic coefficients from wind-tunnel testing, flight testing, CFD, another prediction method, or a previously reduced engineering database. Missile DATCOM can transform these data to the active case reference convention, interpolate them in angle of attack, and insert them at a selected configuration level.

This branch is not a general multidimensional database interface. One dataset is a one-dimensional coefficient-versus-angle-of-attack table for one configuration and, when stated explicitly, one Mach number. Separate dataset elements are required for separate Mach numbers or configuration levels. Sideslip and aerodynamic roll remain case-level flight conditions; a dataset has no independent beta or roll dimension.

The experimental-data inputs correspond to the published EXPR capability documented in Table 17 of the Missile DATCOM User's Manual [1].

B.14.1. Experimental-Data Root Fields

Table B.56.: Fields accepted under `input.caseInput.experimentalData`.

Field	MATLAB form	Availability/default	Description
<code>input.caseInput.experimentalData.incrementMode</code>	Logical scalar	Optional; default false	Requests the legacy configuration-incrementing control path. Normal direct-substitution and hybrid build-up requests leave this false. See section B.14.7 .
<code>input.caseInput.experimentalData.datasets</code>	Structure row or column vector	Optional; required when experimental data are used	Ordered experimental datasets. One generated experimental-data record is produced per element.

When `datasets` is empty and `incrementMode=false`, the complete branch is inactive and nothing is added to the generated request. Setting `incrementMode=true` without at least one dataset is an error.

B.14.2. Dataset Fields

Table B.57.: Fields accepted in each `input.caseInput.experimentalData.datasets(i)` element.

Field	Units/form	Availability/default	Description
<code>input.caseInput.experimentalData.datasets(i).mach</code>	Dimensionless scalar	Optional; omission-significant	Mach number associated with this dataset. Use a separate dataset for each Mach number. Omission delegates matching to the legacy path; an explicit value is recommended for traceability.

Field	Units/form		Availability/ default	Description
<code>input.caseInput.experimentalData.datasets(i).configuration</code>	Text enum		Required	Configuration level at which the supplied coefficient channels are inserted. Exact values are listed in table B.58 .
<code>input.caseInput.experimentalData.datasets(i).alphaDeg</code>	Degree vector		Required; 1–20 finite values	Angle-of-attack stations for the coefficient arrays. The generated count is derived from the vector length.
<code>input.caseInput.experimentalData.datasets(i).reference</code>	Scalar	structure	Optional; omitted fields inherit the active case convention	Reference area, reference lengths, and moment-reference location attached to the supplied data. See section B.14.4 .
<code>input.caseInput.experimentalData.datasets(i).coefficients</code>	Scalar	structure	Required; at least one non-empty channel	Coefficient arrays at the stated angle-of-attack stations. See section B.14.5 .

The dataset Mach number is a scalar, not a schedule. No Mach interpolation is performed between separate experimental records. For a two-Mach dataset, construct two elements:

```
dataset1.mach = 0.8;
dataset1.configuration = "Body";
dataset1.alphaDeg = [0 5 10];
dataset1.coefficients.cn = [0.00 0.30 0.65];

dataset2 = dataset1;
dataset2.mach = 1.2;
dataset2.coefficients.cn = [0.00 0.34 0.73];

input.caseInput.experimentalData.datasets = ...
    [dataset1, dataset2];
```

Listing B.52: Creating separate datasets for two Mach numbers.

The analysis may interpolate a supplied coefficient channel in `alphaDeg` when the requested case angles differ from the dataset stations. Cover the intended analysis range with experimental stations rather than relying on behaviour outside the supplied alpha interval.

B.14.3. Configuration Selection

The cumulative body-fin selections are intentionally contiguous from fin set 1. There is no configuration enum for, for example, body plus fin sets 1 and 3 while omitting fin set 2.

A dataset applies only to its selected configuration level. If both "Body" and "BodyFinSet1" datasets are supplied, the body dataset governs the body-alone table and the cumulative dataset governs the body-plus-fin-set-1 table. The cumulative result is not rebuilt as experimental body data plus a theoretical fin set when an authoritative cumulative dataset is present.

Table B.58.: Allowed values of `input.caseInput.experimentalData.datasets(i).configuration`.

MATLAB value	Selected configuration	Required model definition
"Body"	Body alone	An axisymmetric or elliptical body.
"FinSet1"	Fin set 1 alone	Fin set 1.
"FinSet2"	Fin set 2 alone	Fin set 2.
"FinSet3"	Fin set 3 alone	Fin set 3.
"FinSet4"	Fin set 4 alone	Fin set 4.
"BodyFinSet1"	Body plus fin set 1	A body and fin set 1.
"BodyFinSets12"	Body plus fin sets 1 and 2	A body and both fin sets 1–2.
"BodyFinSets123"	Body plus fin sets 1–3	A body and every fin set from 1 through 3.
"BodyFinSets1234"	Body plus fin sets 1–4	A body and every fin set from 1 through 4.

B.14.4. Dataset Reference Quantities

Experimental coefficients are meaningful only together with the reference convention under which they were formed. Dataset reference fields therefore are not descriptive labels: they participate in force and moment coefficient conversion before the data are inserted into the active case.

All dimensional dataset reference quantities use `input.settings.lengthSystem` in the same literal way as `input.caseInput.reference`: lengths use the selected length unit and `sref` uses its square.

Table B.59.: Fields accepted under each dataset's reference structure.

Field	Units	Availability/default	Description
<code>input.caseInput.experimentalData.datasets(i).reference.sref</code>	Selected length squared	Optional; inherits the active resolved value	Reference area used by the supplied force and moment coefficients.
<code>input.caseInput.experimentalData.datasets(i).reference.lref</code>	Selected length	Optional; inherits the active resolved value	Longitudinal reference length used by the supplied pitching-moment coefficient.
<code>input.caseInput.experimentalData.datasets(i).reference.latref</code>	Selected length	Optional; inherits the active resolved value	Lateral reference length used by the supplied yawing- and rolling-moment coefficients.
<code>input.caseInput.experimentalData.datasets(i).reference.xcg</code>	Selected length	Optional; inherits the active resolved value	Longitudinal reference point of the supplied moment data. Explicit zero is preserved.
<code>input.caseInput.experimentalData.datasets(i).reference.zcg</code>	Selected length	Optional; inherits the active resolved value	Vertical reference point of the supplied moment data. Explicit zero is preserved.

When the dataset and active-case reference conventions differ, SpearDAT rescales the coefficient

channels and shifts moment coefficients to the active reference point before insertion. Consequently, omitting a reference value means that the corresponding active case value is already the dataset convention; it does not mean that reference conversion is irrelevant.

Use positive physical values for s_{ref} , l_{ref} , and l_{atref} . Every supplied reference field must be finite. Known corrections and compatibility details affecting experimental lateral-moment reference conversion are documented separately in [appendix A](#).

B.14.5. Coefficient Channels

At least one coefficient channel must be supplied. Every non-empty channel must contain exactly the same number of values as the dataset `alphaDeg` vector. Omitted channels are not emitted as zeros and remain available to the normal theoretical build-up.

The arrays are aligned element by element. If `alphaDeg` contains $a_1, a_2, \dots, a_N$, then the first value of every coefficient array belongs to a_1 , the second to a_2 , and so on. In particular, `coefficients.cn(k)` is the supplied normal-force coefficient at `alphaDeg(k)`. All populated channels in one dataset share this single alpha schedule; a channel cannot carry a separate set of angle-of-attack stations.

Table B.60.: Fields accepted under each dataset's coefficients structure.

Field	Coefficient	Availability	Meaning
<code>input.caseInput.experimentalData.datasets(i).coefficients.cn</code>	C_N	Optional vector	Normal-force coefficient.
<code>input.caseInput.experimentalData.datasets(i).coefficients.cm</code>	C_m	Optional vector	Pitching-moment coefficient.
<code>input.caseInput.experimentalData.datasets(i).coefficients.ca</code>	C_A	Optional vector	Axial-force coefficient.
<code>input.caseInput.experimentalData.datasets(i).coefficients.cy</code>	C_Y	Optional vector	Side-force coefficient.
<code>input.caseInput.experimentalData.datasets(i).coefficients.csn</code>	C_n	Optional vector	Yawing-moment coefficient. The public field is <code>csn</code> to distinguish it from normal-force <code>cn</code> .
<code>input.caseInput.experimentalData.datasets(i).coefficients.csl</code>	C_l	Optional vector	Rolling-moment coefficient.

Each populated array must contain finite values. A partial dataset is valid. For example, a measured normal-force and pitching-moment dataset may omit all other channels:

```
dataset.mach = 0.8;
dataset.configuration = "BodyFinSet1";
dataset.alphaDeg = [0 2 4 6 8 10];

dataset.coefficients.cn = ...
    [0.00 0.18 0.37 0.58 0.81 1.05];
dataset.coefficients.cm = ...
    [0.02 0.00 -0.03 -0.07 -0.12 -0.18];
```

```
input.caseInput.experimentalData.incrementMode = false;
input.caseInput.experimentalData.datasets = dataset;
```

Listing B.53: Supplying only measured normal-force and pitching-moment channels.

Only the two supplied channels are substituted at the selected configuration level. The omitted axial, lateral, yawing, and rolling channels are not materialised as zero-valued experimental data.

B.14.6. Direct Substitution and Hybrid Build-Up

With the default `incrementMode=false`, each dataset substitutes its supplied coefficient channels at the selected configuration level. This supports three principal workflows:

Direct experimental or CFD substitution. Supply data for the configuration whose reported coefficients should be replaced. The analysis can transform reference conventions, shift the moment reference point, interpolate in alpha, and produce the usual reports.

Partial-channel substitution. Supply only the trusted or available coefficient channels. Other channels remain theoretical.

Hybrid build-up. Supply data for a cumulative baseline and allow a later, physically aft fin set to be added theoretically. For example, measured body-plus-fin-sets-1–2 data may form the baseline from which the normal theoretical fin-set-3 increment is applied.

Hybrid build-up follows the same cumulative, front-to-back configuration order as the normal Missile DATCOM synthesis. The experimental baseline must therefore be one of the contiguous body-fin configurations "BodyFinSet1", "BodyFinSets12", "BodyFinSets123", or "BodyFinSets1234".

Important

A wind-tunnel baseline containing front canards and rear tail fins cannot be used directly as a body-plus-fin-sets-1-and-3 baseline while a new mid-body wing is inserted as fin set 2. The physical numbering would have to be:

$$\text{canards} = 1, \quad \text{new wing} = 2, \quad \text{tail fins} = 3.$$

The measured canard–tail baseline would then require a non-contiguous configuration equivalent to body plus fin sets 1 and 3. No such experimental configuration selector exists. Assigning the tail as set 2 and the physically forward wing as set 3 is also invalid because fin-set numbers must progress strictly from front to back.

For that exact engineering study, use an external increment calculation or obtain experimental/CFD data for the modified complete configuration. The standard EXPR hybrid path cannot insert a new fin set between two components already contained in the measured cumulative baseline.

The nearest valid related workflow uses a contiguous experimental baseline and adds a component aft of it. The following example uses:

- fin set 1: four front canards;
- fin set 2: four mid-body wings included in the measured baseline; and
- fin set 3: four tail fins added theoretically.

```

% Fin set 1: front canards.
canards.set = 1;
canards.spanStations = [0 0.70];
canards.chord = [0.80 0.45];
canards.xLeadingEdge = [2.20 2.35];
canards.nPanels = 4;
canards.panelRollDeg = [0 90 180 270];
canards.section.type = "Naca";
canards.section.naca.sectionType = "FourDigit";
canards.section.naca.designation = "0010";

% Fin set 2: mid-body wings included in the measured baseline.
wings.set = 2;
wings.spanStations = [0 1.20];
wings.chord = [1.10 0.65];
wings.xLeadingEdge = [5.00 5.30];
wings.nPanels = 4;
wings.panelRollDeg = [0 90 180 270];
wings.section.type = "Naca";
wings.section.naca.sectionType = "FourDigit";
wings.section.naca.designation = "0010";

% Fin set 3: tail fins added theoretically.
tails.set = 3;
tails.spanStations = [0 0.90];
tails.chord = [1.00 0.55];
tails.xLeadingEdge = [8.40 8.60];
tails.nPanels = 4;
tails.panelRollDeg = [0 90 180 270];
tails.section.type = "Naca";
tails.section.naca.sectionType = "FourDigit";
tails.section.naca.designation = "0010";

input.caseInput.finSets = [canards wings tails];

baseline.mach = 0.9;
baseline.configuration = "BodyFinSets12";
baseline.alphaDeg = [0 2 4 6 8 10];

baseline.coefficients.cn = ...
    [0.00 0.40 0.80 1.20 1.60 2.00];
baseline.coefficients.cm = ...
    [0.00 -0.12 -0.24 -0.36 -0.48 -0.60];
baseline.coefficients.ca = ...
    [0.08 0.08 0.08 0.08 0.08 0.08];

input.caseInput.experimentalData.incrementMode = false;
input.caseInput.experimentalData.datasets = baseline;

```

Listing B.54: A valid contiguous experimental baseline with a theoretical aft fin set.

The wind-tunnel dataset is authoritative for the cumulative body, canards, and mid-body-wing configuration. The body-plus-fin-sets-1-3 output is then formed by applying the normal theoretical contribution associated with the aft tail set and its interference. At every channel, the six coefficient values align with the six alpha values [0, 2, 4, 6, 8, 10] in the same order.

B.14.7. Configuration Incrementing

incrementMode=true emits the historical INCRMT control card. It is different from the hybrid build-up in [section B.14.6](#).

In a configuration-incrementing run, the first case is a calibration case:

1. the solver computes the theoretical complete-configuration coefficients;
2. it compares those coefficients with the supplied complete-configuration experimental data; and
3. it stores a correction for every coefficient channel supplied in the first-case dataset.

Later cases in the *same legacy solver run* recompute the modified configuration theoretically and apply the stored first-case corrections to the corresponding channels. The purpose is to estimate the effect of a parametric change while retaining calibration to the measured baseline. The later cases do not treat a later datasets value as a new baseline.

The calibration case must contain both the increment request and the experimental dataset. It must use exactly the dataset Mach number and exactly the dataset angle-of-attack stations. The following is a conceptual three-case sequence:

```
% CASE 1 - calibration case.
% Same Mach and alpha schedule as the experimental dataset.
case1.flight.alphaDeg = [0 2 4 6 8 10];
case1.flight.speed.mach = 0.8;
case1.finSets = baselineFinGeometry;

case1.experimentalData.incrementMode = true;
case1.experimentalData.datasets = calibrationDataset;
% calibrationDataset describes the complete baseline configuration.

% CASE 2 - first parametric variation in the same solver run.
% Keep the same number of fin sets and the same BETA or PHI condition.
case2.finSets = modifiedFinGeometryA;

% CASE 3 - second parametric variation in the same solver run.
case3.finSets = modifiedFinGeometryB;
```

Listing B.55: Conceptual legacy configuration-incrementing sequence.

Conceptually, case 1 establishes the measured-versus-theoretical corrections. Cases 2 and 3 then receive those corrections after their own theoretical coefficients have been calculated.

The legacy restrictions are important:

- all cases must retain the same number of fin sets;
- all cases must retain the same sideslip or aerodynamic-roll condition;
- the calibration case must use the experimental Mach and alpha values exactly;
- later alpha values should remain inside the calibration alpha range;
- all cases must remain in the same subsonic, transonic, or supersonic Mach regime; and
- experimental data belong only to the first case and must represent the complete calibration configuration.

Important

The current public MATLAB call accepts one caseInput request at a time, and each call starts an independent solver execution. Executing case1, case2, and case3 above as three separate speardat_mex calls would therefore lose the stored calibration between calls. For ordinary MATLAB analyses, leave incrementMode=false. The field is retained for advanced legacy-deck and compatibility workflows in which a complete stacked case sequence is managed deliberately.

B.14.8. Complete Direct-Substitution Example

This example first defines an actual theoretical body case and then replaces the supplied body coefficient channels with wind-tunnel data at Mach 0.8.

```
input = struct;

% Units and flight conditions.
input.settings.lengthSystem = "Feet";

input.caseInput.id = "BODY_EXPERIMENTAL_M08";
input.caseInput.flight.alphaDeg = [0 2 4 6 8 10];
input.caseInput.flight.speed.mode = "Mach";
input.caseInput.flight.speed.mach = 0.8;
input.caseInput.flight.atmosphere.mode = ...
    "StandardAtmosphere";
input.caseInput.flight.atmosphere.altitude = 0;

% Active case reference convention.
input.caseInput.reference.sref = 12.0;
input.caseInput.reference.lref = 4.0;
input.caseInput.reference.latref = 4.0;
input.caseInput.reference.xcg = 6.5;
input.caseInput.reference.zcg = 0;

% Theoretical axisymmetric body definition.
input.caseInput.body.shape = "Axisymmetric";
input.caseInput.body.axisymmetric.geometryMode = "Stations";
input.caseInput.body.axisymmetric.xOrigin = 0;
input.caseInput.body.axisymmetric.stations.x = ...
    [0 1.5 8.0 10.0];
input.caseInput.body.axisymmetric.stations.radius = ...
    [0 1.0 1.0 0.6];

% Wind-tunnel dataset for this body.
dataset = struct;
dataset.mach = 0.8;
dataset.configuration = "Body";
dataset.alphaDeg = [0 2 4 6 8 10];

dataset.reference.sref = 12.0;
dataset.reference.lref = 4.0;
dataset.reference.latref = 4.0;
dataset.reference.xcg = 6.5;
dataset.reference.zcg = 0;
```



```
% Each coefficient vector is aligned with dataset.alphaDeg.
dataset.coefficients.cn = ...
    [0.00 0.10 0.21 0.33 0.46 0.60];
dataset.coefficients.cm = ...
    [0.01 -0.01 -0.04 -0.08 -0.13 -0.19];
dataset.coefficients.ca = ...
    [0.07 0.07 0.08 0.08 0.09 0.10];
```

```
input.caseInput.experimentalData.incrementMode = false;
input.caseInput.experimentalData.datasets = dataset;
```

Listing B.56: A complete theoretical body definition with direct experimental substitution.

The coefficient values have the following direct correspondence:

- at $\alpha = 0$ degrees, $C_N = 0.00$, $C_m = 0.01$, and $C_A = 0.07$;
- at $\alpha = 6$ degrees, the fourth entries give $C_N = 0.33$, $C_m = -0.08$, and $C_A = 0.08$; and
- at $\alpha = 10$ degrees, the sixth entries give $C_N = 0.60$, $C_m = -0.19$, and $C_A = 0.10$.

The theoretical body remains necessary: it defines the configuration, geometry, reference quantities, and any coefficient channels not supplied by the dataset. Because the case and dataset use the same Mach, alpha schedule, and reference convention here, no Mach matching ambiguity, alpha interpolation, or reference conversion is required.

If the requested case alpha schedule used intermediate values inside the dataset range, the supplied channels could be interpolated in alpha. The dataset contains no independent sideslip or roll coordinate; it applies under the scalar attitude condition of the prepared case.

B.14.9. Experimental-Data Validation Summary

Before execution, SpearDAT checks that:

- incrementing mode is not requested without a dataset;
- every dataset selects an allowed configuration;
- the selected body and fin-set definitions exist;
- each dataset has 1–20 finite angle-of-attack values;
- an explicit dataset Mach number is finite;
- every supplied dataset reference value is finite;
- at least one coefficient channel is populated;
- every populated coefficient channel is finite and has exactly the same length as alphaDeg; and
- cumulative body-fin configuration selections contain all required contiguous fin sets.

No omitted coefficient channel is silently converted to zero. No Mach interpolation is inferred between separate datasets.

B.15. Enumeration Index

This section is placed at the end of the appendix because it is an appendix-wide quick reference, not a subordinate part of the experimental-data branch. The preceding sections explain each field in engineering context; the index collects the exact spellings so that users do not need to search the entire appendix when checking a text selector.

An *enumerated field* accepts one value from a fixed set of exact text values. The term *enumeration index* here means a consolidated lookup table; it is not a numeric index that the user must supply.

The index is complete for the current public MATLAB reader fields that are parsed as fixed-value enums or enum vectors. It deliberately excludes logical flags, numeric set and panel indices, free-form identifiers and comments, and family-specific free text such as a NACA designation, because those fields are not enumerations.

All values in this section are case-sensitive MATLAB string scalars or character rows unless the detailed field section states that the field is a text vector. The detailed field sections remain authoritative for defaults, omission rules, units, and cross-field restrictions.

Table B.61.: Consolidated index of enumerated MATLAB input fields.

Field	Allowed values	Detailed section
input.settings.lengthSystem	"Inches", "Feet", "Centimetres", "Metres"	section B.3
input.settings.derivativeSystem	"Degrees", "Radians"	section B.3
input.caseInput.theory.bodyMethod	"VanDykeHybrid", "Sose", "Hyper"	section B.6
input.caseInput.output.printGeometry	"Body", "FinSet1", "FinSet2", "FinSet3", "FinSet4", "Inlet"	section B.5
input.caseInput.output.printAero	"Body", "FinSet1", "FinSet2", "FinSet3", "FinSet4", "Synths", "Trim", "Bend", "Hinge", "Inlet"	section B.5
input.caseInput.flight.attitudeMode	"BodyAlphaBeta", "TotalAlphaRoll"	section B.7.2
input.caseInput.flight.speed.mode	"Mach", "Velocity"	section B.7.3
input.caseInput.flight.atmosphere.mode	"StandardAtmosphere", "ReynoldsSpecified", "WindTunnel"	section B.7.4
input.caseInput.reference.boundaryLayer	"Turbulent", "Natural"	section B.8.4
input.caseInput.body.shape	"Axisymmetric", "Elliptical"	section B.9.1
input.caseInput.body.axisymmetric.geometryMode	"Analytic", "Stations"	section B.9.2
input.caseInput.body.elliptical.geometryMode	"Analytic", "Stations"	section B.9.3
input.caseInput.body.axisymmetric.nose.type	"Conical", "Ogive", "Power", "Haack", "Karman"	section B.9.2
input.caseInput.body.elliptical.nose.type	"Conical", "Ogive", "Power", "Haack", "Karman"	section B.9.3
input.caseInput.body.axisymmetric.aft.type	"Conical", "Ogive"	section B.9.2
input.caseInput.body.elliptical.aft.type	"Conical", "Ogive"	section B.9.3
input.caseInput.body.protuberances.items(i).type	"VerticalCylinder", "HorizontalCylinder", "Block", "Fairing", "Lug", "Shoe"	section B.9.6
input.caseInput.finSets(i).section.type	"Hex", "Arc", "User", "Naca"	section B.10.5
input.caseInput.finSets(i).section.naca.sectionType	"OneSeries", "FourDigit", "FiveDigit", "SixSeries", "Supersonic"	section B.10.5
input.caseInput.inlet.type	"TwoDimensionalSide", "Axisymmetric", "TwoDimensionalTop"	section B.13.2

Field	Allowed values	Detailed section
<code>input.caseInput.experimentalData.datasets(i).configuration</code>	"Body", "FinSet1", "FinSet2", "FinSet3", "FinSet4", "BodyFinSet1", "BodyFinSets12", "BodyFinSets123", "BodyFinSets1234"	section B.14.3

The detailed sections remain authoritative for omission behaviour, conditional availability, vector shape, units, and cross-field validation. The index is intended only as a quick spelling and value reference.

References

- [1] William B. Blake. *Missile DATCOM User's Manual—1997 Fortran 90 Revision*. AFRL-VA-WP-TR-1998-3009, Air Force Research Laboratory, Air Vehicles Directorate, Wright-Patterson Air Force Base, Ohio, February 1998. Approved for public release; distribution is unlimited.
- [2] Steven R. Vukelich, Stan L. Stoy, Keith A. Burns, Joseph A. Castillo, and Marvin E. Moore. *Missile DATCOM, Volume I—Final Report*. AFWAL-TR-86-3091, McDonnell Douglas Missile Systems Company, St. Louis, Missouri, December 1988. Prepared for the Flight Dynamics Laboratory, Air Force Wright Aeronautical Laboratories, Wright-Patterson Air Force Base, Ohio. Approved for public release; distribution is unlimited.
- [3] Donald L. Kassner and Brian Wettlaufer. *Aerodynamic Characteristics of a Canard-Controlled Missile at Mach Numbers of 0.8, 1.3, and 1.75*. NASA TM X-73,218, Ames Research Center, Moffett Field, California, July 1977.
- [4] Jerry M. Allen. *Aerodynamics of an Axisymmetric Missile Concept Having Cruciform Strakes and In-Line Tail Fins From Mach 0.60 to 4.63*. NASA/TM-2005-213541, NASA Langley Research Center, Hampton, Virginia, March 2005.
- [5] Leland H. Jorgensen and Edgar R. Nelson. *Experimental Aerodynamic Characteristics for Bodies of Elliptic Cross Section at Angles of Attack From 0 Degrees to 58 Degrees and Mach Numbers From 0.6 to 2.0*. NASA TM X-3129, NASA Ames Research Center, Moffett Field, California, February 1975.
- [6] Clyde Hayes. *Aerodynamic Characteristics of a Series of Twin-Inlet Air-Breathing Missile Configurations, III—Axisymmetric and Two-Dimensional Inlets at Subsonic-Transonic Speeds*. NASA TM-84560, NASA Langley Research Center, Hampton, Virginia, March 1983.